

CS224R 学习笔记

Deep Reinforcement Learning · Stanford 2025 春

非官方个人笔记 · 根据公开课视频字幕整理 · 图为自绘示意

2026-09-24

CS224R 学习笔记

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 讲者
Chelsea Finn, 个别讲次为客座讲座 非官方个人笔记, 根据 YouTube 公开课
视频的字幕整理 · [播放列表](#) · [课程主页](#) (现在挂的是 2026 春的课表; 2025
春的课表有各讲的选读论文)

一句话: 这门课讲怎么让智能体从经验里学会做决定——先从模仿人的示范学起, 再到策略梯度、actor-critic、Q-learning 三条核心算法路线, 然后是从离线数据学、把奖励学出来、用 RL 训 LLM、基于模型的 RL、多任务与元学习、探索、分层, 最后落到机器人。

CME295 第 5—6 讲和 CS329A 第 6 讲里直接拿来用的 PPO、GRPO、DPO 公式, 在这门课里会从 MDP 和策略梯度开始一步步推出来。

课程地图

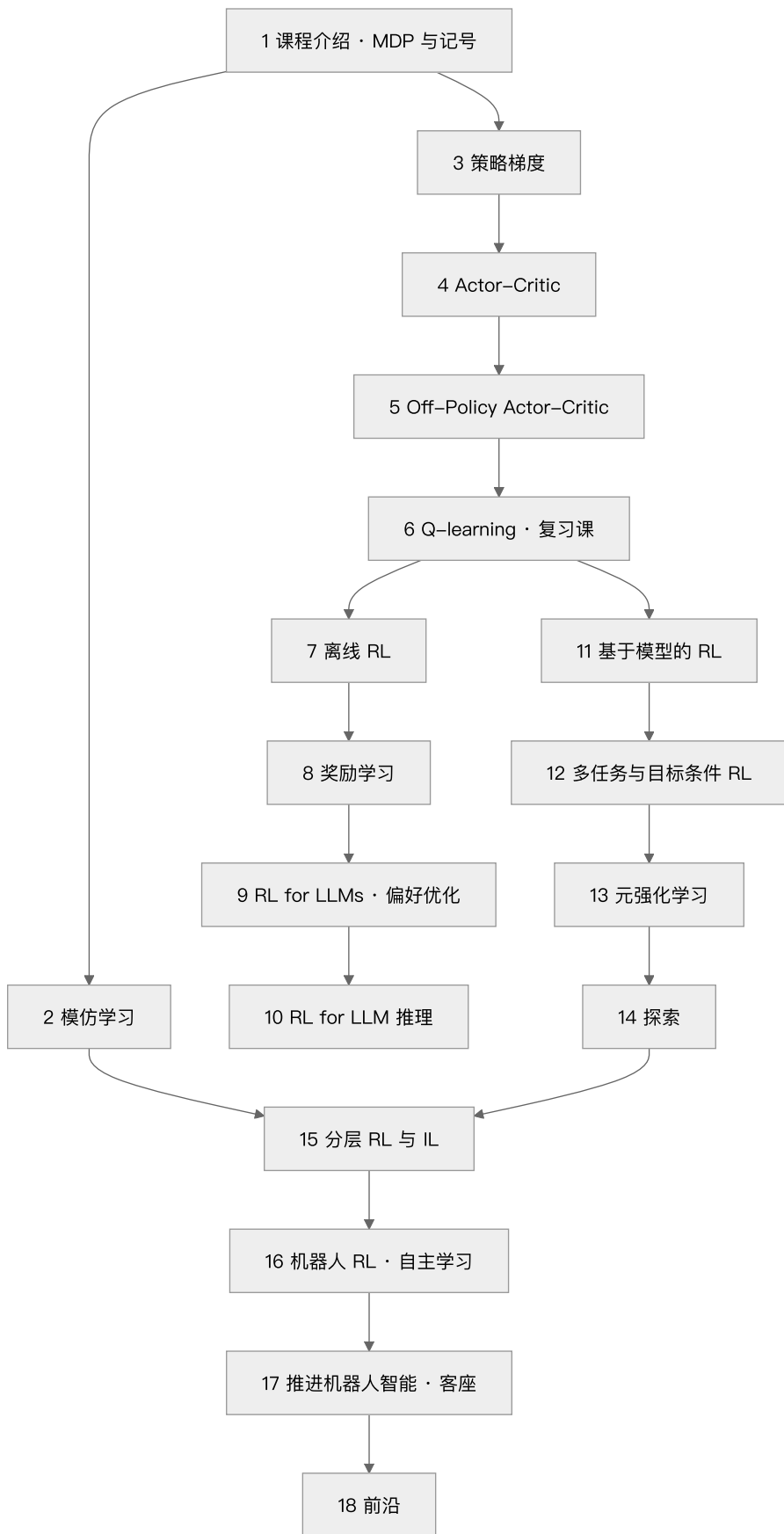


图 0-1 | 第 1-6 讲是算法主干；第 7-10 讲讲数据和奖励从哪里来，并把这套东西用到 LLM 上；第 11-15 讲讲怎么学得更省、更通用；第 16-18 讲落到机器人和研究前沿（自绘示意）

各讲索引

讲	主题	时长	视频
第 1 讲	课程介绍与 MDP：RL 问题的定义、全课的记号	52:58	YouTube
第 2 讲	模仿学习：行为克隆、误差累积、Diffusion Policy	1:07:05	YouTube
第 3 讲	策略梯度：RL 目标、REINFORCE、降方差	1:02:37	YouTube
第 4 讲	Actor-Critic：价值函数做基线、优势估计、PPO	1:03:30	YouTube
第 5 讲	Off-Policy Actor-Critic：importance sampling、PPO clip 与 GAE、replay buffer、拟合 Q 得到 SAC 骨架	1:09:21	YouTube
第 6 讲	Q-learning：policy iteration、DQN、Double DQN、N-step 目标	1:01:40	YouTube
复习课	助教复习课：Q-learning 回顾	50:39	YouTube
第 7 讲	离线 RL：分布偏移、过滤式 BC、AWR / AWAC、IQL	1:07:50	YouTube
第 8 讲	奖励学习：离线 RL 收尾（CQL）、成功分类器当奖励、成对偏好与 Bradley-Terry、RLHF / RLAIFF	1:05:58	YouTube
第 9 讲	RL for LLMs：RLHF 流水线、KL 惩罚、DPO 推导（客座 Archit Sharma）	1:02:51	YouTube
第 10 讲	RL for LLM 推理：推理即 MDP、RFT、逐步 credit assignment、PAV（客座 Aviral Kumar）	1:10:30	YouTube
第 11 讲	基于模型的 RL：学动力学模型、random shooting / CEM / MPC 规划、PDDM	1:13:20	YouTube
第 12 讲	多任务与目标条件 RL：MBRL 收尾、任务条件策略、hindsight relabeling	1:10:29	YouTube
第 13 讲	元强化学习：RL ² 、任务推断视角、PEARL、探索难题	1:09:10	YouTube
第 14 讲	探索：bandit、UCB、后验采样、DREAM	1:12:42	YouTube
第 15 讲	分层 RL 与 IL：两层策略、子目标表示、分层模仿、SayCan	1:09:32	YouTube
第 16 讲	机器人 RL：自主学习、reset-free、MEDAL、Single-Life RL	1:05:44	YouTube

讲	主题	时长	视频
第 17 讲	推进机器人智能：sim-to-real、域随机化、RMA、Tesla Optimus（客座 Ashish Kumar）	49:48	YouTube
第 18 讲	前沿：三类开放问题，以及怎么做研究	1:10:49	YouTube

和前两门课怎么对照

这门课	前两门课里对应的内容	关系
第 3—4 讲 策略梯度、Actor-Critic、PPO	CME295 第 5 讲 RLHF 与 PPO；CS329A 第 6 讲 GRPO、DAPO	那边直接用公式，这边从头推导
第 8—9 讲 奖励学习、偏好优化	CME295 第 5 讲奖励模型与 DPO；CS329A 第 3 讲验证器	那边讲 LLM 上的用法，这边讲一般形式和来历
第 10 讲 RL for LLM 推理	CME295 第 6 讲；CS329A 第 2、6 讲	同一批工作，客座讲者的视角
第 7 讲 离线 RL	—	前两门课没讲，agent 从日志数据里学时会用到
第 14 讲 探索	CS329A 第 2 讲重复采样的多样性	同一个问题在 RL 里的正式提法

怎么用这份笔记

- **每讲的结构是固定的**：一句话 → 时间轴 → 核心内容（带图和公式）→ 关键图表速查 → 提到的工作 → 术语对照 → 字幕勘误 → 带走的问题。赶时间可以只读"一句话"、图和"带走的问题"。
- **这门课公式多**：关键公式单独成块，后面一两句话解释每个符号；推导过程用文字说"这一步在做什么、为什么可以这么做"。
- **时间戳都能点**，直接跳到视频里对应的位置。图下面黄色的「▶ 看原幻灯片」会跳到老师讲那一页 PPT 的时刻。
- **图是重新画的示意图**，用来解释机制，不是课件截图。
- **"小注"** 是补充的课外事实或对口误的更正；**"应为 / 应出自"** 表示这是根据内容推断的，老师没有明说。
- 字幕对人名、算法名的识别错误不少，每讲末尾的"字幕勘误"列了会影响理解的那些。想对照原文：在播放器「设置 → 字幕 → 自动翻译 → 中文（简体）」可以开中文字幕。

CS224R 第 1 讲 | 课程介绍与 MDP (Class Intro)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 1 讲，
2025 年 4 月 2 日 · 课程表上的题目是 Course intro + MDPs，没有指定阅
读 视频：https://www.youtube.com/watch?v=EvHRQhMX7_w (52:58，英
文字幕是人工校对过的 CC，错得很少，见文末勘误) 讲者：Chelsea Finn
(课程主讲，不是客座) 课程主页：<https://cs224r.stanford.edu/>

一句话：强化学习和监督学习的差别不在模型而在数据——没人告诉你正确动作是什么，只有奖励这种间接反馈；而且数据是你正在学的策略自己采出来的，策略一变，看到的東西就变。本讲把"经验"写成数据：状态 s 、观测 o 、动作 a 、奖励 r 、轨迹 τ ，用马尔可夫性质把长序列拆成一步一步，把策略 π_θ 定义成一个从状态到动作的神经网络，再把 RL 的目标写成"在策略诱导出的轨迹分布下，最大化期望总奖励"。收尾先让值函数 V 、 Q 认个脸，并画出全课的算法地图：模仿学习、策略梯度、actor-critic、值方法、基于模型——它们优化的是同一个目标，只是在数据成本、监督形式、稳定性之间做了不同取舍。

时间轴

时间	内容
0:05	开场：讲者自我介绍、今天的计划与三个学习目标
0:55	什么是 deep RL：观察→行动→再观察的决策问题；课程覆盖的主题清单
1:50	和监督学习的两处根本不同：反馈是间接的、数据不是 i.i.d.
4:15	课程范围与目标；想学理论去 CS234
5:12	为什么学 deep RL：五条理由（Spotify、Cursor 的例子，和一句玩笑）
8:24	已部署的系统：腿式机器人、叠衣服、AlphaGo 与 Move 37、LLM 后训练、交通、生成模型、芯片
11:32	讲者的故事：看着机器人学会拼玩具飞机；从"闭眼"到加相机
13:33	开放问题与本课后半程主题：曲棍球机器人与 reset；问答：真实世界里奖励从哪来
16:25	把经验写成数据：状态、观测、动作、轨迹、奖励
20:23	马尔可夫链视角：动态与马尔可夫性质；只有观测时为什么不成立；三个问答
26:31	两个例子：20 Hz 的机器人 vs 事件驱动的聊天机器人
30:04	课堂练习与问答：自动驾驶的状态、稀疏奖励与示范、网页 agent

33:44 策略 π_θ 是一个神经网络；rollout / episode；只有观测就得给策略记忆

38:22 RL 的目标：最大化轨迹总奖励；随机性从哪来

42:34 轨迹的概率分布与目标函数；折扣因子；为什么策略要随机

48:22 值函数与 Q 函数先认个脸；五个算法家族；MDP / POMDP 收尾

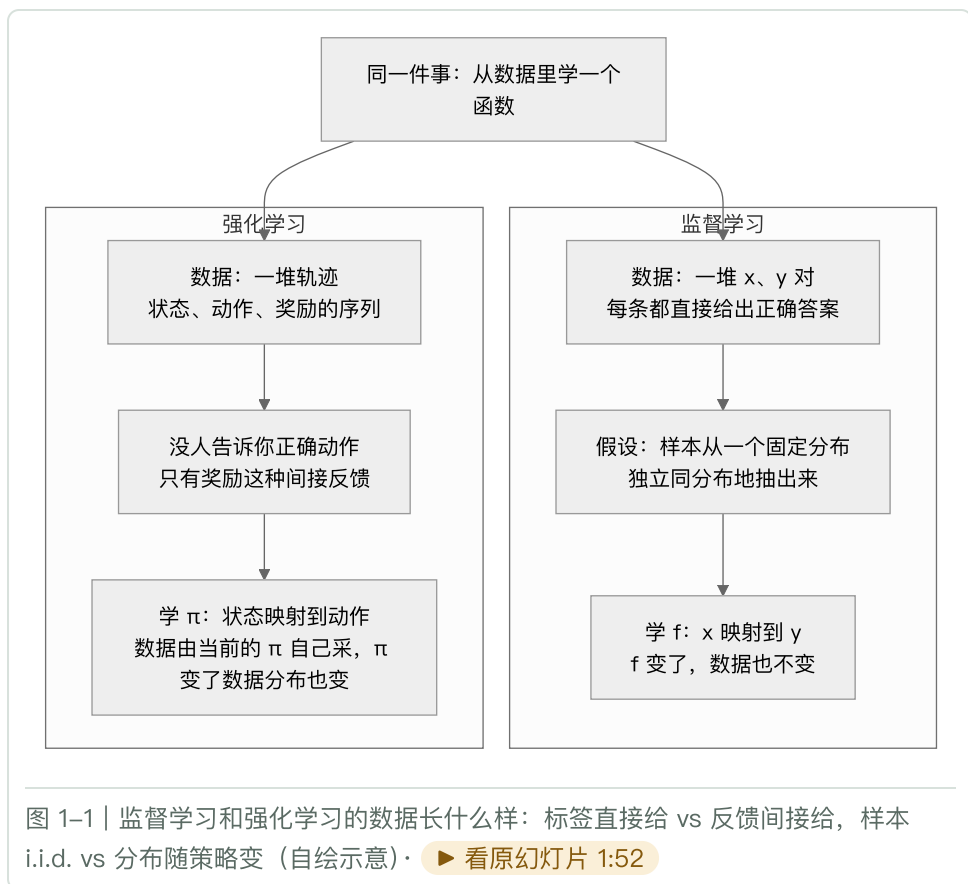
核心内容

1. 这门课讲什么、不讲什么

- 今天的三个学习目标：deep RL 到底指什么；怎么把"行为"表示成一种可以被优化的东西；怎么把一个实际问题写成强化学习问题。前两节是动机，后面全是记号——讲者说得很直白，今天引入的记号接下来每一讲都要用。
- 什么是 RL：决策问题。一个系统基于一个信息流做一连串决策：观察一次，据此行动一次，再观察，再行动。课程研究这类问题和它们的解法：模仿学习（第 2 讲）、model-free 与 model-based（第 3—6 讲、第 11 讲）、online 与 offline（第 7 讲）、multi-task 与 meta-RL（第 12—13 讲），再额外关照两块应用：RL for LLMs（第 9—10 讲）和 RL for robots（第 16—17 讲）。
- 什么是 deep：只讲能 scale 到深度神经网络的解法，跟神经网络无关的经典解法基本不碰。
- 课程目标与范围（logistics 就这几条）
 - 抓核心概念，让你之后能自己读懂更前沿的方法；动手实现算法，实现到自己有把握的程度；例子来自机器人、控制和语言模型，方法本身更通用。
 - 想要更多理论和其他应用，讲者推荐 CS234。
 - 录像里没有评分、作业和项目的细节——讲者说这些都在网上，见课程主页。

小注：CS234 是 Stanford 另一门强化学习课（Emma Brunskill 主讲），偏理论分析；CS224R 的定位是"深度网络 + 动手实现 + 机器人与 LLM 两块应用"。

2. 和监督学习的两处根本不同



- **监督学习的默认设定：**数据集是一堆输入输出对 (x, y) ，每条都直接告诉你正确答案；并且假设这些样本是从某个固定分布里独立同分布（i.i.d.）抽出来的。学习目标是一个从 x 到 y 的函数。
- **RL 的设定：**要学的是行为——从状态 s 到动作 a 的映射，记作 $\pi(a|s)$ 。讲者点出两处根本不同：
 1. 没人直接告诉你正确动作是什么。反馈是间接的：一个奖励信号，说这一步（或这一整段）好不好。
 2. 数据不是从一个固定分布来的。它来自经验，而且往往是你正在学的那个策略自己采出来的——策略一变，之后看到的状态就变了。
- **"行为"有多宽：**机器人的运动控制、和人对话的聊天机器人、玩游戏、开车、在网页上操作的 agent，都算。
- 对已经学过 RLHF 的读者：CME295 第 5 讲用一张表把 agent、state、action、policy、reward 对到 LLM 上，那是"用法"；本讲是这些词的原始定义，第 6 节的聊天机器人例子会把两边再对一次。

小注：第二处不同是后面好几讲的伏笔。第 2 讲的模仿学习之所以难，就是因为用别人的数据训出的策略一上场，看到的状态分布就变了（compounding errors, DAgger 为此而生）；第 5、7 讲的 off-policy 和 offline RL 也都是在和"数据分布随策略变"这件事打交道。

3. 为什么学 deep RL：五条理由和一串例子

- **理由一：走出监督学习。**三种情形监督学习接不住：
 - 预测有后果。Spotify 给你推一首歌（甚至直接播），这个决定会影响下一次该推什么——不该再推同一首，也许要留在同一流派。要把后果纳入优化，就需要一个能谈论后果的框架。
 - 没有直接监督。给 Cursor 或 Copilot 这类编程助手一句"这次做得好 / 做得差"，这不是它该输出的答案，是对结果的间接评价。从这种反馈、或者从任何一个目标（包括不是准确率、甚至不可微的目标）里学，都可以写成 RL 问题——所以 RL 比监督学习灵活得多。
 - 决策问题无处不在：任何 AI agent；任何和人交互的系统（聊天机器人、推荐系统）；任何"这次的决定会影响未来观测"的反馈回路。
- **理由二：已经在很多高性能系统里跑着。**讲者放了一串：
 - 腿式机器人：在模拟器里用 RL 学，再迁移到真机——狗形机器人、会跳舞的人形，都是真实世界里的真机。
 - 机械臂操作：一台从洗衣机里掏衣服并叠好，一台折简单的折纸。这两个用的是模仿学习，仍在本课范围内。
 - 游戏：DeepMind 用 deep RL 训练的系统击败了李世石。更有意思的是 Move 37——一步让所有围棋专家都意外的棋，人类之前没想到过的策略。这说明 RL 可以是**发现新解法的工具**，而不只是模仿数据；讲者把这一点延伸到需要做实验才能发现的事，比如寻找疾病的治疗方法。
 - 语言模型：几乎所有现代 LLM 的后训练都用某种形式的 RL，尤其是高级推理（第 9—10 讲）。
 - 交通控制：研究用 RL 把自动驾驶车插进人类车流，提高道路的吞吐量。
 - 生成模型：用 RL 让扩散模型的生成更贴合提示——基模型画"海豚骑自行车"画得不像，RL 优化后就对了。
 - 芯片设计：用于 Google 生产 TPU 的芯片布局。

小注：这几个例子课上都没给出处，以下是推断。叠衣服和折纸应出自讲者参与创办的 Physical Intelligence 的 $\pi 0$ (Black et al., 2024)；"海豚骑车"应出自 DDPO (Black et al., 2023, 用 RL 训练扩散模型，目标之一正是 prompt-image alignment)；芯片布局应为 Mirhoseini et al., 2020 (2021 年发表于 Nature, 用于 Google 的 TPU 设计)；AlphaGo 是

Silver et al., 2016, 2016 年 3 月以 4 比 1 击败李世石, Move 37 出现在第二局。

- **理由三：从经验中学习，像是智能的根本。**讲者讲了自己的故事：本科毕业后进了一个做机器人 RL 的实验室，座位旁边就是一台机器人，一位博士后经常在上面跑 RL 实验——学习把玩具飞机的两个零件装到一起。她着迷的是**能亲眼看到它随练习变好**：一开始什么都不会，最后能熟练完成。"越练越好"这种能力，其他机器学习系统里没有。
 - 但那台机器人是"闭着眼"的——没用相机。她觉得这是根本性限制，于是和那位博士后做了带相机的后续：机器人在真实世界里学会把红色积木插进形状分类盒，成功率练到大约 95% 到 100%；年轻版的讲者把积木举在手里，机器人也能接过去放进去。
 - 任务很简单、年代也久远，但它们把"真实世界里的 RL 过程"和"随练习进步"直观地放在你眼前。

小注：推断——那位博士后应为 Sergey Levine。"闭眼"拼玩具飞机应出自 Levine, Wagener & Abbeel, 2015 (guided policy search, 摘要里列的第一个任务就是拼装玩具飞机)；带相机的后续是 Levine*, Finn* et al., 2016 (End-to-End Training of Deep Visuomotor Policies, 形状分类盒是其中一个任务)。

- **理由四：开放的研究问题很多。**机器人怎么学到"什么对任务是好的"；行为怎么泛化到许多场景而不是一个；能不能在大规模多样数据上运行并跨任务迁移；能不能学长时程任务（做一顿饭、解一道难数学题）；机器人能不能完全自主地练习。最后一点她放了个反例：机器人学把曲棍球打进球门，每次尝试完必须有人把球放回去，因为它不会"撤销"任务——画面里她的朋友 Yevgen 干的活比机器人还多，这样收数据显然不现实。这些问题对应本课后半程：奖励学习（第 8 讲）、offline RL（第 7 讲）、多任务与 meta-RL（第 12–13 讲）、层级（第 15 讲）、推理（第 10 讲）、reset-free RL（第 16 讲）。

小注：推断——Yevgen 应为 Yevgen Chebotar，曲棍球任务应出自 Chebotar et al., 2017。

- **理由五（玩笑）：**朋友说这问题根本不难，答案就是"OpenAI 一百万美元的总包"。
- **问答：机器人在真实世界里学，奖励从哪来？**那两个例子里奖励是物体位置到目标位置的负距离。形状盒的例子里积木一直握在机械臂

里，所以位置能从臂的位置算出来；曲棍球的例子里球上贴了动作捕捉标记。这些做法都是任务特定的，换个任务就不成立——这正是为什么需要奖励学习（第 8 讲）。

4. 把经验写成数据：状态、观测、动作、轨迹、奖励

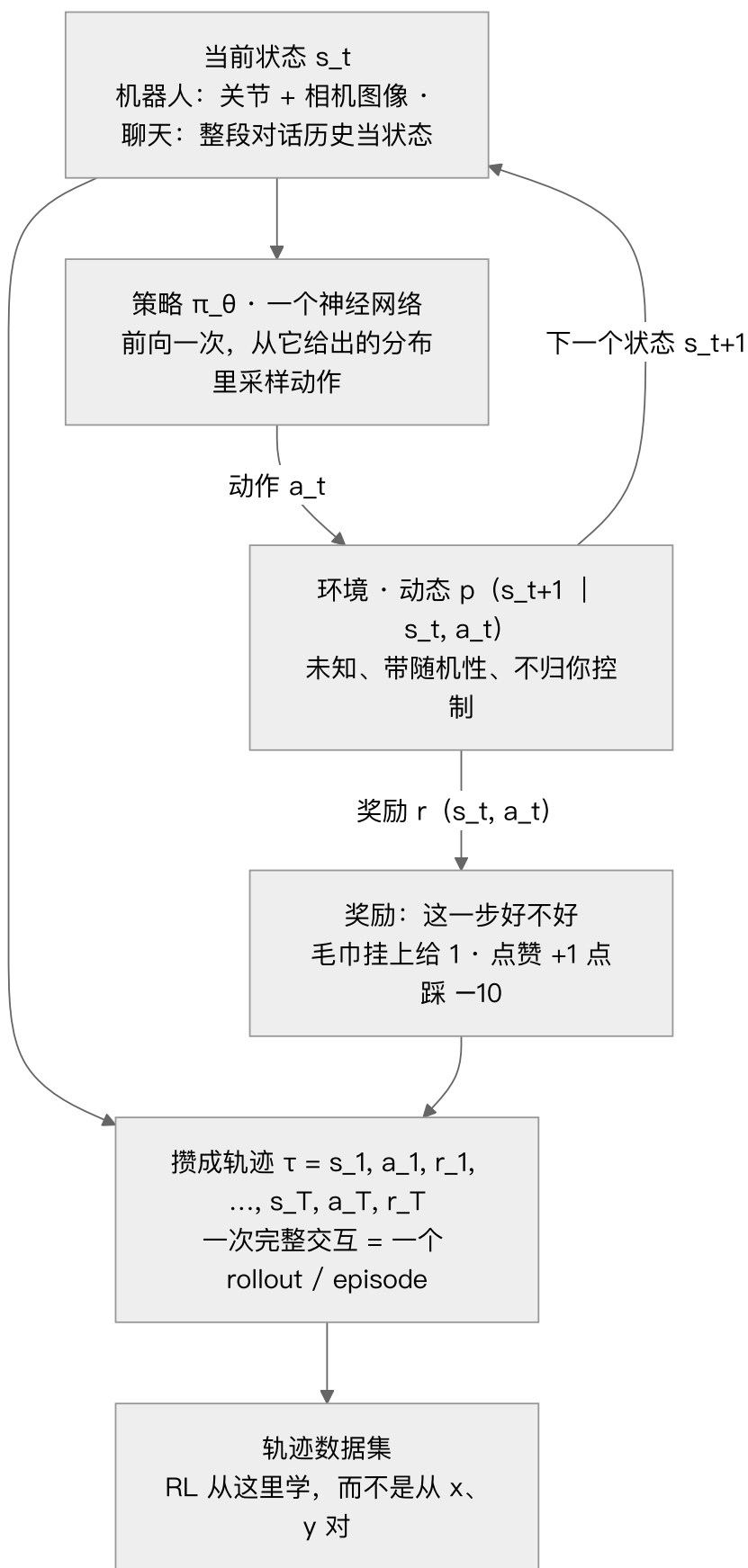


图 1-2 | agent 与环境的回路：策略出动作、环境出下一状态和奖励，一圈圈攒成轨迹（自绘示意）· [▶ 看原幻灯片 16:37](#)

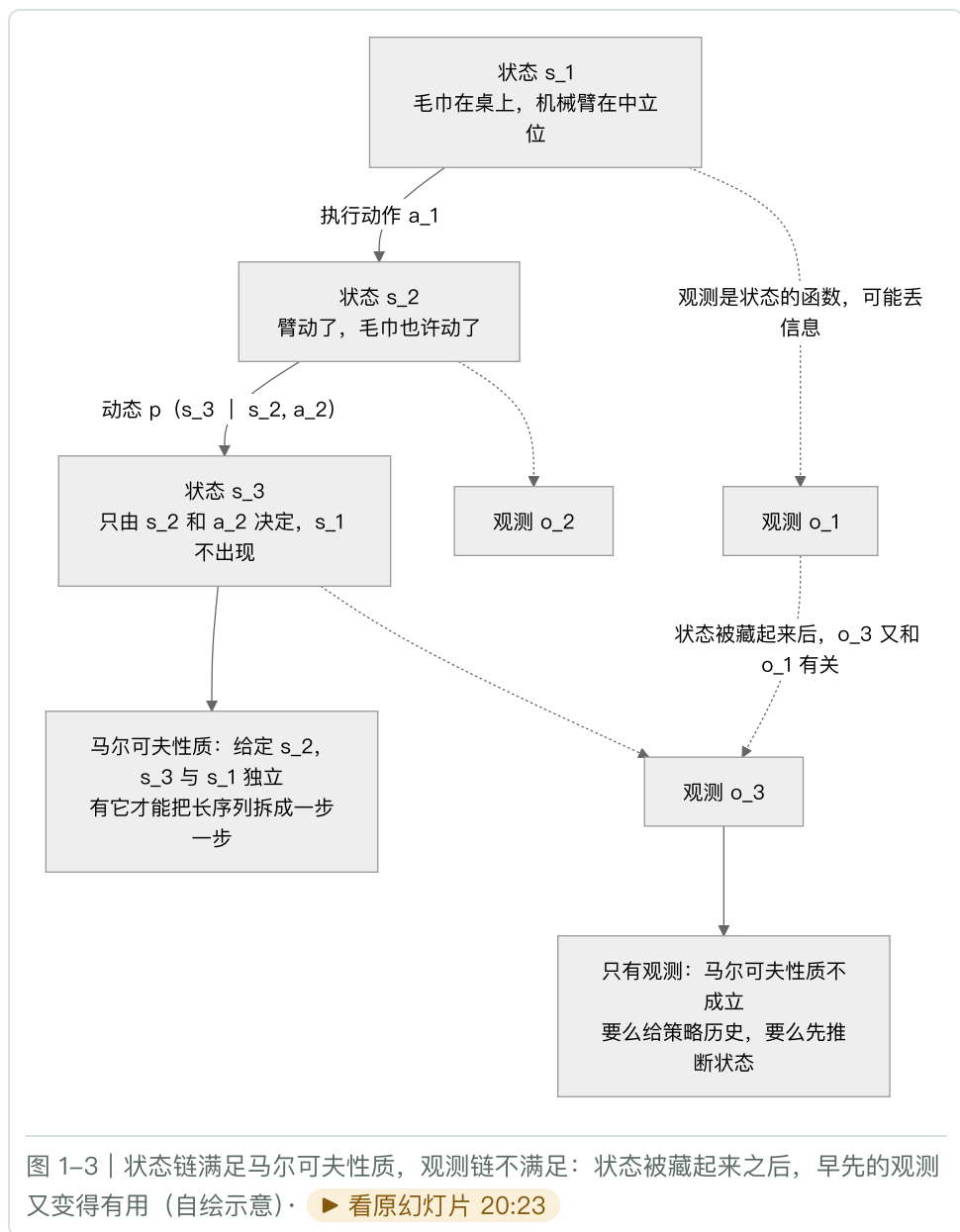
- **出发点**：机器人练习的那段视频、聊天机器人的一段对话，怎么写成能学习的数据？RL 的标准答案是先定义几个量。
- **状态 s** ：世界当前的完整描述。机器人挂毛巾的例子里，可以是机械臂的位置、毛巾的位置、挂钩的位置；如果拿不到毛巾的信息，就是相机图像加关节位置。
- **观测 o** ：很多时候拿不到真正的状态，只能拿到观测。观测和状态的区别在于：观测可能不包含做决策所需的全部信息，你可能要回看过去的观测。聊天机器人的例子：把某一步的观测定义为用户最新的一条消息，只看它显然不够决定下一句该说什么，得把历史一起给它。
- **动作 a** ：agent 做出的决策。
- **轨迹 τ** ：一次和环境的完整交互，也就是状态和动作的序列（或观测和动作的序列）。于是 RL 的数据集是一堆轨迹，而不是一堆 (x, y) 对。
- **奖励函数 $r(s, a)$** ：告诉你一个状态加一个动作有多好。很多时候只依赖状态：毛巾离挂上还有多远、有没有挂上；用户满不满意、给了赞还是踩。
- **经验**就是带上奖励的轨迹。每 rollout 一次策略，就得到一条这样的数据：

$$\tau = (s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T), \quad r_t = r(s_t, a_t)$$

s_t 、 a_t 是第 t 步的状态和动作， r_t 是那一步的奖励， T 是这条轨迹的长度；后面所有算法吃的都是这种序列。

- 对照 CME295 第 5 讲的 RLHF：那里一条轨迹就是 prompt 加一整条回答，奖励只在末尾给一次。本讲的定义更一般——每一步都可以有奖励，“只在最后给”是特例，也就是后面会反复提到的稀疏奖励。CS329A 里的 verifier、ORM，用这里的话说都是奖励函数 r 的一种实现。

5. 马尔可夫性质：为什么下一步只看当前这一步



- **状态和动作串起来像一条马尔可夫链。**以挂毛巾为例：s₁ 是毛巾在桌上、机械臂在中立位；机器人执行 a₁（把臂移到某处）；当前状态和这个动作共同决定 s₂——也许臂只是在空中移动、毛巾没动，也许毛巾被带动了；再执行 a₂，得到 s₃.....
- **动态函数（dynamics）** p(s₃ | s₂, a₂) 描述世界怎么变：臂真的动了吗，毛巾怎么动，作为"之前在哪、机器人做了什么"的函数。关键观察：这个分布里没有 s₁。也就是说，给定 s₂, s₃ 和 s₁ 独立。这个独立性叫**马尔可夫性质**：

$$p(s_{t+1} | s_t, a_t) = p(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_1, a_1)$$

左边只用当前状态和动作，右边把整段历史都摆上——两者相等，说明历史里没有额外信息。有了它，长序列可以拆成"一步一步"来处理，不必操

心更早的所有依赖；这也是 RL 里很多推导能成立的前提。

- **动态是未知的。**我们不知道世界会怎么演化、不能预测未来；RL 算法要在不知道 p 的情况下工作（第 11 讲的 model-based 方法会尝试去学它）。
- **只有观测时，情况就变了。**观测 o_t 是状态 s_t 的函数，可能丢掉信息。如果状态观测不到， o_3 就不只是 o_2 和 a_2 的函数，还会依赖 o_1 ——马尔可夫性质在观测层面不成立。
- **问答**
 - 实际要多少过去的观测才能推出状态？强烈依赖领域。机器人里常常只用当前的传感器读数就够，但要看传感器是什么：触觉传感器不接触时几乎没有信息，比视觉"部分可观测"得多；再加上图像就完整多了。
 - 奖励为什么要依赖动作，只依赖状态不行吗？人形机器人学走路，自然的奖励是前进了多少，那只看状态；但如果不想让它把所有电机绷得死紧、耗太多能量，就要对输出的扭矩加惩罚，这就依赖动作。多数情况下只看状态即可。
 - 观测只是状态的一部分，为什么未来的观测就不再独立于过去的观测？还是触觉：接触、松开、再接触，最近的那次读数 o_2 信息很少，反而是更早的 o_1 告诉你更多关于当前状态的事。熟悉图模型的人可以把状态节点边缘化掉，这种依赖会直接显现出来。

小注：这套形式化的名字课上留到最后才说（第 10 节）：状态、动作、奖励、初始状态分布、动态合起来叫 Markov decision process (MDP)，只有观测的版本叫 partially observable MDP (POMDP)；MDP 的提法一般追溯到 Bellman 1957 年的工作。LLM 语境下把"整段对话到当前为止"当作状态，正是为了让马尔可夫性质成立——状态里已经含有全部历史，下一步只需看它。

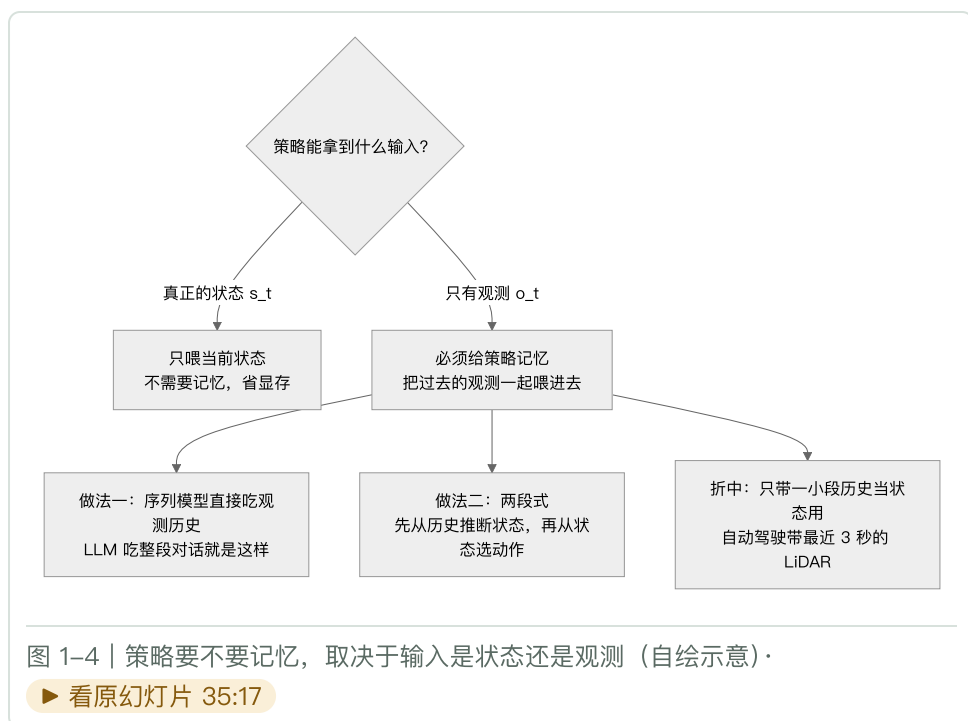
6. 两个具体的 MDP：20 Hz 的机器人和事件驱动的聊天机器人

机器人挂毛巾	聊天机器人
状态 / 观测 状态：RGB 图像 + 各关节位置 + 各关节速度（三者维度不同，网络要能分别接收）	观测：用户最新的一条消息（通常没有紧凑的状态表示，所以用观测）
动作 下一步的目标关节位置：现在在 90° ，命令它去 92°	下一条回复，可能是一句话、一段话，可能很长
轨迹 图像序列（其实就是视频）+ 关节位置序列 + 指令序列	整段对话记录

时间步	把时间离散化成固定 20 Hz：每秒 20 帧、20 次关节读数、20 次动作	不固定频率：用户什么时候回，就什么时候进入下一步
奖励	毛巾在钩上给 1，否则 0；可以再加扭矩惩罚	点赞 +1，点踩 -10，没有反馈 0

- 20 Hz 这一点讲者说自己初学时也觉得反直觉——人不会每秒做 20 次决定——但对物理系统这是最实用的表示。两个例子放在一起看的价值在于：**时间步可以是 1/20 秒，也可以是"用户回复一次"**，MDP 不关心时间步在物理上是什么。
- **课堂练习**：为自动驾驶、网页 agent 等例子（或自选）写出状态或观测、动作、轨迹、奖励。问答里有几条值得记：
 - 自动驾驶用状态还是观测？取决于传感器。LiDAR 加前视相机的当前一帧缺少周围车的速度，而速度对预测未来很重要；可以把最近 3 秒的 LiDAR 历史拼起来当状态。这是一个反复出现的主题：**拿一小段历史当状态用**。
 - 奖励是"挂上给 1 否则 0"，几乎永远拿 0，怎么学？那段视频里其实先示范了几次作为初始化，帮它完成了探索。奖励稀疏、又没有朝目标"塑形"时，示范数据常常是启动 RL 的关键（第 2 讲、第 14 讲）。
 - 网页 agent：状态是页面的 HTML 和 URL，动作是和页面交互，奖励是任务有没有完成。
 - 传感器不完美，是不是一切都该叫观测？可以那样建模，但带很多历史有代价。哪怕传感器万分之一的概率黑屏，实践里也常把它近似当状态用，因为这会改变可用的算法。观测该用全量还是相对上一帧的增量？看取舍——历史很长时用增量压缩可能合理。
- 对照 CME295 第 5 讲：那里 LLM 的动作是"下一个 token"，本讲的例子里动作是"下一整条消息"。两种粒度都是合法的 MDP，选哪种会改变 T 的长短、奖励的稀疏程度和策略随机性落在哪一层（带走的问题 1）。

7. 策略：一个从状态到动作的神经网络



- **策略 π** ：把状态或观测（或观测序列）映射到动作的函数，几乎总用 π 表示。它可以就是一个神经网络——卷积网络、Transformer、全连接网络，甚至线性函数； θ 记它的参数，也就是要学的东西。写成 $\pi_{\theta}(a | s)$ ，含义是“给定状态，各个动作的概率”。
- **用策略跑一遍**：观察到状态 $\rightarrow$ 网络前向一次，输出动作的分布，从里面采样一个动作 $\rightarrow$ 在真实世界执行 $\rightarrow$ 观察下一个状态 $\rightarrow$ 循环。跑出来的结果就是一条轨迹；由策略生成的轨迹叫 **rollout** 或 **episode**——一个 episode 就是一次尝试，然后再来一次；roll out 的意象是策略和动态交替往前滚。
- **只有观测时，给策略记忆**：把多个观测一起喂进去，用序列模型处理——语言模型正是用序列模型吃过去的观测。有状态就不需要记忆，只喂当前状态即可；这很划算，因为记忆会占显存。
- **问答**
 - 大写 T 和小写 t ： T 是轨迹的长度（状态动作的个数）， t 是某一步， t 从 1 到 T 。
 - 能不能同时喂状态和观测？状态严格包含更多信息，观测加进去没有意义。
 - 历史观测为什么有用？它们让你更了解底层状态。对话的例子：用户先要你写一段代码，你写了，用户给了反馈——要把这些过去的观测都带上，才算抓住了“世界的完整状态”。

- 也可以两段式：先从历史推断状态，再从状态预测动作。如果你对"好的状态表示"有想法，这条路有用；否则直接从观测到动作也行。

8. RL 的目标：在策略诱导的轨迹分布下最大化期望总奖励

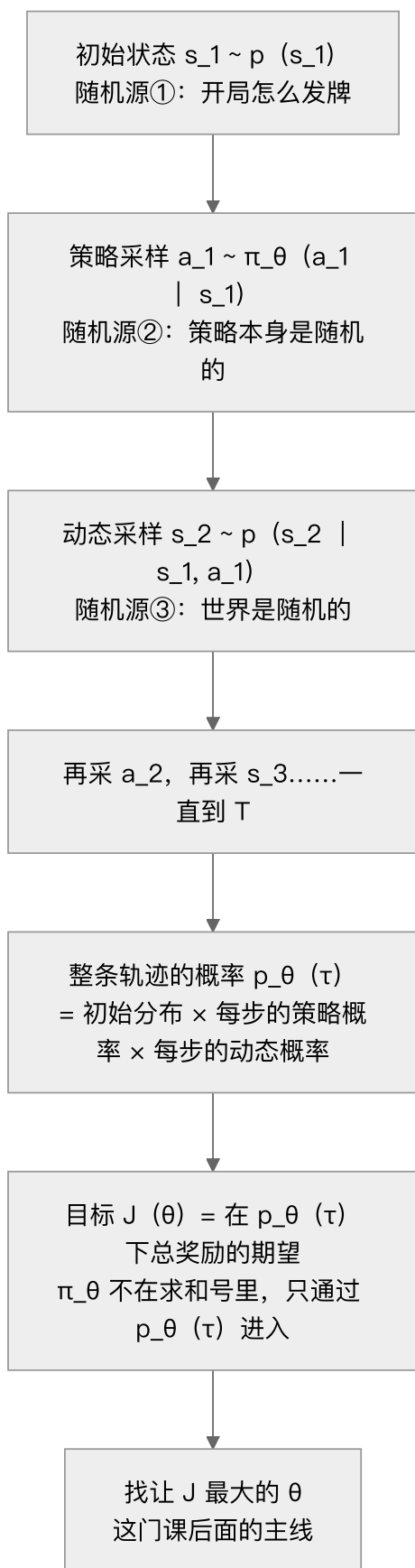


图 1-5 | 随机性从三处进入一条轨迹，连乘起来就是轨迹分布，目标是在这个分布下的期望总奖励（自绘示意）· [▶ 看原幻灯片 43:07](#)

- 目标不是某一步的奖励，而是整条轨迹的奖励之和最大。
- 这个和不是确定的量。同一个策略每次跑，总奖励都可能不同。讲者让学生找原因，最后归纳出两个主要来源：世界是随机的（动态是分布，不是函数），策略也是随机的。开车的例子：别的车行为随机，你的车也不会每次做同样的决定。此外还有开局的随机性——初始状态 s_1 本身有分布，比如牌怎么发。
 - 学生问：奖励函数会不会也是随机的？讲者的回答很有代表性：通常把奖励建模成确定的函数，把随机性归到状态和动作里。只有观测时奖励看起来会是随机的；奖励学习里对奖励有不确定性，那是另一回事。
 - 目标会随时间变怎么办？通常建模成另一个决策过程；也可以把看不见的目标放进状态，让奖励函数保持确定——不失一般性，随机性可以推进状态里。
- 轨迹的概率。要谈“期望”，就得先写出轨迹的分布：先有初始状态，策略据此选动作，动态据此给出下一状态，如此交替：

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

$p(s_1)$ 是初始状态分布， $\pi_{\theta}(a_t | s_t)$ 是策略选每个动作的概率， $p(s_{t+1} | s_t, a_t)$ 是动态；一条轨迹的概率就是这三类因子沿时间连乘。

- 目标函数：

$$\theta^* = \arg \max_{\theta} J(\theta), \quad J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T r(s_t, a_t) \right]$$

$J(\theta)$ 是在 $p_{\theta}(\tau)$ 下总奖励的期望， θ^* 是让它最大的参数。讲者特意指出： π 并不出现在求和号里面，它只通过轨迹分布 $p_{\theta}(\tau)$ 进入——你要找的是“能导出高奖励轨迹分布”的策略。这门课的大部分内容，就是在这个目标下怎么优化行为。

- 问答：T 必须固定吗？不必，可以是可变的，固定只是写起来方便；聊天里 T 可以理解为对话的长度。
- 折扣因子。上面的写法对所有时刻的奖励一视同仁——一千年后的奖励和今天的一样重要。有时候你更在乎眼前，于是给每步奖励乘上一个随 t 衰减的权重：

$$J_{\gamma}(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \gamma^t r(s_t, a_t) \right], \quad 0 < \gamma \leq 1$$

γ 是折扣因子： $\gamma = 1$ 回到原式； γ 越小越“贪”，越只看近期。几讲之后会细说它。

小注：指数写成 γ^t 还是 $\gamma^{(t-1)}$ 各家不同，只差一个常数因子，不影响最优策略。CME295 第 5 讲的 RLHF 把一整条回答当一步， γ 实际上是 1；机器人和游戏里常取 0.99 上下。

• 为什么策略要是随机的（讲者说这是个小岔题）：

1. 要学习就得尝试不同的东西。学打网球不能一直重复同一个动作，得试几种打法再看哪种好——随机策略就是“同时尝试几种策略并判断哪种最好”的表示方式（探索，第 14 讲）。
2. 数据来自人或别的系统时，不同的人用不同的打法。要建模他们，策略也得是随机的。这时可以借用生成模型的工具：策略不再是“A 到 B 的映射”，而是“给定状态或观测之下，动作的生成模型”。

小注：LLM 里 temperature 大于 0 的采样就是随机策略；CS329A 与 CME295 里 GRPO 对同一个 prompt 采一组回答，靠的正是这一点。第 2 讲的模仿学习会碰到“示范是多模态的”问题，就是这里的第二条理由。

9. 值函数与 Q 函数：先认个脸

- **为什么需要**：如果不知道一个策略有多好，就没法让它变得更好。
- **值函数 $V^\pi(s)$** ：从状态 s 出发、之后一直按 π 行动，未来能拿到的期望奖励。它就是第 8 节的目标函数“从 s 起算”的版本：

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \mid s_t = s \right]$$

期望里的随机性来自策略 π 和动态；条件是第 t 步正处在 s 。

- **Q 函数 $Q^\pi(s, a)$** ：和 V 几乎一样，区别是第一步的动作 a 由你指定——它可以不是 π 会选的那个——之后再按 π 走：

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \mid s_t = s, a_t = a \right]$$

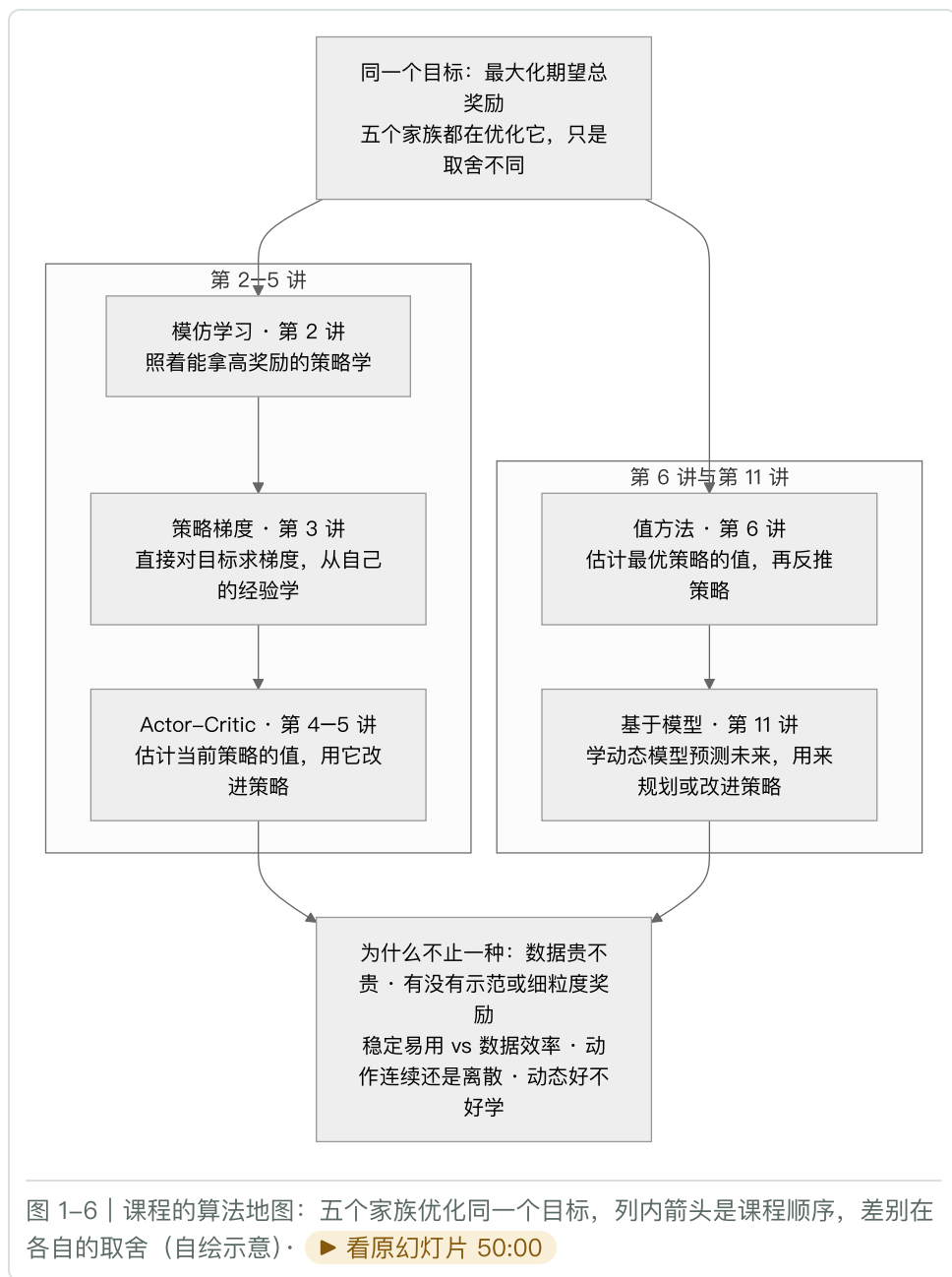
多出的条件 $a_t = a$ 就是“先试这个动作”；有了 Q ，比较同一状态下不同动作的好坏就有了尺子。

- 讲者今天不展开，只是先让大家眼熟：周五的模仿学习（第 2 讲）用不到，下周起（第 3—4 讲）会大量使用。

小注：CME295 第 5 讲里 PPO 的 value head 估的就是这里的 V^π ，只不过 MDP 是 token 级的；那里的 advantage A_t 等于 Q^π 减 V^π ，本

讲没有出现，第 3—4 讲会引入。CS329A 里给每一步打分的 PRM，可以理解为在学一个 V 或 Q 的近似。

10. 算法地图：五个家族，为什么不止一种



• 五类算法，都在优化第 8 节那个目标：

1. 模仿学习：照着一个能拿高奖励的策略学（第 2 讲）。
2. 策略梯度：直接对目标求导，从自己的练习和经验里学（第 3 讲，下周开始）。
3. Actor-critic：估计当前策略的值，用它来改进策略（第 4—5 讲）。

4. 值方法：估计最优策略的值，再从中反推出一个好策略（第 6 讲）。
5. Model-based：学世界的动态模型、预测未来，用它来改进策略或做规划（第 11 讲）。

- 为什么监督学习一个梯度下降就够，RL 却要这么多算法？ 因为 RL 算法各自做了不同的取舍，在不同假设下才好用：
 - 收集数据贵不贵：模拟器里数据几乎免费，人手写回复或真人交互就很贵。
 - 有哪些监督形式可用：有没有示范，有没有细粒度的奖励。
 - 稳定、易用 vs 数据效率：数据不缺时可以多花时间调超参。
 - 动作是什么样：维度多高，连续还是离散。
 - 动态模型好不好学。
- 收尾：状态、观测、动作、奖励函数、初始状态分布、动态——合起来叫 MDP；只有观测叫 POMDP。讲者故意把名字留到最后：叫什么不重要，但以后别人说 MDP，你知道指的就是这套问题定义。再加上轨迹、策略、目标函数、值函数、Q 函数，就是今天全部的记号。

记号速查

记号	名字	一句话解释	本讲的例子或备注
s_t	状态 state	时刻 t 世界的完整描述；有了它，决策不必再看更早的历史	臂的位置 + 毛巾位置 + 钩位置；或图像 + 关节位置
o_t	观测 observation	状态的函数，可能丢信息；拿不到 s 时用它，通常要配历史	用户最新的一条消息；触觉读数
a_t	动作 action	时刻 t 的一次决策	目标关节位置；下一条回复
$r(s_t, a_t)$	奖励函数 reward	这一步有多好；很多时候只依赖 s	毛巾挂上给 1；点赞 +1、点踩 -10
$\pi_\theta(a_t s_t)$	策略 policy	给定状态下各动作的概率分布； θ 是网络参数	CNN、Transformer、MLP，甚至线性函数
τ	轨迹 trajectory	$s_1, a_1, r_1, \dots, s_T, a_T, r_T$ ；由策略生成时叫 rollout / episode	20 Hz 的视频 + 关节 + 指令；整段对话
t, T	时间步、时程 horizon	t 是某一步， T 是轨迹长度，可以不固定	机器人 1/20 秒一步；聊天以用户回复一次为一步
$p(s_1)$	初始状态分布	开局的随机性	牌怎么发

记号	名字	一句话解释	本讲的例子或备注
$p(s_{t+1} s_t, a_t)$	动态 / 转移 dynamics	世界怎么变；未知；满足马尔可夫性质	臂动没动、毛巾怎么动
$p_\theta(\tau)$	轨迹分布	$p(s_1)$ 乘以每步的策略概率和动态概率； π 只通过它进入目标	图 1–5
$J(\theta)$	RL 目标 objective	在 $p_\theta(\tau)$ 下总奖励的期望；求让它最大的 θ	全课的主线
γ	折扣因子 discount	$0 < \gamma \leq 1$ ，用 γ^t 给奖励加权，越小越看重近期	$\gamma = 1$ 回到不折扣的原式
$V^\pi(s)$	值函数 value	从 s 出发按 π 走的期望未来奖励	下周起使用
$Q^\pi(s, a)$	动作值函数 Q	从 s 先做 a 、之后按 π 走的期望未来奖励	下周起使用
$A^\pi(s, a)$	优势 advantage	本讲未出现： Q^π 减 V^π ，第 3–4 讲引入	CME295 第 5 讲 PPO 里的 A_t 就是它
MDP / POMDP	马尔可夫决策过程 / 部分可观测版本	s 、 a 、 r 、 $p(s_1)$ 、动态五件套；只有 o 时是 POMDP	收尾 52:06 才点名

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
监督学习 vs 强化学习	左边 x 、 y 对与 i.i.d. 假设，右边 $\pi(a s)$ 、间接反馈、数据随策略变	1:52	—
已部署系统拼图	腿式机器人、掏洗衣机叠衣服、折纸：哪些是 RL，哪些是模仿学习	8:24	叠衣服应为 π_0 （推断）
AlphaGo 与 Move 37	RL 作为"发现新解法"工具的论据	9:30	Silver et al., 2016
海豚骑车前后对比	基模型的生成 vs RL 优化后更贴合提示的生成	10:31	应为 DDPO
拼玩具飞机与形状分类盒	真实世界里机器人随练习变好；"闭眼"版和带相机版的差别	11:32	应为 Levine et al., 2015、Levine, Finn et al., 2016
曲棍球机器人与 Yevgen	reset 问题：每次尝试后人比机器人干得多	14:35	应为 Chebotar et al., 2017
经验即数据	s 、 o 、 a 、 τ 、 r 各自的定义与机器人 / 聊天两个例子	16:37	—

图	看什么	跳转	出处
马尔可夫链图	$s_1 \rightarrow s_2 \rightarrow s_3$ 的箭头和 $p(s_3 s_2, a_2)$ ；观测版本多出的依赖	20:23 / 22:27	—
机器人 vs 聊天机器人对照	20 Hz 与事件驱动的时间步；两种奖励的写法	26:31	—
策略幻灯片	网络吃状态或观测序列、吐动作分布；rollout 的过程	33:44	—
轨迹分布与目标函数	$p(\tau)$ 的连乘和期望总奖励； π 只出现在分布里	43:40	—
折扣因子	γ^t 权重； γ 的取值范围与"贪"的方向	46:21	—
V 与 Q 的定义	两者只差在"第一步动作谁选"	48:55	—
算法家族与取舍	五类算法各一句话；为什么不止一种算法	50:00 / 50:33	—

提到的工作

名称	在本讲里的作用
CS234	想学更多理论和其他应用时的替代课
Spotify 推荐	"预测有后果"的例子：这次推了，下次就不该再推同一首
Cursor / Copilot	"只有间接反馈"的例子：做得好 / 做得差不是标签
腿式机器人 sim-to-real（狗形、跳舞人形）	已部署的 deep RL 例子；先在模拟器学再迁移到真机
掏洗衣机叠衣服、折纸的机械臂	模仿学习的部署例子；推断出自讲者参与创办的 Physical Intelligence 的 $\pi 0$ (Black et al., 2024)
AlphaGo (Silver et al., 2016)	击败李世石；Move 37 说明 RL 能发现人类没想到的策略
LLM 后训练中的 RL	几乎所有现代 LLM 都在用，尤其是推理；第 9—10 讲
混合车流的交通控制	把自动驾驶车插进人类车流以提高吞吐量；应出自 Berkeley 的 Flow 项目 (Wu 等人)，课上没点名
DDPO (Black et al., 2023)	用 RL 训练扩散模型使生成贴合提示；"海豚骑车"应出自此篇
芯片布局 (Mirhoseini et al., 2020)	用于 Google 生产 TPU 的芯片设计；应为此篇

名称	在本讲里的作用
Guided policy search (Levine, Wagener & Abbeel, 2015)	讲者故事里"闭眼"学拼玩具飞机的机器人；应为此篇
End-to-End Training of Deep Visuomotor Policies (Levine*, Finn* et al., 2016)	加相机后的后续：红积木进形状分类盒
曲棍球机器人（应为 Chebotar et al., 2017）	reset 问题的反例；Yevgen 应为 Yevgen Chebotar
马尔可夫链 / 马尔可夫性质 / MDP / POMDP	本讲的形式化框架
序列模型、Transformer	只有观测时给策略记忆的工具
动作的生成模型	随机策略的一种实现手段，课上没点名具体方法
图模型与边缘化	解释"观测之间为什么有依赖"的另一种视角

术语对照

English	中文
deep reinforcement learning (deep RL)	深度强化学习：用深度神经网络表示策略、值函数或模型的 RL
decision-making problem	决策问题：基于一个信息流连续做多次决策
supervised learning	监督学习：从带正确答案的 (x, y) 对里学一个函数
i.i.d. (independent and identically distributed)	独立同分布：监督学习对样本的默认假设，RL 里不成立
behavior	行为：从状态到动作的映射，RL 要学的对象
agent	智能体：做决策的那一方
environment	环境：agent 之外的一切，按动态演化并给出奖励
state s	状态：世界当前的完整描述
observation o	观测：状态的函数，可能丢信息
partially observable	部分可观测：只能拿到观测、拿不到状态
action a	动作：agent 的一次决策
trajectory τ	轨迹：一次交互的状态、动作（和奖励）序列
rollout / episode	由策略跑出一条轨迹 / 一次完整尝试
reward function $r(s, a)$	奖励函数：这一步有多好
sparse reward	稀疏奖励：绝大多数时候是 0，只在完成时给分
shaped reward	塑形奖励：设计成朝目标逐渐增加的奖励（课上提到稀疏奖励"没有 shape 好"）
demonstration	示范：人先做几遍，用来启动学习

English	中文
exploration	探索：尝试不同的动作以发现更好的策略
reset / reset-free RL	复位 / 免复位 RL：每次尝试后把环境恢复原状；免复位是让机器人自己接着练
Markov chain	马尔可夫链：下一状态只依赖当前状态的随机序列
Markov property	马尔可夫性质：给定 s_t 和 a_t , s_{t+1} 与更早的历史独立
dynamics / transition function	动态 / 转移函数：世界怎么变，通常未知
initial state distribution	初始状态分布：开局的随机性
MDP (Markov decision process)	马尔可夫决策过程：状态、动作、奖励、初始分布、动态的合称
POMDP (partially observable MDP)	部分可观测马尔可夫决策过程：只有观测的版本
graphical model / marginalize	图模型 / 边缘化：把隐藏的状态积掉，观测之间的依赖就显现
policy π_θ	策略：给定状态下动作的分布； θ 是网络参数
deterministic / stochastic policy	确定性 / 随机策略：后者输出一个分布再采样
memory / history	记忆 / 历史：只有观测时喂给策略的过去观测
sequence model	序列模型：处理观测历史的网络，Transformer 是一种
horizon T / time step t	时程（轨迹长度） / 时间步
discretize time	时间离散化：机器人常用固定的 20 Hz
expected sum of rewards	期望总奖励：RL 的目标
objective $J(\theta)$	目标函数
discount factor γ	折扣因子： $0 < \gamma \leq 1$ ，越小越看重近期
greedy	贪心：只顾眼前的奖励
value function $V^\pi(s)$	值函数：从 s 出发按 π 走的期望未来奖励
Q function $Q^\pi(s, a)$	动作值函数：先做 a 再按 π 走
optimal policy	最优策略：让目标最大的策略
imitation learning	模仿学习：照着示范策略学
policy gradient	策略梯度：直接对目标求 θ 的梯度
actor-critic	演员—评论家：估计当前策略的值来改进策略
value-based methods	值方法：估计最优值函数再反推策略
model-based / model-free RL	基于模型 / 无模型：是否显式地学动态
online / offline RL	在线 / 离线 RL：边交互边学 / 只用固定数据集学
multi-task RL / meta-RL	多任务 / 元强化学习

English	中文
hierarchical RL	分层 RL：高层选子目标、低层执行
reward learning	奖励学习：从人类反馈等数据里学出奖励函数
sim-to-real	模拟到真机迁移
generative model over actions	动作的生成模型：随机策略的一种实现
motion capture (mocap)	动作捕捉：给物体贴标记测位置
tactile sensor	触觉传感器：不接触时几乎没有信息
torque penalty	扭矩惩罚：奖励里依赖动作的那一项
nonstationary	非平稳：奖励或动态随时间变
Move 37	AlphaGo 对李世石第二局的第 37 手：人类没想到过的策略

字幕勘误

人工字幕，整体很准，只有几处会误导："same currency" → shape-sorting cube（形状分类盒）；"curious diseases" → cures for diseases；"teapot" → chatbot；"current space" 和 "observe the space" → state；"supervised example" → supervised learning；"performance systems" → performant systems；"an error from here to here" → an arrow；"the dynamic" → the dynamics；"straight from observations side" → 应为 observations to actions（推断）；收尾那句把 MDP 和 POMDP 的名字串在了一起，应读作 Markov decision process（MDP），只有观测时叫 POMDP。

带走的问题

1. CME295 把 LLM 的动作定义为"下一个 token"，本讲把聊天机器人的动作定义为"下一整条消息"。两种都是合法的 MDP。换一种粒度，T、奖励的稀疏程度、策略的随机性各落在哪里？对你自己的 agent（一次工具调用算一步？一个 token 算一步？）哪种更合适，为什么？
2. 目标函数里 π 不出现在求和号里，只藏在 $p_{\theta}(\tau)$ 中。这为什么让"直接梯度下降"不再显然？提示：期望所依赖的采样分布本身随 θ 变——第 3 讲的策略梯度就是在处理这件事。
3. 讲者说"不失一般性，随机性可以推进状态里"。对一个调用外部工具、返回结果带噪的 agent：什么是 s，什么是 o，你会给策略多长的历史？马尔可夫性质在哪一层成立？

4. 20 Hz 的机器人和事件驱动的聊天机器人， T 的含义完全不同。对一个多轮的网页 agent， γ 取 1 还是小于 1 分别意味着什么？什么时候你希望它"贪"近期？
5. 0/1 的稀疏奖励靠几次示范启动，曲棍球机器人靠人来 reset。在 LLM 后训练里，SFT 起的正是"示范启动"的作用（CME295 第 4—5 讲）；那 reset 对应什么？一个 coding agent 的 episode 从哪开始、到哪结束、谁来复位环境？

CS224R 第 2 讲 | 模仿学习 (Imitation Learning)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 2 讲, 2025 年 4 月 4 日 视频: https://www.youtube.com/watch?v=WxRDyObrm_M (1:07:05, 英文字幕是自动生成的, 术语拼写错得不少, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座) 课程主页: <https://cs224r.stanford.edu/> · 本讲指定阅读: Diffusion Policy: Visuomotor Policy Learning via Action Diffusion (Chi et al., 2023; v5 为 2024)

一句话: 模仿学习不写奖励函数, 直接把专家示范里的 (状态, 动作) 对当训练集做监督学习, 这就是行为克隆。它有两个坑。第一, 示范常常是多峰的——同一个路口有人直行、有人并线——用 L2 回归只能学到"平均动作", 所以策略必须输出一个足够有表达力的分布 (categorical、混合高斯、离散化 + 自回归、扩散), 再用最大似然去拟合; 讲者用实验说明这一步能决定成败。第二, 策略一旦部署, 下一个状态由它自己的动作决定, 一个小错会把它带到示范没覆盖的地方, 错误越滚越大 (compounding errors); 补救办法是让专家在策略自己访问到的状态上给纠正数据, 也就是 DAgger, 或者更实用的"专家接管"式干预。

时间轴

时间	内容
0:06	开场: 第 1 讲记号回顾 (state / observation、action、trajectory、reward、policy); 今天的计划与学习目标
2:41	问题设定: 专家示范 D 来自未知的 π_{expert} , 目标是追平专家; 自动驾驶例子
4:13	Version 0: 对专家动作做 L2 回归, 训完直接部署
7:16	什么会出错: 直行 vs 并线, 动作分布是双峰的, L2 只学到均值
11:29	问答: 为什么不直接查表做最近邻; 多峰在多人采集的数据里很常见
14:35	让网络输出分布的参数: 离散动作用 categorical; 单高斯为什么不够; 问答: 完美拟合多峰不会"犹豫", 目标是追平而不是超越
20:49	更有表达力的分布: 借生成模型的思路; 混合高斯 GMM; "表达力"到底指什么
25:54	离散化 + 自回归: 转向角分桶、加速度以转向角为条件逐维预测; 序列模型; 维度顺序; 为什么离散化

时间	内容
36:41	训练目标：最大化数据里动作的 log 概率；GMM 写密度求导；自回归逐维交叉熵，训练时喂数据里的动作
41:16	问答：离散化丢掉桶之间的距离、桶粒度是关键超参数；离散标签怎么反向传播
43:53	扩散策略：从噪声迭代去噪出动作；关键问题：把网络做大 + L2 不等于分布更有表达力
47:57	实证：Transport 任务 diffusion vs GMM、单人 vs 多人数据；真机挂衬衫 diffusion vs L1；业界大模型的选择
51:30	小结；offline 与 online 算法；模仿学习的优缺点；问答：大模型里除了课上讲的还有什么
55:10	什么会出错 II：和监督学习的根本差别；compounding errors；covariate shift
58:17	解决办法：堆数据；收集纠正数据；DAgger 的循环；问答：纠正数据太窄会过拟合
1:03:24	从干预中学习：专家接管、给部分示范；自动驾驶安全员；上限仍是专家

核心内容

1. 问题设定：给我示范，我来模仿

- **第 1 讲的记号，这一讲全要用**（讲者开场先复习了一遍）：
 - **状态 s_t** (state)：能比较完整地描述"世界此刻什么样"的信息；如果只能看到一部分，叫**观测 o_t** (observation)，比如摄像头的一帧。
 - **动作 a_t** (action)：每一步做出的决策，比如转向角和油门。
 - **轨迹 τ** (trajectory)：一段时间里 $s_1, a_1, s_2, a_2, \dots$ 这样的序列，是"一段行为"的记录。
 - **奖励函数 $r(s, a)$** (reward)：给"在状态 s 做动作 a "打分，说这一步有多好。
 - **策略 π** (policy)：给定当前状态（或者观测的历史）决定做什么动作；它就是我们要学的东西。第 1 讲提过策略可以用生成模型来表示，今天会真正用上。
 - RL 的目标是让期望的累计奖励最大。今天讲的是达到这个目标的一条路——去模仿一个已经很好的策略。
- **模仿学习的设定：手里有一批专家采集的轨迹，叫示范**
(demonstrations)，记作 D ，本质是一堆（状态, 动作）对。我们不知道专家用的是什么策略，只把 D 看成从一个未知的 π_{expert} 里采

出来的样本；目标是学一个 π_θ ，表现达到或接近专家。自动驾驶是贯穿全讲例子：人开车大多数时候开得不错，记录传感器读数和转向、加速指令，就得到示范。

- **问答里两个关于目标的澄清**

- 是不是默认专家在最大化某个奖励？——只假设专家"足够好"，不要求最优；现实里很多任务的最优策略根本没法指望。
- 模仿学习能不能超过专家？——本讲所有方法的目标都是追平示范者。想超越，要靠之后的 RL（第 3 讲起），或者主动筛选数据，只留你认为更好的那部分。

- **学习目标**（讲者说这部分和课程主线有点"正交"，但没有它做不出能用的模仿学习系统）：怎么用神经网络表示一个分布而不是一个确定性函数；为什么表达力强的分布对模仿学习很重要；什么是 compounding errors，怎么治。

小注：对做 LLM 的读者，SFT 就是文本上的模仿学习：状态是 prompt 加已生成的 token，动作是下一个 token，示范是人写的回答。CME295 第 4 讲的 SFT loss 正是本讲第 6 节那个负对数似然目标的特例。

2. Version 0：行为克隆就是监督回归



图 2-1 | 行为克隆的流水线：示范数据集上的监督学习，部署之后才和监督学习分道扬镳（自绘示意）· [▶ 看幻灯片 6:16](#)

- **做法：**从 D 里采一个 mini-batch 的 (s, a) ，把 s 喂给策略网络得到预测动作 $\hat{a}$ ，让 $\hat{a}$ 尽量接近数据里的 a，对 D 里所有的对做这个最小化。训练就是普通的前向、算 loss、反向传播、用你喜欢的 SGD 变种迭代。训完直接部署——它应该会模仿数据里的动作。

$$\min_{\theta} \sum_{(s,a) \in D} \|\pi_{\theta}(s) - a\|^2$$

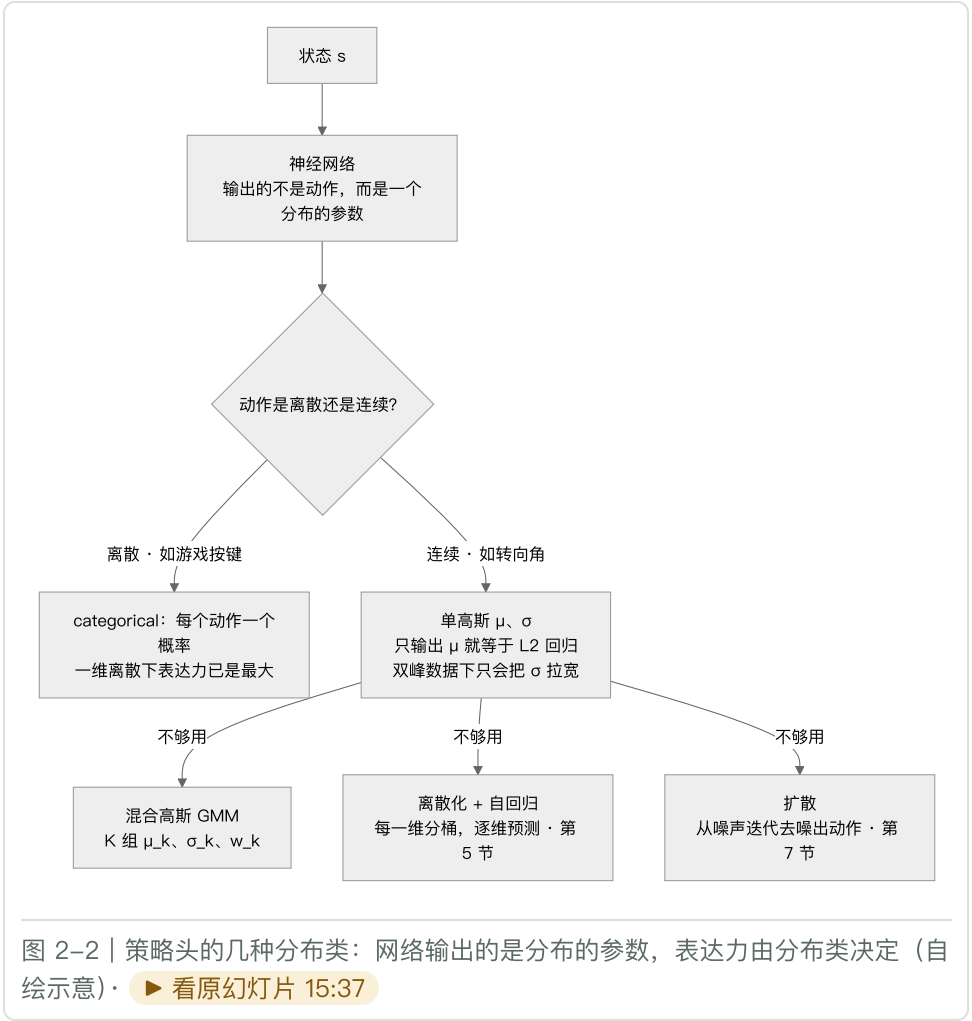
$\pi_{\theta}(s)$ 是策略网络对状态 s 输出的那一个动作，a 是示范者在同一状态下的动作；对数据集里每一对都算平方误差再相加。

- **这是确定性策略：**讲者当场改了幻灯片——原来写成从 π_{θ} 里采样，其实这一版的策略只输出一个动作，做的就是 L2 回归。而专家未必是确定性的，这就埋下了第 3 节的问题。
- 本讲把它叫 version 0；文献里的名字是**行为克隆**（behavior cloning, BC）。

3. 什么会出错 I：示范是多峰的，L2 只学得到均值

- **驾驶例子：**同一个路况，有的司机直行（转向角 0），有的并线到左边的快车道（转向角约 -2）。数据集里两种都有，直行的更多。把这个状态下的动作画成分布：直行的司机在 0 附近一个峰，并线的在 -2 附近一个峰，整个数据集是两个峰叠起来的**多峰分布**（multimodal）。
- **L2 回归学到什么：**均值——两个峰之间的某个转向角，车会骑在车道线上。而且这个动作在数据分布下本身概率很低，所以它不只是"看起来蠢"，在数学上也是一个差的选择。
- **问答**
 - 为什么不直接在数据里找最像的状态、照抄动作？——那是最近邻，在有些机器学习问题上可行；但车一上路就会遇到数据里没有的情形，没法照抄。（这一点到第 10 节会变成模仿学习的核心难题。）
 - 动作不是数值怎么办？——离散动作马上讲。
- **这在实践里常见吗：**讲者说"一直在发生"，尤其是数据由多个人采集时：每个人有自己的偏好和习惯，这些因素观测不到，同一个状态下自然有不同做法。只有当示范来自一个策略始终一致的专家时，确定性策略才够用。
- **自然的下一步：**既然示范本身是一个分布，能不能不只学均值，而是把整个分布学下来？

4. 让神经网络输出一个分布，而不是一个动作



- **核心做法**：网络输入状态，输出的不是动作，而是**某个分布类的参数**；分布类越有表达力，能学到的分布就越丰富。所有参数都隐含地依赖状态——同一个网络在有的状态下输出多峰分布，在有的状态下输出单峰分布（问答里有人确认了这一点）。
- **离散动作**：比如玩超级马里奥，动作是上、下、A、B、右这几个键。网络输出每个动作的概率，就是一个 **categorical 分布**，训练相当于分类。在一维离散动作上它是“最大表达力”的：没有任何动作分布是它表示不了的。可惜世界上大多数动作不是一维离散的。
- **连续动作**：最自然的是高斯分布。只输出均值 μ ，就等价于 L2 回归；再让它输出方差也没用——在双峰数据上它只会把方差撑大去盖住两个峰，还是不像样。所以要换更有表达力的分布类。
- **"表达力"是什么意思**（问答）：一个分布类能**完美表示**的分布有多少。只用高斯就只能完美表示高斯；混合高斯能表示任何单个高斯，也能表示任何“若干高斯叠加”的分布，所以严格更强。不同分布类之间有时不好比，但 GMM 对高斯是严格包含。

- **问答：完美建模了双峰分布，策略会不会"犹豫不决"，一会儿选这个峰、一会儿选那个？**——只要状态表示够好就不会：一旦采样到并线并开始向左，速度朝左这件事已经写进了状态，下一步的条件分布会集中在"继续向左"上。好的示范加好的状态表示，再把分布拟合好，出来的就是好策略。
- **借生成模型的思路：**图像扩散模型能生成彼此完全不同的人脸和视频，语言模型一个 token 一个 token 地预测——两者都远比"输出均值和方差"或"一维 categorical"有表达力。它们通常用来建模"给定文字的图像"或"给定前文的下一个词"，但底层技术是通用的，同样可以拿来学给定状态的动作分布 $p(a | s)$ 。本讲讲三种：混合高斯、离散化 + 自回归、扩散。
- **混合高斯 (GMM)：**网络输出 K 组参数，每组是一个高斯的均值、标准差和权重。 K 要事先定，当超参数调，看不同 K 拟合得好不好。

$$\pi_{\theta}(a | s) = \sum_{k=1}^K w_k(s) \mathcal{N}(a; \mu_k(s), \sigma_k(s)^2), \quad \sum_{k=1}^K w_k(s) = 1$$

K 个高斯各有自己的均值 μ_k 、标准差 σ_k 和权重 w_k ，三者都是网络读入 s 之后输出的；权重非负且加起来是 1，所以整体仍是一个合法的概率密度。

5. 离散化 + 自回归：像语言模型一样一维一维地出动作

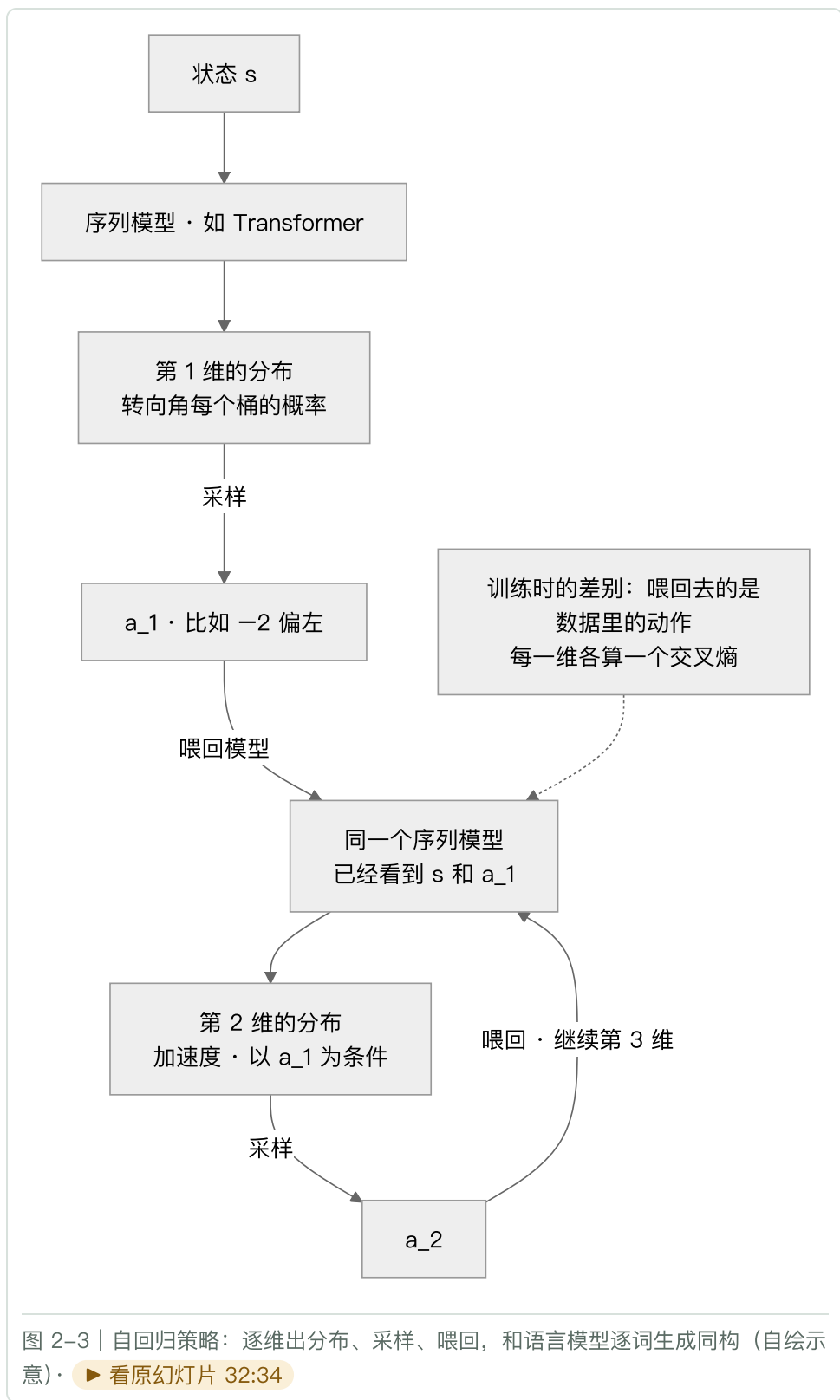


图 2-3 | 自回归策略：逐维出分布、采样、喂回，和语言模型逐词生成同构（自绘示意）· [▶ 看原幻灯片 32:34](#)

- **什么时候用：**动作本来就是离散的但有多维度；或者动作连续且多维——先离散化（tokenize），再上自回归模型。
- **简化的驾驶例子：**动作有两维，转向角和加速度。
 1. **离散化：**转向角的真实分布高斯画不出来，就把角度切成若干桶，网络给每个桶输出一个概率，得到一个"阶梯"形的近似。桶切得越

细越像原分布，但要输出的数也越多——这是个权衡。

2. **自回归**：加速度要以转向角为条件——并线时因为要插进左边的车流，通常要加速；直行则倾向保持车速。所以先输出转向角的分布，采样一个 a_1 （比如 -2 ），把 a_1 喂回模型，再输出以它为条件的加速度分布：条件是 -2 时，高加速度的概率大；条件是 0 时，分布落在小加速度上。第三维同理，以前两维为条件——用序列模型（比如 Transformer）的话，它天然看到之前所有的维度。

- **和语言模型完全同构**：起始 token $\rightarrow$ 下一个词的分布 $\rightarrow$ 采样 $\rightarrow$ 喂回 $\rightarrow$ 再下一个词。只是这里迭代的是动作的各个维度，不是句子里的词。

$$\pi_{\theta}(a | s) = \prod_{d=1}^D \pi_{\theta}(a_d | s, a_1, \dots, a_{d-1}), \quad \mathcal{L}_{AR}(\theta) = - \sum_{d=1}^D \log \pi_{\theta}(a_d^{\text{data}} | s, a_1^{\text{data}}, \dots, a_{d-1}^{\text{data}})$$

a_d 是动作的第 d 维（离散化之后是一个桶的编号），每个因子都是一个 categorical 分布；loss 是各维交叉熵之和，条件里用的是数据里的动作 a^{data} ，测试时才换成自己采样的。

• 问答

- 维度顺序怎么定？——语言里词序天然存在；动作维度的顺序一般手工挑，也可以当超参数；有时有自然的选择，比如机械臂沿运动链（kinematic chain）从基座依次到各关节。好消息是表达力不随顺序变：你表示的始终是所有维度的联合分布，一维一维出只是因为一次性预测整个联合分布要输出的格子数是桶数的维度次方（每维 100 个桶、10 维动作就已经爆炸），逐维分解便宜得多。
- 用自回归是不是就必须离散化？——离散化有两个好处：每个桶的高度随意，能表示比混合高斯复杂得多的多峰分布（上限只受桶粒度限制）；自回归则负责解决高维带来的组合爆炸。原则上自回归也能配 GMM 或高斯，只是既然都用自回归了，离散化通常是更有力的选择，所以不常见。
- 训练数据怎么造？——采一对 (s, a) ，按分桶策略把 a 的每一维映射到桶，每一维用交叉熵 loss，等于每一维做一次分类。
- 交叉熵不知道“隔壁桶”和“隔十个桶”的差别？——这是离散化的一个缺点。实践中这种距离关系学得挺快，结果通常没问题，但可能需要更多数据；桶的粒度是非常重要的超参数——切太细，稍微偏一个桶就算全错，信号很难学到。更进一步可以做标签平滑，让标签不是一个 one-hot，而是在邻近的桶上也分一点。

- 离散的东西怎么反向传播？——离散的只是标签；网络输出的是每个桶的概率（连续值），交叉熵直接对它求导，和分类问题一样。只有权重本身离散时反向传播才有麻烦。

小注：训练时喂数据里的动作、推理时喂自己采样的动作，这套做法在序列建模里叫 teacher forcing，讲者描述了做法但没用这个词。两者不一致带来的问题（exposure bias）和第 10 节的 compounding errors 是同一种病。

6. 训练目标：让数据里的动作在你的策略下概率最大

$$\min_{\theta} - \mathbb{E}_{(s,a) \sim D} [\log \pi_{\theta}(a | s)]$$

(s, a) 从示范数据集 D 里采， $\pi_{\theta}(a | s)$ 是正在学的策略给这个“专家动作”的概率；取 $\log$ 、加负号变成最小化，就是负对数似然。

- 所有生成式策略共用的目标：**网络输出的是分布，就应该让数据里出现的动作在这个分布下的概率尽量大。训练循环不变：采 (s, a) ，前向得到分布，算 $-\log \pi_{\theta}(a | s)$ ，反向传播。
- 不同模型里这一项长什么样**
 - GMM：直接写出第 4 节的密度函数，对网络输出的 μ_k 、 σ_k 、 w_k 求导。
 - 离散化 + 自回归：第 5 节的逐维交叉熵，每一维一个 categorical 的负对数似然相加。
 - 扩散：似然写不出来，改用第 7 节的去噪目标。
- 讲者的幻灯片把这个式子写成对示范数据的期望，蓝色标出的是“示范里的动作”：概率是在正在学的策略下算的，动作是专家的。拟合得越好，策略和专家的行为越接近，任务表现也越好。

小注：L2 回归其实是这个目标的一个特例——把 $\pi_{\theta}(a | s)$ 取成方差固定的高斯， $-\log$ 密度就只剩 $\| \mu_{\theta}(s) - a \|^2$ 加常数。所以 version 0 不是另一个目标，而是同一个目标配了最没表达力的分布类。CME295 第 4 讲的 SFT 交叉熵、第 5 讲奖励模型的 $-\log \sigma(\text{方差})$ ，形式上也都是负对数似然。

7. 扩散策略；把网络做大不等于把分布做宽

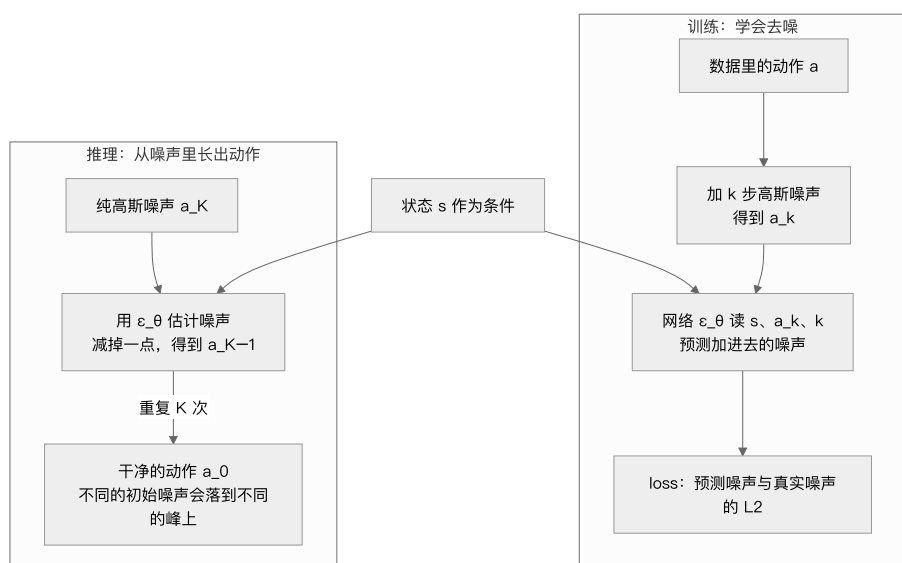


图 2-4 | 扩散策略：训练学的是“猜噪声”，推理靠迭代去噪把噪声变成动作（自绘示意）· [▶ 看原幻灯片 43:53](#) · 出处：Chi et al., 2023

- **扩散** (diffusion)：连续动作的另一个好选择，尤其是高维连续动作——图像也是高维连续变量，扩散模型在那里已经证明了自己。做法是去噪：从一团纯噪声出发，一步一步把它变得更不像噪声，最终得到一个干净的动作。
- **两种迭代**：自回归和扩散都是迭代式的，区别在迭代的方向——自回归沿着动作的维度迭代，扩散沿着去噪步数迭代。
- **课堂上的关键问题**：这些分布类都没改网络结构。那能不能还用 L2，只把网络做大？大网络不是更有表达力吗？——学生答对了：在驾驶例子里，网络再大，输出的还是均值，只是均值估得更准。这里有两种彼此独立的表达力：**函数的表达力**（从 s 到参数能表示多少种映射）和**分布的表达力**（这些参数能刻画多少种分布）。只输出 μ 、 σ 的网络可以把 s 到 (μ, σ) 的映射学得极其复杂，但它能表示的分布永远是一个高斯。

$$a^{k-1} = \alpha(a^k - \gamma \varepsilon_{\theta}(s, a^k, k)) + \mathcal{N}(0, \sigma^2 I), \quad k = K, \dots, 1$$

这是一步去噪： a^k 是第 k 步的带噪动作， ε_{θ} 是读入状态 s 、当前带噪动作和步数 k 后预测噪声的网络， α 、 γ 、 σ 是按步数预先定好的系数；从纯噪声 a^K 出发做 K 步就得到动作 a^0 。

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{(s, a^0) \sim D, k, \varepsilon^k} \left\| \varepsilon^k - \varepsilon_{\theta}(s, a^0 + \varepsilon^k, k) \right\|^2$$

训练时随机挑一个步数 k ，给数据里的动作 a^0 加上对应量级的噪声 ε^k ，让网络把这团噪声猜出来；loss 就是猜的噪声和真噪声的平方误

差。

小注：上面两个式子出自指定阅读 Diffusion Policy 论文，课上只讲了直觉。论文把每一步去噪解释成沿动作分布的 score（对数密度的梯度）走一步 Langevin 动力学，多峰分布因此天然能被表示：不同的初始噪声落到不同的峰上。论文报告在 4 个基准的 12 个任务上平均提升 46.9%，并给了三项落地技术——receding horizon（每次预测一段动作序列、只执行前几步）、视觉条件化、时序扩散 Transformer。

8. 实证：表达力值多少，业界怎么选

- **仿真：Transport 任务。**两台机械臂，把一个工具从一台传到另一台，一个策略同时控制两台。比较 diffusion 和 GMM（diffusion 的表达力远超 GMM），各在两份数据上训：一份由一个人采集，一份由多个人采集，指标是成功率。结果：diffusion 在两份数据上都不错，多人数据上略降——数据越多样，难度越高；GMM 首先整体不如 diffusion，其次在多人数据上掉得多得多。两个结论：表达力有用；数据里的行为越多样，表达力越关键。

小注：这组结果应出自 Diffusion Policy 论文的 robomimic 实验：Transport 任务的两份数据分别是 ph（一位熟练操作者）和 mh（多位操作者），GMM 对照应为 robomimic 自带的 LSTM-GMM 策略。

- **真机：把衬衫挂上衣架。**任务本身很难，数据集很大——规模一上去就没法由一个人用一致的策略采完，所以是很多人、很多种做法。比较 diffusion 和 L1 loss（后者是确定性策略）。总体成功率不如仿真任务，但结论一致：换成 L1，成功率掉了接近一半。

小注：应为 ALOHA Unleashed（Zhao et al., 2024，讲者是合著者）：ALOHA 2 双臂平台上的大规模示范加 Diffusion Policy，挂衣架是其中一个真机任务。

- **问答：有没有场景想用"其他分布的混合"而不是混合高斯？**——其他分布往往本身已经够有表达力，讲者没见过，但也不是完全不可想象。
- **业界的大模型都建在这些基础上：**机器人领域 NVIDIA、Figure、Physical Intelligence 的基础模型，以及开源的视觉-语言-动作模型（VLA），都是大规模示范数据配表达力强的分布；自动驾驶里 Waymo 和 Wayve 的模型用的是离散化 + 自回归预测。

小注：讲者没点论文名，推断分别指 GR00T N1、Helix、 $\pi 0$ 和 OpenVLA；其中 OpenVLA 把动作离散成 token 走自回归， $\pi 0$ 和 GR00T

N1 用 flow matching (扩散的近亲)。Waymo 的应为 EMMA, 把轨迹写成文本 token 让多模态大模型自回归生成。

- **问答：这些大模型里除了课上讲的还有什么？——不止这些。**第一个例子的功夫花在数据筛选和模型架构上；NVIDIA 的人形机器人数据很难采，重点在怎么用仿真数据；Figure 的架构设计是为了让模型跑在很高的控制频率上；OpenVLA 也有不少架构选择；此外还有各种联合训练目标。但今天讲的内容是这些系统背后最主要的驱动力。

9. 小结：什么时候要分布；offline 还是 online

- **小结：**示范来自一个行为一致的人，学单峰策略就够；示范来自多个人、多峰，就要学分布，而且要用表达力强的生成模型。
- **这套算法是完全 offline 的——这是 RL 里非常重要的一对概念：**
 - **offline 算法**只用一份现成的数据集，不从正在学的策略那里拿任何新数据。
 - **online 算法**会用学到的策略去采新数据。代价有两条：边学边采，贵；策略没练好时上线可能不安全，比如医疗场景你不会想让半成品策略去收数据。

小注：offline / online 说的是"学习过程中要不要和环境交互"，和 CME295 里见过的 on-policy / off-policy 不是一回事：后者说的是"训练用的数据是不是当前策略生成的"。online 的 RL 既可以是 on-policy (只用当前策略刚采的数据，第 3、4 讲)，也可以是 off-policy (把旧数据存进 replay buffer 反复用，第 5、6 讲)；完全不交互的 offline RL 是第 7 讲。行为克隆是 offline 的，数据来自专家而非当前策略，也就天然 is off-policy 的。

- **模仿学习的优点：**不需要策略自己采数据 (offline)；不需要写奖励函数——第 1 讲说过驾驶这类任务的奖励很难写全，这里完全绕开。
- **缺点：**要想可靠，往往需要非常多的示范。这就引出了后半讲：从在线的纠正和干预中学习。
- 讲者原计划"时间允许的话"再讲怎么采集示范，这一讲没讲到。

10. 什么会出错 II：错误会滚雪球

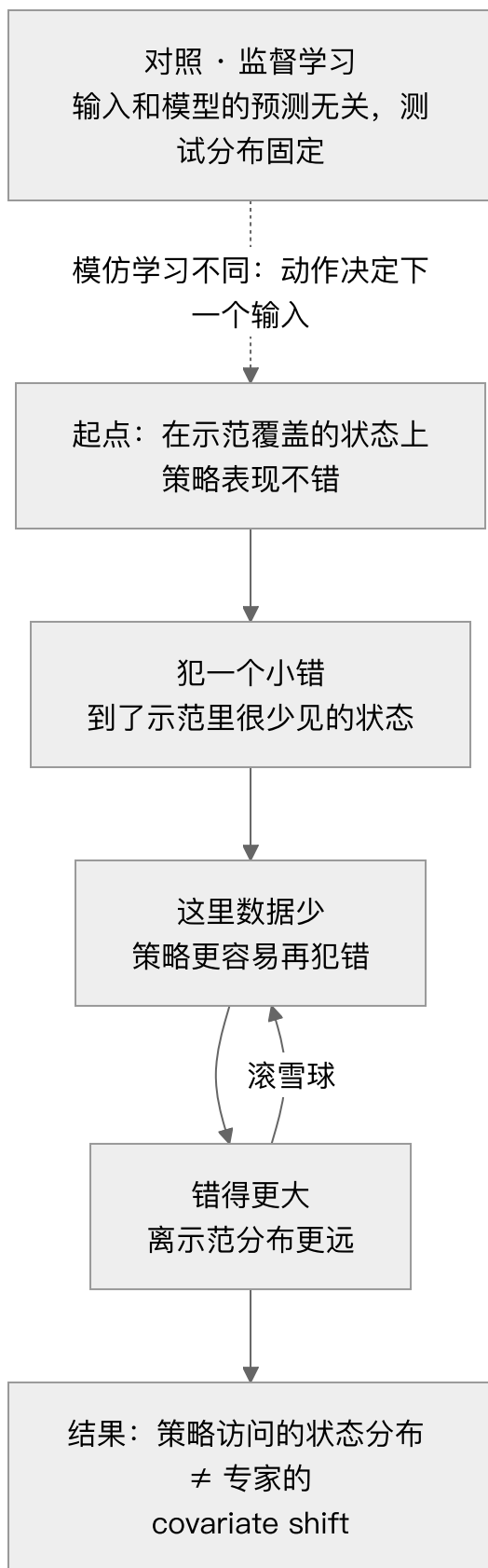
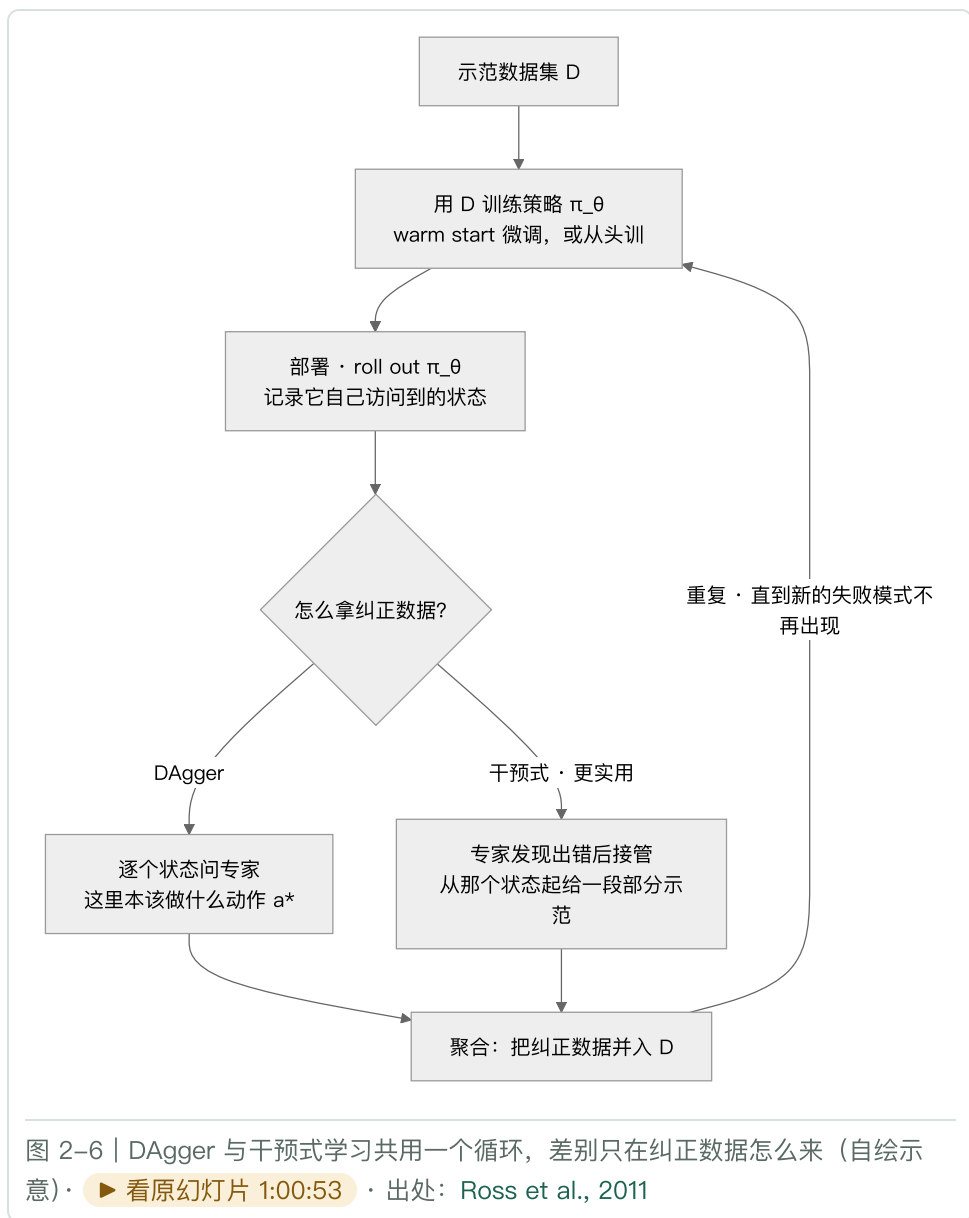


图 2-5 | compounding errors: 一个小错把策略推离示范分布，数据越稀错越大
(自绘示意) · [▶ 看原幻灯片 55:40](#)

- **和监督学习的根本差别**：监督学习用一份固定的数据集，模型的输入和它自己的预测无关。模仿学习不是——策略的动作决定它下一步会看到什么状态。
- **滚雪球的过程**：把一条示范想成一串状态点。策略从起点出发，一开始表现不错；但它不可能完美，某一步犯了个小错，跑到了示范附近但没被覆盖的状态。这个状态在数据里很少见，于是策略在这里更容易再犯错，错了就离示范更远，数据更稀，错得更大……这就是 **compounding errors**。
- **结果**：策略实际访问的状态分布，和专家访问的状态分布不一样。这叫 **covariate shift**，是分布偏移的一种。它让模仿学习从根本上区别于监督学习，也直接拖低表现。
- **和前几节的关系**：分布拟合得不好（比如只学均值），就更容易迈出第一个错步；但就算拟合得很好，也还是会犯错、会漂移。所以表达力缓解不了这个问题的根源。

小注：这条直觉有定量版本（应为 Ross & Bagnell, 2010 和下一节的 DAgger 论文）：单步错误率为 ϵ 时，行为克隆在长度 T 的轨迹上的期望代价可以差到 $\epsilon \cdot T^2$ 量级，而 DAgger 能把它压回 $\epsilon \cdot T$ 量级。CME295 第 5 讲说 DPO 吃现成偏好对会有分布偏移、PPO 在线采样没有，本质上是同一件事。

11. DAgger 与从干预中学习



- **最笨的办法：**堆数据、训到底、祈祷。如果有这个条件，讲者说这并不算坏。
- **更受欢迎的办法：收集纠正数据（corrective data）。**策略开始跑偏时，记录"在这个状态下正确的动作是什么"。把这些纠正数据和原始示范合在一起重训，策略就有了"犯错之后怎么回来"的例子——关键在于纠正是在**策略自己访问到的状态**上收集的，正好补上 covariate shift 造成的空洞，把数据覆盖的状态分布拓宽到"专家会去的地方"加"策略犯错后会去的地方"。
- **DAgger 的循环**（Dataset Aggregation；讲者说这名字起得很糟，但术语得认识）
 1. 用当前策略 roll out，记下它访问的状态；
 2. 在这些状态上向专家查询"本该做什么"，得到 a^* ；

3. 把这些 (s, a*) 聚合进原数据集；
 4. 在聚合后的数据上更新策略——通常从原策略 warm start 微调，担心偏差也可以从头训；
 5. 重复：新策略可能以新的方式失败，再针对它收纠正数据。
- **DAGger 的代价：**数据效率高，但"查询专家"可能很难做到——要拿着策略走过的一串 $s_1, a_1, s_2, a_2, \dots$ ，逐个状态问人"这里转向角本该是多少"，很多场景下人给不出这种逐步的动作标签。
 - **干预式：让专家接管。**不逐状态问，而是让策略跑，专家看到出错就整个接过控制（比如直接开车），从那个状态起给一段**部分示范**；把这些部分示范聚合进数据集、更新策略、重复。
 - 什么时候算"出错"是主观的，由示范者决定；通常要主动一点，在撞上之前就接管。
 - 缺点：有些错来得又快又不可预测（比如动作突然大变），人来不及接；最适合的是漂移得慢的错误，比如慢慢滑向隔壁车道时把它拉回来。
 - 现实中自动驾驶公司就是这么干的：让策略开，安全员在策略"卡住不动"或开始犯错时接管。也出过安全员没能阻止严重事故的案例——把"防大错"寄托在示范者身上有风险，但作为提供纠正的交互方式它很实用，对性能提升很大。
 - 问答：不管哪种办法，策略的上限都是专家。

小注：接管式的变体文献里叫 HG-DAGger (human-gated DAGger, Kelly et al., 2019)，提出动机正是讲者说的两点：逐状态标注时人没有真正在控制系统，标签质量差，也不安全。DAGger 原文是 Ross, Gordon & Bagnell, 2011。

- **问答：纠正数据太窄会不会过拟合？**——会。原始示范来自全美各地，纠正数据只在 Stanford 校园采，合并后策略可能只在校园那种场景里学会纠正，还可能整体向校园数据过拟合。纠正数据要在不比原数据窄的、足够多样的场景里采。
- **往后看**（按课程目录推断）：第 8 讲 Reward Learning 应会讲另一条从示范出发的路——不克隆动作，而是从示范里反推奖励再做 RL；第 15 讲会回到模仿学习与分层策略；第 16、17 讲的机器人内容会大量用到本讲的方法。

图	看什么	跳转	出处
模仿学习问题设定	示范 D 是 (状态, 动作) 对, 来自未知的 π_{expert} ; 目标是学 π_{θ} 追平专家	2:41	—
直行 vs 并线的三条曲线 (板书)	两个子群各一个峰, 合起来是双峰; $L2$ 的均值落在两峰之间、概率很低	8:53	—
幻灯片版多峰示意	黄色直行、蓝色并线、黑色虚线是合并数据; 确定性策略选在黑线的均值	12:30	—
网络输出分布参数 (板书)	状态 $\rightarrow$ 网络 $\rightarrow$ 分布的参数 $\rightarrow$ 分布; 表达力取决于最上面那层分布类	15:07	—
离散化的阶梯分布 (板书)	转向角分桶后每桶一个概率; 桶越细越像原分布但输出越多	28:32	—
序列模型逐维出动作 (板书)	采样 a_1 喂回模型再出 a_2 的分布, 和语言模型逐词生成同构	32:34	—
负对数似然目标 (板书)	所有生成式策略共用的目标: 让数据里的动作概率最大	38:12	—
三种生成式策略一页	离散化自回归逐维预测的示意, 以及扩散从噪声去噪到动作	43:53	Diffusion Policy
Transport 成功率柱状图	diffusion vs GMM、单人 vs 多人数据; GMM 在多人数据上掉得最狠	47:57	应出自 Diffusion Policy
真机挂衬衫对比	多人采集的大数据集上 diffusion vs $L1$; $L1$ 成功率掉近一半	49:28	应为 ALOHA Unleashed
业界大模型一页	机器人基础模型与 Waymo、Wayve 驾驶模型各自的动作分布选择	50:29	—
compounding errors 板书	小错 $\rightarrow$ 罕见状态 $\rightarrow$ 更大的错; 策略访问的状态分布偏离示范	55:40	—
DAgger 算法一页	roll out $\rightarrow$ 查询专家 $\rightarrow$ 聚合 $\rightarrow$ 更新 $\rightarrow$ 重复; 绿色是专家在策略状态上的动作	1:00:53	Ross et al., 2011
从干预中学习一页	专家接管后给部分示范, 聚合后更新; 和 DAgger 的差别只在怎么拿纠正数据	1:04:25	应为 HG-DAgger

提到的工作

名称	在本讲里的作用
第 1 讲的记号: state / observation、action、trajectory、reward、policy	开场回顾; 今天全部用上
Behavior cloning ($L2$ / $L1$ 回归)	version 0: 对专家动作做监督回归; 实验里作为 $L1$ 对照
自动驾驶示范; Super Mario	贯穿全讲的连续动作例子; 离散动作的例子
高斯 / 混合高斯 (GMM)	连续动作的分布类: 单高斯不够, GMM 更强, K 是超参数

名称	在本讲里的作用
图像 / 视频扩散模型；自回归语言模型	生成模型的两个例子，说明同样的技术能建模 $p(a s)$
Transformer	自回归策略里用的序列模型
标签平滑 (label smoothing)	缓解离散化后交叉熵不知道桶间距离的办法
Diffusion Policy (Chi et al., 2023)	指定阅读；扩散策略；Transport 实验应出自此文
ALOHA Unleashed (Zhao et al., 2024)	应为真机挂衬衫 diffusion vs L1 的出处
NVIDIA、Figure、Physical Intelligence 的机器人基础模型；开源 VLA	大规模示范加表达力强的分布的工业实例（推断：GR00T N1、Helix、 $\pi 0$ 、OpenVLA）
Waymo、Wayve 的驾驶模型	离散化 + 自回归动作预测的例子（Waymo 的应为 EMMA，推断）
Dagger (Ross, Gordon & Bagnell, 2011)	在策略访问的状态上查询专家、聚合数据、重训
HG-Dagger (Kelly et al., 2019；应为)	专家接管、给部分示范的干预式变体
自动驾驶安全员	干预式学习在工业界的现实版本，也有失败案例
Offline RL (第 7 讲)	offline / online 的区分在后面课程里反复出现

术语对照

English	中文
imitation learning	模仿学习：从专家示范里学策略，不需要奖励函数
demonstration (D)	示范：专家采集的轨迹，一堆（状态, 动作）对
expert policy π_{expert}	专家策略：生成示范的那个未知策略
state / observation	状态 / 观测：完整的世界描述 / 只看到其中一部分
action / trajectory	动作 / 轨迹：每步的决策 / 一段状态与动作的序列
reward function	奖励函数：给（状态, 动作）打分；模仿学习不需要它
policy π_{θ}	策略：给定状态（或观测历史）输出动作或动作分布， θ 是网络参数
behavior cloning (BC)	行为克隆：对示范做监督学习

English	中文
supervised regression / L2 loss / L1 loss	监督回归 / 平方误差 / 绝对误差；两者训出的都是确定性策略
deterministic / stochastic policy	确定性 / 随机策略：输出一个动作 / 输出一个分布
multimodal distribution	多峰分布：同一状态下有几种不同的合理做法
expressiveness / expressive distribution class	表达力：一个分布类能完美表示的分布有多少
categorical distribution	类别分布：每个离散动作一个概率，一维离散时表达力最大
Gaussian / mixture of Gaussians (GMM)	高斯 / 混合高斯：K 个高斯按权重叠加
discretize / tokenize / bin	离散化 / 词元化 / 分桶：把连续动作切成有限个桶
autoregressive model	自回归模型：一维一维地出，后一维以前面的为条件
cross-entropy loss	交叉熵：分类的标准 loss，等于 categorical 的负对数似然
label smoothing	标签平滑：one-hot 标签在邻近桶上也分一点概率
kinematic chain	运动链：机械臂从基座到末端的关节顺序
diffusion model / denoising	扩散模型 / 去噪：从噪声迭代生成
negative log-likelihood (NLL) / maximum likelihood	负对数似然 / 最大似然：让数据在模型下的概率最大
generative model	生成模型：能表示并采样一个复杂分布的模型
vision-language-action model (VLA)	视觉-语言-动作模型：吃图像和语言、吐动作的机器人基础模型
success rate	成功率：机器人任务的常用指标
offline / online algorithm	离线 / 在线算法：只用现成数据 / 用学到的策略去采新数据
compounding errors	错误累积：小错把策略带到没数据的状态，接着错得更多
covariate shift / distribution shift	协变量偏移 / 分布偏移：策略访问的状态分布和专家不一样
corrective data	纠正数据：在策略自己访问的状态上由专家给的正确动作
DAgger (dataset aggregation)	数据集聚合：roll out → 查询专家 → 聚合 → 重训
intervention / partial demonstration	干预 / 部分示范：专家接管后从当前状态起给的一段示范

English	中文
safety driver	安全员：自动驾驶测试里随时准备接管的人
warm start / fine-tune	热启动 / 微调：从上一版策略继续训而不是从头训
overfitting / generalization	过拟合 / 泛化

字幕勘误

"decision-m problem" → decision-making problem; "gaussian / gausion" → Gaussian; "DMM" → GMM; "auto reggressive / autogressive" → autoregressive; "stoastic" → stochastic; "den noiseise" → denoise; "covariant shift" → covariate shift; "dagger" → DAgger; "Whimo" → Waymo; "Wave" → Wayve; "open VA" → OpenVLA; "PI" → Physical Intelligence (π); "L2 law" → L2 loss; "discreet" → discrete; "lay the way from" → lead away from; "PBS" → p of s (专家的状态分布); "code training" → co-training; "a T1 / a T2" → a_1、a_2 (动作的第一、二维); "oftenimes" → oftentimes。

带走的问题

- LLM 的 SFT 就是行为克隆，那 compounding errors 在 LLM 上对应什么？（训练时喂真实前文、推理时喂自己生成的 token，错一个 token 后面全在没见过的前文上继续。）RLHF 的在线采样为什么能缓解？这和 CME295 第 5 讲里 DPO 的分布偏移是不是同一回事？
- categorical 在一维离散动作上"表达力最大"，语言模型的动作正是一个 token——所以它从不需要 GMM 或扩散。把这个逻辑反推到机器人：什么时候离散化 + 自回归就够，什么时候必须上扩散？桶粒度、动作维度和控制频率各起什么作用？
- DAgger 假设专家能在策略访问到的任意状态上给出动作。在你做的 agent 产品里，"专家"是谁？用户的重试、编辑、手动接管算不算干预式纠正数据？怎么避免"纠正数据只在 Stanford 校园采"式的过拟合？
- 讲者说完完美拟合多峰分布加上好的状态表示就够了。如果状态表示不好——比如只看当前一帧、看不出已经开始并线——策略会怎样？这 and 第 1 讲里 observation 与 state 的区别、以及"以观测历史为条件的策略"有什么关系？

5. 自回归和扩散都是迭代过程，一个迭代在动作维度上，一个迭代在去噪步上。对需要高频控制的机器人，两者的推理延迟各意味着什么？Diffusion Policy 论文的 receding horizon（一次预测一段动作、只执行前几步）是怎么摊薄这个成本的，又会带来什么新问题？

CS224R 第 3 讲 | 策略梯度 (Policy Gradients)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 3 讲, 2025 年 4 月 9 日 · 课程的第一个 online RL 算法; 下一讲的 actor-critic 与 PPO 直接接在它后面 视频: <https://www.youtube.com/watch?v=KCAOXd4lO9o> (1:02:37, 英文字幕是自动生成的, 数学符号几乎全被听成别的词, 见文末勘误) 讲者: Chelsea Finn (课程主讲; 本讲不是客座) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: Williams, 1992 — Simple statistical gradient-following algorithms for connectionist reinforcement learning (REINFORCE 原文)

一句话: RL 的目标 $J(\theta)$ 是"按当前策略采样出的轨迹, 回报的期望"。它不能直接求导: θ 藏在采样过程里, 采样又要经过我们不知道的环境动力学。log-derivative trick 把梯度搬进期望, 再把轨迹概率展开, 初始分布和动力学项不含 θ 、求梯度时自动消失, 只剩策略自己的 $\nabla \log \pi_\theta(a_t | s_t)$; 用采样轨迹的平均去估计它, 就是 REINFORCE——本质是"模仿自己采的数据, 按回报加权"。这个估计量噪声极大, 课上用三个机器人例子演示了它会把策略推向哪些荒唐方向, 并给了两个不改变期望梯度的降噪手段: 只用动作之后的奖励 (reward-to-go)、减掉一个 baseline。最后指出它是 on-policy 的——每更新一步就得重新采样——用 importance sampling 的概率比可以在同一批数据上多走几步, 下一讲的 actor-critic 与 PPO 从这里接着讲。

时间轴

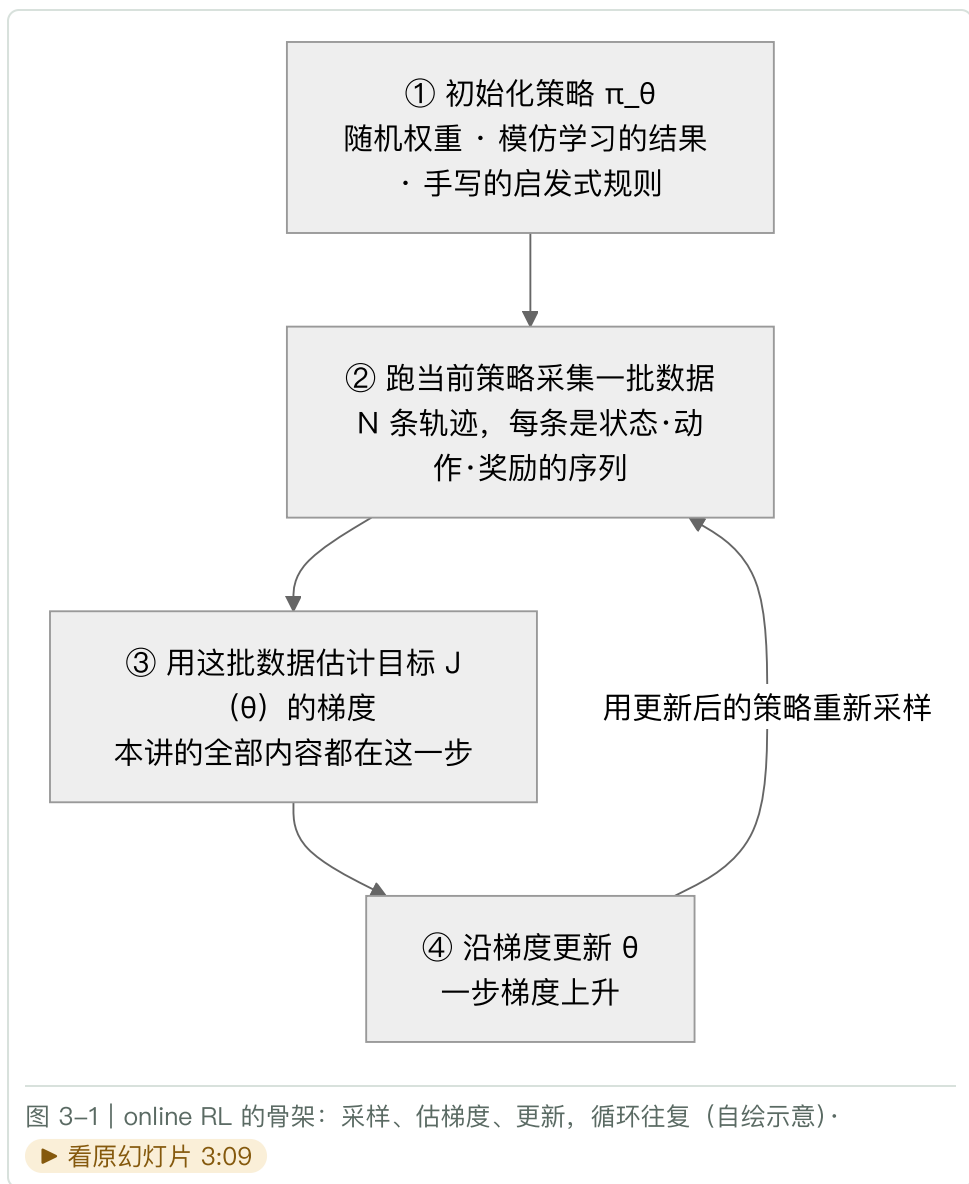
时间	内容
0:06	回顾: 状态、动作、轨迹、奖励、策略; RL 目标是期望回报; 第 2 讲模仿学习的两个短板: offline 与 online; 今天讲第一个 online 算法
3:09	online RL 的骨架: 初始化 → 采样 → 用数据改进 → 重复; 评估一个策略只能靠跑、取样本平均
5:44	目标 $J(\theta)$: 轨迹分布 $p_\theta(\tau)$ 与回报 $r(\tau)$; 为什么不能直接对采样求导
7:50	推导: 期望写成积分、梯度搬进积分、log-derivative trick、再写回期望
13:33	展开 $\log p_\theta(\tau)$: 初始分布和动力学不含 θ , 只剩 $\sum \nabla \log \pi_\theta$; 最终公式与 REINFORCE 的名字
19:14	完整算法三步; 问答: 怎么从数据算 $\nabla \log \pi$ (前向、取对数概率、反向); 自动微分
21:17	直觉: 第一项就是模仿学习的梯度, 第二项按回报加权; 问答: 和模仿学习的区别、稀疏奖励、初始状态、先模仿再 RL; 红绿轨迹图

时间 内容

26:56	练习一：人形机器人学走路，奖励是前向速度；答案是"鼓励向前摔、不鼓励迈步"——高方差；为什么要采多条轨迹
30:59	改进一：因果性 / reward-to-go；问答：先进再退为什么仍难、固定步长时状态要带剩余步数
37:12	练习二：四条都在前进的轨迹，全是正回报，连踉跄也被鼓励；对奖励尺度敏感
41:46	改进二：减去 baseline；板书证明减常数不改变期望梯度；问答：无偏与降方差、b 每批算一次、最优 baseline
48:32	练习三：叠夹克的稀疏奖励，三条失败轨迹得到一模一样的负梯度；小结：要大 batch 与稠密奖励；问答
52:37	实现：写一个 surrogate objective，一次前向一次反向；离散动作是交叉熵，高斯策略是平方误差
54:42	麻烦：on-policy——每走一步梯度都要重新采样；on-policy 与 off-policy 的定义
56:15	Importance sampling：用旧策略的样本估计新策略下的期望；推导与两个条件
59:20	按轨迹的比值是连乘会失控，改成按时间步，状态分布之比近似为 1；最终形式；收尾：下一讲 actor-critic 与 PPO

核心内容

1. 回顾与今天的位置：第一个 online RL 算法



- **RL 的基本词汇**（第 1 讲定义过，这里再过一遍）：环境在每个时刻处于一个**状态** s_t （机器人的关节角、游戏画面；在 LLM 里就是 prompt 加已经生成的 token）；智能体（agent）选一个**动作** a_t ；环境给一个**奖励** $r(s_t, a_t)$ ，一个数，说明这一步做得好不好。一条从头到尾的“状态、动作、状态、动作……”序列叫**轨迹** τ ；沿轨迹把奖励加起来叫**回报**（return），本讲直接写成 $r(\tau)$ 。要学的是**策略** $\pi_\theta(a_t | s_t)$ ：给定状态输出动作的概率分布， θ 是神经网络的权重。RL 的目标是让期望回报最大；讲者把它写成“对轨迹分布求期望”，这个分布同时由策略、环境动力学（可能是随机的）和奖励函数决定。
- **第 2 讲的模仿学习**：有专家示范时，最大化示范动作的似然，让策略学着做专家会做的事。简单、好扩展，但两个短板：超不过示范者；没有“练习后变好”这回事。

- **offline 与 online**: offline 算法只用一份现成的数据集，不拿学到的策略去采新数据；online 算法要用当前策略去采新数据。今天开始讲 online RL，第一个算法就是**策略梯度** (policy gradient)。讲者说它是训练腿式机器人和语言模型的那些算法的底座，也是课程默认项目的一部分；今天只走到一半，周五（第 4 讲）补齐剩下的部分——那就是 PPO。

小注：CME295 第 5 讲的 PPO 和 CS329A 第 6 讲的 GRPO 都直接给出了“ $\nabla \log \pi$ 乘以 advantage”这个形状，本讲是它的来历：第 2–4 节推公式，第 7–8 节解释 advantage 里的两个成分 (reward-to-go 与 baseline)，第 11 节推出 PPO 里那个概率比。

- **骨架** (图 3–1)：初始化策略（随机权重、模仿学习的结果、或手写的启发式规则，看问题而定）；跑策略采一批数据；用这批数据改进策略；再用改进后的策略采新数据。讲者的 2D 导航例子里，策略就是这样一轮轮从“多数失败”走到“稳定成功”的——这就是 online RL：一边采数据一边改。
- **怎么知道一个策略好不好**：看权重看不出来，只能跑。要估计“期望回报”这个量，就跑策略多次、观察每次的回报、取平均，这个样本平均就是对期望的估计（统计里叫 Monte Carlo 估计）。采数据本身不难：从策略采一个动作，观察下一个状态，再喂给策略，循环到结束，这一整条叫一次 rollout。难的是采完之后怎么改策略。

2. 目标函数： θ 藏在采样里，梯度不能直接求

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(\tau)], \quad r(\tau) = \sum_{t=1}^T r(s_t, a_t), \quad p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

τ 是一条轨迹， $r(\tau)$ 是它 T 步奖励的总和； $p_{\theta}(\tau)$ 是“用策略 π_{θ} 跑出这条轨迹”的概率，等于初始状态的概率、每一步策略选中该动作的概率、每一步环境转到下一状态的概率三者连乘。 $J(\theta)$ 就是平均而言一条轨迹能拿多少回报。

- **符号约定**：讲者把轨迹分布写成 $p_{\theta}(\tau)$ ，下标 θ 强调这个分布随策略变； $p(s_{t+1} | s_t, a_t)$ 是环境的**动力学** (dynamics)，我们不知道它长什么样，只能与环境交互时观察到样本。
- **为什么难**：我们习惯用梯度法训神经网络，那就需要 $\nabla_{\theta} J(\theta)$ 。但 θ 对 J 的影响完全是通过“采样出哪些轨迹”实现的：不能对采样过程求导，而且采样还经过未知的动力学。所以要绕一条路——目标是把它改写成“某个可以逐样本求导的量的期望”，这样就能先采样、再对每个样本求导、最后平均。

小注：本讲的回报是有限步长 T 内不打折扣的和。折扣因子 γ （未来第 k 步的奖励乘 γ^k ，让远处的奖励算得轻一些）要到后面讲 actor-critic 和 Q-learning 时才出现，这里不需要。

3. 推导：log-derivative trick 把梯度搬进期望

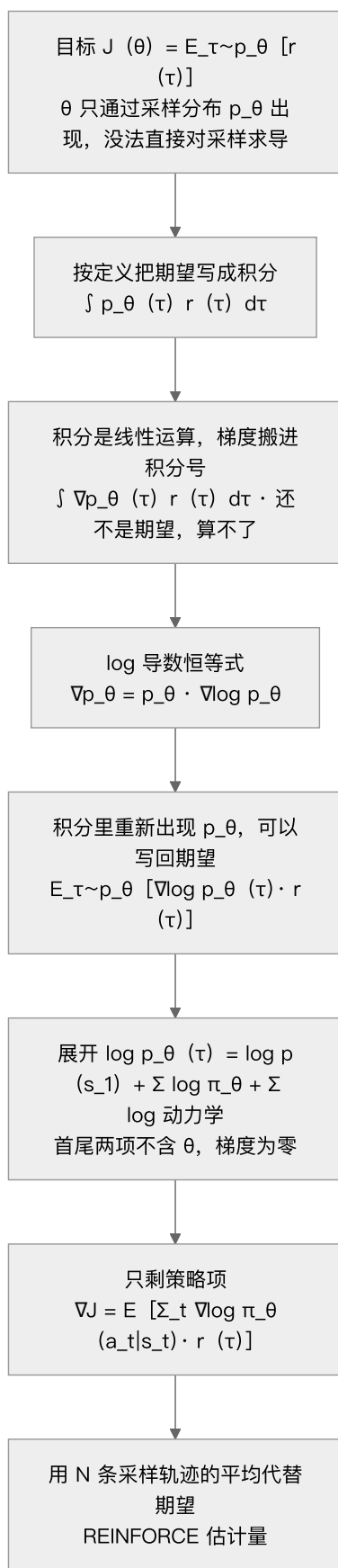


图 3-2 | 从"不能求导的期望"到"能用样本估计的梯度": 推导的六步 (自绘示意) ·

讲者在黑板上推了一遍，再回到幻灯片对照。逐步说清每一步做了什么、为什么允许：

1. **期望写成积分**。按定义， $E_{\{\tau \sim p_\theta\}}[r(\tau)] = \int p_\theta(\tau) r(\tau) d\tau$ （动作离散时就是求和）。这一步只是定义。
2. **梯度搬进积分**。积分是线性运算，梯度可以直接放到被积函数上： $\nabla J = \int \nabla p_\theta(\tau) r(\tau) d\tau$ 。到这里其实没有进展——积分里不再是" p_θ 乘以什么"的形状，不能靠采样估计；要真算这个积分，又绕不开未知的动力学。
3. **log-derivative trick**。看恒等式 $p_\theta(\tau) \cdot \nabla \log p_\theta(\tau)$ ： $\log$ 的导数是 $1/x$ ，链式法则再乘上 $\nabla p_\theta(\tau)$ ，分子分母的 $p_\theta(\tau)$ 约掉，结果就是 $\nabla p_\theta(\tau)$ 。这条恒等式"什么也没做"，却把 ∇p_θ 改写成了" p_θ 乘一个东西"。代回积分，得到 $\int p_\theta(\tau) \cdot \nabla \log p_\theta(\tau) \cdot r(\tau) d\tau$ ——现在被积函数又是" p_θ 乘某个函数"，正是期望的定义，可以写回 $E_{\{\tau \sim p_\theta\}}[\nabla \log p_\theta(\tau) \cdot r(\tau)]$ 。
 - **问答**：这跟第一行有什么区别？不还是要采样吗？——是的，还是要采样，也仍然不对采样过程求导。区别在于 θ 现在出现在期望里面、一个可以求导的表达式上：只要从 p_θ 采出很多轨迹，对每条算方括号里的量再平均，就得到一个随样本数变准的梯度估计。

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) r(\tau) d\tau = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) r(\tau) d\tau = E_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau) r(\tau)]$$

三个等号分别是"梯度进积分"、"代入恒等式 $\nabla p_\theta = p_\theta \nabla \log p_\theta$ "、"按定义写回期望"；最右边这个期望可以用采样估计， θ 只出现在 $\nabla \log p_\theta(\tau)$ 里。

4. **展开 $\log p_\theta(\tau)$** 。 $\nabla \log p_\theta(\tau)$ 仍然不是我们能算的东西——它是整条轨迹的概率，不是策略 $\pi_\theta(a | s)$ 。把 $p_\theta(\tau)$ 按定义写成连乘，取 $\log$ 变连加：初始状态的 $\log$ 概率、每一步策略的 $\log$ 概率、每一步动力学的 $\log$ 概率。对 θ 求梯度时，初始状态分布和动力学都与 θ 无关，这两项直接为零，只剩下 $\sum_t \nabla \log \pi_\theta(a_t | s_t)$ ——这是把状态喂进网络、取输出分布里那个动作的 $\log$ 概率、反向传播一次就能得到的量。**未知的动力学在这里消失了**，整个方法能成立全靠这一点。

$$\nabla_\theta \log p_\theta(\tau) = \nabla_\theta \left[\log p(s_1) + \sum_{t=1}^T \log \pi_\theta(a_t | s_t) + \sum_{t=1}^T \log p(s_{t+1} | s_t, a_t) \right] = \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t)$$

方括号里三项分别来自初始状态分布、策略、动力学；只有中间一项含 θ ，所以整条轨迹的 $\log$ 概率梯度就是各步动作 $\log$ 概率梯度之和。

5. 代回去，换成样本平均。把第 4 步塞进第 3 步的期望，再用 N 条采样轨迹的平均代替期望，就得到最终可以写进代码的估计量：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right)$$

上标 i 是第 i 条采样轨迹；第一个括号是这条轨迹里所有动作的 log 概率梯度之和，第二个括号是它的总回报；N 条相乘再平均，就是梯度的估计。N 越大估计越准，无穷多样本时是精确值，但那办不到。

- 这个式子叫 **vanilla policy gradient**（最朴素的策略梯度），对应的算法叫 **REINFORCE**——就是指定阅读 Williams 1992 那篇。

小注：REINFORCE 在 Williams 的论文里是个缩写：REward INcrement = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility，其中 offset 就是第 8 节的 baseline，1992 年那篇已经证明了减 baseline 不改变期望梯度。第 3 步的技巧在统计里也叫 likelihood-ratio 或 score-function 估计，任何“对采样分布的参数求梯度”的场合都能用。

4. 算法：从一批数据到一次更新

- **三步循环**（对应图 3-1）：从当前策略采 N 条轨迹；按上面的公式算梯度估计；对策略参数走一步梯度；用新策略重新采样，如此往复。式子里对 i 的求和就是“用 N 个样本估期望”，N 越大越好。
- **问答：第一项具体怎么从数据算**——手里是 N 条轨迹，每条 T 步的状态和动作。把状态过一遍策略网络（前向），得到动作的分布；在这个分布里查数据中实际采到的那个动作的概率（离散动作就是 categorical 分布里那一项），取 log；对它做一次反向传播，就是 $\nabla \log \pi_{\theta}(a_t | s_t)$ 。要不要手推梯度？——不用，PyTorch 这类自动微分框架替你算。

5. 直觉：按回报加权的模仿学习

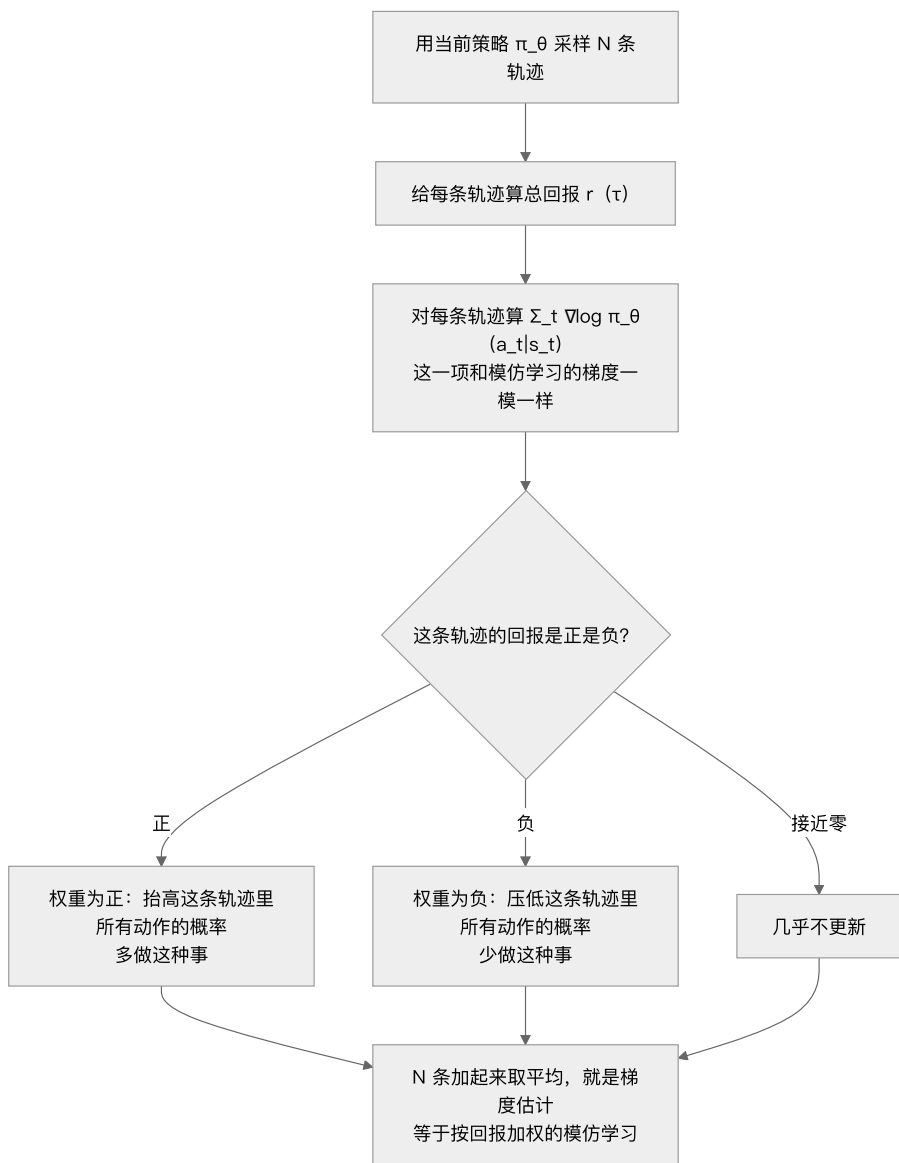


图 3-3 | 策略梯度的直觉：模仿自己采的数据，但按回报给每条轨迹加权（自绘示意）· [▶ 看原幻灯片 21:49](#)

- **第一项眼熟：** $\sum_t \nabla \log \pi_\theta(a_t | s_t)$ 正是第 2 讲模仿学习目标（最大化数据里动作的 log 似然）的梯度。单看它，意思是“抬高数据里出现过的动作的概率”。
- **第二项是权重：**整条轨迹的回报。所以策略梯度 = 模仿学习的梯度 × 回报（图 3-3）。模仿的对象是策略自己采出来的数据；回报高的轨迹权重大，动作被抬高；回报为负的轨迹权重是负数，梯度反向，动作被压低——被推离坏的、推向好的。
- **和模仿学习的两点不同：**数据是自己 rollout 出来的，不是别人给的；用了奖励——模仿学习不管数据好坏一律模仿，默认给的都是好数据。

- **图示：**几条轨迹，红色是初始的轨迹分布，其中一条回报高标成绿色。梯度一步之后，分布往绿的那条挪、离红的远。讲者的一句话总结：多做观察到的好事，少做观察到的坏事——这就是"试错"的数学化。
- **问答**
 - 奖励极稀疏（成功 1 失败 0）也能用吗？——原则上能，只是那种情况下这个算法未必好用，后面会看到。
 - 轨迹的第一个状态怎么采？——由环境决定：弹球游戏由机器给出初始画面，语言模型里就是用户的第一个问题。
 - 先做模仿学习、再切到策略梯度，合理吗？——合理。随机初始化的策略只会做随机动作，奖励稠密时也许还行；有示范就用示范初始化，再在它之上改进。反过来给示范按回报加权来改进模仿学习，也可以。
 - 不知道奖励函数该怎么定，能从示范里摸出来吗？——这是 reward learning，第 8 讲讲；实践里并不容易。

6. 练习一：五条轨迹会把人形机器人推向哪里

- **设定：**仿真里的人形机器人学走路，奖励 = 前向速度：前进为正、后退为负、不动为零。初始策略很差，采到五条轨迹，讲者让全班两两讨论"梯度会鼓励什么"：

轨迹	发生了什么	回报的符号
1	向后摔倒	负
2	向前摔倒	正——摔的方向朝前，速度为正
3	站着不动	约零
4	小步向前，然后向后摔	先正后负，总和为负
5	大步向后，然后小步向前	先负后正，总和为负

- **答案：**梯度对 1、4、5 里的所有动作是负的，对 2 是正的，对 3 约为零。于是策略被鼓励向前摔，被劝阻迈步——不管往前还是往后——也被劝阻向后摔。我们想要的是迈步，它学到的是摔。
- **这就是高方差 (high variance)：**梯度估计强烈依赖于恰好采到哪几条轨迹。换一批样本，比如碰巧有一条真的在走，梯度可能完全不同。"噪声大"和"方差高"在这里是一回事。
- **问答：**为什么同一个策略要采多条轨迹，采一条不行吗？——两个原因：策略本身是随机的，多采几条才能探索到不同的做法；环境也可

能是随机的，同样的动作会有不同结局，多采才能学到这些结局。

7. 改进一：动作只影响它之后的奖励（reward-to-go）

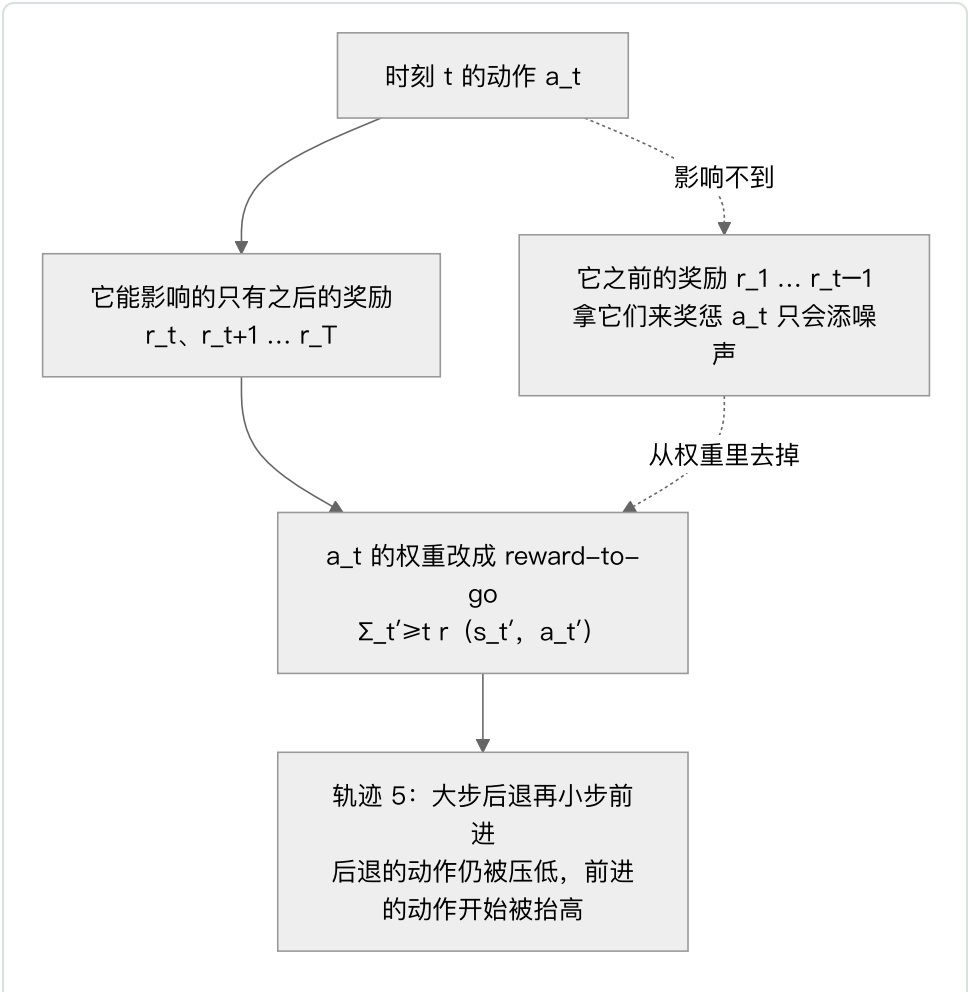


图 3-4 | 因果性：给动作 a_t 加权时，只算它之后的奖励（自绘示意）·

► 看原幻灯片 31:29

- **观察：**时刻 t 的动作影响不了 t 之前的状态和奖励。可现在的公式里，每个动作都被整条轨迹的回报加权，包括它之前的奖励；甚至会因为“前面拿了高分”而鼓励后面的动作。轨迹 5（大步后退再小步前进）本来能教会我们“小步前进是好的”，但整条回报为负，前进那几步也一并被压低。
- **改法：**把加在 $\nabla \log \pi_{\theta}(a_t | s_t)$ 上的权重，从整条回报改成从 t 开始求和 $\sum_{t' \geq t} r(s_{t'}, a_{t'})$ ——只算这个动作之后的奖励，叫 **reward-to-go**。有学生提醒了一个细节：原来是两个独立的和相乘，现在奖励的求和要搬到对 t 的求和里面，每个时刻有自己的权重。

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) \right)$$

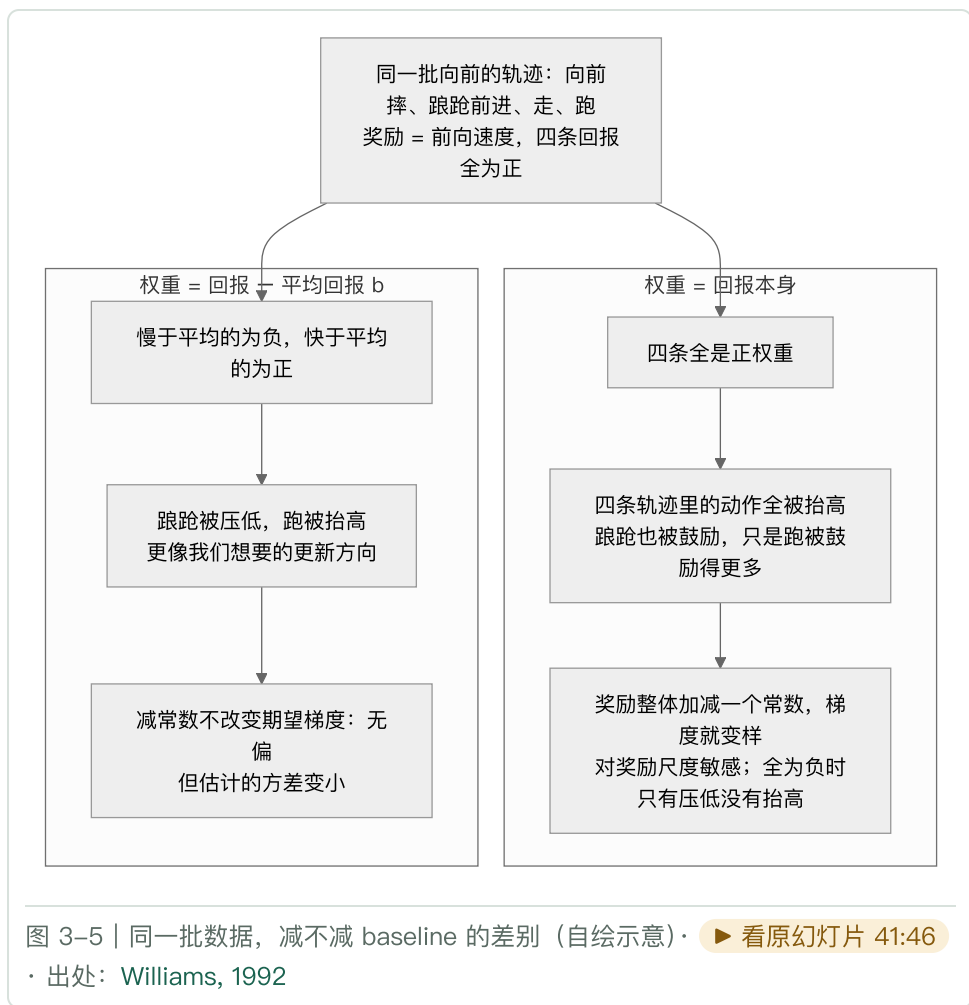
和第 3 节的式子比，奖励的求和从 $t' = 1$ 改成从 $t' = t$ 开始，并且移到了对 t 的求和里面：动作 a_t 只被它之后的奖励加权。

- **为什么允许：**讲者的解释是因果性—— a_{100} 只影响 s_{100} 之后的事，和 s_{50} 、 s_{99} 上的奖励独立，所以那些奖励可以从它的权重里去掉。她坦言课上不给严格的数学证明，这只是关于世界的因果结构的一个事实。

小注：严格证明和第 8 节 baseline 的证明是同一招。对 $t' < t$ ， $E[\nabla \log \pi_\theta(a_{t'} | s_{t'}) \cdot r(s_{t'}, a_{t'})]$ 里，先固定 t' 之前发生的一切，再对 $a_{t'}$ 求期望： $E_{a_{t'} \sim \pi_\theta}[\nabla \log \pi_\theta(a_{t'} | s_{t'})] = \nabla \sum_a \pi_\theta(a | s_{t'}) = \nabla 1 = 0$ ，所以“过去的奖励”那些项期望为零。去掉它们不改变期望梯度，只减少方差。

- **效果：**轨迹 5 里后退动作的权重仍为负，前进动作的权重变正，开始被抬高。
- **问答**
 - 先前进再后退呢？——还是难：从算法的角度看，前进那一步“可能”就是导致后来摔倒的原因，单看一条轨迹分不清；弄清动作对未来的影响正是 RL 要从经验里学的，所以后面的奖励不能丢。
 - $T = 10$ 、第 6 步做了件好事，来得及被奖励吗？——第 6 步的权重是从 6 加到 10 的奖励和，只要好处在接下来几步里兑现就能被抬高；能不能兑现取决于奖励函数。
 - 固定步长的坑：同一个状态在“还剩 4 步”和“还剩 10 步”时权重不同，那这个状态就不是真正 **Markov** 的（Markov 意味着当前状态已经包含了预测未来所需的全部信息）；正确做法是把剩余步数放进状态里，两种情形就成了不同的状态。

8. 练习二与改进二：减一个 baseline



- **练习二：**用上 reward-to-go，策略也变好了一些，四条轨迹分别是向前摔、慢慢跟跑前进、走、跑。回报全为正。课堂答案：跑得最快的回报最高，被鼓励得最多；但另外三条也是正回报，同样被抬高，只是幅度小——策略也会被鼓励去跟跑，而我们只想要跑。
- **问答：**这是局部最优吗？——不是，是方差：只要有跑的轨迹，它会一轮轮被强化，最终收敛到全局最优，只是被其他数据拖慢了。会鼓励翻滚前进吗？——没有翻滚的数据就不会。能只从部分轨迹学吗？——现在用的就是“未来的部分”；下一讲有更聪明的估计。
- **对奖励尺度敏感：**全是正回报就全部抬高；反过来若奖励全为负（比如每步一个惩罚），就只有压低、没有抬高。同一个任务给奖励整体加个常数，行为就变——这不合理。
- **baseline：**简单的修法是从回报里减一个常数 b ，比如这批数据的平均回报。高于平均的权重为正、被抬高；低于平均的为负、被压低（图 3-5）。
- **为什么可以随便减一个数：**讲者在黑板上证明，减去 b 对梯度的影响是 $E[\nabla \log p_{\theta}(\tau) \cdot b]$ 。把它写成积分；把第 3 节的恒等式反着用，

$p_{\theta} \nabla \log p_{\theta}$ 又变回 ∇p_{θ} ； b 是常数、梯度是线性运算，两者都提到积分号外面；剩下的 $\int p_{\theta}(\tau) d\tau$ 是概率的归一化，恒等于 1；对常数 1 求梯度是 0。

$$\mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) b] = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b d\tau = b \nabla_{\theta} \int p_{\theta}(\tau) d\tau = b \nabla_{\theta} 1 = 0$$

b 是与轨迹无关的常数；这条链说明“减 b ”这一项在期望上贡献为零，所以减 baseline 后的估计量和原来的估计量指向同一个期望梯度。

- 所以减 baseline **在期望上不改变梯度**——统计里叫**无偏** (unbiased)；但它能减小估计的方差。
- **问答**
 - 如果能精确算出期望， b 就没用了？——对，它只在“用有限样本估期望”时才有意义。
 - b 真的是常数吗？它看起来依赖于轨迹。——一批数据算一个 b ，这一批里的所有更新都用同一个值。
 - 方差为什么会降？——可以把梯度估计的方差写出来分析，课上不展开；直觉就是“比平均好的多做，比平均差的少做”。平均回报还不是**最优 baseline**：把方差对 b 求导取零能解出最优的 b ，它和平均回报很接近，只多了些加权项。

小注：最优 baseline 应出自 Greensmith, Bartlett & Baxter (2004, JMLR)：按 $\|\nabla \log \pi_{\theta}\|^2$ 加权的平均回报，实践里几乎没人用；更常见的是下一讲的 value function $V(s_t)$ ，一个依赖状态、同样无偏的 baseline。CS329A 第 6 讲 GRPO 的“组内平均奖励”就是本节的 baseline 在 LLM 里的样子：同一个 prompt 采 G 条回答，用组内平均当 b 。

9. 练习三：稀疏奖励下失败样本无差别

- **设定**：机器人叠夹克，奖励 1 = 叠整齐，0.5 = 叠了但有褶皱，0 = 没叠。四条轨迹：没碰夹克、只叠了袖子、把夹克铺平但没叠、叠好了。回报 0、0、0、正。
- **有 baseline 时的梯度**：第 4 条权重为正、被鼓励；1—3 权重为负、被压低。好的一面是确实在鼓励成功；不满意的有两点：
 1. 1—3 的梯度完全一样，没有任何区分——但“叠了袖子”明显比“没碰”更接近成功。这是奖励太稀疏的后果。
 2. “叠袖子”这一段其实是好的，本该拿来鼓励策略，现在整条轨迹都被压低。下一讲 (actor-critic) 有办法利用轨迹里好的那一段。

- **讲者的小结：**策略梯度依然噪声大、方差高。它最适合**稠密奖励**和**大 batch**——如果这一批里恰好没采到成功的那条，四条全是零回报，梯度就是零，策略动不了。
- **问答**
 - 改奖励函数不就行了？——是，更稠密的奖励对这个例子肯定有帮助；但"先叠好袖子、然后把衣服掉地上"这种轨迹还是用不上前半段，那要等下一讲。
 - baseline 和 reward-to-go 能一起用吗？——能，但 baseline 要想得仔细些，因为不同时刻的 reward-to-go 求和项数不一样。
 - 用平均回报当 baseline，要不要一直更新？——要。策略变好之后"平均"就变了，每批数据都重算。

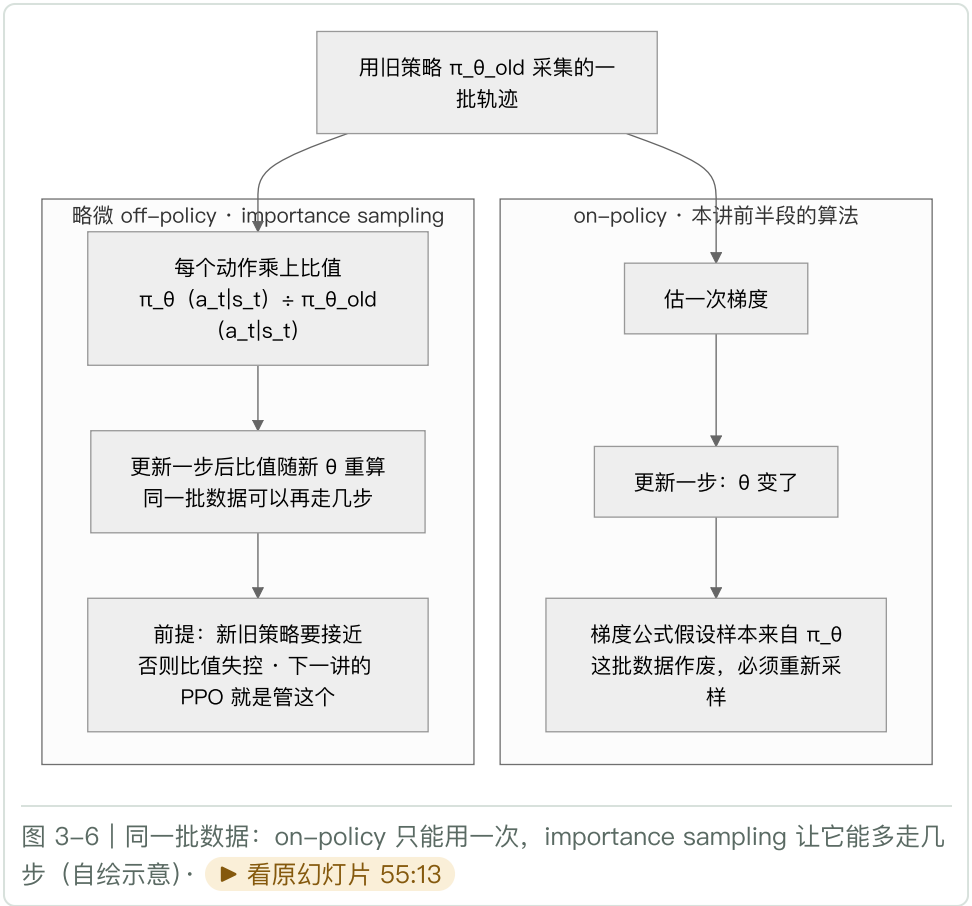
小注：LLM 里一条回答就是一条轨迹、奖励只在结尾给一次，于是每个 token 的 reward-to-go 都等于最终奖励——这就是 CS329A 第 6 讲里 GRPO 一条回答内所有 token 的 advantage 相同的原因。"稀疏奖励下失败样本无差别"也原样出现在那里：一个 prompt 采 G 条全错（或全对），组内减去平均后权重全为零，这一组对梯度没有贡献；DAPO 的 dynamic sampling 就是为这个补的。

10. 实现：写一个 surrogate objective

- **直接照公式算太慢：**估计量里有 $N \times T$ 个 $\nabla \log \pi$ 项，逐项反向传播就是 $N \times T$ 次。办法是写一个**替代目标**（surrogate objective）——它不是"最大化期望回报"那个原始目标，但对 θ 的梯度恰好等于策略梯度：把每个 $\log \pi_{\theta}(a_t^i | s_t^i)$ 乘以一个当常数处理的权重（总回报、reward-to-go、或减了 baseline 的版本），加起来除以 N，交给自动微分，一次前向一次反向。权重不含梯度，求导时只剩 $\nabla \log \pi$ 乘权重。讲者说幻灯片上这个式子有笔误：第二个求和应该在第一个求和里面。
- **落到具体的策略类：**离散动作的 categorical 策略，这个替代目标就是按权重加权的交叉熵；连续动作常用**高斯策略**（网络输出动作的均值和方差），替代目标就是加权的平方误差；也可以用更复杂的策略类，比如 diffusion policy。

小注：把同一思路用到第 11 节的 off-policy 版本，替代目标就变成"概率比 $\times$ 权重"：因为 $\nabla(\pi_{\theta} / \pi_{\text{old}}) = (\pi_{\theta} / \pi_{\text{old}}) \cdot \nabla \log \pi_{\theta}$ ，对它求导恰好得到第 11 节的梯度。CME295 第 5 讲 PPO-Clip 里的 $r_t(\theta) \cdot A_t$ 就是这个替代目标，clip 是在它上面加的限制。

11. 最后的麻烦：on-policy，以及用 importance sampling 松一松



- **采样过程本身的问题：**梯度公式假设样本来自 π_{θ} ；可第三步一更新， θ 变了，这批数据就不再来自当前策略——每走一步梯度都要重新采一批数据，而神经网络通常要走成千上万步。这种性质叫 **on-policy**：估计梯度必须用当前这个策略采的数据；**off-policy** 算法则可以复用过去的策略采的数据。这一对词是 online 算法的一个属性，和 online / offline 是两组不同的词。
- **能不能推一个 off-policy 的版本？** 讲者说只做“一点点 off-policy”：目标是在同一批数据上多走几步梯度，工具是 **importance sampling**（重要性采样），RL 之外也很有用。
- **问题设定：**想估计 $E_{x \sim p}[f(x)]$ ，但只能从另一个分布 q 采样， q 叫 **提议分布**（proposal distribution）。对应到这里： q 是采数据的旧策略， p 是刚更新过的新策略。
- **推导：**把期望写成积分 $\int p(x) f(x) dx$ ；只要 q 在 p 非零的地方也非零（术语叫有**支撑**），就可以乘一个等于 1 的因子 $q(x)/q(x)$ ，重新组合成 $\int q(x) \cdot [p(x)/q(x)] \cdot f(x) dx$ ——这又是一个期望，只不过是对 q 的。结论：从 q 采样，把每个样本的 f 乘以 p/q 这个权重再平均，就是对“ p 下的期望”的估计。

$$\mathbb{E}_{x \sim p(x)}[f(x)] = \int p(x) f(x) dx = \int q(x) \frac{p(x)}{q(x)} f(x) dx = \mathbb{E}_{x \sim q(x)}\left[\frac{p(x)}{q(x)} f(x)\right]$$

p 是想要的分布， q 是实际能采样的分布， $p(x)/q(x)$ 叫重要性权重；样本在 p 下比在 q 下更可能出现就加权，反之减权。

- **用到策略上**：目标写成对旧策略轨迹分布的期望，里面的回报乘以 $p_{\theta}(\tau)/p_{\{\theta_{old}\}}(\tau)$ 。两个条件：旧策略在新策略会走的地方必须有非零概率，否则分母为零、比值没有定义；新旧策略越接近效果越好，差得远了权重会极端。
- **从轨迹到单步**：轨迹概率之比展开成连乘，初始分布和动力学上下约掉，剩下 T 个 $\pi_{\theta}(a_t | s_t)/\pi_{\{\theta_{old}\}}(a_t | s_t)$ 的连乘。轨迹一长，几十上百个数相乘，大于 1 就爆、小于 1 就归零，数值上没法用。实践里改成对时间步（状态—动作对）求期望。这里有个小坑：严格说还要乘上新旧策略下状态分布之比，那个量算不出来，实践里近似为 1——讲者明说这是不对的，但只能这么办。于是得到最终形式：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta}(a_t^i | s_t^i)}{\pi_{\theta_{old}}(a_t^i | s_t^i)} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) - b \right)$$

样本来自旧策略 $\pi_{\{\theta_{old}\}}$ ；每一步多了一个新旧策略对同一动作的概率比，其余（reward-to-go 减 baseline）和第 7、8 节一样。 θ 更新后比值随之重算，同一批数据可以多走几步。

- **算法**：和之前一样，只是同一批数据可以走多步梯度。讲者说今天没时间讲 KL 约束——也就是怎么保证“新旧策略接近”。

小注：这个比值就是 CME295 第 5 讲 PPO-Clip 里的 $r_t(\theta)$ ，“新旧策略必须接近”在 PPO 里由 clip 或 KL 惩罚兜底；GRPO 用的也是同一个单步比值。“状态分布之比近似为 1”的合法性由 TRPO (Schulman et al., 2015) 给出：新旧策略足够接近时，用旧策略的状态分布算的替代目标与真目标之差有上界。

- **收尾**：直觉始终是“多做高回报的事、少做低回报的事”，梯度依然噪声大，大 batch 与稠密奖励下最好用。下一讲 actor-critic 紧接着今天讲，包括 PPO。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
online RL 算法骨架	初始化、采样、改进三步的循环；2D 导航例子几轮之后的变化	3:09	—

图	看什么	跳转	出处
从期望到积分的板书	$J(\theta)$ 写成积分、梯度搬进去、log trick 那几行	7:50	—
推导小结幻灯片	恒等式 $p \nabla \log p = \nabla p$, 以及代回之后的期望形式	13:03	—
轨迹概率的展开	取 log 变连加; 求梯度时首尾两项消失, 只剩 π 项	17:41	Williams, 1992
REINFORCE 三步算法	采样、估梯度、梯度步; 对 i 的求和就是 N 个样本	19:14	—
红绿轨迹分布图	更新后的分布往高回报轨迹挪、离低回报的远	25:55	—
人形机器人五条轨迹	每条的奖励正负; 小组讨论"梯度会鼓励什么"	26:56	—
reward-to-go 公式	求和下标从 $t' = t$ 开始; 奖励和搬进对 t 的求和里	32:00	—
四条前进轨迹	全正回报下四条都被抬高; 引出对奖励尺度敏感	37:12	—
baseline 无偏证明板书	反用 log trick、 $\int p = 1$ 、梯度为零	43:22	Williams, 1992
叠夹克四条轨迹	奖励 $1 / 0.5 / 0$; 三条失败轨迹的梯度完全一样	48:32	—
surrogate objective	一个能交给自动微分的替代目标 (讲者说有笔误: 第二个和应在第一个和里面)	53:08	—
importance sampling 推导	乘 q/q 等于 1, 把对 p 的期望改写成对 q 的期望	56:46	—
轨迹比值的连乘	动力学约掉后剩策略比值连乘; 为什么要改成按时间步、状态分布之比近似为 1	59:51	应为 TRPO

提到的工作

名称	在本讲里的作用
模仿学习 / behavior cloning (第 2 讲)	策略梯度的第一项就是它的梯度; 也可以用来初始化策略
REINFORCE / vanilla policy gradient (Williams, 1992)	本讲主角, 指定阅读; baseline 的无偏性也出自这篇
log-derivative trick (likelihood-ratio / score-function 估计)	推导的核心一步: $\nabla p = p \nabla \log p$
腿式机器人、语言模型的 RL 训练	讲者点名的应用, 算法底座是策略梯度
PyTorch 自动微分	算 $\nabla \log \pi$ 不用手推
Markov 性质	固定步长时要把剩余步数放进状态, 否则状态不 Markov
最优 baseline (应出自 Greensmith et al., 2004)	平均回报不是最优 baseline, 但足够好

名称	在本讲里的作用
categorical / Gaussian policy	离散 / 连续动作的策略类；替代目标分别是加权交叉熵 / 加权平方误差
diffusion policy（应为 Chi et al., 2023）	更复杂的策略类，一句带过
importance sampling	推 off-policy 版本的工具
actor-critic、PPO（第 4 讲预告）	下一讲；今天的概率比就是 PPO 的 $r_t(\theta)$
KL 约束（TRPO / PPO）	保证新旧策略接近；今天没时间讲
reward learning（第 8 讲预告）	不知道奖励函数怎么定时的办法

术语对照

English	中文
state s_t / action a_t	状态 / 动作：环境此刻的情况 / 智能体此刻的选择
trajectory τ	轨迹：一整条状态、动作交替的序列，也叫 episode、rollout
reward $r(s_t, a_t)$ / return	奖励：每一步的一个分数 / 回报：沿轨迹把奖励加起来，本讲记作 $r(\tau)$
policy $\pi_\theta(a_t s_t)$	策略：给定状态输出动作的概率分布； θ 是网络权重
trajectory distribution $p_\theta(\tau)$	轨迹分布：初始状态分布、策略、环境动力学三者相乘
dynamics / transition $p(s_{t+1} s_t, a_t)$	动力学：环境从一个状态转到下一个状态的规律，通常未知
horizon T	步长：一条轨迹走多少步
RL objective $J(\theta)$	RL 目标：期望回报
Monte Carlo / sample estimate	样本估计：用有限次采样的平均代替期望
rollout	跑一遍策略得到一条轨迹
online / offline	在线：用当前策略采新数据 / 离线：只用现成数据集
on-policy / off-policy	同策略：只能用当前策略采的数据 / 异策略：可以复用旧策略的数据
policy gradient	策略梯度：目标 J 对策略参数 θ 的梯度
log-derivative trick	对数导数技巧： $\nabla p = p \nabla \log p$ ，把梯度搬进期望
REINFORCE / vanilla policy gradient	最基础的策略梯度算法
gradient ascent	梯度上升：沿梯度方向更新以最大化目标
automatic differentiation	自动微分：框架替你算 $\nabla \log \pi$
variance / noisy gradient	方差 / 噪声：梯度估计随采到的样本剧烈变化

English	中文
unbiased	无偏：估计量的期望等于真值
causality / reward-to-go	因果性 / 未来回报：动作 a_t 只用它之后的奖励加权
Markovian state	Markov 状态：当前状态包含预测未来所需的全部信息
baseline b	基线：从回报里减去的常数，常用平均回报
optimal baseline	最优基线：让方差最小的 b ，比平均回报多一些加权项
dense / sparse reward	稠密 / 稀疏奖励：每步都有信号 / 只在结尾或极少时刻有信号
batch	一批：一次采集的 N 条轨迹
surrogate objective	替代目标：梯度与策略梯度相同、可直接交给自动微分的目标
categorical / Gaussian policy	离散动作策略 / 高斯策略：连续动作，网络输出均值与方差
diffusion policy	扩散策略：用扩散模型表示的策略
importance sampling	重要性采样：从 q 采样、乘以 p/q 权重来估计 p 下的期望
proposal distribution q	提议分布：实际用来采样的分布，这里是旧策略
support	支撑：分布取值非零的区域； q 要覆盖 p
probability ratio $\pi_\theta / \pi_{\theta_{old}}$	概率比：新旧策略对同一动作的概率之比，PPO 里的 $r_t(\theta)$
KL constraint	KL 约束：限制新旧策略的距离，下一讲

字幕勘误

"toao""tow""to""town""directories" → τ (τ)、trajectories；"marovian" → Markovian；"heristics" → heuristics；"pietorch" → PyTorch；"important sampling" → importance sampling；"kale constraints" → KL constraints；"gaussian" → Gaussian；"arrowl""orall" → RL；"waiting terms" → weighting terms；"on principal" → in principle；baseline 证明里的 " v to the"" V " → b (baseline)；"key of data" → p_θ ；"recompute" → recompute；"polic" → policy；"reinforce" → REINFORCE；"ST50" → s_{50} 。

带走的问题

1. 今天的估计量最后写成 $\nabla \log \pi_{\theta} \cdot (\text{reward-to-go} - b)$ 。CME295 第 5 讲的 PPO 用 value function 当 b ，CS329A 第 6 讲的 GRPO 用组内平均再除以组内标准差。按今天的证明， b 依赖状态时还无偏吗（提示：把第 8 节证明里的 b 换成 $b(s_t)$ ，对 a_t 求期望）？除以标准差呢——它改的是方向还是步长？
2. 讲者说 reward-to-go 的严格证明"不好写"。试着用 baseline 的证明思路把它写出来：对 $t' < t$ ，为什么 $E[\nabla \log \pi_{\theta}(a_t | s_t) \cdot r(s_{\{t'\}}, a_{\{t'\}})] = 0$ ？哪一步用到了" a_t 是在看到 s_t 之后才采样的"？
3. Importance sampling 要求新旧策略接近，否则比值失控；把状态分布之比近似为 1 更是"明知不对"。PPO 的 clip 和 KL 惩罚分别在补哪一个漏洞？如果在同一批数据上走 50 步梯度而不是 5 步，哪个问题先爆？
4. 在 LLM 里，一条回答是一条轨迹，奖励只在结尾给一次，所以所有 token 的 reward-to-go 相同，失败回答里的每个 token 都被同等压低——对应今天叠夹克的困境。你会怎么给中间步骤（工具调用成功、子目标完成）设计更稠密的奖励，又不引入 CME295 里讲的 reward hacking？
5. 今天的三个课堂例子都在说"梯度方向取决于恰好采到了什么"。你手上的 agent 如果要用策略梯度改进，一批要采多少条轨迹才够？"这一批里一条成功都没有"时梯度为零——DAPO 的 dynamic sampling 是怎么绕开这个问题的，代价是什么？

CS224R 第 4 讲 | Actor-Critic 方法 (Actor-Critic Methods)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 4 讲, 2025 年 4 月 11 日 · 紧接第 3 讲的策略梯度; 开头预告的 off-policy 变体没讲到, 是第 5 讲的内容 视频: <https://www.youtube.com/watch?v=oejFZShW9hU> (1:03:30, 英文字幕是自动生成的, 符号和上下标常被念白, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: Policy Gradient Methods for Reinforcement Learning with Function Approximation (Sutton et al., 1999) · Proximal Policy Optimization Algorithms (Schulman et al., 2017)

一句话: 第 3 讲的策略梯度拿"这条轨迹后面实际拿到的奖励和"来决定一个动作该被鼓励还是打压——只有一个样本, 噪声大, 走对一步却摔倒的轨迹会被整体否定。本讲加一个专门评估"这个状态值多少"的价值网络 (critic), 用它算出的 advantage 替换那个单样本奖励和, 策略网络 (actor) 的梯度方差立刻小很多。critic 怎么训是重头戏: Monte Carlo 回归到实际看到的奖励和, 无偏但方差大; bootstrapping 用"这一步的奖励加上自己对下一状态的估值"当标签, 方差小但把估值误差带了进来; n-step return 在两者之间插值, 折扣因子 γ 让无限长回合的估值不至于爆掉。PPO 的 value head 就是这里的 critic, 本讲只讲到"怎么估、怎么用", clip 和 off-policy 都留给后面。

时间轴

时间	内容
0:05	回顾第 3 讲: 在线 RL 的循环、策略梯度与 baseline、on-policy 的代价、importance weight 的局限
3:09	今天讲什么: actor-critic、policy evaluation; PPO 建立在它之上
4:12	价值函数 $V^\pi(s)$: 从 s 出发按 π 走到底的期望奖励和
6:52	Q 函数 $Q^\pi(s, a)$; V 是 Q 的期望; advantage = $Q - V$; 问答: 归一化
12:01	练鼓的例子: 三个动作下自己算 V 、 Q 、 A ; 问答: Q 是学会的概率; 下棋的胜率
15:36	策略梯度的两个不满意场景: 迈步后摔倒、折衣服的稀疏奖励
18:10	把 reward-to-go 换成 Q , 再减 V 当 baseline, 得到 advantage 形式的梯度; 问答: 仍是同一目标、无偏
25:47	算法总览: 跑策略 $\rightarrow$ 拟合价值估计 $\rightarrow$ 改进策略

时间	内容
26:50	推导: A 只用 V 表示, $A \approx r + V(s') - V(s)$; 问答: 为什么是近似、要不要新交互
33:29	为什么估 V ; critic 是输出标量的网络; Monte Carlo: 重置做不到, 改用多条轨迹回归
41:11	Bootstrapping: 用 $r + V_\phi(s')$ 当标签; TD learning; 问答: 只输入 s 、初始化
48:24	两条轨迹的例子: MC 与 bootstrap 对蓝色、粉色状态的估值
52:00	偏差-方差; n -step returns: 在 $n = 1$ 到 $n = T$ 之间插值
56:08	折扣因子 γ : 等价于每步有 $1 - \gamma$ 的概率进入死亡状态; γ 与 n 的关系
1:00:12	完整的 actor-critic 算法: 五步; actor 与 critic 两个网络
1:02:17	回顾: 策略梯度 vs actor-critic; 三种估值方式统称 policy evaluation

核心内容

1. 回顾: 策略梯度为什么"不省数据"

- **在线 RL 的循环** (第 3 讲): 用当前策略在环境里跑一批数据 $\rightarrow$ 用这批数据改进策略 $\rightarrow$ 重复, 靠不断练习越来越好。策略 $\pi_\theta(a | s)$ 是"给定状态 s , 选各个动作 a 的概率", θ 是策略网络的参数; 从头跑到回合结束得到的状态、动作、奖励序列叫一条轨迹 τ 。
- **策略梯度的更新**: 从 π_θ 采 N 条轨迹, 对每一步算"这个动作的 log 概率对 θ 的梯度", 再乘上这一步之后实际拿到的奖励和 (reward-to-go)。后面奖励高, 就抬高这个动作的概率; 再减一个 baseline (比如平均奖励), 变成"高于平均的多做、低于平均的少做"。
- **两个毛病**
 1. **完全 on-policy**: 梯度里的期望是对 π_θ 自己采的样本算的, θ 一变, 手里的样本就不再来自当前策略, 所以每批数据只能走一步梯度, 要不停地重新采。第 3 讲讲 importance weight (新策略下动作的概率 $\div$ 旧策略下的概率) 能让旧数据复用, 但讲者特意提醒: 数学漂亮不等于万能——两个策略只要差得稍多, 这个比值就会极大或极小, 梯度估计跟着变差。
 2. **数据用得糙**: 走路例子里, 机器人向前迈了一小步、随后向后摔倒, 整条轨迹的 reward-to-go 为负, 策略梯度会连"向前迈步"一起打压, 因为它分不清哪一步是功、哪一步是过。折衣服例子里奖励稀疏, 只折了袖子、只铺平了衣服的轨迹奖励为零, 等于白

采。如果我们能判断"向前迈一步本身是有利的", 这些数据就都能用上。

- **本讲的目标**：actor-critic 方法，建在策略梯度之上。核心学习目标是 **policy evaluation**——估计一个状态、一个动作对当前策略来说值多少；然后用这个估计造出更好的 RL 算法。讲者说这类方法是 PPO 的基础，PPO 既用来训练语言模型，也用来训练仿真里的人形机器人。

小注：CME295 第 5 讲见过 PPO 的 clipped objective，里面那个 A_t 就是本讲要造的 advantage，PPO 的 value head 就是本讲的 critic。clip、KL 这些"别走远"的约束本讲完全没碰，开头预告的 off-policy 变体也没讲到，都在第 5 讲。

2. 三个价值量：V、Q、advantage

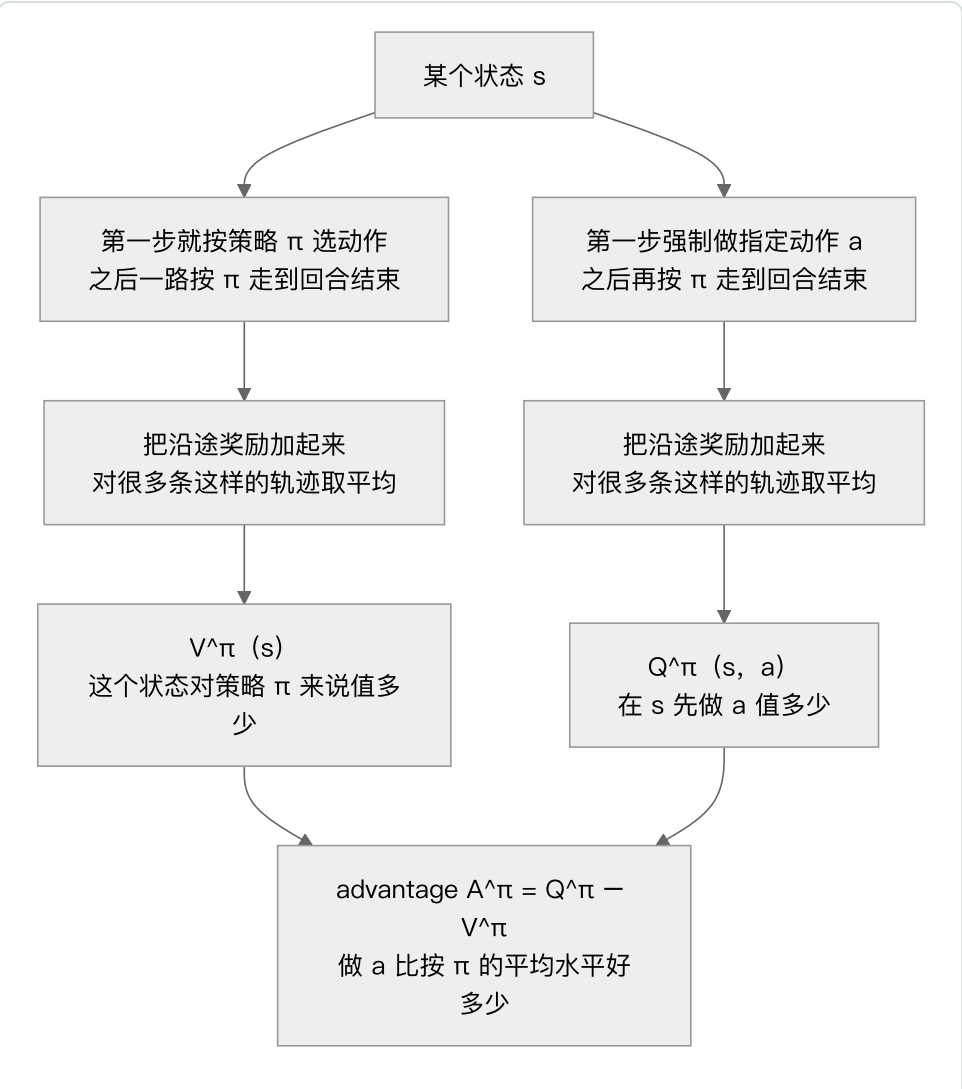


图 4-1 | V、Q、advantage 三个量分别在问什么：区别只在第一步是按策略选还是强制指定（自绘示意）· ▶ 看原幻灯片 7:53

- **价值函数 $V^\pi(s)$** (第 1 讲出现过)：从状态 s 出发、之后一直按策略 π 走，到回合结束能拿到的奖励总和的期望。要取期望，是因为两处有随机性：策略选动作是抽样的，环境给出的下一状态也可能是抽样的（环境是随机的，stochastic）。直观理解：从这个状态出发跑很多条轨迹，每条把从这一刻到结束的奖励加起来，再对所有轨迹取平均。
- **Q 函数 $Q^\pi(s, a)$** ：和 V 只差一处——第一步不按 π 选，而是强制执行指定的动作 a ，之后再按 π 走。这个 a 可能是 π 根本不太会选的动作；由于环境随机，同一个 a 也可能落到不同的下一状态。

$$V^\pi(s_t) = \mathbb{E}_\pi \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \mid s_t \right], \quad Q^\pi(s_t, a_t) = \mathbb{E}_\pi \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \mid s_t, a_t \right]$$

s_t 、 a_t 是时刻 t 的状态和动作， r 是每一步的奖励， T 是回合长度；两个期望都对"之后按 π 走会遇到的状态和动作"取，区别只是 Q 额外把第一个动作 a_t 固定住。

- **两者的关系**：把 Q 里那个"指定的 a "换成按 π 抽样，再对抽样取期望，就回到了 V —— V 是 Q 在策略自己的动作分布下的平均。
- **advantage $A^\pi(s, a)$** ： Q 减 V ，"在 s 做 a ，比按 π 的平均水平好多少"。这是本讲真正要塞进策略梯度里的量。

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)], \quad A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

左式说 V 是 Q 对策略动作分布的期望，右式定义 advantage； A 为正表示这个动作好于策略的平均表现，为负表示差于平均。

- **练鼓的例子** (讲者让全班算)：奖励是"一个月后能演奏得 1 分，否则 0 分"， T 就是一个月；动作有三个：躺沙滩、看电视、练鼓；当前策略是在所有状态下都躺沙滩，而且现在还不会打鼓。那么 $V^\pi(\text{当前状态}) = 0$ ，因为照这个策略永远学不会； $Q(\text{躺沙滩})$ 和 $Q(\text{看电视})$ 都是 0； $Q(\text{练鼓})$ 是"练这一步之后又躺回沙滩"仍能学会的概率，可能接近 1，也可能远小于 1——取决于一个时间步有多长、这个人有没有天赋；advantage 只有练鼓是正的，另外两个是 0。
- **问答**
 - V 和 Q 之间有归一化吗？——期望就是平均，用样本估的时候各自除以各自的样本数，天然就是归一的。
 - 讲者补了个更常见的直觉：下棋时，价值函数就是"当前局面下这个棋手的胜率"——它评的是局面对某个特定的人有多好。

3. 把单样本的 reward-to-go 换成 advantage

- 策略梯度里真正决定"抬还是压"的，是每一步后面的奖励和。它其实在估"在 s 做 a 之后还能拿多少"——这正是 $Q^\pi(s, a)$ 的定义，只不过用了一条轨迹里的一个样本。如果换成真正的 Q^π （相当于从那个状态、那个动作出发把策略跑无穷多次再平均），梯度会准得多：Q 高的动作抬概率，Q 低的压概率。
- 再加 baseline：第 3 讲的 baseline 是平均奖励，这里对应的自然是"Q 的平均"；而 Q 在策略动作分布下的平均恰恰是 V^π 。于是权重变成 Q 减 V，也就是 advantage：抬高 advantage 为正的動作，压低为负的。回到练鼓的例子：练鼓的 advantage 是正的，梯度会增加练鼓的概率、减少另外两个。

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) A^{\pi}(s_{i,t}, a_{i,t})$$

$J(\theta)$ 是策略的期望奖励和，i 编号 N 条采样轨迹，t 编号轨迹里的时间步；和第 3 讲的 REINFORCE 相比，只是把每一步的权重从"这条轨迹后面的奖励和"换成了 advantage。

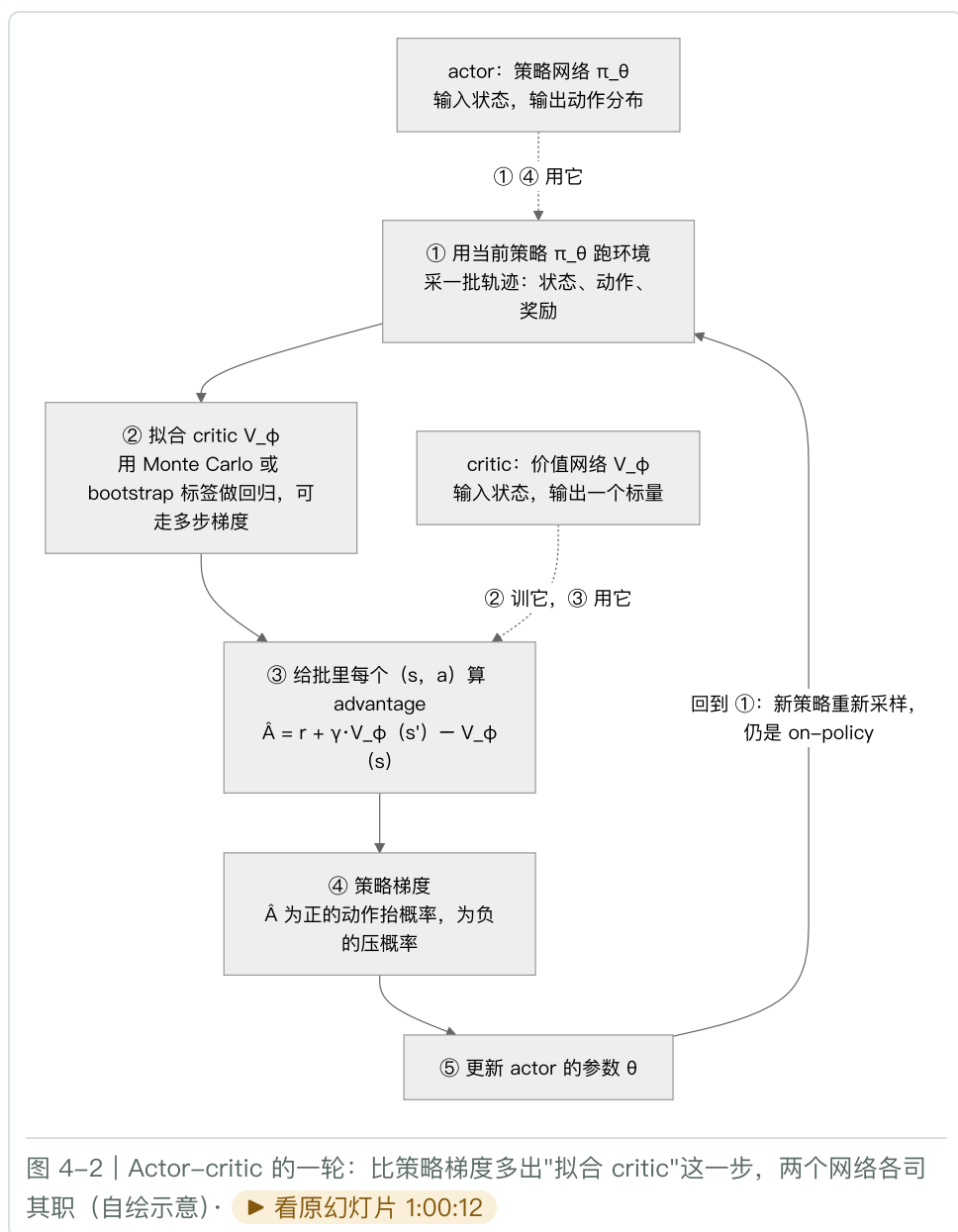
- 为什么更好：advantage 估得越准，梯度越不噪；如果知道真实的 A^π ，这就是一个非常好的梯度。讲者确认：它优化的仍是第 3 讲那个"期望奖励和"目标，只是估计方式变了，用真实 advantage 时这个梯度估计是无偏的（推导略过，有偏的东西后面才出现）。真正的难题是我们不知道真实的 A^π ，接下来几十分钟都在讲怎么估它。

小注：把 reward-to-go 换成 Q^π 这一步的合法性，正是指定阅读 Sutton et al., 1999 的 policy gradient theorem；那篇还证明了 critic 用和策略"兼容" (compatible) 的线性特征时，不会给梯度引入偏差。

• 问答

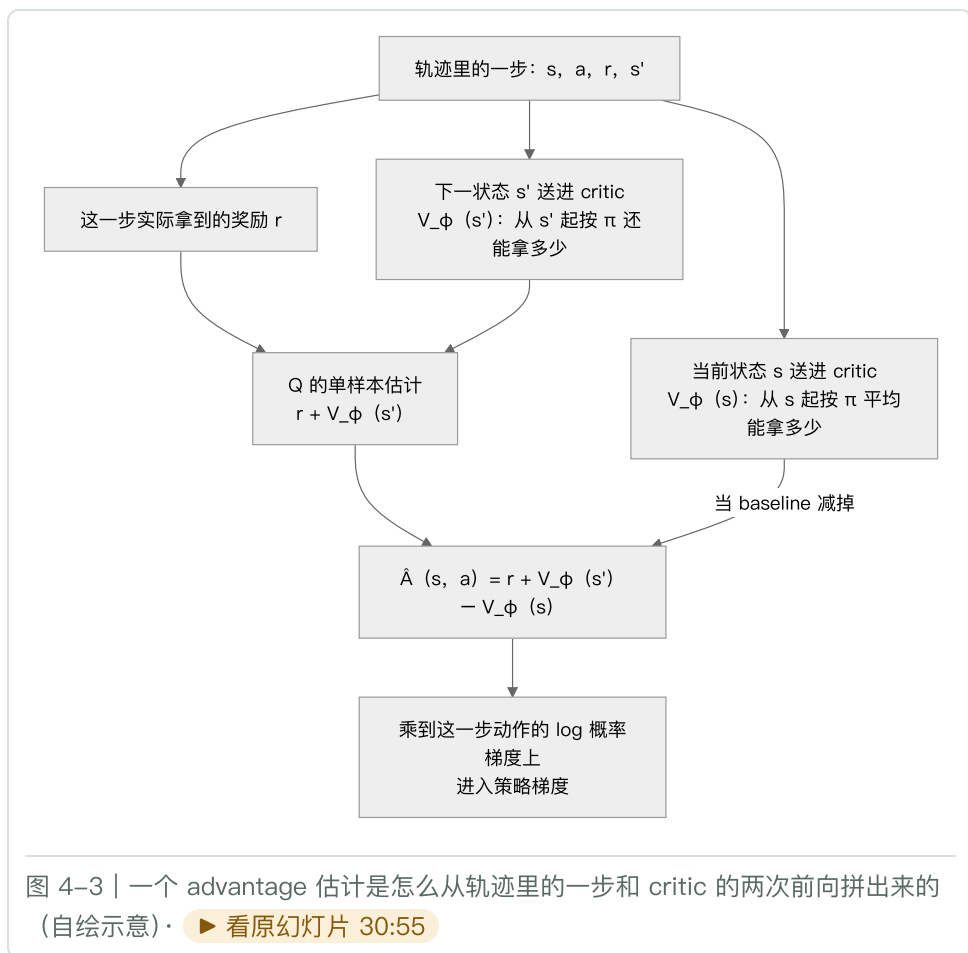
- 这和原来的采样有什么不同？——原来对每个 (s, a) 只有它所在那一条轨迹的一个奖励和，是极噪的单点估计；现在要的是"这个动作的优势"，概念上等于从这个状态、动作出发把策略滚无穷次。
- baseline 是对当前这条轨迹平均，还是对所有轨迹？——设成 V^π 的话，概念上是无穷多条轨迹的平均；怎么用有限数据估它，马上讲。

4. Actor-critic 的循环：多出来的一步"拟合"



- 原来的循环只有两步：跑策略、改策略。现在中间插一步：**拟合一个模型去估计期望回报**——估 V^{π} 、 Q^{π} 或 A^{π} 都行；改策略那一步照旧，只是用这个估计代替单样本的奖励和。
- **两个网络**：策略网络是 **actor**（执行者），价值网络是 **critic**（评论者，它在“评价”一个状态、一个策略有多好）。名字就是这么来的。
- **估哪一个**：直接拟合 A 拿不到现成的监督信号； Q 和 V 都估要维护两个模型。讲者的选择是只拟合 V ，再从 V 推出 A ——下一节就是这个推导。问答里她补充：这只是实践上简单一点，直接估 Q 的办法下周会讲（第 5、6 讲）。

5. 只拟合 V 就够了：advantage 的一步近似



- **推导** (板书): $Q^{\pi}(s_t, a_t)$ 是"当前这一步的奖励, 加上之后所有奖励"。之后的奖励取决于下一状态 s_{t+1} ——它由环境的动态 (dynamics) 抽样决定; 而"从 s_{t+1} 出发按 π 走到底的奖励和"按定义就是 $V^{\pi}(s_{t+1})$ 。于是 Q 等于当前奖励, 加上"下一状态的价值"对环境动态的期望。
- **近似**: 我们不能对环境动态随便抽样, 但采数据时确实看到了一个 s_{t+1} ——就用这一个样本代替期望。这样 Q 和 A 都只用 V 就能写出来, critic 只需要拟合 V :

$$Q^{\pi}(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim p(\cdot | s_t, a_t)} [V^{\pi}(s_{t+1})] \approx r(s_t, a_t) + V^{\pi}(s_{t+1}), \quad \hat{A}(s_t, a_t) = r(s_t, a_t) + V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

$p(\cdot | s_t, a_t)$ 是环境的转移分布; $\approx$ 处用轨迹里实际出现的那一个 s_{t+1} 代替对 p 的期望; V_{ϕ} 是参数为 ϕ 的 critic 网络对 V^{π} 的估计, $\hat{A}$ 是最终塞进策略梯度的 advantage 估计。

- **问答**
 - 为什么是近似? ——下一状态可能有多种, 我们只用了轨迹里看到的那一个。环境越确定 (deterministic), 这个近似越好; 越随机越差。好消息是后面会看到怎么用大量数据、多步梯度把它抹平。

- 需要和环境新的交互吗？——不需要。 s_t 、 a_t 、 s_{t+1} 都取自同一条已经采好的轨迹。
- 那个上标 π 很多论文为了省事会省掉，但讲者强调它很重要：这里估的是**某个特定策略**的价值，不是任意策略的。

小注： $r + V(s_{t+1}) - V(s_t)$ 在文献里叫 TD error，记作 δ_t 。CME295 第 5 讲提过的 GAE (Schulman et al., 2015) 就是把第 9 节里不同 n 的 advantage 估计按 λ 做指数加权平均， (γ, λ) 两个超参由此而来。

6. 拟合 critic 之一：Monte Carlo 回归

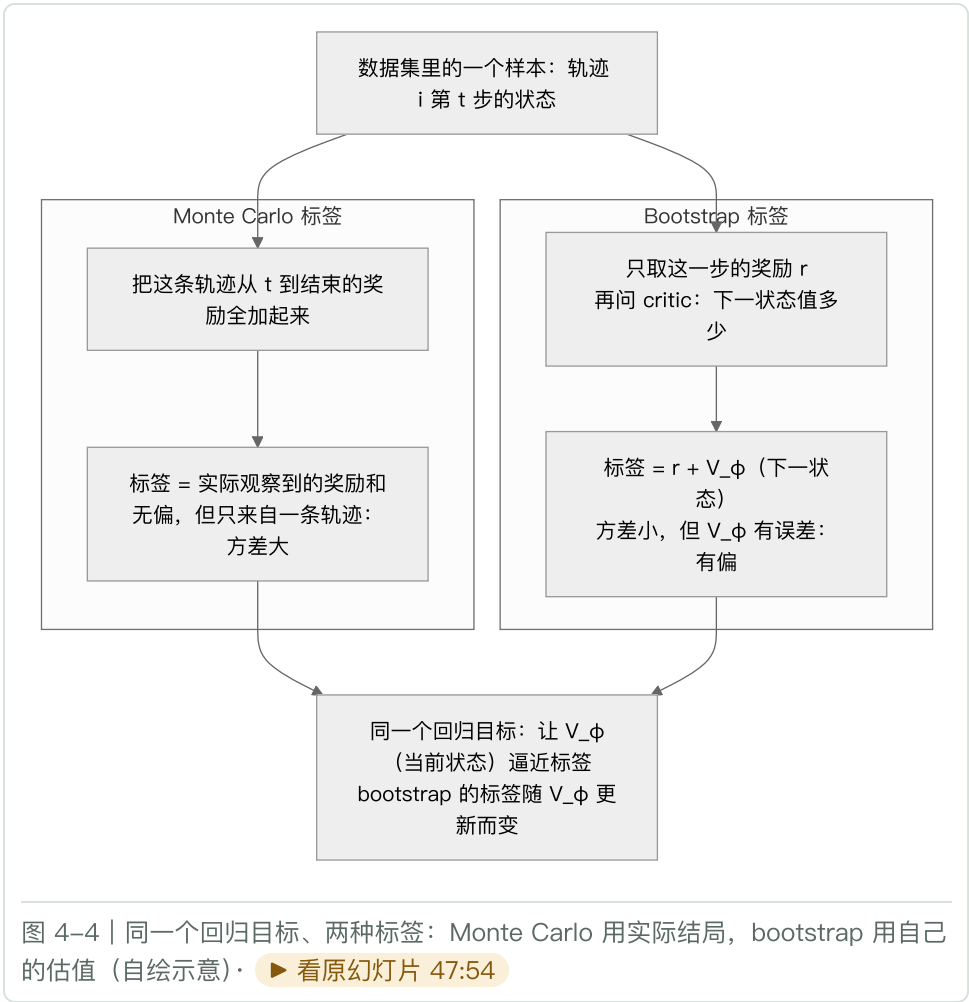
- **critic 长什么样**：一个神经网络 V_ϕ ，输入一个状态，输出一个标量（讲者特意说：不像幻灯片上画的那样是多维的），参数记作 ϕ 。
- **理想做法做不到**：想知道某个状态的价值，最直接的是把世界重置（reset）到那个状态、从那里跑很多次取平均。现实世界没法重置；仿真器有时可以，但实践中一般也不这么干。
- **能做的**：手里有很多条轨迹，每条轨迹的每个时刻 t 都对应一个“从 t 到结束的实际奖励和”。把所有（状态，之后的奖励和）配成对，就是一个监督学习数据集：输入状态、标签是奖励和，用 L2 回归训练 V_ϕ 。数据点数 = 轨迹数 $\times$ 平均步数。不同轨迹里会出现相似的状态，网络在它们之间共享信息（讲者的词是 amortize），等于变相对相似状态取了平均。

$$\min_{\phi} \sum_{i=1}^N \sum_{t=1}^T \left(V_{\phi}(s_{i,t}) - y_{i,t} \right)^2, \quad y_{i,t}^{\text{MC}} = \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'})$$

$s_{i,t}$ 是第 i 条轨迹第 t 步的状态， $y_{i,t}$ 是它的回归标签；Monte Carlo 的标签就是这条轨迹从 t 起实际看到的奖励和，后面几节只换标签、不换这个平方误差目标。

- 这叫 **Monte Carlo 估计**：跑很多次，直接回归到看到的结果。最简单，讲者自嘲也最傻。
- **问答**
 - 每次策略一变就得重采、重训吗？——用这个方法是的， V^{π} 是绑定策略的；后面会有能复用旧数据的办法。
 - critic 每走一步梯度都要重新采数据吗？——不用。和策略梯度不同，拟合 critic 不改变策略，同一批数据上可以走很多步梯度。
 - 能不能顺便学“给定 s 、 a 预测下一状态”？——可以，那叫 model-based 方法（学一个世界模型），过几周讲（第 11 讲）。

7. 拟合 critic 之二：bootstrapping / TD learning



- **想法：**训到一半时， V_ϕ 对某些状态的估值已经像样了。那么对时刻 t 的状态，与其用整条轨迹后面的奖励和当标签，不如用"这一步的奖励加上 V_ϕ 对下一状态的估值"——理想标签本来就是"当前奖励加下一状态的期望价值"，只是把期望价值换成了自己的估计。

$$y_{i,t}^{\text{TD}} = r(s_{i,t}, a_{i,t}) + V_\phi(s_{i,t+1})$$

回归目标和上一节完全一样，只是标签换成了"这一步的奖励加 critic 对下一状态的估值"；标签里有 V_ϕ 自己，所以它会随训练不断变化。

- **bootstrapping** 这个词的意思是"用自己的估计当监督"（拽着自己的鞋带把自己提起来）。 V_ϕ 一更新，标签跟着变；实践中通常每走一步梯度就刷新一次标签，网络特别大、更新特别贵时可以刷得慢一点。
- **它换来了什么：**MC 标签只看这一条轨迹碰巧发生的一个结局；bootstrap 标签里的 $V_\phi(s_{t+1})$ 是用**所有**轨迹训出来的，隐含了"从 s_{t+1} 出发可能发生的各种结局"。这正是它对第 5 节那个"单样本近似"的补偿。

- 另一个名字是 **temporal difference (TD) learning**：看的是相邻两个时间步之间的差。讲者提醒别和反向传播混了：训练时仍然是对那个 L2 目标做 backprop；"信息沿时间往回传"（下一状态的估值传给当前状态）是另一回事。

- **问答**

- 为什么 critic 只输入当前状态，不输入历史？——未来的奖励只取决于现在在哪（Markov 假设），当前奖励又是确定的，所以标签只需条件在 s_t 上。
- 初始估值很差时标签不就全错了？——一般不用特别处理，只要别把网络初始化到输出 -1000 这种离谱值；真担心的话可以先用几轮 Monte Carlo 把量级拉对，再切到 bootstrap。

小注：实现上 bootstrap 标签要当常数处理（stop-gradient，不让梯度流过 $V_\phi(s_{t+1})$ ），这叫 semi-gradient TD；用一个更新慢一拍的旧网络算标签的 target network 技巧，应在第 6 讲 Q-learning 里正式出现。TD learning 这个名字出自 Sutton 1988。

8. 两条轨迹的例子：bootstrap 为什么能跨轨迹传信息

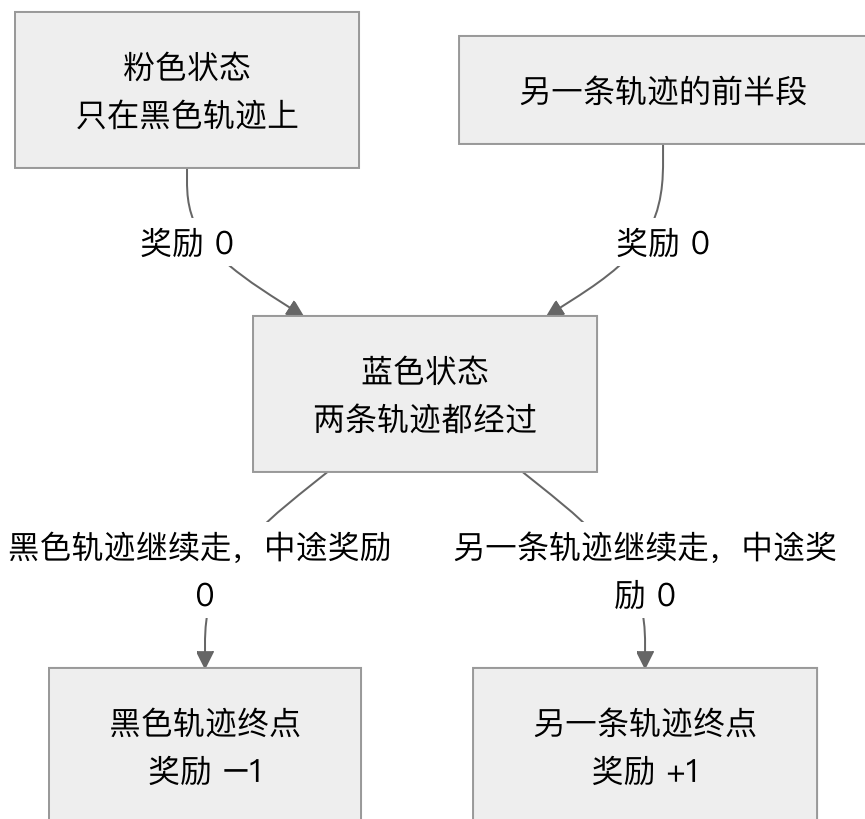


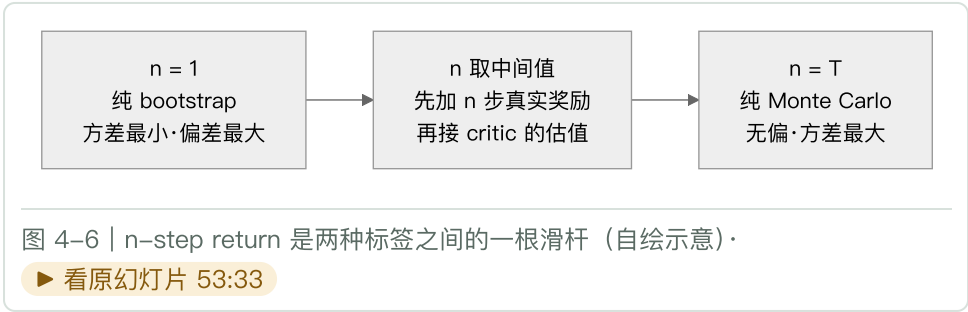
图 4-5 | 课上的练习：两条轨迹共享一个蓝色状态，粉色状态只在通往 -1 的那条上（自绘示意）· [▶ 看原幻灯片 48:24](#)

- 只采了两条轨迹：黑色那条最后奖励 -1，另一条最后奖励 +1，中途奖励全是 0；蓝色状态两条轨迹都经过，粉色状态只在黑色轨迹上、位于蓝色之前。问：两种方法各给这两个状态什么估值？

状态	Monte Carlo	Bootstrap
蓝色（两条轨迹共有）	0：数据集里同一个状态带着 -1 和 +1 两个标签，回归取平均	0：标签是 0 加下一状态的估值，两条轨迹的下一状态分别估成 -1 和 +1，平均仍是 0
粉色（只在黑色轨迹上）	-1：只看得到自己这条轨迹的结局	0：标签是 0 加 $V_\phi(\text{蓝}) = 0$ ，蓝色状态的估值把另一条轨迹的信息带了过来

- 蓝色之后，黑色轨迹上的状态估值都是 -1，另一条上都是 +1，两种方法一致；差别出在蓝色之前。讲者的评价：bootstrap 的估值可以说更准，因为它能通过自己的估值把不同轨迹的信息聚合起来；MC 只有在数据集里出现一模一样的状态时才会做这种聚合。

9. 偏差-方差：n-step returns



- 两种标签的取舍：Monte Carlo 完全无偏，但方差高；bootstrap 方差低得多，但一定有偏——当前的 V_ϕ 总有误差，误差会被当成标签的一部分学进去。
- 中间路线：**先累加接下来 n 步实际看到的奖励——趁方差还没涨起来——再在第 t+n 步接上 critic 的估值。n = 1 就是纯 bootstrap，n = T 就是纯 Monte Carlo，实践中通常取中间某个值最好。

$$y_{i,t}^{(n)} = \sum_{t'=t}^{t+n-1} r(s_{i,t'}, a_{i,t'}) + V_\phi(s_{i,t+n})$$

前 n 项是轨迹里真实观察到的奖励，最后一项是 critic 对第 t+n 步状态的估值；n 越小越像 bootstrap，n 越大越像 Monte Carlo。

- 什么时候该大 n：**时间步很短的场景（比如 10 ms 一步），相邻两步之间几乎没发生什么，一步 bootstrap 学不到东西；方差随 n 涨得慢就

多看几步，涨得快就少看。 n 不改变"贪不贪心"——各种 n 估的都是同一个 V^π ，变的只是估计的偏差和方差。

- **问答：**能不能把 MC 标签和 n -step 标签都放进数据集？——原则上可以，实践中一般只挑一种：高方差的标签会拖慢学习，估值不准的标签也是。

10. 折扣因子 γ ：给无限长的回合"设个寿命"

- **问题：**前面一直在把未来奖励全加起来。回合很长甚至无限长时（第 1 讲提过），奖励和会大到没边，估值就没法学。
- **做法：**在标签里给下一状态的估值乘一个系数 γ (0.9、0.99 之类)，意思是"更在乎眼前的奖励，越远的越不在乎"； $\gamma = 1$ 就回到原来。

$$y_{i,t} = r(s_{i,t}, a_{i,t}) + \gamma V_\phi(s_{i,t+1}), \quad y_{i,t}^{(n)} = \sum_{t'=t}^{t+n-1} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'}) + \gamma^n V_\phi(s_{i,t+n})$$

γ 在 0 和 1 之间；一步标签里下一状态的估值打一次折， n 步标签里第 k 步之后的奖励打 k 次折、末尾的估值打 n 次折（课上只点到了估值项上的 γ^n ，奖励项上的折扣是标准写法）。

- **它到底改了什么：**加折扣等价于稍微改了决策过程本身（MDP，即"状态、动作、转移概率、奖励"这一整套定义）：每一步都有 $1 - \gamma$ 的概率进入一个"死亡状态"——奖励为零、再也出不来；原 MDP 里所有转移概率都乘上 γ 。 $\gamma = 0.99$ 就是"每步有 1% 的概率当场结束"。
- **γ 和 n 的关系（问答）：**两者互相牵连——环境越确定、时间步越细，越会往前多看几步，这同样会影响 γ 的选择；但 n 管的是"往前看几步再交给估值"， γ 管的是"远处的奖励算几分"，不是一回事。

小注： γ 把有效视野压到大约 $1/(1 - \gamma)$ 步， $\gamma = 0.99$ 大约看 100 步。

LLM 的 RLHF 通常直接取 $\gamma = 1$ ——一条回答只有几十到几千个 token、奖励在末尾，不折扣才不会把长回答末尾的奖励打薄。

11. 完整算法，以及它通向哪里

- **五步（图 4-2）：**① 用当前策略采一批轨迹；② 用 n -step 标签在这批数据上走多步梯度，拟合 V_ϕ ；③ 对每个 (s_t, a_t) 算 $\hat{A} = r + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ ，它回答"按 critic 和实际奖励看，数据里这个动作有多好"；④ 用 $\hat{A}$ 做策略梯度，抬高有利动作、压低不利动作；⑤ 更新 θ ，回到 ①。
- **回到开头的例子：**向前迈步然后摔倒的轨迹现在能用上了——尤其是用 bootstrap 估值时，critic 会学到"迈步之后的状态"比"摔倒之后的

状态"值钱，迈步的 advantage 为正、摔倒为负，各得其所。数据利用率高得多。

- **一句话对比：**策略梯度是"看到什么好就多做、看到什么坏就少做"；actor-critic 是"先学会判断什么好什么坏，再按判断多做少做"。三种估值方式——直接看奖励和、bootstrap、两者混合——统称 policy evaluation。
- **接下来：**这个算法仍然是 on-policy 的—— V_ϕ 估的是当前策略的价值，每轮都要重新采样。第 5 讲会把它改成 off-policy，让旧策略的数据（攒在 replay buffer——一个存历史转移样本的缓冲区——里）也能用；第 6 讲直接学 Q，不经过 V。

小注：CS329A 第 6 讲的 GRPO 走的是另一条路——干脆不要 critic，对同一个 prompt 采一组回答，用组内平均奖励当 baseline。按本讲的语言，那是 Monte Carlo 估计配一个只依赖状态（prompt）的 baseline：省掉了 critic 的偏差和显存，代价是回到高方差，靠一组多条采样来压。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
V 与 Q 的两张示意图	同一个 s 出发：直接按 π 跑 vs 先做 a 再按 π 跑，各自对奖励和取平均	7:53	—
V、Q、A 的关系式	V 是 Q 在策略动作分布下的期望； $A = Q - V$	8:53	—
练鼓的例子	三个动作、一个"永远躺沙滩"的策略，自己算 V、Q、A	12:01	—
走路摔倒与折衣服	策略梯度浪费数据的两个场景：功过不分、稀疏奖励	16:07	—
用 V 当 baseline 得到 advantage 的梯度式	reward-to-go $\rightarrow Q \rightarrow Q - V$ 的三次替换	21:43	Sutton et al., 1999（课上没点名）
三步算法框图	跑策略 $\rightarrow$ 拟合价值估计 $\rightarrow$ 改进策略，中间那步是新增的	26:18	—
板书： $A \approx r + V(s') - V(s)$	Q 拆成当前奖励加下一状态价值的期望，再用单样本近似	30:55	—
Monte Carlo 数据集	(状态, 之后的奖励和) 配对成监督学习数据，数据量 = 轨迹数 $\times$ 步数	37:04	—
bootstrap 标签	理想标签 = 当前奖励加下一状态的期望价值，把期望价值换成 V_ϕ 自己	43:13	—

图	看什么	跳转	出处
两条轨迹的例子	蓝色、粉色两个状态在 MC 与 bootstrap 下的估值差异	48:24	—
n-step return 公式	前 n 步用真实奖励、之后接 critic; $n = 1$ 与 $n = T$ 两个极端	53:33	—
折扣即"死亡状态"	每步以 $1 - \gamma$ 的概率进入零奖励的吸收态; 其余转移乘 γ	58:11	—
完整 actor-critic 算法	五步循环; 两个网络分别叫 actor 和 critic	1:00:12	—

提到的工作

名称	在本讲里的作用
策略梯度 / REINFORCE 与 baseline (第 3 讲)	出发点: 本讲把它的单样本 reward-to-go 换成 critic 给的 advantage
importance weights (第 3 讲)	回顾里的提醒: 只在新旧策略很接近时可靠, 不是万能药
价值函数 V、Q (第 1 讲引入)	本讲的主角, policy evaluation 的对象
PPO (Schulman et al., 2017; 指定阅读)	被点名为建在 actor-critic 之上的算法, 用于训练 LLM 和仿真人形机器人; clip 部分本讲未讲
Policy gradient theorem (Sutton et al., 1999; 指定阅读)	课上未点名; 把 reward-to-go 换成 Q^π 的理论依据
Monte Carlo estimation	拟合 critic 的第一种方法: 回归到实际奖励和
Bootstrapping / temporal difference learning	第二种方法: 用自己的估值当标签
n-step returns	两者之间的插值
折扣因子与 MDP 的"死亡状态"改写	处理长回合下估值爆炸
Model-based RL (第 11 讲预告)	问答: 学一个预测下一状态的模型是另一条路
Off-policy actor-critic (第 5 讲预告)	本讲开头预告、没讲到的部分
直接估 Q 的方法 (第 5、6 讲预告)	问答: 为什么这里选择估 V
国际象棋的胜率	价值函数的直觉例子

术语对照

English	中文
state / action / policy π_θ	状态 / 动作 / 策略：给定状态下各动作的概率， θ 是策略网络参数
trajectory τ / episode / rollout	轨迹 / 回合 / 一次滚动：从头跑到结束的状态、动作、奖励序列
return / sum of rewards	回报：一条轨迹上奖励的总和
reward-to-go	未来奖励和：从某一步起到结束的奖励之和
baseline	基线：从权重里减掉的"平均水平"，用来降方差
on-policy / off-policy	只能用当前策略自己采的数据 / 能复用别的策略采的数据
importance weight	重要性权重：新策略下的动作概率 $\div$ 旧策略下的动作概率
value function $V^\pi(s)$	价值函数：从 s 出发按 π 走到底的期望回报
Q function $Q^\pi(s, a)$	动作价值函数：在 s 先做 a 、之后按 π 走的期望回报
advantage $A^\pi(s, a)$	优势： Q 减 V ，做 a 比按 π 平均好多少
policy evaluation	策略评估：估计一个给定策略的 V 、 Q 或 A
actor / critic	执行者（策略网络） / 评论者（价值网络）
Monte Carlo estimation	蒙特卡洛估计：跑很多次，回归到实际看到的回报
bootstrapping	自举：用自己当前的估值当训练标签
temporal difference (TD) learning	时序差分学习：bootstrapping 的另一个名字，看相邻时间步的差
n-step return	n 步回报：前 n 步用真实奖励，之后接估值
bias / variance	偏差 / 方差：估计的系统性偏离 / 估计的随机波动
unbiased estimate	无偏估计：平均下来正好等于真值
discount factor γ	折扣因子：未来奖励每往后一步乘一次 γ
MDP (Markov decision process)	马尔可夫决策过程：状态、动作、转移概率、奖励的整套定义
absorbing / death state	吸收态：进去就出不来、奖励为零的状态，折扣的等价解释
dynamics / transition	环境动态：给定 s 、 a 时下一状态的分布
deterministic / stochastic	确定性 / 随机性（说环境或策略）
horizon T	时间范围：回合的最大步数
reset	重置：把环境放回某个状态，现实中通常做不到
sparse reward	稀疏奖励：只有完成任务才有奖励
supervised regression / L2 loss	监督回归 / 平方误差
model-based RL	基于模型的 RL：学一个预测下一状态的世界模型

English	中文
replay buffer	经验回放缓冲区：攒历史数据供 off-policy 复用（第 5 讲）

字幕勘误

自动字幕把符号念白了的地方多：开头 "the pi of S" 应为 $V^{\pi}(s)$ ；"used to write ingredients" 是无意义的误识别；"S S I and A I" 应为 s_i 、 a_i ；"n equals t" 里的 t 应为大写的 T（回合长度）；"gamma to the end" 应为 γ^n (gamma to the n)；"block trajectory" 应为 black trajectory；"the hat of zero" 应为估值 $\hat{V} = 0$ 。人名和算法名本讲几乎没有出现，没有会误导的错误。

带走的问题

1. 用真实的 A^{π} 时梯度无偏；换成 $r + V_{\phi}(s') - V_{\phi}(s)$ 后偏差从两处来：单样本的 s' 和 V_{ϕ} 自己的误差。哪一处能靠多采数据抹平，哪一处不能？n-step 分别对这两处做了什么？
2. Monte Carlo 的 critic 和策略绑死，策略一变就得重采。bootstrap 标签里的 $V_{\phi}(s_{t+1})$ 估的是"当前策略"的价值——如果这批数据是旧策略采的，它估的到底是谁的价值？第 5 讲的 off-policy actor-critic 要回答的就是这个。
3. 在 LLM 的 RLHF 里每个 token 是一步、奖励只在末尾：Monte Carlo 标签和 bootstrap 标签分别长什么样？PPO 的 value head 学的是哪一个？GRPO 用组内平均当 baseline，相当于回到本讲的哪种估计，为什么在"一个 prompt、一条回答"的结构里可以接受？
4. 按"死亡状态"的解释，你的 agent 如果平均要 30 步工具调用才能完成任务， $\gamma = 0.9$ 和 $\gamma = 0.99$ 各意味着什么"寿命"？会不会让它对最后那一步的任务完成奖励视而不见？
5. 两条轨迹的例子 bootstrap 更准，是因为蓝色状态在两条轨迹里**完全相同**。真实任务里状态几乎不会重复，critic 靠什么把信息传过去？这对 critic 的网络结构和输入表示提出了什么要求？

CS224R 第 5 讲 | Off-Policy Actor-Critic (Off-Policy Actor Critic)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 5 讲, 2025 年 4 月 16 日 · 承接第 4 讲的 actor-critic, 引出第 6 讲的 Q-learning 视频: <https://www.youtube.com/watch?v=cRGKc-nAWho> (1:09:21, 英文字幕是自动生成的, PPO、SAC、KL 这些缩写几乎全错, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座讲座) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: Playing Atari with Deep Reinforcement Learning (Mnih et al., 2013, 即 DQN; 课上没有展开, 它里面的 replay buffer 和 bootstrap 目标是本讲后半段思路的 Q-learning 版本, 第 6 讲细讲)

一句话: 第 4 讲的 actor-critic 是 on-policy 的——每采一批数据只能走一步梯度就得扔掉, 而和环境交互是 RL 里最贵的事, 这样用太浪费。本讲沿一条线把它改成 off-policy: 先用 importance weight 让一批数据走多步, 发现 advantage 会过期、步数一多就崩, 于是用 KL penalty 或者把 importance weight 截到 $1 \pm \epsilon$ 来拴住新策略, 这就是 PPO; 再把过去所有的数据存进 replay buffer, 为此把拟合 V 换成拟合 Q——Q 的递归式对任何 (s, a) 都成立, 动作是谁选的无所谓——并把目标值和策略梯度里的动作都改成从当前策略现采, 这就是 SAC 的骨架。代价也说清了: PPO 稳但费数据, SAC 省数据但难调。

时间轴	
时间	内容
0:06	回顾: 策略梯度与 importance weight; V、Q、advantage 的定义; AlphaGo 例子
4:41	回顾第 4 讲 actor-critic: 用 advantage 替换回报; policy evaluation 的三种拟合目标
6:11	今天的目标: 从 on-policy actor-critic 出发, 做出 PPO 和 SAC 两个真实算法
7:12	一批数据走多步: importance weight 版梯度; 为什么要写成 surrogate objective
13:22	走多了会崩: advantage 过期; 三条学习曲线; 问答 (过拟合的策略长什么样、为什么要 importance weight)
21:13	办法一: KL penalty 把新策略拴在旧策略旁边; β 怎么调; 要多存一份旧参数

时间	内容
28:21	办法二：把 importance weight 截到 $1 \pm \epsilon$ ——PPO 的核心
32:26	PPO 的 trick 2：与原目标取 min 保证是下界； ± 1000 的板书例子
37:32	trick 3：GAE，把不同步长的 advantage 加权混合；问答
42:11	完整 PPO 算法与典型超参数；half-cheetah 曲线；问答（循环怎么接、超参数怎么调）
46:48	更 off-policy：replay buffer 存下全部历史；要拆掉哪些 on-policy 假设
49:52	天真版本坏在哪：在 buffer 上拟合的 V 属于哪个策略（课堂讨论）
53:26	改拟合 Q ：板书推导 Q 的递归式，它对任意 (s, a) 都成立
57:31	目标 $y = r + \gamma Q(s', a')$ ， a' 从当前策略现采；覆盖度问答
1:05:17	策略更新：用 Q 不用 A 、动作也从当前策略现采；完整算法就是 SAC 的骨架
1:07:19	SAC 与 PPO 的数据效率；真机机器人两小时学会走；PPO 在仿真和 LLM 里的位置

核心内容

1. 回顾：V、Q、advantage 三个"有多好"

- **先把词摆齐**（前几讲建立的词汇，本讲开头又过了一遍）：agent 在时刻 t 看到状态 s_t ，按策略 $\pi_\theta(a_t | s_t)$ 选动作 a_t ——策略就是"给定状态，各个动作的概率"， θ 是策略网络的参数；环境给一个奖励 $r(s_t, a_t)$ ，并把它送到下一个状态 s_{t+1} 。一整段交互 $(s_0, a_0, s_1, a_1, \dots)$ 叫轨迹 τ ；从某时刻起把之后的奖励加起来叫回报（return）。折扣因子 γ （0 到 1 之间的数）给越远的奖励打越重的折，本讲板书为了简洁把它省了，幻灯片上的目标值里有。
- **值函数**： $V^\pi(s)$ 是从 s 出发、之后一直按 π 行动，平均能拿到的未来奖励总和。上标 π 很要紧：同一个状态换个策略，值就变。
- **Q 函数**： $Q^\pi(s, a)$ 是从 s 出发，先固定做动作 a ，之后再按 π 行动的期望回报。它比 V 多做一件事：把第一步钉死。
- **advantage**： $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ ，在 s 做 a 比"按 π 的平均水平"好多少。正表示比平均好，负表示比平均差。
- **AlphaGo 的例子帮助建立直觉**：奖励是终局的胜负； V (棋局) 就是在这个局面下按当前策略下完，预期的结果。策略更强，值更高；局面更好，值也更高—— V 同时取决于状态和策略。 Q 则是"先走这一手，再按策略下完"的预期结果。

$$V^\pi(s_t) = \mathbb{E}_\pi \left[\sum_{t' \geq t} r(s_{t'}, a_{t'}) \mid s_t \right], \quad Q^\pi(s_t, a_t) = \mathbb{E}_\pi \left[\sum_{t' \geq t} r(s_{t'}, a_{t'}) \mid s_t, a_t \right], \quad A^\pi = Q^\pi - V^\pi$$

$V^\pi(s_t)$ 是从 s_t 起按 π 行动的期望回报， $Q^\pi(s_t, a_t)$ 多了"第一步固定做 a_t "这个条件，两者之差就是 advantage；期望里的随机性来自环境的状态转移和策略的采样。

- 第 3 讲的策略梯度：**用当前策略采一批轨迹，把之后奖励高的动作的概率往上推；减掉一个 baseline（平均奖励）之后，就变成"高于平均的多做、低于平均的少做"，方差也小了。它有一条硬约束：数据必须来自你正要更新的那个策略。第 3 讲末尾已经提过，用 importance weight 可以放松这条约束，让一批数据走多步——本讲前半段就从这里展开。
- 第 4 讲的 actor-critic：**梯度公式的形状和策略梯度一样，只是把"之后的奖励总和"换成了 advantage 的估计——先估计哪些动作好、哪些坏，再让策略多做好的。估计的过程叫 policy evaluation：训一个输入 s 、输出一个数的网络 $V(s)$ ，训练目标有三种：整条轨迹的奖励和（Monte Carlo）、一步奖励加下一状态的 $V^\wedge$ (bootstrapping，拿自己的估计当目标)、或者几步奖励再加 $V^\wedge$ 。advantage 则近似为 $r + V(s') - V(s)$ 。

2. 今天的路线：一批数据用几次，决定了你有多 off-policy

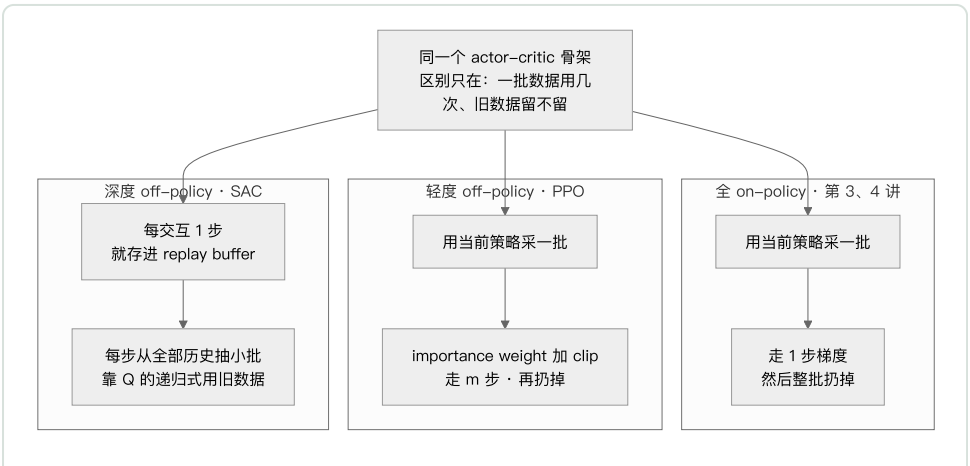


图 5-1 | 同一个 actor-critic 骨架的三个 off-policy 层级：数据用一次、用多次、永远留着（自绘示意）· [▶ 看原幻灯片 46:48](#)

- on-policy 与 off-policy：**on-policy 指更新策略用的数据必须由当前这个策略采出来；off-policy 指可以用别的策略——通常是自己以前的版本——采的数据。CME295 第 5 讲用"SFT 是 off-policy、RL 阶段是 on-policy"介绍过这对词，本讲讲的是 RL 算法内部怎么一级一级变得更 off-policy。

- **为什么想 off-policy**：和环境交互是最贵的东西（真机上是时间和磨损，仿真里也要算力），梯度步相对便宜。on-policy 算法每采一批只走一步就扔，数据浪费得厉害。
- **三个层级**（图 5-1）：全 on-policy，一批一步（第 3、4 讲）；轻度 off-policy，一批走多步、然后扔掉——PPO；深度 off-policy，所有历史都留在 replay buffer 里反复用——SAC。三者共用同一个 actor-critic 骨架，区别只在数据怎么用、公式里哪些 on-policy 假设被拆掉。
- **讲者给的学习目标**：看懂 PPO 和 SAC 这两个实际在用的算法背后的关键概念；最后比较 PPO、SAC 和模仿学习各自适合的场景。

3. 一批数据走多步：importance weight 与 surrogate objective

- **on-policy actor-critic 的五步**：① 用 π_θ 采一批数据；② 用这批数据的奖励拟合 V ；③ 对批里每个 (s, a) 算 advantage；④ 估计策略梯度；⑤ 更新参数。走完第 ⑤ 步，参数从 θ 变成 θ' ，而数据是 π_θ 采的——想再走一步，就得在 θ' 处估计梯度，可手里没有 $\pi_{\theta'}$ 的数据。
- **importance weight**（第 3 讲的工具）：给批里每个样本乘一个比值 $\pi_{\theta'}(a | s) / \pi_\theta(a | s)$ ——新策略给这个动作的概率，除以采数据的旧策略给它的概率。旧策略爱选、新策略不爱选的动作被打折；反过来的被放大。这样用旧数据也能得到新策略的梯度。advantage 估计不动，仍是旧策略的；反向传播走的是 θ' 。之后每走一步，变的只有这个比值。
- **surrogate objective 是怎么来的**：实现时没人手写梯度，而是写一个目标函数交给 PyTorch / JAX 自动求导。所以要找一个函数，它对 θ' 的梯度恰好等于上面的加权梯度。看一眼加权梯度：里面有 $\pi_{\theta'}$ 乘 $\nabla \log \pi_{\theta'}$ 。而 $\nabla \log \pi = \nabla \pi / \pi$ ，所以这个乘积就是 $\nabla \pi_{\theta'}$ ——这是第 3 讲推策略梯度时那个 log 技巧倒过来用。于是把 $\nabla \log$ 去掉、只留 $\pi_{\theta'}$ 本身，就是要找的函数。记作 $\tilde{J}$ ，加波浪号是因为它不是真目标（期望回报 J ），只是一个梯度相同的替身。

$$\tilde{J}(\theta') = \sum_i \frac{\pi_{\theta'}(a_i | s_i)}{\pi_\theta(a_i | s_i)} \hat{A}^{\pi_\theta}(s_i, a_i), \quad \nabla_{\theta'} \tilde{J}(\theta') = \sum_i \frac{\pi_{\theta'}(a_i | s_i)}{\pi_\theta(a_i | s_i)} \nabla_{\theta'} \log \pi_{\theta'}(a_i | s_i) \hat{A}^{\pi_\theta}(s_i, a_i)$$

左边是 surrogate objective：对批里每个样本 i ，新策略与旧策略给动作 a_i 的概率之比，乘上旧策略下的 advantage 估计 $\hat{A}$ ；右边是它对 θ' 的梯度，正好是 importance weight 版的 actor-critic 梯度。分母和 $\hat{A}$ 相对 θ' 都是常数。

- **怎么读它：**策略能动的只有分子。要把 J 做大，办法很直接： $\hat{A}$ 为正在的动作把概率抬高——旧策略给的概率越低，抬高后的比值越大，收益越大； $\hat{A}$ 为负的动作把概率压低。第一次走梯度之前，比值恒为 1；走了一步才开始偏离。CME295 第 5 讲 PPO-Clip 里的 $r_t(\theta)$ 就是这个比值。
- **问答**
 - 为什么要看 surrogate 而不是直接看梯度？——是同一个东西。但看 surrogate 更容易看出“策略被鼓励去做什么”；PPO 写成目标也比写成梯度好读。
 - 为什么非要 importance weight？——没有它，旧数据估出来的就不是新参数处的梯度，多走一步都没有依据。

4. 走多了会怎样：advantage 会过期

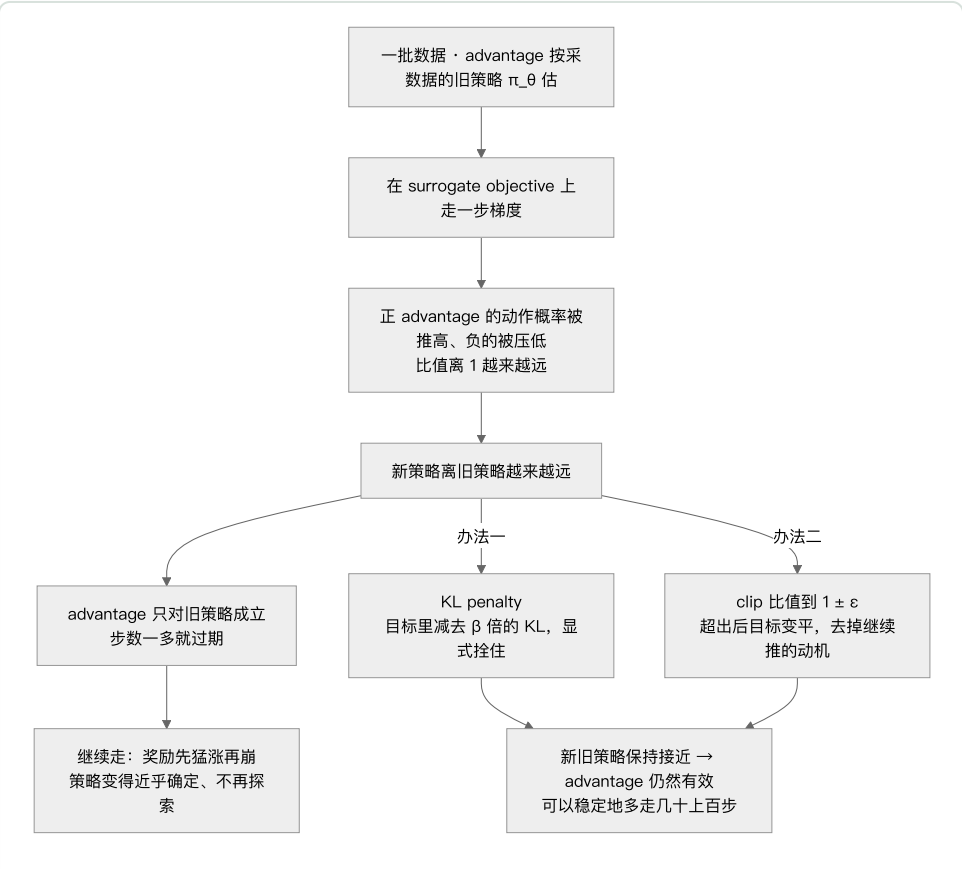


图 5-2 | 一批数据走太多步为什么会崩，以及两个断点在哪（自绘示意）。

► 看原幻灯片 13:52

- **哪里会出问题：**advantage 是从一批有限的的数据（讲者给的量级是几千个时间步）、按旧策略估出来的，它只对旧策略成立。多走几十上百步以后策略已经变了，advantage 还是老的——讲者管这叫对 advantage 过拟合（overfit）。更麻烦的是，surrogate 的动机本身就

在推着策略远离旧策略（比值越大越好），所以越走 advantage 越失效，是个正反馈。

- **板书上的三条学习曲线**（横轴是采到的数据量，纵轴是奖励）：每批只走 1 步——学得很慢，每个数据点只被用一次；走 5 步——快一些；贪心地走很多步——奖励先猛涨，然后不稳，甚至崩掉。崩的一种形态：策略把某个高 advantage 动作的概率推到接近 1，变得几乎确定、不再探索；神经网络是平滑的，相邻状态也一起被带偏。
- **问答里的网格例子**：agent 在 2D 网格里要走到某个目标，一批数据告诉它"向右走是对的"，却没告诉它走到某处要停下转向上；对这批 advantage 拟合过头的策略会一路向右冲进角落。再举一例：旧策略在某个状态选了向上，advantage 说"这条路好，因为之后会向上"；新策略若在那个状态改了主意，这些 advantage 就整个不成立，只能重新采数据。
- **另两个问答**："走 5 步"指在一批数据上连做 5 次梯度更新，再采新批、重训 V ；现在 V 只在当前批上训，怎么复用旧批是本讲后半段的事。比值的分母是旧策略、固定不动，分子随策略变，所以旧策略给的概率很小时，比值可以冲到很大，反过来也可以压到零——约束的目的就是让它待在 1 附近。
- **对症的思路**：只要新策略离旧策略不太远，advantage 就还大致有效。两个办法：办法一在目标里显式加约束；办法二直接去掉"越推越好"的动机。图 5-2 画的就是这条因果链和两个断点。

5. 办法一：KL penalty，把新策略显式拴在旧策略旁边

$$\tilde{J}_{\text{KL}}(\theta') = \tilde{J}(\theta') - \beta \text{KL}(\pi_{\theta'}(\cdot | s) \parallel \pi_{\theta}(\cdot | s))$$

$\tilde{J}$ 是第 3 节的 surrogate objective；KL 项衡量新策略 $\pi_{\theta'}$ 与旧策略 π_{θ} 在同一状态下给出的动作分布差多远； β 决定罚多重。

- **KL 散度**（CME295 第 5 讲讲过）：衡量两个分布差多远的量，非负、不对称。对离散动作，两个 categorical 分布的 KL 直接按定义算；对连续动作，策略常是高斯，KL 是两组均值和方差的函数——查公式加进目标即可，求导仍交给框架。加了这一项，策略在多走几步的同时会被拉着不离旧策略太远，advantage 因而更久有效，就能稳定地走更多步；但它也在限制步长，是个平衡。板书上的第四条曲线是"多步加 KL 罚"，比前三条都好。
- **β** ：罚多重的超参数。讲者的经验：它重要，但按数量级调就够了，不必精确到小数点后几位。

- **问答里的两个工程点**

1. 算 KL 要用旧策略的参数，得在显存里再放一份——LLM 上就是整份旧模型的权重，很烦。折中办法是只缓存旧策略给这批数据的概率，用完即弃。
2. 图像输入的策略要走很多步梯度才能把视觉特征学好；KL 约束的价值正在于允许多走步来打磨特征，同时不让动作分布变太多。

- **去向：**本讲之后不再用 KL 版。PPO 有一个带 KL 罚项的变体，不是最常用的，但在偏好优化里用得很多，几周后（第 9 讲 RL for LLMs）会再见到。

小注：CME295 第 5 讲 RLHF 目标里的 KL，罚的是"当前策略 vs 冻结的 reference (SFT 模型)"；本讲罚的是"新策略 vs 上一轮策略"——同一件工具，锚点不同，LLM 实践里常两个都要。PPO 论文里这个变体叫 PPO-penalty， β 还会按实测 KL 的大小自动增减，不完全靠手调。

6. 办法二：clip importance weight，去掉"越推越好"的动机——这就是 PPO

- **做法：**记 importance weight 为 w 。把 w 截在 $[1 - \epsilon, 1 + \epsilon]$ 之间：先取 $\max(w, 1 - \epsilon)$ 挡住下界，再取 $\min(\cdot, 1 + \epsilon)$ 挡住上界。 ϵ 取 0.2 左右——不是"无穷小"的那种 ϵ ，区间就是 0.8 到 1.2。
- **为什么有用：**比值一旦推过 1.2，目标就停在 $1.2 \times \hat{A}$ 不再增长，继续抬高概率没有收益；压到 0.8 以下同理。它并不禁止策略变化，只是把变化的动机拿掉了。讲者说，给 importance weight 加边界是 PPO 的核心思想——trick 1。把 surrogate 里的 w 换成截断后的 w ，就得到 clip 版 surrogate。

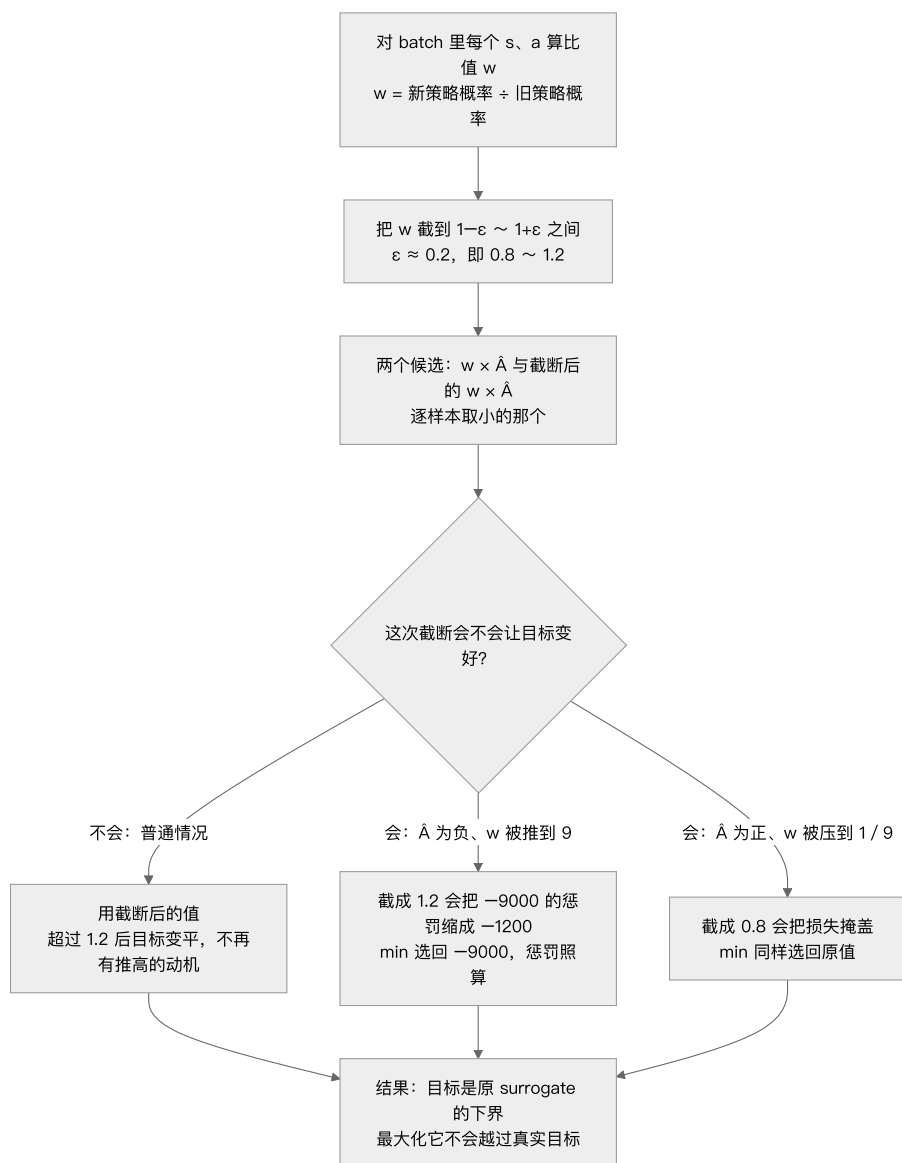


图 5-3 | PPO 的两个 trick 叠在一起: 截断去掉动机, 取 min 保证截断只会让目标变差不会变好 (自绘示意) · [▶ 看原幻灯片 32:57](#) · 出处: Schulman et al., 2017

- trick 2: 和原目标取 min** (图 5-3)。clip 有个漏洞: 极少数情况下, 截断反而会让目标变好。板书的例子: 某个 (s, a) 的 $\hat{A}$ 是 -1000 (很坏的动作), 新策略却把它的概率从 0.1 推到了 0.9 ——可能是为了照顾附近的好动作顺带推上去的, 比值是 9 ; 截到 1.2 之后, 惩罚从 -9000 缩成 -1200 , clip 帮它把错误掩盖了。反过来, $\hat{A}$ 为 $+1000$ 而概率从 0.9 压到 0.1 , 比值 $1/9$, 截到 0.8 同样把损失掩盖。修法是逐样本取 $\min(w \times \hat{A}, \text{clip}(w) \times \hat{A})$: clip 永远不会让目标变好, 于是整个目标是原 surrogate 的下界。优化一个下界是安全的: 你不可能把它推到超过真目标的地方去。
- 问答:** 为什么两边都截? ——上面两个例子正好是一上一下两种事故, 只截一边挡不住另一种。讲者也坦言, 不确定这种事故在实践中

多常发生。

$$\tilde{J}_{\text{PPO}}(\theta') = \sum_i \min(w_i \hat{A}_i, \text{clip}(w_i, 1 - \epsilon, 1 + \epsilon) \hat{A}_i), \quad w_i = \frac{\pi_{\theta'}(a_i | s_i)}{\pi_{\theta}(a_i | s_i)}, \quad \epsilon \approx 0.2$$

w_i 是第 i 个样本的 importance weight, $\hat{A}_i$ 是旧策略下的 advantage 估计, clip 把 w_i 限制在 $1 \pm \epsilon$ 内; 对每个样本取截断前后两项中较小者, 再求和。这就是 CME295 第 5 讲的 PPO-Clip (那里记作 L^{CLIP} , $r_t(\theta)$ 就是 w), 这里补上了它从 importance weight 出发的来路。

小注: $\epsilon = 0.2$ 也是 PPO 论文的推荐默认值。CS329A 里 DAPO 的 clip-higher 改的就是上界 $1 + \epsilon$, 让低概率动作能被抬得更多——按本讲的逻辑, 就是有意放宽"去掉动机"的那一侧。

7. PPO 的第三个 trick: GAE, 把不同步长的 advantage 混着用

- 作用在 advantage 上。 V^* 照常拟合 (Monte Carlo 或 bootstrap 都行), 但算 advantage 时不再只看一步: n 步版本先加 n 步的真实奖励, 再加第 n 步之后那个状态的 V^* , 最后减去当前状态的 V^* 。再把不同 n 的估计加权混合——权重随 n 增大快速衰减, 主要还是靠 $n = 1, 2$ 这些短的。这就是 generalized advantage estimation (GAE)。

$$\hat{A}_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \hat{V}(s_{t+n}) - \hat{V}(s_t), \quad \hat{A}_t^{\text{GAE}} = \sum_{n \geq 1} w_n \hat{A}_t^{(n)}$$

$\hat{A}^{(n)}$ 是 n 步 advantage: 前 n 步真实奖励按 γ 折扣相加, 加上 n 步后状态的值估计, 减去当前状态的值估计; GAE 把各个 n 的估计用权重 w_n 混合, w_n 随 n 快速衰减。

- 偏差与方差的权衡 (第 4 讲的老话题): n 小, 靠 V^* 多, 偏差大方差小; n 大, 靠真实奖励多, 偏差小方差大。混合是在两者之间取一条折中线。讲者很坦率: 她自己没做过系统对比, 不确定它有多必要; 她能给的直觉是——时间步很密的时候 (比如高频控制), 一步的 $r + V(s') - V(s)$ 里两个 V 几乎相等, 估计对 V^* 的误差很敏感, 拉长视野会稳一点; 混合不同 n 则是在不确定哪个 n 对时的对冲。
- 问答: 用了这些权重还要 γ 吗? ——要, 两者一起用, 展开后会出现 γ 和 λ 相乘的项。这和" n 步回报的加权平均"一样吗? —— V^* 的拟合方式不变, 只是 advantage 换成不同 n 的加权平均, 可以这么理解。

小注: GAE 论文里的权重是 $w_n = (1 - \lambda) \lambda^{n-1}$, λ 就是那个第二超参数: $\lambda = 0$ 退化为一部 TD advantage, $\lambda = 1$ 退化为 Monte Carlo

advantage, 实践中常取 0.9 几。CME295 第 5 讲说 GAE 有 γ 、 λ 两个超参数却没展开, 展开就是这里。

8. 完整的 PPO: 算法、典型超参数、half-cheetah 曲线

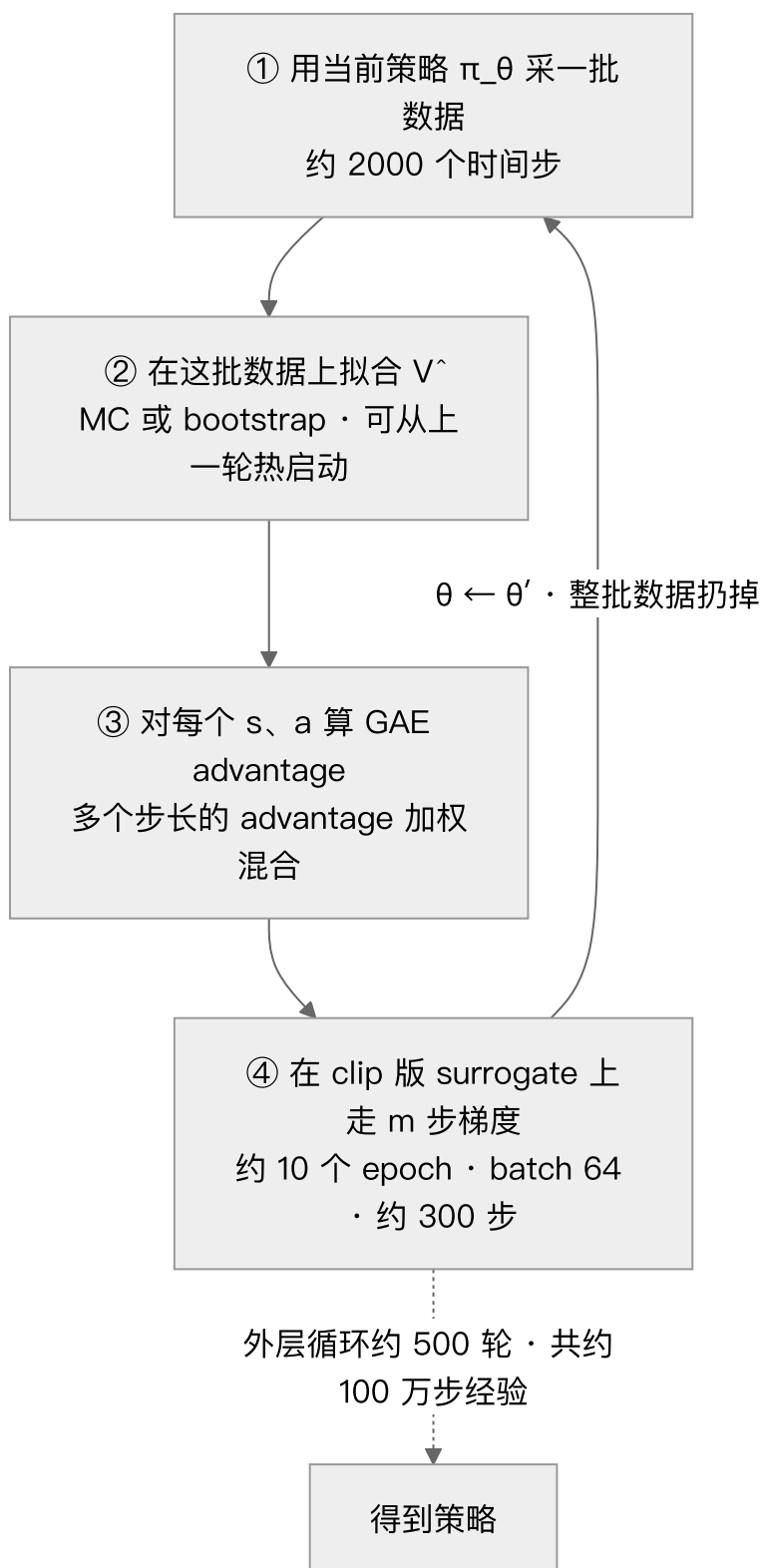


图 5-4 | PPO 的一轮：采一批、拟合 $V^\wedge$ 、算 GAE、走几百步，然后整批扔掉（自绘示意）· [▶ 看原幻灯片 42:11](#) · 出处：Schulman et al., 2017

- 算法：① 采一批 → ② 拟合 $V^\wedge$ → ③ 算 GAE advantage → ④ 在 clip 版 surrogate 上走 m 步梯度 → 回到 ①，用最新策略采下一批。

- **典型超参数**（讲者强调只是量级，实际按领域调）：一批约 2000 个时间步——如果一条轨迹就有 1000 步，那这批只有两三条轨迹；短视野任务则是很多条。第 ④ 步在这批数据上跑约 10 个 epoch（一个 epoch 是把这批数据过一遍），mini-batch 64，算下来约 300 步梯度——和 on-policy 的 1 步相比，这就是 clip 换来的东西。 $\epsilon = 0.2$ 。外层循环约 500 轮，总共约 100 万步经验。
- **half-cheetah 曲线**：half-cheetah 是仿真里“半只猎豹”跑步的常用 benchmark。横轴是经验步数，纵轴是奖励。米色的是（稍微聪明一点的）vanilla policy gradient——不拟合 advantage、不 clip；PPO 在上方，用少得多的数据涨得快得多。讲者归因于两点：拟合了 advantage，以及 clip 让每批数据能走几百步。

小注：这条曲线应出自 PPO 论文的 MuJoCo 对比图（推断，课上没点名）。

• 问答

- 第 ④ 步到第 ① 步的箭头是什么意思？——走完 m 步后用最新策略重新采一批，此时 θ 变成刚才的 θ' ，旧批扔掉。 V^* 可以从上一轮热启动，也可以重训，看领域。
- 曲线里每个算法的超参数怎么来的？——各自调各自的。规范做法是在一两个环境上调好，然后原封不动用到其他环境；按环境逐个调等于偷偷多花了样本，曲线上看不出来。控制频率差很多（1 Hz 对 50 Hz）或者对象差很多（LLM 对 half-cheetah）时，超参数确实要重调。

9. 更 off-policy: replay buffer, 以及它会弄坏什么

- **从轻度到深度**：PPO 仍然每轮扔掉旧批。下一步是把过去所有的试错经验都留着用。这需要两件事：(1) 一个 replay buffer——存历史数据的集合（作业 1 里已经用过），新数据不断加进去，太老的可以丢，名字来自“回放”过去的经验；(2) 把公式里隐含的 on-policy 假设一条条拆掉。
- **算法形态的变化**：不再“采一批、更新一批”，而是更在线：与环境交互一步，就更新一次；每次更新用的 mini-batch 从整个 buffer 里随机抽，而不只是最近这批。

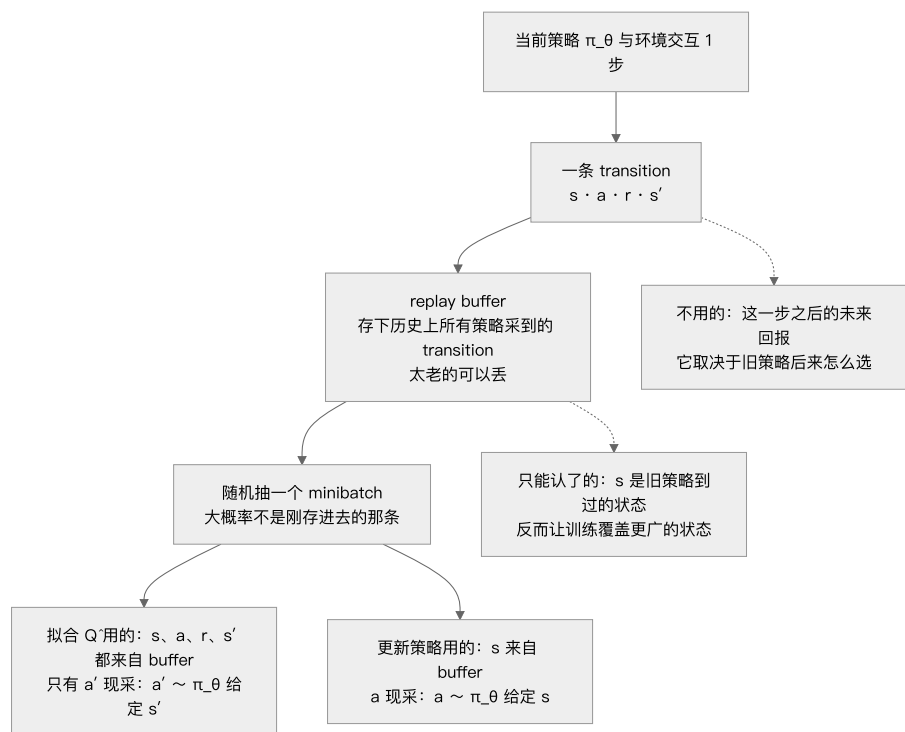


图 5-5 | replay buffer 进什么、出什么：四元组照用，动作现采，未来回报不用（自绘示意）· [▶ 看原幻灯片 48:50](#)

- **天真版本：**buffer 从空开始 → 用当前策略交互、把 transition (s, a, r, s') 存进去 → 随机抽一个 mini-batch（有很小的概率抽到刚存的那条）→ 用和以前一样的 bootstrap 目标 $r + V(s')$ 拟合 V ；只是对 mini-batch 取平均 → 算 advantage → 走一步策略梯度 → 循环。
- **它坏在两处。**讲者让全班讨论了两分钟第一处：在整个 buffer 上按 $r + V(s')$ 拟合出来的 V ，是哪个策略的值函数？答案是——不是当前策略的。当前策略在 buffer 里也许只有一条数据，其余全是历史策略采的；不同策略在同一状态选不同动作，随之而来的 s' 和 r 都不同。硬要说的话，它是所有历史策略的某种混合体的值函数，而且这个混合还随状态变（有的状态只有某个旧策略去过），根本写不清楚。可我们要的是当前策略的 V ，否则 advantage 和梯度都不对。第二处：buffer 里的动作 a 是旧策略选的，不是当前策略会选的——它既出现在 V 的目标里，也出现在策略梯度里。

小注：replay buffer 在指定阅读 DQN (Mnih et al., 2013) 里就是主角之一：存最近 100 万帧的经验，从中均匀抽 mini-batch，一条经验会被用很多次；更早的出处是 Lin (1992) 的 experience replay。

10. 修法一：不拟合 V ，改拟合 Q ——递归式对任何 (s, a) 都成立

- **核心问题：**怎么用别的策略采的数据，拟合出当前策略的值函数？讲者的办法是：别拟合 V ，改拟合 Q 。

- **板书推导，用话说一遍**

1. 从定义出发： $Q^{\pi}(s_t, a_t)$ 是"从 s_t 做 a_t 、之后按 π "的未来奖励之和的期望。
2. 把求和的第一项拆出来：第一项是 $r(s_t, a_t)$ ，而 s_t 、 a_t 是 Q 的输入，是给定的——这一项没有任何随机性。
3. 剩下的从 $t + 1$ 起的奖励和，要对 s_{t+1} （由环境动态决定）和之后的动作（由 π 决定）取期望。仔细看，它正是 $V^{\pi}(s_{t+1})$ 的定义。
4. 再往前走一步： $V^{\pi}(s_{t+1})$ 是对 $a_{t+1} \sim \pi$ 取期望的 $Q^{\pi}(s_{t+1}, a_{t+1})$ 。于是 Q 被自己递归地定义：当前奖励，加上下一步 Q 的期望。
5. 关键观察：这个等式对任意 (s_t, a_t) 都成立。推导里没有任何地方假设 a_t 是谁选的——策略 π 只在 $t + 1$ 之后出场。

$$Q^{\pi_{\theta}}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(\cdot | s_t, a_t), a_{t+1} \sim \pi_{\theta}(\cdot | s_{t+1})} [Q^{\pi_{\theta}}(s_{t+1}, a_{t+1})]$$

$r(s_t, a_t)$ 是这一步的奖励， p 是环境的状态转移分布（不随策略变）， a_{t+1} 从当前策略 π_{θ} 采样；等式说的是" Q 等于当前奖励加下一步 Q 的期望"，且对任何 (s_t, a_t) 成立。

- **它为什么解决了问题：**buffer 里一条数据是 (s, a, r, s') 加上之后的奖励。之后的奖励取决于旧策略后来怎么选，关系很复杂，不用；但 (s, a, r, s') 四样都能用—— a 是谁选的无所谓，因为它是 Q 的输入； s' 是环境对 (s, a) 的反应，环境动态不随策略变。唯一要换掉的是下一步的动作：不用 buffer 里的 a' ，而是把 s' 喂给当前策略现采一个 $a' \sim \pi_{\theta}(\cdot | s')$ 。于是目标里的 $Q(s', a')$ 是当前策略的 Q ，拟合出来的也就是当前策略的 Q 。

$$y_i = r_i + \gamma \hat{Q}_{\phi}(s'_i, a'_i), \quad a'_i \sim \pi_{\theta}(\cdot | s'_i), \quad \min_{\phi} \sum_i (\hat{Q}_{\phi}(s_i, a_i) - y_i)^2$$

y_i 是第 i 条样本的目标值：数据里的奖励，加上 γ 倍的"下一状态 s'_i 与现采动作 a'_i 处的 Q "； $\hat{Q}_{\phi}$ 是参数为 ϕ 的 Q 网络，训练就是把 $Q(s_i, a_i)$ 往 y_i 上拉的 L2 回归，算 y 时 $\hat{Q}$ 当常数看。

- **监督学习的形式：**输入 (s, a) ，网络输出一个数，用 L2 把它往 y 上拉——和第 4 讲拟合 V 一模一样，只是多了动作输入。期望用单样本估计： s' 用数据里那一个， a' 采一个；也可以对同一个 s' 采多个 a' 取

平均，而且一个 mini-batch 里每条样本各采一个，本身就是多个样本。

- **代价：覆盖度**（问答里反复出现的词）。目标 y 要准，buffer 里有“和当前策略会选的动作相近”的数据。某个状态从没去过，不行；去过但只试过很不一样的动作，也不行—— Q 在没见过的动作上只能靠神经网络插值，插值靠谱到什么程度，目标就准到什么程度。所以旧数据并不是白给的，仍然需要当前策略持续采数据来补覆盖。有学生问 KL 约束能不能帮忙（让策略别离数据太远）——讲者说下面要讲的算法没这么做，但这是个好主意，“别偏离太远”这类约束在 RL 里会反复出现。

小注：板书上的递归式省了 γ ，幻灯片上的目标写作 $r + \gamma Q$ 。另外，动作离散且数量少时，对 a' 的期望可以精确算出来；如果把“按 π_θ 采 a' ”换成“取让 Q 最大的 a' ”，就是 Q-learning / DQN（指定阅读，第 6 讲）——那时连策略网络都不用了。覆盖度问题在第 7 讲 offline RL 里会成为主角。

11. 修法二：策略更新的动作也从当前策略现采——SAC 的骨架

- **策略更新做两处改动**：(1) 直接用 Q 而不是 advantage——也就是不减 baseline。方差会大一些，但现在用的是整个 buffer 的数据而不只是最近一批，样本多得多，扛得住。(2) 动作不用 buffer 里的 a ，而是把 s 喂给当前策略现采 $a \sim \pi_\theta(\cdot | s)$ ——理由是当前策略的动作多半比历史策略的好；这和 Q 目标里换 a' 是同一个技巧，只是用在当前状态而不是下一状态。

$$\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(a_i | s_i) \hat{Q}_\phi(s_i, a_i), \quad s_i \sim \mathcal{D}, \quad a_i \sim \pi_\theta(\cdot | s_i)$$

s_i 从 replay buffer $\mathcal{D}$ 里抽， a_i 从当前策略现采， $\hat{Q}_\phi$ 是第 10 节拟合好的 Q 网络；形状和第 3 讲的策略梯度一样，只是回报换成 Q ，且没有 baseline。

- **剩下没法修的一处**： s 来自 buffer，是历史策略走到过的状态，不是当前策略的状态分布。讲者说这一点没有办法，但也未必是坏事：策略在更宽的状态分布上被训练，而不只在自己会去的那些状态上。

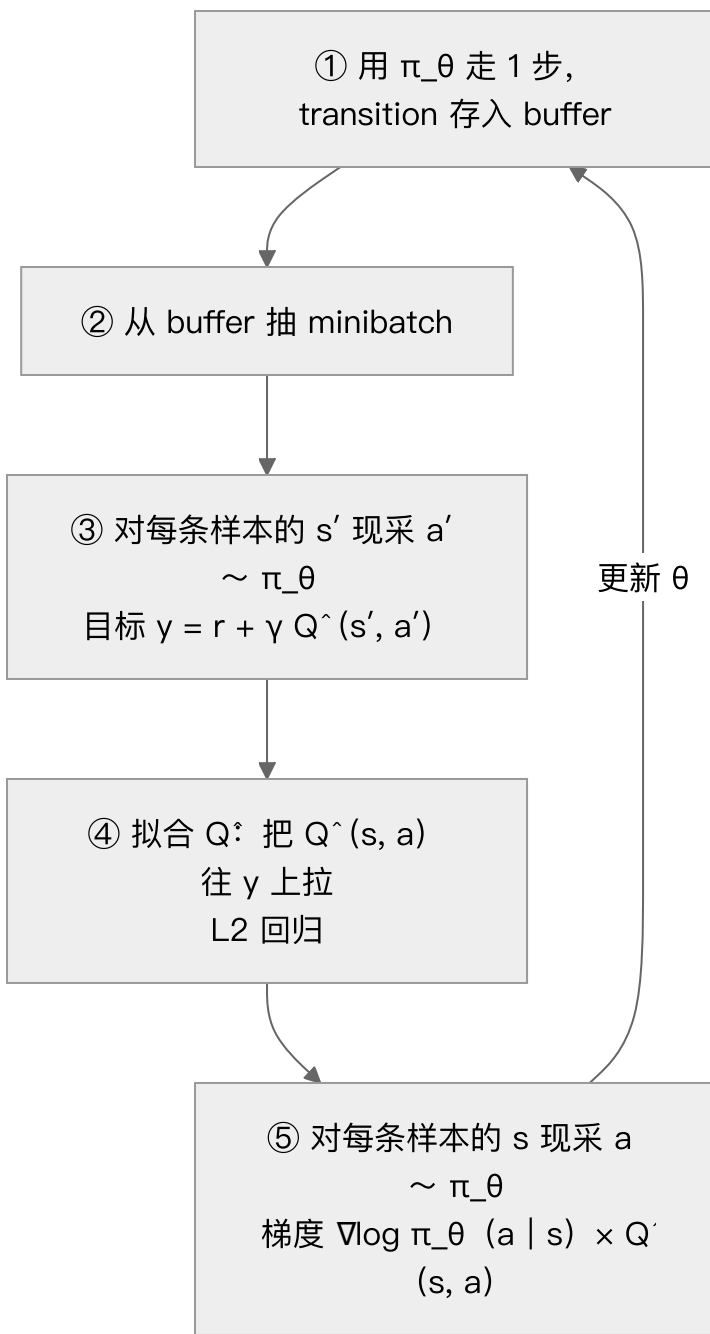


图 5-6 | off-policy actor-critic 的一轮：两处“动作现采”是它能用旧数据的全部秘密（自绘示意）· [▶ 看原幻灯片 1:06:18](#) · 出处：Haarnoja et al., 2018

- **完整算法**（图 5-6）：交互一步存 buffer → 抽 mini-batch → 对每条样本的 s' 现采 a' 、算目标 y → 用 L2 拟合 $Q^{\wedge}$ → 对每条样本的 s 现采 a ，用 $\nabla \log \pi_{\theta}(a | s) \times Q^{\wedge}(s, a)$ 更新策略 → 循环。讲者说这基本上就是 Soft Actor-Critic (SAC)；估计这个策略梯度还有别的方式，拟合 Q 也有更讲究的做法，下一讲（Q-learning）会见到。

小注：真正的 SAC (Haarnoja et al., 2018) 在这个骨架上还加了几样课上有意略过的东西：“soft”指目标里加了策略的熵当奖励，鼓励探索、也让

Q 目标更平滑；Q 网络训两份取较小值，压住 Q 的高估；算目标 y 用一份缓慢跟随的 target network 而不是正在训的 $Q^{\hat{}}$ ，免得目标跟着自己跑；连续动作下策略梯度用重参数化技巧而不是 $\log \pi \times Q$ 。这些就是讲者说的"其他估计梯度的方式"和"更讲究的拟合"。

12. PPO 还是 SAC：数据效率、稳定性，以及谁在真机上用

- **数据效率**：课上一张曲线，橙色是带 replay buffer 的 SAC，棕色是 PPO——SAC 用少得多的经验达到同样的奖励。代价是 Q 函数这套东西超参数更难调，训练通常没有 PPO 稳。
- **真机上的 RL**：够 off-policy 的算法才能把交互量压到真机可承受的程度。课上的例子：一台四足机器人用本讲的算法，大约两小时学会向前走；一个用示范起步的精细操作任务，RL 在所有测试任务上拿到 100% 成功率，模仿学习低得多，而且 RL 学出的策略动作更快。
- **PPO 的地盘**：要稳、不在乎数据时用它。仿真里数据不要钱，可以在仿真里训，把仿真器调准后迁移到真机（sim-to-real）；操作类任务的 sim-to-real 难得多，但课上放了用 PPO 在仿真里训出、真机上花约五分钟拧好魔方的例子。PPO 也是语言模型上的常用选择——下周（第 9 讲）开始讲。

	PPO	SAC（本讲推出的骨架）
数据怎么用	一批走 m 步（约 300 步）后扔掉	全部历史进 replay buffer，每步都抽来用
评估谁	拟合 $V^{\hat{}}$ ，算 GAE advantage	拟合 $Q^{\hat{}}$ ，目标 $r + \gamma Q(s', a')$ ， a' 现采
拴住策略的方式	clip importance weight 加取 min	没有显式约束；靠 Q 的递归式让旧数据合法
策略梯度用的动作	数据里的 a	从当前策略现采的 a
数据效率	低	高得多
稳定性与调参	稳，超参数好调	不如 PPO 稳，超参数难调
典型用武之地	仿真、sim-to-real、LLM	真机机器人

小注：三个真机例子课上没给出处，应为：Haarnoja et al., 2018 《Learning to Walk via Deep RL》（Minitaur 四足，真机约两小时，用的是最大熵 RL 即 SAC 一系）；HIL-SERL（Luo et al., 2024：示范加人工纠正起步，1 到 2.5 小时训练达到近乎 100% 成功率，比模仿学习成功率高约 2 倍、执行快约 1.8 倍）；OpenAI 2019 的机械手拧魔方（仿真里用 PPO 训练）。SAC 对 PPO 的曲线应出自 SAC 论文的 MuJoCo 对比图（推断）。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
AlphaGo 的 V 与 Q	奖励是终局胜负；V 同时随棋局和策略变，Q 先钉死一手	3:39	—
importance weight 版梯度与 surrogate objective	认出 $\pi_{\theta'} \times \nabla \log \pi_{\theta'} = \nabla \pi_{\theta'}$ ，所以目标里只剩比值乘 $\hat{A}$	12:22	—
三条学习曲线（板书）	每批 1 步、5 步、太多步各自的形状：慢、快、先涨后崩	13:52	—
KL penalty 板书	在 surrogate 后面减 β 倍的 KL(新 旧)； β 按数量级调	21:45	PPO (KL 变体)
clip 的 max / min 写法	先 max 挡下界 $1 - \epsilon$ ，再 min 挡上界 $1 + \epsilon$ ； $\epsilon \approx 0.2$	29:54	PPO
PPO 最终目标与 ± 1000 例子	clip 项与原项取 min，整体是原 surrogate 的下界； $\hat{A} = -1000$ 、比值 9 时 clip 反而缩小惩罚	32:57	PPO
GAE 的多步 advantage	n 步奖励加 $V(s_{t+n})$ 减 $V(s_t)$ ，不同 n 加权；权重随 n 快速衰减	38:35	GAE
PPO 算法与典型超参数	2000 步一批、10 epoch、batch 64 约 300 步、 $\epsilon = 0.2$ 、500 轮约 100 万步	42:41	PPO
half-cheetah 曲线	米色 vanilla PG 对 PPO：拟合 advantage 加 clip 带来的数据效率差距	44:15	应为 PPO 的 MuJoCo 对比图
Q 递归式板书	拆出第一项 $r(s_t, a_t)$ ，剩下的是 $V^{\pi}(s_{t+1})$ ，再写成 $a_{t+1} \sim \pi$ 下的 Q	54:57	—
Q 的目标 y	s, a, r, s' 来自 buffer，只有 a' 从前策略采；未来回报不用	1:00:04	—
完整 off-policy actor-critic	五步循环；标出 Q 目标和策略梯度里两处"动作现采"	1:06:18	SAC
SAC 对 PPO 曲线	橙色 SAC 用少得多的经验到达同样奖励；代价是难调、不稳	1:07:19	应为 SAC 的对比图
真机例子	四足两小时学会走；示范起步的操作任务 RL 100% 对模仿学习；仿真训的 PPO 拧魔方	1:07:50	见第 12 节小注

提到的工作

名称	在本讲里的作用
策略梯度与 importance weight（第 3 讲）	开场回顾；一批数据走多步的工具就是 importance weight

名称	在本讲里的作用
Actor-critic 与 policy evaluation (第 4 讲)	本讲的出发点：把回报换成 advantage 估计
AlphaGo (Silver et al., 2016)	解释 V 和 Q 直觉的例子：奖励是终局胜负
PPO (Schulman et al., 2017)	本讲前半主角：clip 版 importance weight、与原目标取 min；带 KL 罚项的变体
GAE (Schulman et al., 2015)	PPO 的第三个 trick：多步 advantage 加权混合
PyTorch / TensorFlow / JAX	surrogate objective 存在的理由：把目标交给自动求导
half-cheetah	PPO 对 vanilla PG、SAC 对 PPO 两张曲线用的仿真跑步任务
replay buffer (作业 1; DQN, Mnih et al., 2013, 指定阅读)	深度 off-policy 的第一件事：存下全部历史 transition
SAC (Haarnoja et al., 2018)	本讲后半推出的算法"基本上就是它"；熵项等细节课上未讲
Q-learning (第 6 讲、复习课)	讲者预告："更讲究的拟合 Q 的方法"
真机四足行走 (应为 Haarnoja et al., off-policy 够省数据, 真机约两小时学会走 2018)	
示范起步的精细操作 (应为 HIL-SERL, Luo et al., 2024)	RL 在所有测试任务上 100%，模仿学习低得多且更慢
机械手拧魔方 (OpenAI, 2019)	PPO 在仿真里训练、迁移到真机的例子
模仿学习 (第 2 讲)	结尾比较：数据够时 RL 成功率和速度都更高
偏好优化 / RL for LLMs (第 9 讲)	KL 罚项版 PPO 的去处；PPO 是 LLM 上的常用选择

术语对照

English	中文
on-policy / off-policy	同策略 / 异策略：训练数据必须来自当前策略 / 可以来自别的 (旧的) 策略
importance weight / importance ratio	重要性权重：新策略概率除以采数据策略的概率，用来纠正数据来源不同
surrogate objective	替代目标：梯度和真目标相同、专门写给自动求导的函数
policy evaluation	策略评估：估计某个策略的 V 或 Q
bootstrapping	自举：用自己的估计 (V 或 Q) 当训练目标的一部分
Monte Carlo return	蒙特卡洛回报：整条轨迹的奖励直接加起来
n-step return	n 步回报：加 n 步真实奖励，再接一个值估计
baseline	基线：从回报里减掉的平均水平，用来降方差

English	中文
advantage	优势：某个动作比策略平均水平好多少
KL divergence / KL penalty	KL 散度 / KL 罚项：衡量并惩罚新旧策略分布的差异
clipping	截断：把比值限制在 $1 \pm \epsilon$ 内，去掉继续偏离的动机
lower bound	下界：PPO 目标不超过原 surrogate，优化它不会"越界"
GAE (generalized advantage estimation)	广义优势估计：不同步长 advantage 的加权混合
epoch / mini-batch	轮 / 小批：把整批数据过一遍叫一个 epoch；每步梯度用的的小份数据叫 mini-batch
replay buffer	回放缓冲：存历史 transition 的数据集，更新时从中随机抽
transition	转移四元组 (s, a, r, s') ：一步交互的记录
target y	目标值： Q^* 回归的标签， $r + \gamma Q^*(s', a')$
coverage	覆盖度：数据里有没有当前策略会选的那类动作、会到的那类状态
mixture of policies	策略混合体：在 buffer 上直接拟合 V 时，实际拟合的对象
data efficiency	数据效率：达到同样奖励需要多少交互
warm start	热启动：新一轮的 V^* 从上一轮的参数开始训
sim-to-real	仿真到真机的迁移
half-cheetah	仿真里"半只猎豹"跑步的基准任务
soft actor-critic (SAC)	软演员-评论家：本讲推出的 off-policy actor-critic 的正式版本

字幕勘误

"PO" → PPO; "SACE""soft factor critic" → SAC / soft actor-critic; "actor credit" → actor-critic; "kale divergence" → KL divergence; "Alph Go" → AlphaGo; "GA" → GAE; "endstep returns" → n-step returns; "pietorch" → PyTorch; "skew functions" → Q functions; "circuit objective""sur objective" → surrogate objective; "sum of awards" → sum of rewards; "overfitit" → overfit; 讲 clip 范围时的 "something like 2" → 0.2, "uh8""08" → 0.8; "bygative a,000" → by negative 1,000; "1 over9" → 1/9; 网格例子中的 "needs the thumb" → needs to turn; "vi" → V; "fit QA""Q4 our current policy" → fit $Q(s, a)$ 、Q for our current policy; "ST""A+1" → s_t 、 a_{t+1} 。

带走的问题

1. PPO 的 clip 只是"去掉动机", 不是硬约束: 一个 mini-batch 里其他样本的梯度照样能把某个 (s, a) 的比值推出 $1 \pm \epsilon$ 之外。这和 KL penalty 的差别在实践中意味着什么? 什么时候值得两个一起用 (LLM 的 RLHF 就是这么做的)?
2. 在 buffer 上直接拟合 V 得到的是"历史策略混合体"的值函数, 换成 Q 就绕开了——递归式里究竟是哪一项吸收了"谁选的动作"? 如果动作离散且很少, $a' \sim \pi$ 的期望能不能精确算? 第 6 讲的 Q -learning 会怎么改这一项?
3. SAC 的目标 y 依赖覆盖度。把 replay buffer 换成一份固定的、别人采的数据集, 再也不和环境交互——本讲的算法会在哪一步出事? 这正是第 7 讲 offline RL 的起点。
4. GRPO (CS329A) 去掉了 value function、用组内均值当 baseline; 本讲的 off-policy 更新反过来去掉了 baseline、直接用 Q 。两者各自靠什么压方差? 为什么 LLM 上很少见 replay buffer 式的深度 off-policy?
5. 你的 agent 系统里, 历史日志就是一个 replay buffer。按本讲的逻辑, 哪些量可以直接从旧日志学 (Q 的递归式对任何 (s, a) 成立), 哪些不能 (未来回报依赖旧策略的后续选择)? 日志里"动作"的覆盖度够不够, 怎么判断?

CS224R 第 6 讲 | Q-learning

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 6 讲, 2025 年 4 月 18 日 · 在线 RL 算法的最后一讲; 之后有一节 Q-learning 复习课, 第 7 讲转入 offline RL 视频: <https://www.youtube.com/watch?v=-7kv6jf0isQ> (1:01:40, 英文字幕为自动生成, "Q" 几乎全被听成 p / t / key / two, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: Deep Reinforcement Learning with Double Q-learning (van Hasselt et al., 2015)、A Distributional Perspective on Reinforcement Learning (Bellemare et al., 2017; 课程页面误链了另一篇, 以这篇为准。课上没有讲到它, 见第 10 节末尾的小注)

一句话: 前几讲的算法都要学一个策略网络, 这一讲把它扔掉——只学 Q 函数, 行动时对每个候选动作查一次 Q、取 $\arg\max$ 。之所以行得通, 是因为"对准确的 Q^π 取 $\arg\max$ "得到的新策略一定不比原策略差 (policy improvement), 反复做就是 policy iteration; 再把 $\arg\max$ 直接写进回归目标 $r + \gamma \cdot \max Q$, 评估和改进合成一步, 就是 Q-learning, 它逼近的是 Bellman 最优方程的解。这个目标对任何一条 (s, a, r, s') 都成立, 所以天然 off-policy, 可以吃 replay buffer 里所有旧数据; 但"max + 神经网络"既不稳又系统性高估, 靠 DQN 的 target network、Double DQN 的选评分离、N-step 目标三个技巧, 它才在 Atari 和机器人抓取上真正跑起来。

时间轴

时间	内容
0:05	回顾 V 与 Q 的定义、policy gradient / actor-critic / off-policy actor-critic 三种循环; 今天: value-based RL, 三个目标
4:17	思考题: 手里有准确的 Q^π , 取 $\arg\max$ 定义新策略; 2D 导航例子, 同桌讨论
7:23	逐行推 Q^π : 新策略更好但不是最优; 问答: γ 小于 1 时 Q 随距离衰减, 稠密奖励一轮到位
12:27	新策略永远不比旧的差; 迭代算法: 收数据 $\rightarrow$ 拟合 $Q^\pi \rightarrow \arg\max$; 唯一的网络是 Q
14:32	问答: 连续动作怎么做 max; 稀疏奖励下要迭代多少轮 (backup 与 horizon)
16:36	写成算法: policy evaluation + policy improvement = policy iteration; 板书
19:41	问答引出 Q-learning: 把 max 放进目标; 两种理解; Bellman equation 与 Bellman optimality equation

23:20 问答：为什么用 Q 不用 V

26:29 术语小结；收敛性：表格能收敛，函数逼近连线性都可能发散

29:05 Q-learning 是 off-policy 的；为什么数据要比策略更"广"； ϵ -greedy 与 Boltzmann 探索

33:49 完整算法：replay buffer、均匀抽 batch、L2 loss 的梯度、K 步更新；餐馆比喻；一串关于 buffer 的问答

40:31 实践一：moving target 不稳 $\rightarrow$ target network $\rightarrow$ DQN (2013, Atari 从像素学起)

44:02 实践二：Q 值准不准？loss 上升不必慌；真实回报远低于估计：overestimation

47:09 为什么 max 会高估；Double Q-learning 用两张网选评分离；Double DQN 的实现

53:27 实践三：N-step 目标；off-policy 带来的偏差和几种应对

58:05 成功案例：Atari 超过人类、机器人抓取 96%；PPO / DQN / SAC 怎么选；总结

核心内容

1. 回顾与今天的三个目标

- 两个价值量（第 4、5 讲定义过，这里再压一遍）：价值函数 $V^{\pi}(s)$ 是从状态 s 出发、一直按策略 π 行动，未来奖励之和的期望；Q 函数 $Q^{\pi}(s, a)$ 是先在 s 执行动作 a 、之后再按 π 行动的期望回报。两者只差第一步是"随策略"还是"指定"。有折扣因子 γ 时，越远的奖励权重越小。
- 已经学过的三种循环：
 1. policy gradient（第 3 讲）：用当前策略采样轨迹，提高未来回报高的动作的对数概率；减掉 baseline，让高于平均的动作被强化、低于平均的被抑制。
 2. actor-critic（第 4 讲）：在循环里多加一步——先估计当前策略的期望回报（critic），再拿它来改策略（actor）。
 3. off-policy actor-critic（第 5 讲）：不只用最新一轮的数据，而是想用过去所有数据。为此要从旧策略的数据里估出 Q ，办法是把 Q 写成递归式： $Q(s, a)$ 等于当下的奖励加上"下一个状态、下一个动作"的 Q 。这个等式对所有 (s, a) 都成立，所以从 buffer 里抽 (s, a, s') 、从当前策略采一个 a' ，就能做监督回归。这种"用自己的估计当标签"叫 bootstrapping；前提是数据要覆盖新策略会选的动作。

- **今天：**value-based RL，也就是 Q-learning。Finn 说它可以算是第一个深度 RL 方法（DQN）的基础。三个目标：Q 函数和策略是什么关系；怎么完全不学显式策略也能做 RL；Q-learning 不稳定时怎么救。

2. 思考题：拿 Q^π 取 $\arg\max$ ，能不能得到更好的策略

- **设定：**有一个策略 π ，并且对它的 Q^π 估得很准。现在不再按 π 行动，而是在每个状态把所有候选动作的 Q^π 比一比，永远执行最大的那个：

$$\pi'(a | s) = 1 \left[a = \arg \max_{a'} Q^\pi(s, a') \right]$$

$\mathbb{1}[\cdot]$ 是指示函数：条件成立取 1，否则取 0。这是一个确定性策略 (deterministic policy)：概率全压在 Q 值最大的动作上。注意里面的 Q 仍然是旧策略 π 的 Q，不是新策略自己的。

- **例子（2D 导航）：**起点在一个蓝色方框里，目标是星星；到星星奖励 1，其他地方 0；为简单起见 $\gamma = 1$ 。当前策略是“永远向右”。同桌讨论：新策略 π' 比 π 好、坏、一样，还是已经最优？

- **逐行推**（课上一一起算的）：

- 星星所在那一行：一直向右就能撞到星星，“向右”的 $Q^\pi = 1$ ；但如果某一步“向上”，之后又回到向右，就会错过，所以同一行里 Q 依动作而异。
- 星星下面一行：向右的 $Q^\pi = 0$ （永远到不了），向上的 $Q^\pi = 1$ （上去一格之后按 π 向右，正好撞上）。
- 再往下一行：向上的 Q^π 也是 0——上去之后按 π 向右，还是错过。
- 于是 π' 在星星那行向右，在下一行向上，在更下面的行因为所有动作 Q 都是 0，只能沿用默认的向右。

- **结论：** π' 比 π 好（多了一整行状态能到目标），但不是最优——从起点出发它还是一路向右。有同学马上问“多迭代几次是不是就最优了”，Finn 说这正是接下来几张幻灯片要讲的。

- **问答**

- γ 小于 1 会怎样？例子里为了简单用了 $\gamma = 1$ ；若 $\gamma = 0.9$ ，星星旁边的 Q 是 1，再远一格是 0.9，再远是 0.9^2 ……随距离按 γ 的幂衰减；但到不了星星的那些行仍然全是 0。

◦ 奖励稠密一点呢？如果奖励是"到星星的负距离"，Finn 没在黑板上推，但自己算会发现：从同一个"向右"策略出发，一次改进就得到直奔星星的策略——稀疏奖励下"目标在哪"要一行一行来回传，稠密奖励一步就把方向指明了。

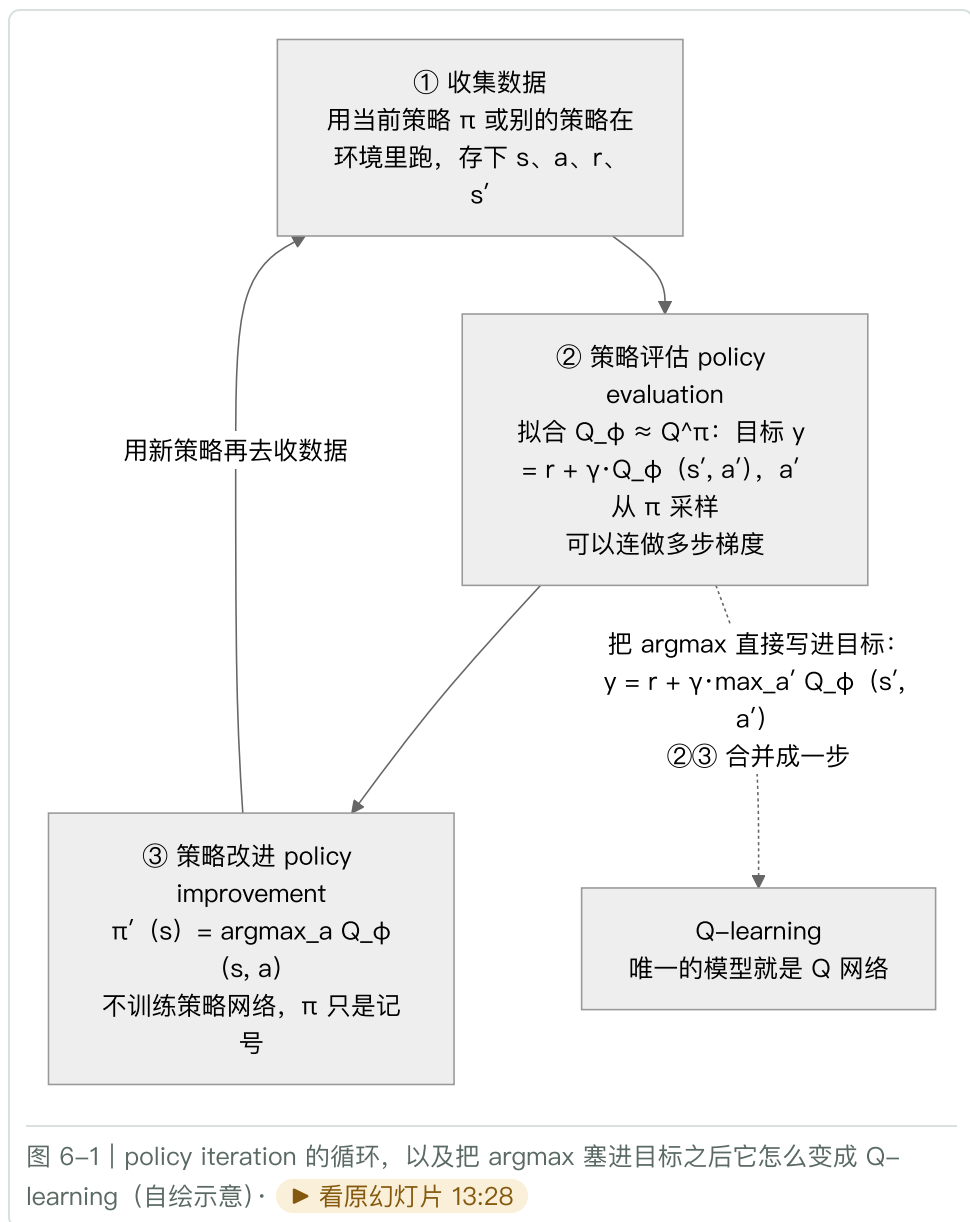
- **一般结论 (policy improvement)**：只要 Q^π 准确， π' 永远至少和 π 一样好；两者相等当且仅当 π 已经最优。理由一句话：在任何状态，"先执行 Q^π 最大的动作、之后再按 π 走"的期望回报，不会低于"按 π 随机抽一个动作再按 π 走"：

$$\max_a Q^\pi(s, a) \geq \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)] = V^\pi(s)$$

左边是新策略 π' 第一步的价值，右边是旧策略的状态价值 $V^\pi(s)$ ；最大值不小于平均值，所以哪怕之后仍按旧策略走，也已经不亏。

小注：这就是教科书里的 policy improvement theorem (Sutton & Barto)。它还能往下推一步： π' 之后每一步也都换成 $\arg\max$ ，回报只会更高，所以 $V^{\pi'} \geq V^\pi$ 对所有状态成立。

3. Policy iteration：只学 Q，不学策略网络



- 既然 argmax 总能改进，那就干脆别学策略网络了。迭代算法：① 用策略跑环境收数据；② 用这批数据拟合 Q^π （也可以像第 5 讲那样 off-policy 地用旧数据）；③ 把策略定义成 $\operatorname{argmax} Q$ ；然后拿新策略收数据、估计新策略的 Q^π ，如此往复。整个系统里唯一的模型是 Q 网络， π 只是为了说话方便的记号。
- 两个步骤的名字：② 叫 policy evaluation（策略评估：算清楚一个策略值多少），③ 叫 policy improvement（策略改进）；两者交替就叫 policy iteration。评估这一步可以连做多步梯度——它不是 on-policy 的策略梯度，只是在拟合固定目标。板书里的目标和第 5 讲一样：

$$y_i = r(s_i, a_i) + \gamma Q_\phi(s'_i, a'_i), \quad a'_i \sim \pi(\cdot | s'_i)$$

(s, a, s') 来自 buffer, a' 从当前策略采样, ϕ 是 Q 网络的参数; 然后最小化 $Q_\phi(s, a)$ 与 y 的平方误差。CME295 第 5 讲里 PPO 的 value head 估的是 V^π , 就是 policy evaluation 在 LLM 里的样子; 这里评估的对象换成了 Q。

• 问答

- 动作是连续的怎么取 max? 离散动作 (上、下、左、右) 最省事, 直接枚举。连续动作时 argmax 本身成了一个优化问题: 对动作做梯度上升原则上可行, 实际效果不好; 动作维度不高时, 随机采一批动作、挑 Q 最高的反而更好; 还有各种基于采样、逐轮细化的优化器。动作离散但空间巨大 (比如文本), 可以在 Q 值上做某种搜索。

小注: 机器人抓取里常用的"采样再细化"是 cross-entropy method (CEM), 第 10 节的 QT-Opt 就是这么取 max 的; 另一条路是再训一个网络专门输出 argmax, 那就回到第 5 讲的 actor-critic。对 LLM 来说, "在文本动作上按价值搜索"正是 CS329A 里 PRM 引导 beam search 的思路——价值模型当 Q, 搜索当 argmax (我的类比, 课上没这么说)。

- 奖励稀疏还能用吗? 能, 但要有心理准备: 每一轮改进只把"目标在哪"的信息往回传一步 (术语叫 backup), 环境越大、horizon 越长, 需要的迭代轮数就越多。
- 这个策略是确定性的吗? 是。这也是为什么后面不建议用它本身去收数据——每个状态只会出一种动作, 数据里没有多样性。

4. Q-learning: 把 max 写进目标, 逼近 Bellman 最优方程

- 由一个问答引出: 目标里的 a' 是"下一步做什么", 为什么从 π 采样, 而不是取 max 或平均? Finn: 平均 (多采几个 a') 可以, 能让目标更准; 更妙的是把改进步骤直接写进 Q 的目标里:

$$y_i = r(s_i, a_i) + \gamma \max_{a'} Q_\phi(s'_i, a')$$

同样的 (s, a, r, s') , 唯一的改动是下一步动作不再从 π 抽, 而是取让 Q_ϕ 最大的那个。用这个目标去拟合 Q, 就是 Q-learning。

• 两种理解:

1. 这些目标恰好就是"新策略" (对当前 Q 取 argmax 的那个策略) 的 Q 值——改进步骤已经内嵌在目标里, 评估的对象直接是改进后的策略。

2. 第 5 讲的递归式对任意策略都成立，当然也包括最优策略 π^* 。但最优策略还多一条性质：它在下一个状态一定选 Q 最大的动作。于是可以写一条**只对最优策略成立**的方程——把"对下一动作取期望"换成"取最大"：

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [\mathbb{E}_{a' \sim \pi(\cdot|s')} [Q^\pi(s', a')]], \quad Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [\max_{a'} Q^*(s', a')]$$

前者叫 Bellman equation，对任何策略 π 成立，是策略评估时用的；后者叫 Bellman optimality equation，只有最优 Q 函数 Q^* 满足（当且仅当）。 $\mathbb{E}_{\{s'\}}$ 是对环境转移的期望，Finn 特意说明动力学未知，这一项只能靠样本 s' 来近似。 Q -learning 的更新可以看成在努力让第二个等式的左右两边相等：等式一旦成立，取 argmax 得到的就是最优策略。

- **术语提醒：**读论文会反复遇到这两个名字；口头上有人把第二个也叫 Bellman equation。两者都有对应 V 的版本。

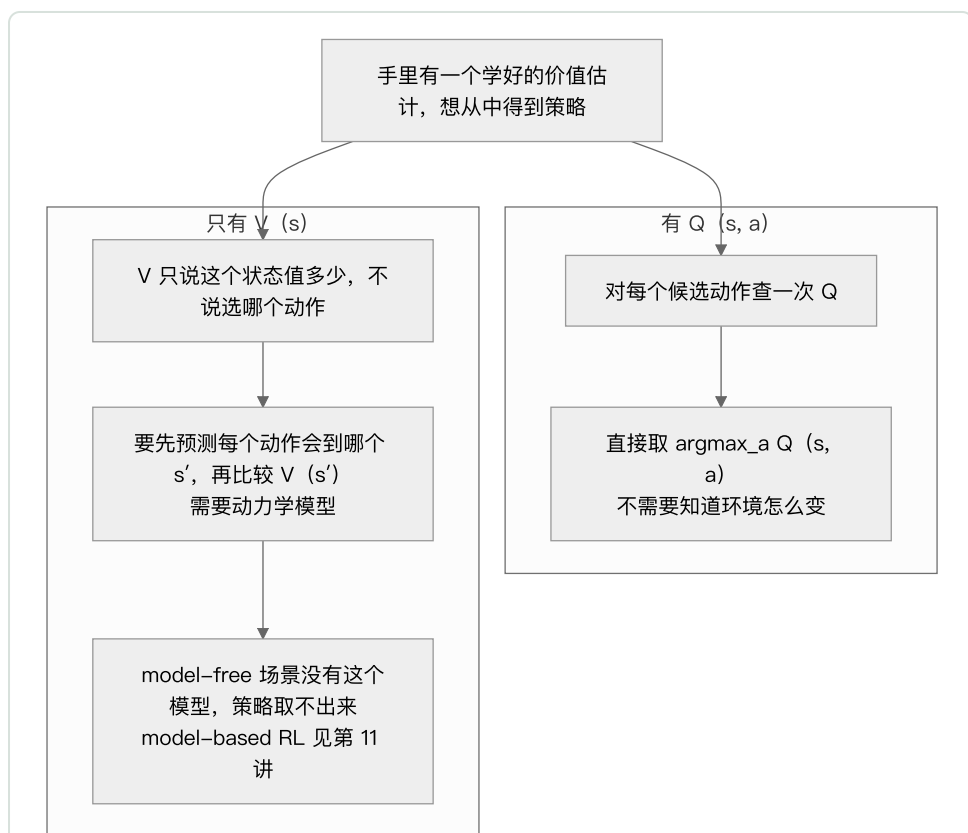


图 6-2 | 为什么是 Q 而不是 V ：从 Q 取策略只需 argmax ，从 V 取策略要预知动作会把你带到哪（自绘示意）。▶ [看原幻灯片 23:20](#)

- **问答：**为什么整堂课都在说 Q ，不说 V ？ V 的方程同样能写，但 Q 的好处是能直接取出策略：知道 Q 就能对动作 argmax 。只知道 $V(s)$ 的话，不知道哪个动作会带来更高的价值；原则上可以做前瞻——预测每个动作之后到哪个 s' ，比较 $V(s')$ ——但那需要动力学模型，而我们

是 model-free (第 11 讲的 model-based RL 才走这条路)。所以从 V 取策略、用 V 改进策略，都比用 Q 难得多。

- **会不会收敛？** 好消息：如果状态和动作空间小到能把每个 (s, a) 的 Q 存进一张表 (tabular)，而且探索充分——数据把所有状态动作都覆盖到——这个算法保证收敛到最优。坏消息：除此之外没有保证；哪怕只用线性函数逼近 Q ，也能构造出发散的例子。不过实践中它被调到非常好用，课末有例子。有同学追问“如果”指什么：不是 Q 估得多准 (Q 本来就是被这个过程拟合出来的)，而是空间大小和数据覆盖两件事。

小注：表格版 Q -learning 出自 Watkins 1989 年的博士论文，收敛证明见 Watkins & Dayan (1992)。“函数逼近 + bootstrapping + off-policy”三者同时出现就可能发散，Sutton & Barto 称之为 deadly triad—— Q -learning 三样全占，所以第 7-9 节的稳定化技巧不是锦上添花，而是必需品。

5. Q -learning 是 off-policy 的：数据该从哪来

- **为什么 off-policy**：Bellman 最优方程对所有 (s, a) 成立，所以训练数据里的动作不必来自当前策略——第 5 讲定义过，用别的策略产生的数据来学，就叫 off-policy。
- **而且数据最好比当前策略更“广”**：目标里那个 $\max$ 要把所有候选动作都比一遍（离散动作时就是全部枚举），这就要求每个动作的 Q 都估得靠谱。某个动作如果从来没在数据里出现过，它的 Q 值可能被随手估高， $\max$ 一挑就挑中它，目标就错了。所以收数据要用一个探索策略 (exploration policy)，专门比确定性的 $\arg\max$ 策略多试一些动作。
- **两种常见探索策略**：
 - **ϵ -greedy**：以概率 ϵ 均匀随机选一个动作，以概率 $1 - \epsilon$ 按 $\arg\max Q$ 行动。随机那一份保证所有动作都有覆盖。训练初期 ϵ 大、多探索，随着 Q 越估越准逐渐调小。名字的意思：大部分时间贪心 (greedy)， ϵ 的时间不贪心。
 - **Boltzmann 探索**：按 Q 值的大小按比例选动作——把每个动作的 Q 取指数（保证为正）再归一化成一个分布， Q 高的动作选得多，低的选得少。稍复杂一些。
- **问答：连续动作怎么探索？** ϵ -greedy 主要用于离散动作；连续动作要先定义一个“均匀”分布（得知道动作范围），或者用标准差很大的高

斯。连续动作上 Boltzmann 更常见，但具体怎么从这个分布采样，Finn 当场没想起细节，说课后在 Ed 上补。

小注：Boltzmann 探索就是对 Q 做 softmax，通常还带一个温度系数；Nature 版 DQN 的 ϵ 是在前一百万帧里从 1.0 线性退到 0.1。第 14 讲会系统讲探索。

6. 完整算法：replay buffer、均匀抽样、K 步更新

- **一轮 Q-learning**：① 用某个策略（实践里就是 ϵ -greedy 或 Boltzmann）在环境里收数据；② 把转移 (s, a, r, s') 加进 replay buffer；③ 从 buffer 里**均匀抽**一个 batch——刻意不用刚收回来的那几条，而是从整个 buffer 里随机抽，这样 batch 内的样本彼此不相关、近似 iid；④ 对 batch 算目标 y ，最小化 L2 loss，更新 Q；⑤ 步骤 ③④ 重复 K 次，再回到 ①。

$$\phi \leftarrow \phi - \alpha \sum_i \left(Q_\phi(s_i, a_i) - y_i \right) \nabla_\phi Q_\phi(s_i, a_i)$$

L2 loss 对 ϕ 求导，链式法则得到“误差 $\times$ Q 对参数的梯度”； α 是学习率。 y_i 虽然也含 ϕ ，求导时当常数处理，梯度不传进目标里。

- **K 的选择**：实践中最常见的是 $K = 1$ ，收一步数据、更新一步；K 越大，每批数据被榨得越干，数据效率越高。Finn 后面说 DQN 的数据效率“看细节”，指的主要就是这类设置。
- **问答：用别的策略收数据，不会比用好策略收的数据更差吗？** 会，但这是有意为之：先学出更准的 Q，后面才能得到更优的策略——前期收点次优数据、换后期更好。Finn 的比喻：让 agent 帮你挑餐馆，一开始没吃过几家，就该多试、包括可能不好的，才知道什么叫好；等 Q 估准了再收敛到少数几家。
- **训练完的产物是 Q**；部署时的策略就是对它取 argmax 的贪心策略，不再探索。
- **问答（一串关于 buffer 的）**
 - 还叫 off-policy 吗？收数据的策略和当前策略很像啊。——叫。buffer 里有当前策略的数据，也有过去所有策略的数据；off-policy 的意思不是“不用自己的数据”，而是“不只用自己的数据”。
 - buffer 大小为 1 是不是就 on-policy 了？——差不多，但只要收数据的策略（ ϵ -greedy）和被评估的策略（argmax）不同，严格说仍是 off-policy。

- 每轮重新初始化 Q 吗？——不，warm start，接着上一轮的参数继续训。
- buffer 怎么管，会不会某些状态被反复抽？——这确实重要。通常 buffer 固定容量、先进先出（FIFO），满了就丢最老的；Q-learning 的 buffer 一般很大。开头 buffer 太小，可以第一轮多收一些预填。还有异步方案：一些线程往 buffer 里写，另一些线程读 buffer 更新 Q——因为收数据和更新网络都耗时——这时更要盯紧 buffer 管理，别让某条数据被更新一千次、另一条只更新一次。

7. 实践技巧一：target network, DQN 的诞生

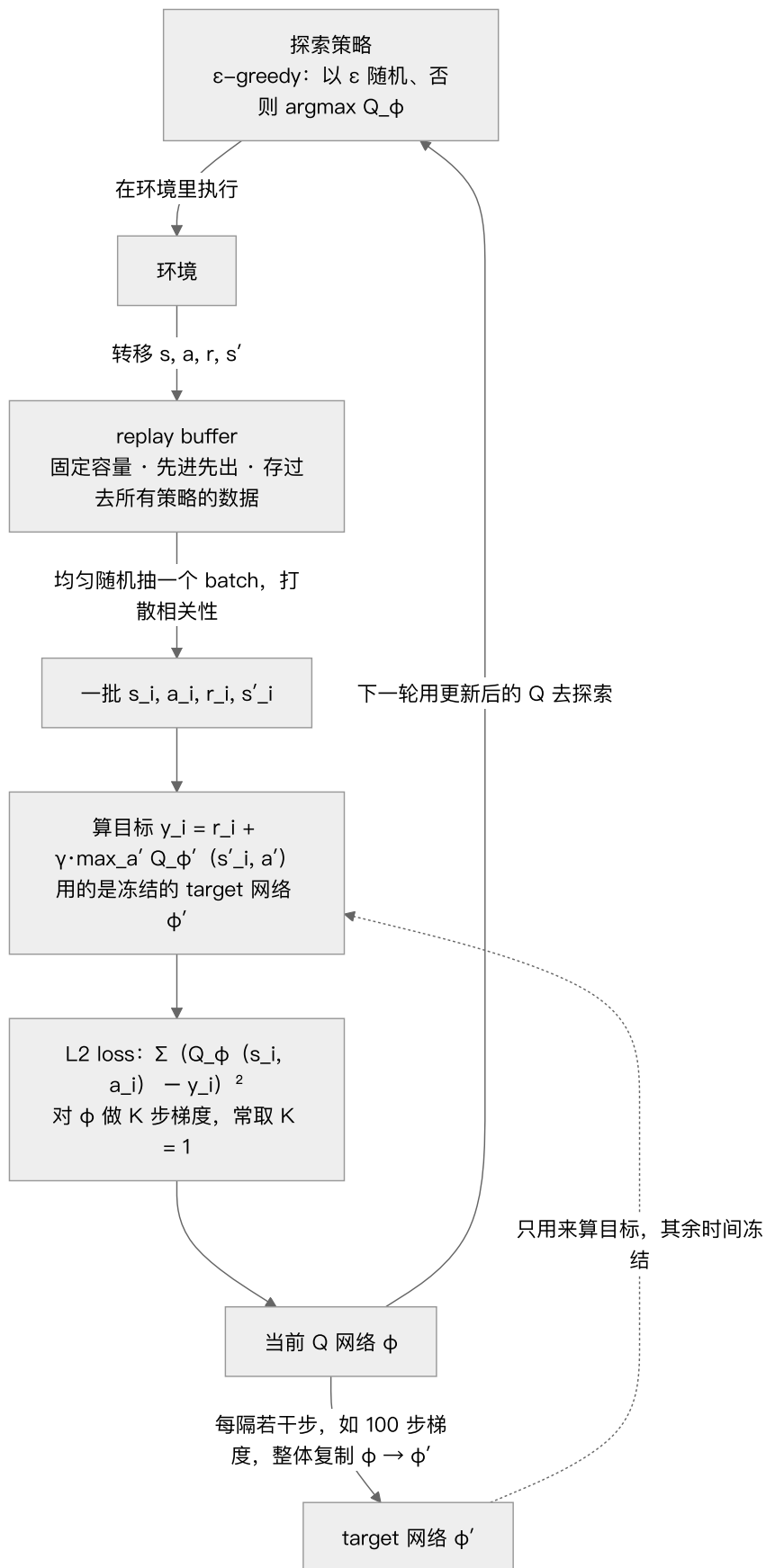


图 6-3 | DQN 的完整循环：探索策略喂 replay buffer，均匀抽 batch，用冻结的 target 网络算目标，定期整体复制（自绘示意）· [▶ 看原幻灯片 42:31](#) · 出处：Mnih et al., 2013

- **问题：**Q-learning 用神经网络时常常不稳定。原因之一是目标 $r + \gamma \cdot \max Q_\phi$ 里含有正在被更新的 ϕ ——每改一次参数，标签就跟着变（moving target），优化问题高度非平稳， $K = 1$ 时尤其明显。
- **办法：**别让标签变得那么勤。另存一份 Q 网络参数 ϕ' （target network），算目标时用它，且冻结不动；只每隔一段时间（比如 100 个梯度步）把最新的 ϕ 整体复制过去。这样内层循环的标签固定，本质上就是在做监督学习，而监督学习稳定得多。
- **DQN：**用了 target network 的 Q-learning 就是 DQN。Finn 把它算作 2013 年出现的、按她的看法第一个深度 RL 方法：直接从游戏画面像素训练神经网络玩多款 Atari 游戏，在此之前 RL 的例子多半是低维的玩具问题。作业里要自己实现它。

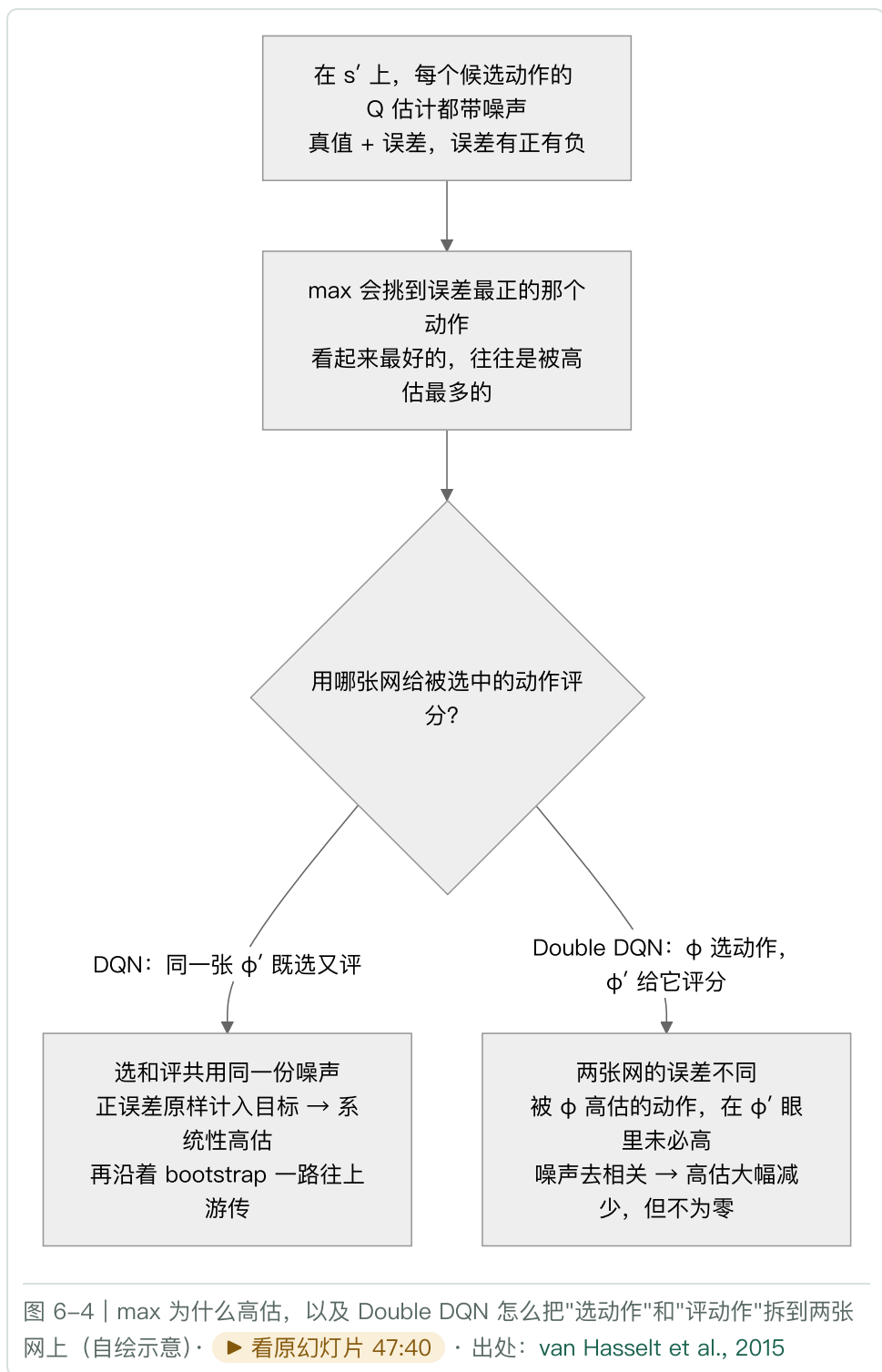
小注：2013 年那篇 workshop 论文已有 replay buffer，target network 是 2015 年 Nature 版加进去的：每 10,000 次更新同步一次 ϕ' ，buffer 容量一百万条转移。

- **问答：**能不能存两张 Q 值表，而不是两套参数？不行——循环里一直有新数据进来，新的 s' 需要现算目标，只有参数才能对没见过的状态算 Q。

8. 实践技巧二：max 会系统性高估，Double DQN

- **先看 Q 值准不准：**训练曲线上，回报在涨，Q 的估计值也在涨——符合预期，Q 就是在估平均回报。一个实用提醒：Q 的 loss 在训练中往往上升，别慌——新收的数据回报更高，Q 值的量级变大，loss 的尺度自然跟着变大；开头到处都是 0 奖励时 loss 当然小。把 Q 值画到 Pong 的画面上也像样：必须向上接球时，“上”的 Q 明显高于“下”和“不动”；球在对面时三者差不多。
- **但和真实回报一比就露馅：**把训练后策略跑出来的真实折扣回报画成一条水平线，DQN 估的值在它上面很远——估计值一直系统性地高于真值。这叫 overestimation（高估）。Double DQN 这一算法估得准得多，高估小得多。

小注：这张“估计值 vs 真值”的图出自指定阅读 van Hasselt et al., 2015；图里 DQN 的估计不仅偏高，还会随训练不稳定地跳动，Double DQN 的曲线则平且贴近真值。



- **高估从哪来：**目标里的 max 在数学上等于两步——先用 $Q_{\phi'}$ 在 s' 上挑一个 Q 最大的动作，再用同一个 $Q_{\phi'}$ 给这个动作打分。神经网络的估计总带噪声：挑动作时，噪声偏正的动作更容易被挑中；评分时用的又是同一份噪声，那份正误差就原封不动进了目标。选和评共用一份噪声，等于在系统性地"利用"噪声，目标就偏高；这个偏高又通过 bootstrapping 传给上游的 Q 。

- **Double Q-learning 的想法**：用两张不同的网络——网络 A 挑动作，网络 B 给挑出的动作评分。两张网的噪声不同，A 高估的动作在 B 眼里未必高，噪声去相关之后就不再被利用。
- **实践中怎么做 (Double DQN)**：我们本来就有两张网——当前网络 ϕ 和定期冻结的 target 网络 ϕ' 。于是：用当前网络 ϕ 选动作，用 target 网络 ϕ' 评分：

$$y_{\text{DQN}} = r + \gamma Q_{\phi'}(s', \arg \max_{a'} Q_{\phi'}(s', a')), \quad y_{\text{Double}} = r + \gamma Q_{\phi}(s', \arg \max_{a'} Q_{\phi}(s', a'))$$

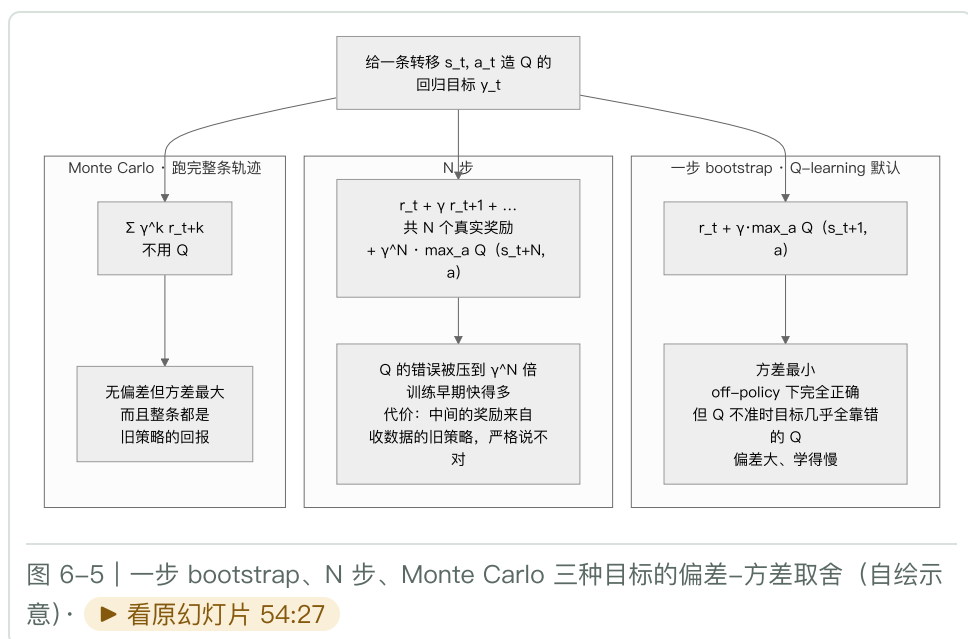
第一行是 DQN 的目标，把 max 显式写成“用 ϕ' 选、再用 ϕ' 评”；第二行只把 argmax 里面的下标换成 ϕ ——不多一张网，也不多一次前向。

• 问答

- 能保证两张网噪声不同吗？——不能保证；有些噪声来自网络结构本身，两张网都有。但实践中效果很好，高估大幅减少——仍然不为零，图里 Double DQN 的曲线还是略高于真值。
- ϕ' 只是 ϕ 的旧版本，为什么噪声就不一样了？—— ϕ' 是过时的， ϕ 自那之后又吃了很多梯度更新和新数据；噪声不只来自初始化，也来自“做了哪些更新”，两者的误差模式确实不同。
- 三张、四张网会更好吗？——Finn 觉得值得一试：如果噪声来源是随机的，更多独立样本可能有帮助；如果来自结构这类固定因素，就未必。

小注：原始 Double Q-learning (van Hasselt, 2010) 是表格版，维护两张独立的 Q 表、每次随机更新其中一张；Double DQN 的贡献正是“不加网络，直接借用 target network 当第二个估计器”。连续动作的 actor-critic 里同样的高估问题由 TD3 的 clipped double Q (两个 critic 取 min) 处理，SAC 也沿用了这个做法。

9. 实践技巧三：N-step 目标



- **动机：**目标 $r + \gamma \cdot \max Q$ 里， Q 准的时候 $\max Q$ 那一项承担了大部分信息； Q 不准的时候（训练初期）真正可靠的只有那一个 r ，而单步奖励对“这个 (s, a) 到底值多少”说明不了太多，学得慢。
- **回忆第 4 讲：**估回报有三种做法：Monte Carlo 把整条轨迹的奖励加起来（无偏但方差大）；一步 bootstrap 用 $r + V(s')$ （方差小，偏差来自 V 的错误）；N-step 折中——先加 N 步真实奖励，第 N 步之后接价值估计。搬到 Q 上：

$$y_t = \sum_{k=0}^{N-1} \gamma^k r_{t+k} + \gamma^N \max_a Q_{\phi'}(s_{t+N}, a)$$

前 N 步用 buffer 里记录的真实奖励，第 N 步之后的状态 $s_{\{t+N\}}$ 上再取 $\max Q$ 来 bootstrap； Q 的错误被乘上 γ^N ，影响变小。

- **有同学一眼看出问题：**off-policy 时这 N 个奖励来自当初收数据的那个策略，不是我们想评估的最优策略——中间那几步它可能走了别的动作。 $N = 1$ 时没这个问题，因为 r 只取决于 (s, a) 。所以 N-step 目标在 Q-learning 里**严格说是错的**：训练早期更少依赖坏 Q 、学得快得多，代价是估的不完全是 Q^* 。
- **怎么办：**
 1. 最常见、也最不讲道理的答案——当它不存在，照用 N 大于 1。尽管不正确，N-step 在实践里非常经常地显著提升效果。
 2. 动态选 N ：只用和当前策略一致的那一段数据——看记录里的动作是否等于当前 Q 的 $\arg\max$ ，一致就往下数，不一致就在那里截断。只在动作空间小、数据基本 on-policy 时值得做。

3. 训练后期把 N 调小—— Q 准了之后一步目标就够。

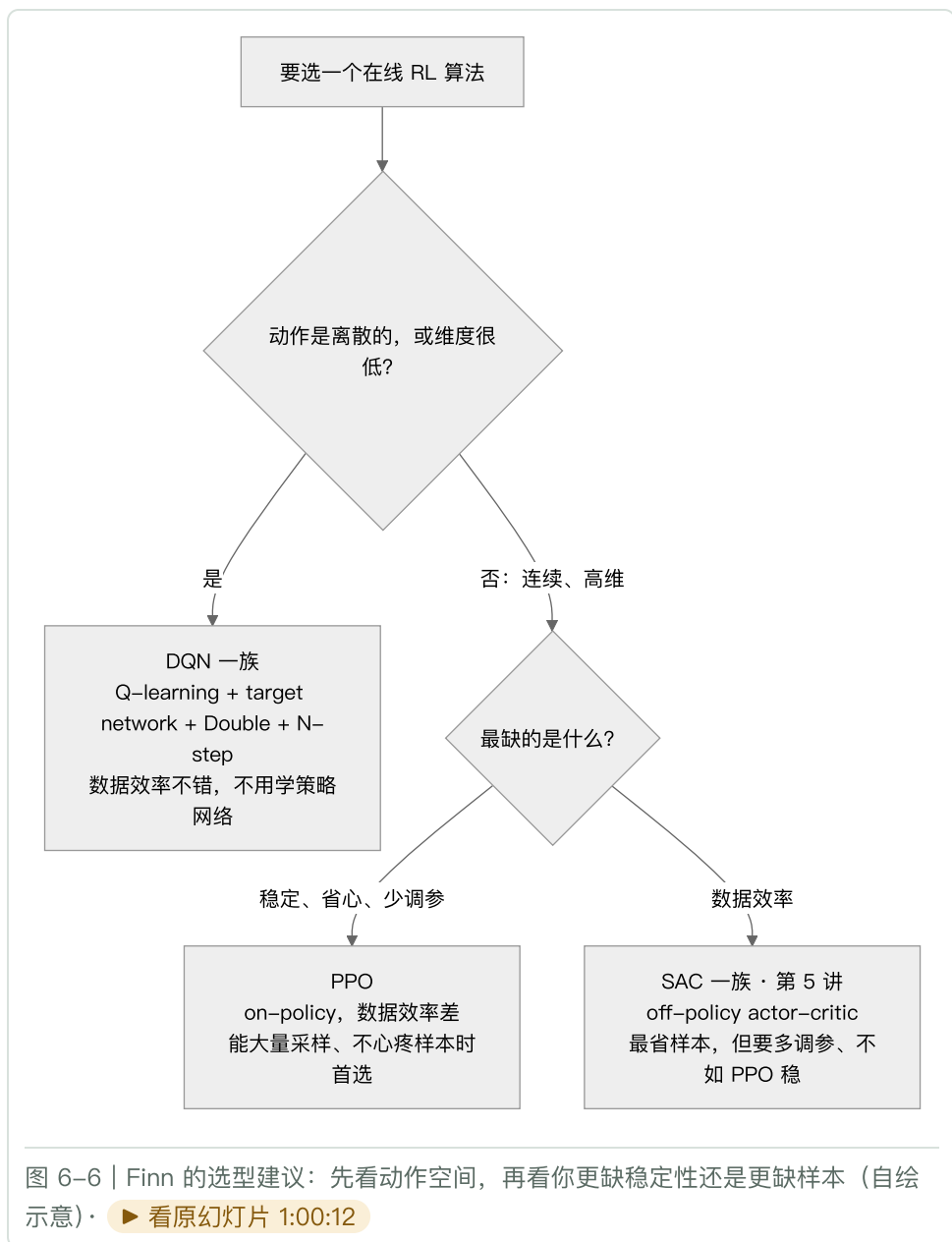
4. 用 importance sampling 给这些奖励加权，修正策略不一致。

小注：Rainbow (Hessel et al., 2018) 的消融里， $N = 3$ 的多步目标是对 DQN 提升最大的组件之一，用的就是“照用”的第 1 种做法；带 importance-sampling 修正并截断权重的版本是 Retrace (Munos et al., 2016)。CME295 第 5 讲的 GAE，本质是对不同 N 的 N -step 估计做指数加权平均。

10. 成功案例、选型建议与总结

- **Atari**：Q-learning 在很多 Atari 游戏上超过大多数人类玩家；幻灯片上那张长条形图，横线以上是达到或超过人类水平的游戏，以下是不如人类的。
- **机器人抓取**：一个连续动作的例子—— Q 函数由 Q-learning 训练，行动时对动作做连续优化来近似 argmax ；抓取成功率约 96%。上一讲说过 RL 能给出成功率更高、更可靠的策略，这里又是一例；而且需要先绕着物体调整位姿再抓时，它也能自己学会——整个过程被当成一个序贯决策问题在优化。

小注：应为 QT-Opt (Kalashnikov et al., 2018)：用 7 台机械臂收集了约 58 万次真实抓取，用 CEM 在动作空间里取 max，在没见过的物体上 96% 成功率。



- **三种在线算法怎么选**（这是最后一个在线 RL 方法，Finn 顺便给建议）：
 - **PPO**：最看重稳定性和易用性时选它，基本不用怎么调参；但数据效率很差，适合能大量采样、不心疼样本的场景。CME295 第 5 讲和 CS329A 里 LLM 的 RL 用它，正是因为对 LLM 来说采样只是推理，而稳定性更值钱。
 - **DQN**：动作离散或低维时很好，数据效率可以相当不错，取决于实现细节（例如 K）。
 - **SAC 及其变体**：最在乎数据效率时选它，但要多调参，稳定性不如 PPO。

- **总结：**Q-learning 与它在实践中的三个技巧——target network、Double DQN、N-step 目标；两个术语——Bellman equation 与 Bellman optimality equation；核心思想——不学显式策略也能做 RL。在线 RL 算法到此告一段落，第 7 讲进入 offline RL：手里只有一份固定数据集、不能再交互时怎么办——那是 off-policy 推到极限的情形，本讲"数据要覆盖 max 会考虑的动作"这条警告会变成主角。

小注：指定阅读的第二篇（Bellemare et al., 2017，算法名 C51）课上没有讲。它的想法是把 $Q(s, a)$ 这一个标量换成"回报的整个分布"来学：把回报离散成 51 个支点，用分布版的 Bellman 算子做目标，再用交叉熵拟合。"平均 10 分"和"一半 0 分、一半 20 分"在标量 Q 下无法区分，分布式 RL 能。读它时可以对照第 4 节：Bellman 最优方程作用的对象从一个数变成一个分布。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
2D 导航思考题	蓝框起点、星星目标、"永远向右"的策略；按行标出各动作的 Q^{π} 是 1 还是 0	5:50	—
迭代算法三步	收数据 $\rightarrow$ 拟合 $Q^{\pi} \rightarrow \pi$ 定义为 argmax ；注意没有策略网络	13:28	—
Q-learning 目标	与 policy iteration 板书的唯一区别： a' 的采样换成 $\max$	20:43	—
Bellman equation 与 Bellman optimality equation	两条方程并排，差别只在"对 a' 取期望"还是"取 $\max$ "	26:29	—
收敛性的好消息与坏消息	表格 + 充分探索能收敛；线性函数逼近也可能发散	27:34	—
ϵ -greedy 与 Boltzmann	两种探索策略的写法； ϵ 随训练退火	30:06	—
完整 Q-learning 算法	收数据 $\rightarrow$ 进 buffer $\rightarrow$ 均匀抽 batch $\rightarrow$ 梯度更新 $\times K$ ；梯度式里是误差乘 ∇Q	33:49	—
target network / DQN 算法	目标里用冻结的 ϕ' ；内层循环成了监督学习；定期复制	41:31	Mnih et al., 2013
回报与 Q 值的训练曲线	两者一起上升；顺带记住 loss 上升不必慌	44:33	Mnih et al., 2013（推断）
Pong 画面上的 Q 值	该向上时"上"的 Q 明显高；球在对面时三个动作差不多	45:36	Mnih et al., 2015（推断）

图	看什么	跳转	出处
估计值 vs 真实回报	红色水平线是真值，DQN 的估计远在其上；Double DQN 贴近得多	46:37	van Hasselt et al., 2015
高估的拆解	max 写成"用同一张网选、再用同一张网评"；Double DQN 只改 argmax 里的下标	47:40	同上
N-step 目标板书	N 个真实奖励加 γ^N 倍的 max Q；随后列出 off-policy 偏差的应对	54:58	—
Atari 条形图与机器人抓取	横线以上超过人类；抓取是连续动作的 Q-learning，96% 成功率	58:05	Mnih et al., 2015 · 应为 QT-Opt

提到的工作

名称	在本讲里的作用
policy gradient（第 3 讲）、actor-critic（第 4 讲）、off-policy actor-critic（第 5 讲）	开场回顾的三种循环；Q-learning 从第 5 讲的 Q 目标改一个 max 得来
policy iteration（policy evaluation + policy improvement）	本讲第一个算法：只学 Q，策略是 argmax；Q-learning 是它的合并版
Bellman equation / Bellman optimality equation	策略评估与最优性的两条方程；Q-learning 在逼近后者
Q-learning（应出自 Watkins, 1989；课上未点名）	主线算法
ϵ -greedy、Boltzmann 探索	收数据用的两种探索策略
replay buffer	off-policy 数据的容器：均匀抽样去相关、FIFO、预填、异步读写
DQN（Mnih et al., 2013；Nature 版 2015）	target network + replay buffer 的 Q-learning；Atari 从像素学起；作业要实现
Double DQN（van Hasselt et al., 2015，指定阅读）	高估的诊断图与"当前网选、target 网评"的修法
Double Q-learning（van Hasselt, 2010；推断）	两张网选评分离的原始想法
N-step return（第 4 讲已用于 actor-critic）	第三个技巧；off-policy 偏差与几种应对
importance sampling	N-step 偏差的修正手段之一，课上只提名字
Atari 基准	DQN 超过人类的成功案例
机器人抓取（应为 QT-Opt，Kalashnikov et al., 2018）	连续动作 Q-learning 的成功案例，96% 成功率

名称	在本讲里的作用
PPO、SAC	结尾选型建议里的另外两种在线算法
Distributional RL / C51 (Bellemare et al., 2017, 指定阅读)	课上没讲；见第 10 节末尾的小注
Ed (课程论坛)、作业	Boltzmann 连续采样的细节课后补；DQN 是作业内容

术语对照

English	中文
value function $V^\pi(s)$	状态价值函数：从 s 出发按 π 行动的期望回报
Q function / action-value $Q^\pi(s, a)$	动作价值函数：先做 a 、再按 π 行动的期望回报
value-based RL	基于价值的 RL：只学价值函数，不学显式策略
greedy policy / argmax policy	贪心策略：每个状态选 Q 最大的动作
deterministic policy	确定性策略：每个状态只出一个动作
policy evaluation	策略评估：估计给定策略的 V 或 Q
policy improvement	策略改进：对 Q 取 argmax 得到更好的策略
policy iteration	策略迭代：评估与改进交替
Bellman equation	Bellman 方程： Q^π 的递归式，对 a' 取期望
Bellman optimality equation	Bellman 最优方程： Q^* 的递归式，对 a' 取 max
optimal policy π^* / Q^*	最优策略与最优 Q 函数
discount factor γ	折扣因子：越远的奖励权重越小
bootstrapping	自举：用自己的估计当回归目标
backup	回传：把下游的价值信息传回上游状态一步
horizon	决策步数的长度；稀疏奖励下要迭代的轮数与之相当
tabular	表格法：每个 (s, a) 单独存一个 Q 值
function approximation	函数逼近：用神经网络等参数化模型表示 Q
convergence / divergence	收敛 / 发散
off-policy	用其他策略（含过去的自己）产生的数据训练
coverage	覆盖：数据是否包含 max 会考虑的所有动作
exploration policy	探索策略：为收数据而故意多试动作的策略
ϵ -greedy	ϵ 概率随机、其余贪心
Boltzmann exploration	玻尔兹曼探索：按 $\exp(Q)$ 归一化后的概率选动作
anneal ϵ	退火：训练中逐步调小 ϵ

English	中文
replay buffer	经验回放池：存过去转移，随机抽样训练
transition (s, a, r, s')	一条转移记录
i.i.d. / decorrelate	独立同分布 / 去相关：均匀抽样打散相邻样本的相关性
FIFO	先进先出：buffer 满了丢最老的
warm start	热启动：接着上一轮参数继续训
moving target / non-stationary	移动的目标 / 非平稳：标签随参数一起变
target network ϕ'	目标网络：冻结的 Q 参数副本，只用来算目标
DQN (Deep Q-Network)	深度 Q 网络
overestimation	高估：估计值系统性高于真实回报
Double DQN	双 DQN：当前网络选动作、目标网络评分
N-step return	N 步回报：N 个真实奖励加第 N 步后的价值估计
Monte Carlo return	蒙特卡洛回报：整条轨迹奖励之和，不用价值估计
bias / variance	偏差 / 方差
importance sampling	重要性采样：按策略概率比给样本加权以修正分布差异
data efficiency	数据效率：达到同样效果需要多少环境样本
SAC (Soft Actor-Critic)	软 actor-critic：off-policy 的连续动作算法

字幕勘误

自动字幕把 "Q" 听成了各种词，通读时先把它们都换回来："p value / t value / key value / key function / two function / tube function / tree function / cube function / C function" → Q value、Q function；"qpi / qi / 2 pi / Q pi" → Q^π ；"the 2" → the Q。其他："value functions entry functions" → value functions and Q functions；"active criticisms / actritic" → actor-critic；"last semester" → last lecture；"GQM / DQ / DQM" → DQN；"Belman / solving equation / solvality equation / sol equation" → Bellman equation、Bellman optimality equation；"optional policy / optical policy" → optimal policy；"F1 3D policy" → ϵ -greedy policy；"Bolton distribution" → Boltzmann distribution；"five star / five prime / U5 prime" → ϕ' （target 网络参数）；"double D" → Double DQN；"Tucka" → 应为 Seaquest（推断）；"mstep / ends up / nstep returns" → N-step returns；"bonner" → Monte Carlo；"bootstropping" →

bootstrapping; "important sampling" → importance sampling; "PTO / SAT / soft critic" → PPO、SAC、Soft Actor–Critic; "homeless" → homework; "edge" → Ed; "deorrelating" → decorrelating。

带走的问题

1. 第 2 节的 2D 例子里，从"永远向右"出发，policy iteration 要迭代多少轮才能让起点也走对？把奖励换成"到星星的负距离"为什么一轮就够？这对你给 agent 设计奖励有什么提示？
2. Q-learning 的目标对任意 (s, a, r, s') 成立，所以能用任何来源的数据。那第 7 讲的 offline RL 直接拿一份固定数据集跑 Q-learning，会在哪一步出问题？提示：max 会考虑数据里没出现过的动作。
3. Double DQN 只把 $\arg\max$ 里的下标从 ϕ' 换成 ϕ 。如果反过来——用 ϕ' 选动作、用 ϕ 评分——高估会减轻吗？为什么论文选前者？
4. N-step 目标在 off-policy 下"严格说是错的"，但实践里几乎总是有用。你能说清它错在哪、又为什么有用吗？如果 buffer 里的数据来自一个非常差的旧策略，N 该取大还是取小？
5. 把 Q-learning 搬到 LLM：状态是 prompt 加已生成的 token，动作是下一个 token，Q 是"接下来这么写最终得分的期望"。max 要遍历整个词表，target network 和 replay buffer 还需要吗？哪些困难是 PPO / GRPO (CME295 第 5 讲、CS329A) 没有的？

CS224R 复习课 | Q-learning 回顾 (Tutorial Session: Review of Q-Learning)

Stanford CS224R: Deep Reinforcement Learning (2025 春, 主讲 Chelsea Finn) · 课程表上的 "Extra section on Q-learning" 助教复习课, 2025 年 4 月 18 日, 排在第 6 讲之后 视频: <https://www.youtube.com/watch?v=07MQNMcxhZU> (50:39, 英文字幕为自动生成, 人名、缩写错得多, 见文末勘误) 讲者: 助教 Anikait (开场自报姓名, 字幕写作 "Anakiette"; 应为 Chelsea Finn 组的博士生 Anikait Singh, 推断) · 不是 Chelsea Finn, 也不是客座讲座 课程主页: <https://cs224r.stanford.edu/> · 本讲无指定阅读; 理论推导见第 6 讲笔记; 助教发的 PDF 附有可以自己改 grid world 的 Colab (课上提到, 链接未给)

一句话: Q-learning 的全部内容是一条自洽关系——一个状态-动作对的价值, 等于当下奖励加上 γ 倍"下一状态里最好动作的价值"。表格环境里把这条关系当更新规则反复套, 价值就从终止格一圈圈往外传, argmax 就是最优策略; 换成神经网络后, 同一条关系变成回归问题, 用一步 TD 目标换低方差、付出偏差。复习课后半段全是让这个回归稳定下来的工程手段: stop-gradient 与 target network、gradient clipping 与 Huber loss、按 transition 存的 replay buffer, 以及针对 $\max$ 算子带来的系统性高估的 ensemble 取 $\min$ 。

时间	内容
0:06	开场: 助教自我介绍; 四段提纲——MDP 回顾、表格型 Q iteration、参数化 Q-learning 的 TD 与 MC 取舍、实现细节
1:11	MDP 复习: 状态、动作、转移动力学、奖励、初始状态、折扣 γ 、horizon; grid world 里的机器人与宝石
2:41	三个量: V^π 、 Q^π 、advantage 各自回答什么问题
4:44	Bellman 最优方程改写成迭代更新; grid world 里价值从终止格往外传 (6:16 起逐轮演示)
7:47	问答: Bellman 方程与迭代式的区别; "Q 出现在等号两边就是 DP"的判别法; Q_0 怎么初始化
11:21	最优 V 、 Q 、 π 三图对照: $\gamma = 0.9$ 时价值按步数衰减; 策略就是 argmax
12:55	问答: 只学策略行不行——policy gradient 的方差与 Q 函数的偏差
14:28	advantage 在 grid world 里的样子: 最优策略下处处不大于 0; 次优策略的 Q^π 与 critic 的用法 (16:30)

18:02	表格法为何不可扩展；V 网络、连续 Q 网络、离散 Q 网络三种参数化
21:38	白板推导：从 Monte Carlo 回报拆出一步 TD 目标（改编自 Sutton & Barto）
25:54	MC 与 TD 的偏差-方差；n-step return；两张图与一张三行表（29:29）；语言场景与机器人场景的差别
32:06	Stitching：一步 DP 能把两段轨迹拼成最优路径
33:08	实现细节一、二：stop-gradient 与 target network（白板写 TD 回归的 loss，35:45）；gradient clipping 与 Huber loss（38:18）
39:49	实现细节三：replay buffer 为什么按 transition 存；问答：带历史的 critic（41:53）
42:54	过估计：max 算子把零均值噪声变成正偏差，并在 DP 里层层叠加
46:58	Double DQN 论文里的图；critic ensemble 取 min；在线两个够用、离线要更多（SAC-N、EDAC）；语言奖励模型的类比

核心内容

1. 这节课在整门课里的位置

- **定位：**排在第 6 讲之后的助教复习课，目标不是推新理论，而是把 Q-learning 从表格例子一路走到训得动的神经网络版本。助教的提纲有四段：MDP 记号、表格环境里的 fitted Q iteration、参数化 Q-learning 里 TD 与 Monte Carlo 的取舍、让训练稳定的实现细节。理论推导见第 6 讲，这里只保留跟着例子走所需的部分。
- **MDP 的词汇**（第 1 讲定义过）：agent 在环境里一步步行动。一个 MDP（Markov decision process）由状态集合 S 、动作集合 A 、转移动力学 $p(s' | s, a)$ （在 s 做 a 之后落到 s' 的概率）、奖励 $r(s, a)$ 、初始状态分布、折扣因子 γ 和 horizon H （一条轨迹的长度）组成。一串状态-动作叫轨迹 τ ；把轨迹上的奖励按 γ^k 折扣后加总叫回报（return）。目标是最大化期望回报。表格环境里，策略 π 就是"每个格子该走哪一步"的一张表。
- **贯穿全课的例子：**网格里的小机器人要走到右上角的宝石格，中途有一格岩浆。

2. V、Q、advantage：三个量各回答什么问题

- **$V^\pi(s)$** (value function, 价值函数)：站在状态 s 、之后一直按策略 π 行动，期望还能拿到多少回报。它只看状态。
- **$Q^\pi(s, a)$** (Q function)：先在 s 执行动作 a ，之后再按 π 行动，期望还能拿多少。比 V 多一个可以"试探"的自由度——同一个状态下比

较不同动作。

- **两者的关系**：对动作按策略取期望， Q 就变回 V 。
- **advantage $A^\pi(s, a)$ (优势)**：这个动作比"按策略的平均水平"好多少。CME295 第 5 讲 PPO 里减 baseline 用的就是它，这里是它在 MDP 里的原始定义。

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^\pi(s, a)], \quad A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

V^π 是 Q^π 对策略 π 选出的动作取的平均； A^π 是某个具体动作的 Q 减去这个平均，为正说明这个动作比策略的平均表现好。

3. 表格型 Q iteration：价值从终止格往外传

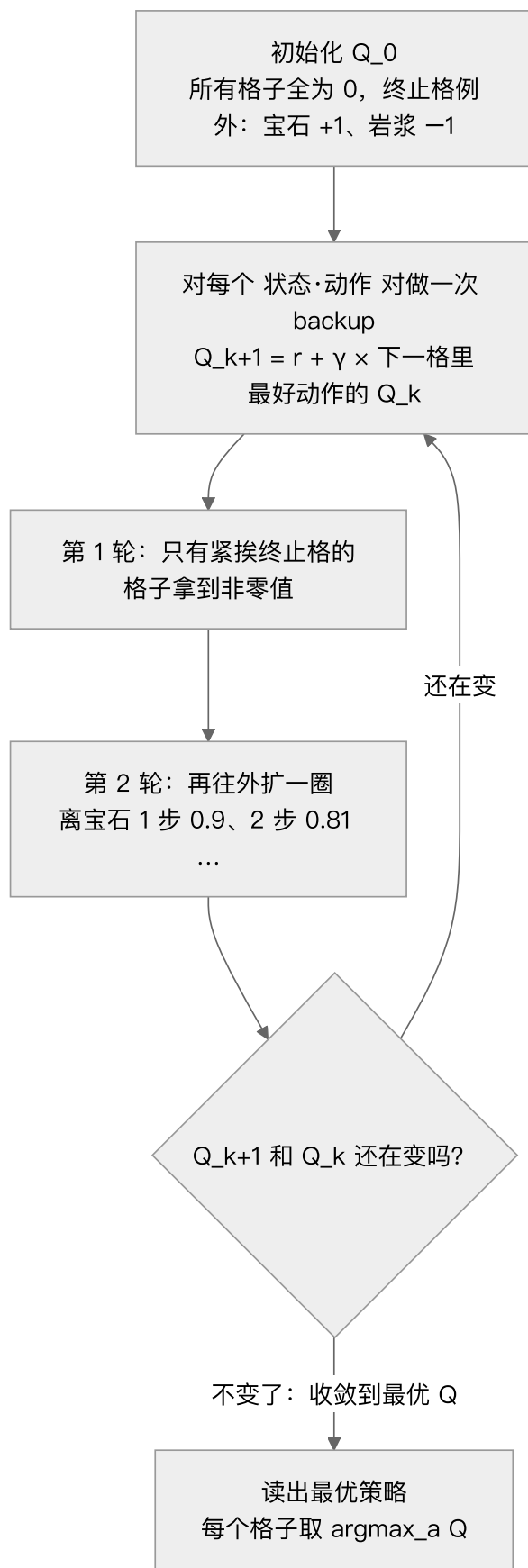


图 T-1 | 表格型 Q iteration：把 Bellman 关系当更新规则反复套，价值从终止格一圈圈向外传播（自绘示意）· [▶ 看原幻灯片 6:16](#) · 出处：Sutton & Barto, 2018

- **Bellman 最优方程**说的是一条自洽关系：最优 Q 在 (s, a) 上的值，等于当下奖励加上 γ 倍"下一状态里最好动作的最优 Q "，对转移的随机性取期望。它没有任何"第几轮"的下标，只是最优解必须满足的性质。
- **变成算法**：给等号两边加上轮次下标——右边用第 k 轮的 Q_k 算，得到第 $k+1$ 轮的 Q_{k+1} ——就是 Q iteration，一种动态规划 (dynamic programming, DP)。

$$Q^*(s, a) = \mathbb{E}_{s' \sim p(\cdot|s, a)} \left[r(s, a) + \gamma \max_{a'} Q^*(s', a') \right] \implies Q_{k+1}(s, a) \leftarrow \mathbb{E}_{s' \sim p(\cdot|s, a)} \left[r(s, a) + \gamma \max_{a'} Q_k(s', a') \right]$$

左边是 Bellman 最优方程： Q^* 是"当下奖励加 γ 倍下一状态最优动作价值"这个关系的不动点；右边把同一关系写成第 k 轮到第 $k+1$ 轮的更新， s' 按转移动力学 p 取期望，表格里就是对可能的下一格加权求和。

- **grid world 走一遍**：五个动作（原地、左、右、上、下）；走进右上角宝石格得 +1，走进中间偏上的岩浆格得 -1，两者都是终止格； γ 小于 1 让机器人尽快到达、别乱逛。 Q_0 在所有格子都是 0，只有终止格例外——在那里不管做什么动作都立刻拿到 +1 或 -1。第 1 轮更新后，只有紧挨宝石的格子拿到非零值；第 2 轮再往外扩一圈；如此反复，整张网格被填满， Q 收敛。
- **问答里的三个澄清**
 - Bellman 方程和迭代式的区别：前者无下标、只是最优解的性质；后者带轮次下标，是从终止格出发反复套用它的过程。
 - 怎么一眼分辨 DP 和 Monte Carlo：更新式的等号两边都出现 Q ——用自己的估计更新自己，术语叫 bootstrapping——就是 DP 风格；只出现奖励的，是 Monte Carlo。
 - Q_0 不是随手编的：表格设定里约定全零初始化， Q_1 就是套一次更新的结果。

小注：这个迭代为什么一定收敛、收敛到的为什么是 Q^* ，靠的是 Bellman 算子在 γ 小于 1 时是压缩映射，推导见第 6 讲。

4. 最优 V 、 Q 、 π 三图对照，以及 advantage 长什么样

- 最优价值 V^* ： $\gamma = 0.9$ 时，宝石格 +1，离它一步的格子 0.9，两步 0.81，依此衰减；岩浆格 -1。
- 最优 Q^* ：既可以由上面的迭代收敛得到，也可以解析算出。
- 最优策略 π^* ：在完全可观测的 MDP 里最优策略是确定性的，就是每个格子对 Q^* 取 argmax ；画出来是从任意格子到宝石的最短路径。

- **问答：只学策略不就够了？** 表格里策略正是从 Q 的 argmax 里读出来的，没有 Q 就没有全局的“哪里值多少”。只学策略的方法（第 3 讲的 policy gradient）也存在，但奖励或动力学一有噪声，方差就上来了； Q 函数是用一点偏差换掉这些方差。哪种更好取决于环境。
- **advantage 图：** 在最优策略下 advantage 处处小于等于 0——最优动作为 0（并列最优也是 0，例如左下角格子往上和往右一样好），其余为负，朝岩浆走的动作负得最多。
- **次优策略：** 换一个随机的策略 π ， Q^π 和 Q^* 明显不同，策略甚至会走进岩浆。但正因为 Q^π 是“对着当前策略”算的，它能告诉你哪些动作比现在做的更好——这就是第 4 讲 actor-critic 里 critic 的用法：拿 Q^π 做一步策略改进。 Q^* 已经收敛，对应的 π^* 没有改进空间。

5. 从表格到神经网络：三种参数化

- **表格为什么不可扩展：** 状态或动作一多，表就有成千上万甚至上百万行，学不动，也不会泛化。用神经网络参数化 Q ，相近的状态（语义上或字面上相近）自然得到相近的 Q 值——这种平滑性是表格没有的。
- **三种网络形状**
 - V 网络：输入 s ，输出一个标量。 V 不依赖动作，所以只有这一种参数化。
 - 连续动作的 Q 网络：输入 (s, a) ，输出一个标量。动作没法穷举，要对动作取期望时只能从策略里采样，得到的是采样估计。
 - 离散动作的 Q 网络：输入 s ，输出一个向量，每个动作一个 Q 值（grid world 就是 5 维）。可以精确地对所有动作取期望或取 max；只要动作集合可枚举，就推荐这种。

6. 白板推导：从 Monte Carlo 回报到一步 TD

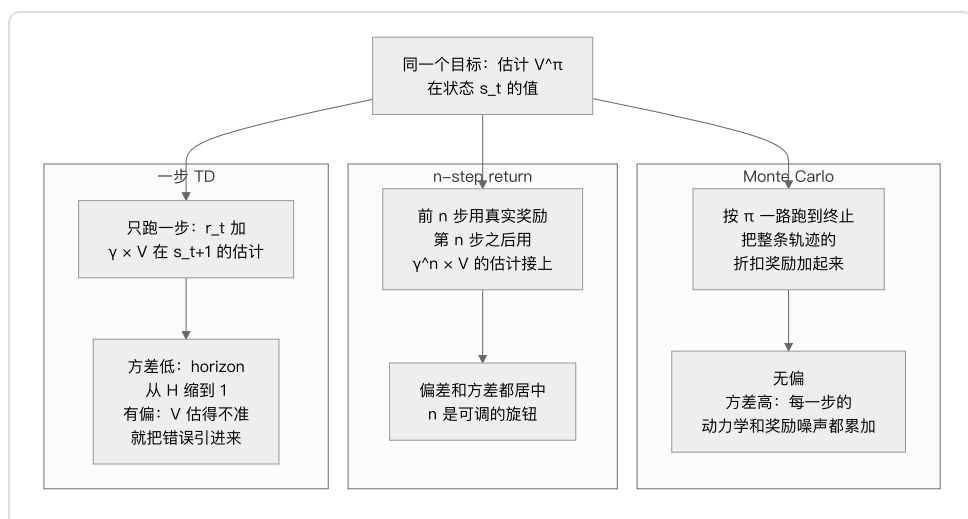


图 T-2 | 同一个价值，三种估法：真实奖励用几步、估计值从哪一步接上（自绘示意）· [▶ 看原幻灯片 29:29](#) · 出处：Sutton & Barto, 2018

- **起点：** $V^\pi(s)$ 是从 s 出发、按 π 行动的期望回报，回报展开成折扣奖励之和。直接按定义估计——跑完整条轨迹把奖励加起来、多跑几条取平均——就是 Monte Carlo (MC) 估计。
- **拆一步：**把 $k = 0$ 那一项（当下奖励）单独拎出来，剩下的求和整体提出一个 γ ；剩下那部分正好是“从下一个状态出发的回报的期望”，也就是 $V^\pi(s_{t+1})$ 。于是 $V^\pi(s)$ 等于 r_t 加 $\gamma V^\pi(s_{t+1})$ 的期望。这就是一步 TD (temporal difference) 目标：真实奖励只用一步，后面全部交给自己的估计。
- **拆 n 步：**同样的展开做 n 次，前 n 步用真实奖励、第 n 步之后用 $\gamma^n V^\pi(s_{t+n})$ 接上，就是 n -step return。助教说 Q 函数的推导完全平行。

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}) \mid s_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{n-1} \gamma^k r(s_{t+k}, a_{t+k}) + \gamma^n V^\pi(s_{t+n}) \mid s_t = s \right]$$

左边是 MC 的定义：整条轨迹的折扣奖励之和；右边是 n -step 形式，前 n 步真实奖励、之后用 V^π 的估计代替。 $n = 1$ 是一步 TD， n 取到整条轨迹就回到 MC。

- **偏差—方差：**MC 无偏，但每一步的动力学噪声和奖励噪声都会累加，horizon 越长方差越大；TD 把 horizon 缩到 1，方差大幅下降，代价是把 V 的估计误差引进来（有偏）。 n 是两者之间的旋钮。
- **问答：**表格设定下 MC 和 TD 收敛到同一个答案；用了函数逼近之后未必。
- **环境决定选哪个：**语言场景里动力学几乎确定、数学和代码的奖励可以精确验证，所以大家直接用 MC rollout——CS329A 里的 GRPO 就

是用组内 MC 回报做基线，第 10 讲会回到这里；换到 agent 或机器人场景，噪声回来了，TD 的取舍才变得重要。

小注：CME295 第 5 讲提到的 GAE，就是把所有 n 的 n-step 估计按 λ 的幂加权平均，用一个连续旋钮代替选定某个 n。

	一步 TD	n-step	Monte Carlo
bootstrap 深度	1 步	n 步	整条 episode
偏差	高	居中，随 n 变	无
方差	低	居中	高

7. Stitching：DP 独有的"拼接"能力

- 迷宫里策略只走过两段轨迹：A 到 B、C 到 D。因为每次 backup 只看前一步，DP 能把两段拼起来，找到从 A 到 D 的最优路径——MC 做不到，它只能评价完整走过的轨迹。这让 DP 类算法对状态空间的覆盖更充分、样本效率更高，也是第 7 讲离线 RL 看重 Q-learning 的原因之一。

8. 实现细节一：stop-gradient 与 target network

- **问题**：目标 $r + \gamma \max Q(s', a')$ 里的 Q 和被更新的 Q 是同一个网络。如果让梯度穿过目标，等于追着一个自己会动的靶子，优化非常难。
- **semi-gradient**：目标照算，但不往目标里回传梯度 (stop-gradient)。参数还是同一套，只是目标被当成常数。
- **target network**：更进一步，目标用一套单独的、滞后的参数 w' 。两种更新法：hard——每隔固定步数把 w 整体复制给 w' ；soft——每一步按 Polyak / EMA 平滑， ρ 通常取 0.99 上下。两种经验上都行，助教觉得现在更多人用 soft；更新频率是要调的超参数。
- **白板上的 loss**：一步 TD 目标 y 等于 $r + \gamma V(s')$ ，loss 是 $V_w(s)$ 与 y 之差的平方。等号两边都是可学习网络时非常不稳定；stop-gradient 让右边固定、只有左边收梯度。

$$y = r + \gamma \max_{a'} Q_{w'}(s', a'), \quad \mathcal{L}(w) = (Q_w(s, a) - \text{sg}[y])^2, \quad w' \leftarrow \rho w' + (1 - \rho) w$$

y 是用 target network 参数 w' 算出的一步 TD 目标，sg 表示不回传梯度；loss 只对当前网络 Q_w 求导；最后一式是 soft 更新， ρ 越接近 1， w' 跟得越慢、靶子越稳。

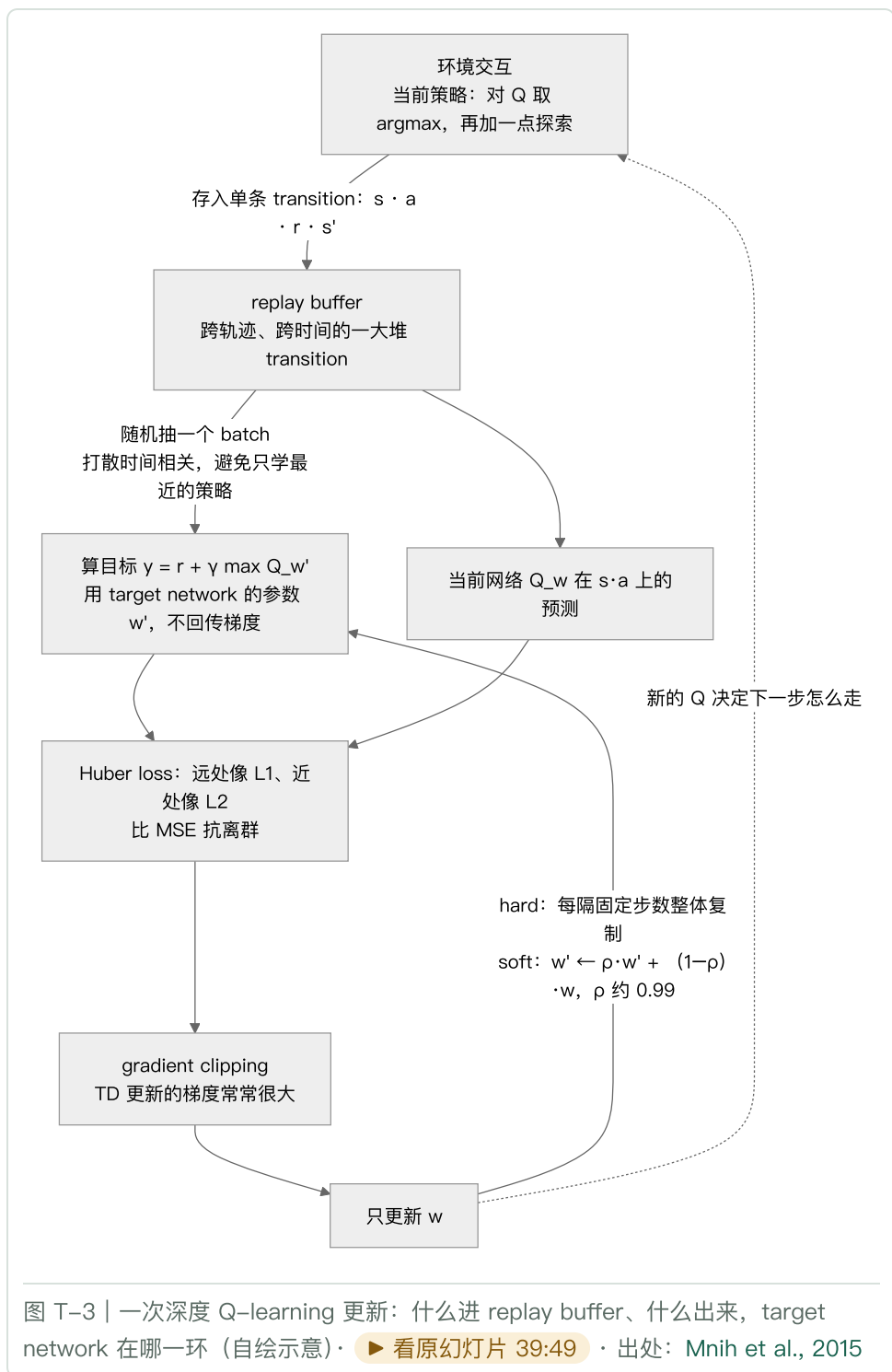
小注：hard 与 soft 两种做法分别由 DQN (Mnih et al., 2015, Nature 版每 10,000 步复制一次) 和 DDPG (Lillicrap et al., 2015, $\tau = 0.001$, 相当于 $\rho = 0.999$) 带起来；SAC 常用 $\tau = 0.005$ 。

9. 实现细节二：gradient clipping 与 Huber loss

- **gradient clipping**: TD 更新常常冒出很大的梯度；截断之后参数能平滑地逼近最优点，而不是在它周围打转（助教用一张 loss 等高线图对比了有无截断的轨迹）。
- **Huber loss**: 把 L1 和 L2 拼在一起——离目标远时像 L1，不被离群样本带偏；靠近目标后切换成 L2，精细收敛。助教说这是训 value function 的常用配置，不是唯一选择。

小注：这两招在 Nature 版 DQN 里就是一套：把 TD 误差截断到 -1 与 1 之间，等价于 Huber loss。

10. 实现细节三：replay buffer



- **是什么：**一块存过往经验的内存。关键是按单条 transition (s, a, r, s') 存，而不是按整条轨迹存。第 5 讲讲 off-policy actor-critic 时它已经出现过：训练用的数据不必来自当前策略，这正是 Q-learning 能 off-policy 的原因。
- **两个好处**
 - 打散时间相关：一个 batch 可以从整个数据集、任意轨迹的任意位置抽，相邻样本不再高度相关。

- 避免 recency bias: 不只学最近一个策略的数据。Q 函数受益于对状态空间的广覆盖, 被当前策略窄化分布反而坏事。
- 合起来就是样本效率更高, 经验可以反复用。
- 容量: buffer 通常有上限; 真受限时可以重新加权或丢掉一部分, 多数时候不必操心。
- 问答: critic 能不能带历史? 可以, RNN 或 transformer 都有人做, Decision Transformer 就是用长上下文同时预测动作和价值的例子。语言模型的策略本来就条件在整段历史上; 机器人数据时间相关性强, 带历史要更小心。

11. 过估计: max 把噪声变成系统性偏差

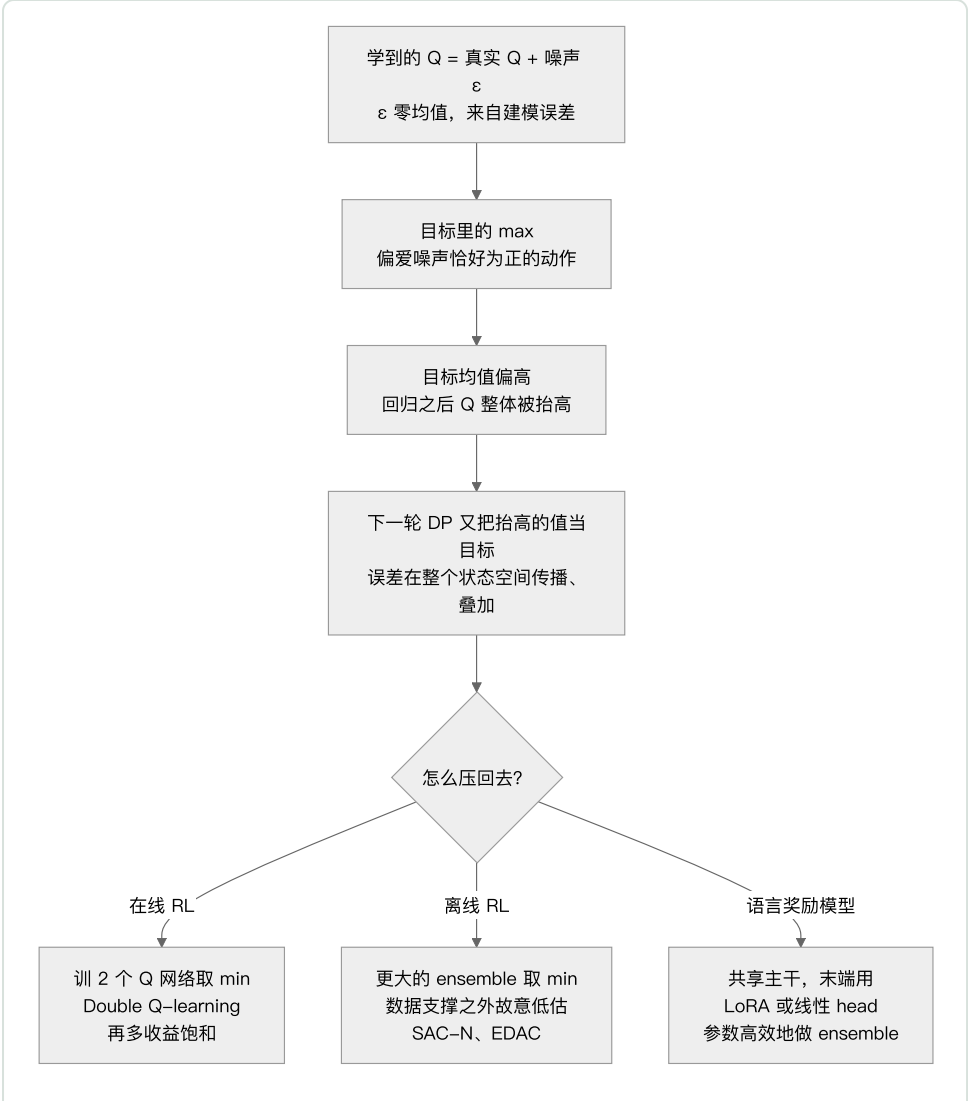


图 T-4 | 过估计怎么产生、怎么在 DP 里滚雪球, 以及三种场景下的 ensemble 解法 (自绘示意) · [▶ 看原幻灯片 46:58](#) · 出处: van Hasselt et al., 2016

- **推导思路**（助教说是改自函数逼近下的 RL 分析）：把学到的 Q 写成真实 Q 加噪声 ε , ε 来自建模误差、均值为 0。目标里对下一步动作取 $\max$, 会系统性地挑中噪声恰好为正的那个动作——所以即使每个动作的噪声零均值, "max 之后的噪声"均值为正。回归到这个偏高的目标, Q 整体被抬高；下一轮 DP 又拿抬高的值当目标, 误差在整个状态空间传播、叠加, 最后 Q 值爆掉。
- **助教的自我纠正**：问题不在"只看了部分动作"——离散场景下动作是全部枚举的——而在 Q 本身的建模误差。
- **论文里的图**（应出自 Double DQN 论文）：一列画学到的估计和真实值, 一列画所有动作的估计和它们的 $\max$, 一列画估计与真实值的差；标准 Q -learning 的误差曲线很大, 两套网络的 double Q -learning 把它压下去。

$$\mathbb{E}_{\varepsilon} \left[\max_{a'} (Q(s', a') + \varepsilon_{a'}) \right] \geq \max_{a'} Q(s', a') \text{ even if } \mathbb{E}[\varepsilon_{a'}] = 0, \quad y_{\text{ens}} = r + \gamma \min_{i \in \{1, \dots, N\}} \max_{a'} Q_{w_i}(s', a')$$

左式是过估计的根源：对"真值加零均值噪声"取 $\max$, 期望不小于真值的 $\max$, 多出来的部分就是正偏差；右式是解法, N 个独立初始化的 target 网络各算一个目标, 取最小的那个当 y 。

小注：这个"零均值噪声经 $\max$ 变成正偏差"的分析最早应出自 Thrun & Schwartz, 1993；表格版 Double Q -learning 是 van Hasselt, 2010, Double DQN 把它搬到神经网络上。

12. 解法：critic ensemble 取 $\min$

- **做法**：训 N 个 Q 网络, 初始化不同, 各自在不同状态上犯错；对它们取 $\min$, 得到一个偏保守的估计——用可控的低估换掉失控的高估。这本质上是在估计 Q 的 epistemic uncertainty, 即"模型不知道自己不知道"的那部分。
- **N 取多大**：在线 RL 里 $N = 2$ (double Q -learning) 就够, 再多收益饱和。离线 RL 面对的是静态数据集, 一旦查询到数据支撑之外的状态-动作, 误差传播得更凶, 所以要更大的 ensemble, SAC- N 、EDAC 是代表 (第 7 讲)。
- **语言里的类比**：奖励模型也会遇到类似的高估——CME295 第 5 讲的 reward hacking 就是奖励模型被钻空子。参数高效的做法是共享大部分层, 只在末端用 LoRA 或线性 adapter 做 ensemble。

小注：An et al., 2021 (NeurIPS) 里, SAC- N 只是把 clipped double Q 的 N 从 2 加大到几十上百；EDAC 加了一个让各网络梯度方向分散的正则项, 把需要的网络数降到约十分之一。"取 $\min$ "的具体写法各家不同：

Double DQN 用一套网络选动作、另一套评估；TD3 和 SAC 的 clipped double Q 是在选定动作上对两个 critic 取 min。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
MDP 框图与 grid world	agent 与环境的回路上每个符号是什么；右上角的宝石格就是本讲所有例子的目标	1:11	—
Q iteration 逐轮演示	终止格先有值，每一轮多传一圈，直到整张网 格填满	6:16	Sutton & Barto, 2018
最优 V、Q、 π 三图	0.9、0.81 的衰减；每格五个方向的 Q；箭头就是最短路径	11:21	—
advantage 网络	最优动作为 0、其余为负；红色的是朝岩浆走 的动作	14:28	—
三种网络参数化	V 网络、连续 Q 网络、离散 Q 网络各自的输 入输出	19:03	—
MC rollout 对比 TD backup	左边多条完整轨迹取平均，右边只展开一步； 下面的三行表	29:29	—
Stitching 迷宫	A 到 B、C 到 D 两段轨迹怎么拼成 A 到 D	32:06	—
过估计三列图	标准 Q-learning 的误差曲线与 double Q- learning 的对比	46:58	应为 Double DQN 图 2

提到的工作

名称	在本讲里的作用
Sutton & Barto, Reinforcement Learning: An Introduction	白板上 MC 到 TD 推导的出处
Bellman 最优方程、动态规划	表格型 Q iteration 的依据；"Q 在等号两边"是 DP 的判别法
Fitted Q iteration (Ernst et al., 2005)	提纲里表格环境那一段用的算法名，课上按 Q iteration 讲
Policy gradient (第 3 讲)	对照：只学策略的方法，噪声环境下方差大
Actor-critic (第 4 讲)	次优策略的 Q^π 作为 critic 做一步策略改进
n-step return	TD 与 MC 之间的旋钮
Target network、Polyak / EMA 更新	稳定 TD 回归的靶子；应出自 DQN 与 DDPG
Huber loss、gradient clipping	让 TD 回归对离群值和大梯度更稳

名称	在本讲里的作用
Replay buffer（第 5 讲）	按 transition 存经验；打散时间相关、避免 recency bias
Decision Transformer（Chen et al., 2021）	问答里"带历史的 critic 或 policy"的例子
Double DQN（van Hasselt et al., 2016）	过估计的图示； $N = 2$ 的 ensemble 解法
SAC-N、EDAC（An et al., 2021）	离线 RL 里更大的 Q ensemble
LoRA（Hu et al., 2021）	语言奖励模型做参数高效 ensemble 的手段
离线 RL（第 7 讲）	stitching 与大 ensemble 的应用场景预告
助教 PDF 与 Colab	可以自己改的表格环境

术语对照

English	中文
MDP (Markov decision process)	马尔可夫决策过程：状态、动作、转移、奖励、初始分布、折扣、horizon 的打包
state / action	状态 / 动作
transition dynamics $p(s' s, a)$	转移动力学：做完动作落到哪个状态的概率
reward $r(s, a)$	奖励
discount factor γ	折扣因子：越远的奖励打的折越多，也鼓励尽快到达目标
horizon H	一条轨迹的长度
trajectory / rollout	轨迹 / 把策略跑一遍得到的轨迹
return / reward-to-go	回报：从当前时刻起折扣奖励之和
policy π	策略：每个状态下选什么动作（表格里就是一张查表）
value function $V^\pi(s)$	价值函数：从 s 出发按 π 行动的期望回报
Q function $Q^\pi(s, a)$	动作价值函数：先做 a 再按 π 行动的期望回报
advantage $A^\pi(s, a)$	优势：某个动作比策略平均水平好多少
optimal $V^* / Q^* / \pi^*$	最优价值 / 最优 Q / 最优策略
Bellman equation	Bellman 方程：价值必须满足的自洽关系
dynamic programming (DP)	动态规划：反复套用 Bellman 关系
(fitted) Q iteration	用 Bellman 关系迭代更新 Q；fitted 指用函数逼近器去拟合
bootstrapping	自举：用自己的估计去更新自己
Monte Carlo (MC) estimate	蒙特卡洛估计：跑完整条轨迹把奖励加起来

English	中文
TD (temporal difference)	时序差分：一步真实奖励加下一状态的估计
n-step return	n 步回报：前 n 步真实奖励，之后接估计
bias—variance trade-off	偏差—方差权衡
tabular / parametric	表格型 / 参数化（用神经网络等函数逼近器）
function approximation	函数逼近
terminal state	终止状态：到此结束，宝石格和岩浆格
stitching	拼接：DP 能把两段不完整轨迹拼出更好的路径
semi-gradient / stop-gradient	半梯度 / 截断梯度：目标当常数，不回传
target network	目标网络：一套滞后的参数专门算 TD 目标
hard / soft target update	硬更新（定期整体复制） / 软更新（每步平滑）
Polyak averaging / EMA	指数滑动平均：w' 按 ρ 向 w 缓慢靠拢
gradient clipping	梯度截断
Huber loss	远处 L1、近处 L2 的损失，抗离群值
replay buffer	经验回放池：按 transition 存过往经验
transition	一条 (s, a, r, s') 记录
temporal correlation / recency bias	时间相关性 / 近因偏差：只学最近策略的数据
sample efficiency	样本效率
on-policy / off-policy	用当前策略自己产生的数据训练 / 用别处或过去的的数据训练
overestimation	过估计：Q 被系统性地高估
epistemic uncertainty	认知不确定性：模型因为没见过而不确定的部分
critic ensemble	多个 Q 网络的集成
Double Q-learning	两套 Q 网络互相制约，压过估计
offline RL	离线 RL：只有静态数据集，不能再交互
support / distribution shift	数据支撑 / 分布偏移：查询到数据没覆盖的状态-动作

字幕勘误

"Anakiette" → Anikait（推断）；"marov" → Markov；"Bman""Bowman""Belman" → Bellman；"sutin embarto" → Sutton & Barto；"key value""cube functions""Q store""QS star" → Q value、Q functions、Q*；"polyac""poliac" → Polyak；"endstep""nups return" → n-step return；"vias" →

bias; "edac""sacken" → EDAC、SAC-N; "Laura" → LoRA; "LA state""U level state" → lava state; "grid roles""good world" → grid world; "policy grading" → policy gradient; "aentic" → agentic; "MSE stall loss" → MSE-style loss; "auto grading clipping" → gradient clipping; "heristic" → heuristic; "tupless" → tuples; "collab" → Colab; 噪声项的 "Y parameter" 应为 ϵ 。

带走的问题

1. 表格设定下 TD 和 MC 收敛到同一个答案，换成神经网络就未必。bootstrapping 的偏差和函数逼近的误差是怎么互相放大的？把 n 从 1 调到整条轨迹，你会按什么信号来选？
2. 过估计的推导只用了“噪声零均值加一个 max”。policy gradient 和 actor-critic 里用 V 做 baseline 时没有 max，是不是就没有这个问题？那第 4 讲的 critic 有没有别的系统性偏差？
3. ensemble 取 min 是用“低估”换“稳定”。离线 RL 里对数据支撑之外的动作低估是好事；在线探索时，低估会不会反过来压制探索？为什么在线 $N = 2$ 够用、离线要更多？
4. 语言场景里奖励可验证、动力学确定，MC 就够，GRPO 的组内基线就是 MC。如果你的 agent 任务奖励噪声大、轨迹很长（多轮工具调用），你会把 TD 的哪些技巧搬过来——target network、Huber、replay buffer？哪些和 LLM 的规模或 on-policy 训练不兼容？
5. Stitching 是 DP 相对 MC 独有的能力。你手里的用户会话日志里，有没有“两段各自不完整的轨迹能拼出一条更好路径”的情形？这需要日志以什么粒度存？

CS224R 第 7 讲 | 离线 RL (Offline RL)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 7 讲, 2025 年 4 月 23 日 · 在线 RL 算法讲完之后的第一讲: 手里只有一份固定数据集、不能再和环境交互时怎么办 视频: <https://www.youtube.com/watch?v=IRDaXnPlzks> (1:07:50, 英文字幕为自动生成, "Q" 常被听成 key、"KL" 被听成 kale, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: Conservative Q-Learning for Offline Reinforcement Learning (Kumar et al., 2020; 课上没有讲到它, 见第 12 节的小注)、Offline Reinforcement Learning with Implicit Q-Learning (Kostrikov et al., 2021; 课程页面误链了另一篇, 以这篇为准)

一句话: 在线 RL 每换一个策略都要重新和环境交互收数据, 而很多场景 (自动驾驶、医疗、机器人) 要么交互不安全, 要么数据早就躺在那里; 离线 RL 就是只用一份由未知的行为策略 π_β 收集的固定数据集训练策略、全程不再交互。直接把第 5 讲的 off-policy actor-critic 搬过来会翻车: Q 在数据里没出现过的动作上是随机数, 策略更新专挑那些被随手估高的动作, 离线时又没有新数据来纠正, 虚高的 Q 被 bootstrap 一路放大。本讲的解法一条路走到底——策略只在数据里出现过的动作上被监督: 过滤式行为克隆是基线; AWR 用 $\exp(\text{advantage})$ 给模仿目标加权, 能把两条半好的轨迹拼接起来; AWAC 让目标里的 a' 也从数据里取, 好做 bootstrap; IQL 再用 expectile regression 把价值函数从 " π_β 的平均结局" 抬到 "数据支撑内更好策略" 的水平, 从头到尾不查询任何数据外的动作。指定阅读里的 CQL 走的是另一条路 (把数据外动作的 Q 压低), 课上没讲到。

时间轴

时间	内容
0:06	回顾在线 RL: 策略梯度、价值函数、actor-critic (PPO、SAC)
1:08	拟合价值函数的三种标签: Monte Carlo、TD bootstrap、用 Q 吃旧策略的数据; 最后一种要求动作有覆盖
7:14	完整的 off-policy actor-critic 循环; 今天的三个目标; 问答: 这和 Q-learning 差在哪
9:50	在线 vs 离线; 为什么要 offline RL: 已有数据、在线交互不安全 (驾驶、医疗)、实验数据反复用; 离线在线混用; 问答: 离线是不是 off-policy 的特例
14:03	形式化: 数据集、行为策略 π_β (可以是混合策略)、目标函数; 分布偏移只来自策略; 问答: 数据够全会不会没有偏移; 数据从哪来

时间	内容
19:45	把 SAC 直接跑在离线数据上会怎样：板书 a_1 – a_6 ，随机初始化的 Q 在数据外是任意数，策略专挑虚高的动作；幻灯片版；问答：离散动作、能不能排除 OOD 动作
26:26	缓解高估：模仿学习天然只监督数据里的动作，但超不过数据；轨迹拼接 $s_1 \rightarrow s_3$ 加 $s_7 \rightarrow s_9$
29:58	基线：过滤式 BC——只模仿回报靠前的轨迹；不能拼接，但一定要先跑
32:34	用 $\exp(\text{advantage})$ 给模仿目标加权；它近似一个 KL 约束目标；问答：状态不必精确相交、温度、与 actor–critic 的差别
39:44	先估行为策略的 advantage；版本一：Monte Carlo 拟合 V ， $A = \text{回报} - V$ ，这就是 AWR；问答：GAE、大数据集方差小
46:05	同桌讨论：行为策略是确定性的会学到什么——advantage 全 0、退化成本 BC；AWR 完整算法与优缺点；+10 / +1 / -5 的例子
52:16	想 bootstrap 又不查数据外动作： a' 也从数据里取——AWAC；估的仍是 π_β 的价值
53:48	估一个比 π_β 更好的策略的价值：同一状态的结局有分布，L2 拟合的是均值，改拟合更高的分位；不对称 loss 与 expectile regression， λ 的含义
1:01:12	IQL：expectile 拟合 V 、MSE 拟合 Q （目标用 V ）、 $A = Q - V$ 、AWR 抽策略；问答：会不会高估、 λ 怎么选、要不要随训练调；优点
1:05:18	命名 implicit Q–learning；问答：为什么要 Q 不直接 Monte Carlo；总结：选型建议、offline–to–online、IDQL

核心内容

1. 回顾：拟合价值函数的三种标签，最后一种埋着今天的雷

- **先把记号压一遍**：智能体在时刻 t 处于状态 s_t ，按策略 $\pi_\theta(a | s)$ （给定状态输出动作的分布， θ 是网络参数）选动作 a_t ，环境给奖励 r_t 并跳到 s_{t+1} ；一整段状态、动作、奖励交替的序列叫轨迹 τ ，轨迹上奖励的总和叫回报，折扣因子 γ 让远处的奖励权重更小。价值函数 $V^\pi(s)$ 是从 s 出发一直按 π 行动的期望回报， $Q^\pi(s, a)$ 是先做 a 、之后再按 π 行动的期望回报，advantage $A^\pi = Q^\pi - V^\pi$ 是“做 a 比按 π 平均做好多少”（第 4 讲第 2 节）。用当前策略自己刚采的数据训练叫 on-policy，用别的策略（含过去的自己）的数据训练叫 off-policy，存旧数据的容器叫 replay buffer（第 5 讲）。
- **在线 RL 的三条线**（第 3–6 讲）：策略梯度——用当前策略采一批轨迹，抬高带来高于平均回报的动作的概率；actor–critic——先拟合一个价值函数（估“这个状态或动作接下来能拿多少奖励”），再拿它告诉

策略哪些动作该多做，PPO 和 SAC 都是这个套路；Q-learning——连策略网络都不要，只学 Q。

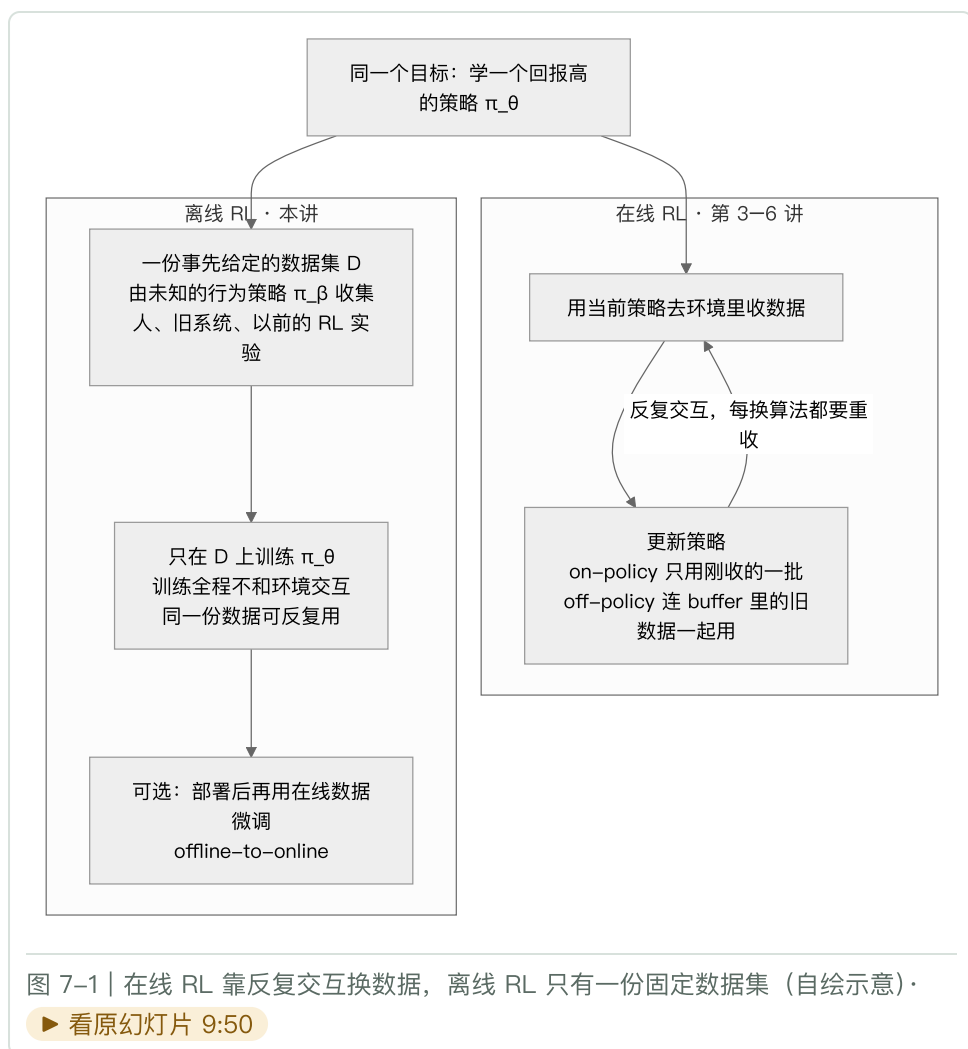
- **拟合价值函数的三种标签**（讲者特意重放一遍，因为今天全靠它们）。都是同一个平方误差回归，换的只是标签：
 1. **Monte Carlo**：标签是这条轨迹从 t 起实际拿到的奖励之和。最简单。
 2. **bootstrap / TD**：标签是“这一步的奖励，加上自己对下一状态的估值 $V(s')$ ”。用自己的估计当标签叫 bootstrapping，也叫 temporal-difference 更新。
 3. 前两种都假设数据是被评估的那个策略自己采的（on-policy）。数据若来自一堆旧策略的混合，改拟合 $Q(s, a)$ ：(s, a, s') 从数据里取，下一步动作 a' 却从正在学的策略里现采。第 5 讲第 10 节推过：Q 的递归式对任何 (s, a) 都成立，所以数据里的动作是谁选的无所谓。

$$y_t^{\text{MC}} = \sum_{t' \geq t} r_{t'}, \quad y_t^{\text{TD}} = r_t + \gamma V_\phi(s_{t+1}), \quad y_t^{\text{off}} = r_t + \gamma \hat{Q}_\phi(s_{t+1}, a'), \quad a' \sim \pi_\theta(\cdot | s_{t+1})$$

三个标签喂给同一个回归目标 $\min_\phi \sum (\text{估值} - y)^2$ ； ϕ 是价值网络的参数。第三个式子里，只有 a' 不来自数据。

- **第三种标签的隐患**：标签里用的是自己对 (s', a') 的估计，所以 Q 必须在“和将来要评估的动作长得像”的动作上训练过。正在学的策略如果和收数据的旧策略差得远，标签就不准，Q 就学不准——它需要数据对动作有好的覆盖（coverage）。第 5 讲的问答里这只是个提醒，今天它是主角。
- **完整的 off-policy actor-critic**（第 5 讲图 5-6）：策略去环境里收数据 → 存进装着过去所有策略数据的 replay buffer → 抽一批 → 用上面的标签更新 Q → 更新策略，抬高 Q 值高的动作的概率 → 循环。
- **问答：这是 Q-learning 吗？** 不是，这是第 5 讲的东西（SAC 一类）。Q-learning（第 6 讲）再改一处：目标里不从 π_θ 采 a' ，而是对 a' 取 max，学的是最优策略的 Q。它同样能 off-policy，前提也是覆盖够好。
- **今天的三个目标**：offline RL 的核心难点是什么；两类方法以及它们为什么行；offline RL 凭什么比同样从离线数据学的模仿学习更好。

2. 什么是 offline RL，为什么要它



- **定义：**在线 RL 的流程是"收数据 → 更新策略（用最新一批或至今全部数据）→ 再收"；offline RL 则是数据集事先给定，只在这份数据上训练策略，训练过程中不再收集任何新数据。
- **三个理由**
 1. **数据已经有了，而在线试错不安全。**自动驾驶：人类驾驶的数据海量，而把一个还不会开车的策略放上路去收数据显然不行。
 2. **医疗：**用 RL 决定治疗方案，讲者提到一个癫痫发作预防的项目——有现成系统的数据，就不必让一个糟糕的策略去病人身上试。
 3. **实验数据可以反复用**（更微妙的一点）：跑过几次在线 RL 之后，手里已经有一堆策略跑出来的数据，每次改算法都从头收太浪费——训机器人抓取时，第一次跑的算法几乎不会是最终版本。所以在线数据比离线数据贵：离线数据可以用很多遍。
- **混着用：**先用离线数据训出一个还不错的策略，再上线（比如带安全员）用在线数据继续改进；部署之后还能持续变好，是很多真实场景的刚需。这条线课末会回来（第 12 节）。

- **问答：**offline RL 是不是"off-policy RL 但不知道数据来自谁"的特例？ 有几分道理，但纯 offline 有一条硬约束——完全不能再收数据——所以它不算在线 off-policy RL 的特例；算法层面确实会借用 off-policy 的技巧，因为那些技巧本来就是为"用旧策略的数据"设计的。

小注：第 2 讲第 9 节区分过 offline / online（学习过程中要不要交互）和 on-policy / off-policy（训练数据是不是当前策略生成的）——offline 必然 off-policy，反之不然。癫痫那个项目应为 Pineau 组的 Guez et al., 2008（用批量 RL 学电刺激策略），课上没给出处。

3. 形式化：行为策略 π_β 与分布偏移

- **数据集：**D 是一堆轨迹，比如很多人、很多地方、很多辆车的驾驶记录。我们不知道产生它的策略是什么（多个人开的，就是多个策略），但数学上可以假设它是由一个**行为策略**（behavior policy） π_β 跑出来的；多个策略的情况把 π_β 看成它们的混合（按权重平均），不损失一般性。数据背后于是有一个状态分布、一个动作分布、环境的动力学和奖励函数。
- **目标：**学一个策略 π_θ ，最大化它自己跑出来的轨迹的奖励之和。

$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t r(s_t, a_t) \right], \quad \mathcal{D} = \{\tau_i\} \sim \pi_{\beta}, \quad \pi_{\theta} \neq \pi_{\beta}$$

期望是对 π_θ 自己产生的轨迹取的，而我们手里只有 π_β 的轨迹，也没法去采 π_θ 的——这就是 offline RL 的核心难点：**分布偏移**

（distribution shift）。环境的动力学在数据和部署时是同一个，偏移只来自策略不同。

- **问答：**数据足够全面时，学到的策略和 π_β 会不会同分布？ 几乎不会。通常你想比数据里的平均司机开得好，学出来的分布反而比 π_β 更窄；如果 π_β 本来就很好，那更像模仿学习的设定，策略仍可能慢慢漂离数据分布。
- **数据从哪来：**人、手写的控制器或现有系统、以前的 RL 实验，或者这些的混合。

小注：CME295 第 5 讲第 10 节说 DPO 直接吃现成的偏好数据、因而有分布偏移，PPO 在线采样没有——那正是本讲的设定：DPO 是 offline 的偏好优化。第 9 讲会正面处理这件事。

4. 直接把 SAC 跑在离线数据上会怎样：OOD 动作被"钓"出来

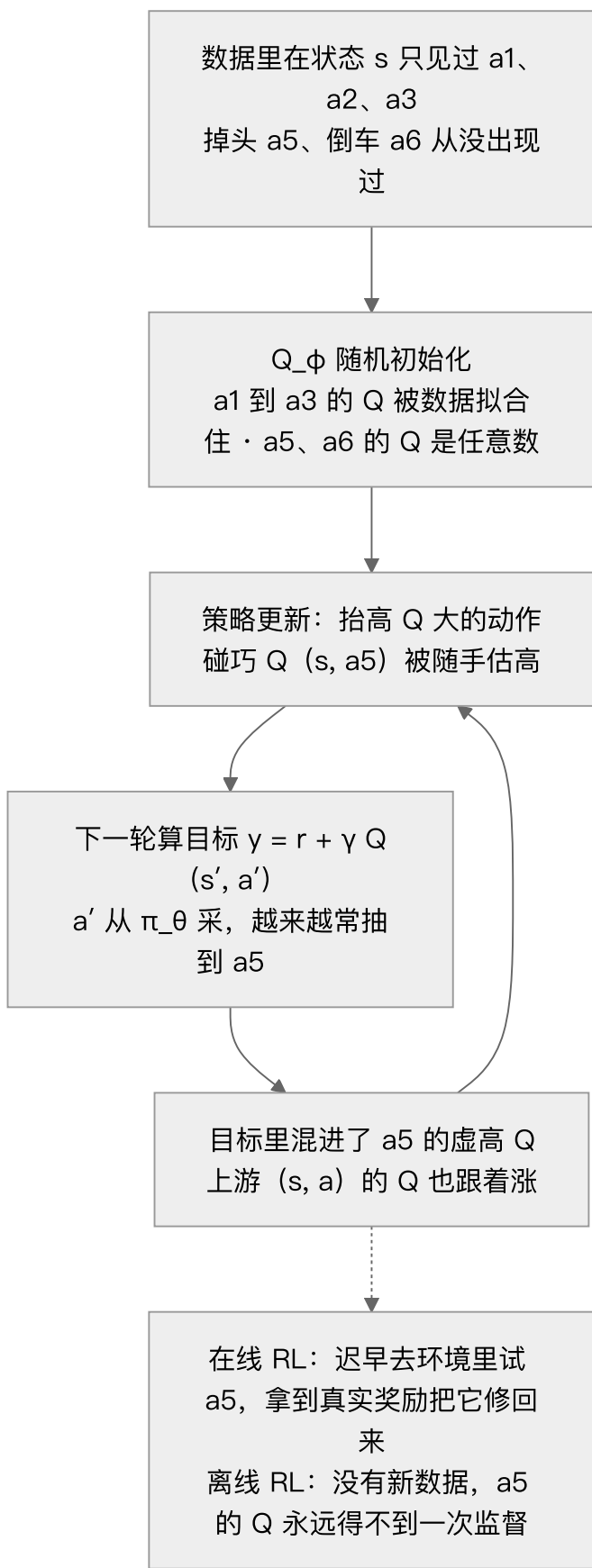


图 7-2 | 一个 OOD 动作的虚高 Q 怎样被策略“钓”出来、又怎样通过 bootstrap 传回上游，而且离线时永远得不到纠正（自绘示意）· [▶ 看原幻灯片 24:15](#)

- **自然的想法**：既然 off-policy 算法能用旧策略的数据，那把 SAC 拿来、只是不再收数据，行不行？有同学答对了要害：策略更新会抬高 Q 值高的动作，而 Q 是在从策略采出的动作上评估的；策略一旦和 π_β 不同，这些动作在数据里可能根本没出现过，用来监督 Q 的标签就不可靠， Q 不准，策略也就好不了。
- **板书推演**（图 7-2）：某个状态 s 下，数据里只见过 a_1 、 a_2 、 a_3 ；车其实还能开出去探险 a_4 、在高速上掉头 a_5 、倒车 a_6 。 Q 网络随机初始化，训练只能把 a_1 到 a_3 那一段拟合住，数据之外的 Q 是任意数。策略更新一看 a_5 的 Q 碰巧挺高，就抬高它的概率；之后算目标时 a' 越来越常抽到 a_5 ，虚高的 Q 混进标签、传给上游的 (s, a) ，而 a_5 在数据里永远不会出现，它的 Q 永远得不到一次真实监督。术语：这些动作叫 **out-of-distribution (OOD) 动作**，数据覆盖到的范围叫数据的**支撑** (support)；策略在“利用” (exploit) Q 在 OOD 动作上的错误。
- **幻灯片上的说法**： Q 在数据支撑之外不可靠 $\rightarrow$ 策略更新专门去找 Q 过分乐观的地方 $\rightarrow$ 更新之后这些 Q 值被高估 (overestimation)，而且会被反复强化，因为没有任何数据说它不好。换个角度：这发生在学到的策略偏离 π_β 太远的时候——OOD 动作正是 π_β 从不做的事；离 π_β 近一点，问题就轻。
- **和第 6 讲的高估对照**：Double DQN 治的是 max 挑中噪声偏正的动作；那里是在线的， ϵ -greedy 迟早会去试那个动作、拿到真实奖励把它修回来。离线没有这一步——图 7-2 的循环没有出口。
- **问答**
 - 离散动作会不会好些？——一样。若数据里每个状态都覆盖了全部动作，就没有 OOD 问题；实际上总有些状态没覆盖全，那里照样高估。
 - 更新 Q 时把 OOD 动作排除掉不就行了？——有这样做的方法，但麻烦在于**不知道什么算 OOD**：手里只有样本，一个状态选了 a_1 ，一个非常相似的状态选了 a_2 ，对前者来说 a_2 严格说没见过，可你没法划这条线。
- **由此得到 offline RL 方法的两条设计原则**：别偏离 π_β 太远；避开 OOD 动作。本讲后面的方法都在落实它们——具体做法是让策略只在数据里的动作上被监督。另一条路线是不拴策略、直接把数据外动作的 Q 压低，那是指定阅读 CQL 的做法，课上没讲到（第 12 节小注）。

5. 用模仿学习避开 OOD，但要能超过数据：轨迹拼接

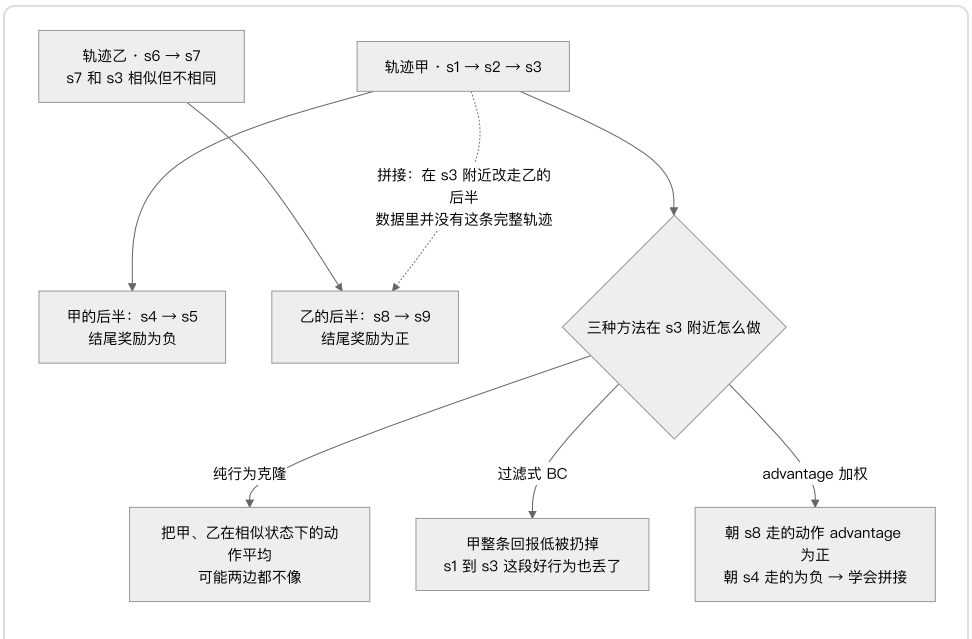


图 7-3 | 轨迹拼接：两条各自半好的轨迹，只有会用奖励的方法才拼得起来（自绘示意）· [▶ 看原幻灯片 28:27](#)

- **为什么模仿学习没有这个病：**行为克隆（第 2 讲）只在数据里出现过的 (s, a) 上做监督，从不评估别的动作，等于隐式地避开了 OOD 动作。
- **但纯模仿不是我们要的：**离线数据未必好——数据集里的人类司机不见得按你想要的方式开车。offline RL 该做的，是利用奖励信息超过数据背后的策略：翻一遍数据，这段行为好、那段不好，据此学出更好的策略；纯模仿最多和数据一样好。
- **拼接 (stitching)：**数据里两条轨迹，甲结尾拿到负奖励，乙结尾拿到正奖励。甲的前半段本身没问题——在中途改走乙的后半段，就是一条好轨迹。好的离线方法应该认出这一点：在甲、乙相近的那个状态，选乙的动作。符号化： $s_1 \rightarrow s_3$ 好， $s_7 \rightarrow s_9$ 好，offline RL 原则上能学出从 s_1 走到 s_9 的策略；模仿学习不能——它在相近的状态只会把甲、乙的动作平均一下。
- **问答（后面补的）：**两条轨迹要精确相交吗？不用。讲者画图时故意让 s_3 和 s_7 不是同一个状态：策略和 advantage 都是神经网络，相似的状态和动作自然有相似的输出，能泛化过去；泛化得多好取决于两个状态有多近、数据有多大。

小注：第 4 讲第 8 节两条轨迹共享一个蓝色状态的练习，讲的正是 bootstrap 能跨轨迹传信息；拼接是同一件事在离线数据上的名字。第 2

讲第 3 节说 L2 行为克隆只学得到多峰示范的均值，图 7-3 里"纯 BC 把动作平均"就是它。

6. 基线：过滤式行为克隆

- **做法**：从 $\mathcal{D}$ 里只挑回报高的轨迹——比如回报在第 70 百分位以上、也就是前 30%——组成新数据集，在上面做标准的行为克隆：最大化数据里动作的 $\log \pi_{\theta}(a | s)$ 。
- **评价**：用奖励的方式很原始，而且做不到拼接——甲整条回报低，会被整条扔掉， s_1 到 s_3 那段好行为也没了。但它是很好的基线：任何 offline RL 方法上手前先跑它，确保新方法比它强。
- **伏笔**：讲者说，一些最好的 offline RL 方法其实和它相去不远——它们都带一个模仿学习的成分。

小注：LLM 里的对应物是 CS329A 笔记里的 STaR / rejection-sampling fine-tuning：只在答对的样本上做 SFT，同样是"按结果过滤、然后模仿"，同样做不到拼接（我的类比，课上没说）。

7. 用 advantage 给模仿加权

- **想法**：不按整条轨迹取舍，而是给数据里每一个 (s, a) 的模仿目标乘一个权重，权重反映这个动作有多好。衡量"有多好"的现成量就是 advantage $A(s, a)$ 。权重取 $\exp(A/\alpha)$ ， α 是温度：

$$\max_{\theta} \sum_{(s,a) \in \mathcal{D}} \log \pi_{\theta}(a | s) \exp\left(\frac{A(s,a)}{\alpha}\right)$$

和行为克隆唯一的区别是多了指数权重：advantage 高的动作被使劲模仿，低的几乎不管， α 越小权重越"尖"。 (s, a) 全部来自数据，策略从不在数据外的动作上被监督。

- **它能拼接**：回到图 7-3，在 s_3 附近，朝 s_8 走的动作 Q 是 1，朝 s_4 走的（就数据来看） Q 是 0， V 是两者的平均，于是前者 advantage 为正、后者为负——加权之后策略被明确告知该走哪边。前提是 advantage 估得准，这是接下来几节的全部内容。
- **为什么是指数**：这个目标可以推导出来（幻灯片上有，课上没推）：它近似地在解一个带 KL 约束的优化——最大化按 Q 衡量的未来奖励，同时要求策略离 π_{β} 不远。对 offline RL 这是个合理的目标：离 π_{β} 太远，就没有可靠的信息告诉你怎么改进。

$$\max_{\pi} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi(\cdot | s)} [Q(s, a)] \quad \text{s.t.} \quad \mathbb{E}_{s \sim \mathcal{D}} [\text{KL}(\pi(\cdot | s) \| \pi_{\beta}(\cdot | s))] \leq \epsilon$$

KL 衡量两个动作分布差多远（CME295 第 5 讲）， ϵ 是允许偏离 π_β 的幅度；解这个问题、再把解投影到策略网络上，就得到上面的加权目标。CME295 第 5 讲 RLHF 目标里拴住 SFT 模型的那根 KL 绳子，和这里是同一根，只是锚点换成了 π_β 。

• 问答

- 指数会不会爆？——实现里有温度 α ，写成 $\exp(A/\alpha)$ ，可以压住。
- 和 actor-critic 的目标像不像？——像，但两处不同：那里权重是 A 本身、没有指数；那里动作从当前策略采（SAC）或来自在线数据，这里 (s, a) 都从离线数据集来。更深一层的原因讲者只点到为止：在线方法往往在优化一个 reverse KL，离线方法优化的更像 forward KL，展开说要很久。

小注：那个 KL 约束问题有闭式解 $\pi^*(a | s) \propto \pi_\beta(a | s) \cdot \exp(A(s, a)/\alpha)$ ——和 CME295 第 5 讲 DPO 推导第 2 步的 $\pi^* \propto \pi_{\text{ref}} \cdot \exp(r/\beta)$ 是同一个式子，只是锚点从 SFT 模型换成 π_β 。用 forward KL ($\text{KL}(\pi^* \parallel \pi_\theta)$) 把 π_θ 拟合到 π^* 上，需要的是 π_β 的样本按 $\exp(A/\alpha)$ 加权做最大似然，也就是只用数据里的动作；用 reverse KL ($\text{KL}(\pi_\theta \parallel \pi^*)$) 则要从 π_θ 采动作再问 Q ——正是第 4 节会出事的那一步。这就是离线方法偏爱 forward KL 的原因（推断，课上只提了名字）。指数权重的推导应出自 AWR（Peng et al., 2019）和 AWAC（Nair et al., 2020）两篇。

8. 估 advantage 之一：Monte Carlo，得到 AWR

- 先回答"谁的 advantage"：advantage 总是相对某个策略定义的。眼下最省事的选择：估行为策略 π_β 的 advantage——数据就是 π_β 采的，评估它是 on-policy 的，第 1 节的前两种标签都能直接用。
- 版本一：Monte Carlo。先用平方误差把 $V_\phi(s_t)$ 回归到这条轨迹从 t 起实际拿到的奖励之和，纯监督学习；再算 advantage。算 advantage 有两条路：一是第 4 讲的一步 TD 式 $r + V(s') - V(s)$ ，它依赖 V 相当准；二是更简单的——直接用实际的未来奖励和减去 $V(s)$ ，不依赖两个估值之差。后者就是 **advantage-weighted regression (AWR)**。两种都合法，后者在离线数据上用得多、效果也好；GAE 也可以用。

$$\min_{\phi} \sum_{(s_t, \tau) \in \mathcal{D}} \left(V_{\phi}(s_t) - \sum_{t' \geq t} r_{t'} \right)^2, \quad \hat{A}(s_t, a_t) = \sum_{t' \geq t} r_{t'} - V_{\phi}(s_t)$$

左式是 Monte Carlo 回归，标签是轨迹 τ 从 t 起的奖励和（带折扣时加 γ 的幂）；右式的 advantage 是"这条轨迹实际拿到的"减"这个状态平均能

拿到的"。

- **问答：为什么这里敢用高方差的 Monte Carlo？** 一部分原因是离线数据集通常很大，方差没那么要命；更大的原因是简单——拟合 V 是监督学习，训练策略是加权的监督学习，整个算法没有一处不稳。后面更复杂的方法会更好，因为这个估计的方差仍然比 bootstrap 大。
- **同桌讨论：行为策略是确定性的会怎样？** 比如车在某些车道永远直行，数据里从不出现转向。答案：每个状态只见过一个动作，实际奖励和就等于 V ，advantage 全是 0， $\exp(0) = 1$ ，目标退化成普通的行为克隆。讲者说这其实挺好：数据里没有动作的多样性，你本来就不可能比 π_β 做得更好，算法优雅地退回模仿。
- **问答**
 - 那收数据时是不是该故意偏离专家、多试几种动作？——是，动作多样性对 offline RL 是好事，尤其当专家并不最优时：多样性是超过专家的唯一本钱。
 - advantage 全 0 常见吗？——真实的 advantage 在没见过的动作上并不为 0，只是数据里看不到；只要动作有变化，估出来的 advantage 就会明显非零，取决于数据有多"广"。
 - advantage 网络在数据外的动作上输出什么？——这个目标从不在数据外评估它，所以不成问题；真要问，离数据近可能还靠谱，远了就是任意值。
- **完整算法与评价：**① Monte Carlo 回归拟合 V ；② 用 $\exp((\text{奖励和} - V)/\alpha)$ 加权做行为克隆， α 控制权重多"尖"。优点：极其简单——大概是最简单的、有可能超过过滤式 BC 的方法；从不查询、也从不在 OOD 动作上训练，所以稳。缺点：Monte Carlo 有噪声；估的是 π_β 的 advantage 而不是新策略的，改进幅度因此有限。
- **数值例子**（回答"advantage 是不是总接近 0"）：同一个（或相近的）状态下三种结局 +10、+1、-5， V 拟合到平均值，三个动作的 advantage 就是各自结局减 V ——一正两负，差异很大。只有从相似状态出发、动作不同、结局不同，advantage 才拉得开。

小注：课上口算把平均说成 3，得到 7、-2、-8；三个数的平均实际是 2，advantage 应为 8、-1、-7，结论不变。AWR 原文是 Peng, Kumar, Zhang & Levine, 2019。

9. 估 advantage 之二： a' 也从数据里取，得到 AWAC

- **想要 bootstrap**: Monte Carlo 方差大, 最好能像第 1 节第三种标签那样用 TD 目标——但那个目标坏就坏在 $Q(s', a')$ 里的 a' 是从正在学的策略采的。最简单的修法: a' 也从数据里取, 就用这条转移里实际发生的下一个动作。这样 (s', a') 一定在数据里, 一定有监督。

$$y = r + \gamma \hat{Q}_\phi(s', a'), \quad (s, a, r, s', a') \sim \mathcal{D}$$

和第 1 节第三个标签的唯一区别: a' 不再来自 π_θ , 而是数据里紧接着记录的那个动作。

- **算法**: 用这个目标拟合 Q , 由 Q 算 advantage, 再套第 7 节的加权目标——课上称之为 **advantage-weighted actor-critic (AWAC)**。它去掉了 AWR 的一个弱点 (可以 bootstrap 了), 但另一个还在: 估的仍然是 π_β 的价值, 不是更好的策略的。

小注: 这种" a' 取数据里的下一个动作"的目标在文献里常叫 SARSA 式目标; AWAC 原文 (Nair et al., 2020) 里 critic 的 a' 其实是从当前策略采的, 靠加权 BC 的隐式约束让策略离数据不远、查询不至于太 OOD (推断; 两种写法都只估 π_β 附近的价值, 讲者要说的要点相同)。AWAC 的设计初衷正是"离线预训练 + 在线微调"。

10. 估一个比 π_β 更好的策略的价值: 拟合分位而不是均值

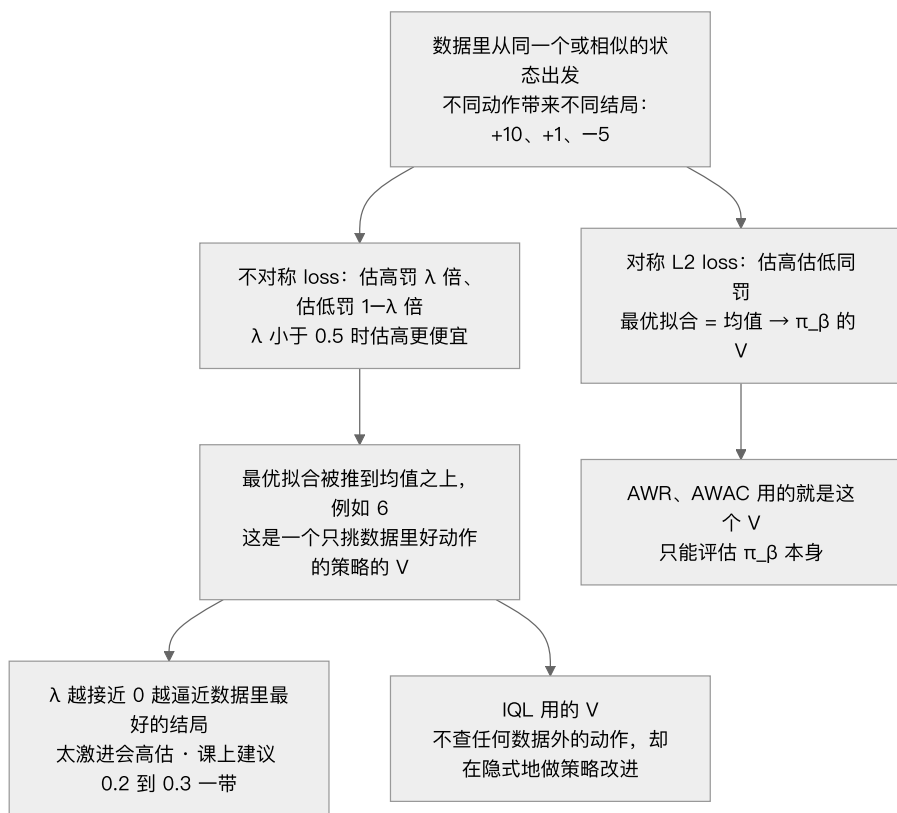


图 7-4 | expectile regression: 把对称 loss 换成不对称 loss, 拟合值就从"数据的平均结局"挪到"数据里较好的结局" (自绘示意) · [▶ 看原幻灯片 1:00:10](#) · 出处: Kostrikov et al., 2021

- **换个角度看 TD 目标:** $r + \gamma Q(s', a')$ 可以看成 $V(s')$ 的一个样本。同一个状态出发、数据里不同的动作带来不同的结局, 所以对于一个状态而言, 数据里其实有一个**结局的分布**——很多条落在某个值附近, 少数几条更高。用 L2 (平方误差) 回归, 拟合到的是这个分布的**均值**, 也就是 π_β 的 V 。
- **想法:** 要估一个比 π_β 更好的策略的 V , 就别拟合均值, 去拟合更高的**分位** (percentile), 比如第 70 百分位; 直方图的最右端对应"数据支撑内最好的策略"的 V 。怎么让回归拟合分位? 换 loss。
- **不对称 loss:** L2 关于误差 e 是对称的抛物线 e^2 , 估高估低同罚。改成两边斜率不同: 估高 (e 大于 0) 罚 λ 倍, 估低罚 $1 - \lambda$ 倍。 $\lambda = 0.5$ 就是 L2; λ 小于 0.5 时估高更便宜、估低更贵, 最优拟合值被推到均值之上; λ 大于 0.5 反之。这叫 **expectile regression**, 讲者说是个"很酷的技巧"; 幻灯片上画了不同 λ 下 loss 的形状, 以及同一个随机变量的均值 (红线) 和各个 expectile。

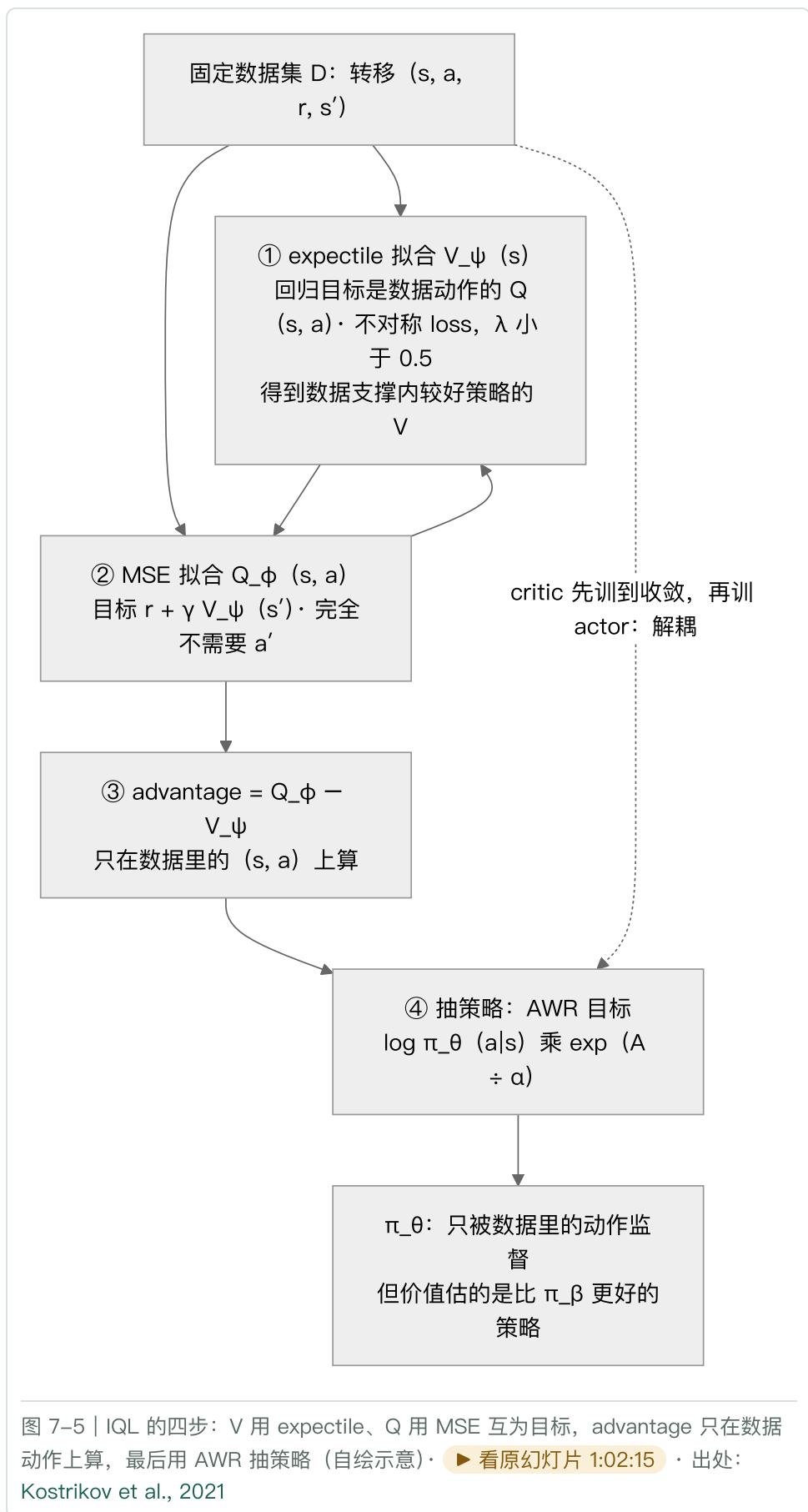
$$L_\lambda(e) = |\lambda - 1[e < 0]| e^2, \quad e = V_\phi(s) - y$$

e 是估值减目标； $e \geq 0$ （估高）时权重是 λ ， $e < 0$ （估低）时权重是 $1 - \lambda$ ； $\lambda = 0.5$ 退回普通平方误差。最小化它得到的不是均值，而是分布的 λ -expectile。

- **效果：**估高和估低都还挨罚，只是稍微估高一点罚得轻。回到 +10 / +1 / -5 的例子：均值是 3（课上的口算），某个 λ 下最优输出会跑到 6 一带——比均值高，但不是简单取最大。

小注：expectile 之于不对称平方损失，正如分位数之于不对称绝对值损失、均值之于普通平方损失； λ 趋于 0 时逼近支撑内的最大值。IQL 论文的写法是 $L_2^\tau(u) = |\tau - \mathbb{1}(u < 0)| u^2$ ， u 是目标减估值，方向和课上的 e 相反，所以论文的 $\tau = 1 - \lambda$ ：论文在 D4RL 运动任务上取 $\tau = 0.7$ 、AntMaze 上取 0.9，对应课上说的 $\lambda \approx 0.3$ 和 0.1。

11. IQL：expectile 拟合 V、MSE 拟合 Q，从不查询数据外的动作



• 算法

1. 用 expectile loss 拟合 $V_\psi(s)$ ，回归目标是数据里动作的 $Q(s, a)$ ——在"这个状态下数据里各个动作的 Q "这个分布上取上侧 expectile。
 2. 用普通的均方差拟合 $Q_\phi(s, a)$ ，目标是 $r + \gamma V_\psi(s')$ ——目标里只有 V ，根本不需要 a' ，无论从策略还是数据里取都不需要。
 3. $\text{advantage} = Q_\phi - V_\psi$ ，只在数据里的 (s, a) 上算。
 4. 用第 7 节的加权目标抽策略： $\log \pi_\theta(a | s)$ 乘 $\exp((Q - V)/\alpha)$ 。
- 两个价值网络里只有 V 用不对称 loss； V 和 Q 交替训练、互为目标。

$$L_V(\psi) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [L_\lambda(V_\psi(s) - \hat{Q}_\phi(s, a))], \quad L_Q(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(\hat{Q}_\phi(s, a) - r - \gamma V_\psi(s'))^2], \quad \max_{\theta} \mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi_\theta(a | s) \exp(\frac{\hat{Q}_\phi(s, a) - V_\psi(s)}{\alpha})]$$

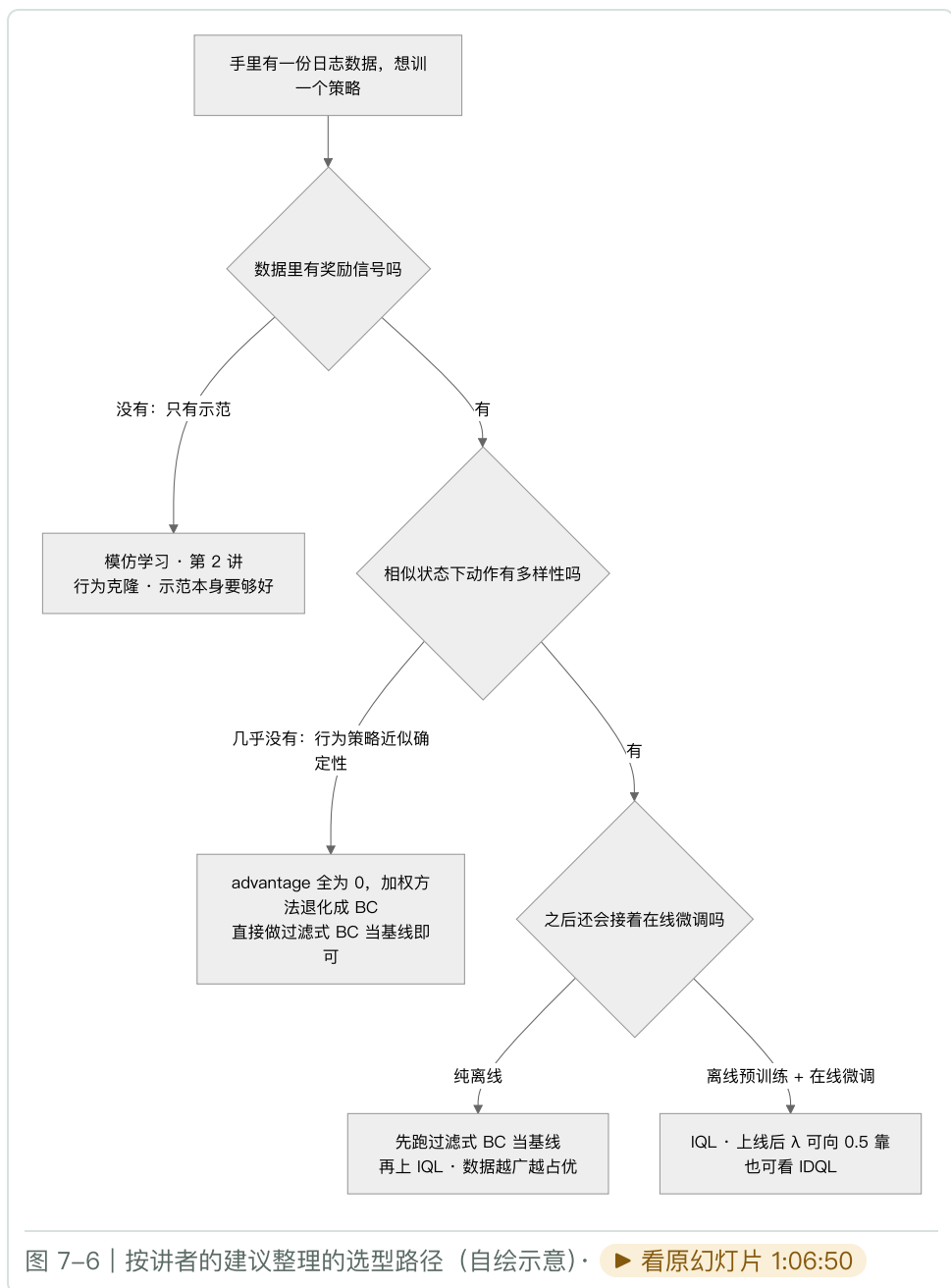
ψ 、 ϕ 、 θ 分别是 V 、 Q 、策略的参数；三个损失里的 s 、 a 、 s' 全部来自 $\mathcal{D}$ ，没有任何一处需要在数据外的动作上评估 Q 。

小注： V 的回归目标是数据动作的 $Q(s, a)$ ，课上板书没写明，按论文补上。"implicit"的意思：第 6 讲 Q -learning 目标里对 a' 的 $\max$ ，被换成了数据动作上的上侧 expectile—— λ 越小越接近 $\max$ ，但永远只在支撑之内取，所以策略改进是"隐式"完成的，不用真的去比较没见过的动作。

• 问答

- 这样会不会高估？——从不把数据外的动作喂进网络，之前最大的高估来源已经没了。但 λ 设成 0 大概率会高估；实践里取 0.2、0.3 这种不那么激进的值。而且这里是故意高估 π_β 的 V ：要估的本来就是一个比 π_β 更好的策略的 V 。
 - 策略在变好， λ 要不要逐步调？——不用。数据全程是离线的，你想要的一直是"比 π_β 好一个固定的量"， λ 保持不变。如果之后开始收在线数据，数据来自更好的策略， λ 反而应该往 0.5 靠。
 - 为什么还要 Q ，不直接用 Monte Carlo 回报配 expectile？——可以，这套想法对 Monte Carlo 同样适用；用 Q 是因为 bootstrap 目标方差更低，而 bootstrap 只有 Q 能做。
- 优点：不查询 OOD 动作；策略只在数据动作上训练；actor 和 critic 解耦——先把 V 和 Q 训到收敛（或训够），再单独训策略。名字：implicit Q-learning (IQL)。

12. 总结、选型建议与各方法对数据的要求



- **讲者的总结：**在线数据贵，每次跑算法都要重收；离线数据可以反复用。核心难点是 π_β 和学到的策略之间的偏移。过滤式或加权的模仿学习是基线；把策略显式拴在 π_β 上——只在数据动作上监督策略——避开了分布偏移；轨迹拼接则是它超过纯模仿的来源。应用案例的幻灯片跳过了。
- **建议：**纯离线训练，过滤式 BC 和 IQL 都是好选择，IQL 大概率更好，数据越广越明显；要做“离线预训练 + 在线微调”，IQL 非常合适，配合在线收数据效果也好。IQL 还有一个变体 IDQL，策略换成扩散策略（第 2 讲第 7 节），更新方式也不同，想深入 IQL 一系可以看它。
- **各方法对数据集的要求（给要从日志里学的人）：**

方法	学什么	从不评估什么	对数据集的要求	能拼接	能超过 π_β
行为克隆 (第 2 讲)	只学策略	数据外的动作	示范本身就要够好	否	否
过滤式 BC	前 k% 轨迹上的策略	数据外的动作	要有整条轨迹的回报；好轨迹得有一定数量	否	只到数据里好轨迹的水平
AWR	Monte Carlo 的 V + 加权 BC	数据外的动作	要有每步奖励和完整轨迹 (算 reward-to-go)；相似状态下动作要有多样性，否则退化成 BC	能 (advantage 准的话)	有限：advantage 是 π_β 的
AWAC (课上的版本)	SARSA 式的 Q + 加权 BC	数据外的动作	单步转移 (s, a, r, s', a') 即可；动作多样性	能	有限：价值仍是 π_β 的
IQL	expectile 的 V + MSE 的 Q + 加权 BC	数据外的动作	单步转移 (s, a, r, s') 即可；动作多样性； λ 要调	能	能：估的是支撑内更好策略的价值
CQL (指定阅读，课上没讲)	保守的 Q + actor	会评估数据外动作，但把它们的 Q 压低	单步转移；正则化系数要调	能	能

小注：指定阅读的第一篇 CQL (Kumar et al., 2020) 课上没有讲到。它代表另一条路线：不拴策略，而是在 Q 的 Bellman 损失上加一个正则项，把策略会选的（也就是可能 OOD 的）动作的 Q 压低、把数据里动作的 Q 抬高，从而让学到的 Q 成为真实价值的下界——图 7-2 里那个虚高的 $Q(s, a_5)$ 被直接按下去，策略再也钓不到它。摘要里的数字：在离散和连续控制的离线基准上，最终回报比之前的方法高 2 到 5 倍。第一作者 Aviral Kumar 就是第 10 讲的客座讲者。

小注：IDQL (Hansen-Estruch et al., 2023) 把 IQL 的 critic 重新解释成一个带行为正则的隐式 actor 的 actor-critic；指出高斯策略配 AWR 表达不了这个隐式 actor 的多峰形状，于是改成：用扩散模型拟合 π_β ，从中采一批动作，用 critic 算出的权重做重要性重采样来抽策略。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
拟合价值函数的三种标签	Monte Carlo、TD、off-policy 的 Q 目标并排；第三种里只有 a' 来自策略	1:08	—

图	看什么	跳转	出处
在线 vs 离线流程	在线是"收 → 更新 → 再收"的循环，离线只有一份给定的数据集	9:50	—
板书：数据集与 π_β	一堆驾驶轨迹被看成由一个（混合的）行为策略采出；目标函数的期望却在 π_θ 下	14:03	—
板书：a1 到 a6 与随机的 Q	数据里只有 a1 到 a3；随机初始化的 Q 曲线在数据外任意起伏，策略专挑高的	21:19	—
幻灯片：OOD 动作上的 Q	数据支撑只盖住一部分动作；策略更新去找过分乐观的 Q；更新后被高估并反复强化	24:15	—
轨迹拼接	绿色轨迹结尾正奖励、另一条负奖励；把前半段和绿色后半段拼起来	28:27	—
过滤式 BC	只保留回报超过某个百分位的轨迹再做 BC；注意它不能拼接	30:30	—
advantage 加权目标	行为克隆目标乘 $\exp(\text{advantage})$ ；旁边写着它近似 KL 约束目标，附参考文献	33:36	AWR
版本一：Monte Carlo 板书	V 回归到奖励和；advantage 的两种算法： $r + V(s') - V(s)$ 与"奖励和 $- V$ "	40:44	AWR
AWAC 的一处改动	TD 目标里的 a' 从"策略采样"改成"数据里的下一个动作"	52:16	AWAC
结局直方图	同一状态的价值样本有分布；L2 给均值；支撑内最好策略的 V 在右端	56:28	IQL
不对称 loss 板书与 expectile 曲线	e^2 两侧权重不同； $\lambda = 0.5$ 回到 L2；不同 λ 的 loss 形状与各个 expectile	57:32	IQL
IQL 算法	V 用 expectile、Q 用 MSE 且目标是 V、策略用 $\exp(Q - V)$ 加权；下一页是三条优点	1:02:15	IQL
总结与建议	纯离线：过滤式 BC 或 IQL；离线转在线：IQL；IDQL 用扩散策略	1:06:50	IDQL

提到的工作

名称	在本讲里的作用
策略梯度（第 3 讲）、actor-critic（第 4 讲）、PPO / SAC（第 5 讲）、Q-learning（第 6 讲）	开场回顾；SAC 是"直接跑在离线数据上会翻车"的例子
拟合价值函数的三种标签（Monte Carlo、TD、off-policy 的 Q 目标）	本讲的工具箱；第三种标签是问题的根源
自动驾驶、癫痫治疗（应为 Guez et al., 2008）、机器人抓取	为什么要 offline RL 的三个场景
行为克隆（第 2 讲）	天然避开 OOD 动作的参照系，但超不过数据
过滤式 BC（filtered BC）	第一个基线：只模仿回报靠前的轨迹

名称	在本讲里的作用
AWR (Peng et al., 2019)	exp(advantage) 加权的行为克隆，配 Monte Carlo 的 V
KL 约束的策略优化；forward / reverse KL	加权目标的来历；在线与离线目标差别的深层原因，课上只提了名字
GAE (第 5 讲)	问答里提到也能用来估 advantage
AWAC (Nair et al., 2020)	a' 取自数据的 bootstrap 版本（课上的讲法；见第 9 节小注）
expectile regression	用不对称平方损失拟合分位而不是均值
IQL (Kostrikov et al., 2021, 指定阅读)	本讲的终点：expectile 的 V、MSE 的 Q、加权 BC
IDQL (Hansen–Estruch et al., 2023)	IQL 的扩散策略变体，课末推荐
扩散策略（第 2 讲）	IDQL 的策略类
CQL (Kumar et al., 2020, 指定阅读)	课上没讲；保守 Q 的另一条路线，见第 12 节小注
offline-to-online fine-tuning	IQL 的推荐用法；上线后 λ 往 0.5 靠

术语对照

English	中文
offline RL / batch RL	离线强化学习：只用给定的固定数据集训练，不再交互
online RL	在线强化学习：训练过程中反复收集新数据
off-policy	用其他策略（含过去的自己）的数据训练；离线必然 off-policy
behavior policy π_β	行为策略：收集数据集的那个（未知的、可能是混合的）策略
mixture of policies	混合策略：多个策略按权重平均，用来统一描述多来源的数据
distribution shift	分布偏移：目标在 π_θ 的轨迹上取期望，数据却来自 π_β
support / coverage	支撑 / 覆盖：数据实际出现过的状态和动作范围
out-of-distribution (OOD) action	数据外的动作：某状态下数据里从未出现的动作
overestimation	高估：Q 在 OOD 动作上偏高且被策略反复强化
exploit (Q 的误差)	利用：策略更新专挑 Q 估错偏高的动作
bootstrapping / TD target	自举 / 时序差分目标：用自己的估值当回归标签

English	中文
Monte Carlo return / reward-to-go	蒙特卡洛回报：轨迹从 t 起实际拿到的奖励和
trajectory stitching	轨迹拼接：把两条轨迹各自好的一段接成一条更好的
filtered behavior cloning	过滤式行为克隆：只在回报靠前的轨迹上做 BC
advantage-weighted regression (AWR)	优势加权回归：exp(advantage) 加权的行为克隆
temperature α	温度：控制 exp(A/ α) 权重有多"尖"
KL-constrained objective	KL 约束目标：最大化 Q 同时限制离 π_β 的 KL
forward / reverse KL	前向 / 反向 KL：拟合分布时用目标分布的样本还是自己的样本
advantage-weighted actor-critic (AWAC)	优势加权 actor-critic：bootstrap 的 Q 配加权 BC
SARSA-style target	SARSA 式目标： a' 取数据里实际的下一个动作（小注里的说法）
percentile / expectile	百分位 / expectile：不对称平方损失的最小点， $\lambda = 0.5$ 时是均值
expectile regression	expectile 回归：用不对称平方损失做回归
asymmetric loss	不对称损失：估高与估低罚得不一样重
implicit Q-learning (IQL)	隐式 Q 学习：max 换成支撑内的上侧 expectile，从不查询数据外动作
decoupled actor / critic	解耦：先训好价值网络，再单独训策略
offline-to-online fine-tuning	离线预训练、在线微调
diffusion policy	扩散策略（第 2 讲）：IDQL 的策略类
conservative Q-learning (CQL)	保守 Q 学习：压低数据外动作的 Q，使 Q 成为真值的下界
D4RL	离线 RL 的标准基准数据集（小注里提到）

字幕勘误

"PO" → PPO；"SACE""soft vector critic" → SAC / soft actor-critic；"q hot pi of fi" → Q^{π_ϕ} ；"key value""key function""keying" → Q value、Q function、Q-learning（"implicit keying" → implicit Q-learning）；"kale constraint""reverse kale / forward kale" → KL constraint、reverse / forward KL；"auto distribution""autodistribution" → out-of-distribution；"off-picoloy" → off-policy；"monocolor returns" → Monte Carlo returns；"GA""GE"

→ GAE; "advantage beta regression" → advantage-weighted regression; "expect loss" → expectile loss; "behavior coding" → behavior cloning; "the learn policy" → the learned policy; " e^2 " → e^2 ; "downweing / upweing" → down-weighting / up-weighting。

带走的问题

1. 图 7-2 的循环在线时靠探索打断，离线时没有出口。如果你的 agent 只能从日志里学、永远不能上线试错，"策略只在数据动作上被监督"是不是唯一的出路？CQL 那条路（压低数据外动作的 Q）和它相比，各自在什么样的数据上会吃亏？
2. 确定性数据集让 AWR 退化成行为克隆。你的产品日志里，同一类状态下"动作"的多样性从哪来（不同用户、A/B 实验、随机化）？如果多样性来自不同用户， π_β 是混合策略，那 advantage 衡量的是"比谁好"？
3. IQL 的 λ 越小越激进， λ 趋于 0 时 V 逼近"数据里最好的结局"——这和第 6 讲里 max 带来的高估有什么本质区别？为什么 λ 常取 0.2 到 0.3 而不是更小？在线微调时又为什么该往 0.5 靠？
4. 过滤式 BC 需要整条轨迹的回报，AWR 需要 reward-to-go，AWAC / IQL 只需要单步转移。你的日志能提供哪一种？稀疏奖励（只有会话结束时的一个分）下，三者各自的 advantage 估计会出什么问题？
5. CME295 第 5 讲的 DPO 在固定偏好数据上训练，算不算 offline RL？它拴住 reference 的 KL 和本讲拴住 π_β 的 KL 是同一根绳子吗？如果把 IQL 的想法搬到 LLM——只用日志里的回答做加权 SFT——advantage 该怎么估、 V 的 expectile 又该在什么分布上取？

CS224R 第 8 讲 | 奖励学习 (Reward Learning)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 8 讲, 2025 年 4 月 25 日 视频: <https://www.youtube.com/watch?v=PDlxDhA9Z6Y> (1:05:58, 英文字幕是自动生成的, "Q value" 几乎全被听成 "key value", 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座) 课程主页: <https://cs224r.stanford.edu/> · 本讲指定阅读: Deep Reinforcement Learning from Human Preferences (Christiano et al., 2017)

一句话: 前 24 分钟收尾离线 RL: IQL 为什么只在学 V 时用非对称 loss——动作可以挑、运气不能挑; 以及 CQL——把数据支持之外的 Q 值压低、数据里的拉回来, 加个熵正则之后连那个"专找高 Q 值动作"的分布 μ 都不用显式构造, 惩罚项直接变成 log-sum-exp。后 40 分钟讲奖励从哪来: 倒水、聊天、驾驶都没有现成的奖励, proxy 又靠不住。最省事的办法是拿成功示例训一个分类器当奖励, 但 RL 会专门去找分类器没见过、又误判为好的状态 (reward hacking); 解药是把策略访问过的状态一律标成负例回灌重训, 这恰好就是一个 GAN。更通用的办法是从成对偏好学: 假设"更好的那条轨迹奖励和更大", 差值过 sigmoid 做最大似然, 可以一次排 K 条、可以放进 RL 循环里边采边问——仿真里的 backflip、驾驶偏好、ChatGPT 的 RLHF 用的都是这一个公式。

时间轴

时间	内容
0:06	开场: 今天三件事——收尾离线 RL、为什么任务指定难、怎么从人类监督学奖励; 离线 RL 的设定与分布偏移回顾
2:07	IQL 回顾: 只用数据里的动作学 V 和 Q; 为什么 V 用非对称 loss 而 Q 用 L2——动作可控、运气不可控
7:19	第二条路: 允许查询策略的动作, 但把数据外的 Q 值压下去; CQL 版本一与"下界"性质
11:58	版本二: 把数据里的动作加回来; 问答: 会不会伤泛化、 α 怎么定、 μ 到底是什么、为什么不直接做行为克隆
16:32	数据集状态上的期望下界; CQL 完整算法; μ 加熵正则后的闭式解, 惩罚项变成 log-sum-exp
21:46	应用: LinkedIn 用 CQL 优化通知发送; 问答: 到底为什么要保守; 离线 RL 小结

时间	内容
24:49	奖励学习：奖励从哪来？倒水、聊天、驾驶；proxy 的问题；模仿学习为什么不够
28:25	从目标示例学：成功 / 失败分类器当奖励；两个问题——稀疏、被钻空子
30:55	防钻空子：把策略访问过的状态标成负例重训分类器，和 CQL 是一个思路
34:33	问答：正例太少会过拟合；负例越加越多要 50/50 平衡；假负例为什么没关系；直接用概率当奖励
39:42	问答：这不就是 GAN？机器人例子 50 条示范 26% → 62%；GAN 旁白；扩展成完整奖励函数；目标示例小结
45:16	从偏好学：哪条更好比打分容易；一条偏好 = 一个关于奖励和的不等式
48:21	奖励网络；差值过 sigmoid 变概率；最大似然目标；可以比较部分轨迹
51:59	问答：人没说"好多少"怎么办；一次排 K 条；完整算法；放进 RL 循环里边采边问
56:36	例子：backflip 只问了 900 次；驾驶偏好；LLM：煎蛋卷的两个回答、预训练 → SFT → RLHF 三步
1:00:13	问答：x 是 prompt 还是整段历史、排序怎么拆成对、 σ 是什么；RLAIF；小结；其他监督形式；无监督 RL 让 agent 自己提目标

核心内容

1. 收尾离线 RL：IQL 为什么只对 V 用非对称 loss

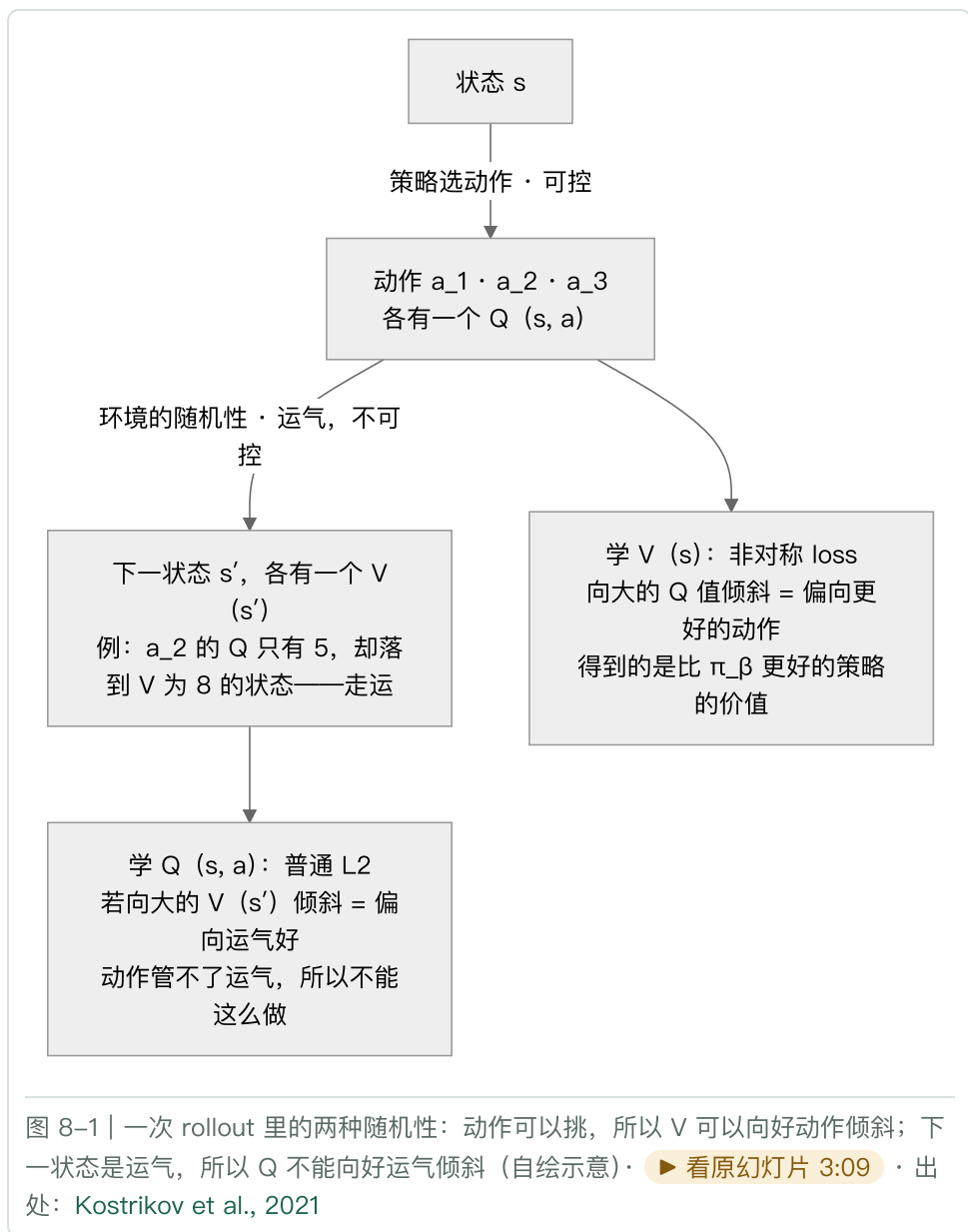


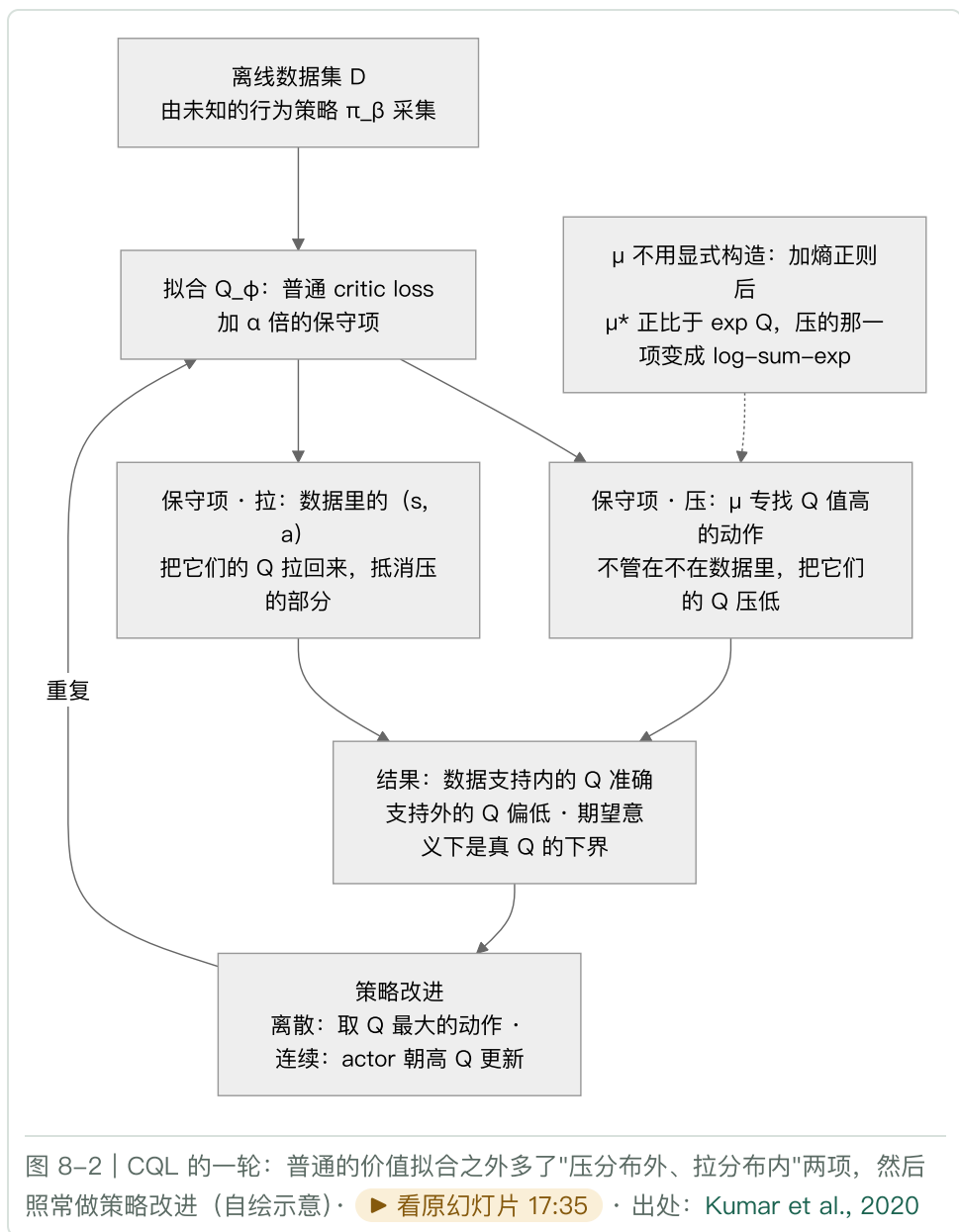
图 8-1 | 一次 rollout 里的两种随机性：动作可以挑，所以 V 可以向好动作倾斜；下一状态是运气，所以 Q 不能向好运气倾斜（自绘示意）· [▶ 看原幻灯片 3:09](#) · 出处: [Kostrikov et al., 2021](#)

- **上周三的设置回顾（第 7 讲）：离线 RL（offline RL）**是从一批固定的数据里学策略，训练期间不再和环境交互。数据由某个未知的**行为策略**（behavior policy） π_β 采集，目标是让要学的 π_θ 拿到高奖励。两者之间有**分布偏移**（distribution shift）： π_θ 想采取的动作，数据里未必有。
- **为什么传统算法会翻车**：如果学一个 Q 函数（第 6 讲： $Q(s, a)$ 是“在 s 做 a 、之后照策略走下去能拿到的期望回报”）再用它改进策略，改进这一步会在 π_θ 自己采出来的动作上查询 Q ，而这些动作可能不在数据覆盖范围里；随机初始化的 Q 在那些地方随便给个大数，策略就会奔着那个虚高的数去——这就是**高估**（overestimation）。
- **IQL 的回答**：干脆一次也不在数据之外查询。它只用数据里出现过的状态和动作学一个 V 和一个 Q 。可是照常规更新，学到的只是 π_β 自

己的 V^π 和 Q^π ，没法比行为策略好多少；所以 IQL 在学 V 时用**非对称 loss**（第 7 讲的 expectile 回归：向较大的 Q 值倾斜），学到的是“比行为策略好的那个策略”的价值；学 Q 时用普通的 $L2$ 。

- **本讲补的一课：为什么非对称只放在 V 上。**讲者的解释是，rollout 一条轨迹有两种随机性：一是你选了哪个动作，二是选定动作之后环境把你送到了哪个下一状态。前者可控，后者是运气。她举了三个动作的数字例子： a_1 的 Q 值不错，但实际落到的 s' 价值只有 8——这次运气差； a_2 本身更差， Q 只有 5，可实际落到的 s' 价值有 8——走运了； a_3 是个冒险动作， Q 是 -5 ，真做了却没那么糟， s' 的价值是 0。
 - 学 V 时向大的 Q 值倾斜，倾斜的对象是**动作**，等于偏向更好的动作，得到一个更好的策略的价值函数——这正是想要的。
 - Q 的回归目标是下一状态的 V 。如果学 Q 时也向大的 $V(s')$ 倾斜，倾斜的对象就成了**运气**，学到的是一个“更走运的策略”的价值，而运气不是动作能控制的——所以 Q 必须老老实实用对称的 $L2$ 。
- **为什么还要 Q 、不能只有 V ：**改进策略时要知道哪个动作更好，这只有 Q 能回答（比如第 7 讲的 advantage-weighted regression，用的就是 Q 减 V ）。

2. CQL：把数据之外的 Q 值压下去



- **另一条路**：仍然估计价值、仍然允许在策略自己的动作上查询 Q，但设法**劝阻**算法去追那些“看起来高、其实是分布外”的 Q 值。最直接的想法：数据支持（support，数据真正覆盖到的范围）之外的 Q 值，直接往下压。压完之后，对分布外的东西就是**保守 / 悲观的**（conservative / pessimistic），在数据覆盖的地方则不那么保守。
- **版本一**：在普通的 critic loss（第 4–6 讲那种以 TD 目标为回归目标的平方误差）之外再加一项：从数据集里取状态 s ，从某个别的分布 μ 里取动作 a ，把 $Q_\phi(s, a)$ 的期望往小里压。 μ 应该覆盖策略可能考虑的分
布外动作——而策略考虑的正是 Q 值高的动作，所以干脆让 μ 去找 Q 值最高的动作。

$$\min_{\phi} \mathcal{L}_{\text{critic}}(\phi) + \alpha \max_{\mu} \mathbb{E}_{s \sim \mathcal{D}, a \sim \mu(\cdot|s)} [Q_{\phi}(s, a)]$$

ϕ 是 Q 网络的参数, L_{critic} 是普通的价值拟合 loss, α 是惩罚项的权重 (超参数); 内层的 $\max$ 让 μ 专挑当前 Q_ϕ 认为最好的动作, 外层的 $\min$ 再把这些动作的 Q 压低。

- **它的性质:** 只要 α 足够大, 学到的 Q^* 是真实 Q 函数的一个下界 (lower bound)。讲者画的图: 横轴是动作, 真 Q 是一条曲线, 数据只覆盖中间一段, 学到的 Q^* 整条曲线都在真 Q 之下。
- **问题:** 太悲观了。这一项压的是所有动作的 Q 值, 可数据里出现过的动作我们有数据、有把握, 不该压。
- **版本二:** 把同样的项在数据里的 (s, a) 上减回来——分布外的压、数据内的拉, 两相抵消, 希望得到一个"数据支持内准确、支持外偏低"的 Q。

$$\min_{\phi} \mathcal{L}_{\text{critic}}(\phi) + \alpha \left(\max_{\mu} \mathbb{E}_{s \sim \mathcal{D}, a \sim \mu(\cdot|s)} [Q_{\phi}(s, a)] - \mathbb{E}_{(s,a) \sim \mathcal{D}} [Q_{\phi}(s, a)] \right)$$

括号里第二项在数据集自己的 (s, a) 对上取期望; μ 在支持内挑出的动作会被这一项抵消掉, 只有支持外的动作真正被压。

- **性质变了:** 不再是对每个 (s, a) 都成立的逐点下界, 但可以证明在数据集的状态上、期望意义下仍是真 Q 的下界 (证明在 CQL 论文里; $\alpha = 0$ 时不成立, α 要足够大, 具体条件讲者没记住)。讲者说这条性质说明这个目标"是合理的"。
- **问答**
 - 会不会伤害到数据之外的泛化? ——会, 它就是要在支持外更悲观。 α 控制悲观程度: 如果你相信自己的模型在这个问题上泛化得不错, 就把 α 调小, 多用原始 critic 目标、少用正则项。
 - 公式里该写 Q 还是 Q_ϕ ? ——是正在学的那个 Q_ϕ ; 幻灯片把 Q 当作被最小化的对象, 板书按参数写, 记号略有出入。
 - μ 是什么? 是一个"支持外的假策略"吗? ——可以把它看成一个策略, 但不是要运行的那个: 它专门去找 Q 值大的动作, 不管在不在数据里; 一旦找进了数据支持内, 就被第二项抵消, 所以支持内的估计仍然准确。
 - 既然目标是压低分布外的 Q, 为什么不直接做行为克隆? ——因为这个算法有可能比行为克隆好得多: 任何 RL 方法都在利用奖励信息去找 Q 值高的策略; 周三讲的按奖励筛选、排序再模仿的方法也能超过行为克隆。只在数据里的动作上训策略是一条合法路线, CQL 是另一条。

- **完整算法**：在数据集上优化上面的目标得到 $Q \rightarrow$ 更新策略（离散动作直接取 Q 最大的动作；连续动作要做一步 actor 更新，让策略去选高 Q 的动作） $\rightarrow$ 重复。
- **μ 不用真的造出来**。显式构造 μ 又烦又多一个活动部件；而且按上面的写法， μ 会坍缩成只取那一个 Q 最大的动作。我们希望它考虑更宽的一批动作——给 μ 加一个熵正则（entropy：分布越“散”熵越大）。“既要 Q 大、又要熵大”这个问题有闭式解：最优的 μ 正比于 Q 值的指数。代回去，那一项就变成对动作的 $\log\text{-sum-exp}$ ， μ 从公式里消失了。

$$\mu^*(a | s) \propto \exp(Q_\phi(s, a)) \implies \min_{\phi} \mathcal{L}_{\text{critic}}(\phi) + \alpha \mathbb{E}_{s \sim \mathcal{D}} \left[\log \sum_a \exp Q_\phi(s, a) \right] - \alpha \mathbb{E}_{(s, a) \sim \mathcal{D}} [Q_\phi(s, a)]$$

μ^* 是加了熵正则后的最优“找茬分布”； $\log \sum_a \exp Q$ 是一个“软最大值”——比 Q 的最大值略大、随 Q 平滑变化；连续动作空间下用采样来估计这个和。讲者说推导可能课后发到 Ed 上。

小注：这就是 CQL 论文（Kumar, Zhou, Tucker, Levine, 2020）里的 CQL(H) 变体，H 指熵正则；论文还有把熵换成对某个先验分布的 KL 的 CQL(ρ) 变体。连续动作下论文用重要性采样近似 $\log\text{-sum-exp}$ 。

- **一个“真实世界”的应用**：LinkedIn 用 double DQN（第 6 讲）加上 CQL 的惩罚项，在离线数据上学一个决定“要不要给用户发通知”的策略。目标是多目标的：点击率要高（说明用户真感兴趣），通知总数要少（别变成骚扰），还关心周活跃用户。上线做 A/B 测试后，比原有系统点击率更高、通知更少、用户使用量略增。

小注：应为 Prabhakar et al., KDD 2022, [Multi-objective Optimization of Notifications Using Offline Reinforcement Learning](#)，摘要里说的正是“基于 CQL 的 double DQN”。

- **问答：到底为什么要保守**——回到离线 RL 的初衷：数据支持之外你没法再采数据。如果假装能远远走出数据，学到的 Q 就不准，策略就差。保守只针对支持外。
- **离线 RL 小结**：在线采数据贵，离线数据有用；核心难题是策略与数据之间的偏移带来的高估；今天讲的办法是显式惩罚 Q 值；离线方法能超过模仿学习（包括筛选式模仿），靠的是把数据里好的片段拼接起来（stitching）和利用奖励信息。

3. 奖励从哪来：任务指定为什么难

- **问题：**除了模仿学习，前面所有算法都在最大化奖励之和，可奖励是谁给的？最早的 RL 用例是电脑游戏，游戏分数天然就是奖励。真实世界很少这么慷慨：
 - 让机器人把水从壶里倒进杯子——小孩都会，机器人很难。人喝到水有满足感，机器人没有。想手写奖励，得先表示"水在哪、洒了没、进杯子了没"，很难。
 - 聊天机器人——没有人会告诉你"这句回答值 1 分、那句值 10 分"。
 - 自动驾驶——世界不给分；交规、乘客舒不舒服……一堆因素要揉进去。
- **用 proxy：**可以让人打 1 / 0 的成功标签，或者用点击率这种代理指标——我们并不真的在乎人点没点，只是它和我们在乎的东西相关。但 proxy 有很多坏处（CME295 第 5 讲里"用掌声当讲课奖励"的比方，就是 proxy 被优化过头的样子），值得想想有没有别的监督方式。
- **模仿学习**（第 2 讲）不需要奖励，但它只是在模仿动作，没有在想"结果"；有些任务示范难给，或者专家和机器人自由度不同，示范根本对不上。
- **奖励学习**要做的事：推断专家想达到什么结果、人想要什么。人类很擅长这个——讲者放了小孩看一眼就明白大人想干什么、还主动去帮忙的视频。本讲讲两条路：**从目标示例学，从人类偏好学。**

4. 从成功示例学：训一个分类器当奖励，以及它怎么被钻空子

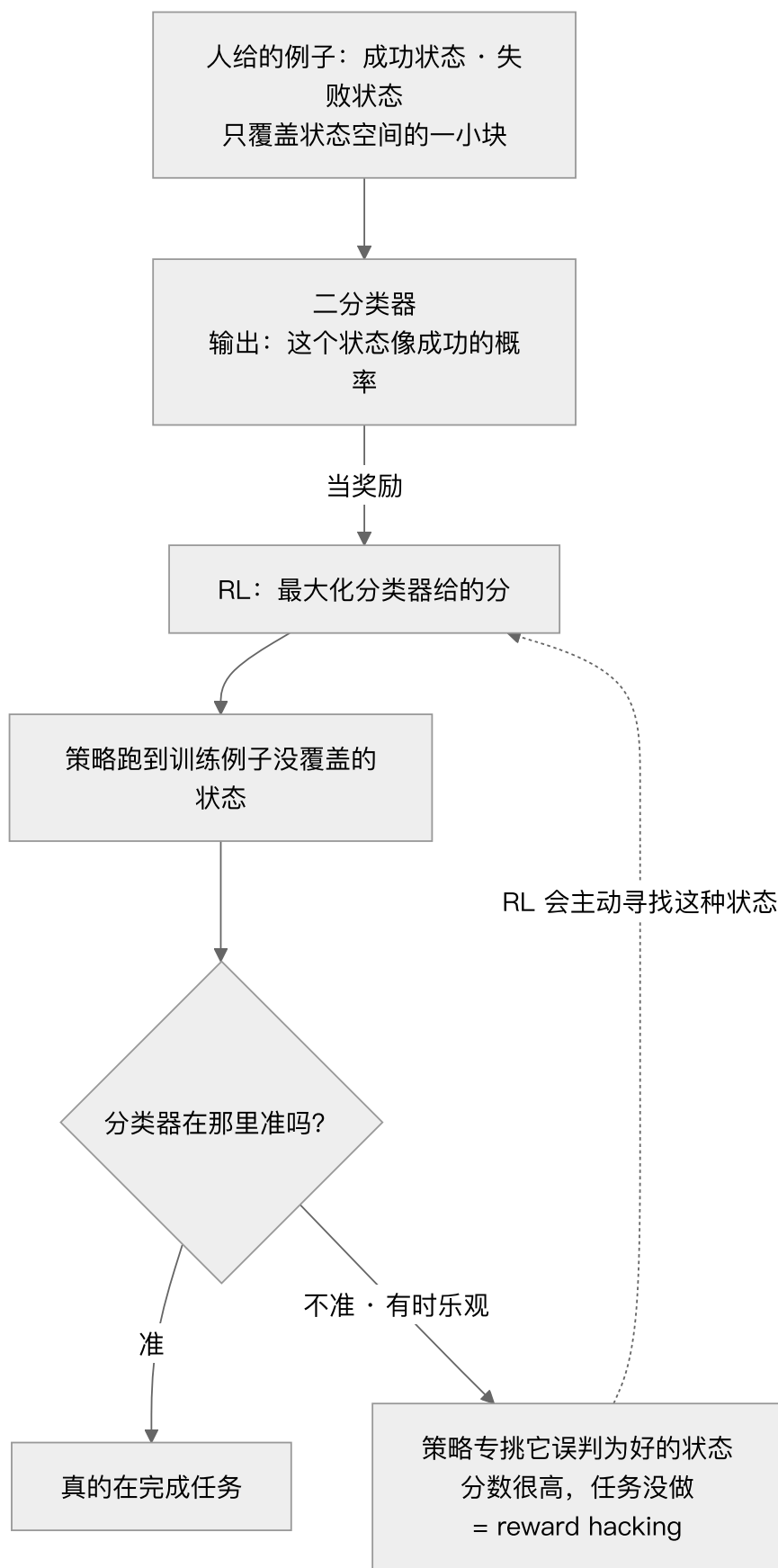


图 8-3 | reward hacking 从哪里进来：分类器只在固定数据上训过，而被它优化的策略会跑到数据之外，专找它判错的地方（自绘示意）· [▶ 看原幻灯片 29:55](#)

- **最简单的做法**：给几个"成功了"的例子（比如铅笔盒放到了笔记本后面），再给几个"没成功"的例子，训一个二分类器，正例标 1、负例标 0，把它的输出当奖励，跑 RL。也就是：收集成功状态（可以想成一个**目标集合**里的状态）和失败状态 → 训分类器 → 用分类器给的奖励做 RL。
- **会出什么问题**（讲者问学生，两个答案都对）：
 1. 奖励很**稀疏**：它只说你在不在目标集合里，进不了目标集合之前没有任何"正在接近"的信号。
 2. 策略会学会**骗过分类器**而不是完成任务。分类器只在你给的那批例子的分布上训过；策略却能跑遍各种状态，包括数据里没有的。这和离线 RL 里的分布外问题是一回事：对着分类器优化，RL 会主动寻找它判错的地方，而它判错时有时会乐观。讲者强调这一点"格外会出问题"——RL 算法会**专门去找**分类器以为好、其实没见过的状态，利用它的弱点。

小注：这就是 CME295 第 5 讲和 CS329A 第 3 讲里的 reward hacking，只不过那里被钻空子的是奖励模型或 verifier，这里是一个成功分类器。机制完全相同：奖励函数是从固定数据学出来的，而被它优化的策略会改变数据分布。

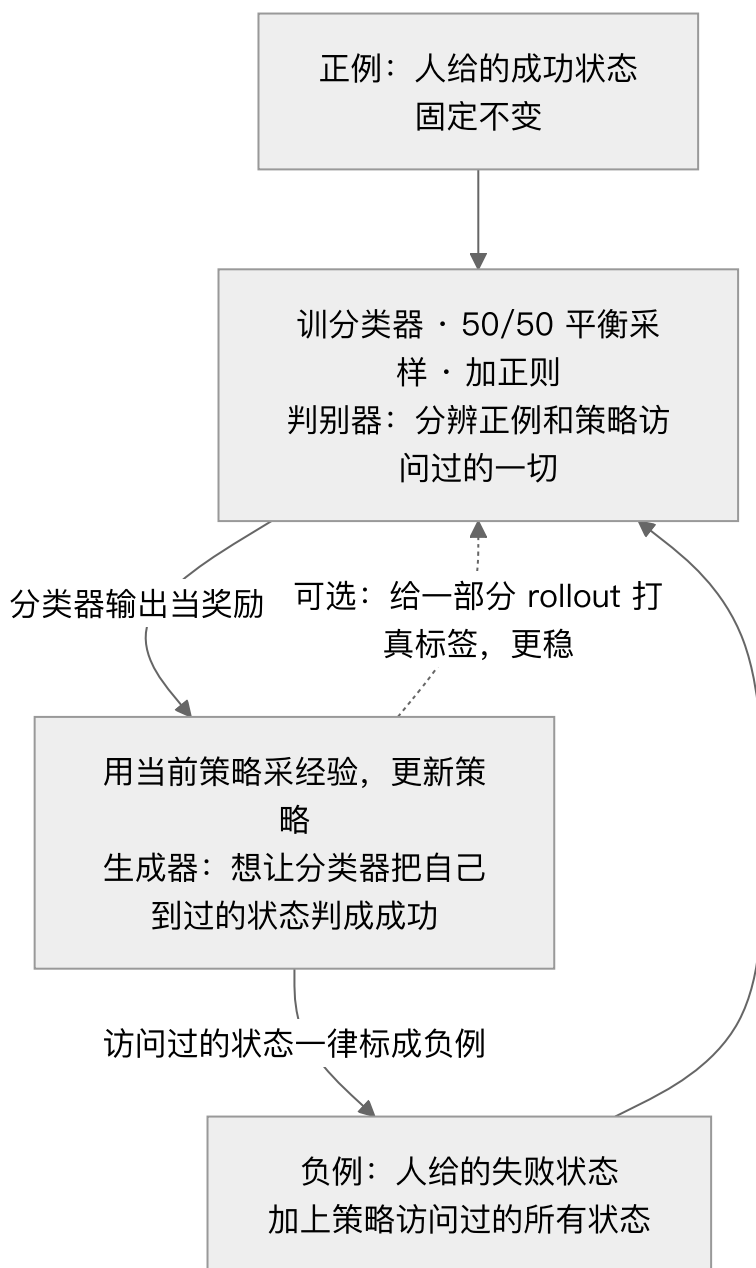


图 8-4 | 防钻空子的循环：策略访问过的状态回灌成负例、重训分类器——结构上就是一个 GAN，分类器是判别器、策略是生成器（自绘示意）· [▶ 看原幻灯片 33:32](#)
· 出处：Fu et al., 2018 (VICE；课上没点名，推断)

- **能不能防？** 讲者说恰好和 CQL 同一天讲，因为解法是同一个思路。画一个抽象的状态空间：正例聚在一处，负例在另一处，分类器学到一条决策边界；RL 训着训着跑到边界外某个区域——那里有的状态其实不错，有的很糟，分类器没见过、说不准。既然没有真奖励、也不想在线要人工标签，就**保守一点**：把策略访问过的这些状态**全部当负例**，回灌重训分类器；边界收紧，策略再往别处试，分类器在那些状态附近越来越准。

- **完整循环**：收集初始的成功 / 失败状态 → 训分类器 → 用当前策略采经验，用分类器奖励更新策略 → 把访问过的状态加进负例 → 重训分类器 → 重复。
- **问答里把这个算法盘了一遍**
 - 正例很少会过拟合：边界可能只圈住那几个正例点，旁边的都判负。——给分类器加正则（普通的正则化手段就有帮助），让它更平滑。
 - 负例越加越多：不做处理的话，分类器最后会到处预测负。——**必须 50/50 重新平衡**：采样时一半正例一半负例。原始负例会不会被采样负例淹没？可以也做平衡，看初始负例有多少。
 - 更要命的：策略会真的走到目标状态，甚至和正例一模一样，却被标成负例——标签明明是错的（**假负例**）。——讲者算了一笔账说明为什么只要平衡了就没事：设采样的一半是真正例，另一半里 40% 是假负例、10% 是真负例（含原始负例）。分类器对一个正例输出的"是正例"概率不会是 1，但一定**大于 0.5**、也大于"是负例"的概率；只有另一半**全部**是假负例时才恰好 0.5，只要还有一点真负例就严格大于。真负例的概率则小于 0.5。所以用作奖励时正负仍然分得开。
 - 那能不能干脆不做 0/1 阈值、直接用概率当奖励？——可以，也很正常。用置信区间？——也许有帮助，但需要更复杂的 RL 方法，讲者没细想过。
 - 遇到很难区分的状态？——有正则的话倾向于判正，因为正则鼓励函数平滑；具体看它和正例差多少。
 - 跑完一条轨迹之后我们看得到真实标签吗？——不，假设除了初始数据没有任何额外的人工监督。当然也可以给一部分 rollout 打标签来继续更新分类器，不必每条都标，这也能让训练更稳。
- **这不就是 GAN 吗？**（一个学生抢在两页幻灯片之前问出来）——对，这正是一个生成对抗网络：分类器是判别器，分辨"你给的正例"和"策略访问的一切"；策略是生成器。不需要懂 GAN 也能用这个算法，但懂的话会发现结构一模一样。GAN 原本是生成模型，常用来生成图像：训一个分辨真图和生成图分类器，再训生成器去骗它。

小注：GAN 出自 Goodfellow et al., 2014。把成功示例当正例、策略状态当负例来学奖励的这一套，应为 VICE (Fu et al., 2018, [Variational Inverse Control with Events](#))，它把任务写成"最大化目标事件发生的概率"。

- **一个机器人例子**（讲者组里的工作，“我们发现”）：50 条示范，取每条的末状态当成功正例；replay buffer（第 5 讲的经验池）用示范初始化。直接模仿这 50 条，成功率约 26%——50 条不多，再多收也很难到 100%；改用这样学出来的分类器做 RL，成功率 62%。不算惊艳，但比模仿高一大截。研究里发现的关键点也是：**分类器一定要正则化**，否则会过拟合那 50 个正例。
- **扩展**（时间不够，讲者跳过了）：不只学“目标分类器”，还能学完整的奖励函数——把示范里**所有**状态当正例、策略 rollout 的所有状态当负例，学到的就是对状态分布的奖励，而不只是对目标状态的。

小注：这条被跳过的路就是对抗式模仿 / 逆强化学习一族：GAIL (Ho & Ermon, 2016) 和 AIRL (Fu et al., 2018)。逆强化学习 (inverse RL) 问的是“找一个奖励，使示范在它下面是最优的”；对抗式做法把“示范 vs 策略”的判别器直接当奖励。本讲预告里说的“从示范学奖励”，实际只讲到这里。

- **目标示例这条路的小结**：框架很实用；对抗式训练可能不稳定，GAN 文献里的各种正则化技巧可以直接搬过来，很有用；最大的缺点是**需要示例**——有时造这些示例本身就难、耗时。

5. 从偏好学：比较比打分容易

- **换一种监督**：不只想到达某个目标集合，而是想区分“好行为”和“坏行为”，这可能不只是到没到某个状态的事；而且不想只靠示范或目标示例，想对**整条 rollout** 给反馈。
- **问哪个问题**：一种是“这条轨迹有多好”——从蓝点出发尽快到星星，让你给一条路径打分，很难。换一种问法：“这两条哪条更好”——蓝线还是粉线？全场都选蓝线。**相对偏好**比绝对评价容易得多，二维画线如此，很多真实任务也如此。
- **一条偏好告诉了我们什么**：如果人说 τ^w (winning) 比 τ^l (losing) 好，最自然的解读是：沿 τ^w 各步奖励之和应该大于沿 τ^l 的。一个二元标签只给一点点关于奖励的信息，我们要把很多这样的标签**汇总**成一个奖励函数。

$$\tau^w \succ \tau^l \implies \sum_t r(s_t^w, a_t^w) > \sum_t r(s_t^l, a_t^l)$$

τ^w 、 τ^l 是被判为更好和更差的两条轨迹， r 是要找的（未知的）逐步奖励； $>$ 读作“优于”。

- **写成可以优化的目标**（讲者一步步来；课上把奖励的参数也记作 θ ，本笔记改记 ψ ，免得和策略参数混淆）：
 1. 把奖励设成一个神经网络 $r_\psi(s, a)$ ，读一个状态和一个动作，输出一个标量；一条轨迹的奖励 $R_\psi(\tau)$ 就是沿途各步之和（沿用前几讲的简写）。
 2. 起点：希望 $R_\psi(\tau^w) - R_\psi(\tau^l)$ 是正数。
 3. 机器学习里我们喜欢概率、喜欢做最大似然，于是把这个差过一个 **sigmoid**，压到 0 和 1 之间，当作“ τ^w 确实优于 τ^l ”的概率。
 4. 在数据集的所有偏好对上让这个概率最大——对每一对，各自求奖励和、相减、过 sigmoid、取 log 再求和，用梯度下降优化 ψ 。

$$P_\psi(\tau^w \succ \tau^l) = \sigma(R_\psi(\tau^w) - R_\psi(\tau^l)), \quad R_\psi(\tau) = \sum_t r_\psi(s_t, a_t), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

σ 是 sigmoid（讲者在问答里现场画了它的 S 形曲线）；两条轨迹的奖励和差得越多，模型认为“ τ^w 更好”的概率越接近 1；差为 0 时恰好 0.5。

$$\max_{\psi} \sum_{(\tau^w, \tau^l) \in \mathcal{D}} \log \sigma(R_\psi(\tau^w) - R_\psi(\tau^l))$$

$\mathcal{D}$ 是偏好对的数据集；这就是最大化“标注结果在模型下的对数似然”，取负号就是训练用的 loss。

小注：这就是 Bradley-Terry 模型（1952），CME295 第 5 讲的奖励模型 loss 是它在 LLM 上的特例——那里 R 是奖励模型给整条回答的一个分数，这里是对整条轨迹逐步奖励求和的一般形式。课上没有点 Bradley-Terry 的名字。CS329A 第 3 讲的 ORM 也是同一个东西。

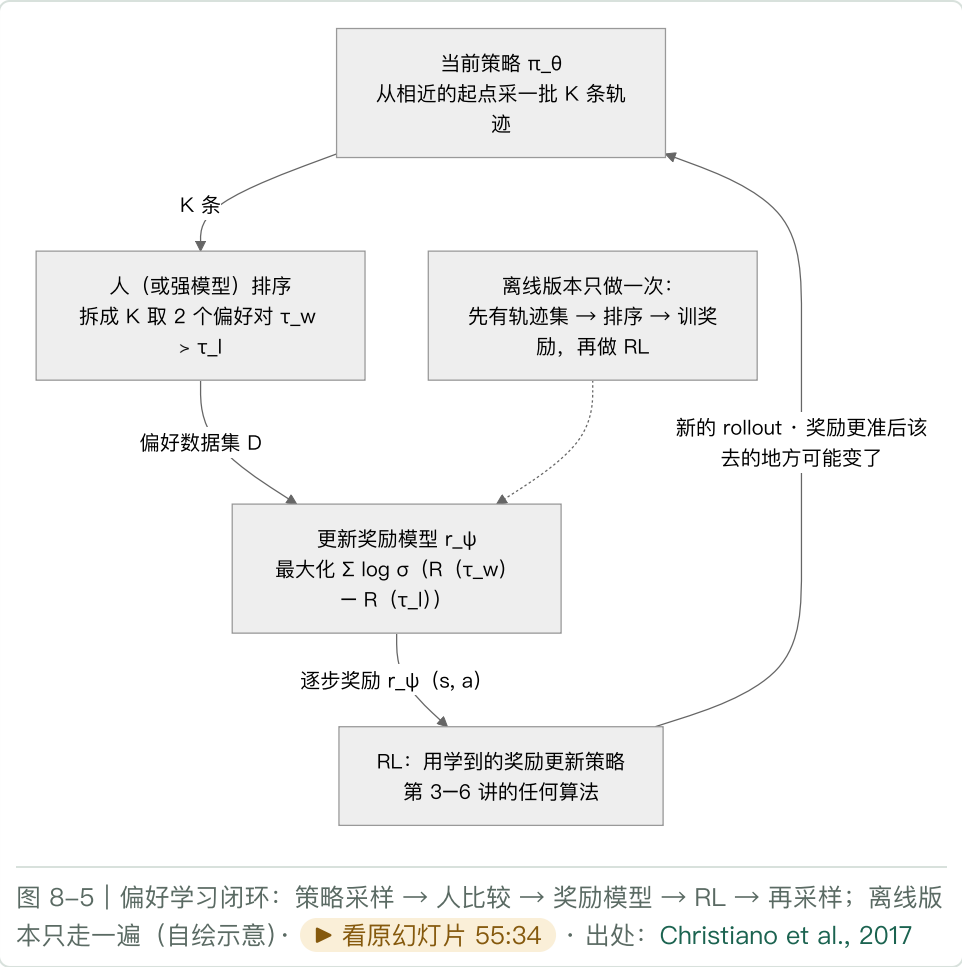
- **不必是整条轨迹**：从 Palo Alto 开到旧金山、把厨房收拾干净，这样的轨迹里有很多子段，可能有的段好、有的段差。所以也可以拿**部分轨迹**来比：“这段比那段好”。

小注：Christiano et al. 正是这么做的：让标注者比较两段等长的短片段（clip），而不是整条 episode。

- **问答：人只说了谁好，没说好多少**。如果三条轨迹一条很好、一条差一点、一条差很远，这个目标不区分“好一点”和“好很多”。——两点回应：其一，数据多了之后，成对偏好会隐含一个排序（A 优于 B、B 优于 C……），排序会**迫使**奖励函数给最好的那条明显更高的分，想要的效果自然就出来了；其二，实际收集时经常直接给标注者一批轨迹让他们**排序**，信息更显式。

- **完整算法**：有一个轨迹数据集 → 采一批 K 条 → 请人排序 → 在当前奖励模型下算每条的奖励（前向一遍、沿途求和） → 对排序里的每一对（共 K 取 2 对）算上面目标的梯度，反向传播 → 更新 ψ → 重复。重复这件事就是在一个固定数据集上做梯度下降。
- **同一起点**：给 LLM 学奖励时通常用同一个 **prompt** 采 K 条回答再排。原因是这些轨迹起点相同才好比：一条从别的起点出发的轨迹可能到得更快，但它起点本来就容易，比较不公平。所以采样时尽量让一批轨迹起点相近。

6. 把偏好学习放进 RL 循环：backflip、驾驶与 RLHF



- **在环学习**：第 5 节是先有数据集再训奖励。也可以把它放进 RL 的循环：策略采样 → 请人排序 → 更新奖励 → 奖励更准了，策略该去的地方可能变了（讲者的例子：本来往这边走，现在应该往那边） → 更新策略 → 新 rollout → 再问人 → 再更新奖励 → 重复。
- **三个例子**
 - **仿真里的 backflip**（指定阅读 Christiano et al., 2017）：给一个仿真小人写“后空翻”的奖励几乎不可能；他们在 RL 循环里只向人问

了 900 次"这个比那个好吗", 学出了会后空翻的策略, 顺带学出了一个"人类偏好的后空翻"奖励函数。

小注: 论文摘要的说法是: 人只对不到 1% 的环境交互给了反馈; 训练这类新行为大约需要一小时的人工时间。

- **驾驶**: 学"保持车道、保持某个速度、避免碰撞"各占多重——可以想见避碰的权重远大于保持速度。

小注: 应为 Sadigh et al., 2017 (RSS), Active Preference-Based Learning of Reward Functions: 奖励是几个手工特征的线性组合, 用主动挑选的成对比较学权重。

- **语言模型**: prompt 是"怎么煎蛋卷", 从模型采两个回答, 问人哪个更好。训练时会花钱请标注员; 你用 ChatGPT 时它偶尔弹出的"这两个回答哪个更好", 就是在给 OpenAI 攒偏好数据。有了偏好就能训一个**奖励模型**, 判断回答 y 对 prompt x 有多好。

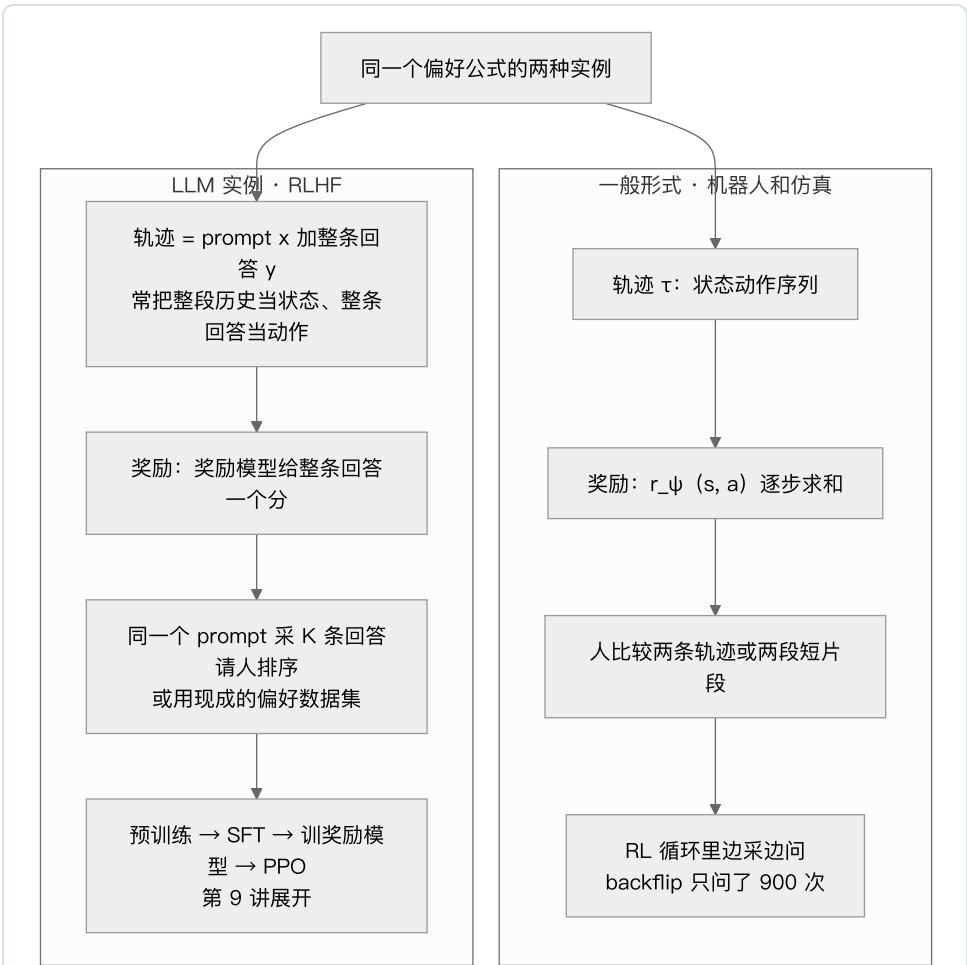


图 8-6 | 同一个偏好目标的两种实例: 机器人的逐步奖励求和, 与 LLM 的整条回答打分 (自绘示意) · [▶ 看原幻灯片 58:42](#) · 出处: Ouyang et al., 2022 (InstructGPT; 课上没点名, 推断)

- **LLM 的完整流水线**（CME295 第 4—5 讲讲过用法，这里是它在 RL 课里的位置）：
 1. **预训练**：在海量、质量参差的数据上预测下一个 token。这一步之后模型知道很多，但不好控制——不一定回答问题、不一定照你说的做，只是接着预测合理的下一个词。
 2. **监督微调（SFT）**：造一份高质量的“这个 prompt 应该这样答”的数据集，在上面做**模仿学习**（第 2 讲）。
 3. **人类反馈**：先收偏好数据（也可以用现成的偏好数据集，不一定非得用自己的模型采）→ 训奖励模型 → 用 RL 最大化它，常用 PPO（第 4 讲）。下周三的课（第 9 讲）专门讲偏好学习。

小注：这条三步流水线应出自 InstructGPT (Ouyang et al., 2022)。第 9 讲的客座讲者 Archit Sharma 是 DPO 的作者之一，会讲怎么把第三步的两阶段并成一个。

• 问答

- x 只是 prompt 还是整段对话历史？——通常是整段历史。把 LLM 写成 RL 问题有两种粒度：按 token 的序列决策；或者更粗糙但更简单的——把整段历史当状态、把下一整条回答当要优化的动作。
- 排序怎么变成公式里的概率？—— K 条的排序拆成 K 取 2 个对，每对都有“谁优于谁”， τ^w 记优选的、 τ^l 记劣选的；采一对代进公式。
- σ 是什么？——sigmoid，把实数压到 0 到 1 之间的那条 S 形曲线。
- **RLAIF**：偏好也可以不问人、问一个强模型：“这两个哪个更好？”而且“更好”有很多维度，可以指定问你关心的那一维——比如把两条回答喂给模型，问哪条**更无害**，用来阻止模型产出有害内容。背后的洞见：**批评比生成容易**，所以用 AI 反馈有机会得到比评判者本身更强的模型；已有工作显示这种反馈在多个维度上都能带来提升。

小注：无害性这个例子和 RLAIF 这个词应出自 Constitutional AI (Bai et al., 2022)。CME295 第 5 讲把它列为 RLHF 的对照项；这里讲者给出了它成立的前提——评判者的判断要比被评判者的生成更可靠。

7. 小结与两条延伸

- 奖励不能当成现成的。两条路对比：

	从目标示例学	从偏好学
要人提供什么	成功（和失败）状态的例子	成对比较或一批排序
好处	很实用，标签便宜	不需要示范、不需要目标示例；比较极易提供；在 LLM 上已大规模部署
代价	对抗式训练难优化；示例有时难造	可能需要人在 RL 循环里持续给监督
被钻空子时怎么办	把策略访问的状态回灌成负例	边采边问，让奖励模型跟着策略的分布走

- **还有别的监督形式：**目标和偏好只是两种反馈。人还可以用语言说"这个不错""这个有帮助但语气有点刻薄".....怎么把这些用起来，讲者认为是一个还有很多事可做的研究方向。
- **无监督 RL：**也可以不让人给目标，让 agent 自己提目标、自己好奇心。一个例子是两个 agent 博弈：一个负责出目标，另一个负责去达成，能涌现出有意思的行为。第 14 讲的探索会回到这个话题。

小注：应为 asymmetric self-play (Sukhbaatar et al., 2018)。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
IQL 三个动作的数字例子	$Q(s, a)$ 和实际落到的 $V(s')$ 对不上的三种情形：倒霉、走运、冒险动作没那么糟；由此看出非对称只能放在 V 上	3:42	IQL
CQL 版本一与下界曲线	横轴是动作、数据只覆盖中间一段；压低所有 Q 之后学到的 Q 整条在真 Q 之下	11:25	CQL
CQL 完整目标的板书	压数据外、拉数据内两项怎么写；问答里关于 Q 还是 Q' 的记号澄清	12:59	同上
μ 的闭式解与 log-sum-exp	加熵正则后 μ^* 正比于 $\exp Q$ ，惩罚项变成 $\log \Sigma \exp Q$ ， μ 从公式里消失	20:08	同上
LinkedIn 通知的 A/B 结果	基线、double DQN、加 CQL 三者的点击率、通知总数、活跃度	22:48	应为 Prabhakar et al., 2022
铅笔盒分类器	正例、负例各几张 → 二分类器 → 输出当奖励	28:25	—
决策边界被钻空子的手绘	策略跑到边界外分类器没见过的区域；把这些状态标成负例后边界收紧	31:25	应为 VICE
50/50 平衡的算账	一半真正例、另一半 40% 假负例加 10% 真负例时，正例的概率仍严格大于 0.5	37:08	—

图	看什么	跳转	出处
机器人 26% vs 62%	50 条示范、未状态当正例、示范初始化 replay buffer；模仿 vs 分类器奖励的 RL	41:12	—
蓝线 vs 粉线	给一条路径打分很难，两条里选一条全场秒答——相对偏好比绝对评价容易	45:46	—
偏好目标的推导板书	奖励和之差 → 过 sigmoid 当概率 → 在所有偏好对上最大似然	49:54	Christiano et al., 2017
排 K 条的完整算法	采 K 条 → 人排序 → 当前奖励模型前向求和 → K 取 2 对的梯度 → 更新；LLM 里用同一个 prompt	54:02	同上
backflip	900 次询问，在 RL 循环里同时学出策略和奖励	56:36	同上
LLM 三步流水线	预训练 → SFT（模仿学习）→ 偏好数据、奖励模型、RL 三小步	58:42	应为 InstructGPT

提到的工作

名称	在本讲里的作用
IQL (Kostrikov et al., 2021)	第 7 讲的主角，本讲补讲为什么非对称 loss 只用在 V 上
advantage-weighted regression (第 7 讲)	讲者说明"为什么还要学 Q"时举的策略提取方法
CQL (Kumar et al., 2020)	本讲前半的主角：压低分布外 Q 值的保守目标；下界证明在论文里
Double DQN (第 6 讲)	LinkedIn 例子的底座算法，加上 CQL 的惩罚项
LinkedIn 通知优化 (应为 Prabhakar et al., KDD 2022)	CQL 的真实世界应用：离线数据学策略，线上 A/B 验证
成功分类器当奖励 (应为 VICE, Fu et al., 2018)	从目标示例学奖励；把策略访问的状态回灌成负例
机器人 50 条示范的实验 (讲者组里的工作，课上没给题目)	模仿 26% vs 分类器奖励的 RL 62%；分类器要正则化
GAN (Goodfellow et al., 2014)	分类器 = 判别器、策略 = 生成器；正则化技巧可以直接搬
GAIL / AIRL (应为)	被跳过的扩展：示范全部状态当正例，学完整奖励函数
Deep RL from Human Preferences (Christiano et al., 2017)	指定阅读；backflip 900 次询问；在 RL 循环里学偏好奖励
驾驶偏好 (应为 Sadigh et al., 2017, RSS)	学车道、速度、避碰各占多重
ChatGPT / InstructGPT (应为)	LLM 三步流水线：用户点选"哪个回答更好"就是在攒偏好数据

名称	在本讲里的作用
PPO（第 4 讲）	RLHF 第三步里常用的 RL 算法
RLAIF / Constitutional AI（应为）	用强模型代替人给偏好；无害性例子；"批评比生成容易"
无监督 RL、目标提议者 vs 达成者（应为 asymmetric self-play）	收尾的延伸：不靠人给目标
第 9 讲预告（客座 Archit Sharma）	偏好学习与 DPO 下周三展开

术语对照

English	中文
offline RL	离线强化学习：只用一批固定数据学策略，训练中不与环境交互
behavior policy π_β	行为策略：采集离线数据的那个未知策略
distribution shift	分布偏移：要学的策略想做的动作，数据里未必有
overestimation	高估：分布外动作的 Q 值被随机地估高，策略奔着它去
support of the data	数据支持：数据真正覆盖到的状态动作范围
in-distribution / out-of-distribution (OOD)	分布内 / 分布外
critic / critic loss	价值网络及其拟合 loss（第 4 讲）：以 TD 目标做回归
asymmetric loss / expectile	非对称 loss / 分位期望回归（第 7 讲）：向大的一侧倾斜
conservative / pessimistic	保守 / 悲观：分布外一律往低估
lower bound	下界：学到的 Q^* 不高于真 Q
entropy regularizer	熵正则：鼓励分布更"散"，别坍缩成一个点
log-sum-exp	对数-求和-指数：最大值的平滑版本
stitching	拼接：把数据里不同轨迹的好片段接成更好的行为
A/B test	线上对照实验
task specification	任务指定：把"想要什么"写成算法能优化的目标
proxy reward	代理奖励：和真正目标相关但不等于它的可测量量
reward hacking / exploiting the classifier	奖励投机：分数高了，任务没做
goal set / success classifier	目标集合 / 成功分类器：判断一个状态算不算成功

English	中文
sparse reward	稀疏奖励：只在目标附近才有信号
decision boundary	决策边界
class balancing	类别平衡：采样时正负各半
false negative	假负例：其实是正例却被标成负
regularization / smoothness	正则化 / 平滑：防止只圈住几个正例点
GAN / discriminator / generator	生成对抗网络 / 判别器 / 生成器
replay buffer	重放缓冲区（第 5 讲）：存经验供离策略更新，可以用示范初始化
preference / pairwise comparison	偏好 / 成对比较
winning / losing trajectory (τ^w / τ^l)	优选 / 劣选轨迹
trajectory return $R(\tau)$	轨迹的奖励和
sigmoid σ	S 形函数：把实数压到 0 到 1
maximum likelihood	最大似然
ranking / K choose 2	排序 / K 取 2 个对
in the loop	在环：奖励学习和 RL 交替进行
RLHF / RLAIFF	人类反馈 / AI 反馈的强化学习
SFT (supervised fine-tuning)	监督微调：文本上的模仿学习
reward model	奖励模型：判断回答 y 对 prompt x 有多好
unsupervised RL / curiosity	无监督 RL / 好奇心：agent 自己提目标

字幕勘误

"key value / key function / key values" → Q value / Q function（全程几十次，是本讲最大的读字幕障碍）；"auto distribution" → out-of-distribution；"mew" → μ (mu)；"RO algorithm" → RL algorithm；"behavior coding" → behavior cloning；"vpi / qpi" → V^π 、 Q^π ；"hati" → Q_ϕ (Q hat phi)；"some x" → sum-exp (log-sum-exp)；"subjective" → this objective；"genative adversarial" → generative adversarial；"adversarial trading" → adversarial training；"chat GBT" → ChatGPT；"PO" → PPO；"TOW W / TOW L" → τ_w 、 τ_l ；"towels" → taus (τ)；"directory" → trajectory；"dispered / disper" → dispreferred；"timeconuming" → time-consuming；"ed" → Ed（课程论坛）。

带走的问题

1. CQL 里"压"用的是专找高 Q 值的 μ ，"拉"用的是数据分布。如果把 μ 直接换成当前策略 π_θ ，闭式解还在吗？ α 调得太大或太小，学到的 Q^* 和最终策略分别会出什么毛病？
2. "把策略访问过的状态一律标成负例"和"把数据外动作的 Q 值压低"是同一种保守。它们各自付出的代价是什么？在你做的 agent 评测里，如果用模型自己的输出当负例去训一个判别式的 verifier（CS329A 第 3 讲），会不会撞上本讲 50/50 平衡和假负例的问题？
3. 偏好目标只约束奖励和的差。给 r_ψ 整体加一个常数 c ，两条等长轨迹的差不变，两条不等长的轨迹差却会变——这说明什么？用部分轨迹比较时应该怎么控制长度？
4. 在 RL 循环里边采边问（Christiano）和先收一大批偏好再训奖励模型（InstructGPT 式）：前者防的是什么、多付了什么代价？对照 CME295 第 5 讲的 on-policy / off-policy 和 DPO 的分布偏移，想一想这两种做法分别对应哪一边。
5. RLAIIF 依赖"批评比生成容易"。当被评判的策略比评判模型还强的时候，评判模型会不会变成新的被钻空子的对象？本讲的哪一招——负例回灌、在环监督、正则化——能移植到 LLM 的奖励模型上？

CS224R 第 9 讲 | RL for LLMs：偏好优化（客座 Archit Sharma）

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 9 讲，2025 年 4 月 30 日 · 客座讲座；第 8 讲讲的是从偏好里学奖励的一般方法，本讲把它落到语言模型上；开场说"推理部分由 Aviral 来讲"，应指第 10 讲（Aviral Kumar） 视频：<https://www.youtube.com/watch?v=XKLGuwvSKvI>（1:02:51，英文字幕是自动生成的，DPO、KL、Gumbel 这类专名错得多，见文末勘误） 讲者：Archit Sharma（客座；视频简介写的是 Gemini 团队研究员、DPO 论文第一作者——论文实为 Rafailov、Sharma、Mitchell 三人共同一作。课上讲到 DPO 时用的是"我们当时想....."的第一人称） 课程主页：<https://cs224r.stanford.edu/> · 指定阅读：Direct Preference Optimization: Your Language Model is Secretly a Reward Model (Rafailov et al., 2023)

一句话：预训练只教模型"续写文本"，不教它"帮人"。变成助手要三步：instruction fine-tuning 用指令—回答对教会格式，但它标注贵、开放任务没有标准答案、而且只有"对 / 错"没有"好一点 / 差一点"；RLHF 改问人"两条回答哪条更好"，用 Bradley-Terry 把偏好对训成一个打分的奖励模型，再用策略梯度让 LLM 追高分，并加一个 KL 惩罚把它拴在起点模型附近，防止它钻奖励模型的空子。DPO 是讲者自己的工作：带 KL 惩罚的目标有闭式最优解，奖励可以反写成"策略相对起点模型的对数概率比"，代回 Bradley-Terry 后算不出来的配分函数 $Z(x)$ 恰好抵消——于是奖励模型和整个采样 RL 循环一起省掉，一个二分类 loss 直接在偏好对上更新 LLM。收尾讲这套范式的裂缝：偏好噪声大、奖励模型准确率只有六七成、reward hacking、以及优化短期偏好带来的谄媚。

时间轴

时间	内容
0:06	开场：模型与算力越做越大；今天回答"预训练模型怎么变成聊天助手"：IFT、RLHF、DPO 三种技术
1:07	预训练学到什么：next-token 预测顺带学到事实、句法和一点世界知识；但语言建模不等于帮人：GPT-3 的登月续写
5:13	Instruction fine-tuning：指令—回答对；越大的模型增益越大；微调前后的对比
8:50	IFT 的四个局限；问答：IFT 教的主要是格式，知识来自底座；指令数据早已不限于问答

时间	内容
12:54	RLHF 的设定：摘要任务、指令 x 与样本 y 、人给的奖励；期望奖励最大化；InstructGPT 三步
15:25	为什么要 RL：奖励不可导；policy gradient 复习——log-derivative trick 与 Monte Carlo 估计
21:04	人打分不校准，改问偏好；Bradley-Terry 奖励模型；问答：分差无界要归一化、为什么用 sigmoid
25:39	RLHF 循环；难题：奖励模型明明可导，为什么还要 RL——离散 token 的采样瓶颈；Gumbel-softmax
29:45	奖励模型不完美，加 KL 惩罚折进奖励；摘要任务上 RL 首次超过人写的摘要
32:47	问答：偏好不可传递与噪声；奖励模型准确率只有 65—70%；能不能主动挑 prompt
35:22	DPO 的动机：RLHF 流水线又贵又难调；为什么不能从偏好直接优化 LLM
38:33	推导：KL 约束目标的闭式最优策略 → 把奖励反写成策略 → 黑板上 $Z(x)$ 抵消 → DPO loss
45:21	读 DPO loss；问答：KL 去哪了、“你其实还是在训奖励模型”、还要不要 RLHF、两处采样、偏好强度
50:57	DPO 的结果；本节小结：RLHF 与 DPO 各自的取舍
53:31	InstructGPT 的例子；DPO 在开源社区；偏好优化后的风格差异；问答：短期与长期偏好、排序数据怎么用
58:15	GPT-4o 谄媚事件；reward hacking；rubric 打分；展望：可验证奖励、个性化、幻觉

核心内容

1. 这讲要回答的问题：预训练模型怎么变成助手

- **背景：**过去五到十年最稳定的趋势是模型越做越大、算力越砸越多，甚至有人据此预测 AGI 2027。但本讲的问题不是“再大一点”，而是：一个被训练来预测下一个 token 的模型，怎么变成聊天助手？
- **预训练学到什么**（讲者声明这不是 NLP 课，只快速过一遍）：模型在海量网络文本上学“下一个 token 是什么”。补全“Stanford University is located in ___, California”要记事实；续写一段 Iroh 和 Zuko 的故事要懂句法和指代；接上“Pat 在真空舱里看保龄球和树叶同时落下，作为物理学家的 Pat 预测……”要懂一点物理。结论：next-token 预测学到的远不止语言，还有某种“世界怎么运转”的知识。
- **但“会建模文本”不等于“会帮人”：**GPT-3 拿到“用几句话给六岁小孩解释登月”，续写出来的是“用几句话给六岁小孩解释万有引力……解释相对论……”——在它见过的语料里，这种句子后面往往跟着更多同款

题目。它只是在预测文本，没有在回答问题。这道落差就是本讲要填的。

- **今天的三种技术**：instruction fine-tuning、RLHF、DPO，最后谈这套范式还缺什么。

小注：本讲前半场的例子（Zuko、真空保龄球、登月、地震摘要 8.0 对 1.2、policy gradient 的三行推导、 p^{PT} 与 RM_ϕ 的记号）应与 Stanford CS224N 讲 RLHF 那一讲的课件同源（推断），想找幻灯片可以去那门课的网站。

- **和前两门课的关系**：CME295 第 4 讲讲过预训练与 SFT，第 5 讲讲过 RLHF、PPO、DPO 的用法。本讲真正新的东西在第 4 节和第 7 节：为什么绕不开策略梯度，以及 DPO 是怎么从 KL 目标一步步推出来的。

2. 第一步：Instruction fine-tuning，以及它的四个局限

- **范式**：先预训练，再针对任务微调（fine-tuning）——收一批输入和目标输出，最大化目标输出的对数似然。早期是单任务，比如影评情感分类；后来发现预训练模型什么都能干，就把任务数堆上去：收集大量“指令—回答”对（“氮的沸点是多少？——320.4”），训完在没见过的任务上也能答（“Geoffrey Hinton 能和 George Washington 对话吗？先给理由再回答”）。
- **规模效应**：同一份指令数据，模型越大，微调后的增益越大（BIG-Bench、MMLU 上的表）；很小的数据就能带来很强的泛化。微调前后对比：问“下面句子里代词指代谁、是否有歧义”，原始模型接着往下编句子，微调后才真正回答。

小注：这几页应出自 FLAN (Wei et al., 2021, 氮沸点与 Hinton 的例子) 和 Flan-PaLM (Chung et al., 2022, “越大的模型增益越大”的表)；讲者没点名。

- **四个局限**（讲者先让大家自己想）：
 1. 标注贵。题越难，能写出标准答案的人越少，逐题请人写答案没法规模化。
 2. 很多任务没有“正确答案”。LLM 的大量用途是创作类的——“写一个关于狗和它的宠物蚱蜢的故事”没有唯一正解。
 3. 所有错误一视同仁。IFT 只有一个预定义答案，偏离它的每个 token 受同样的惩罚，没法表达“这句大体对，只是把 Avatar 说成

奇幻剧不如说成冒险剧好"这种程度差别。更麻烦的是，模型写答案已经比人好，靠人写标签就被人的水平封顶。

4. 目标错位。我们想要的是模型理解意图、和人互动得好；训练目标却只是"把这条答案预测准"。

- **问答**

- IFT 的收益来自学格式还是学知识？——多数证据说主要是格式：故意用错误答案做指令微调，下游还是不错的，知识基本来自底座。
- 指令数据只能是问答格式吗？——早期是，现在极其多样，函数调用、工具使用都在里面。

- **和第 2 讲的关系**：IFT 就是行为克隆——最大化示范数据的似然，误差一视同仁。CME295 第 4 讲讲怎么做 SFT，这里讲为什么它不够。

3. RLHF 的设定：把 RL 的词翻到 LLM 上

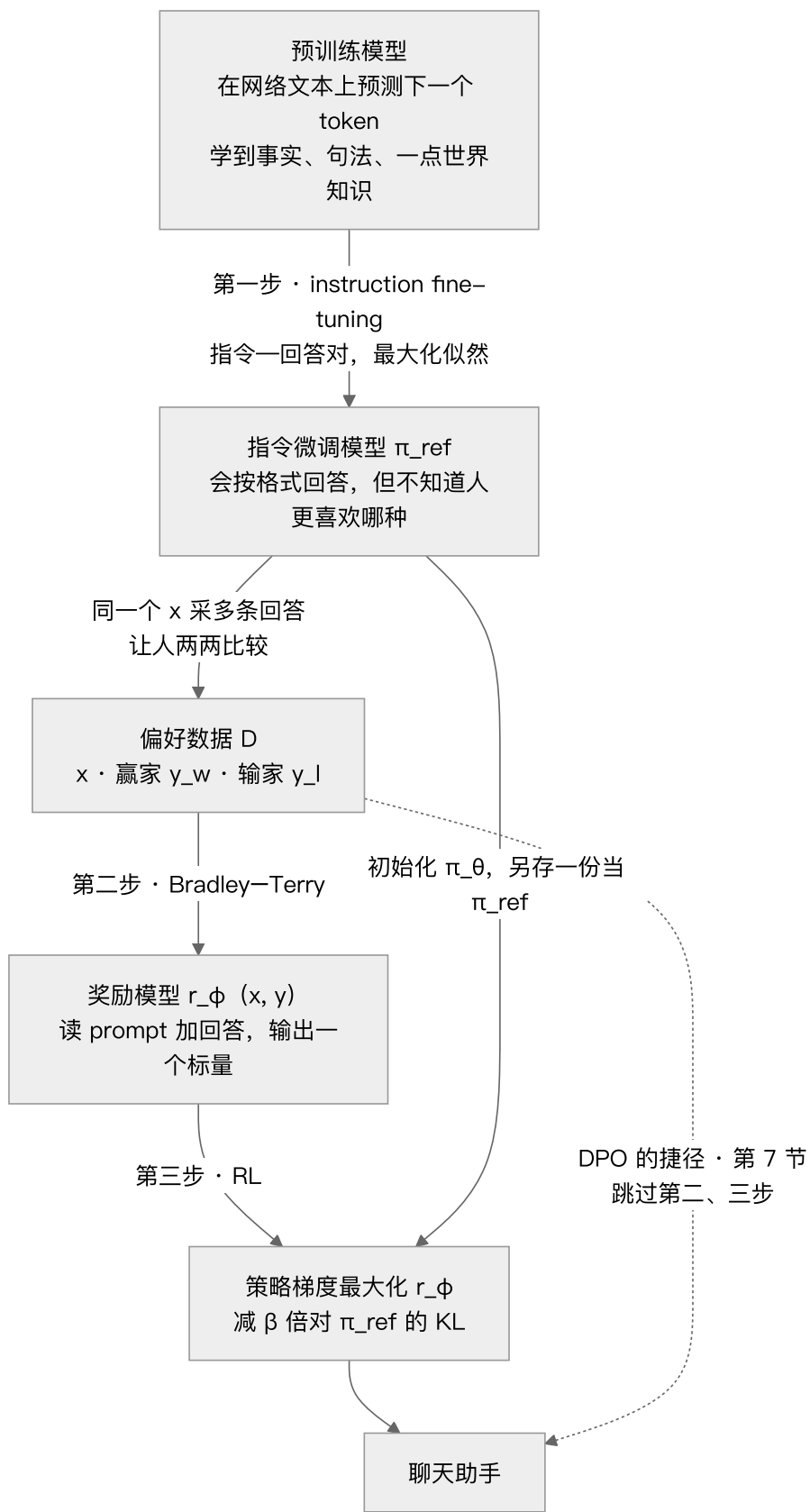


图 9-1 | InstructGPT 式的三步流水线：指令微调教格式，奖励模型学偏好，RL 追分；DPO 是从偏好数据直通结果的捷径（自绘示意）· [▶ 看幻灯片 14:24](#) · 出处：Ouyang et al., 2022

- **贯穿本讲的例子是摘要**：没有唯一正确的摘要，正适合用偏好。记号：指令 (prompt) x ，模型生成的样本 y (课件写 $\hat{y}$)，先假设人能给 (x, y) 一个奖励分——地震新闻的两条摘要，说了地震和无人受伤的得 8.0 分，扯湾区天气和山火的得 1.2 分。
- **目标**：给定 x ，让 LLM 生成的 y 平均得分最高——就是这门课第 1 讲写下的 RL 目标。

$$\max_{\theta} J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} [r(x, y)]$$

x 是从 prompt 集合 $\mathcal{D}$ 里抽的指令， y 是当前策略 π_{θ} (参数为 θ 的 LLM) 对它生成的整条回答， $r(x, y)$ 是这条回答得到的奖励；目标是调 θ ，让自己生成的回答平均得分最高。课件里策略写作 p_{θ} 、样本写作 $\hat{y}$ ，本文统一用 π_{θ} 和 y 。

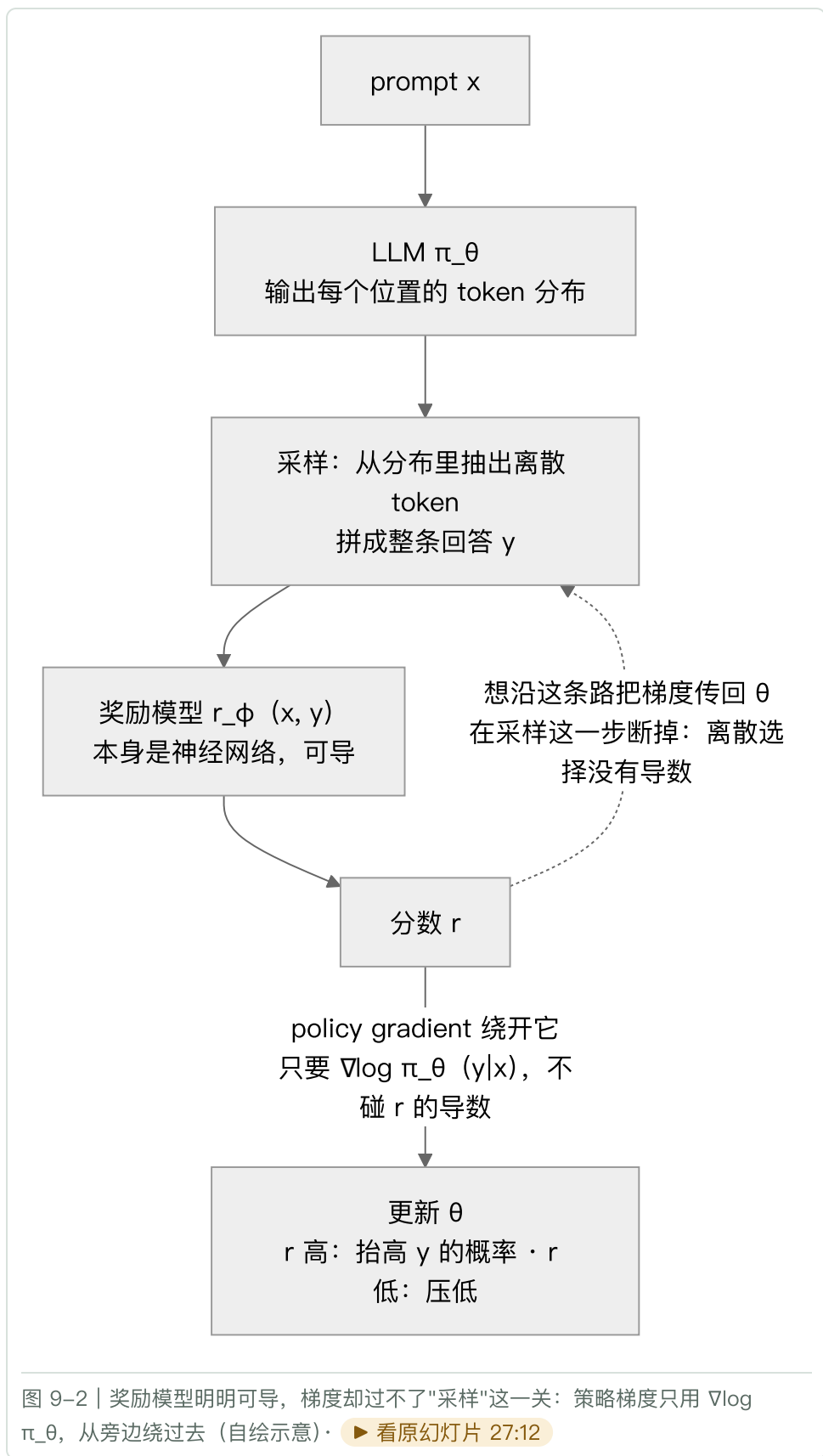
- **RL 的词在这里各指什么** (这门课前几讲的记号，一一对上)：

RL 里的词	本讲里的对应物
state (状态)	prompt x
action (动作)	整条回答 y 。问答里讲者确认：采样的是整句，不是逐个 token
policy π_{θ} (策略)	LLM 本身：给定 x 生成 y 的概率 $\pi_{\theta}(y x)$ ，等于逐 token 概率的连乘
reward (奖励)	人 (或奖励模型) 对 (x, y) 的打分，整条回答只给一次
trajectory / episode (轨迹 / 回合)	一步就结束：读 x 、写 y 、拿分。第 1 讲的 MDP 在这里退化成单步决策，也就是 bandit
environment (环境)	几乎是空的：没有状态转移，只有打分

小注：CME295 第 5 讲把每个 token 当一个动作、整条回答当一条轨迹；本讲和 DPO 论文用的是"整条回答是一个动作"的 bandit 视角。两种视角在"奖励只在结尾给一次"这点上等价 (第 3 讲的 reward-to-go 讨论)，区别只在 value function 要不要按 token 算。

- **三步流水线** (InstructGPT, 图 9-1)：先用示范数据做 IFT 把格式调对；再训一个奖励模型，让它对齐人的意图；最后最大化奖励模型的分。后两步是本讲重点。

4. Policy gradient 复习：为什么绕不开 RL



- **第一个理由：**奖励往往不可导——让人给一个分数，人脑没法反向传播。策略梯度（policy gradient）方法的价值就在于它能优化“不可导奖励的期望”。讲者说 Chelsea 早已讲过（第 3 讲），这里只复习。

- **推导走三步。**要的是 $\nabla_{\theta} E[y \sim \pi_{\theta}][r(y)]$ (为省记号先不写条件 x)。第一步，把期望按定义写成对所有 y 的求和 $\sum_y r(y) \cdot \pi_{\theta}(y)$ ，由梯度的线性性， ∇ 可以移进求和号，落在 $\pi_{\theta}(y)$ 上。第二步，log-derivative trick：由链式法则 $\nabla \log \pi_{\theta} = \nabla \pi_{\theta} / \pi_{\theta}$ ，所以 $\nabla \pi_{\theta} = \pi_{\theta} \cdot \nabla \log \pi_{\theta}$ ，把这个恒等式代回去。第三步，求和式里重新出现了 $\pi_{\theta}(y)$ 这个权重，于是整个式子又是一个“在 π_{θ} 下的期望”——可以采样估计了：从 π_{θ} 采 m 条回答，算 $r \cdot \nabla \log \pi_{\theta}$ 的平均，就是 Monte Carlo 估计，插进任何梯度优化器。

$$\nabla_{\theta} \mathbb{E}_{y \sim \pi_{\theta}}[r(y)] = \sum_y r(y) \nabla_{\theta} \pi_{\theta}(y) = \sum_y r(y) \pi_{\theta}(y) \nabla_{\theta} \log \pi_{\theta}(y) = \mathbb{E}_{y \sim \pi_{\theta}}[r(y) \nabla_{\theta} \log \pi_{\theta}(y)] \approx \frac{1}{m} \sum_{i=1}^m r(y_i) \nabla_{\theta} \log \pi_{\theta}(y_i)$$

三个等号分别是：期望的定义、log-derivative trick、再把求和认回期望；最后的 $\approx$ 是用 m 条采样回答做的 Monte Carlo 估计。 r 只以数值形式出现，从头到尾没有对它求导。

- **为什么要费这番周折：**对句子求期望不可能枚举——每个位置的词表十万量级、长度上千——必须有一个能采样的随机估计量，这个 trick 恰好把“梯度”变回“期望”。
- **直觉：**奖励高的样本，抬高它的对数似然；奖励低的样本往反方向推——好的动作多做、坏的少做，这就是 reinforcement 一词的来历。第 3 讲把它叫 REINFORCE，还讲了减 baseline 降方差；本讲不谈降方差，CME295 第 5 讲的 PPO 就是这个估计量加上 advantage 和 clip 之后的样子。
- **问答：为什么“必须”这样求梯度？**——两个理由：奖励不可导；更精确的数学理由是 θ 在期望的采样分布里，梯度必须穿过采样过程。奖励若可导，完全可以设别的目标端到端反传。
- **讲者的难题**（他说这题“有点难”，学生答得接近）：奖励模型是神经网络，可导，那为什么还要 RL？答案是离散瓶颈（图 9-2）：LLM 输出的是采样出来的离散 token，从奖励模型出发的梯度到“采样”这一步就断了。如果 y 是连续变量，直接反传把分数推高就行，还省掉昂贵的采样；但 y 是几千个离散 token。有学生提 Gumbel-softmax，讲者说这类“让采样可导”的技巧没有在这种规模上成功：一条回答是“十万词表 $\times$ 序列长度”的组合空间，Gumbel-softmax 通常撑不过千维。教训：别默认必须用 RL，先问梯度能不能直接传。

小注：Gumbel-softmax (Jang et al., 2016; 同期 Maddison et al. 的 Concrete distribution) 用“加 Gumbel 噪声再过带温度的 softmax”这个连续松弛替代离散采样，让梯度能穿过采样步，代价是有偏、且高维下方差大。

- **问答：加了奖励模型之后，什么变得可导了？——什么都没变。策略梯度只需要 $\nabla \log \pi_\theta$ ，从不往奖励里反传。**

5. 从偏好到奖励模型：Bradley-Terry

- **问题：**训最大的 LLM 需要百万级查询，不可能让一支"人类军团"给每条回答打分。解决：把"学人的奖励"拆成单独的优化问题，先训一个奖励模型（reward model），再进 RL。
- **为什么不直接要分数：**不同人的刻度不同——"7 分"对两个人意思完全不同；同一个人中午吃了什么都会影响打分；直接要分数训出的奖励模型不好用。改问偏好：给几条摘要，问"这条比那条好吗"。数据形式是 (x, y_w, y_l) ， y_w 是赢家、 y_l 是输家，集合记作 D 。
- **Bradley-Terry 目标：**从偏好数据得到一个"隐含"的奖励分，要求赢家的分高于输家。

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \right]$$

r_ϕ 是参数为 ϕ 的奖励模型， σ 是 sigmoid，把分差映射成" y_w 赢 y_l 的概率"；最小化它就是最大化"观察到这些胜负"的对数似然。CME295 第 5 讲从最大似然一步步推过这个式子，第 8 讲在机器人轨迹上用的也是它——那里比较的是两段轨迹，这里比较的是两条回答。

- **问答里的两点，后面 DPO 要用**
 - 分数无界吗？要归一化吗？——目标只要求分差大：1000 与 10000、1 与 999 都能满足，没有任何东西定住尺度。所以实践里要做事后归一化，让不同 prompt 的分数在同一尺度上，否则 RL 阶段会很"诡异"。讲者特意标注：这是发展 DPO 要用到的关键观察——loss 只看分差。
 - 为什么用 sigmoid？——不是必须，别的函数也行；sigmoid 天然对应"这条赢那条的概率"，所以通行。
- **后面问答里的更多现实**
 - 偏好可能不可传递：A 胜 B、B 胜 C，却 C 胜 A。说不通，但真实数据里就是有，噪声很大，没有好办法。
 - 偏好学习本质是二分类。一般的二分类能到 90—95%，语言模型的奖励模型通常只有 65—70%——勉强好于随机，却已足以带来大幅提升。
 - 能不能主动挑模型"拿不准"的 prompt 去问偏好？——属于主动探索、最大化信息增益那一类，有零散工作，没有标准做法（第 14

讲)。

小注：Bradley–Terry 给每条回答一个标量分，胜率只由分差决定，天然可传递，所以它表达不了讲者说的循环偏好；直接建模两两胜率的"一般偏好模型"（如 Nash learning from human feedback, [Munos et al., 2023](#)）就是冲着这点去的。

6. RLHF 循环与 KL 惩罚：把它拴在起点附近

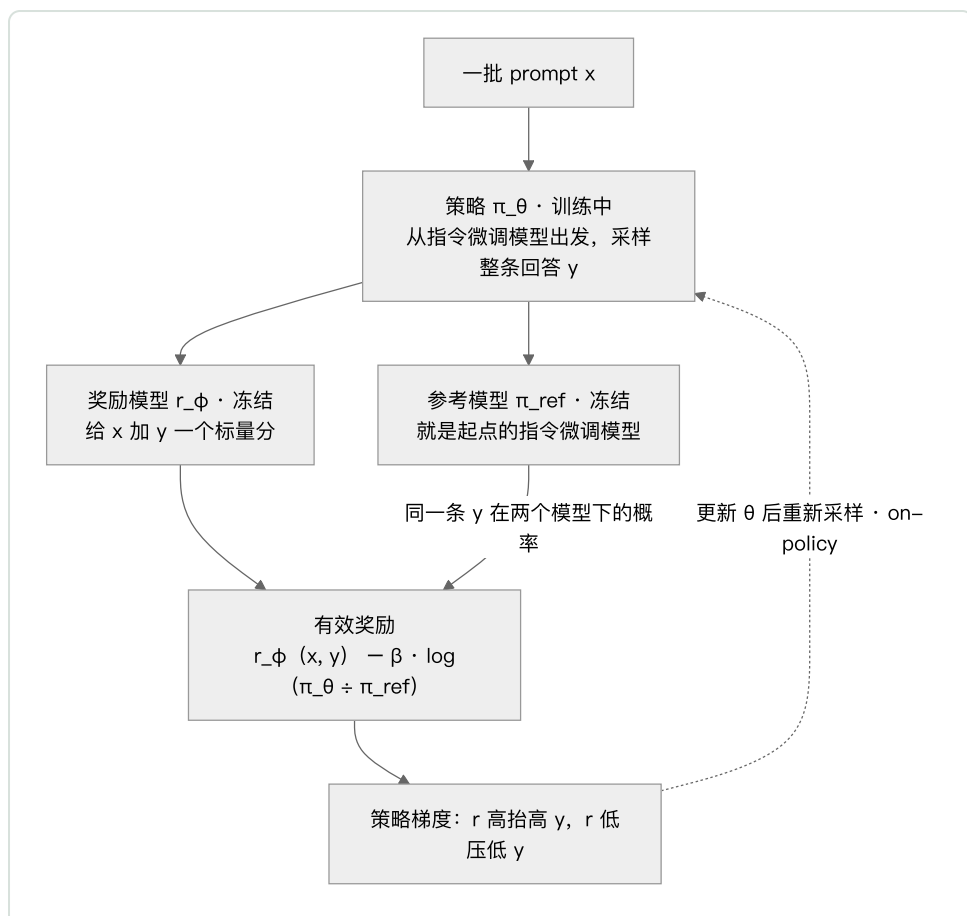


图 9-3 | RLHF 的一轮：策略采样，奖励模型打分，KL 项按“这条回答在策略下比在参考模型下概率高多少”扣分（自绘示意）· [▶ 看原幻灯片 30:15](#) · 出处：[Stiennon et al., 2020](#)（应为）

- **现在手里有：**指令微调模型（课件记作 p^{PT} ，讲者说它可以是指令微调过的，本文写 π_{ref} ）和训好的奖励模型 r_ϕ 。RLHF 循环就是把 r_ϕ 插进第 3 节的目标：从 π_θ 采样 y ， r_ϕ 打分，用第 4 节的策略梯度更新 θ 。它是 on-policy 的：数据必须来自当前策略，每更新一步都得重新采样（第 3 讲）。
- **但奖励模型永远不完美：**它是在某个偏好分布上拟合出来的，只在那附近可信；朴素地优化下去，会得到“钻奖励模型空子”的模型（reward hacking，第 10 节有图）。补救是 **KL 惩罚**：把 $-\beta$

$\log(\pi_\theta(y | x) / \pi_{\text{ref}}(y | x))$ 这一项加进奖励——当前模型给这条回答的概率比起点模型高得越多，扣得越多。对 y 取期望，这一项恰好是 π_θ 与 π_{ref} 之间的 KL 散度。第 5 讲预告过"带 KL 罚项的 PPO 变体在偏好优化里常用"，就是这里。

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)} \left[r_{\phi}(x, y) - \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} \right] = \max_{\theta} \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [r_{\phi}(x, y)] - \beta \text{KL}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)) \right]$$

π_{ref} 是起点的指令微调模型， β 是罚多重的旋钮；等号成立是因为 $\text{KL}(P \| Q)$ 的定义就是 $\mathbb{E}_{y \sim P}[\log P(y) / Q(y)]$ 。左边是课件的写法（KL 折进每条回答的奖励），右边是第 7 节推导用的写法。

- **KL 散度**是什么：衡量分布 P 偏离 Q 多少的量，恒大于等于 0， P 和 Q 相同时为 0，不对称。问答：某条 y 在 π_θ 下的概率小于 π_{ref} 时，这一项是正的奖励，会不会反而鼓励模型往那边走？——单条样本的项可正可负，但整体期望（KL）恒非负，讲者留作练习。

小注：证明用 Jensen 不等式（ $-\log$ 是凸函数），CME295 第 5 讲证过。工程上 InstructGPT 把这一项按 token 折进奖励、只在结尾加奖励模型的分，见那讲的小注。

- **早期最有说服力的结果**（摘要任务）：横轴模型大小，纵轴"摘要被人偏好于参考摘要的比例"。预训练模型随规模上升；SFT 更高；RL 那条曲线是第一条超过人写摘要的——很小的模型也能生成比人更受偏好的摘要，讲者说这是直接优化偏好威力的早期证据。

小注：应出自 Stiennon et al., 2020 《Learning to summarize from human feedback》，DPO 论文的摘要实验用的也是同一套 TL;DR 数据。

7. DPO（上）：从 KL 目标到一个分类 loss

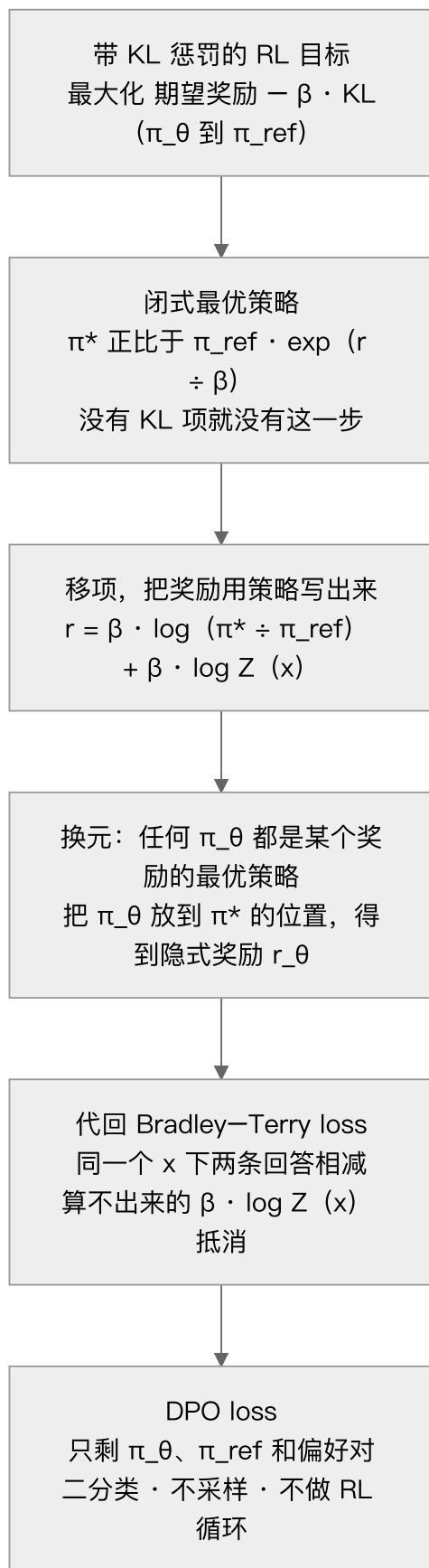


图 9-4 | DPO 的推导链：奖励和策略之间有一个可逆的换算，换过去之后奖励模型就多余了（自绘示意） · [▶ 看原幻灯片 39:36](#) · 出处：Rafailov et al., 2023

- **动机**：RLHF 在千亿、万亿参数上极贵——每条回答都要采样、送奖励模型打分，用 PPO 还要训 value function；讲者放了一张"专门用来吓人"的全流程图，零件越多越难调对。做 DPO 时他们的问题是：手里就是一个语言模型和一堆偏好，为什么不能直接在偏好数据上优化模型参数，非得先学一个中间量（奖励模型）？"direct"指的就是从偏好直达策略。
- **高层思路**：把奖励模型用语言模型自己表示出来。奖励模型是用一个简单的分类 loss (Bradley-Terry) 拟合到偏好上的；若能把两者连起来，就能拿语言模型的参数直接去拟合那个 loss。
- **推导**（讲者说"这段数学有点密，我慢慢讲"）。全程没有引入新假设，只是代数，下面按讲者的顺序分六步走。

第 1 步：起点是第 6 节的 KL 约束目标。此时 π_{ref} 和 r_ϕ 固定，只优化 θ 。

第 2 步：关键事实：这个问题有**闭式解**（能直接写出来、不需要迭代优化的解）。普通的期望奖励最大化没有这种东西，加了 KL 项才有。最优策略 π^* 是把 π_{ref} 按 $\exp(r / \beta)$ 重新加权：奖励高的回答概率被指数级抬高，低的压低； $Z(x)$ 是让它归一化成概率分布的常数，先只看分子。讲者说"这一步先信我，推导之后可以分享"。

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{r(x, y)}{\beta}\right), \quad Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp\left(\frac{r(x, y)}{\beta}\right)$$

π^* 是第 6 节目标的最优策略； $Z(x)$ 是配分函数 (partition function)，要对所有可能的回答求和，实际算不出来，但它只依赖 x 、不依赖 y 。

小注：补上讲者跳过的推导 (DPO 论文附录 A.1)。把目标除以 $-\beta$ ，改写成最小化 $E_{y \sim \pi} [\log \pi(y | x) - \log \pi_{\text{ref}}(y | x) - r(x, y) / \beta]$ ；后两项凑成 $-\log[\pi_{\text{ref}} \cdot \exp(r / \beta)] = -\log[\pi^*(y | x) \cdot Z(x)]$ ，于是被最小化的量等于 $\text{KL}(\pi \parallel \pi^*) - \log Z(x)$ 。 $\log Z(x)$ 与 π 无关，KL 恒非负且只在 $\pi = \pi^*$ 时为零，所以最优解就是 π^* 。没有 KL 项时，最优策略会把全部概率压到奖励最高的那条回答上，没有这样漂亮的形式。

第 3 步：两边取 $\log$ 、乘 β 、移项，奖励就能用最优策略表示。直觉检查：一条回答奖励高，它在 π^* 下的概率相对 π_{ref} 就高， $\beta \log(\pi^* / \pi_{\text{ref}})$ 也高；反之亦然。

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

奖励等于"最优策略相对参考模型把这条回答的概率抬高了多少"（取 $\log$ 、乘 β ），再加一个只和 x 有关的常数。

第 4 步：关键一跃（讲者称之为 the main argument）：这个关系不只对 π^* 成立。随便拿一个策略 π_θ ，它都是某个奖励函数的最优策略——把 π_θ 放进上式 π^* 的位置，得到的 $r_\theta(x, y) = \beta \log(\pi_\theta / \pi_{\text{ref}}) + \beta \log Z(x)$ 就是那个奖励。于是可以用策略的参数 θ 来参数化奖励模型：这是一次换元（reparameterization），不是近似。

第 5 步：把 r_θ 代回第 5 节的 Bradley–Terry loss。讲者到黑板上把上式写了两遍，一遍给 y_w 、一遍给 y_l ：两条回答共用同一个 x （同一篇文章的两条摘要），相减时 $\beta \log Z(x)$ 正好抵消。这就是一直让大家"先忽略 $Z(x)$ "的原因——它算不出来，但从不需要算。剩下的分差就是两个 $\beta \log$ 比值之差。

第 6 步：代进去，得到 DPO loss：

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

形状和第 5 节的奖励模型 loss 一模一样，只是奖励换成了隐式奖励 $\beta \log(\pi_\theta / \pi_{\text{ref}})$ ；这是一个二分类目标，直接对语言模型的参数 θ 求梯度，没有采样，没有 RL 循环。CME295 第 5 讲直接给了这个式子，这里是它的来路。

8. DPO（下）：怎么读这个 loss，以及课堂上的追问

- 训练时发生什么：**一开始 $\pi_\theta = \pi_{\text{ref}}$ ，所有隐式奖励都是 0，每一对的胜率预测都是 0.5。梯度只流过 π_θ ， π_{ref} 全程不动。loss 要让 $r_\theta(y_w)$ 升、 $r_\theta(y_l)$ 降，而抬高 r_θ 的唯一办法是抬高 $\pi_\theta(y_w | x)$ 、压低 $\pi_\theta(y_l | x)$ ——"好的上去、坏的下来"，和策略梯度的直觉一样，却不需要 RL。
- 问答：KL 惩罚去哪了？模型不会漂走吗？**——讲者说 DPO 其实做到了：loss 是 log-sigmoid，分差够大就饱和、梯度消失； β 控制"多小的分差就算够大"—— β 大，一点点偏离就饱和，参数几乎不再动。DPO 的 β 和第 6 节的 β 是同一个超参数，两边都管"允许离起点多远"。

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\theta) = -\beta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)) (\nabla_\theta \log \pi_\theta(y_w | x) - \nabla_\theta \log \pi_\theta(y_l | x)) \right], \quad \hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}$$

r_θ 是隐式奖励；后一个括号是"抬高赢家、压低输家"的方向，前面的 $\sigma(\cdot)$ 是权重：隐式奖励把两条回答排错了序时权重才大，排对且差距够大时趋于 0——这就是讲者说的"饱和"。

小注：这个梯度式课上没写，出自 DPO 论文第 4 节；它把" β 控制饱和和快慢"说得更精确：分差乘了 β 才进 sigmoid。

- **问答：DPO 的主要优点是不用训奖励模型？**——讲者纠正：你其实还是在训奖励模型，用的是同一个 loss。DPO 提供的是"奖励模型 $\leftrightarrow$ 它的最优策略"之间的连接，所以省掉的是从奖励模型走到最优策略的那个显式 RL 循环。奖励模型被巧妙地参数化成了语言模型本身，拟合奖励模型的同时就在优化语言模型。
- **问答：既然便宜，还要不要 RLHF？**——看情况。显式奖励模型有一个潜在好处：它可能泛化到偏好数据里没出现过的生成，给它们打分；所以取决于预算和目标。
- **问答：两处采样。**RLHF 里有两处采样：造偏好数据时采（人来标），RL 优化时采（奖励模型打分）。DPO 只去掉了第二处，第一处照旧。另外在线 RL 只需要 prompt、不需要偏好对，所以 RL 阶段用的 prompt 集通常比偏好数据的大——这是 DPO 放弃的东西之一。
- **问答：能不能利用偏好的强弱？**——RLHF 和 DPO 都只吃二元标签，是共同的局限。实践里让标注员给强度、只保留强偏好；怎么从噪声大、强度弱的偏好里学仍是未解问题。
- **结果：**论文证明了 DPO 目标与 KL 约束 RL 的等价；摘要任务上的胜率（和第 6 节一样的量法）与 PPO 相当或更好——讲者自己也说"这一点有争议"，但总体表现不错。
- **问答：多条回答排序怎么用？**——为效率通常一次排多条，拆成 $C(N, 2)$ 个偏好对；也可以用 Bradley-Terry 的推广 Plackett-Luce 直接在排序上训练，两者都行。

小注：Plackett-Luce 版的 DPO 在论文附录 A.3：排序的似然是一串 softmax 的连乘，同一个 x 下 $Z(x)$ 同样抵消。

9. RLHF 与 DPO 放在一起看：放弃了什么，留下了什么

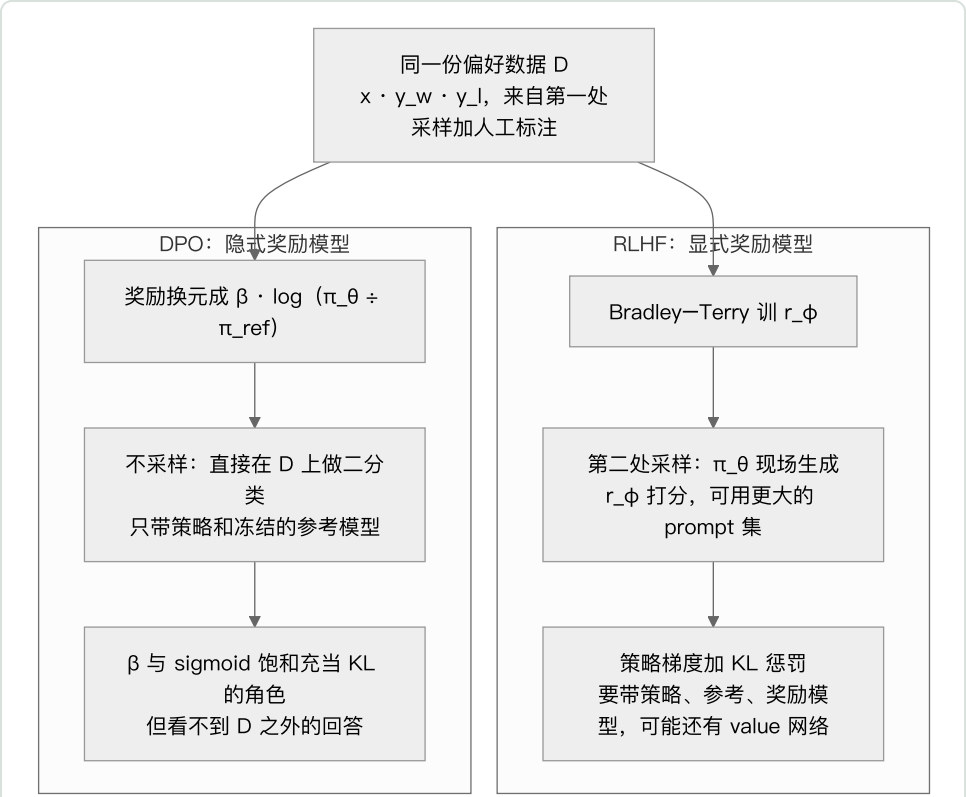


图 9-5 | 同一份偏好数据的两条路: RLHF 多一次采样和一个显式奖励模型, DPO 把奖励换元进策略后两者都省了 (自绘示意) · [▶ 看原幻灯片 37:31](#) · 出处: Rafailov et al., 2023

• **讲者的小结:** 优化偏好是因为它更贴近我们使用模型的方式; 不用各人刻度不同的分数, 而是收偏好, 再以某种形式学一个奖励模型——显式 (RLHF) 或隐式 (DPO)。RLHF 有时比 DPO 更强, 但难调对、因采样而昂贵; DPO 跳过采样, 把问题写成二分类, 简单有效、不需要在线步骤。

	RLHF	DPO
奖励模型	显式 r_ϕ , 单独训	隐式 $\beta \cdot \log(\pi_\theta / \pi_{ref})$, 和策略共用一套参数
采样	两处: 造偏好数据时、RL 时	一处: 造偏好数据时
训练时要带的模型	策略、参考模型、奖励模型 (PPO 还要 value 网络)	策略、参考模型
防漂移	显式 KL 项, 旋钮 β	同一个 β , 通过 sigmoid 饱和起作用
prompt 覆盖	RL 阶段只需要 prompt, 可以用更大的集合	只有偏好数据里的 prompt 和回答
泛化	奖励模型可能给没见过的回答打分	看不到 D 之外的回答
讲者的评价	有时更强; 难调、贵	便宜、稳, 开源社区一度全用它

- **后续采用**：DPO 发布后开源社区大量使用，便宜、性能好，一度所有开源 LLM 都用它做偏好对齐；Llama、Mistral 系列的后训练也用到 DPO。

小注：Llama 3 的后训练是 SFT、拒绝采样、DPO 轮流迭代 (Dubey et al., 2024)；Mistral 底座上 HuggingFace 的 Zephyr (Tunstall et al., 2023) 是最早靠 DPO 出圈的开源模型之一。

- **风格差异**：问“人们压力的五大来源”，只做过 IFT 的模型给一句话的短答案；经过偏好优化后，答案更长、带格式的列表——讲者强调这些是涌现出来的：模型发现人喜欢这种形式。这也埋下了下一节的问题：人喜欢的，未必是对人好的。
- **用这门课的词说**：DPO 只用现成的偏好数据、不与环境交互，是 off-policy 的（第 5 讲），甚至可以说是一种 offline 方法（第 7 讲）；RLHF 每轮重新采样，是 on-policy 的。CME295 第 5 讲说 PPO 调好了整体仍比 DPO 强、DPO 受分布偏移拖累，指的就是表里“看不到 D 之外的回答”这一行。

10. 这套范式的裂缝：噪声、谄媚、reward hacking 与展望

- **人的反馈能信吗？**（学生问：“人不知道自己要什么怎么办？”）讲者的回答：短期激励和长期激励错位——有些行为短期看着好，人按短期感受打分，长期却有害；这是 RLHF 流水线里真正难的问题。例子：GPT-4o 一次模型更新后变得明显谄媚 (sycophantic)，对用户的好坏主意一律附和，公认不健康。讲者声明自己不在 OpenAI、不评论，但强调“反馈数据怎样塑造模型的涌现行为”仍在被理解中。

小注：这件事就发生在本讲前几天：4 月 25 日推送的 GPT-4o 更新因谄媚被 4 月 29 日回滚；OpenAI 事后的说明提到，那次更新把用户点赞 / 点踩这类短期反馈加进了奖励信号——正是讲者说的短期激励。

- **公司会不会有意优化“参与度”而非人的利益？**——超出本讲范围。类比 YouTube 推荐系统曾让人看得比自己想要的多；但据讲者回忆，平台的真正激励是长期健康的参与——一天看 20 小时对产品也不好。真优化长期参与度，理论上不该落到 GPT-4o 那种短期谄媚。
- **Reward hacking**：对着学到的奖励模型训练，一开始确实更合人意，练得越久越偏离我们真正想要的——这是 RL 里“用代理奖励替代真实奖励”的通病，机器学习里也普遍。后果：训到某个点必须停，限制了 RL 的可扩展性；这是活跃的优化问题。

小注：那张图应为 Gao et al., 2022 《Scaling Laws for Reward Model Overoptimization》：横轴是策略离起点的 KL，代理奖励模型的分数一直涨，"金标"奖励模型的分数先涨后跌；奖励模型越大、数据越多，峰值来得越晚。

- **现状与展望**（讲者的清单）
 - 奖励信号在往 rubric（多维评分标准）走，按多个方面分别打分；仍需收集多种类型的数据教模型什么行为可取。不可能覆盖聊天机器人会遇到的所有情境，只能靠泛化，而泛化时常以无法预料的方式失效。CS329A 第 3 讲的验证器和 rubric 就在这条线上。
 - 偏好数据仍然昂贵，且未必代表我们想要的行为，需要更好的反馈形式。
 - 推理：模型自己生成、由可验证奖励打分——下一讲（第 10 讲）的主题；CME295 第 6 讲和 CS329A 第 6 讲的 GRPO 就在那条线上。
 - 个性化：现在的模型对齐的是"所有人的平均偏好"，但每个人是一对一使用的，需要更多关于用户的上下文。
 - 幻觉和失控的涌现行为仍是开放问题。
- **结尾**：课堂上的很多问题都是正在研究的问题，没有现成答案，鼓励大家自己去查。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
真空舱里的保龄球和树叶	要接对下一句就得懂点物理：next-token 预测顺带学到世界知识	2:39	—
GPT-3 的登月续写	语言建模和"帮人"的落差：把指令续写成一串同款指令	4:11	InstructGPT
指令微调的规模表	BIG-Bench、MMLU 上，模型越大微调后增益越大	7:19	应为 Chung et al., 2022
InstructGPT 三步流水线	示范数据 → 奖励模型 → RL；本讲的骨架	14:24	InstructGPT
策略梯度三行推导	展开期望 → log-derivative trick → 变回期望再采样	17:00	—
从打分到偏好	两条地震摘要；为什么不要分数要偏好；Bradley-Terry 目标	23:07	Bradley & Terry, 1952
可导的奖励模型与离散瓶颈	梯度从奖励模型往回传，在采样处断掉	27:12	—

图	看什么	跳转	出处
KL 惩罚折进奖励	$\beta \log(\pi_\theta / \pi_{PT})$ 这一项；为什么它的期望恰好是 KL	30:15	—
摘要任务的三条曲线	预训练、SFT、RL 随模型规模的偏好率；RL 首次超过人写的摘要	31:46	应为 Stiennon et al., 2020
闭式最优策略与反写奖励	π_{ref} 按 $\exp(r / \beta)$ 加权；移项得 $r = \beta \log(\pi^* / \pi_{ref}) + \beta \log Z$	39:36	DPO
黑板：Z(x) 抵消	对 y_w 、 y_l 各写一遍，同一个 x ，相减消掉算不出来的项	43:17	DPO
DPO loss 与结果	认出 Bradley–Terry 的形状；摘要胜率与 PPO 相当或更好	45:21 · 50:57	DPO
IFT 与偏好优化后的风格差异	"压力的五大来源"：一句话的短答案 vs 带格式的长答案，涌现出来的偏好	55:07	—
Reward hacking	对学到的奖励模型训得越久，越偏离真正想要的	59:46	应为 Gao et al., 2022

提到的工作

名称	在本讲里的作用
GPT-3 与 InstructGPT (Ouyang et al., 2022)	登月续写的反例；三步流水线；结尾的前后对比例子
Instruction fine-tuning: FLAN (Wei et al., 2021)、 Flan-PaLM (Chung et al., 2022 ；均为推断)	第一步：指令—回答对；越大的模型增益越大
BIG-Bench、MMLU	衡量指令微调收益的问答基准
Learning to summarize from human feedback (Stiennon et al., 2020 ；应为)	贯穿全讲的摘要任务；RL 首次超过人写摘要的曲线
Bradley–Terry 模型 (1952)	从偏好对到标量奖励；DPO 的 loss 也建在它上面
Policy gradient / REINFORCE (第 3 讲)	复习：log-derivative trick、Monte Carlo 估计
Gumbel-softmax (Jang et al., 2016)	问答：让离散采样"可导"的技巧，讲者说在 LLM 规模上没成功
KL 散度 / KL 惩罚	拴住策略；第 5 讲预告的 KL 版 PPO 的去处
PPO、value function (第 4 讲)	RLHF 流程复杂的来源之一 ("吓人"的全流程图)
DPO (Rafailov, Sharma, Mitchell et al., 2023)	本讲主角：闭式解 → 换元 → 二分类 loss
Plackett–Luce 模型	Bradley–Terry 的排序推广；问答里提到，DPO 论文附录有对应版

名称	在本讲里的作用
	本
Llama、Mistral	用 DPO 做后训练的开源模型
GPT-4o 谄媚事件（2025 年 4 月）	短期偏好与长期利益错位的现实案例
YouTube 推荐系统	"优化参与度"的历史类比
Reward hacking / 代理奖励（应为 Gao et al., 2022）	对学到的奖励模型过度优化会偏离真实目标
Rubric 打分、可验证奖励（第 10 讲、CS329A）	展望：奖励信号的新来源
主动探索 / 信息增益（第 14 讲）	问答：主动挑模型拿不准的 prompt 收偏好，没有标准做法

术语对照

English	中文
pre-training / next-token prediction	预训练 / 下一个 token 预测：在海量文本上学续写
instruction fine-tuning (IFT, 也叫 SFT)	指令微调：在指令—回答对上最大化似然，教格式
preference optimization / alignment	偏好优化 / 对齐：让输出更合人的偏好
preference pair (x, y_w, y_l)	偏好对：同一个 prompt 下的赢家回答和输家回答
RLHF	基于人类反馈的强化学习：显式奖励模型加 RL
reward model r_φ	奖励模型：读 prompt 加回答，输出一个标量分
calibration	校准：不同人、不同时候的打分刻度不一致，所以改问偏好
Bradley–Terry model	成对比较的概率模型：胜率是分差的 sigmoid
Plackett–Luce model	Bradley–Terry 对"排序"的推广
intransitive preference	不可传递的偏好：A 胜 B、B 胜 C、C 却胜 A
policy π_θ	策略：这里就是 LLM，给定 prompt 生成整条回答的概率
bandit	单步决策问题：一个状态、一个动作、一次奖励，回合即结束
expected reward objective	期望奖励目标：自己采样的回答平均得分
policy gradient	策略梯度：不对奖励求导也能优化期望奖励的方法
log-derivative trick	对数导数技巧：∇p = p · ∇log p，把梯度搬进期望
Monte Carlo estimate	蒙特卡洛估计：用采样平均代替期望

English	中文
non-differentiable reward	不可导奖励：比如人给的分
discrete bottleneck	离散瓶颈：采样 token 这一步没有导数，梯度传不回去
Gumbel-softmax	让离散采样近似可导的连续松弛技巧
on-policy	在策略：训练数据必须由当前策略现场采样
KL divergence / KL penalty	KL 散度 / KL 惩罚：衡量策略离起点多远，并按此扣分
reference model π_{ref} (课件 p^{PT})	参考模型：起点的指令微调模型，训练中冻结
β	控制"允许离起点多远"的旋钮，RLHF 和 DPO 里是同一个
reward hacking / proxy reward	奖励投机 / 代理奖励：分数涨了，真正想要的没有
closed-form solution	闭式解：能直接写出来、不需要迭代优化的解
partition function $Z(x)$	配分函数：让概率之和为 1 的归一化常数，只依赖 x ，算不出来
reparameterization	换元 / 重参数化：把奖励改用策略的参数来表示
implicit reward	隐式奖励： $\beta \log(\pi_{\theta} / \pi_{\text{ref}})$ ，由策略自己表达的奖励
DPO (direct preference optimization)	直接偏好优化：从偏好对直接更新策略的二分类 loss
sigmoid saturation	sigmoid 饱和：分差够大后梯度消失，DPO 里起 KL 的作用
win rate	胜率：生成的回答被评审偏好于对照的比例
emergent behavior	涌现行为：没有明确训练却出现的行为，比如长答案、列表格式
sycophancy	谄媚：一味附和用户
rubric	评分标准：按多个维度分别打分
verifiable reward	可验证奖励：能用规则或程序判对错的奖励，第 10 讲
personalization	个性化：对齐到个人而非平均偏好
active exploration / information gain	主动探索 / 信息增益：挑最能减少不确定性的 prompt 去问

字幕勘误

"GPD3 / GPD / GPD40" → GPT-3 / GPT / GPT-4o；"instruct GPD" → InstructGPT；"DPU" → DPO；"RLF / RHF / RLE" →

RLHF; "callback library divergence" → Kullback—Leibler divergence; "kale penalty / scale penalty / scale constrained" → KL penalty / KL—constrained; "Bradley objective" → Bradley—Terry objective; "gumball softmax / Gumba Softmax" → Gumbel—softmax; "placket loose" → Plackett—Luce; "MLU" → MMLU; "SFD" → SFT; "Mistl" → Mistral; "PPD / p pre-trained" → p^{PT} (参考模型); "RM 5" → RM_{ϕ} ; "syopantic / psychopantic" → sycophantic; "diet preference optimization" → direct preference optimization; "instruction puning" → instruction tuning; "stoastic" → stochastic; "non-ifferiable" → non-differentiable; "anticedent" → antecedent; "Iro" → Iroh; 开场的"a is going to cover reasoning"应为 Aviral (Kumar, 第 10 讲); "NC2 pairs" → N choose 2, 即 $C(N, 2)$ 个偏好对。

带走的问题

1. $Z(x)$ 能抵消, 靠的是两条回答共用同一个 x 。如果偏好对来自不同 prompt, DPO 还成立吗? Plackett—Luce 版本把 N 条回答一起排序, 为什么仍能抵消?
2. 讲者说"你其实还是在训奖励模型"。隐式奖励 $\beta \log(\pi_{\theta} / \pi_{ref})$ 和显式奖励模型在"给没见过的回答打分"上有什么不同? 为什么讲者说显式奖励模型可能泛化更好 (提示: 想想两处采样和 prompt 集大小)?
3. 奖励模型的二分类准确率只有 65—70%, 为什么仍能带来大幅提升? 标注噪声会怎样从奖励模型传到策略, 又会怎样传到 DPO 的隐式奖励? β 该调大还是调小?
4. 从你做的 agent 产品看: 短期偏好和长期利益的错位 (谄媚) 在用户反馈信号里长什么样? 如果 reward hacking 意味着"训到某个点必须停", 你会用什么信号决定何时停?
5. 讲者说 RL 阶段用的 prompt 集通常比偏好数据的大, 这是 DPO 放弃的东西。把 DPO 改成"在线"版本 (自己采样、找评审标注、再做 DPO) 会补回什么, 又会把哪部分成本加回来? 和 CME295 第 5 讲的分布偏移讨论对照着想。

CS224R 第 10 讲 | RL for LLM 推理（客座 Aviral Kumar）

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 10 讲, 2025 年 5 月 2 日 · 客座讲座, 课表上的题目是 "RL for LLMs: Reasoning", 紧接第 9 讲的偏好优化 视频:
<https://www.youtube.com/watch?v=O2VpNnwB4IM> (1:10:30, 英文字幕是人工校对的 CC, 比自动字幕干净得多, 但缩写仍有几处错, 见文末勘误) 讲者: Aviral Kumar (Carnegie Mellon University) ——客座, 视频简介确认: 讲的主要是他自己组里的两篇论文 课程主页: <https://cs224r.stanford.edu/> · 本讲没有指定阅读; 对应论文: [RL on Incorrect Synthetic Data...](#) (Setlur et al., NeurIPS 2024) 与 [Rewarding Progress](#) (PAV, Setlur et al., ICLR 2025)

一句话: 讲者把"解数学题"写成一个没有环境、转移确定、只有末端 0/1 奖励的 MDP, 然后用一条数据缩放曲线串起三类方法: 在人写的解上做 SFT, 误差随数据量降得极慢; 让模型自己采样、只留答对的解再 SFT (RFT) 能省 2 倍数据, 但采得越多反而越差, 病根是答对的解里混着"多余步骤" (spurious steps); 真正的 RL 味道在于逐步的 credit assignment——从每个前缀让另一条策略接着写, 用成功率的差分当每一步的优势, 据此过滤步骤做 SFT、或构造共享前缀的偏好对做 DPO, 数据效率能到 8 倍; 同样的逐步优势训成 PAV 塞进在线 RL, 比只用 0/1 奖励省 5—6 倍样本。到了 DeepSeek-R1 这一代, 算法没变, 变的是"一步"能是什么: 基座模型学会了自我验证与回溯, 再放开 token 预算, RL 才能把它们放大。

时间轴

时间	内容
0:05	开场: 推理是什么; 本讲只看数学竞赛题 (AIME 例子); 常规范式 = 在参考解上做 next-token prediction
2:41	为什么监督不够: 误差随数据量降得慢、高质量数据快用完; IMO 2021 例子里 o1 的解像样但逻辑错
6:14	本讲结构: 以 DeepSeek-R1 (2025 年 1 月) 为界; 模仿学习 / 离线 RL / 在线 RL 三类
8:19	把解题写成 MDP: 题目是初始状态、步骤是动作、答案对错是末端 0/1 奖励; 板书; 问答: 动作空间无穷、证明题、答对但步骤差
12:26	NeurIPS 2024 论文的数据缩放分析: SFT / RFT / RL 三条轴; 板书 RFT 算法

时间	内容
19:39	问答：RFT 会不会鼓励啰嗦；错误方法答对了怎么办；会不会偏向简单题
21:40	结果一：SFT 的误差随题数降得极慢，RFT 省 2 倍数据；结果二：每题正确解采得越多，误差先降后升；去重
27:50	病因：spurious steps；GSM8K 里 100 乘 2 的例子；和模仿学习里 causal confusion 的关系
32:31	RL 的药方：逐步 credit assignment——从每个前缀让另一条策略接着写，成功率就是值函数
36:09	写成公式：rollout 策略的 Q 函数， $\gamma = 1$ 、只有末端奖励；问答：rollout 该用哪条策略、和 stitching 的关系
40:14	优势 = 相邻前缀 Q 之差；负优势就是可疑步骤；NLP 圈叫它 PRM
44:23	方法一：按优势过滤步骤再 SFT；只训这一步、不训前缀；问答：结果：上翘的曲线被压平
51:42	方法二：共享前缀构造偏好对做 DPO；rollout 策略要质量还是多样性；板书算法；一步怎么切
58:36	结果三：逐步 DPO 比 SFT 省 8 倍数据；Pass@5 是什么；温度 0.6
1:01:40	在线 RL：RFT 推广成策略梯度、GRPO；逐步优势训成 PAV，稠密奖励省 5—6 倍样本；问答：只训 MATH 不涨 GSM8K
1:07:22	思考模型（R1、o 系列）：算法没变，变的是"一步"是什么——自我验证、回溯、更长的预算

核心内容

1. 推理为什么轮到 RL：监督学出来的解"像对，但错"

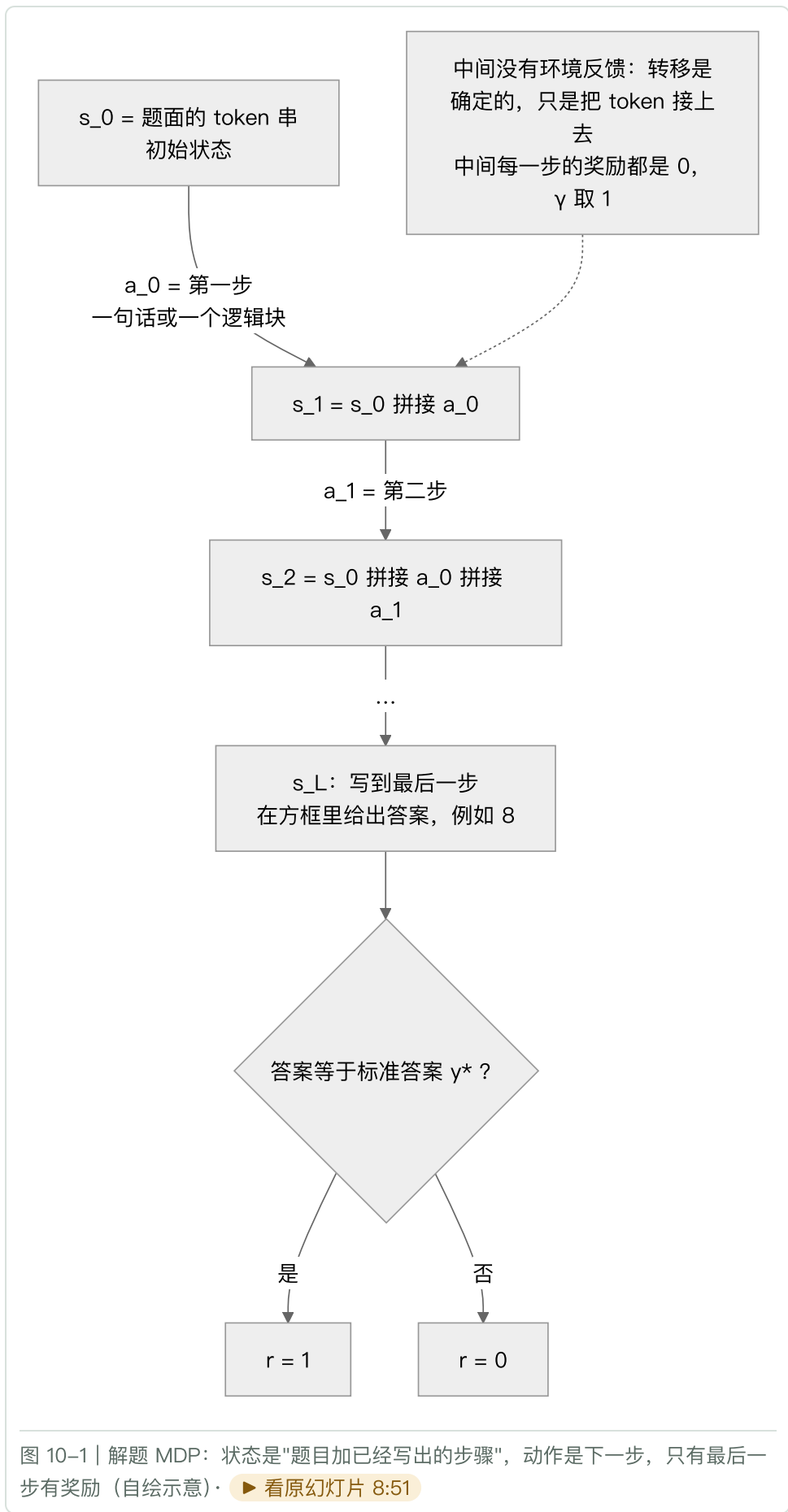
- **范围：**讲者一上来就说"推理"对不同人意思不同，一节课讲不完，所以只看一类问题——数学竞赛题，比如 AIME（American Invitational Mathematics Examination）里的数论题：输入是自然语言题面，输出是一段自然语言的解，末尾给一个确定的数值答案。"末尾有唯一答案"这个限定后面会反复用到。
- **常规范式：**把题面和参考解各自 tokenize，用 next-token prediction（负对数似然）训模型在给定题面时续写参考解——也就是 SFT。它为什么有效，讲者给了一个他喜欢的视角：学到的模型 p_θ 只是对真实"专家分布" p^* 的近似，两者之差有一个随训练数据量下降的上界。数据足够多，就能逼近专家；数据不够，就学不到专家水平。
- **问题正在这里：**推理题的高质量数据远远不够。讲者引了一个估算——照现在的用法，互联网上的高质量数据到 2028 年前后就会用完；机器人和具身智能同样数据受限。

小注：这个估算应出自 Epoch AI 的 Villalobos et al., 2022 (Will we run out of data?), 2024 年的更新版给的区间是 2026—2032 年。

- **数据不够时监督学到的是什么：**讲者拿一道 IMO 2021 的不等式证明题喂给 OpenAI 的 o1 (他记得是 o1)。模型给出的解从行文看完全像专家写的，但逻辑是错的：这道题必须把整个求和当成一个整体处理，模型却逐项去看，最后径直断言结论，而且完全没意识到自己错了、也不去改。讲者的总结：SFT 训出来的模型会写出“考试里看着能得分”的解，只是逻辑内容不对——RL 要修的就是这一点。
- **本讲的地图 (6:14)：**按时间线切成两段。第一段是 DeepSeek-R1 (2025 年 1 月发布) 之前的“经典”技术，再按方法分成三类：模仿学习、离线 RL、在线 RL；第二段讲这些技术怎么延伸到今天的“思考模型”，以及从前一段到后一段到底什么变了、什么没变。他反复强调这是个人的组织方式，文献太多难免漏掉。

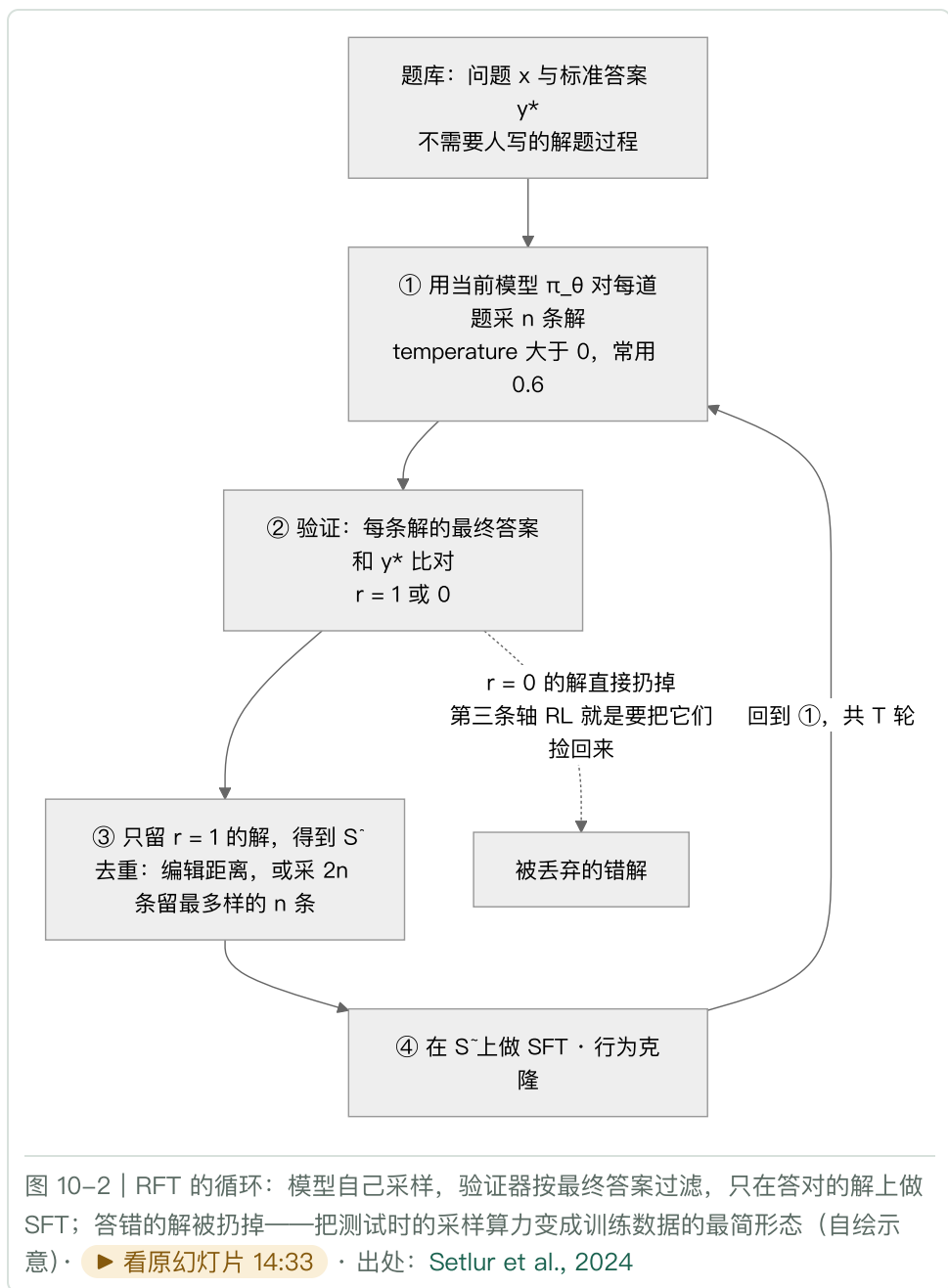
小注：讲者是离线 RL 算法 CQL 的第一作者，第 7 讲讲过它；他开场那句“今天不讲 CQL”被字幕听成了 SQL。

2. 把解题写成 MDP：没有环境、转移确定、奖励只在最后



- **MDP** (Markov decision process, 马尔可夫决策过程) 是 RL 描述问题的标准格式：状态 s 、动作 a 、转移（做了 a 之后到哪个新状态）、奖励 r ，第 1 讲定义过；这里是它在文本上的一个特例。
- **对应关系** (8:51 的幻灯片, 10:23 的板书)：题面的 token 串是初始状态 s_0 ；模型写出的解被切成一串步骤，每一步是一个动作 a_i ；下一个状态 s_{i+1} 就是把 a_i 接到 s_i 后面，没有别的变化；写到最后，模型在方框里给出答案（例子里是 8），和标准答案 y^* 比对，相同得 1，否则 0。中间每一步的奖励都是 0。
- **它是一种什么 MDP**：讲者的说法是 sparse reward（稀疏奖励：整条轨迹只有末端一个信号）加 deterministic dynamics（确定性转移：没有外部环境给你“下一个状态”，下一个状态就是拼接）。这和第 9 讲的 RLHF 设定、CME295 第 5 讲那张“RL 词汇对照 LLM”的表是一致的；区别只在动作的粒度——那边一个 token 一个动作，这里一“步”一个动作，为的是后面能给每一步记功记过。
- **问答**
 - 动作有多少个？——无穷多。但和连续控制里“固定维度的连续向量”不同，这里是变长的离散 token 串。
 - 答案不是一个数、而是一段证明怎么办？——社区还没有公认的好办法，所以本讲只讨论末尾有确定答案的竞赛题。
 - 会不会答对了、但中间步骤很差？——会，而且这正是本讲的主线之一。

3. 三条轴：SFT、RFT、RL，以及 RFT 的循环



- **为什么讲这篇论文：**讲者选了自己在 NeurIPS 2024 的一篇（RL on Incorrect Synthetic Data...），理由不是它发明了什么，而是它对一批现成方法做了系统的实证分析，能给人一个“什么时候该用什么”的概念模型。分析的形式是 data scaling：同一批题目和标准答案，看每种方法的测试误差随数据量怎么变。
- **三条轴** (13:30)。手里有一个题库，每道题带标准答案（定义奖励所必需），可以沿三个方向加数据：
 1. **SFT**：在人写的参考解上做模仿学习（第 2 讲的行为克隆）。加数据 = 加更多题和参考解。
 2. **RFT** (rejection fine-tuning)：让模型对每道题自己采样多条解，按最终答案过滤，只在答对的解上做 SFT。加数据 = 每题多采几条

正确解。

3. **RL**：把答错的解也用起来——第 7 讲讲过，RL 不要求数据是高质量的，这也是作业里的内容。

- **RFT 的板书** (14:33 起)：对题 x ，从当前模型 $\pi_\theta(\cdot|x)$ 采 $y_1 \dots y_n$ ；用标准答案 y^* 算每条的奖励 $r(y_j)$ ；留下 $r = 1$ 的构成 S ；在 S 上做 SFT / BC；重复 T 轮。名字里的 rejection 指扔掉答错的那些。

$$\tilde{S} = \{y_j \sim \pi_\theta(\cdot|x) : r(y_j, y^*) = 1\}, \quad \max_{\theta} \sum_{(x,y) \in \tilde{S}} \log \pi_\theta(y|x)$$

y_j 是模型自己采出来的解， $r(y_j, y^*)$ 是它的最终答案和标准答案比对得到的 0/1 奖励；右边就是普通的 SFT，只不过训练集换成了自己生成、验证通过的解。

- **RFT 的好处**：不需要人写的参考解，只需要题目和标准答案——数据是 on-policy 的（当前策略自己采的），过滤靠的是可计算的奖励。讲者把它定位成“on-policy 的模仿学习”，后面会用这个定位解释它的天花板。

小注：CS329A 里的 STaR 就是这个循环（外加对答错的题“看着答案补写解释”的 rationalization）；RFT 这个名字应出自 Yuan et al., 2023。

- **问答** (19:39 起)
 - 会不会逼模型写得啰嗦？——不会主动鼓励。过滤只看对错：如果答对的解恰好都长，才会间接学到长； $2 + 2$ 这种题多数解直接写 4，学到的就是直接写 4。
 - 用错误的方法碰巧答对了，也会被学进去？——会。这正是后面实验要看的東西。
 - 数据会不会偏向简单题？——会。实践中先在参考解上做一轮 SFT，模型有了基本能力再上 RFT。

4. 两条实验曲线：RFT 省 2 倍，但采得越多越糟

- **基准**：GSM8K 和 MATH。讲者提醒这两个现在已经被刷满，以后不该再拿来比方法，但 2024 年时它们仍有代表性。基座之一是 DeepSeek 的数学模型（讲者说的是 DeepSeek V2），当时最好的开源数学模型之一。

小注：论文里的两个基座应为 Llama2-7B 和 DeepSeek-Math-7B；讲者口中的 “DeepSeek V2” 应指后者。

- **SFT 的缩放曲线** (22:11)：测试误差确实随题数下降，但慢得离谱——拟合出来是 D 的 -0.15 到 -0.05 次方这个量级，而你希望的是 D 的 $-1/2$ 次方那种一加数据就往下掉的曲线。

$$\text{err}(D) \approx C \cdot D^{-\alpha}, \quad \alpha_{\text{实测}} \in [0.05, 0.15] \quad \text{vs} \quad \alpha_{\text{希望}} = \frac{1}{2}$$

D 是训练用的题数， α 是拟合出的指数： α 越大，加数据的回报越快。推理方法的共同目标就是把这条曲线往左下角推。

- **结果一** (24:14)：RFT 在两个基准上都能用一半的题数达到 SFT 的测试误差——on-policy 的模仿比 off-policy 的模仿省 2 倍数据。
- **结果二** (25:15)：那就一直加 n （每题采的正确解条数）？不行。横轴改成每题保留的正确解数量（再乘上题数这个常数），误差先降，过了某个点开始回升，有的数据集上明显变差。
- **问答**：每题的正确解要去重吗？——要。常见做法是按编辑距离去重，或者采 $2n$ 条留最多样的 n 条；讲者本来嫌细节多没放上幻灯片。
- **结论**：在有限的一批初始状态 s_0 上反复拟合自己生成的解，会伤害到新题上的泛化。原因见下一节。

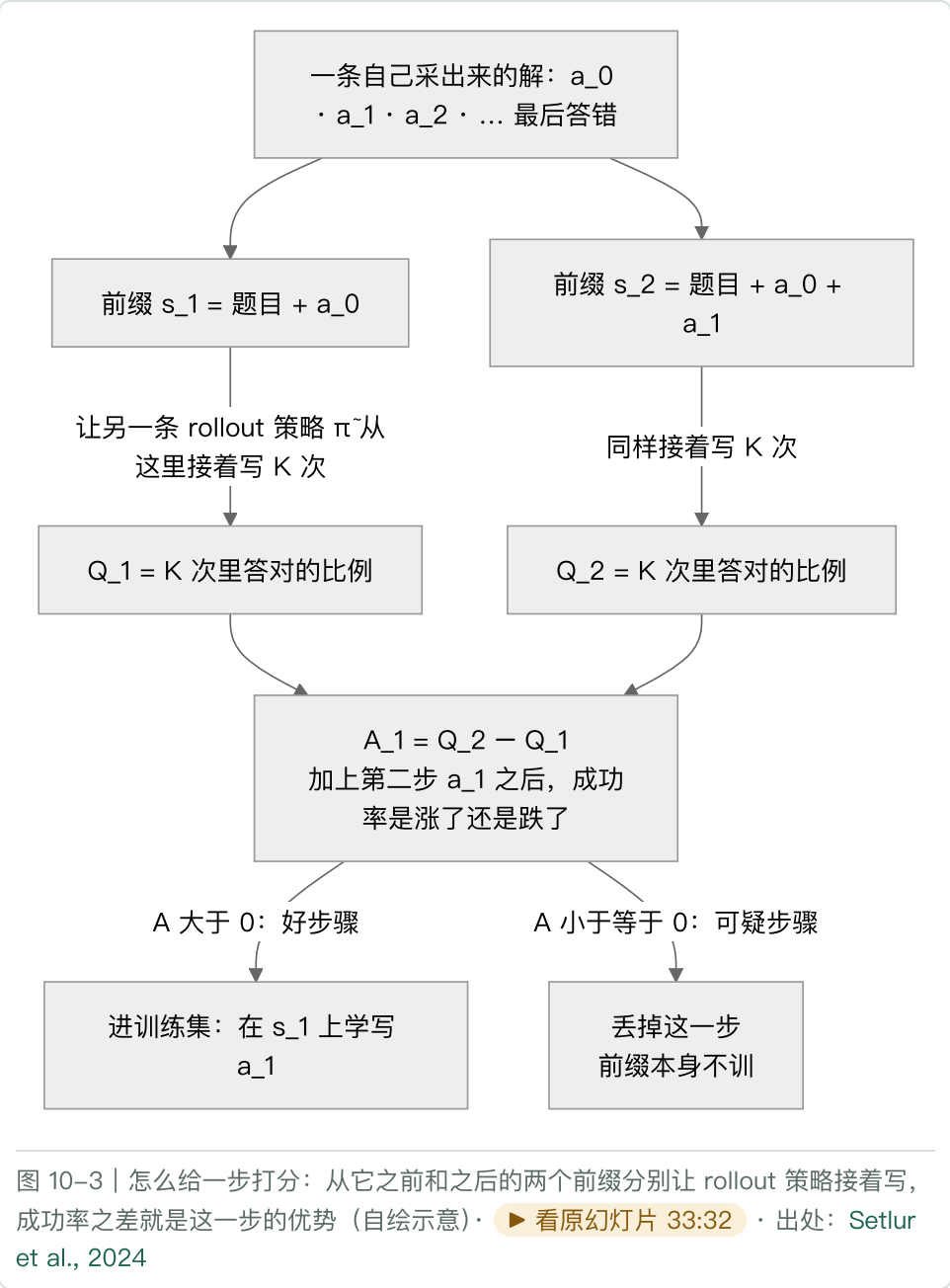
5. 病因：答对的解里混着 spurious steps

- **什么叫 spurious step**：答对的解里某一步本身是错的或多余的，只是碰巧没影响最终答案。GSM8K 里最扎眼的例子 (29:23)：模型先写“总共需要的钱是 100 乘 2 等于 200”，下一行又改口按 100 除以 2 算，最后答对了。中间那句 200 就是 spurious step——不该存在，却被当成正确解的一部分学了进去。
- **它怎么毒害训练**：把这类解当示范做 SFT，模型学到的是“在训练题上恰好能通向正确答案的 token 关联”，到新题上就不成立。论文里量过，训练越久 spurious steps 越多；讲者不敢说这是误差回升的唯一原因，但它是主要原因之一，而且普通的拟合指标看不出来。
- **和模仿学习的联系** (31:59)：这就是模仿学习里的 causal confusion——策略学到了错误的因果关联，在训练分布上照样答对，一换分布就露馅。第 2 讲讲 compounding error 时“训练时对、上场就偏”是同一类病。

小注：causal confusion 一词应出自 de Haan, Jayaraman & Levine, 2019；讲者提到时发现台下没反应，没有展开。

- 问答：spurious step 到底是"错"还是"无关"？——那一步本身未必错，但至少是无关的；无关也有害——想象给模型一百万个无关 token，让它自己挖出有用信息，测试时会很难。一步是一句话吗？——本讲前半段是，但没有公认定义，后半段会看到"一步"是一整块东西。

6. 药方：逐步 credit assignment——从前缀往后 rollout，成功率就是 Q



- credit assignment（功过归因）是 RL 的老问题：整条轨迹只有末端一个奖励，怎么知道功劳或过错在哪一步。RL 的答案是值函数，这里把它落到"步骤"上。

- **做法** (33:32)：拿一条模型采出来的解，截取前 i 步作为前缀，让另一条 rollout 策略从这里接着写完，重复多次。如果多数 rollout 都答对，说明前缀没什么问题；如果无论怎么写都答错，前缀里有毛病；时対时错，介于两者之间。这个"从前缀出发的期望成功率"就是一个值函数——rollout 策略在这个 MDP 里的期望 0/1 奖励。

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right]$$

先回顾一般定义（第 4 讲、第 6 讲）： $Q^{\pi}(s, a)$ 是在状态 s 做动作 a 、之后一直按策略 π 行动，能拿到的折扣奖励总和的期望； γ 是折扣因子，越小越只看眼前。

$$Q^{\tilde{\pi}}(s_i, a_i) = \mathbb{E}_{a_{i+1:L} \sim \tilde{\pi}} [r(\text{answer}, y^*)], \quad s_i = x \circ a_0 \circ \dots \circ a_{i-1}, \quad \gamma = 1$$

在解题 MDP 里只剩一项： s_i 是题目 x 拼上前 i 步， a_i 是当前这一步， $\tilde{\pi}$ 是负责接着写的 rollout 策略， γ 取 1、中间奖励全是 0，所以 Q 就是"从这个前缀出发、由 $\tilde{\pi}$ 写完后答对的概率"。

- **优势** (40:46)：一般定义是 $A(s, a) = Q(s, a) - V(s)$ ，即"这个动作比该状态下的平均水平好多少"（第 4 讲用它做 baseline 降方差）。这里转移是确定的、状态是拼接， $V(s_i)$ 就等于上一步的 Q ，优势退化成相邻两个前缀的 Q 之差：

$$A(s_i, a_i) = Q^{\tilde{\pi}}(s_i, a_i) - V^{\tilde{\pi}}(s_i) = Q^{\tilde{\pi}}(s_i, a_i) - Q^{\tilde{\pi}}(s_{i-1}, a_{i-1})$$

加上这一步之后成功率涨了，优势为正，是好步骤；跌了，优势为负，就是可疑步骤，不该学。

- **rollout 该用哪条策略** (38:43，讲者的个人观点)：公式本身不限定 $\tilde{\pi}$ 。他认为应该用一条和当前模型不同的策略，否则会在同样的地方一起失败；NLP 社区有不少工作用同一条策略也能凑合，但缺少同等规模的对照实验，现有证据多数支持"不同策略"。

小注：后一篇论文（PAV）把这条策略叫 prover policy，摘要里明确要求它不同于被训练的策略。

• 问答

- 这和离线 RL 里的 trajectory stitching、advantage weighting 是不是一回事？——是，论文里就是这么讨论的（第 7 讲和复习课第 7 节讲过 stitching）。
- 要不要真的去拟合一个 Q 函数？——文献里两派：一派显式训一个逐步打分的模型，NLP 叫它 PRM（process reward model，对每

一步打分，区别于给整条回答打分的 RM / ORM) ——讲者不认为 PRM 和 Q 值在语义上该是同一个东西，但大家就这么叫；另一派就用上面的 rollout 直接估优势。CS329A 第 3 讲讲的 PRM 是前者。

- 这怎么帮模型学到通用的解题策略？——它把一条长解的成败拆到每一步：模型能定位是第一步还是第三步出了问题，而不是笼统地知道"错了"，数据效率因此大增。

7. 方法一：按优势过滤，再做 SFT

- **算法** (44:23)：沿用 RFT 的采样，但对每条解的每一步都做 rollout 估优势，把优势为正的步骤放进训练集——哪怕它来自一条最终答错的解；把优势为负或很低的步骤扔掉——哪怕它来自一条答对的解。

$$\mathcal{L}(\theta) = - \sum_i 1[A(s_i, a_i) > 0] \log \pi_\theta(a_i | s_i)$$

$1[\cdot]$ 是指示函数：优势为正的步骤才计入 loss，而且只学"在前缀 s_i 之后写出 a_i "。这是第 7 讲 AWR (advantage-weighted regression) 的过滤版：AWR 用 $\exp(A)$ 做权重，讲者说那个目标要调很多参数，只能跑一次实验的话选过滤。

- **只训这一步，不训前缀** (46:28，板书在 47:00)：假设 s_1 上的 a_1 优势很高、 s_0 上的 a_0 优势很低，训练的只有"给定 s_0 拼 a_0 、写出 a_1 "，不会训模型去写 a_0 。

• 问答

- 部署时要有个东西不断提示模型"下一步"吗？——不需要。训练按步切分，部署时策略一口气写完整条解， s_1 只是"题目加已写的步骤"，同一个策略接着写就是。
- 只训这一步是因为马尔可夫性吗？——状态里保留了全部历史，说它是非马尔可夫策略也行，说状态是拼接、策略是马尔可夫的也行；按步训练不是为了马尔可夫性，是为了能定位哪一步把解带偏了。
- Q 很低但优势为正的步骤要吗？——他们试过留下答错解里的好步骤，没问题；对那种"从任何前缀出发都不可能答对"的解，整条过滤掉可能略好，但没有过硬的结果。既然只学"前缀之后的这一步"，对前缀本身的敏感度不大。

- **结果** (50:40)：横轴仍是每题正确解数，优势过滤把 RFT 那条上翘的曲线压住了；按启发式定义数出来的 spurious steps 也少了。

8. 方法二：共享前缀的偏好对，用 DPO 做离线 RL

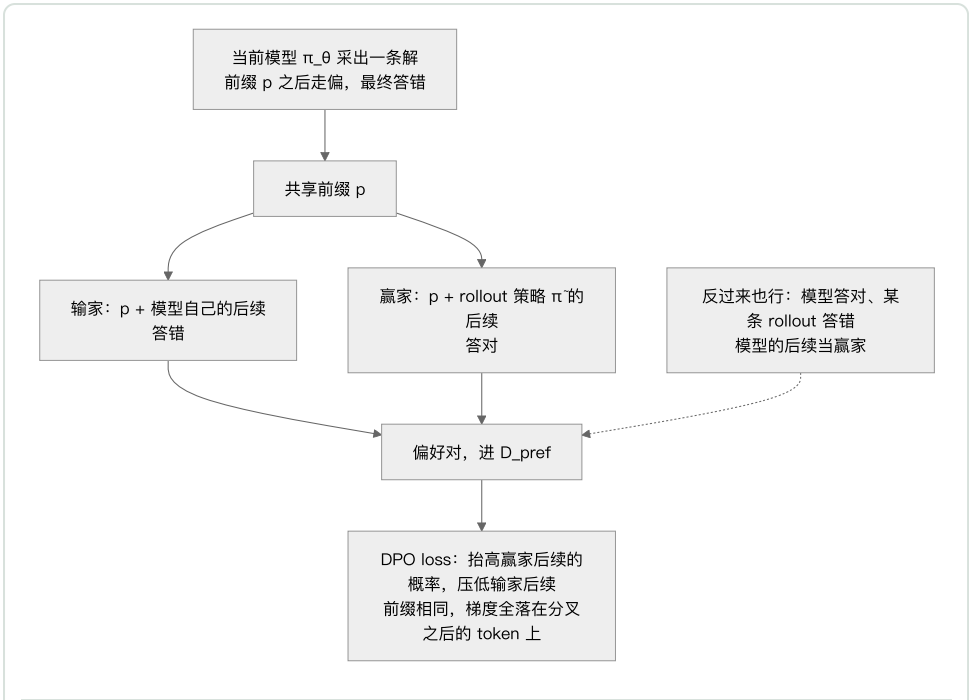


图 10-4 | 从一次 rollout 里造偏好对：前缀相同，模型自己答错的后续做输家，rollout 策略答对的后续做赢家（自绘示意）· [▶ 看原幻灯片 52:15](#) · 出处：Setlur et al., 2024

- 想法 (51:42)：前面的 rollout 只被用来算优势，算完就扔。换个用法：模型自己的一条解在前缀 p 之后走偏、最终答错，而某条 rollout 从同一个 p 出发答对了——两者拼成一个偏好对，赢家是 p 加 rollout 的后续，输家是 p 加模型的后续。反过来也行：模型答对、某条 rollout 答错，模型的后续当赢家。
- 两点不同：一是前缀共享，正负样本的差异被逼到分叉之后那一小段上；二是 rollout 里的 token 本身进了训练集，不再只是估计量。loss 就是第 9 讲和 CME295 第 5 讲的 DPO，条件里多了个前缀：

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(p, y_w, y_l) \sim D_{\text{pref}}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | p)}{\pi_{\text{ref}}(y_w | p)} - \beta \log \frac{\pi_\theta(y_l | p)}{\pi_{\text{ref}}(y_l | p)} \right) \right]$$

p 是共享前缀（题目加已经写出的步骤）， y_w 、 y_l 是分叉之后的两段后续， π_{ref} 是训练起点的冻结副本， β 控制离起点多远；讲者没写这个式子，只说“就是同一个 DPO loss 作用在这两段上”。

- 板书的算法 (55:23)：每轮从 π_θ 采 $y_1 \dots y_n$ ，做 rollout、构造 D_{pref} ，最小化 DPO loss； D_{pref} 里的前缀来自当前模型，一段后续来自 π_θ ，另一段来自 rollout 策略。
- rollout 策略要强还是要杂 (53:49 的问答)：他们在论文里把它写成了一个可以最大化的表达式，本质是探索与利用的平衡（第 14 讲的主

题)。模型很差、几乎全错时，需要一条能偶尔答对的 rollout 策略，否则永远见不到成功；模型已有五成把握时，更需要多样的解法来覆盖它容易失败的方式。

- **一步怎么切** (56:30)：这篇工作按句子切；后续工作用 "Step #k:" 的格式；现在的模型指令跟随够好，可以直接要求它按某种格式写（每个逻辑块之间画横线、不许出现 step 这个词.....），再按格式切。没有通用约定。
- **结果三** (58:36)：横轴回到题数，基座是 Llama 2 和 DeepSeek 的数学模型，虚线是这个逐步 DPO：达到 SFT 同样的测试误差，需要的题数少 8 倍（RFT 那条 2 倍的线为了清爽拿掉了）。
- **问答**：Pass@5 是什么？——给模型 5 次机会、至少答对一次的比例；Pass@n 同理，是评估"多试几次能不能成"的指标（CS329A 第 2 讲的 coverage 就是它）。采样温度用多少？——大家多用 0.6，没有定论。

9. 在线 RL：从 RFT 到策略梯度，再到 PAV 的稠密奖励

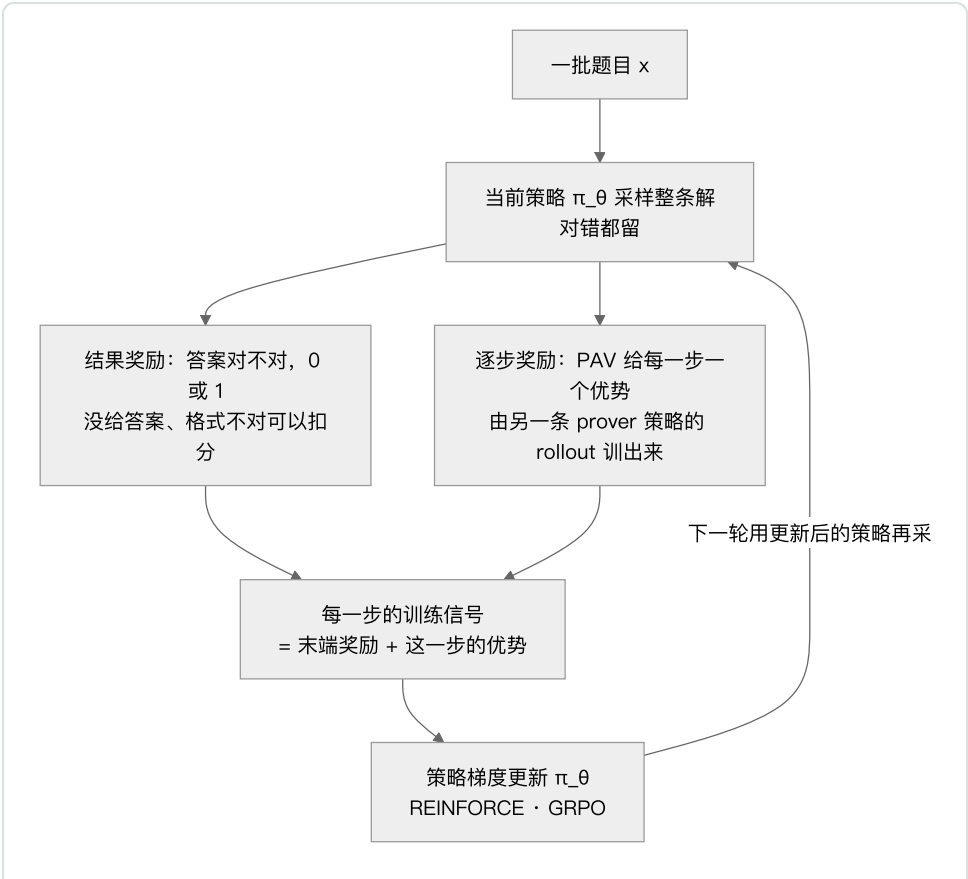


图 10-5 | 在线 RL 的两路信号：末端的 0/1 结果奖励，加上 PAV 给每一步的优势（自绘示意） · [▶ 看原幻灯片 1:03:45](#) · 出处：Setlur et al., 2025

- **最朴素的在线版** (1:01:40)：把 RFT 里的"过滤后 SFT"换成直接做 on-policy RL——采样、按答案给奖励（没给出答案或格式不对可以扣分，这叫 reward shaping），然后用策略梯度最大化奖励。第 3 讲的 REINFORCE 写出来就是 $\nabla \log \pi$ 乘奖励。再往前一步是 GRPO：PPO 的变体，把学出来的 value function 换成组内 rollout 的经验估计（CS329A 第 6 讲、CME295 第 6 讲用过）。
- **加上逐步优势** (1:03:45)：训练过程中同样可以对每条采样的解做 rollout 算优势；更省的办法是训一个网络去拟合这些优势（或 Q 值），像奖励模型一样对每一步打分，然后让策略同时最大化末端奖励和逐步优势：

$$\nabla_{\theta} J = \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} \left[\sum_i \nabla_{\theta} \log \pi_{\theta}(a_i | s_i) (r(y, y^*) + \alpha \hat{A}(s_i, a_i)) \right]$$

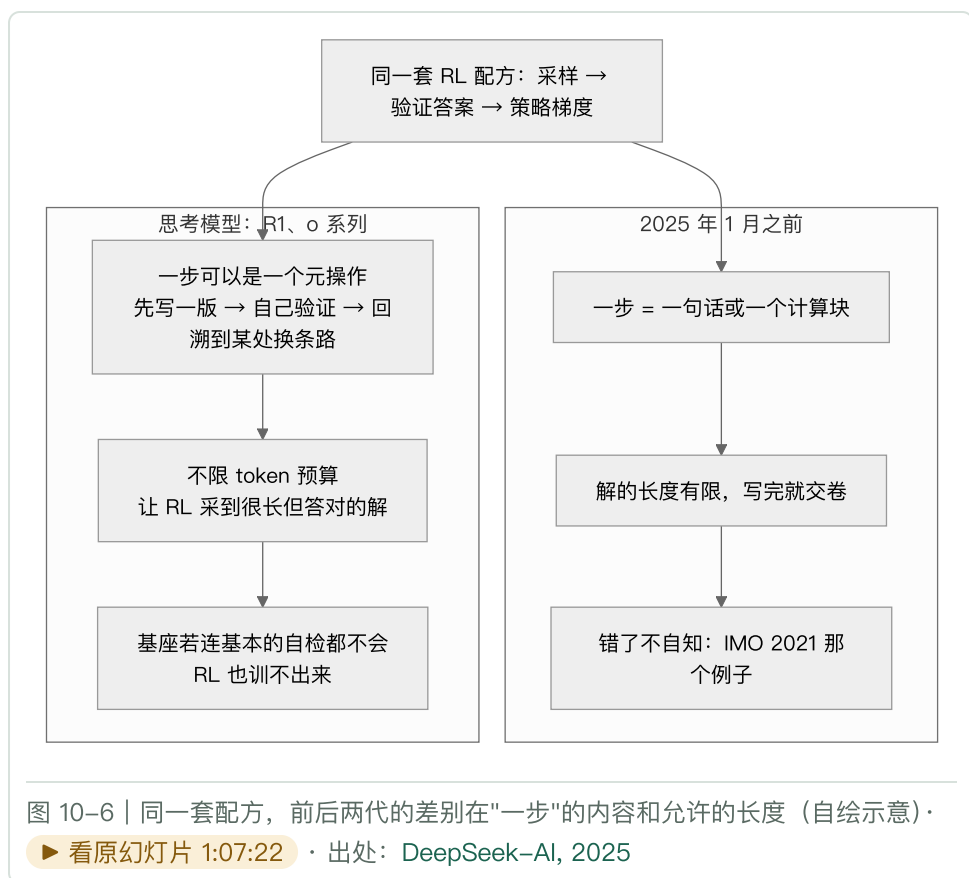
前半是第 3 讲的策略梯度估计；括号里 r 是整条解的 0/1 结果奖励， $\hat{A}$ 是拟合出来的这一步的优势， α 是两者的权重。

- **PAV** (process advantage verifier, 1:05:49)：process 指逐步，advantage 指拟合的是优势函数，verifier 是推理圈对奖励模型的叫法。和只用末端 0/1 奖励（ORM 式）的 RL 比，稠密的优势奖励省 5—6 倍样本、绝对准确率高 6—7 个点；讲者说数字看着小，但在他们用的 2B 和 9B 模型上已是那些数据集上能拿到的最好结果。

小注：PAV 论文摘要给的是在线 RL 5—6 倍样本效率、准确率高 6% 以上；用作测试时搜索也比 ORM 准 8% 以上、省 1.5—5 倍算力。2B / 9B 应为 Gemma 2。

- **问答** (1:04:48)：在 MATH 上训，GSM8K 会涨吗？——不会，很多人验证过。但把不同难度的题目混在一起训，就能泛化——这正是 DeepSeek-R1 这类有公开配方的思考模型所用的基本在线配方，泛化程度相当可观，虽然也不是对所有问题都成立。

10. 思考模型：算法没变，变的是"一步"能是什么



- **讲者的判断** (1:07:55)：DeepSeek-R1、OpenAI 的 o 系列，训练配方和本讲前面讲的基本一样，RL 目标没变。有些新名字——GRPO（PPO 加些小改动）、advantage-induced policy alignment——不必细究，都当作“策略梯度加小改动”就行。
- **真正变的是动作空间** (1:08:26)。他自己说这是个粗糙的说法，严格讲模型还是逐 token 生成；但做 RL 的基座模型已经会执行一些“元操作”：先写一版解，再去验证它，发现不对就回溯到某个点换条路。这可以看成对“一步”的又一种定义——比“再算一个量”有意思得多的动作。
- **第二个变化是预算**：如果告诉模型 token 数不设上限、只要最后答对，RL 就能采到很长但正确的解并强化它。这两点合起来，才有今天思考模型的表现。
- **反事实** (1:10:00)：拿一个不具备基本自检、验证能力的基座去做同样的 RL，现在做不出好结果。所以高层技术没变，是基座变得“能采出更有意思的步骤”了。

小注：这和 CS329A 第 2 讲“测试时算力换准确率”的观察是同一件事从训练侧看：RL 之所以能放大自我验证和回溯，是因为基座里已经有这些行为的雏形。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
AIME 例题与参考解	题面是自然语言，解也是自然语言，末尾一个确定答案——本讲全部方法的前提	1:07	—
IMO 2021 不等式与 o1 的解	行文像专家，逐项处理却是逻辑错误，断言结论而不自知——SFT 模型的典型失败	4:42	—
解题 MDP 一页	题面 = s_0 ，步骤 = 动作，方框答案对错 = 0/1 奖励；板书里 s_{i+1} 就是拼接	8:51	—
三条数据轴	纵轴加题数（SFT），横轴加每题正确解数（RFT），第三轴把错解用起来（RL）	13:30	Setlur et al., 2024
SFT vs RFT 随题数	两条都在降，但指数只有 0.05—0.15；RFT 在一半题数处追平 SFT	22:11	同上
RFT 随每题正确解数	先降后升；采得越多不等于越好	25:15	同上
spurious step 的 GSM8K 例子	"100 乘 2 等于 200"那一步多余且错，下一行改口，最终仍答对	29:23	同上
前缀 rollout 示意	保留前三步，橙色的 rollout 策略往后写多次；全对、全错、时对时错各说明什么	33:32	同上
Q 与优势的定义页	只剩一个期望的 Q；优势写成相邻前缀 Q 之差而不是 Q 减 V	36:39	同上
优势过滤的效果	与 RFT 同一横轴，上翘被压平；spurious steps 计数下降	50:40	同上
共享前缀的偏好对	前缀相同，模型自己答错的后续 vs 橙色 rollout 答对的后续	52:15	同上
8 倍数据效率	Llama 2 与 DeepSeek 两组曲线，虚线（逐步 DPO）在 1/8 题数处追平实线（SFT）	58:36	同上
PAV 的稠密奖励 vs ORM	在线 RL 里样本效率 5—6 倍、绝对准确率高 6—7 个点	1:06:19	Setlur et al., 2025
思考模型一页	目标没变；动作空间变了（验证、回溯）；预算放开	1:07:22	—

提到的工作

名称	在本讲里的作用
AIME、IMO 2021	例题来源：有确定答案的竞赛题；IMO 2021 的不等式用来展示 o1 的逻辑错误
OpenAI o1 / o 系列、Gemini thinking、Claude Sonnet thinking	思考模型的例子；o1 的 IMO 解是"像对但错"的展示
DeepSeek-R1 (DeepSeek-AI, 2025)	本讲时间线的分界（2025 年 1 月）；有公开配方的思考模型，用的是基本在线 RL 配

名称	在本讲里的作用
	方
Will we run out of data? (Villalobos et al., 2022; 应为)	"高质量数据 2028 年前后用完"的估算来源
RL on Incorrect Synthetic Data... (Setlur et al., NeurIPS 2024)	本讲主线: SFT / RFT / 逐步优势 / 逐步 DPO 的数据缩放分析, 2 倍与 8 倍的结论
GSM8K、MATH	实验基准; 讲者提醒已被刷满
Llama 2、DeepSeek 的数学模型 (讲者说 DeepSeek V2, 应为 DeepSeek–Math)	实验基座
SFT / 行为克隆 (第 2 讲)	第一条轴: 在人写的参考解上模仿
RFT (rejection fine-tuning; 名字应出自 Yuan et al., 2023) · STaR (CS329A)	第二条轴: 自采样、按答案过滤、再 SFT
causal confusion (应为 de Haan et al., 2019)	解释 spurious steps 为什么伤泛化
Q 函数、值函数、优势 (第 4、6 讲)	逐步 credit assignment 的数学载体
PRM / ORM (CS329A 第 3 讲)	讲者把 PRM 对应到逐步的 Q 值, 但不认为语义上该等同
AWR (Peng et al., 2019; 第 7 讲)	方法一是它的过滤版: 指示函数代替 $\exp(A)$ 权重
trajectory stitching (第 7 讲、复习课第 7 节)	逐步优势和它是同一类机制, 论文里有讨论
DPO (Rafailov et al., 2023; 第 9 讲)	方法二的 loss, 作用在共享前缀的两段后续上
Pass@k	评估指标: "给 k 次机会至少对一次"
REINFORCE (第 3 讲)、PPO、GRPO (DeepSeekMath)	在线 RL 的更新规则; GRPO 用组内 rollout 替代 value 网络
PAV (Setlur et al., ICLR 2025)	在线 RL 里的逐步优势验证器: 5–6 倍样本效率
advantage-induced policy alignment (应为 Zhu et al., 2023)	讲者点名的"策略梯度小改动"之一, 不必细究
CQL (第 7 讲; 讲者自己的工作)	开场说今天不讲它

术语对照

English	中文
reasoning (本讲的限定)	推理: 这里特指解有确定答案的数学竞赛题
next-token prediction	下一个 token 预测: SFT 用的目标
MDP (Markov decision process)	马尔可夫决策过程: 状态、动作、转移、奖励四件套

English	中文
state s_i / action a_i	状态 = 题目加已写的步骤；动作 = 下一步
sparse reward	稀疏奖励：只有最终答案对错这一个信号
deterministic dynamics	确定性转移：下一状态就是拼接，没有外部环境
trajectory / rollout / trace	一条完整的解；rollout 也指"从某个前缀让策略往后写"
on-policy / off-policy	用当前模型自己采的数据训 / 用别处来的数据训
SFT / behavior cloning (BC)	监督微调 / 行为克隆：在示范上做最大似然
RFT (rejection fine-tuning)	拒绝采样微调：自采样、按答案过滤、再 SFT
data scaling analysis	数据缩放分析：测试误差随数据量的曲线
spurious step	多余步骤：错误或无关、但没影响最终答案的一步
causal confusion	因果混淆：学到在训练分布上成立、换分布就失效的关联
credit assignment	功过归因：把末端奖励分到各步
value function V / Q function	从某状态（做某动作）出发的期望回报；这里就是接着写能答对的概率
advantage A	优势：加上这一步之后成功率的变化
discount factor γ	折扣因子；本讲取 1
rollout policy π^- / prover policy	负责从前缀接着写的另一条策略
PRM / ORM	过程奖励模型（逐步打分） / 结果奖励模型（整条打分）
verifier	验证器：推理圈对奖励模型的叫法
AWR (advantage-weighted regression)	优势加权回归：按 $\exp(A)$ 加权的模仿
trajectory stitching	轨迹拼接：把不同轨迹里好的片段接起来
preference pair / DPO	偏好对 / 直接偏好优化
exploration vs exploitation	探索与利用：rollout 策略要多样还是要准
Pass@k	k 次尝试至少对一次的比例
reward shaping	奖励整形：没给答案、格式不对等额外扣分
policy gradient / REINFORCE	策略梯度： $\nabla \log \pi$ 乘回报
GRPO	用组内采样估优势的 PPO 变体
PAV (process advantage verifier)	过程优势验证器：拟合逐步优势的网络
dense reward	稠密奖励：每一步都有信号
thinking model	思考模型：R1、o 系列这类
self-verification / backtracking	自我验证 / 回溯：思考模型的"元操作"

字幕勘误

"SQL" → CQL (讲者说今天不讲的那个离线 RL 算法, 推断); "RFP""RFD" → RFT; "SFP" → SFT; "DPR""BPO" → DPO; "RHF""RHLF" → RLHF; "GRP" → GRPO; 1:07:55 处的 "TRPO" → GRPO (他前面刚讲完的那个, 推断); "DeepSeek carbon" → DeepSeek-R1; "QL's" → Q values; "S naughts" → s_0 ; "01 reward" → 0/1 reward; "first-step BPO" → per-step DPO。

带走的问题

1. RFT 的曲线为什么会上翘? 如果你的 agent 用"任务成功的轨迹做 SFT"这一招, 什么样的 spurious steps 会混进去 (多余的工具调用、碰巧成功的错误参数)? 怎么量它?
2. 优势 = 相邻前缀的 Q 之差, 这个等式依赖"转移确定、状态是拼接"。如果一步里有真实环境反馈 (工具返回、网页变化), 哪一项要改? $V(s_i)$ 还等于上一步的 Q 吗?
3. rollout 策略为什么最好和被训的策略不同? 用同一条策略估优势时, 会系统性地高估或低估哪类步骤?
4. 共享前缀的 DPO 把梯度集中在分叉之后; 如果输家和赢家在分叉后仍有大段相同内容, 会发生什么? 这和 CME295 第 5 讲讨论过的"DPO 里两条概率一起下降"是什么关系?
5. 讲者说思考模型的关键是基座已经会自检和回溯。对你的产品: 若基座不会某种元操作 (比如"回头核对用户最初的约束"), 你会先用 SFT 教它这个动作再上 RL, 还是直接 RL? 依据本讲的哪条证据?

CS224R 第 11 讲 | 基于模型的 RL (Model-Based RL)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 11 讲, 2025 年 5 月 7 日 · 开场先用一页幻灯片把第 2–7 讲的算法排成一张地图; 本讲只讲完"用模型做规划"这一半, "用模型生成数据"那一半 (指定阅读 MBPO 所属的路线) 因时间关系推到了第 12 讲开头 视频: <https://www.youtube.com/watch?v=PvqyGnOirgA> (1:13:20, 英文字幕是人工校对的 CC, 不是自动生成, 错词很少, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座; 她说 model-based RL 在她心里有特殊位置, 早年做过不少这方面的工作) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: [When to Trust Your Model: Model-Based Policy Optimization](#) (Janner et al., 2019, 即 MBPO; 课上只在案例的对照组里出现了名字, 它的机制留到下一讲, 第 8 节末尾的小注先做了预告)

一句话: 前面十讲的算法都把环境当黑箱, 只能靠采样和策略梯度摸索动作的后果。这一讲先学一个"模拟器"——读入 (s_t, a_t) 、吐出 s_{t+1} 的动力学模型 (必要时配一个奖励模型), 然后在模拟器里"先想后做": 站在当前状态搜一段未来 H 步的动作, 让想象中的奖励之和最大, 可以对动作做梯度上升, 也可以用 random shooting 或 cross-entropy method 撒样本挑好的。麻烦在于模型只在数据覆盖到的地方准, 优化器又专挑它出错的地方钻, 所以要边执行边把新数据加回去重训, 并且只执行第一个动作就重新规划 (model predictive control), 用闭环纠错换鲁棒。代价是测试时算力重、只看得了短视野; 出路是把规划器蒸馏成策略、在规划末尾接价值函数, 或用模型生成数据训策略 (下一讲的 MBPO)。案例 PDDM 用 3 个模型的 ensemble 加改良 CEM, 让 24 关节五指手靠 4 小时真机数据学会转保定球, 数据量只有 model-free 的五分之一。

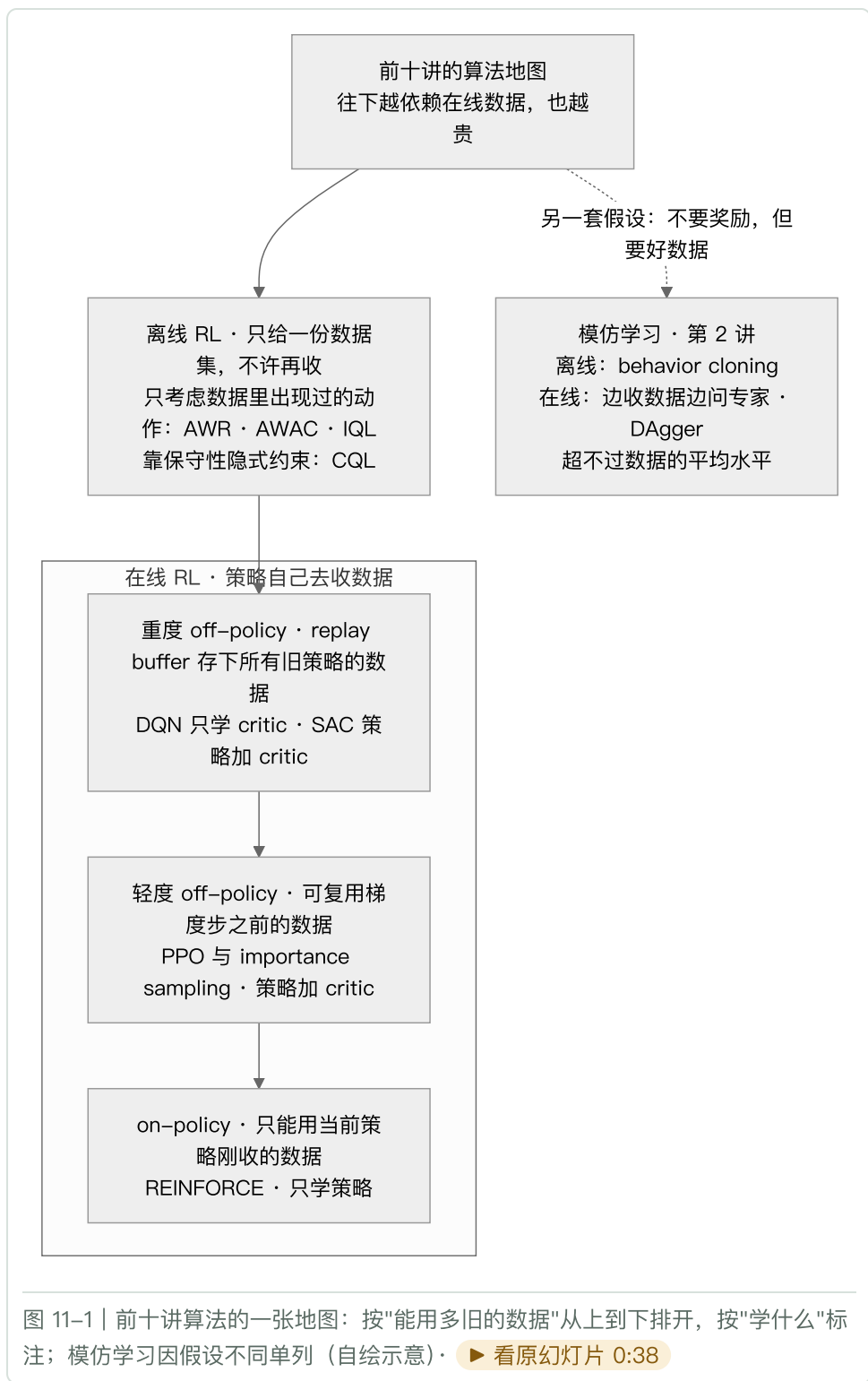
时间轴

时间	内容
0:05	开场: 一页总览前十讲的算法——online 与 offline、on-policy 与 off-policy、只学策略 / 策略加 critic / 只学 critic、离线 RL 两类、模仿学习
4:45	今天的主题: model-based RL; 它能套在上一页所有方法之上; 预告灵巧手案例与本讲目标
5:52	核心想法: 学一个模拟器——物理模型或以动作为条件的视频预测; Veo 2、Sora; 股市、棋类规则、对手建模

时间	内容
8:25	算法草图：收数据 → 学模拟器 → 在里面跑你喜欢的方法；问答：这么做会出什么问题
10:30	两个坑：数据覆盖不到（迷宫例子）、模拟器不准（Sora 叠方块）；模型设计依赖领域；有了好模拟器也还得做 RL
13:11	怎么学动力学模型：已知 / 大部分已知只拟合几个参数 / 从头学；图像观测：整段视频还是潜空间；还得学奖励模型
20:03	用模型 · 规划：把奖励沿模型反传到动作上；板书四步；问答：连续动作、随机模型、误差被利用、加引号的"策略"
31:57	不用梯度的优化：随机采样、往好样本附近重采、cross-entropy method；问答：有限差分、策略存在哪、算慢了状态变了怎么办
43:28	梯度法与采样法的优缺点；random shooting 与 CEM 两版采样式规划；优化维度是动作维数乘 H
48:09	爬山例子：模型只学过"往右更高"，规划一路向右掉下悬崖；把执行过的数据加回去重训，才成了 RL 算法
51:21	开环还是闭环：只执行第一个动作再重新规划；model predictive control；问答：多模态振荡、warm start、隔几步再规划、模型误差帮探索
1:00:47	规划小结：同一模型换奖励就换任务；缺点是测试时算力重、只适合短视野或稠密奖励
1:03:27	用模型训策略：把规划器蒸馏成策略；长视野的两条路——规划末尾接价值函数、用模型生成数据（留到下一讲）
1:07:07	案例 PDDM：24 关节五指手、3 个模型的 ensemble、改良 CEM；与 SAC、NPG、MBPO、PETS 的对比；真机 4 小时数据

核心内容

1. 开场：把前十讲的算法放到一张地图上



- 为什么有这一页：office hours 里不少人反映这些名词搅在一起分不清，讲者就做了一页几乎装下所有已讲算法的总览。
- 第一道分界：online 还是 offline。online 指策略会被拿去收集数据；offline 指只给一份数据集，不能再和环境交互（第 7 讲）。
- online 里再按"能复用多旧的数据"分三档

- on-policy: 只能用当前这个策略刚收的数据, 走一步梯度就得扔——REINFORCE 和 vanilla policy gradient (第 3 讲)。
- 轻度 off-policy: 可以复用本轮梯度步之前那个策略的数据——PPO 和 importance sampling (第 5 讲)。
- 重度 off-policy: 用 replay buffer 存下过去所有策略的数据反复用——DQN、SAC (第 5–6 讲)。
- **按"学什么"上色:** 只学策略的叫 policy gradient 方法; 策略和 critic (价值函数) 都学的叫 actor-critic, PPO 和 SAC 属于这类; 只学 critic、不显式学策略的叫 Q-learning, DQN 是代表。讲者补了一句: 外面常把 PPO 叫 policy gradient 方法, 但它确实学了价值函数, 本课为免混淆一直叫它 actor-critic。
- **offline RL 两类:** 一类明确只考虑数据里出现过的动作 (AWR、AWAC、IQL); 另一类靠保守性隐式地把你约束在数据附近 (CQL)。
- **越往右越贵:** 从 on-policy 到 offline, 对在线数据的依赖递减; 在线数据反复大量收集很难, 通常比离线数据贵得多。
- **模仿学习单列 (第 2 讲):** 离线的 behavior cloning 直接模仿数据里的动作; DAgger 是在线的——既用当前策略收数据, 又拿新数据去问专家该怎么做。它们的假设不一样: 不需要奖励函数, 但需要相当好的数据, 而且没法超过数据里的平均水平。

小注: 讲者说到 online 那一组时口误成了 "offline setting", 幻灯片和上下文都是 online。另外, 本讲的 model-based RL 不是这张图上的第四列——它是一层可以套在任何一列之上的做法, 这正是她接下来那句话的意思。

- **今天的主题:** model-based RL 可以套在上面所有类别之上。内容四块: 核心想法、怎么学模型、怎么用模型 (规划和生成数据)、灵巧手案例; 目标是理解动力学模型怎么学、怎么用, 以及这一类方法的取舍。

2. 核心想法: 学一个模拟器

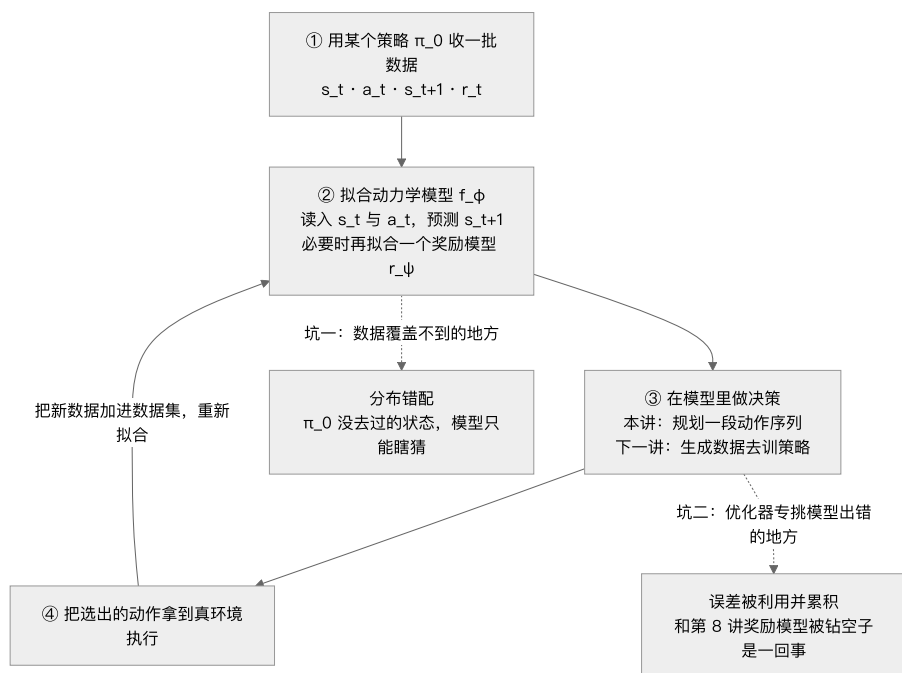


图 11-2 | model-based RL 的基本循环，以及讲者让全班找出来的两个坑（自绘示意）· [▶ 看原幻灯片 8:56](#)

- **模拟器就是动力学模型。**第 1 讲的 MDP 里有一个转移概率 $p(s_{t+1} | s_t, a_t)$ ，描述"在状态 s_t 做动作 a_t ，世界会变成什么样"。前面所有算法都不去学它，只从采样到的 (s, a, r, s') 里间接受益；model-based RL 的核心想法是把它学出来——能预测"如果我这么做，接下来会发生什么"。
- **模型长什么样取决于能观测到什么**
 - 能直接看到底层物理量（关节角、位置、速度）时，它就是一个物理模型。
 - 看不到时，它更像"以动作为条件的视频预测"：我说了这句话对方会怎么反应，我这样迈腿会不会摔。Veo 2、Sora 这类模型已经能生成很逼真的视频，可以想象用它们来教汽车、机器人、四旋翼理解自己的动作会怎样改变世界。
 - 模型也不必是物理世界：想在股市赚钱，模拟器要预测行情以及你的操作是否影响行情；玩游戏，模拟器就是游戏规则——棋类的规则已知，根本不用学；多人游戏里还要建模其他玩家，通常单独建模，一个偷懒办法是拿自己的模型当对手模型。
- **算法草图**（图 11-2 的实线部分）：① 用某个策略收一批数据；② 训一个读入 (s_t, a_t) 、输出 s_{t+1} 的网络，最简单的损失是预测值和真实

下一状态的 L2 距离；③ 在这个模拟器里跑你喜欢的方法——前面学过的任何 RL 算法，或者本讲重点的规划方法。

- **讲者的提问：照这个草图做会出什么事？** 学生答出了两点，讲者又补了三点：
 1. **模拟器不准会被利用。**和第 8 讲的奖励模型被策略钻空子、第 6 讲 Q 函数被 argmax 挑出过估计是同一回事：优化器专门找模型出错的地方，误差还会一步步累积，在模拟器里学到的东西搬回真环境就不灵了。CME295 第 5 讲讲 reward hacking 时用的"掌声当奖励"的比方，换到动力学模型上一样成立。
 2. **数据覆盖。**收数据的初始策略 π_0 和最终要学的策略不同，模型在后者会去的状态上根本没见过数据。迷宫例子：数据只沿一条黄色路线，模型对迷宫其他区域一无所知，那些区域的策略自然学不出来。
 3. **不准的例子：**Sora 按提示生成"人形机器人把红绿蓝方块按顺序叠起来"，视频里方块会变色、凭空多出一块。讲者说她很想让机器人做到这样，但这不是机器人在现实里能做的事。
 4. 模型怎么设计依赖领域：视频预测、带摩擦的低层物理、股市，输入、架构、训练方式都不同，有的领域天生好模拟，有的不好。
 5. 就算模拟器很好，也不等于能轻松得到好策略——RL 过程还是要在模拟器里做一遍，而且要考虑模型不准的地方，因为模型几乎从不完美。

小注：这就是 "model-based" 的全部含义：多学一个模型，把它当环境的替身。前面十讲的算法因为不学模型，统称 model-free。

3. 动力学模型怎么学

- **三种处境**
 - 模型已知：直接用，皆大欢喜。
 - 大部分已知：推桌上的物体，物理大体清楚，只是不知道桌面的摩擦系数；开车，加速和转向的响应大致知道，缺几个参数。这时保留已知结构、用数据拟合未知参数——讲者说这是把领域知识塞进 RL 里最漂亮的方式之一。
 - 几乎不知道（实践中绝大多数情况）：物体属性太多、人和其他司机的行为没法写成公式。做法是用神经网络或别的函数逼近器从头学。

- **训练目标：**随机动力学就最大化下一状态的对数似然；动力学接近确定性时，直接最小化预测误差。有学生问是否一定端到端——讲者说这里的目标就是"能不能生成未来状态"，这个目标本身的利弊后面会谈。

$$\min_{\phi} \sum_{(s_t, a_t, s_{t+1}) \in \mathcal{D}} \|f_{\phi}(s_t, a_t) - s_{t+1}\|_2^2 \quad \text{or} \quad \max_{\phi} \sum_{(s_t, a_t, s_{t+1}) \in \mathcal{D}} \log p_{\phi}(s_{t+1} | s_t, a_t)$$

f_{ϕ} 是参数为 ϕ 的确定性模型，直接回归下一状态； p_{ϕ} 是随机版本，给下一状态一个分布。 $\mathcal{D}$ 是收来的转移数据集，两个目标都是普通的监督学习。

- **图像观测的两条路**（讲者在板上画了一辆车、几辆别的车、地上一点垃圾）
 - 直接生成整段未来视频（以动作为条件），就像上一节的 Veo 2 和 Sora。贵：要预测所有像素在未来怎么变。
 - 先学一个编码器把图像 s_t 压成低维表示 z_t ，再在表示空间里学模型预测 z_{t+1} 。省算力，但要先解决"好的表示从哪来"。

$$z_t = E_{\theta}(s_t), \quad \hat{z}_{t+1} = f_{\phi}(z_t, a_t), \quad \hat{r}_{t+1} = r_{\psi}(\hat{z}_{t+1})$$

E_{θ} 是编码器， f_{ϕ} 在潜空间里预测下一步， r_{ψ} 是配套的奖励模型；三者都只在低维的 z 上工作，像素细节从此不必建模。

- **问答：压到低维会丢信息，主要是为了省算力吗？**——是。而且丢掉的正该是无关信息：路面是纯黑还是磨旧了一点，对开车没影响，学到一个丢掉这些细节的表示，就不用花算力去建模它们。
- **别忘了奖励模型：**用模型往前看，看到的不能只是状态，还得知道这些状态值多少奖励，所以通常还要学一个 r_{ψ} （第 8 讲的奖励学习方法）。奖励已知时——比如学走路用前进速度、导航用到目标的距离——就不必学。

小注：潜空间里学模型的代表作应为 PlaNet 和 Dreamer 一族（Hafner et al., 2019 起），课上没有点名；它们的动力学、奖励、价值都在 z 上学，是这条路线的后续。

4. 用模型 · 路线一：规划——把奖励的梯度反传到动作上

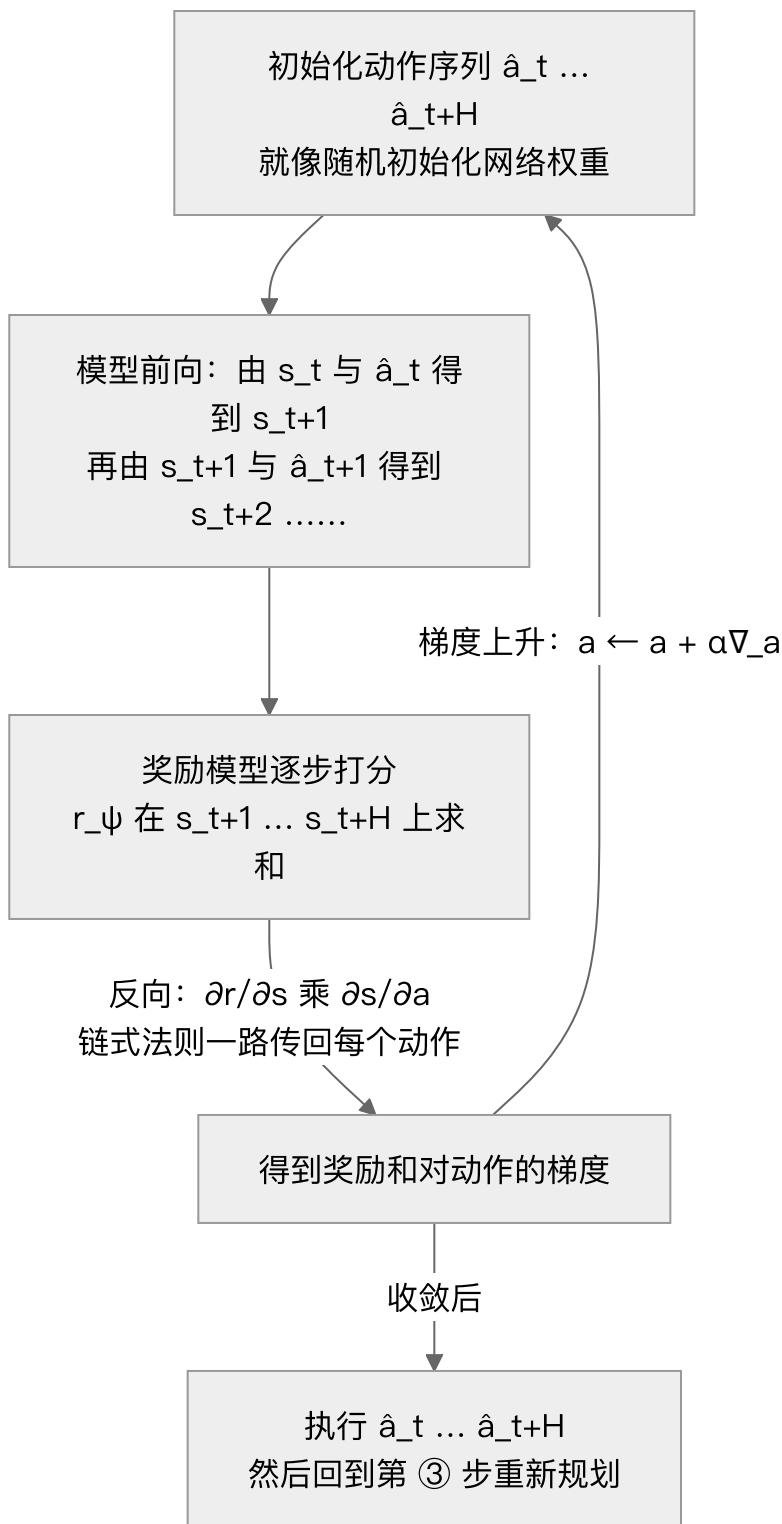


图 11-3 | 通过动力学模型和奖励模型把梯度传回动作序列：更新的是动作，不是网络参数（自绘示意）· [▶ 看原幻灯片 21:35](#)

- 为什么以前不能这么干：MDP 是一条链—— s_t 做 a_t 得到 s_{t+1} 和奖励，再做 $a_{t+1} \dots$ 。前面的算法假设不知道动作怎么影响未来状态，所以只能绕开这段不可导的环境去优化奖励，第 3 讲的策略梯度整套

推导都是在绕这个弯。现在有了动力学模型和奖励模型，两个都是神经网络，就可以直接从奖励一路反向传播到动作。

- **什么叫规划：**站在当前状态，往前想 H 步，找一串动作 $a_t \dots a_{t+H}$ ，让这 H 步的奖励之和最大。 H 叫视野 (horizon)。

$$\hat{a}_{t:t+H} = \arg \max_{a_{t:t+H}} \sum_{t'=t}^{t+H} r_{\psi}(s_{t'}, a_{t'}) \quad \text{s.t.} \quad s_{t'+1} = f_{\phi}(s_{t'}, a_{t'})$$

优化变量是整段动作序列 $a_{\{t:t+H\}}$ ，未来状态由模型 f_{ϕ} 递推给出， r_{ψ} 给每一步打分；解出来的 $\hat{a}$ 是“我现在打算这么走”的计划。

- **板书四步：**① 跑某个策略收数据；② 训模型，最小化预测的下一状态和真实的差；③ 找动作：初始化一段动作（就像随机初始化网络权重），算奖励和对动作的梯度，做一步更新，反复；④ 执行这 H 个动作，然后回到 ③。梯度按链式法则拆成两段：奖励对状态的梯度来自奖励模型，状态对动作的梯度来自动力学模型。

$$\frac{\partial}{\partial a_{t:t+H}} \sum_{t'=t}^{t+H} r_{\psi}(s_{t'}) = \sum_{t'} \frac{\partial r_{\psi}}{\partial s_{t'}} \cdot \frac{\partial s_{t'}}{\partial a_{t:t+H}}, \quad \hat{a}_{t:t+H} \leftarrow \hat{a}_{t:t+H} + \alpha \frac{\partial}{\partial a_{t:t+H}} \sum_{t'} r_{\psi}(s_{t'})$$

$\partial r_{\psi} / \partial s$ 是奖励模型给的“哪个方向的状态更值钱”， $\partial s / \partial a$ 是动力学模型给的“动作怎么改变状态”， α 是步长。和平时训网络的区别只在于：被更新的是动作，网络参数 ϕ 、 ψ 全程冻结。

小注：讲者板书写的是减号 ($a \leftarrow a - \alpha \nabla$)，随后自己指出幻灯片上也有同样的笔误——这里是最大化奖励，应该是加号，也就是梯度上升。

• 问答摘要

- 动作是连续的吗？——这一版是；离散动作后面用采样法处理。
- 奖励是确定性的？——本课一直这么假设，随机性都可以塞进动力学和状态变量里。奖励模型 r_{ψ} 可以用第 8 讲的方法学，也可能本来就知道。
- 动力学模型是随机的怎么反传？——看怎么建模：高斯有对样本求导的技巧，diffusion 模型又不一样。目前先讨论确定性模型。
- 会不会利用奖励的误差？——会，而且动力学和奖励两边的误差都会被利用，任一边不准，选出的动作在真实世界里就可能不好，这是后面要解决的事。
- 这里根本没有策略网络？——对。讲者给“策略”打上引号：所谓策略就是这个规划过程本身，每到一个状态都重新做一遍优化。

- 只看 H 步，无限视野怎么办？——先只考虑短视野，第 8 节再谈。

小注：对动作做梯度反传的做法在控制里叫轨迹优化；反传要穿过 H 次模型前向， H 越长梯度越容易爆炸或消失，这是后面偏爱采样法的原因之一（课上没展开，推断）。

5. 不用梯度也能优化：random shooting 与 cross-entropy method

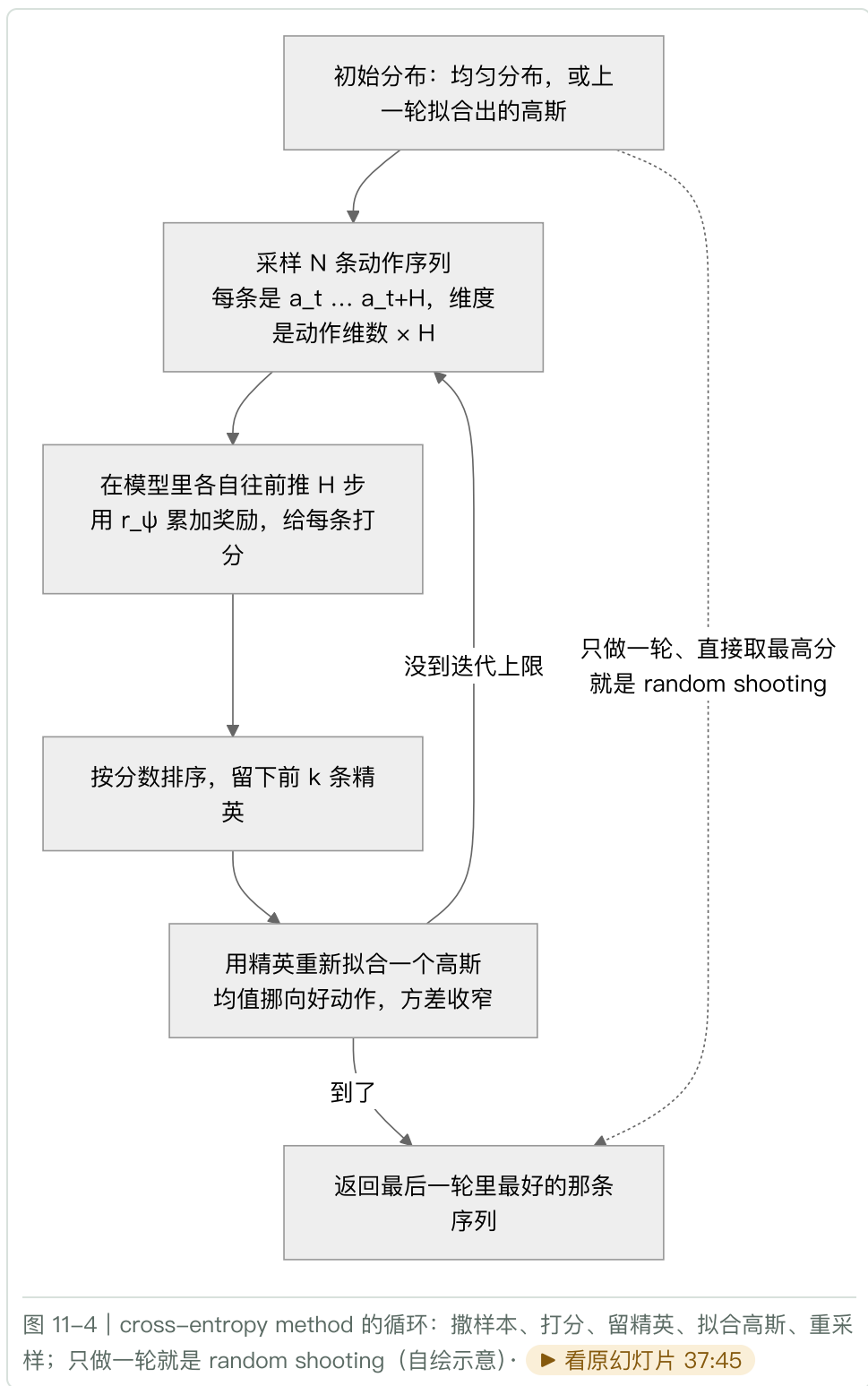


图 11-4 | cross-entropy method 的循环：撒样本、打分、留精英、拟合高斯、重采样；只做一轮就是 random shooting（自绘示意）· [▶ 看原幻灯片 37:45](#)

- **梯度法在一维奖励曲线上长什么样**：随机起一个动作，算这点的斜率，朝上坡挪一步；再算，斜率变缓就少挪一点；斜率为零就停。整个过程是串行的，一次只看一个点。
- **采样法**（也叫 gradient-free 或 zeroth-order 优化）：完全不用梯度，撒一把动作看各自的奖励。最简单的版本是并行采一堆、取最好的一个——random sampling，讲者说它常常比你以为的好用。

- **再进一步**：既然某几个样本明显更好，下一轮就往它们附近多采——把最好的几个拟合成一个简单分布（比如高斯），从中重采样，再打分、再拟合，逐轮收紧。把这个迭代写正式，就是 cross-entropy method (CEM) ——别和交叉熵损失搞混，两者只是同名。

1. 从初始分布采样（可以是均匀分布）。
2. 按奖励（幻灯片上是按损失）排序。
3. 留下最好的几条（精英）。
4. 用精英拟合一个高斯。
5. 回到第 1 步，从新分布采样；迭代若干轮后返回最后一轮里最好的样本。它是通用优化器，任何目标都能用。

$$A^{(i)} \sim \mathcal{N}(\mu, \Sigma), \quad J(A^{(i)}) = \sum_{t'=t}^{t+H} r_{\psi}(\hat{s}_{t'}^{(i)}), \quad \mathcal{E} = \text{top-}k\{A^{(i)}\}_{i=1}^N, \quad \mu \leftarrow \frac{1}{k} \sum_{A \in \mathcal{E}} A, \quad \Sigma \leftarrow \text{Cov}(\mathcal{E})$$

$A^{(i)}$ 是第 i 条候选动作序列 $a_{\{t:t+H\}}$ ， J 是把它放进模型里推 H 步得到的想象奖励之和， $\mathcal{E}$ 是得分最高的 k 条精英；均值挪向精英、方差按精英收窄，下一轮就在好动作附近搜索。

• 问答摘要

- 能不能用采样近似梯度？——可以，叫有限差分：采两个很近的点，用差值估斜率，有利有弊，但是正经方法。
- "策略"为什么打引号？——没有参数化的策略网络，背后是一个迭代优化过程。它确实是从状态到动作（甚至到未来多个动作）的映射，但为了和网络那种紧凑的闭式映射区分，通常不叫它策略。
- 那策略存在哪？——存的只有模型参数和奖励模型参数。每个时间步都现场想"做这个、这个、这个会怎样"，再挑好的。这意味着测试时要花多得多的算力，而且是实时花——CS329A 里 test-time compute 的那笔账，在 RL 里就是规划。
- 采出来的动作会不会超出模型训练数据的分布？——会，下一页处理。
- 算完一轮状态已经变了怎么办？——讲者坦言没有好答案：实践里就是把规划做得尽量快；也可以估计耗时，比如耗了两步就从 a_{t+2} 开始执行，代价是你到的状态未必是你以为的状态。异步规划仍是开放问题。

- **两类优化器的取舍**（讲者说这解释了为什么规划里采样法常是好选择）

	梯度法	采样法
维度	能扩展到极高维；过参数化的大网络尤其合适	维度一高样本就盖不住空间，很难找到好点
对目标的要求	需要平滑的优化地形	不需要梯度：离散动作、只有奖励值没有梯度、奖励不连续都行
并行	天然串行：算一次梯度、更新、再算	天然并行：样本各算各的，有时反而更快
在本讲的角色	第 4 节的反传到动作	random shooting、CEM，以及案例 PDDM 的规划器

- **采样式规划的两个版本**：记 A 为整段动作序列 $a_t \dots a_{t+H}$ 。版本一 random shooting：从某个分布采 N 条 A ，在模型里各推 H 步算奖励和，挑最高的执行——"朝未来射一堆子弹，选打得最准的一枪"。版本二用 CEM 迭代改进采样分布。并行化之后很快，动作维数低时实现也简单；但优化变量的维数是动作维数乘以 H ，两者一大就吃力。

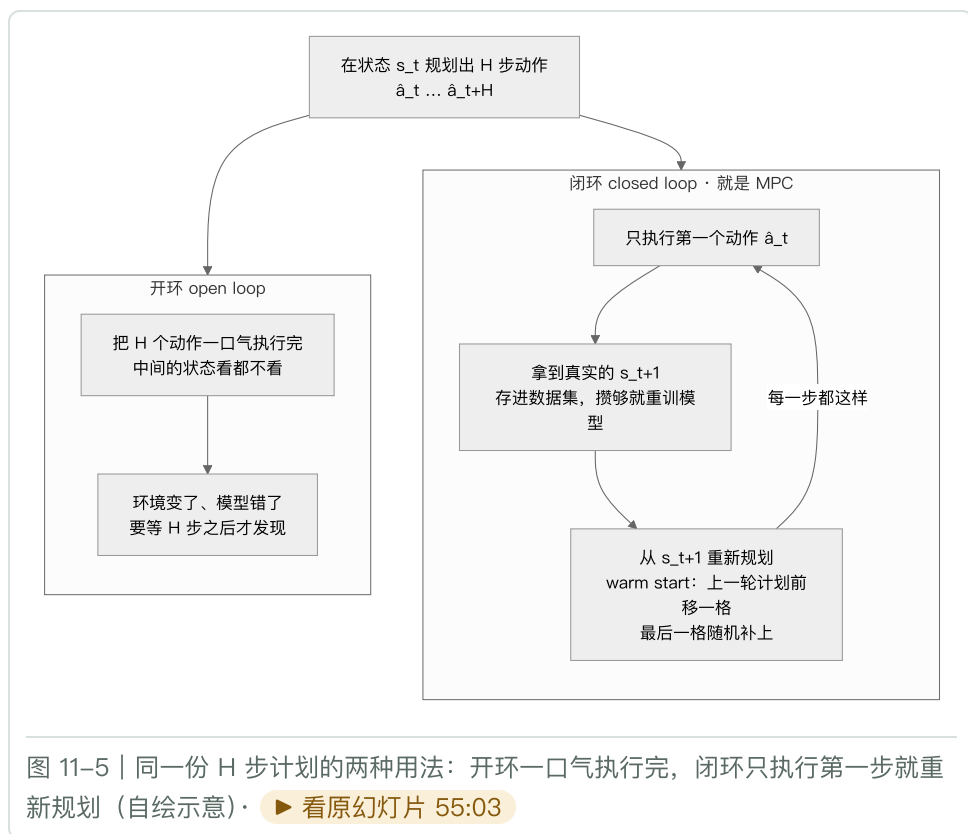
6. 为什么会掉下悬崖：分布错配，以及把执行数据加回去

- **一个"简单而美丽"的环境**：一座山，奖励就是所在位置的海拔（越高风景越好）。
- **第一轮**：某个策略在山脚附近跑了一段，数据显示"往右走海拔升高"，模型学会了这条规律。规划器于是给出"一路向右"的计划，执行之后确实爬了一阵——然后越过山顶，掉下悬崖。
- **病因**：正是第 2 节说的分布错配。开头那批数据没覆盖悬崖那一侧，模型在那里只是把"往右更高"外推过去，规划器又恰好最信这条外推。
- **药方很朴素**：把刚才执行过的轨迹（包括掉下去的那段）加进数据集，重新拟合模型，再规划、再执行。第二轮模型学到"过了顶点再往右，海拔骤降、奖励很低"，规划器就会走到顶上停住。
- **意义**：加上这个回环，它才真正成了一个 RL 算法——迭代地收数据、拟合（这里是动力学）模型、选动作、执行。图 11-2 里从执行回到拟合的那条边，就是这一步。
- **后面问答里的两个补充 (1:00:17)**
 - 模型误差反而帮助探索：规划器会被引到模型不准的地方（它在那里"以为"有高奖励），而那里正是最缺数据的地方。
 - 这是个 off-policy 算法：模型不挑数据是谁收的，用一个很宽的策略收数据反而更好——模型在更多地方准，前提是模型容量够。

小注：这个"边执行边把数据加回去"的循环和第 2 讲 DAgger 治误差累积的思路是同一个：让训练分布追上执行分布。区别是 DAgger 要专家来标

动作，这里只需要环境给出真实的下一状态。

7. 开环还是闭环：model predictive control



- **到目前为止是开环的：**在 s_t 规划出 $a_t \dots a_{t+H}$ ，然后把 H 个动作全执行完，中间到达的状态看都不看。控制里这叫 open loop——不用环境的反馈。
- **闭环：**同样规划 H 步，但只执行第一个动作 a_t ，拿到真实的 s_{t+1} ，再从 s_{t+1} 重新规划一段新序列。每一步都这样。
- **开环什么时候糟糕（学生答）：**环境变化大、发生了你没预料到的事时，开环不会做出任何反应；闭环因为每步都看最新状态，能及时改道，有时策略质量会好非常多。世界比较确定时可以稍微开环，但 H 不能太大。
- **这就是 model predictive control (MPC)：**规划一段 → 只执行第一步 → 观察 → 重规划；同时像第 6 节那样把新数据存起来、定期重训模型。它在机器人里极其常见（腿式机器人用它规划接下来几秒），低维的简单控制系统也常用。
- **重规划为什么值钱：**不只是应对环境变化——脚滑了、撞到了东西，下一步就能换一套完全不同的动作；它也能纠正模型误差。模型在状

态空间大部分地方准、少数地方不准时，闭环规划对这种局部错误鲁棒得多。

• 问答摘要

- 有多条同样好的路会不会来回摇摆？——会有，尤其每步都重规划时；但状态一旦明显显示你已经选定一边，往往就不摇了。
- 算力：每步都重新做一遍优化很贵。解法一是 warm start——把上一步的计划整体前移一格当本步的初始值，最后一格随机补上，优化几步就够，顺带也让你更愿意坚持上一步的选择；采样式方法可以在上一计划附近加噪声初始化，别只盯着那一条。解法二是折中：视野 100 步，但每 3 步或 10 步才重规划一次。
- 模型变好之后能不能少重规划？——完全可以；最终拿去用的模型和学习过程中的模型可以用不同的重规划超参数。

小注：MPC 的另一个名字是 receding horizon control——窗口随时间往前推。它在过程控制里用了几十年，这里只是把手写的物理模型换成了学出来的神经网络。

8. 规划的边界，以及怎么越过去

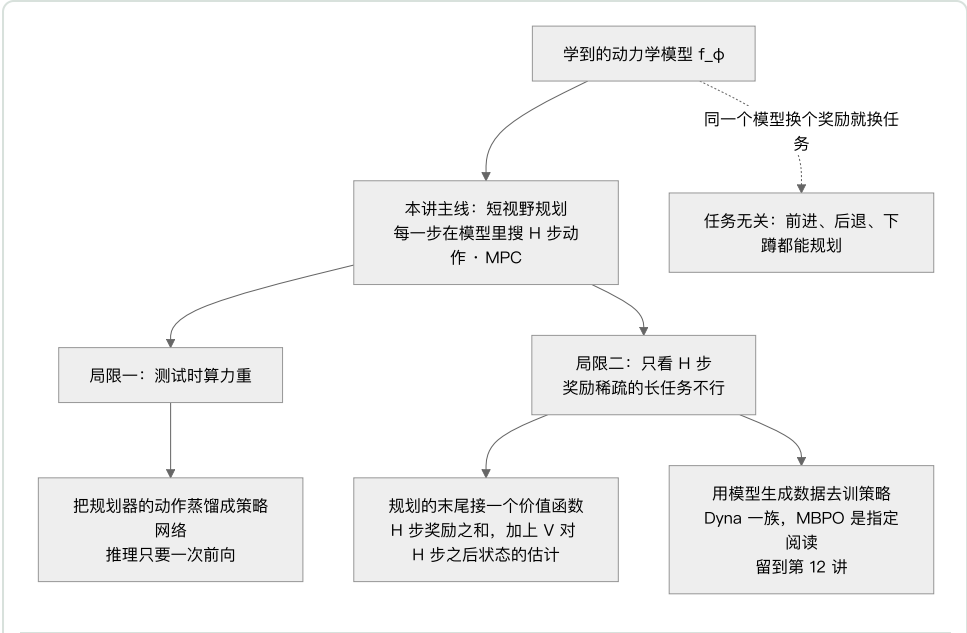


图 11-6 | 短视野规划的两个局限各自的出路；"用模型生成数据"那一支留到第 12 讲（自绘示意）· [▶ 看原幻灯片 1:03:27](#)

- 小结：规划 = 用梯度或采样找一段动作 → 执行 → 用执行数据更新模型 → 重规划纠错。
- 最迷人的一点：任务无关。模型学的是"世界怎么变"，不带任务。同一个模型在测试时换一个奖励或目标就是另一个任务：机器人往前走、

往后走、蹲下，全用同一个模型规划出来。这和 model-free 方法一个策略只会一件事形成鲜明对比。

• 两个局限

1. 测试时算力重：每一步都要跑一遍优化，模型大、规划费时不现实。
2. 只适合短视野。要想看得远，目前的办法只有把 H 拉长，优化维数随之膨胀，模型误差也随步数累积。奖励足够稠密时能补救：想学一个永远往前走的策略（无限视野），只要奖励是前进速度这种每步都有信息的 shaped reward，短视野规划就能近似长视野问题。奖励稀疏的长任务则不行——一方面是模型精度，另一方面是规划长序列本身太贵，而且这里根本没学策略。

- **出路一：蒸馏。**把规划器在各个状态选出的动作存下来，监督训练一个策略网络去模仿。推理只剩一次前向，解决算力问题；但动作还是规划器给的，视野问题没解决。
- **出路二：末尾接一个价值函数。**价值函数 $V(s)$ （第 4 讲）估计“从 s 出发按策略走下去还能拿多少回报”。规划时只显式优化 H 步，H 步之后到达的状态交给 V 打分，长期后果就折进来了。代价是要同时学模型和价值函数，部件多了很多——讲者因此不想多讲。

$$\hat{a}_{t:t+H} = \arg \max_{a_{t:t+H}} \left[\sum_{t'=t}^{t+H} r_{\psi}(s_{t'}, a_{t'}) + V(s_{t+H+1}) \right]$$

前一项是想象中 H 步的奖励，后一项用价值函数替代 H 步之外的全部未来；V 越准，H 就可以越短。

- **问答：那为什么不直接用 Q 函数？**——讲者的回答有两面。一面是这确实部件太多；另一面是如果模型很好甚至已知，在模型里 rollout 到算力允许的深度、再问价值函数，对“这个动作到底多好”的估计比单独一个价值函数准得多——游戏里这是标配。只有模型很好或很容易学好时才值得。

小注：她说的“游戏里”应指 AlphaZero、MuZero 这一类——树搜索在已知或学到的模型里展开若干步，叶子节点用价值网络估值，正是上面这个公式的搜索版本（推断，课上没点名）。

- **出路三：用模型生成数据去训策略。**讲者因为时间关系把这部分推到了下一讲，本视频没有讲。

小注：这条线是指定阅读 MBPO (Janner et al., 2019) 所在的位置，先做个预告。祖先是 Sutton 1990 年的 Dyna：模型充当“额外的环境”，生成

的转移和真实转移混在一起去更新策略或 Q 函数。MBPO 回答的是"多信模型一点还是少信一点": 从 replay buffer 里的真实状态出发, 只在模型里往前 rollout 很短的 k 步, 把这些短分支喂给 SAC; 模型是多个概率网络的 ensemble, 预测时随机挑一个成员。论文给了一个"每步单调改进"的界 (作者自己也承认过于保守), 实证上样本效率远好于之前的 model-based 方法, 渐近性能追平 model-free, 而且能撑到别的 model-based 方法失效的长视野。短 rollout 的道理和第 6 节的悬崖是一回事: 模型误差随想象步数累积, 走得越远越不可信。

9. 案例：PDDM——五指灵巧手上的规划

项目	设置
任务	手掌里转两个保定球 (Baoding balls); 手持笔写数字; 真机上把物体转 90° 或 180°
状态	手的各关节位置 (手上的传感器) 加物体的 3D 位置 (视觉系统估计, 或贴动作捕捉标记)
动作	控制五指手的 24 个关节
奖励	让物体沿目标轨迹走, 掉了球有惩罚
模型	很朴素: 两层各 500 个隐单元的全连接网络; 训 3 个不同随机种子、不同小批次顺序的副本组成 ensemble
规划器	改良的 CEM: 迭代重采样, 精英均值改成按奖励加权的软平均, 并对动作序列做时间平滑, 偏好平滑的轨迹
循环	收 30 条轨迹 → 更新模型 → 再收; 规划始终在执行回路里

- **为什么讲这个**: 讲者认为它仍是五指手上最令人印象深刻的结果之一, 而且对照实验清楚说明了 model-based 规划里什么最要紧。
- **ensemble 的用处**: 一个模型对某个动作过分乐观, 别的模型的误差会把它拉回来; 各模型错的地方不同, 规划器就没法只钻某一个模型的空子。这是对付第 2 节"误差被利用"的一种便宜办法。
- **仿真对照**: model-free 的 SAC 和 natural policy gradient (一种策略梯度法); model-based 的 MBPO、PETS (另一种规划器), 以及一个不用 ensemble、不迭代采样的版本 (相当于 random shooting)。
- **结果**
 - 转保定球这种难任务, 只有 PDDM 学会; 只做随机采样的版本 (黄线) 很差——采不到分布里好的那一块, 也没法收紧。
 - 写字任务简单些, SAC 和 NPG 也能达到高分, 但数据量差得远: model-based 大约 10 万步, model-free 大约 50 万步。它也比其他 model-based 方法更强。

- 三条消融：模型要够大；ensemble 不能少；规划视野有取舍——太长则维度太高，反而变差。

- **真机**：五指手硬件极其脆弱（Gates 楼里就有一只，学生都不敢碰，怕修不起），所以数据效率生死攸关。4 小时真机数据后，90° 转动成功率约 100%，180° 更难、成功率低一些但已很可观。球掉了由另一台机器人负责捡回来重置。
- **下一讲**：model-based RL 的最后一部分（用模型生成数据），然后进入多任务与目标条件 RL（第 12 讲）。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
算法总览一页	横向从 on-policy 到 offline，越往右越依赖在线数据；三种颜色对应只学策略、策略加 critic、只学 critic	0:38	—
学模拟器：物理模型 vs 视频预测	Veo 2、Sora 生成的视频；模型也可以是股市、游戏规则、其他玩家	5:52	—
算法草图	收数据 → 拟合 $f(s, a) \approx s'$ → 在模型里跑 RL 或规划；讲者让全班找茬	8:56	—
迷宫与 Sora 叠方块	数据覆盖不到的区域模型瞎猜；视频模型会变色、凭空长出方块	11:05	—
反传到动作的板书四步	收数据、训模型、对动作做梯度更新、执行后回到第三步；减号应为加号	23:16	—
梯度法 vs 采样法的曲线	一维奖励曲线上，梯度法沿斜率走，采样法撒点再往好点附近加密	32:29	—
Cross-entropy method	采样 → 排序 → 取精英 → 拟合高斯 → 重采样；别和 cross-entropy loss 混	37:45	—
两类优化器的优缺点	梯度法可扩展但要光滑；采样法可并行、免梯度但高维吃力；random shooting 与 CEM 两版规划	43:28	—
爬山掉悬崖	模型只见过“往右更高”，规划一路向右；把执行数据加回去后学会停在顶上	48:43	—
开环 vs 闭环的板书	一口气执行 H 步 vs 每步只执行第一个动作、拿到新状态再重规划	51:51	—
MPC 一页	规划、执行首动作、观察、重规划、定期重训模型；对模型误差鲁棒	55:03	—
规划的优点与局限	任务无关是优点；算力重、短视野是局限；蒸馏成策略、末尾接价值函数是出路	1:01:18	—
PDDM 的设置	24 关节手、3 模型 ensemble、改良 CEM、每 30 条轨迹更新一次模型	1:07:37	PD DM

图	看什么	跳转	出处
PDDM 的视频与曲线	只有它能转保定球；写字任务 model-based 约 10 万步对 model-free 约 50 万步；消融看模型大小、ensemble、视野	1:10:41	同上

提到的工作

名称	在本讲里的作用
REINFORCE / vanilla policy gradient (第 3 讲)	总览图里 on-policy 的代表
PPO、importance sampling (第 5 讲)	轻度 off-policy：可复用梯度步之前的数据
DQN、SAC (第 5–6 讲)	replay buffer 一档的代表；SAC 又是案例里的 model-free 对照
AWR、AWAC、IQL、CQL (第 7 讲)	离线 RL 的两类：只看数据里的动作 / 靠保守性约束
behavior cloning、DAgger (第 2 讲)	模仿学习的离线与在线版本
Veo 2、Sora	"学到的模拟器"长什么样的例子；Sora 叠方块的视频说明物理未必靠谱
奖励学习 (第 8 讲)	规划需要的奖励模型 r_ψ 从哪来
random shooting	一轮采样、取最优的规划器
cross-entropy method (CEM)	迭代收紧采样分布的通用优化器；本讲采样式规划的主力
finite differences	问答：没有梯度时用相近样本的差估梯度
model predictive control (MPC)	只执行第一步就重规划的闭环规划；机器人里的标配
natural policy gradient (NPG)	案例里的另一个 model-free 对照
PDDM (Nagabandi et al., 2019)	案例主角：ensemble 加改良 CEM 的灵巧手规划
MBPO (Janner et al., 2019)	案例里的 model-based 对照；本讲指定阅读；机制属于下一讲
PETS (Chua et al., 2018)	案例里的另一种 model-based 规划器
ensembles	3 个模型互相制衡，防止规划器钻单个模型的空子
游戏里的 rollout 加价值函数	规划末尾接价值函数的现成范例（应指 AlphaZero / MuZero 一类，推断）

术语对照

English	中文
model-based RL / model-free RL	基于模型 / 无模型的 RL：学不学环境的动力学模型
dynamics model / transition model / simulator	动力学模型 / 转移模型 / 模拟器：读入 (s_t, a_t) 预测 s_{t+1}
action-conditioned video prediction	以动作为条件的视频预测：看不到底层物理量时的模型形态
latent representation z_t	潜表示：把图像压成的低维向量，模型在这个空间里预测
reward model r_ψ	奖励模型：给预测出的状态打分，规划时和动力学模型配套
planning	规划：站在当前状态往前想 H 步，搜一段动作让想象中的奖励最大
horizon H	视野：往前规划的步数
backpropagation into actions	反传到动作：梯度对的是动作序列，网络参数冻结
gradient ascent	梯度上升：沿梯度方向最大化（这里是奖励）
sampling-based / gradient-free / zeroth-order optimization	采样式 / 免梯度 / 零阶优化：只用函数值不用梯度
random shooting	随机射击：采一批动作序列，取模型里奖励最高的那条
cross-entropy method (CEM)	交叉熵方法：采样、排序、取精英、拟合高斯、重采样的迭代优化器
elites	精英：每轮得分最高的那几个样本
finite differences	有限差分：用相近两点的函数值之差近似梯度
distribution mismatch / data coverage	分布错配 / 数据覆盖：模型只在数据到过的地方准
model exploitation / compounding errors	利用模型误差 / 误差累积：优化器专挑模型错的地方，且一步错步步错
open loop / closed loop	开环 / 闭环：执行时不看 / 看环境反馈
model predictive control (MPC) / replanning	模型预测控制 / 重规划：规划一段、只执行第一步、观察、再规划
warm start	热启动：用上一步的计划前移一格当本步的初始值
task-agnostic	任务无关：同一个模型换奖励就换任务
shaped / dense reward vs sparse reward	稠密奖励 vs 稀疏奖励：每步都有信息 vs 只在终点有信息
distillation	蒸馏：用规划器的动作监督训练一个策略网络
value function V(s)	价值函数：从 s 出发继续走下去的期望回报；规划末尾用它替代 H 步之外的未来

English	中文
ensemble	集成：多个随机种子不同的模型，各自的误差互相制衡
degrees of freedom (DoF)	自由度：可独立控制的关节数，案例里的手是 24
data / sample efficiency	数据效率：达到同样性能要多少交互数据
online / offline; on-policy / off-policy	在线 / 离线：能否继续收数据；同策略 / 异策略：数据是否必须来自当前策略
policy gradient / actor-critic / Q-learning	只学策略 / 策略加 critic / 只学 critic 的三类方法

字幕勘误

字幕是人工校对的，错词很少："SQL" → CQL (conservative Q-learning，上下文说的是"靠保守性")；"bounding balls" → Baoding balls (保定球)；"[? soft ?] actor critic" → soft actor-critic，即 SAC；"video production" → video prediction；"masonries" → 应为 many scenarios；"gearing a given thing" 一处含混，讲的是转向和挡位。另外两处不是字幕问题：开头 online 那一组被讲者说成了 offline；动作更新式里的减号应为加号，讲者在 32:29 自己更正了。

带走的问题

- 掉悬崖的例子，模型错在"没见过的地方外推"，把执行数据加回去就能修——但这依赖"能安全地掉一次悬崖"。在你做的 agent 系统里（比如让 LLM agent 在工具环境里试错），哪些错误允许发生一次再修，哪些不行？后者是不是正好要靠 ensemble 的分歧来事先挡一下？
- 规划的优化维度是动作维数乘 H，PDDM 又发现视野太长反而变差。除了维度之外，还有什么原因让长视野的规划更糟？如果模型是完美的，视野越长是不是一定越好？
- MPC 每步都重规划，本质是把算力花在测试时；蒸馏成策略是把算力搬回训练时。这和 CS329A 里 test-time compute 与训练的取舍是同一件事吗？什么条件下你愿意在推理时花 100 倍算力？
- "只在模型里做 RL"会被优化器钻空子，和第 8 讲奖励模型被钻空子、第 6 讲 Q 函数的过估计同源。MBPO 的答案是只信短 rollout，PDDM 的答案是 ensemble。还有哪些"什么时候该信模型"的判据？在

LLM agent 里, "模型"换成对工具和环境的预期时, 这个问题变成了什么?

5. 讲者说模型误差反而帮助探索——规划器会被引到模型不准的地方, 那里正是需要数据的地方。这和第 14 讲要讲的不确定性驱动的探索是什么关系? 什么时候这种"误打误撞的探索"会变得危险?

CS224R 第 12 讲 | 多任务与目标条件 RL (Multi-Task RL)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 12 讲, 2025 年 5 月 9 日 · 前 27 分钟先把第 11 讲的 model-based RL 收尾 (用学到的模型造合成数据、什么时候值得学模型), 之后才进入多任务; hindsight relabeling 是作业要重新实现的内容 视频:

https://www.youtube.com/watch?v=qNdsI_4AQJw (1:10:29, 英文字幕为自动生成, relabeling、MDP、Euclidean 这类词几乎都错, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座) 课程主页:

<https://cs224r.stanford.edu/> · 指定阅读: Hindsight Experience Replay (Andrychowicz et al., 2017; 课程页面这一条误链到了一篇元学习论文, 那篇属于第 13 讲, 以这篇为准)

一句话: 前半小时收尾 model-based RL——学到的模型除了拿来规划, 还能当"学出来的模拟器": 从真实数据里的状态出发跑几步短 rollout, 造出来的合成数据和真实数据一起喂给任何 RL 算法, 等于用算力换真实交互; 值不值得学模型, 要看模型是不是比策略更好学。后半段讲多任务 RL: 一个任务就是一个 MDP, 给每个任务一个标识符 z (编号、一句话、一段视频、或者一个目标状态) 并进状态里, n 个 MDP 就合成一个大 MDP, 一个 $\pi_\theta(a | s, z)$ 通吃, 标准的模仿学习和 RL 照常跑。RL 里还多一手: 为任务 i 收的数据可以事后按任务 j 的奖励函数重新打分 (hindsight relabeling) ——想射门却传出一记好球, 对"传球"来说就是成功样本; 前提是动力学相同、奖励能算出来、算法是 off-policy 的。目标条件 RL 是它最漂亮的特例, 指定阅读 HER 讲的就是把没到达的目标换成实际到过的状态。

时间轴

时间	内容
0:06	开场与今日安排; 回顾第 11 讲: 学模型 → 规划这条路的优缺点
3:09	模型的第二种用法: 当学出来的模拟器, 从真实数据里的状态出发做短 rollout
7:48	板书算法; 问答: 短 rollout 会不会拖慢收敛、奖励从哪来、算力换数据、稳定性、兼容 on-policy / off-policy
16:07	什么时候用 model-based RL: 三大优点、三大缺点
19:40	问答: 额外超参有哪些; 在线与离线版本; 迷宫 vs 倒水; 机器人上什么时候管用

时间	内容
24:20	模型的其他形态：逆模型、多步逆模型、不带动作的未来预测、状态插值
27:23	多任务 RL 是什么、为什么要做：通才策略、共享结构摊薄数据、通才更可靠
32:02	任务 = 一个 MDP；四个例子里各自什么在变；任务标识符：编号、语言、视频
38:14	板书：把任务标识符并进状态、n 个 MDP 合成一个；目标条件 RL 与负距离奖励；问答：one-hot 之外
44:25	多任务模仿学习：loss 对任务取平均；分层抽样的 mini-batch；逐任务归一化
48:08	架构：语言 + 视频双编码器；LLM 的 prompt 就是任务描述；VLA 式的 transformer 策略
52:11	问答与演示视频：IL vs RL；中途切换指令；一个策略 vs 每任务一个；层级化留到第 15 讲
57:20	多任务 RL：任务描述也喂给 Q 和 V；每任务一个 replay buffer；数据共享要重标奖励
58:22	冰球例子；板书 relabeling 算法；问答：挑哪个任务 j；这就叫 hindsight relabeling
1:05:07	什么时候能重标：同状态动作空间、动力学必须相同、要有奖励函数、必须 off-policy
1:09:47	目标条件版本：负距离奖励的三种形态；本讲到此结束

核心内容

1. 收尾 model-based RL：模型的第二种用法是当"学出来的模拟器"

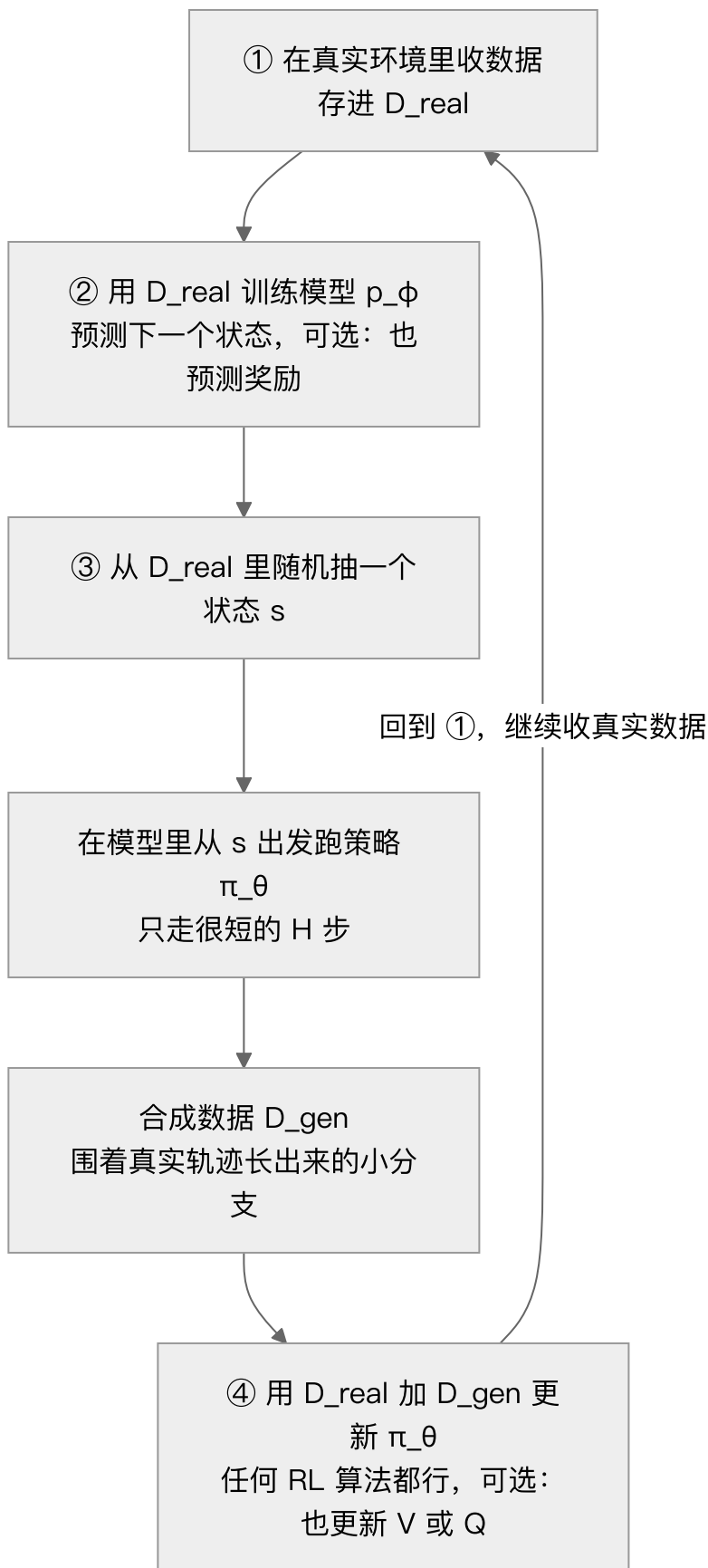


图 12-1 | 把学到的模型当模拟器：从真实状态出发做短 rollout，用合成数据补贴策略更新（自绘示意） · [▶ 看原幻灯片 13:00](#) · 出处：Janner et al., 2019 (MBPO；课上没点名，应为)

- **上一讲的回顾：**model-based RL（基于模型的 RL）先从数据里学一个模型（dynamics model） $p_\phi(s_{t+1} | s_t, a_t)$ ——给定当前状态和动作，预测下一个状态，可以是确定性的，也可以是一个分布。第 11 讲拿它做规划（planning）：不学“状态 $\rightarrow$ 动作”的映射，而是在每个状态现场用梯度或采样优化未来一小段 horizon（时间跨度）上的动作序列，边跑边更新模型、定期重新规划。优点是简单，模型够通用时换目标、换奖励很方便，甚至测试时就能换；缺点是测试时算力大，而且只对短 horizon 或奖励很密（shaped reward）的问题实用——horizon 一长，算力更贵，模型被反复套用后误差也会累积。
- **第二种用法：用模型来学策略。**思路是把它当成一个学出来的模拟器，给手头的数据“增补”。课堂上学生提了两种做法：从真实轨迹的末尾接着往下生成；或者像第 2 讲的 DAgger 那样，前半段由策略在真实环境里走、后半段交给模型接。Finn 的推广是：起点不必是末尾，数据集（包括离线给你的数据）里任何一个状态都可以当起点。
- **rollout 一定要短。**rollout 指从某个状态出发、按策略一步步走出来的一段轨迹。模型走远了就不准，所以合成轨迹不要和真实轨迹一样长，而是做成围着真实轨迹长出来的许多小分支，给数据周围补一圈覆盖——实践里这种效果最好。三种造数据的方式对比：从初始状态生成完整轨迹（长 horizon 上不准，覆盖也差）；从初始状态只生成一小段（覆盖不到后面的状态）；从真实数据里的所有状态出发各生成一小段（短 rollout 加全程覆盖）——取第三种。
- **算法**（板书和幻灯片各写了一遍，图 12-1）：① 在真实环境收数据 D_{real} ；② 用它更新模型，幻灯片版里模型也可以顺带学预测奖励；③ 从 D_{real} 里抽状态 s ，在模型里从 s 出发按当前策略（或某个探索策略）走一小段，得到合成数据 D_{gen} ；④ 用 D_{real} 和 D_{gen} 一起更新 π_θ ，可选地也更新价值函数；然后回到 ①。第 ③ 步的一条合成轨迹就是：

$$s_0 \sim \mathcal{D}_{\text{real}}, \quad a_k \sim \pi_\theta(\cdot | s_k), \quad s_{k+1} \sim p_\phi(\cdot | s_k, a_k), \quad k = 0, \dots, H-1, \quad H \ll T$$

起点 s_0 从真实数据里抽，之后每一步的动作由当前策略 π_θ 给、下一个状态由学到的模型 p_ϕ 给，只走 H 步； H 远小于真实回合长度 T ，这是整个配方的关键。

- **问答里补齐的细节**

- 只做短 rollout 会不会拖慢收敛？——合成数据是对真实数据的补贴，不要求自己走到目标：靠近目标的起点很快会碰到奖励，远处的分支会连回数据集里的状态，第 4 讲起的 bootstrapping（用对后续状态的估值来更新当前估值）会把信号传回去；它甚至能让策略在真实数据没走过的地方发现捷径。真正想省的是**真实数据**的量。
- 合成数据的奖励从哪来？——要么奖励函数已知，要么像第 8 讲那样学出来，要么就让这个模型（或另一个模型）用监督学习顺带预测奖励。没有奖励的估计，这套方法就没法用。
- 算力？——训练时更贵（要训模型，还要在模型里 rollout），本质上是拿算力换环境交互，真实交互不安全或很贵（物理设备）时划算。和规划不同，测试时不额外花算力，最终跑的就是一个普通策略。
- 稳定吗？——生成多少合成数据、更新时怎么配比，取决于你多信任模型。真正的新麻烦是多了一整套超参：模型的架构（生成图像时可能和策略完全不同）、学习率、epoch 数、正则化，再加上 rollout 长度和合成数据量。模型训得好一般没问题；模型训砸了，整个算法跟着变差。
- 第 ④ 步兼容各种 RL 算法，包括 on-policy 的（要求数据来自正在更新的那个策略）——只要合成数据也是 π_θ 自己在模型里采的就行。也有方法只用模型数据更新策略，效果也不错。
- 板书写的是在线版本（不断收新的真实数据）；也有完全离线的版本：固定一份数据集，用模型增补，训完策略就结束。更新时只用本轮的 D_{real} 、 D_{gen} 还是连同以前的一起用，决定了它是 on-policy 还是 off-policy——模型本身非常灵活。

小注：这个“真实转移 + 模型里的短分支”的配方，源头是 Sutton 1990 年的 Dyna；深度 RL 里的标准版本应为 MBPO (Janner et al., 2019, 第 11 讲的选读)：rollout 从 1 步起逐渐放长，策略更新用的数据绝大多数来自模型。

2. 什么时候值得学模型

	第 11 讲：拿模型做规划	本讲：拿模型造合成数据
学不学策略	不学，测试时现场优化动作	学，策略照常用 RL 更新
测试时算力	大	和普通策略一样

训练时算力	训模型	训模型，还要在模型里 rollout
对 horizon 的要求	只能短	rollout 短，但价值传播能把信号接到长 horizon
换任务或奖励	方便，甚至测试时换	要重训策略，模型本身可以复用

• 大优点（三条）

1. 只要能学到一个还不错的模型，"预测未来"用处极大：真实数据可以少得多，还能顺带做测试时规划。Finn 补了一句幻灯片上没有的话：能预见自己动作的后果，本来就该是模型的一种基本能力。
2. 模型可以在**没有奖励标签**的数据上训练。真实数据里往往只有一小部分带奖励，model-free 方法只能用那一部分，模型却能把没奖励的大头也吃进去——大量无奖励数据加少量有奖励数据，是很常见的处境。
3. 模型是任务无关的：在足够宽的数据上训出来，可以跨任务、跨奖励函数复用。如果目标本来就不是学一个任务而是学很多个，这一点直接引出本讲的下半场。

• 大缺点（三条）

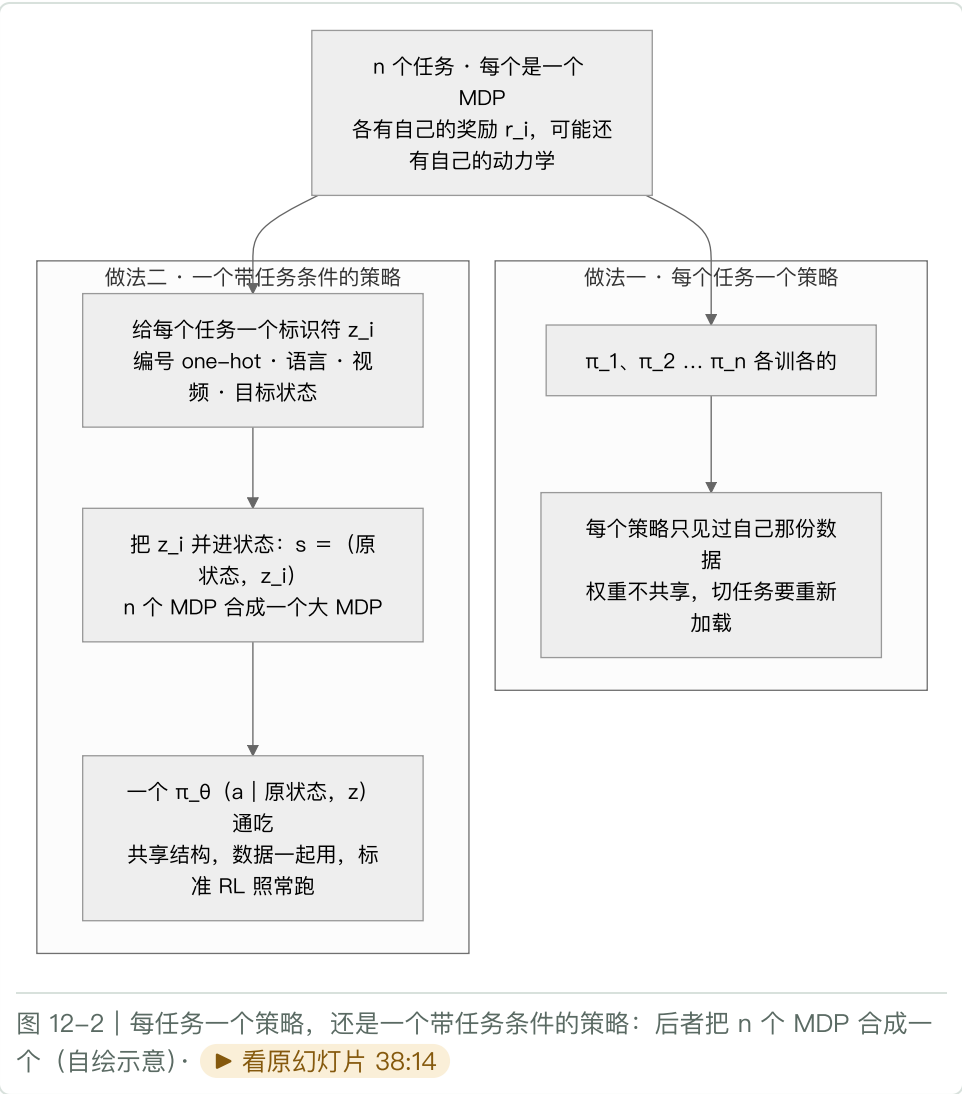
1. 模型优化的不是任务表现。以图像为状态时，它忙着把每个像素对上（天空的颜色对不对），而这往往和"学出好策略"无关：训练目标和下游目标错位。
2. 有时模型比策略更难学。这种情况下学模型等于先给自己出一道更难的问题。
3. 多一样要训的东西：更多超参、更多算力。

- **模型和策略哪个更难学，完全看领域**（问答）：迷宫导航里，模型极简单——往右走就往右挪，除非撞墙——策略却很复杂，每个位置都要往不同方向拐，这时学模型划算；倒水时，策略可能只是转一下手腕，模型却要懂流体力学、水碰到碗边和碗心会怎样，这时直接学策略更省事。

- **机器人上的战绩**（问答）：状态是紧凑的关节角、物体位置时相当成功——四足机器人的"盲走"、上一讲看过的灵巧手转球（已知两个球的位置），后者用 model-free 甚至模仿学习都会非常难；状态是高维图像时则明显吃力。视频生成模型在变强，Finn 觉得这条路未来可能会通，但现在还没有。

- **模型不止一种形态**（最后一张幻灯片）：除了“给 s 、 a 预测 s' ”的前向模型，还可以学逆模型（给 s_t 和 s_{t+1} ，反推动作 a_t ）、多步逆模型（给 s_t 和 s_{t+N} ，反推一串动作）——它们更像策略，只是多了一个未来状态当输入，而且不需要奖励；不带动作的未来预测（环境里可能发生什么，即 affordance）；两个状态之间的插值；转移的生成模型。数据源不同，各有用处。
 - 问答：模型用了无奖励数据，会不会预测状态很准、预测奖励很差？——可能，也可能反过来。一般奖励比状态好预测，但确实会遇到奖励估计不准、要专门补数据的情况。

3. 多任务 RL：一个任务就是一个 MDP，把任务标识符并进状态



- **要解决什么：**不想只训一个“会倒水”的策略，想要一个**通才策略** (generalist policy)：能订旅行也能网购的语言模型助手；能走、跑、跳舞、蹲的四足机器人；能挂毛巾、卸洗碗机、擦水渍的移动机器人；给不同的人做个性化推荐的音乐推荐系统；能玩 Pokemon、弹

珠台和 Flappy Bird 的游戏 agent。把每个任务看成独立的 RL 问题，它们的奖励函数可能不同，动力学可能不同（不同的游戏、不同的用户），甚至动作空间都可能不同。

- **核心思路一句话**：照常做 RL，只是策略除了看状态，还要看"现在要做哪个任务"—— $\pi_\theta(a \mid s, z)$ 。
- **为什么要一起训**：这门课反复回到数据效率——要多少数据才能算出一个靠谱的梯度。如果目标本来就是做很多任务，学任一任务的成本可以在任务之间摊薄，因为任务之间通常有共享结构（语言助手要懂的语法、不同网站相似的操作；机器人前进和后退都得先保持平衡；口味相近的用户）。更深一层的理由是经验性的：近几年反复看到，通才系统比单任务系统更可靠、表现更好。
- **"任务"是什么**：其实没有严格定义。一种可用的定义是，不同任务就是不同的 MDP（Markov decision process，第 1 讲的记号）：各有自己的状态空间、动作空间、初始状态分布、转移模型和奖励函数，用下标 i 区分。

$$\mathcal{M}_i = (\mathcal{S}_i, \mathcal{A}_i, p_i(s_1), p_i(s' \mid s, a), r_i(s, a)), \quad i = 1, \dots, n$$

第 i 个任务 $\mathcal{M}_i$ 由五样东西定义：状态空间 $\mathcal{S}_i$ 、动作空间 $\mathcal{A}_i$ 、初始状态分布 $p_i(s_1)$ 、转移模型（动力学） $p_i(s' \mid s, a)$ 和奖励函数 $r_i(s, a)$ ；任务不同，就是这五样里至少有一样不同。

- 注意 MDP 意义上的"任务"和日常语义不是一回事：在草地上往前走和在平地上往前走，奖励函数相同、动力学不同，按这个框架算两个任务，虽然都叫"走路"。
- **四个例子，各自什么在变**（课堂问答一起过的；策略在所有例子里都会变）：

例子	一个任务是什么	跨任务在变的量	不变的量
个性化音乐推荐	一个用户	奖励（有人爱摇滚、有人爱嘻哈）、动力学（听歌频率之类的行为模式）	—
角色动画	一种动作：后空翻、跑步、侧手翻	奖励	物理引擎的动力学、初始状态（都从站立开始）
角色穿不同衣物	一件衣物：短袖、长袖、连衣裙	动力学（布料的物理）、初始状态	—

例子	一个任务是什么	跨任务在变的量	不变的量
不同机械臂抓东西	一台机械臂：4、6、7 个自由度	状态的维度、动作的维度、动力学	—

- **任务标识符** (task identifier, 记作 z_i)：策略得知道现在该做哪个任务。最朴素的是编号——这台机械臂是任务 0、那台是任务 1——实现上就是 one-hot 向量 (第 k 位是 1、其余全 0)，没有更好办法时就用它；也可以是一句语言描述、一段演示视频，形式很自由。
- **板书：把 z 并进状态**。原来是一堆 MDP，现在构造一个覆盖全部任务的单个 MDP：新的状态是"原状态加任务标识符"，奖励和转移按 z 分派给对应任务的函数。

$$s = (\bar{s}, z_i), \quad r(\bar{s}, z_i, a) = r_i(\bar{s}, a), \quad p(s' \mid \bar{s}, z_i, a) = p_i(s' \mid \bar{s}, a)$$

$\bar{s}$ 是原来的状态， z_i 是任务标识符；合成后的大 MDP 里，奖励和转移都只是"查一下 z 是谁，再调用那个任务自己的 r_i 和 p_i "。这个构造要求各任务的状态空间和动作空间一致；维度都不一样时，得先想办法统一表示。

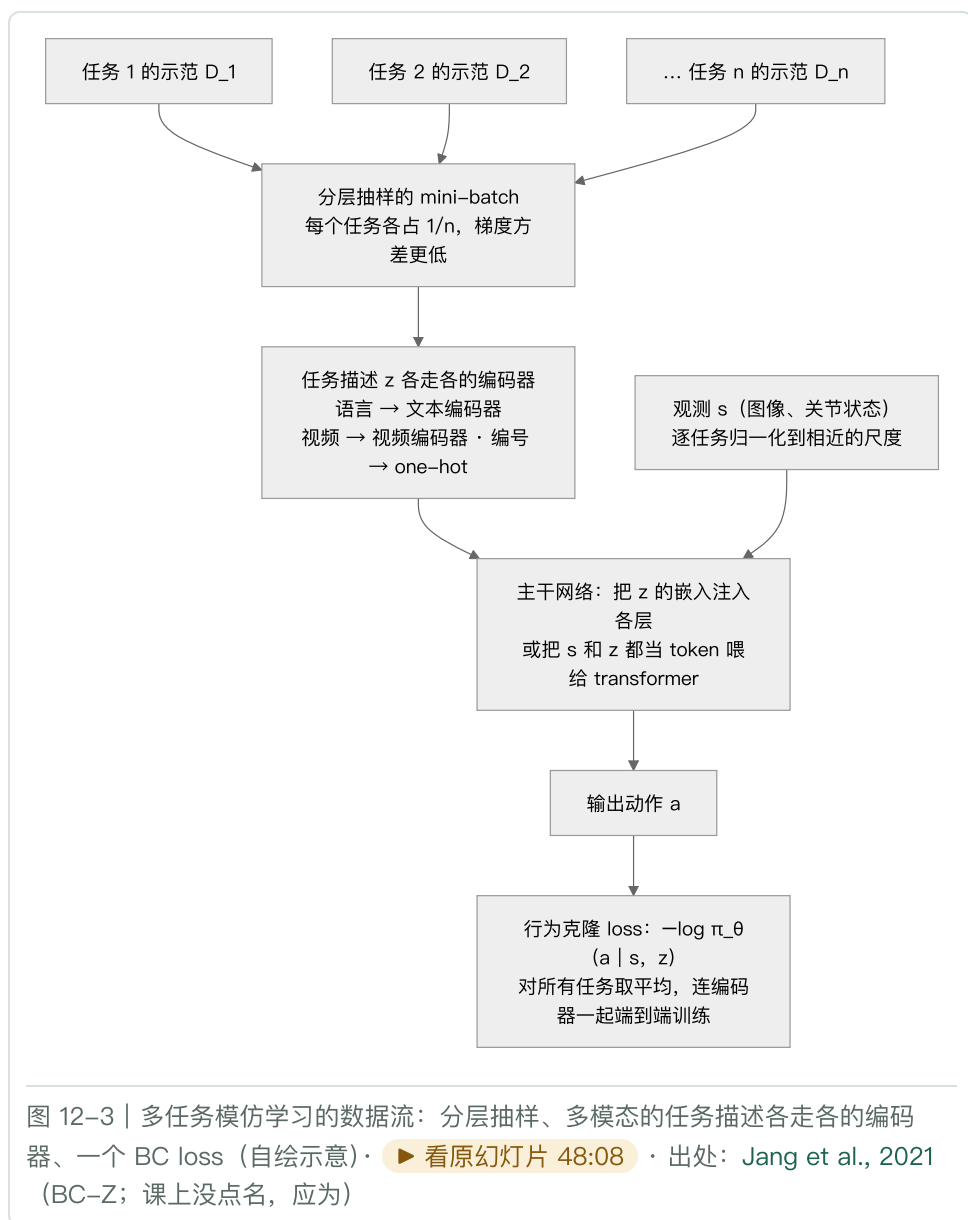
- **目标条件 RL** (goal-conditioned RL) 是特例： z 就是一个想到达的目标状态 g ，奖励通常写成当前状态到目标的负距离——欧氏距离 (问答里确认：L2 就是均方误差)、稀疏的 0/1，等等。

$$r(s, a, g) = -d(s, g), \quad d(s, g) \in \left\{ \|s - g\|_2^2, \|\varphi(s) - \varphi(g)\|, \mathbb{1}[\|s - g\| > \epsilon] \right\}$$

g 是目标状态， d 是某种距离：可以是状态空间里的均方误差，可以先用编码器 ϕ 映到嵌入空间再算距离，也可以是稀疏的 0/1——只要不在目标的 ϵ 球内就算没到 (本讲结尾给的三种)。

- 合并之后仍然是标准 MDP，第 3—6 讲的算法直接可用，很多场景下这就是好办法。但 Finn 铺垫说，有些情况下利用"这个大 MDP 是由若干任务拼起来的"这一结构能做得更好，目标条件 RL 正是这样的情况——这就是第 5—7 节。
- **问答**：把用户当任务时，one-hot 是不是太粗？——是的，RL 里 one-hot 很常见但确实有更好的表示：用一段语言描述用户偏好；或者把用户喜欢和不喜欢的几个例子直接放进 z —— z 里含有关于任务的数据，策略做的就成了上下文学习 (in-context learning)，是下周第 13 讲元 RL 的内容。

4. 多任务模仿学习：loss 对任务取平均，再加两个小技巧



- **设定：**任务数 n 不大，每个任务 i 有一份专家示范 D_i 。第 2 讲的行为克隆是最小化 $-\log \pi_{\theta}(a | s)$ ，也就是让策略给专家动作的概率尽量高；多任务版本只是用同一组参数 θ ，把各任务的 loss 取平均：

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}_i(\theta), \quad \mathcal{L}_i(\theta) = \mathbb{E}_{(s,a) \sim \mathcal{D}_i} [-\log \pi_{\theta}(a | s, z_i)]$$

$\mathcal{L}_i$ 是第 i 个任务上的行为克隆 loss， (s, a) 从该任务的示范里抽，策略同时看到状态 s 和任务标识符 z_i ； n 个任务共用一个 θ 。

- **技巧一：分层抽样的 mini-batch。**如果任务差别很大——一个让机器人往前走、一个往后走、一个躺下——随机抽的 mini-batch 可能这批几乎全是前进、下批几乎全是躺下，相邻两步的梯度方向完全不同，方差很大。理论上网络该靠 z 把它们分开，实际上不总那么容易。办

法是让每个 mini-batch 都含有每个任务的数据，想按任务均匀优化就各占 $1/n$ ，梯度方差会低不少。

- **技巧二**（幻灯片上没有）：逐任务把状态和动作归一化到相近的尺度。
- **架构：输入本身是多模态的。**策略吃 (s, z) ，可 z 可能是一句话或一个 one-hot， s 可能是图像。Finn 举了一篇她喜欢的老论文（还没用 transformer）：机器人做简单的操作任务，图像先降采样再进主干；任务描述有两种——一句话“把瓶子放进陶瓷碗”，或者一段人做这件事的视频——分别走语言编码器和视频编码器，嵌入被“乘进”主干的各层；训练时随机用其中一种描述，loss 就是行为克隆，编码器和主干一起端到端训。

小注：这篇应为 BC-Z (Jang et al., 2021)：语言或人类视频的预训练嵌入做条件，“乘进各层”的做法是 FiLM 式调制；100 多个任务上训练后能零样本泛化到没见过的任务描述。

- **更现代的架构：**语言模型的任务描述天然就是 prompt——从这个角度看，LLM 的整个 SFT (CME295 第 4 讲) 就是“不同 prompt 当不同任务”的多任务模仿学习。机器人这边现在也常用 transformer：预训练的图像编码器把图像投到语言模型的输入空间，任务串“机器人该怎么做才能把茄子放进碗里”也切成 token 一起喂进去，输出动作，整个模型连图像和文本编码器一起端到端训。 z_i 在这里就是 prompt。

小注：这一页应为 RT-2 (Brohan et al., 2023)：把动作也写成文本 token，让视觉-语言模型直接输出动作，靠网络数据带来语义泛化；课上那句提问式的任务串就是它的 prompt 格式。

- **问答：和 RL 比呢？**——任务条件这一招 IL 和 RL 都能用；取舍和前几讲一样：模仿学习用不了奖励、超不过示范者，但简单、容易堆到大数据；RL 有奖励信息时能明显超过给定的数据。
- **演示：**一段视频里只有铺毛巾、挂毛巾两个任务——毛巾铺得差不多时，人当场把指令切成“挂毛巾”，同一个 transformer 策略接着做；另一段有十几条语言指令：把盘子放进抽屉、关微波炉、底盘后退、底盘左转……**任务的抽象层级可以不一样**，“往前开”“右夹爪放低”和“把餐巾扔掉”都算任务。同一个模型还训过看图问答——模型够大就不难，但要做的事越多，模型得越大。
- **问答：一个策略，还是每个任务一个？**——视频里是一个策略加 prompt。每任务一个策略意味着完全不共享权重，每套权重只见过自己那份更小的数据，通常更差；还要存更多权重，切任务时可能得重

新加载。但把很多任务塞进一个模型确实会有优化上的麻烦，有些场合分开训反而合适。

小注：课上说的"优化上的麻烦"，最常被点名的是任务间梯度冲突——两个任务的梯度方向相反，一起平均就互相抵消；PCGrad (Yu et al., 2020, Finn 是作者之一) 把冲突的分量投影掉，蒸馏（先各训各的再合成一个学生）是另一条路。本讲都没展开。

- **问答：更高层的任务** ("把厨房收拾干净") ——视频里的都是单个扁平策略，能做一些层级不低的任务，但直接说"收拾厨房"它做不了，层级化留到两周后的第 15 讲。**长 horizon 任务**指要走很多步才能完成的任务，麻烦在于该做什么的选择太多（多模态）、奖励稀疏、反馈来得很晚。

5. 多任务 RL 与 hindsight relabeling：一次失误对另一个任务可能是成功

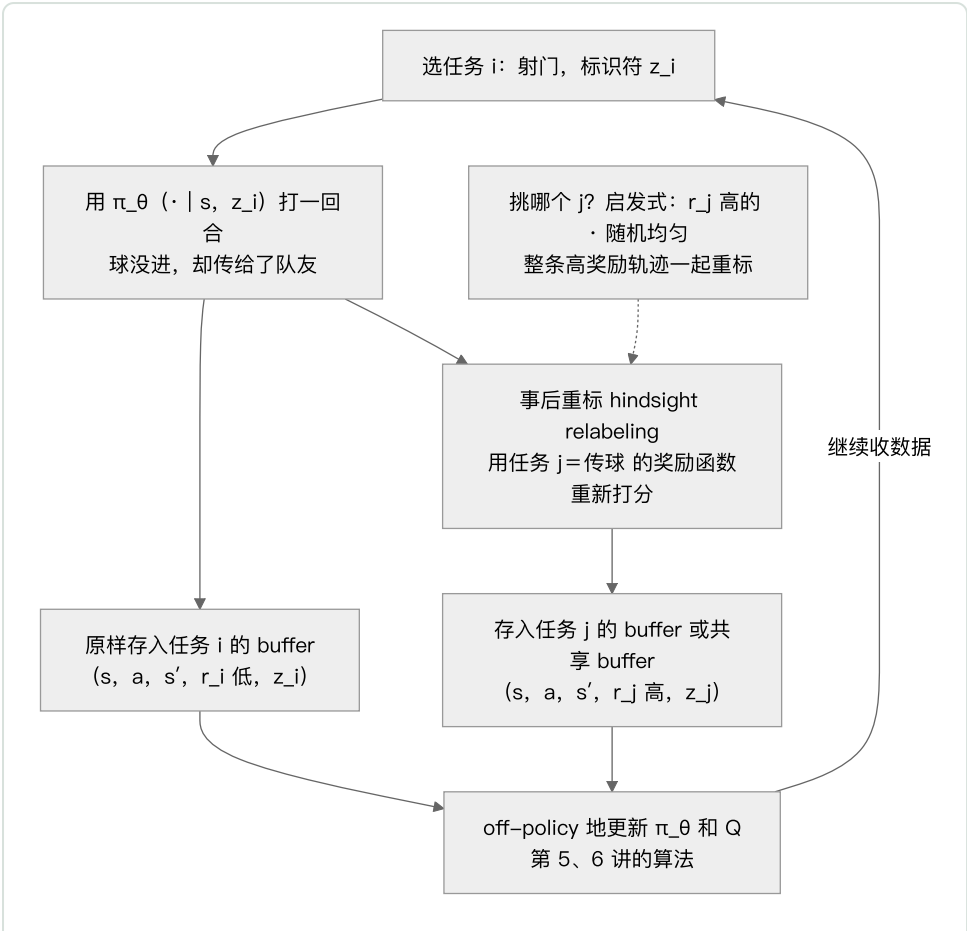


图 12-4 | hindsight relabeling 的数据流：一段经验存两份，第二份换上另一个任务的奖励和标识符（自绘示意） · [▶ 看原幻灯片 59:24](#) · 出处：Andrychowicz et al., 2017

- **多任务 RL 的做法和 IL 一样**：任务描述 z 不但喂给策略，也喂给 Q 函数或价值函数——凡是在学的东西都接上这个“prompt”。任务数离散且不多时，给每个任务单独开一个 replay buffer（第 5 讲：存历史转移供反复抽样的仓库）会方便些，尤其想控制每个 batch 里各任务的配比时。
- **RL 独有的新问题：数据共享**。为任务 i 探索时收的数据——比如让策略“拿起海绵”，它没拿到——对别的任务也可能有用；可是按任务分桶之后，这段经验永远不会拿去更新别的任务。想让它帮到另一个任务，就得**重标** (relabel)：动力学相同、奖励函数不同时，把奖励标签换成另一个任务的。
- **冰球例子**：蓝色是队友，橙色是对方守门员，任务一是传球，任务二是射门。你想射门，打歪了，球却恰好到了队友脚下——对射门是失败，对学传球却是一条好样本。做法：这段经验照常以射门的 z 存一份；再把奖励换成传球任务的奖励、标识符换成传球的 z ，存进传球任务的 buffer（或共享 buffer）。
- **板书算法**（图 12-4）：选一个任务 i 和它的 $z_i \rightarrow$ 用 $\pi_\theta(\cdot | s, z_i)$ 收数据，每条转移是 $(s_t, a_t, s_{t+1}, r_t, z_i)$ ，存进任务 i 的 buffer $\rightarrow$ 对另一个任务 j ，把同一条转移重标 $\rightarrow$ 存进任务 j 的 buffer 或共享 buffer $\rightarrow$ 对若干个任务这样做 $\rightarrow$ 更新 $\pi_\theta \rightarrow$ 再收数据。重标本身只是一个替换：

$$(s_t, a_t, s_{t+1}, r_i(s_t, a_t), z_i) \longrightarrow (s_t, a_t, s_{t+1}, r_j(s_t, a_t), z_j)$$

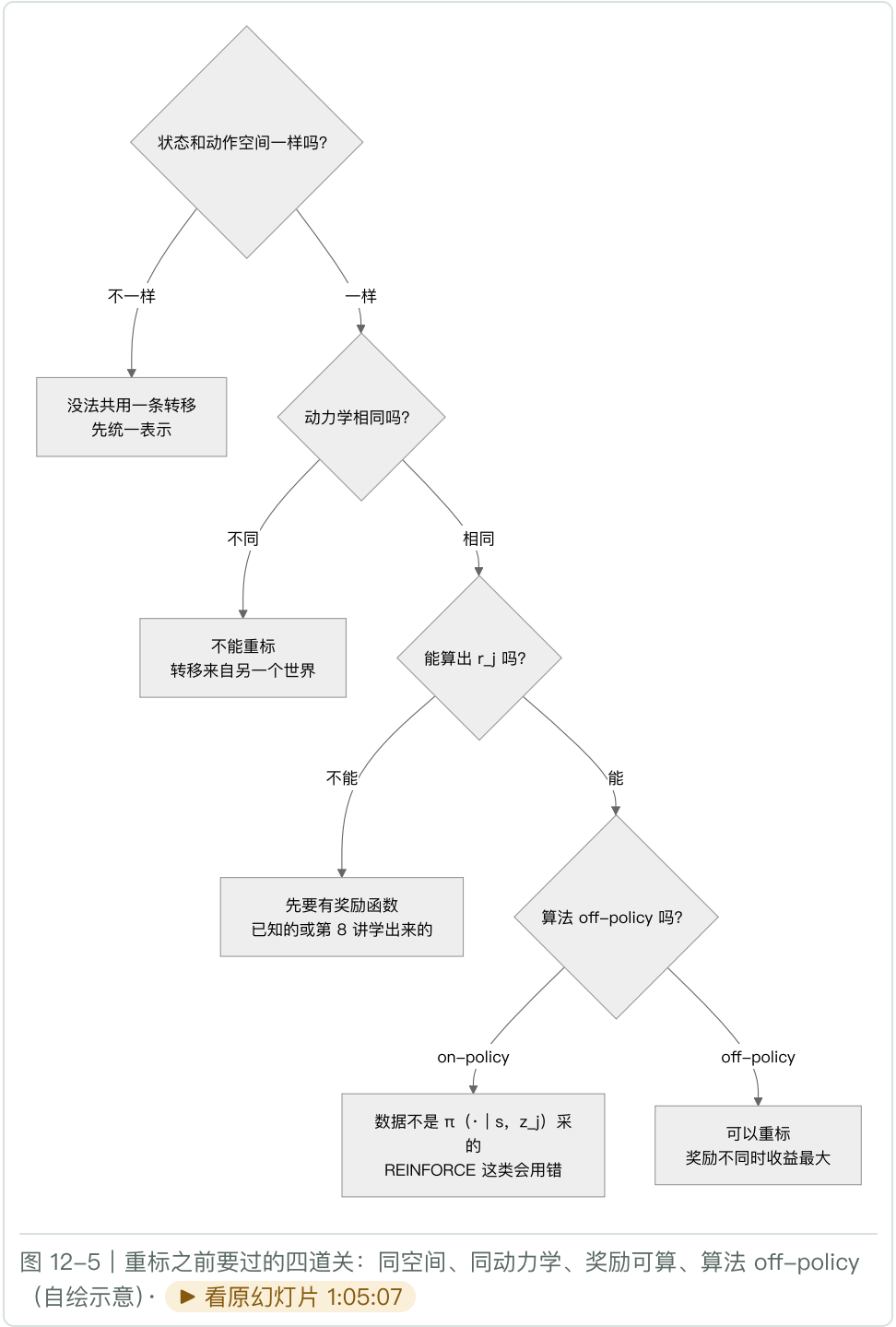
状态、动作、下一个状态原样保留，只把奖励换成用任务 j 的奖励函数重新算出来的值，把标识符换成 z_j ；这就是“事后” (hindsight) 二字的含义——回头看自己做了什么，再决定它算哪个任务的样本。

- **哪些数据值得重标、标给谁**（问答）：说实话是启发式的——学习是个“无定形”的过程，很难预先判断一段数据对某个任务有没有用。两条常用启发：一是 r_j 高的转移，说明你无意中替任务 j 做对了什么；二是干脆随机、均匀地给所有任务都重标一份。幻灯片版补了一条：可以按整条轨迹来——某处奖励高，其余部分多半也做对了什么。连续状态没有任何障碍，算法里没有一步假设状态离散。
- **重标后的数据怎么用**：把第 6 讲的 Q -learning 目标加上任务条件，就能看出为什么它吃得下这种数据（这是笔者的写法，课上只说“用 off-policy 算法更新”）：

$$y = r_j(s_t, a_t) + \gamma \max_{a'} Q_\theta(s_{t+1}, a', z_j), \quad \mathcal{L}(\theta) = (Q_\theta(s_t, a_t, z_j) - y)^2$$

$Q_{\theta}(s, a, z)$ 是带任务条件的 Q 函数（在任务 z 下、状态 s 做动作 a 之后能拿到的折扣回报）， Q_{θ^-} 是第 6 讲的 target network， γ 是折扣因子；目标 γ 只用到重标后的奖励和下一个状态，完全不关心当初这个 a_t 是在哪个任务条件下选出来的。

6. 什么时候能重标：对 MDP 的要求和对算法的要求



- 对 MDP 的要求（Finn 一条条问出来的）

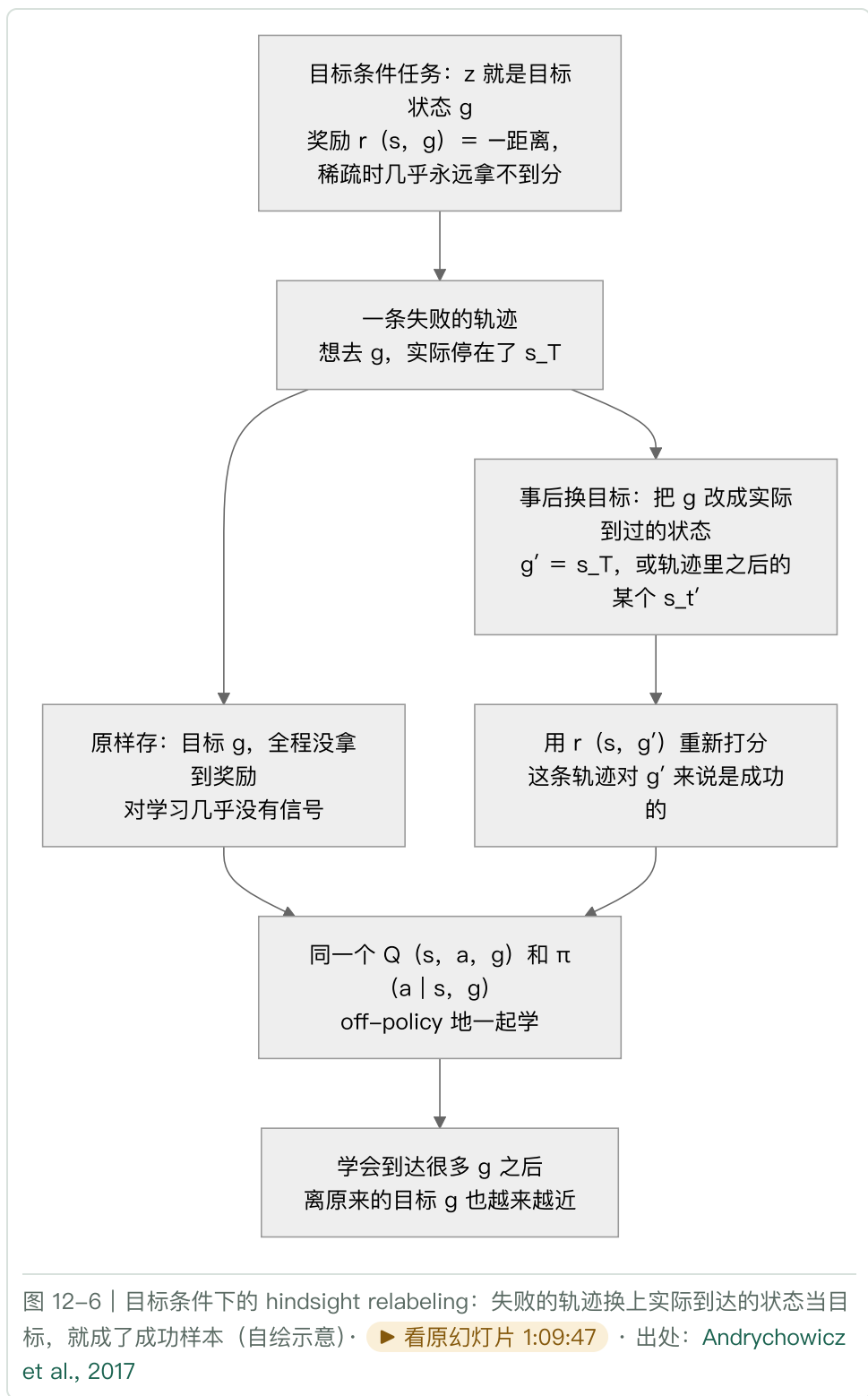
- 状态空间和动作空间要相同，尤其是共用一个 buffer 时，维度必须对得上。
- 初始状态分布 $p(s_1)$ 要相近；差太远的话，共享数据意义不大。
- 奖励尺度相近有帮助。有学生担心密奖励和稀疏奖励混在一起——这是多任务 RL 的通病，不过重标用的是目标任务自己的奖励函数，在这里问题没那么大。
- **动力学必须相同**。重标只改 r 和 z , (s, a, s') 原样保留，等于拿任务 i 的转移去教任务 j 的动力学；动力学不同，教的就是错的东西。
- 奖励函数不同时才最有用；奖励相同的话，这条数据本来就以那个奖励躺在 buffer 里了。
- **对算法的要求：必须 off-policy**。数据里的动作是 $\pi_{\theta}(\cdot | s, z_i)$ 选的，换成 z_j 条件时策略可能会选别的动作，所以重标数据对 $\pi_{\theta}(\cdot | s, z_j)$ 来说是 off-policy 的（不是当前要更新的策略自己采的）。第 3 讲的 REINFORCE 这种要求数据来自正在更新的策略的算法用不了，要用第 5、6 讲的 off-policy actor-critic 或 Q-learning。有学生追问：用重要性权重（importance weights，用概率之比修正数据来源的差异）的话，是不是要记下采集数据的策略？——Finn 先说要，随即改口：重标出来的数据应当被当成“在 z_j 条件下采到的”，不需要记录来源。

小注：严格说动作是在 z_i 条件下采出来的，真要做重要性加权，比值应是 $\pi(a | s, z_j) \div \pi(a | s, z_i)$ ，得知道原来的 z_i 。但 Q-learning 类方法（第 6 讲）本来就不需要重要性权重——回归目标对任何一条 (s, a, r, s') 都成立——这正是“必须 off-policy”这一条的用意（笔者的理解）。

- **还要能算奖励**：重标要给转移重新打分，所以任务 j 的奖励函数得是已知的，或者是第 8 讲那样学出来的。
- **三句话总结**：奖励函数可算、动力学一致、算法 off-policy。

小注：这套“按任务重标”在真实机器人上的大规模版本应为 MT-Opt (Kalashnikov et al., 2021)：十几个抓取、放置任务共用一个 Q 函数，每条回合用各任务的成功判别器重标后互相共享数据；课上没提。

7. 目标条件版本与 HER：把“没到的地方”当成目标



- **本讲最后半分钟：**目标条件版本和前面完全一样，只是奖励函数变成状态到目标的负距离——连续状态下可以是均方误差、嵌入空间里的距离，或者“在目标的 ϵ 球内算 1、否则算 0”的稀疏形式（第 3 节的公式）。视频到这里结束，“换目标”的具体做法没来得及展开，指定阅读 HER 讲的就是它。

- 为什么这是"利用结构能做得更好"的那个案例（接第 3 节的铺垫，笔者的推断）：目标条件任务把第 6 节的四道关全部自动过掉——所有目标共享同一个环境，动力学天然一致；奖励 $r(s, g)$ 是写得出来的距离函数，对任何 g 都能现算；目标状态就在状态空间里，任务空间和状态空间是同一个。于是"重标给哪个任务"有了不需要启发式的答案：标给这条轨迹实际到过的状态。稀疏奖励下随机策略几乎永远碰不到指定的 g ，原始数据里全是 0 分；换成到过的状态当目标，每条轨迹对某个 g' 都是成功的，正样本要多少有多少。

$$(s_t, a_t, s_{t+1}, g) \longrightarrow (s_t, a_t, s_{t+1}, g'), \quad g' = s_{t'} \text{ for some } t' > t, \quad r' = r(s_{t+1}, g')$$

g 是这条轨迹原本要去的目标， g' 是同一条轨迹里之后某个时刻实际到达的状态；用 g' 重新算奖励 r' ，其余不动。它和第 5 节的公式是同一个替换，只是"任务 j "换成了"到过的状态"。

小注：以下是 HER 论文（Andrychowicz et al., 2017）的内容，课上没讲到。它把这个替换接在任意 off-policy 算法上（论文用 DDPG），奖励只用"到没到"的二元稀疏信号；比较了四种取 g' 的策略——整条轨迹的终点、同一轨迹里之后的状态、同一回合里任意状态、buffer 里任意状态——其中"之后的状态"最好；在推、滑、抓放三个机械臂任务上从零学会，并迁移到了真机。论文把它称为一种隐式的课程（implicit curriculum）：先学会到近处，再逐渐够到远处。作业要实现的就是这个。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
规划路线的优缺点	简单、换目标方便 vs 测试时算力大、只适合短 horizon 或密奖励	2:07	—
合成数据的三种画法	黑色是真实轨迹、红色是模型想象的：整条、从头一小段、从每个真实状态各长一小段——取第三种	5:13	应为 MBP O
板书算法	收 $D_{\text{real}} \rightarrow$ 训模型 $\rightarrow$ 从 D_{real} 抽 s 在模型里短 rollout 得 $D_{\text{gen}} \rightarrow$ 合并更新 π_{θ}	7:48	—
幻灯片版四步算法	模型可顺带学奖励；第四步可换任意 RL 算法、可选更新价值函数	13:00	—
什么时候用 model-based RL	三大优点对三大缺点；注意"无奖励数据也能训模型"和"训练目标错位"两条	16:07	—
模型的其他形态	逆模型、多步逆模型、不带动作的未来预测、插值、转移的生成模型	24:20	—
多任务 RL 的定义	任务 = MDP，五个分量都带下标 i	32:02	—

图	看什么	跳转	出处
四个例子	推荐、角色动画、衣物、机械臂——各自哪些分量在变	33:35	—
任务标识符与合并 MDP 的板书	状态 = (原状态, z_i)，奖励和转移按 z 分派	38:14	—
多任务 IL 目标与分层 mini-batch	单个 θ 最小化各任务 loss 的平均；每个 mini-batch 各任务占 $1/n$	45:27	—
语言 + 视频双编码器架构	两种任务描述各走各的编码器，嵌入乘进主干各层，BC loss 端到端	49:08	应为 BC-Z
VLA 式 transformer 策略	预训练图像编码器投到语言模型输入空间，任务串当 token，输出动作	51:40	应为 RT-2
冰球重标例子	想射门却传出好球：原样存一份，再按传球任务的奖励重标一份	59:24	—
hindsight relabeling 幻灯片 算法与适用条件	收数据 → 存 buffer → 重标 → 更新；挑任务：随机或高奖励，整条轨迹也行；条件：同动力学、有奖励函数、off-policy	1:04:35	HER

提到的工作

名称	在本讲里的作用
第 11 讲的 model-based RL 与规划	开场回顾：学模型 → 规划这条路的优缺点
Dyna (Sutton, 1990)、MBPO (Janner et al., 2019)	合成数据算法的来源；课上没点名，应为
DAgger (第 2 讲)	学生用它类比"前半段真实、后半段模型"
灵巧手转球 (应为 PDDM, Nagabandi et al., 2019)	model-based RL 在紧凑状态上成功的例子
视频生成模型	图像状态下学模型的未来希望
逆模型、多步逆模型、affordance、插值、转移的生成模型	模型的其他形态，只点了名
one-hot、语言描述、演示视频	任务标识符 z 的三种形式
上下文学习、元 RL (第 13 讲)	把任务的示例放进 z 的延伸
BC-Z (Jang et al., 2021；应为)	语言 + 视频双任务描述的多任务 IL 架构
SFT (CME295 第 4 讲)	LLM 的 SFT 就是拿 prompt 当任务标识符的多任务 IL
RT-2 (Brohan et al., 2023；应为)	VLA 式的 transformer 策略， z 就是 prompt
毛巾任务与十几条指令的演示视频	单个扁平策略做多任务、任务抽象层级不一的例子；课上没点名
分层 RL (第 15 讲, SayCan)	"收拾厨房"这类更高层的任务留到那里

名称	在本讲里的作用
Hindsight relabeling、HER (Andrychowicz et al., 2017)	本讲主角；指定阅读；作业要实现
PCGrad (Yu et al., 2020；课上没提， 小注补充)	多任务里梯度冲突的一种处理
MT-Opt (Kalashnikov et al., 2021；课 上没提，小注补充)	真实机器人上大规模按任务重标、共享数据

术语对照

English	中文
model-based RL	基于模型的 RL：先学环境的转移模型，再拿它规划或造数据
dynamics model $p_\phi(s' s, a)$	动力学模型 / 转移模型：预测下一个状态
planning	规划：不学策略，在每个状态现场优化未来几步的动作
horizon	时间跨度：往前看几步
rollout	按策略从某个状态出发走出来的一段轨迹
synthetic / imagined data	合成数据：在学到的模型里走出来的转移，区别于真实环境的数据
compounding error	误差累积：模型被反复套用，预测越走越偏
shaped / dense reward vs sparse reward	密奖励 / 稀疏奖励：每步都有反馈 vs 只有到达时才有
inverse model	逆模型：给两个状态，反推中间的动作
affordance	环境里"可能发生什么"，不依赖自己的动作
multi-task RL	多任务 RL：一个策略学多个任务
generalist policy	通才策略
task identifier z_i	任务标识符：告诉策略现在做哪个任务的输入
one-hot vector	独热向量：第 k 位是 1、其余全 0，用来编码"第 k 个任务"
task conditioning	任务条件化：策略、Q、V 都额外吃 z
goal-conditioned RL	目标条件 RL：任务标识符就是要到达的目标状态
aggregated MDP	合并的 MDP：把 z 并进状态之后覆盖所有任务的那一个 MDP
stratified mini-batch	分层抽样的 mini-batch：每个任务在批里占固定比例
per-task normalization	逐任务归一化：各任务的状态和动作缩放到相近尺度
behavior cloning (BC)	行为克隆：最大化示范动作的对数概率（第 2 讲）

English	中文
prompt as task descriptor	把 prompt 当任务描述：LLM 的 SFT 就是多任务 IL
VLA (vision-language-action)	视觉-语言-动作模型：图像和任务串进语言模型，输出动作
weight sharing	权重共享：多个任务用同一套参数
hierarchy / long-horizon task	层级化 / 长时程任务：要走很多步才完成的任务，第 15 讲
replay buffer	重放缓冲区：存历史转移、反复抽样的仓库（第 5 讲）
data sharing	数据共享：一个任务收的数据拿去更新另一个任务
relabeling / hindsight relabeling	重标 / 事后重标：回头把一段经验按另一个任务的奖励重新打分
off-policy	用不是当前策略采的数据来更新（第 5 讲）
importance weights	重要性权重：用概率之比修正数据来源不同
ϵ -ball	ϵ 球：离目标不超过 ϵ 的范围，稀疏奖励的"算到达"判据
HER (hindsight experience replay)	事后经验重放：把实际到过的状态当目标重标
implicit curriculum	隐式课程：先学会近的目标，再自然够到远的

字幕勘误

"reabeling""reabel""reabelled" → relabeling; "MTP""MVP""mark decision process""marov" → MDP、Markov decision process; "uklidian" → Euclidean; "shaped wards" → shaped rewards; "denser words""sparse words" → dense rewards、sparse rewards; "Dagger" → DAgger; "recommener" → recommender; "invitation learning" → imitation learning; "D genen" → D_gen; "repo buffer" → replay buffer; "Pytha""PI theta" → π_θ ; "ZI""Zj" → z_i 、 z_j ; "ST A T S + one" → s_t 、 a_t 、 s_{t+1} ; "heristic" → heuristic; "highle" → high-level; "onpolicy" → on-policy; "directories" → trajectories; 讲架构时的 "and pen" → end-to-end; "four two states" → for two states。

带走的问题

1. 合成数据的 rollout 长度是个超参：长了模型误差累积，短了覆盖不够。如果模型完全准确（比如就是游戏引擎本身），第 1 节的算法退化成什么？如果模型完全不准呢？"从真实状态出发做短分支"和第 2 讲 DAgger 的"从策略走到的状态出发请专家标注"，结构上像在哪里、差在哪里？
2. 把任务标识符并进状态之后，理论上第 3—6 讲的标准 RL 就能解，那 relabeling 到底多给了算法什么信息？（提示：奖励函数是已知的、动力学在任务间共享——这两条"结构"标准 RL 并不知道。）
3. 冰球例子里，重标数据对 $\pi_{\theta}(\cdot | s, z_j)$ 是 off-policy 的。如果你用的是第 3 讲的 REINFORCE，会出什么问题？喂给第 6 讲的 Q-learning 为什么就没事？如果非要做重要性加权，分母该是谁？
4. 在你做的 agent 里，一次失败的工具调用轨迹能不能被"事后重标"成另一个任务的成功样本？逐条对照第 6 节的四道关：LLM agent 的环境在任务之间共享吗？另一个任务的奖励能不能对这条轨迹现算？你的训练方法吃得下 off-policy 数据吗？
5. Finn 说通才系统经验上比单任务系统更可靠，可第 4 节又说把很多任务塞进一个模型会有优化上的麻烦。什么情况下你会选择拆开训、再想办法合并？如果任务之间的梯度方向真的相反，分层抽样的 mini-batch 帮得上忙吗？

CS224R 第 13 讲 | 元强化学习 (Meta RL)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 13 讲, 2025 年 5 月 14 日 · 承接第 12 讲的多任务 RL, 为第 14 讲的探索铺路 视频: <https://www.youtube.com/watch?v=wSiyEpvoGkA> (1:09:10, 英文字幕是自动生成的, 算法名和记号错得不少, 见文末勘误) 讲者: Chelsea Finn (本讲不是客座讲座) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: RL²: Fast Reinforcement Learning via Slow Reinforcement Learning (Duan et al., 2016) · 课上讲到但没点名的: PEARL (Rakelly et al., 2019)

一句话: 普通 RL 每个任务都从零学, 人却能靠旧经验几分钟学会一台新咖啡机。元强化学习 (meta-RL) 把"快速适应"本身当成训练目标: 在一个任务分布上训练, 让策略拿到新任务里一点点经验 (几步或几个 episode) 就能上手。最成功的一类是黑箱方法: 把策略做成有记忆的序列模型, 状态、动作、奖励一起喂进去, 记忆跨越多个 episode, 端到端最大化所有任务的总回报, 探索和利用的权衡由模型自己学出来 (RL²、SNAIL)。换个角度看, 这是在用经验推断一个隐藏的任务变量 z , 整个问题是一个"只有任务看不见"的 POMDP——PEARL 显式维护 z 的后验, 还借此换成 off-policy 算法提速。代价是难优化: 探索和利用互相依赖, 端到端信号只在两件事同时做对时才出现; 后验采样这类辅助探索目标是缓解办法, 第 14 讲继续。

时间轴

时间	内容
0:06	回顾第 12 讲: 多任务 RL 的聚合 MDP; 目标条件 RL 与它的自监督好处
2:42	今天的主题: meta-RL 的问题定义与黑箱方法; 为什么要它——咖啡机例子, 人不是从零开始
5:48	迁移的三种设定: 单任务 fine-tune、多任务 zero-shot、元学习 few-shot; 任务描述符可以是一份数据集; RL 之外的 few-shot: 图像分类、GPT-3 的 in-context learning
12:27	meta-RL 的例子: 走迷宫——第一个 episode 探索, 第二个直奔目标; 探索是 RL 特有的难点; 腿式机器人、用户偏好等场景
17:04	正式定义: 多任务 vs 迁移 vs 元学习; 任务就是 MDP; 适应数据的两种形式; 探索策略与任务策略
20:20	黑箱方法: 把策略做成有记忆的序列模型; 和普通循环策略的几点差别 (奖励当输入、跨 MDP 训练、记忆跨 episode)

时间	内容
24:41	算法四步：采任务 → 连跑 n 个 episode → 存 buffer → 最大化全部任务的回报；测试时最多跑 n 个 episode；问答：episode 边界、初始状态
31:23	架构：RNN (RL ²)、自注意力加 1D 卷积 (SNAIL)、deep set；视觉迷宫实验：1000 个小迷宫训练，大迷宫上也能用
36:29	问答：为什么会学到"尽快找到"——折扣因子；输入是什么；探索和利用藏在目标函数里；能不能普通 RL 加 test-time training
39:31	test-time compute 就是 meta-RL：解数学题当成 MDP，解法是要探索的东西；DeepSeek-R1 对 2 + 2 也长想；把 test-time compute 的效率纳入训练
42:05	三种视角：经验就是任务描述符；meta-RL 推广了目标条件 RL；任务推断视角；问答：为什么不直接给任务编号
46:08	meta-RL 没有聚合 MDP：任务不可观测，它是一个特殊的 POMDP
48:40	显式推断任务分布 z 的方法 (PEARL)；底层用 off-policy 算法让 meta-training 快得多；问答：replay buffer 里放整段轨迹
52:46	小结：黑箱方法通用、表达力强，但难优化；样本效率继承自底层优化器（策略梯度 / PPO / SAC）
53:46	in-context 探索为什么难：走廊加纸条的例子，三条轨迹各给什么信号；做饭的比方；问答：是稀疏奖励的锅吗
1:02:30	解法一：后验采样 (Thompson sampling) 的做法、点机器人例子、什么时候会很糟；内在奖励等其他解法，周五第 14 讲继续

核心内容

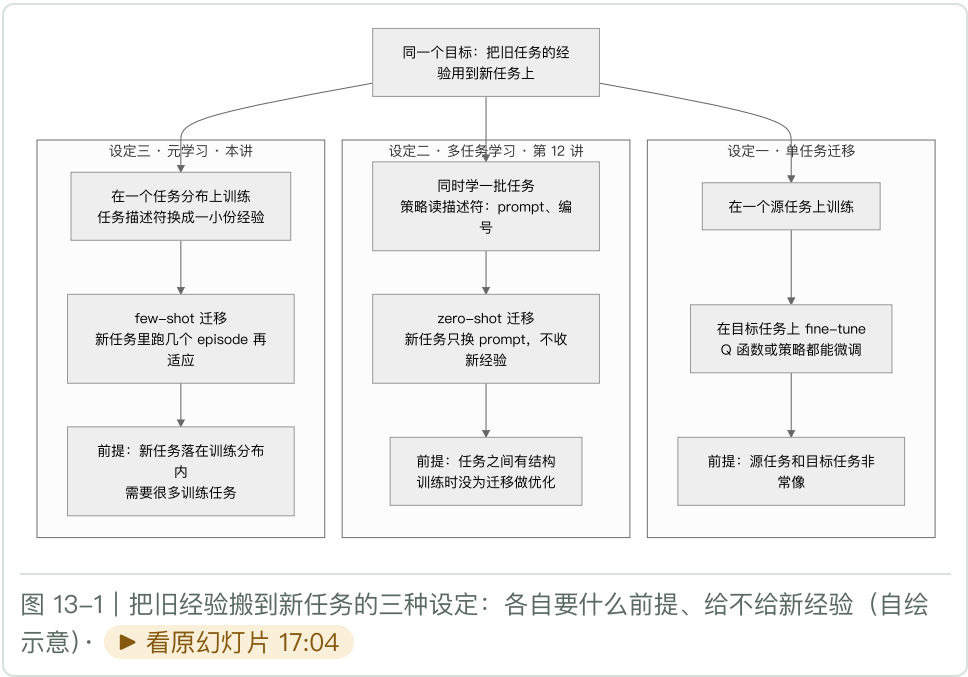
1. 为什么要 meta-RL：人不是从零开始学的

- 回顾第 12 讲：多任务 RL 把一批任务塞进一个聚合 MDP——状态旁边多带一个任务描述符 (prompt、目标，或者干脆一个编号)，采一个任务、在它的 MDP 里跑策略、再采下一个。目标条件 RL 是它的特例：任务是"到达某个目标状态"，描述符就是那个状态；好处是可以自监督，随便一个状态都能当任务，代价是不好训（作业里会做一个简单版本）。
- 今天的差别：多任务 RL 训完直接用；meta-RL 关心的是到了新任务，只给一点点经验，能不能很快学会。
- 咖啡机的例子：让 RL agent 学会操作一台没见过的意式咖啡机要试非常多次——步骤多、要按顺序做、奖励还不细；人只要有一点说明或用过别的机器，几分钟就会。差距在于人不是从零开始：会用手挪东西、用过别的机器、听得懂"拧一下旋钮"。机器人换环境、换任务是

一例；解一道没见过的数学题也是——做过别的题，就有策略可以套。

- 于是问题变成：RL 算法能不能利用以前任务的经验，让新任务用更少的数据学会？这常被看成迁移学习（transfer learning）问题。

2. 三种迁移设定：fine-tune、多任务 zero-shot、元学习 few-shot



- 设定一：单任务迁移。**先在一个任务上训练，再在新任务上 fine-tune——监督学习的微调概念在 RL 里同样可用，微调 Q 函数或策略都行。前提是两个任务非常像。
- 设定二：多任务学习（第 12 讲）。**任务之间有结构时——"拿起水瓶""拿起手机"，而不是"任务一、任务二"——新任务只要换一个 prompt 就可能零样本（zero-shot）迁移，一条新经验都不用收。但训练时并没有为迁移做优化。
- 设定三：元学习（meta-learning）。**关心少样本（few-shot）迁移：几个例子、几次尝试就能适应。和多任务学习的本质区别是，训练时就把"将来要用一小份数据去适应"考虑进去了；换句话说，任务描述符不再是 prompt 或编号，而是一份数据集本身。这就是 in-context learning 的意思。
- 前提对比：**设定一要求源任务和目标任务极像；设定二、三只要求新任务落在训练任务的分布之内，代价是需要很多个训练任务。

- **问答：和"先训练再 fine-tune"到底差在哪？**——训练过程在显式优化"用小数据集适应"的能力，所以迁移更快、需要的数据少得多；手段就是给算法一堆训练任务，而不是一个。

3. RL 之外的 few-shot：图像分类与 LLM 的 in-context learning

- **few-shot 图像分类：**给六张带标签的图，判断一张新图属于哪类。只用六张图从零训分类器毫无希望。元学习的做法是构造许多结构相同的其他分类任务，训练一个"读一个小训练集加一张查询图，输出标签"的模型——**每个 few-shot 问题本身成了一条训练样本。**
- **LLM 的 in-context learning：**把几对"蹩脚英文 → 正确英文"的例子放进 prompt，再给一个新句子让它改。例子就是训练集，新句子就是查询。语言模型并没有专门按这个格式训练过，互联网文本的统计规律自己带出了这种能力；讲者放的是一个 base 模型的输出，不加任何监督训练就能做到，虽然不总是对。

小注：讲者说这应该是 GPT-3；改错英文的例子应出自 GPT-3 论文 (Brown et al., 2020) 的少样本演示。CME295 的 LLM 基础部分讲过 in-context learning 的用法，这里是把它归到"元学习"这个大类里看。

- **共同形状：**模型读训练集 D 和新样本 x ，输出对 x 的预测——预测同时依赖 x 和 D 。
- **问答：LLM 靠 prompting 把数据集塞进上下文，meta-RL 也是这样吗？**——今天讲的这一类，是的。也有靠梯度更新或类似最近邻的办法，但 meta-RL 里最成功的方法长得就像 prompting，本讲只讲它。

小注：靠梯度更新适应的那一族应指 MAML (Finn et al., 2017) ——元学习一个初始化，新任务上走几步梯度；本讲没有展开，2025 春的选读里也只列了 RL² 和 PEARL。

4. meta-RL 的问题定义：任务是 MDP，探索是新增的难点

- **迷宫例子：**agent 要走一个新迷宫，只有第一人称视角。RL 和监督学习不同：不只是输出标签，还要自己去环境里收集经验。目标是只用一个（或几个）episode——第一个 episode 到处探索找目标，之后直奔目标。从零训练做不到，meta-RL 的办法是构造一批相关的训练迷宫，在 meta-training 阶段学会"怎样高效地探索并解决"这一类 MDP。

。术语：一个 **episode** 是从初始状态出发到终止的一段完整交互；这段交互记录下来（状态、动作、奖励的序列）叫一条**轨迹** τ 。

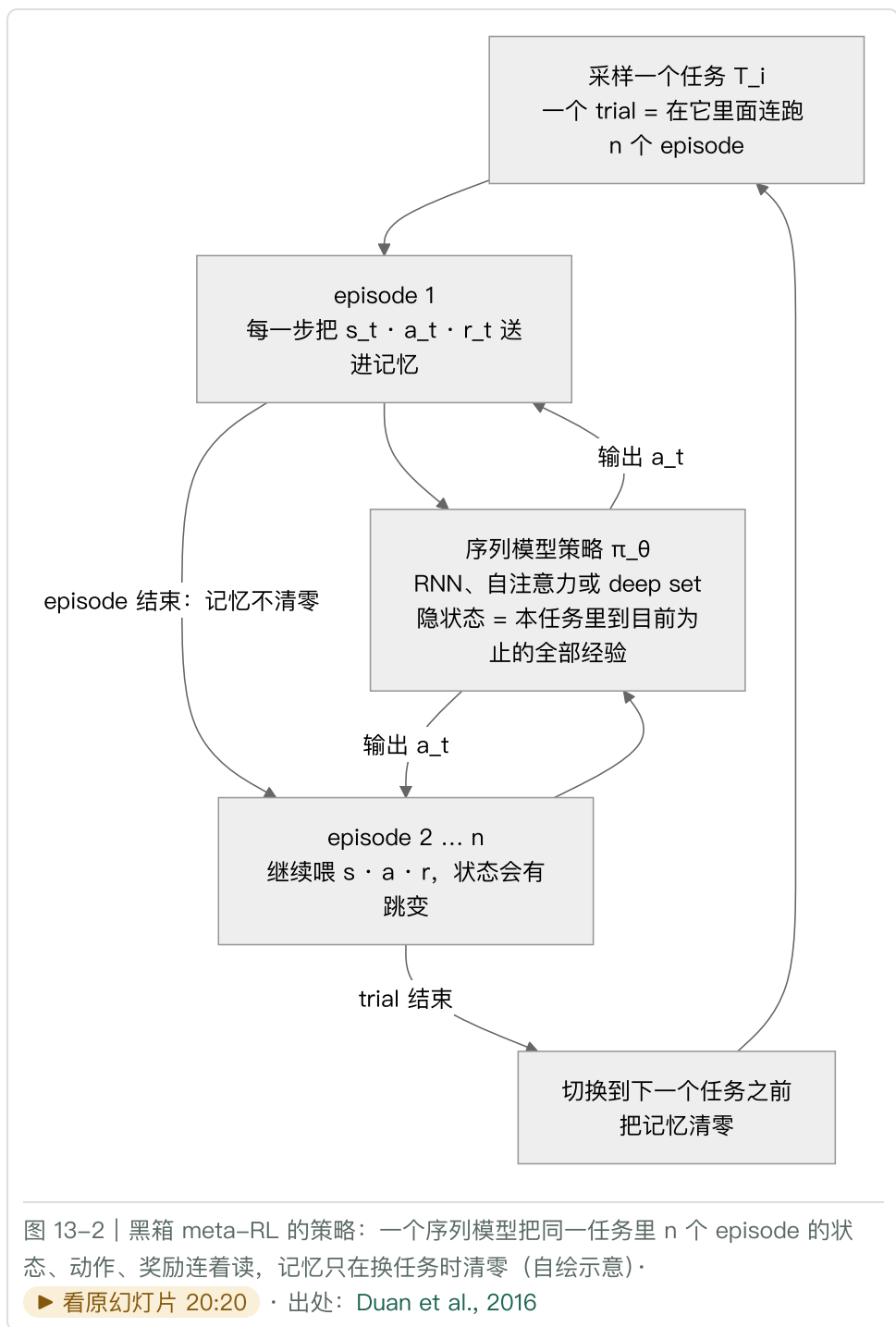
- **新增的难点：探索**。监督学习的 few-shot 拿着现成的训练集做预测；这里训练集要靠探索自己造，还得权衡花多少时间探索、多少时间利用（exploit）已知信息拿分。
- **关键假设**：要有一个**任务分布** $p(T)$ ，新任务落在这个分布里。问答：必须和某个训练任务一样吗？——可以不一样，后面有例子。
- **其他场景**：腿式机器人适应不同地形（仿真里训多种环境，真实世界当成又一个环境）；操作不同物体；语言模型按用户偏好适配——把该用户想要的回答样例放进上下文；还有推理（第 8 节）。
- **正式表述**：多任务学习同时解多个任务，没为迁移优化；迁移学习先解源任务再解目标任务；元学习有一批训练任务的经验，要很快解一个新任务。RL 里每个任务是一个 **MDP**（马尔可夫决策过程，第 1 讲：状态空间、动作空间、动力学、奖励函数），各任务这四样都可以不同。
- **记号**：要学的是一个“读了少量经验之后能解任务的策略”，并且这些经验要自己收集。经验有两种形式，中间是连续谱：

$$\pi_{\theta}(a_t | s_t, D_i), \quad D_i = \{\tau_1, \dots, \tau_k\} \text{ or } \{(s_1, a_1, r_1), \dots, (s_K, a_K, r_K)\}, \quad D_i \sim \pi_{\theta}^{\text{exp}} \text{ in } T_i$$

D_i 是在任务 T_i 里由探索策略 π^{exp} 收集的经验：要么是几条完整轨迹，要么只是几个时间步——前者关心多次尝试后的表现，后者关心一个 episode 内的在线适应。“探索策略”和“任务策略”分开写只是为了讲清楚，实践中可以是同一个网络。

- 顺着这个记号看，策略条件在一份训练数据上，这份数据就像一个任务标识——从这个角度看它又回到了多任务学习的形状（第 9 节会正式讨论）。

5. 黑箱方法：把策略做成有记忆的序列模型



- **好消息**是有一个概念上极简单的办法：训练一个**序列模型**——Transformer、RNN，任何带记忆的网络——把 agent 在当前 MDP 里收集到的全部经验都喂给它，让它根据"到目前为止看到的数据"输出动作，再在所有训练 MDP 上训练这个模型。**坏消息**是它在实践中很难优化，后面几节都在讲这一点。

$$a_t \sim \pi_\theta(a_t | s_t, h_t), \quad h_t = f_\theta(s_{1:t-1}, a_{1:t-1}, r_{1:t-1})$$

h_t 是记忆（RNN 的隐状态或 Transformer 的上下文），由本任务里此前所有的状态、动作、奖励算出来，跨越已经跑完的 episode；策略读当前

状态 s_t 和记忆 h_t 决定动作。

- 和"普通 RL 里用一个循环策略"有什么不同（讲者让全班找差别，凑出五条）：
 1. 普通 RL 常常根本不用记忆；这里记忆是核心。
 2. 奖励被当作输入送进记忆：在用当前 MDP 的奖励信号来变好，因此测试时同样要拿得到奖励——普通 RL 部署时通常不假设有奖励。
 3. 动作也可以当输入：策略接近确定性时过去的动作已隐含在记忆里；随机性强时采样结果不在记忆里，最好喂进去。
 4. 模型是跨 MDP 训练的：不同任务的状态空间、动作空间、动力学、奖励都可能不同。
 5. 记忆跨越多个 episode：要在 episode 之间学习，隐状态就不能在 episode 结束时清零，而要一直保留到换任务。
- 6. 算法：内层 in-context 适应，外层 meta-training

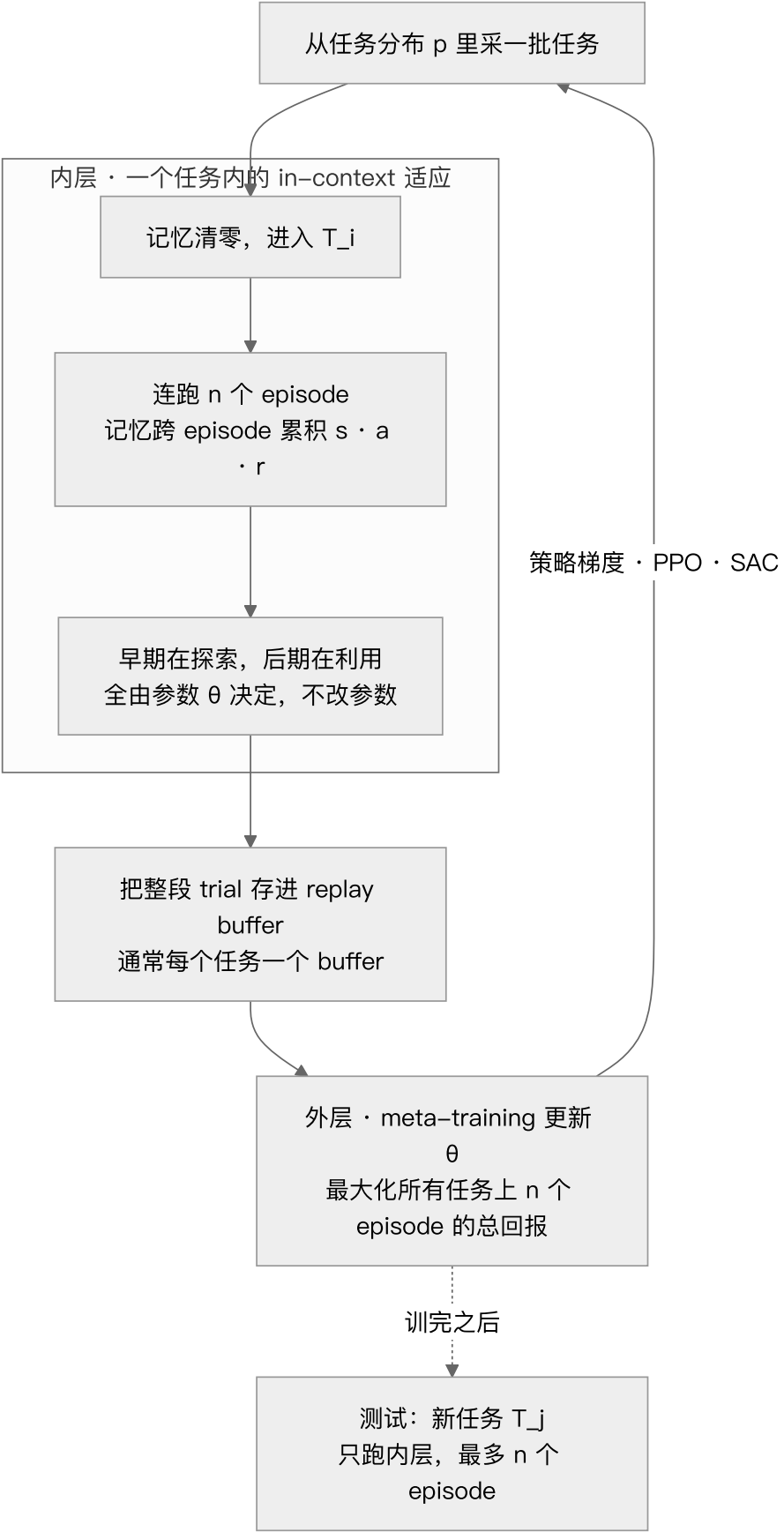


图 13-3 | 两个循环：内层靠记忆在一个任务里适应，参数不动；外层跨任务更新参数，让内层适应得更好（自绘示意）· [▶ 看原幻灯片 27:45](#) · 出处：Duan et al., 2016

- **四步（板书）：**

1. 采一个任务（MDP） T_i 。
2. 在 T_i 里连跑 n 个 episode，记忆跨这 n 个 episode 累积。这 n 个 episode 合起来叫一个 trial。
3. 可选：若底层是带 replay buffer 的 off-policy 算法，把这些 episode 存进 buffer（**replay buffer**：存放过往经验供反复训练的仓库，第 5 讲；这里通常每个任务一个 buffer）。
4. 更新策略，最大化**所有任务**上的回报（**回报** return：一段交互里奖励的总和）——就是普通 RL 目标再对任务求和。然后回到第 1 步。

$$\max_{\theta} \sum_i \mathbb{E}_{T_i: m \sim \pi_{\theta} \text{ in } T_i} \left[\sum_{k=1}^n \sum_t \gamma^t r_t^{(k)} \right]$$

θ 是序列模型策略的参数；外面对一个 mini-batch 里的任务 i 求和，里面是策略在任务 T_i 里连跑 n 个 episode 的期望回报， $r^{(k)}_t$ 是第 k 个 episode 第 t 步的奖励， γ 是折扣因子（问答里确认实际用的是折扣回报）。与普通 RL 的差别只有两处：跨多个 MDP 优化，以及策略的记忆跨一组 episode。

- **n 通常很小**： n 若是一千，整段经验塞不进上下文。切换任务时截断记忆。讲者的图里 $n = 2$ 。
- **训出来是什么**：一个能在 episode 之间靠经验适应的策略，而且它自己决定探索和利用怎么权衡： $n = 2$ 就得在两个 episode 内尽快拿分； n 大一些探索预算多一点，但只要早探索能换来更高回报，它仍会尽快探索。依据只有一条：哪样总回报更高。
- **测试时**：来一个新 MDP T_j ，喂它状态、动作、奖励，**最多跑 n 个 episode**——跑得更长会做什么没有保证；想处理可变长度，训练时就用可变的 episode 数。
- **问答**
 - 模型怎么知道新的 episode 开始了？——可以加位置编码或一个专门的 token；实际上换 episode 时状态通常有明显跳变，模型自己也看得出来。

- 每个 episode 的初始状态 s_1 会变吗？——会，每个 episode 重新从初始状态分布采样。

7. 架构、优化器与迷宫实验

• 架构三选：

- 普通 RNN：最早的 meta-RL 工作就是这么做的。那时 PPO 还没出现，用的是一个假设上和 PPO 类似的算法。

小注：即指定阅读 RL² (Duan et al., 2016) ——GRU 策略加 TRPO (PPO 的前身)；同期的 Learning to Reinforcement Learn (Wang et al., 2016) 思路相同，“最早的工作”应指这两篇。

- 自注意力加 1D 卷积：Transformer 流行之前的一篇工作，可以当成一种 Transformer；注意它把动作也喂进了模型。

小注：应为 SNAIL (Mishra et al., 2017)：temporal convolution 负责聚合过去的经验，soft attention 负责精确取出某一条信息。

- 前馈网络加平均：对每条经验各算一个 embedding 再取平均，叫 **deep set**。能完全并行所以算得快，表达力也出人意料地强，后面会再提到。

小注：deep set 一词出自 Zaheer et al., 2017；后面第 10 节那个方法的任务编码器就是这种“逐条编码再聚合”的形状（推断）。

- 视觉迷宫实验（类 Transformer 架构）：在 1000 个小迷宫上训练，在没见过的小迷宫和**大迷宫**上测试——大迷宫在训练分布之外，是一个能外推的例子。agent 只看图像：第一个 episode 到处走找目标，第二个 episode 直接走向目标；大迷宫里它探索了上半区、判断是死胡同就不再深入。训练时 $n = 2$ 。
- 数字：指标是第一、第二个 episode 里找到目标的平均用时，对比随机策略和 LSTM。小迷宫上第一个 episode 就明显好于随机（探索是高效的），第二个 episode 更快；大迷宫上也显著好于随机；两种迷宫上自注意力架构都强于 LSTM。

小注：这张表应出自 SNAIL 论文的视觉导航实验，其中 LSTM 基线就是 RL² 的架构；具体数字课上没有念。

• 问答

- 奖励只是“找到目标”，为什么会学成“尽快找到”？——用的是折扣回报： γ 大约 0.95 或 0.99 而不是 1，越早拿到奖励目标值越高。

- 输入到底是什么？——之前 episode 的全部视频帧、动作和奖励，加上本 episode 到当前为止的一切。
- 探索和利用在算法哪一步体现？——哪一步都不是，全在目标函数里：agent 不知道自己在哪个迷宫，不探索就拿不到高分，端到端优化回报自然逼出探索。
- 策略很差、拿不到奖励时是不是很难学？——是，这是第 14 讲的主题。
- 能不能普通 RL 训完再做 test-time training？——带记忆的语言模型在某种意义上已经在做 meta-RL 了；test-time compute 则是让它把问题想一遍，下一节讲。

8. 岔路：test-time compute 就是 meta-RL

- **解数学题当成一个 MDP**：证明一个不等式（讲者说可能正是 Aviral 在第 10 讲用的那道），可以试的策略很多——猜测检验、消项……你事先不知道哪条路通。理想情况是把不同策略都探一探，再基于最好的那条给出答案。
- **DeepSeek-R1 的现象**：给它难题，它想很久，到某个点自己截断再输出解释；给它" $2 + 2 = ?$ "，它也认真推一遍再分步解释——明显浪费。
- **用 meta-RL 的眼光**：既然知道测试时要用 test-time compute，就把它纳入训练，鼓励模型**高效地使用**：简单题尽快给答案，难题想它需要的那么久（包括探索不同策略），然后转向利用、给出解释。讲者引的一篇论文显式优化这一点，结果是同样 token 数下性能更高，同样性能下 token 更少。

小注：应为 Qu et al., 2025 (MRT, Aviral Kumar 组)：把 test-time compute 形式化为 meta-RL，用"每写完一段，最终做对的概率涨了多少"作为稠密的进度奖励；摘要称在数学推理上相对 outcome-reward RL 有 2—3 倍的相对提升、约 1.5 倍的 token 效率。CS329A 讲 test-time compute 时是从"多采样加验证器"入手的，这里是同一件事的 RL 视角。

meta-RL 的词	在 LLM 推理里对应什么
任务 T_i	一道题
一个 trial 里的 n 个 episode	一条长思考链里的若干次"尝试"
探索	试不同解法、检验、回头
利用	写下最终答案和解释
记忆 h_t	上下文里已经写出的 token

meta-RL 的词	在 LLM 推理里对应什么
奖励	最终对错（外加"进度"之类的辅助信号）
探索和利用的权衡	想多久才停

9. 三种视角：经验即任务描述符 → 任务推断 → POMDP

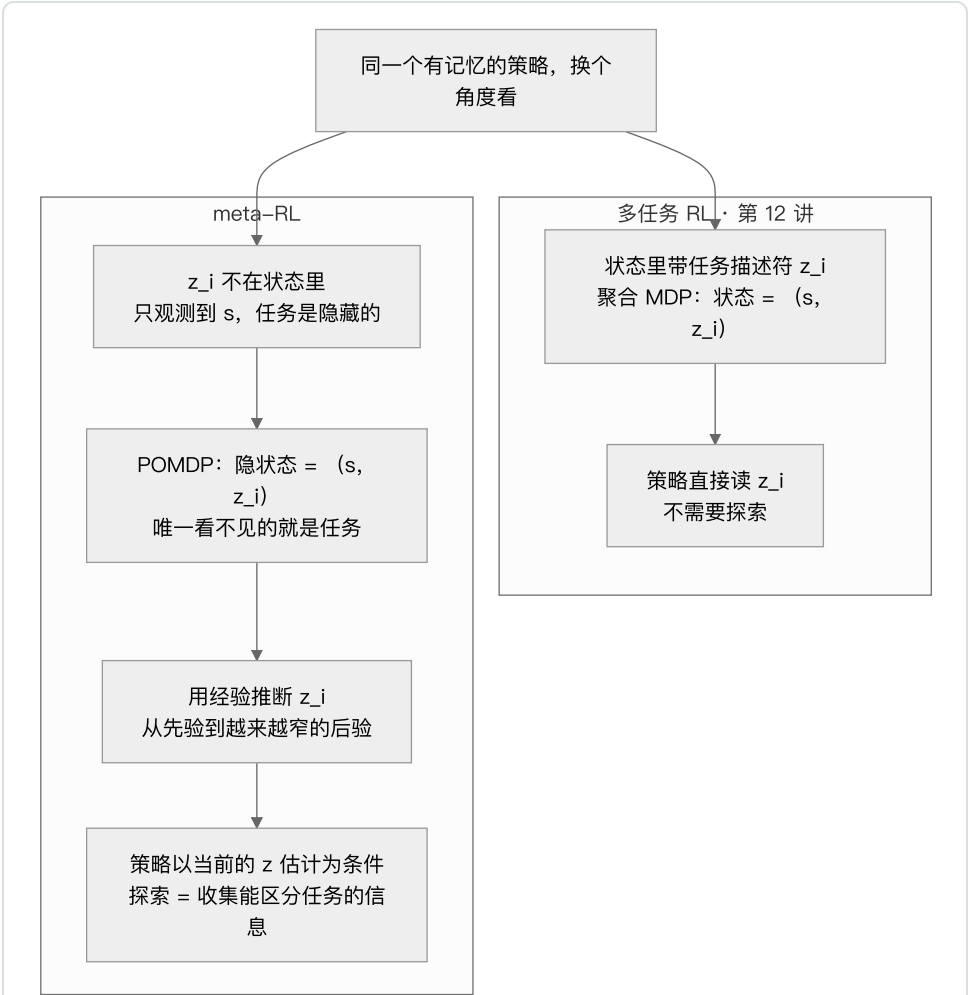


图 13-4 | 任务推断视角：多任务 RL 里任务写在状态里，meta-RL 里任务是隐状态，只能从经验里推断（自绘示意）· [▶ 看原幻灯片 44:07](#)

- **视角一：经验就是任务描述符。**多任务策略读一个描述符（"在哪里堆叠""往哪个方向走"），meta-RL 策略读一段经验。把经验看成任务标识，它就是多任务学习——只是这个"标识"在任务内部会变，类比不完美。
- **视角二：meta-RL 推广了目标条件 RL。**目标到达型 MDP 是奖励最大化型 MDP 的子集；目标条件 RL 做的是对新目标的 **zero-shot** 泛化，meta-RL 做的是靠 K 个 episode 或 K 步经验的 **K-shot** 适应。
- **视角三：任务推断。**图 13-2 里流进策略的那份信息，本质上是关于任务的信息。任务描述符 z_i 不在状态里，你不知道自己在哪个

MDP，经验就是用来辨认它的。于是 meta-RL 可以读作两步：探索以推断未知的任务，然后按当前对任务的估计去执行。

- **问答：既然要推断任务，为什么不直接把任务告诉它？**——因为经常没有紧凑的表示来描述任务。只能用编号的话，测试时来个"任务 100"它什么也做不了；条件在经验上却可能泛化，因为状态、奖励是连续的量。归根结底是表示问题：要学一个连续向量 z 来表示任务。
- **没有聚合 MDP，只有 POMDP。**第 12 讲里，把 z 拼进状态得到一个新的 MDP。meta-RL 里 z_i 不给，问题天生是部分可观测的，拼不出聚合 MDP；硬要写成单个决策过程，就是一个 POMDP（部分可观测 MDP：看不到真实状态 s ，只看到 s 的某个函数 o ）。而且是一种特殊的 POMDP——**唯一看不见的就是任务。**

multi-task RL: $\tilde{s}_t = (s_t, z_i)$ observed

meta-RL: $\tilde{s}_t = (s_t, z_i)$, $o_t = s_t$, z_i hidden

两边的完整状态都是 (s_t, z_i) ；多任务 RL 里 z_i 摆在状态里，策略直接读；meta-RL 里观测只有 s_t ， z_i 是隐藏的。因此"从经验推断任务"就等于"推断 POMDP 的隐状态"，推断出来问题就还原成一个 MDP。

小注：文献里把这种"动力学和奖励未知、其余都可观测"的特殊 POMDP 叫 Bayes-adaptive MDP (BAMDP)；在它上面做最优决策就是最优的探索——第 6 节"端到端原则上能学到最优权衡"的说法，理论上落在这里。

10. 显式推断任务分布，并用 off-policy 算法提速

- **一个显式输出 z 的方法**（前面架构图里出现过）：输出的是 z 的一个分布——一开始只有先验，经验越多估计越准；维护分布而不是点估计，后面能做很有用的事（第 12 节）。它做的是连续控制：往不同方向走、以不同速度跑，以及摩擦系数、惯量等物理参数不同的任务。

小注：应为 PEARL (Rakelly et al., 2019, Finn 是作者之一)：编码器把每条经验编码后聚合成 z 的高斯后验，策略和 Q 函数以 z 为条件，底层是 SAC；摘要称 meta-training 的样本效率比此前方法高 20–100 倍。

- **训练效率来自底层算法。**此前的方法都用 PPO 那一类基本 on-policy 的算法，没有 replay buffer（术语：**on-policy** 只用当前策略刚采的数据，**off-policy** 能反复用旧数据，第 5 讲）。适应很快，但底层若是 on-policy，meta-training 本身会非常慢。课堂小测：第 4 步换成 off-policy 算法更高效还是更低效？——更高效：用 buffer 里的数据更新这个带记忆的策略，meta-training 省数据得多；测试时的适应效

率则差不多。曲线上，带 replay buffer 的方法（蓝线）比 RL^2 等 on-policy 方法学得快得多。

小注：PEARL 论文里对比的三条基线是 ProMP、MAML-TRPO 和 RL^2 ，都用 on-policy 的底层算法；字幕里的 "PPO" 应为其中之一（推断是 ProMP）。

- **问答：replay buffer 里到底存什么？**——每个任务一个 buffer，每一项是该 MDP 里的一段 **episode 序列**（状态、动作、奖励都在）。要训的是带记忆的模型，所以存的不是零散的 (s, a) 转移，而是整段轨迹，训练时才能当记忆喂进去。
- **小结：**黑箱方法通用、表达力强、架构选择多。难点有二：一是难优化——输入太多（整段视频、动作、奖励），模型要自己发现哪些特征对之后的利用有用；二是探索难（下一节）。样本效率完全继承自第 4 步的优化器：vanilla 策略梯度极不省数据，PPO 稳一些但仍不省，SAC 省得多、只是超参数更难调。

11. in-context 探索为什么难：走廊与纸条

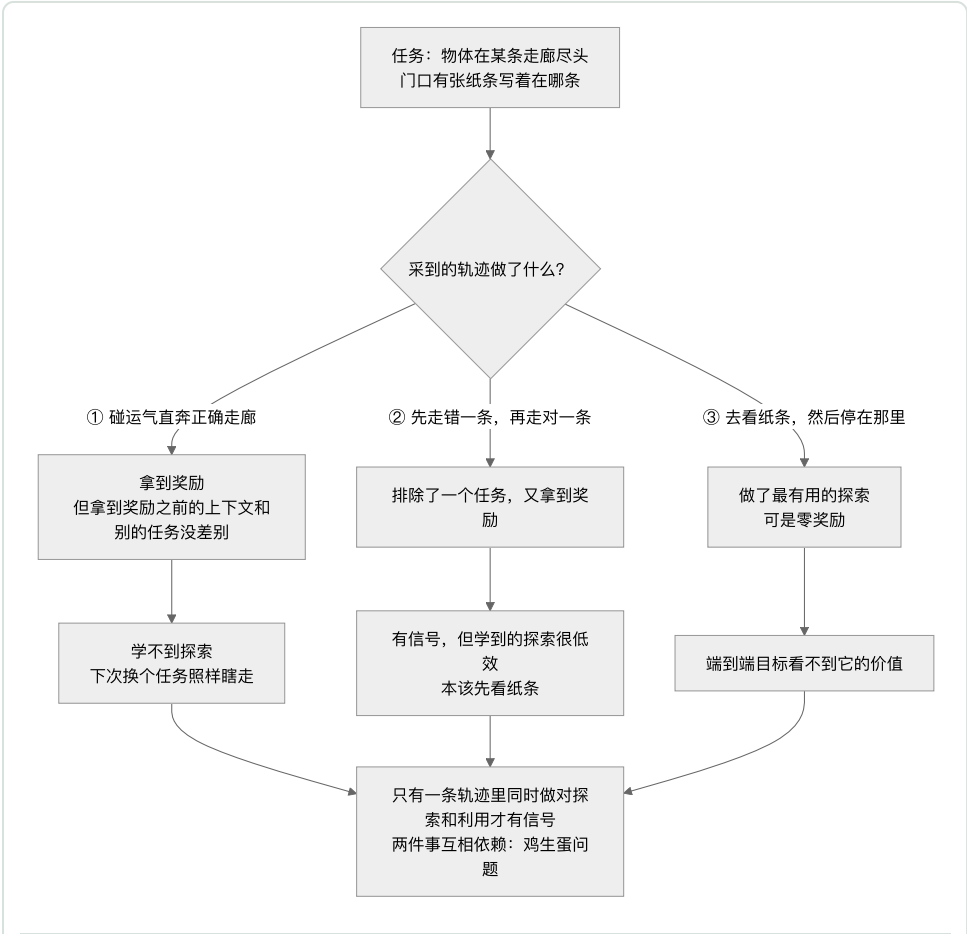
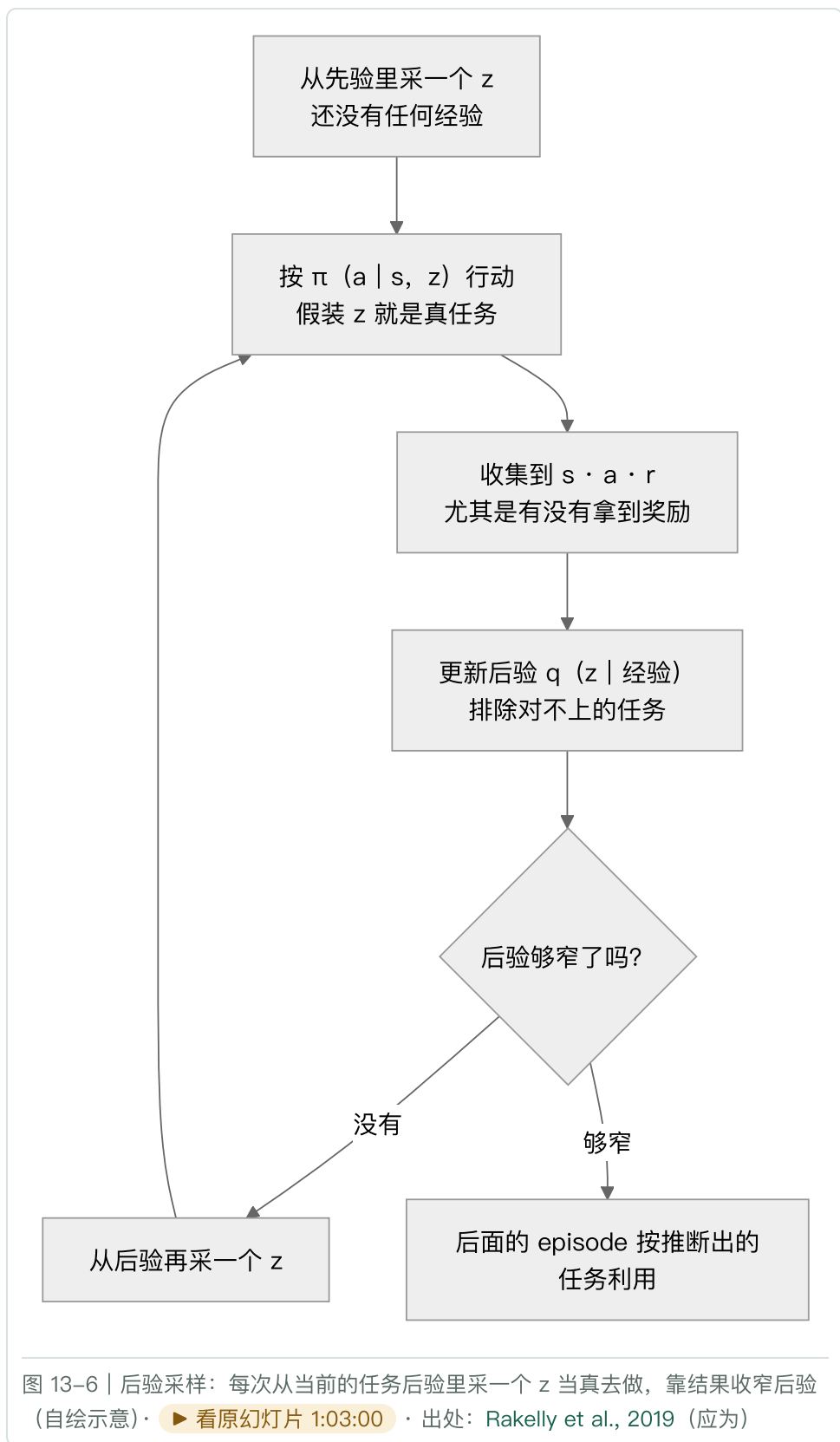


图 13-5 | 走廊加纸条：三种采样到的轨迹分别给端到端训练什么信号（自绘示意）·
▶ 看原幻灯片 56:21 · 出处：Liu et al., 2021 (DREAM; 应为)

- **端到端的承诺与代价。**直接对 n 个 episode 的总回报优化，原则上能学到**最优的探索—利用权衡**——但优化极难。
- **例子：**机器人要找备用灯泡；有好几条走廊，不同任务对应灯泡在不同走廊尽头；门口有张纸条写着在第几条——这个信息在状态里。学习时采到的轨迹有三种（图 13-5）：碰运气直奔正确走廊，拿到了奖励，但拿到奖励之前的上下文和任务换成别的走廊时**没有任何区别**，它没学到探索，也没学到怎么用探索到的信息；先走错一条再走对一条，排除了一个任务又拿到奖励，教了它一种可行但低效的探索——本该先看纸条；去看纸条然后待在那儿，最有用的探索，可是零奖励。
- **结论：**只有当一条轨迹里**同时**做对了探索和利用，才有好的学习信号；两件事分在不同 episode 里就没有信号。于是样本复杂度取决于“运气好到两件事一起做对”的概率——同时学探索和利用很难。
- **做饭的比方：**要在新厨房快速学做饭。不会找食材（探索差）就做不了饭；不会做饭（利用差）就算找到食材也拿不到奖励。鸡生蛋的**耦合问题**：探索和执行互相依赖才能一起成功。
- **问答：根子是不是奖励稀疏？**——奖励稠密的话确实好办得多，因为不需要专门采取探索动作；这个问题只在**探索本身难**的时候才出现，探索容易时只剩利用要学。

小注：走廊加纸条的例子和做饭的比方应出自 DREAM (Liu et al., 2021, Finn 组)，那篇工作的解法——把探索和利用解耦——是第 14 讲的内容。

12. 解法一：后验采样；其他解法留到第 14 讲



- **思路**：不只对端到端回报优化，加上**辅助的探索目标**，逼它去弄清任务。今天讲一种，周五（第 14 讲）讲更多。
- **后验采样**（posterior sampling，也叫 Thompson sampling）：前提是第 10 节那种能推断任务分布的方法—— z_i 现在是一个学出来的隐变

量，不是已知的。做法：从当前对 z 的分布里采一个 z ，然后假装它就是真任务去行动。

$$z \sim q_\phi(z \mid \mathcal{D}_{<t}), \quad a_t \sim \pi_\theta(a_t \mid s_t, z)$$

q_ϕ 是由目前经验算出的任务后验， π_θ 是以任务变量为条件的策略：每个 episode 开头采一个 z ，整段按这个 z 行动，episode 结束后用新经验更新后验再采。只要"给定 z 的策略"已经会做那个任务，探索就自动变成"逐个尝试可能的任务"。

- **点机器人例子**：稀疏奖励，碰到某个圆圈才有分，不同圆圈对应不同任务。先从先验采 z 、按各个 z 跑出几条探索轨迹，据此推断出任务大概在两个候选之一；再采、再跑，一旦某条轨迹拿到正奖励，后验一下子收窄，之后就按推断出的任务执行。
- **什么时候不好**（讲者让大家猜）：动作空间大只是部分原因； z 出分布当然会坏；见到一点奖励就过早利用也可能。讲者心里的答案是**走廊那个场景**：后验采样学到的策略是挨条走廊去试，而不是先看纸条——它就是靠"执行各个候选任务的策略"来探索的。走廊要是很长（走一条要几天），它会花几天试一条、再几天试下一条，而不去主动找能告诉它答案的信息。反过来，没有纸条的话挨条试就是对的。
- **其他解法**（点到为止）：加**内在奖励**（intrinsic reward）鼓励访问新状态；或者鼓励访问能**区分不同任务的动力学和奖励模型**的状态。这类探索比后验采样高效得多，但有很多干扰项或动力学很复杂时也会出问题。它们都比端到端好优化、各有原则，但在某些环境里可能相当次优。

小注：这些就是第 14 讲的主题——DREAM 以及基于"任务可辨识信息"的探索；和 CS329A 第 2 讲讲到的"重复采样要多样"是同一个问题在 RL 里的正式提法。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
迁移的三种设定	fine-tune / 多任务 zero-shot / 元学习 few-shot 各要什么前提	6:19	—
few-shot 图像分类	六张带标签图加一张查询图；每个 few-shot 问题本身是一条样本	9:22	—
GPT-3 的 in-context learning	几对改错英文的例子放进 prompt, base 模型直接会做	9:52	GPT-3 (推断)

图	看什么	跳转	出处
迷宫例子	第一个 episode 探索、第二个直奔目标；探索是 meta-RL 独有的难点	12:27	—
带记忆的策略	状态、动作、奖励一起进序列模型，上下文随经验增长	20:20	—
算法四步与 $n = 2$ 的展开图	采任务 $\rightarrow n$ 个 episode $\rightarrow$ buffer $\rightarrow$ 最大化全任务回报；记忆在哪里截断	27:45	RL ²
架构与优化器	RNN、自注意力加卷积、deep set 三种实现	31:53	SNAIL (应为)
视觉迷宫实验	1000 个小迷宫训练；小、大迷宫上找到目标的用表：随机 / LSTM / 自注意力	35:27	SNAIL (应为)
DeepSeek-R1 对难题和 $2 + 2$	简单题也长想；把 test-time compute 的效率纳入训练	40:01	MRT (应为)
任务推断视角	流进策略的信息就是任务信息；探索是为了推断 z	44:07	—
meta-RL 是 POMDP	隐状态 = 观测加任务；唯一看不见的是任务	47:39	—
off-policy 提速曲线	蓝线（带 replay buffer）比 RL ² 等 on-policy 方法学得快得多	51:12	PEARL (应为)
走廊加纸条的三条轨迹	哪条有信号、哪条没有；为什么要同时做对探索和利用	56:21	DREAM (应为)
后验采样的点机器人	先按先验采样试探，后验收窄后直奔目标	1:04:31	PEARL (应为)

提到的工作

名称	在本讲里的作用
多任务 RL、目标条件 RL、聚合 MDP (第 12 讲)	开场回顾；本讲三种视角的参照物
fine-tuning (监督学习里的微调概念)	迁移设定一：Q 函数或策略都能微调，但要求任务极像
few-shot 图像分类	RL 之外的元学习例子：每个 few-shot 问题本身是一条样本
GPT-3 (Brown et al., 2020) 的 in-context learning	base 模型改错英文的例子；元学习的另一种形态 (出处为推断)
RL ² (Duan et al., 2016) · 指定阅读	最早的黑箱 meta-RL：RNN 策略、奖励当输入、记忆跨 episode；也是 PEARL 曲线里的基线
Learning to Reinforcement Learn (Wang et al., 2016)	同期同思路的 RNN 方法 (应为"最早的工作"之一，课上未点名)
TRPO (Schulman et al., 2015)	"PPO 出现前、假设类似的算法" (推断)；RL ² 的底层优化器

名称	在本讲里的作用
SNAIL (Mishra et al., 2017)	自注意力加 1D 卷积的架构；视觉迷宫实验的来源 (应为)
Deep Sets (Zaheer et al., 2017)	前馈加平均的第三种架构 (课上只提了名字)
LSTM	迷宫实验里的记忆基线，输给自注意力架构
DeepSeek-R1 (2025)	test-time compute 的现象：难题长想、2 + 2 也长想
MRT (Qu et al., 2025)	用 meta-RL 优化 test-time compute 的效率 (应为)
第 10 讲 Aviral Kumar 的不等式例题	讲者说可能是同一道题
PEARL (Rakelly et al., 2019)	显式推断 z 的分布、off-policy 提速、后验采样与点机器人例子 (应为，课上未点名)
PPO (Schulman et al., 2017)、SAC (Haarnoja et al., 2018)	第 4 步可选的底层优化器：PPO 稳但不省数据，SAC 省数据但难调 (第 4、5 讲)
POMDP	描述 meta-RL 的正确形式：只有任务不可观测
posterior sampling / Thompson sampling	今天讲的唯一一种辅助探索方法
内在奖励、区分任务的探索	点到为止的其他解法，第 14 讲展开
DREAM (Liu et al., 2021)	走廊纸条例子与做饭比方的出处 (应为)；第 14 讲

术语对照

English	中文
meta-reinforcement learning (meta-RL)	元强化学习：在一个任务分布上学"怎么快速适应新任务"
transfer learning / fine-tuning	迁移学习 / 微调：先学源任务，再在目标任务上接着训练
multi-task learning	多任务学习：一个策略同时学一批任务 (第 12 讲)
task descriptor	任务描述符：告诉策略在做哪个任务的输入 (prompt、目标、编号，或一份经验)
task distribution $p(T)$	任务分布：训练任务和测试任务都从它采样
zero-shot / few-shot / K-shot	零样本 / 少样本 / K 样本：新任务上不给、给一点、给 K 条经验
in-context learning	上下文内学习：把示例放进输入，靠前向计算而不是改参数来适应
episode / trial	一段完整交互 / 同一任务里连跑的 n 个 episode
trajectory τ	轨迹：状态、动作、奖励的序列
exploration / exploitation	探索 / 利用：收集信息 vs 用已知信息拿分

English	中文
exploration policy / task policy	探索策略 / 任务策略：收集适应数据的策略 / 适应后解任务的策略，可以是同一个网络
black-box meta-RL	黑箱元 RL：用序列模型直接把经验映射到动作，不显式建模"怎么适应"
recurrent policy / sequence model	循环策略 / 序列模型：带记忆（隐状态或上下文）的策略
hidden state / memory	隐状态 / 记忆：到目前为止经验的压缩
deep set	深度集合：逐条编码再取平均的置换不变结构
replay buffer	重放缓冲区：存旧经验反复训练；这里每个任务一个，存整段轨迹
on-policy / off-policy	用当前策略刚采的数据训练 / 能复用旧数据训练
return	回报：一段交互里奖励的（折扣）总和
discount factor γ	折扣因子：越晚的奖励打越多折，逼出"尽快拿分"
MDP / POMDP	马尔可夫决策过程 / 部分可观测 MDP：后者看不到真实状态
aggregate MDP	聚合 MDP：把任务描述符拼进状态得到的一个大 MDP（第 12 讲）
latent task variable z	隐任务变量：用连续向量表示"在做哪个任务"
task inference	任务推断：从经验估计 z
prior / posterior	先验 / 后验：没有经验时、看过经验后对 z 的分布
posterior sampling / Thompson sampling	后验采样 / 汤普森采样：从后验采一个 z ，当真去做
intrinsic reward	内在奖励：为鼓励探索额外加的奖励
auxiliary objective	辅助目标：主目标之外再加之训练信号
coupling problem	耦合问题：探索和利用互相依赖，缺一个另一个学不成
test-time compute	推理时算力：让模型在回答前多想
positional encoding	位置编码：告诉序列模型每个输入在第几步
extrapolation / out-of-distribution	外推 / 分布外：测试任务超出训练分布仍能用
sample efficiency	样本效率：达到某个性能要多少交互数据

字幕勘误

"metal learning" → "metalearning" → meta-learning；"Deep Seek RL one" → "our one" → DeepSeek-R1；"RL squared" → RL^2 （写法而已）；"positioning coding" → positional encoding；"paralyze" → parallelize；"goal condition" → goal-conditioned；"ZI""TI""TJ" →

z_i 、 T_i 、 T_j ；"PPO so algorithm" → PPO-style algorithm；"trading process" → training process；"past as input" → passed as input；"test video points" → test data points（推断）；"few-shot shot" → few-shot；51:12 处对比曲线里的 "RL squared and PPO" 中的 PPO → 应为 ProMP（推断，见第 10 节小注）；"up to me update" → then update。

带走的问题

1. 黑箱 meta-RL 把奖励当输入，测试时也要拿得到奖励。在你做的 agent 产品里，"新用户前几轮"的适应信号是什么？如果线上根本没有奖励，第 5 节那套还能喂什么进记忆？
2. 第 6 节说端到端优化原则上能学到最优的探索—利用权衡，第 11 节又说这个信号几乎拿不到。把两句话放在一起：什么样的任务分布（奖励稠密还是稀疏、有没有"纸条"）会让端到端方法可行？
3. 第 8 节把一条长思考链读成一个 trial：n 个 episode 对应什么？"换任务时清记忆"对应什么？MRT 的"进度"奖励，和第 12 节里"访问能区分任务的状态"是不是同一个思路？
4. 第 10 节里 PEARL 之所以能用 off-policy 算法，是因为把任务推断和控制解耦了。对第 5 节那种把奖励直接喂进 RNN 的黑箱策略，直接换 SAC 会遇到什么麻烦？（提示：buffer 里旧轨迹的探索行为是旧策略做出来的。）
5. 后验采样在走廊场景失效，是因为它探索的单位是"整条候选任务的策略"。如果用 LLM 做 agent，把"读说明书"当成一个动作，你会怎样设计一个辅助探索奖励，让它先看纸条再走？

CS224R 第 14 讲 | 探索 (Exploration)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 14 讲, 2025 年 5 月 16 日 · 承接第 13 讲的 meta-RL, 把那里留下的"怎么学会探索"收尾; 作业 4 要实现本讲的 DREAM 视频:
<https://www.youtube.com/watch?v=4tISKdi8teU> (1:12:42, 英文字幕是自动生成的, 游戏名、算法名和记号错得不少, 见文末勘误) 讲者: Chelsea Finn (本讲不是客座讲座) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: Decoupling Exploration and Exploitation for Meta-Reinforcement Learning without Sacrifices (DREAM, Liu et al., ICML 2021)

一句话: 探索 (exploration) 是 RL 里"要不要去试没做过的事"的问题。Breakout 随手乱按也能撞到分数, Montezuma's Revenge 的奖励是一串稀疏事件, 从零探索几乎不可能。本讲先在最简单的 bandit (单步、无状态、奖励随机) 里把问题说清: 用 regret 衡量探索好坏, "对不确定的动作保持乐观" (UCB) 和"从后验里采一个世界再行动" (后验采样) 是两种能分析或有实效的策略, 课堂选药游戏里后验采样远胜 greedy 和 ϵ -greedy。到了机器人和 LLM 这种大 MDP, 讲者的态度很直接: 从零探索基本不可行, 实际靠示范或预训练基座模型做初始化, 再加塑形奖励。但即便有先验, 部署时仍要探索 (厨房里找食材、推理时试不同思路), 这正是第 13 讲 meta-RL 留下的鸡生蛋难题: 端到端同时学探索和利用, 两边互相等对方先学会。DREAM 的解法是解耦——执行策略拿着任务标识做普通的多任务 RL, 探索策略只负责收集"能猜出这是哪个任务"的经验, 奖励是逐步的信息增益; 再用变分信息瓶颈把任务标识压成只含任务相关信息的表示 z , 探索就不会去看墙上的画。它能恢复最优探索, 样本复杂度从 $A^2 \log A$ 降到 $A \log A$, 还被用来替助教给学生的程序找 bug。

时间轴

时间	内容
0:06	开场: 今天三段——bandit 里的探索、大 MDP (机器人 / LLM) 怎么办、回到 meta-RL 学会探索
0:37	Breakout vs Montezuma's Revenge: 奖励稀疏、事件链长、agent 不懂精灵的含义; Mao 牌的比方
4:15	探索问题的两种看法; 探索 vs 利用; 餐馆、广告投放、钻井三个例子
6:49	什么叫"最优的探索": 从多臂 bandit 到大 MDP 的难度谱; bandit 的定义

时间	内容
9:53	Regret: 定义、oracle 上界、三种曲线; 问答: 期望、为什么线性、为什么只能用于分析
19:46	策略一: 面对不确定保持乐观 (UCB); 策略二: 后验采样; 工业界的推荐与广告
25:22	课堂游戏: 10 种药、5000 个病人; greedy / ϵ -greedy / UCB / 后验采样的对比; 问答: ϵ 的调度
32:10	大 MDP 怎么办: 讲者的悲观看法; 从零探索不可行, 靠示范、基座模型和塑形奖励
35:45	即便有先验也要 test-time 探索: 厨房找食材、LLM 推理; 回顾 meta-RL 的鸡生蛋问题
38:18	替代策略: 先学动力学 / 奖励模型再去探索; 干扰物问题
39:49	不学模型: 用任务标识 μ 解耦; 探索策略 + 任务预测器, 执行策略只做多任务 RL; meta-test 三步
44:54	泛化: 压缩的任务表示 z ; 变分信息瓶颈——加噪 + 幅度惩罚等于 KL; 问答: one-hot 为什么反而不压缩
52:37	执行目标的完整写法: 编码器 f 、执行策略 π 、KL 正则; 问答: 何时可以省掉瓶颈、为什么输出分布
1:00:24	探索奖励: 末端预测误差太稀疏, 改成逐步的信息增益
1:04:30	DREAM: 命名、作业 4; 理论: 最优性与 $A \log A$ 对 $A^2 \log A$; 读墙上颜色的实验; 小结
1:09:38	应用: 用 DREAM 给学生的 Bounce / Breakout 程序找 bug; CS106A 2023 春: 批改快 44%、准 6%

核心内容

1. 为什么探索难: Montezuma's Revenge 与 Mao 牌

- **开场安排:** 三段——先在 bandit 里讲探索 (最简单、能分析); 再看这些想法能不能搬到机器人和 LLM 的大 MDP; 最后回到第 13 讲的 meta-RL, 讲怎么"学会探索"。
- **两个 Atari 游戏的对比:** 训练一个 RL agent 玩 Breakout, 奖励就是得分, 能得到很强的玩家; 换成 Montezuma's Revenge, RL 系统就很难玩好。差别在于第二个游戏的奖励是稀疏的离散事件: 拿钥匙有用、开门有用、要躲滚动的骷髅、别掉下平台——人一眼看懂这些精灵的含义, agent 什么都不知道; 通关只和这些事件弱相关, 事件之间又相隔很远, 一开始 agent 完全拿不到任何奖励, 连"走到钥匙那里"都要一长串动作。

小注：Breakout / Montezuma's Revenge 这对例子几乎是探索文献的标配。课上只带过一句“给新状态、新动作发奖励 bonus”的研究路线，代表作用伪计数（pseudo-count）当 bonus 的 count-based exploration（Bellemare et al., 2016）和用随机网络的预测误差当 bonus 的 RND（Burda et al., 2018）；预测误差类 bonus 有一个著名的失效模式叫 noisy TV：一台随机换台的电视永远“新奇”，agent 会站在那里看个不停。本讲没有展开这些。

- **Mao 牌的比方：**一种“半是游戏半是玩笑”的纸牌游戏——事先不告诉你规则，犯规就罚，罚了也不告诉你为什么，规则未必讲道理，只能靠试错把规则摸出来。RL 算法面对 Montezuma's Revenge 就是这个处境：不知道规则，不知道什么好什么坏，不知道自己能做什么；任务越长，要摸索的动作序列就越长。讲者的极端版本：想象你的人生目标是赢 50 局 Mao，而且这个目标本身也没人告诉你，只能从赢局的奖励和输局的罚分里慢慢体会——稀疏奖励下的 RL 就是这么难。

2. 探索与利用：同一个目标下的两股力

- **两种看问题的角度：**一是“发现高奖励的策略”——单个动作可能都不给奖励，串起来才有；二是“决定试新的还是继续做已知最好的”。后者把问题拆成一对：**利用（exploitation）**——做已知能拿高奖励的事，比如按当前的 Q 函数选动作（第 6 讲： $Q(s, a)$ 是在状态 s 做动作 a 之后预计能拿到的总回报）；**探索（exploration）**——做没做过的事，指望找到更高的奖励。目标是长期总奖励最大，两者必须平衡。
- **三个例子：**搬到新城市，探索是试新餐馆，利用是去最喜欢的那家；广告投放（RL 一个非常常见的用途）——利用是放历史上最成功的广告，探索是随机换一条；石油钻探——利用是在出过油的地方接着钻，探索是换个新地点。
- **“最优的探索”是什么意思：**取决于问题多复杂。讲者画了一条难度谱：多臂 bandit（一步、无状态）→ 上下文 bandit（每一步带特征）→ 小 MDP → 大 MDP。左端可以严格分析一个策略是否最优；右端——本课大部分时候面对的、没有已知结构的大 MDP——不但分析不可行，连设计能保证探索得好的算法都不可行。

3. Bandit：把探索问题剥到最简

- **定义：**bandit 是 RL 的一种极简形式（名字来自“独臂强盗”老虎机，讲者特意说不是抢东西的那种强盗）：每个 episode 只有一步（horizon 为 1），没有状态，而且——和本课之前所有 MDP 不同——

奖励是随机的。选餐馆就是 bandit：每次选一家（一条"臂"），体验好坏有随机性，下一次再根据过去的经验决定选熟悉的还是没去过的。

- **为什么值得单独讲：**它足够简单，能定义并分析"探索得好不好"；同时它在工业界（推荐系统、广告位）被大量使用——很多实际问题确实没有长 episode、没有状态。

4. Regret：衡量探索好坏的尺子

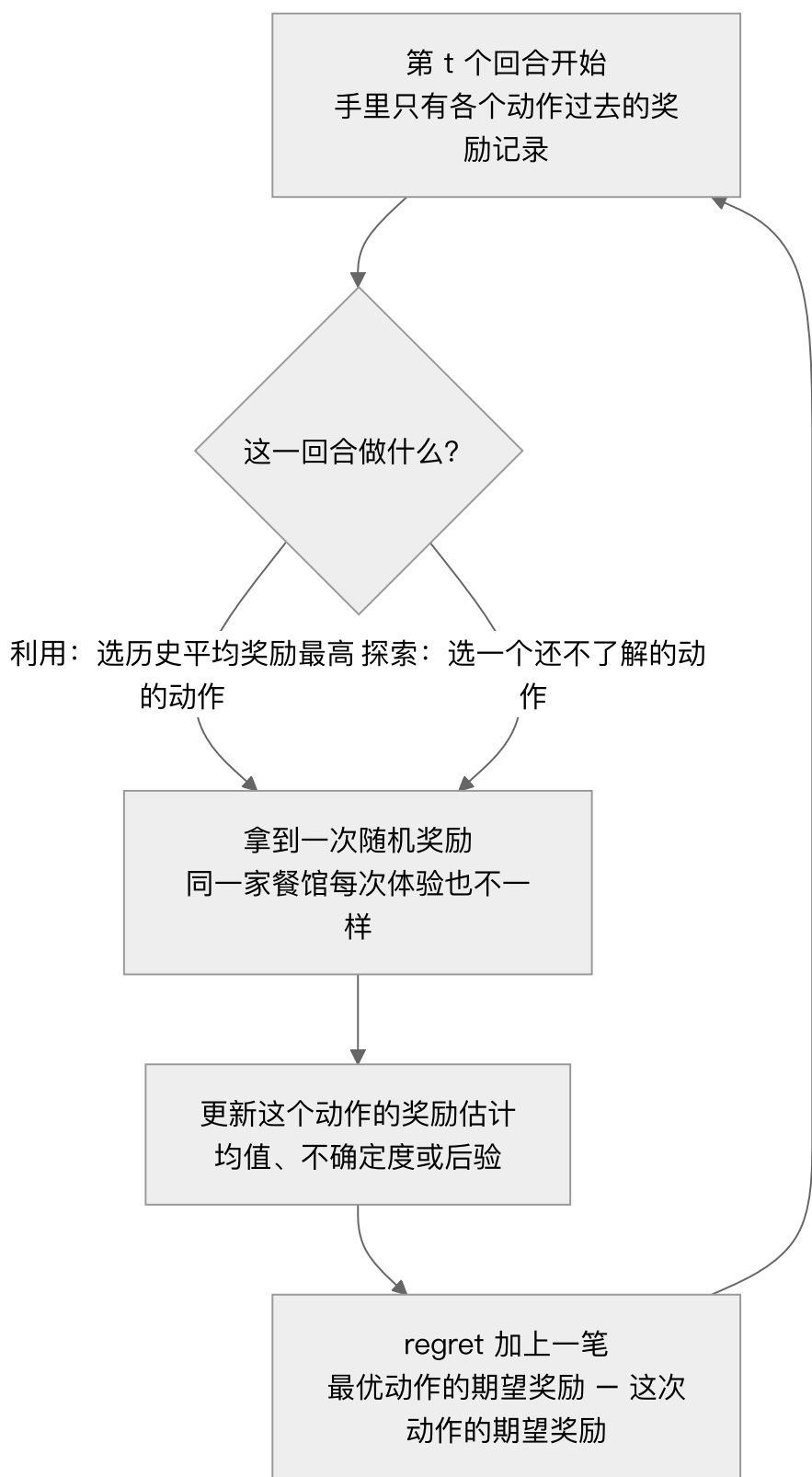


图 14-1 | bandit 的回合循环：每一回合都在"利用"和"探索"之间二选一，代价记在 regret 上（自绘示意）· [▶ 看原幻灯片 8:21](#)

- **换一种记账方式：**之前的课看的是每个 episode 的回报，有时取平均；bandit 里关心的是整个学习过程的最优性——先探索、后利用，

总账算下来离最好能拿到的差多少。

- 假设已经跑了 T 个 episode。事后看，最好的情况是每一回合都选期望奖励最高的动作 a^* ，总共拿到 $T \cdot \mathbb{E}[r(a^*)]$ 。这是一个 oracle 的上界：第一回合你对问题一无所知，只能随机选，真实算法做不到它。把它减去实际拿到的奖励之和，就是 regret ("后悔值")。

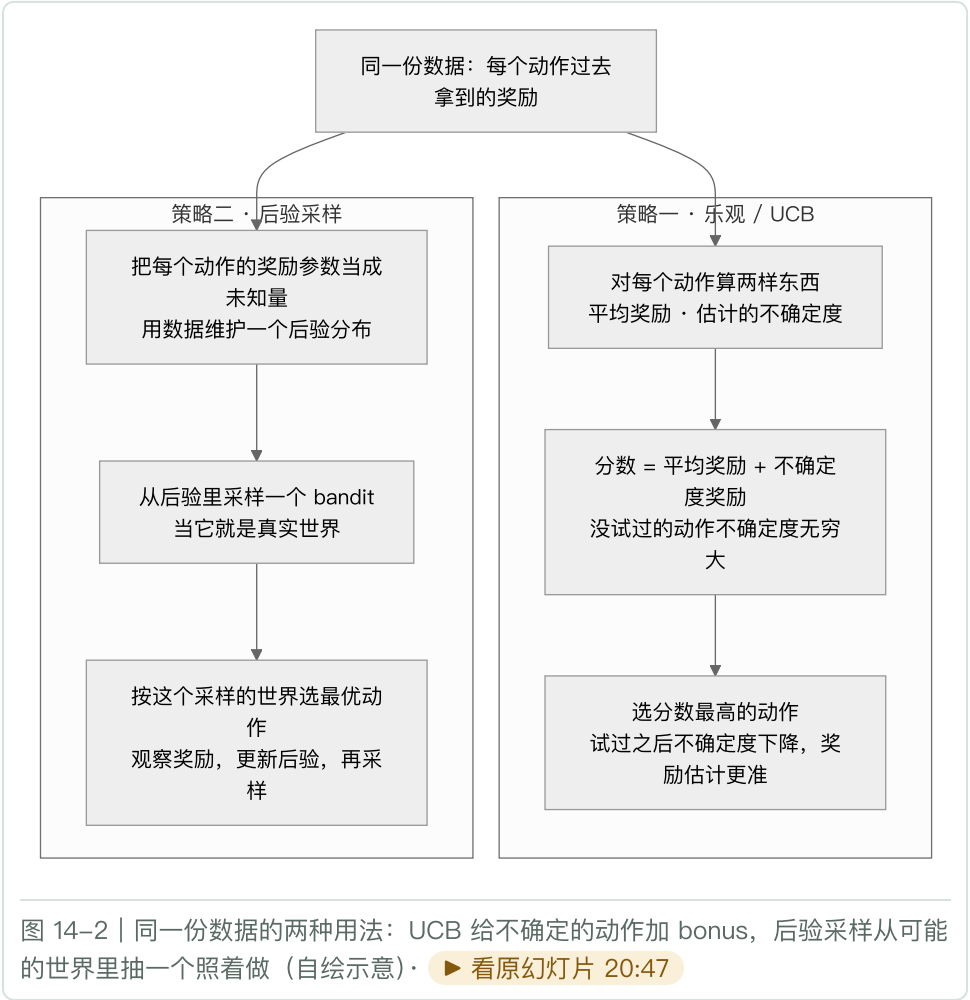
$$\text{Regret}(T) = T \cdot \mathbb{E}[r(a^*)] - \sum_{t=1}^T r(a_t)$$

a^* 是期望奖励最高的动作（只有 oracle 才知道是哪个）， T 是已经跑过的回合数， $r(a_t)$ 是第 t 回合实际选的动作 a_t 拿到的奖励；前一项是"事后看最多能拿多少"，后一项是"实际拿了多少"，差值越小越好。

- **三种 regret 曲线**（横轴是 episode 数；bandit 里每个 episode 只有一步，所以也就是时间）：
 - 一直选最优动作：regret 恒为 0（期望意义下；有随机性时会有小波动）。
 - 一直选同一个次优动作，或者纯随机：期望意义下每回合都多付一笔固定的差价，regret 线性增长——**线性 regret 意味着算法根本没有从探索里学到东西**。
 - 聪明的算法：开头像线性（在探索、在收集信息），之后拐平（进入利用阶段）——这叫次线性（sublinear）regret。
- **问答**
 - 曲线是多次运行取平均吗？——图上每条是一次运行，画得很光滑；实际会有噪声，比如某一步运气差就往上跳一下。对多次运行取期望才是平滑的那条。
 - 一直选 A_1 的话怎么算这个公式？——**算不了**。regret 需要知道最优动作的期望奖励，也就是知道 bandit 的真相，所以它是分析工具，不是训练目标；我们想要的是让它拐平，但不能直接对它做优化。
 - 它一定单调吗？——取期望后单调；单次运行里奖励是随机的（比如最优动作 90% 的时候给高奖励），实际拿到一次好奖励时曲线反而会往下走一点。
- **在 bandit 上很多简单策略可以证明最优或给出 regret 界**——讲者也提醒：有理论保证的算法，实际表现照样会因问题而异。

小注：UCB1 在多臂 bandit 上的 regret 是 $O(\log T)$ 量级（Auer et al., 2002），log 曲线画出来正是"先陡后平"，这就是"次线性"的直观含义。

5. 两种 bandit 策略：乐观（UCB）与后验采样



- **策略一：面对不确定保持乐观。**对每个动作记录过去奖励的平均值；纯利用就是选平均值最高的。为了鼓励去试新的，再加一项和"奖励估计的方差"成比例的 bonus：越不确定的动作分数越高。直觉是"没见过的东西先假设它好"；试过发现差，方差和均值估计都会掉下来，以后自然不选它。这类算法叫 **UCB (upper confidence bound, 置信上界)**，可以推出 regret 保证。
 - 问答：没试过的动作方差是多少？——可以当成无穷大，所以算法会先把每个动作至少试一遍，之后不确定度随数据增多而下降。

$$a_t = \arg \max_a \left[\hat{\mu}_a + c \cdot \hat{\sigma}_a \right]$$

$\hat{\mu}_a$ 是动作 a 过去奖励的平均值， $\hat{\sigma}_a$ 是这个平均值的不确定度（讲者说成"奖励估计的方差"），c 决定乐观到什么程度；没试过的动作 $\hat{\sigma}_a$ 视为无穷大，所以每个动作至少会被试一次。

小注：UCB1 的具体 bonus 是 $\sqrt{(2 \ln t) / N_a}$ ， N_a 是动作 a 被选过的次数 (Auer et al., 2002)。**"置信上界"**的意思就是用"均值加置信区间上沿"去排

序。

- **策略二：后验采样 (posterior sampling)**。第 13 讲末尾见过它。把每个动作的奖励分布看成参数未知的东西——这其实定义了一个部分可观测的 MDP (POMDP: agent 看不到环境的全部状态, 这里看不到的是奖励函数的参数); 用已有数据维护对参数的后验分布, 每回合从后验里采样出一个"可能的 bandit", 假设它就是真的, 选它下面最优的动作; 看到奖励后更新后验, 再采样, 循环往复。它比 UCB 难给理论保证, 但实践里常常很好。

$$\theta \sim p(\theta \mid \mathcal{D}_t), \quad a_t = \arg \max_a \mathbb{E}[r \mid a, \theta]$$

θ 是各个动作奖励分布的未知参数, $\mathcal{D}_t$ 是到目前为止的全部观测, $p(\theta \mid \mathcal{D}_t)$ 是后验; 每回合先抽一个"世界" θ , 再选这个世界里期望奖励最高的动作。

小注: 这就是 Thompson sampling (Thompson, 1933)。第 13 讲说的"用后验采样解决 meta-RL 的探索"是它从 bandit 推广到 MDP 的版本: 采的不是
一组奖励参数, 而是一整个 MDP。

6. 课堂游戏: 给 5000 个病人选药

- **设定**: 你是某研究医院药物研发的负责人, 瘟疫来了, 手上有大约 10 种药、约 5000 个病人, 每个病人只能开一种药, 结果是活或死。目标是累计救活最多的人——既要救眼前的人, 又要为将来积累关于每种药的信息。这就是一个 bandit: 每种药一条臂, 疗效未知。
- **全班玩了一局**: 先随意试了 1、4、10、7、6、5 号药, 10、6、5 出现死亡; 7 号连活几次后大家开始"利用"它, 直到它也出现死亡; 又试了 8、9、1、4、3; 9 号连续救活, 最后大家决定一直开 9 号。揭晓真实疗效: 9 号确实最好, 7 号也不错——全班找到了最优臂。
- **和算法比**: 以"救活人数"和"regret"两张图对比四种算法——greedy (只选目前看起来最好的) 随病人数增多越来越差; ϵ -greedy (第 6 讲: 以概率 ϵ 随机选、否则贪心) 早期比 UCB 好、后期反被 UCB 超过, 因为 UCB 更舍得探索, 长期能选到更优的动作; 后验采样一骑绝尘。全班的曲线甚至比所有算法都好——讲者说这只是一次运行, 有运气成分。
- **问答**: 后验采样赢在哪, 有数学解释吗? ——这个具体问题也许能分析, 但一般情况下很难给它通用的理论保证。 ϵ -greedy 的问题很大程

度在 ϵ 这个超参数：通常要随时间衰减；如果 ϵ 固定，到了后期你已经很了解各种药了，还以固定概率给病人开随机药，代价很大。

7. 大 MDP 里的探索：讲者的悲观看法与实际做法

- **能不能把这些搬到机器人和 LLM**：各家看法不一，讲者自称偏悲观。研究上确实有很多尝试：估计某个状态或动作是否见过，给新状态、新动作发"新奇度"bonus；或者仿照后验采样，对动力学建模再据此行动。但到了很大的 MDP，从零开始探索基本上是不可行的——
 - 随机初始化一个 Transformer 语言模型，只用"用户点赞"这类奖励从零训练，不可能成功：token 序列的空间太大，用户的耐心太少。
 - 机器人从零学倒水，只会在那里随机挥手臂，做不出任何有用的事。
- **实际的探索长什么样**：重度依赖示范（第 2 讲的模仿学习）或在数据和示范上预训练的基座模型。RL 在 LLM 上成功，很大程度是因为基座模型已经会写通顺的句子、大致知道怎么处理一类 prompt（第 9、10 讲）；机器人若有达成高奖励的示范，以它为起点做 RL 威力很大。这些初始化本质上是把模型偏置到状态空间里"可能有奖励"的那一片——一位学生的总结，讲者认可：就是在告诉模型"这些是我想让你做的那类事"。
- **另一件能做就做的事：塑形奖励（shaped reward）**——用稠密奖励替代稀疏奖励来引导探索。仿真里可以手写；从偏好学出来的单步奖励（第 8 讲）本身也相当稠密。这些可以和示范、基座模型组合使用。

小注：这一段和 CS329A 第 2 讲、CME295 第 5 讲说的是同一件事：LLM 的 RL 之所以"探索"得动，全靠预训练分布本身就把好答案放在采样可及的范围里，所以那边把探索落实为采样多样性（temperature、重复采样），而不是从零发现。

8. 即便有先验也要在 test-time 探索：回到 meta-RL 的鸡生蛋问题

- **问题没有消失**：就算有一个训好的、会很多事的策略，部署时仍可能缺少拿高奖励所需的信息，必须在测试时探索。第 13 讲的两个例子：厨房里做饭得先开抽屉、开柜子找食材；LLM 做推理题得试不同的解题思路，而不是直接往下写自以为对的 token（CS329A 第 2 讲的采

样多样性、第 13 讲"test-time compute 就是 meta-RL"说的是同一件事)。

- **meta-RL 的答案与它的病**：把策略做成带记忆（历史）的模型，记忆让它能"先试、记住、再利用"——这就是第 13 讲的黑箱 meta-RL (RL^2)。但如果用一个端到端目标同时优化探索和利用，会撞上**鸡生蛋问题**：还没学会探索，就找不到食材、拿不到奖励；还没学会做任务，任何探索行为也换不来奖励。两边互相等对方先学会，学习信号只在两件事同时做对时才出现——这叫耦合 (coupling) 问题。
- 第 13 讲还讲了几种更容易优化、但会牺牲最优性的替代探索策略（后验采样是其一）。本讲要给的方法要两头都占：既能得到最优的探索—利用权衡，又容易训练。它建立在最后一种替代策略之上，所以先回顾那个。

9. 替代策略：学一个模型再探索，以及它的干扰物问题

- **做法**：对每个环境 / 任务拟合一个动力学模型和一个奖励模型（第 11 讲 model-based 的思路），然后让探索策略去做"能让我对环境模型更了解"的事——探索目标就和执行任务解耦了。在"导航到不同圆圈"的一族 MDP 里，这样训出来的探索轨迹（幻灯片上的紫色轨迹）会专门去访问能暴露奖励函数位置的状态。
- **好处与坏处**：能学到高效的探索策略；但在某些场景下会次优——**干扰物 (distractor)** 问题：如果墙上挂着画，为了把每个环境的模型学准，它会把所有的画都看一遍，而这对任务毫无用处。
- **本讲的目标**：沿着"探索目标与执行解耦"的思路，但不学模型，也不去关心任务无关的信息。

10. DREAM 的核心一步：用任务标识把探索和执行解耦

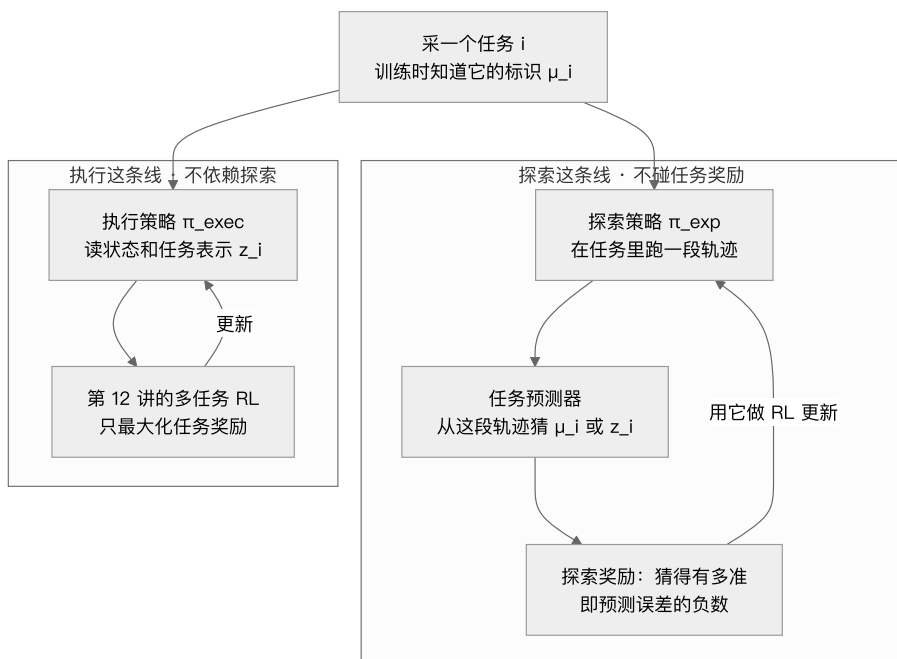


图 14-3 | DREAM 训练时的两条线：探索策略只为“猜出任务”而学，执行策略拿着任务标识只为“做好任务”而学，互不等待（自绘示意） · [▶ 看原幻灯片 40:50](#) · 出处: Liu et al., 2021

- **把“预测环境”换成“预测任务”：**给每个任务贴一个标识 μ_i ——可以是 one-hot 编号、一句语言描述，或者一个连续向量。探索的目标改成：收集能让我猜出“现在是哪个任务”的经验。
- **两条独立的训练线**（讲者说实践中两个策略可以合成一个，但分开想更清楚）：
 1. **探索线：**探索策略 π_{exp} 在任务里跑出一段经验；另训一个任务预测器，从这段经验预测 μ_i 。探索策略的奖励就是预测器猜得多准（负的预测误差）。这个目标和“把任务做好”完全无关。
 2. **执行线：**执行策略 π_{exec} 直接拿任务标识当输入，只优化任务奖励——这就是第 12 讲的多任务 RL，它不依赖探索好不好。
- 耦合于是被拆开：执行不等探索，探索不等执行。
- **问答**
 - 哪部分是探索？——探索策略本身；预测器只是给它提供奖励。
 - 和第 13 讲的黑箱 meta-RL 差在哪？——仍然是采样不同 MDP 来训练；差别有两点：探索的目标函数换了；执行策略被喂了任务标识，所以学做任务时不需要一个好的探索策略。
 - 能泛化到没见过的任务吗？——one-hot 标识会有困难，第 11 节解决。

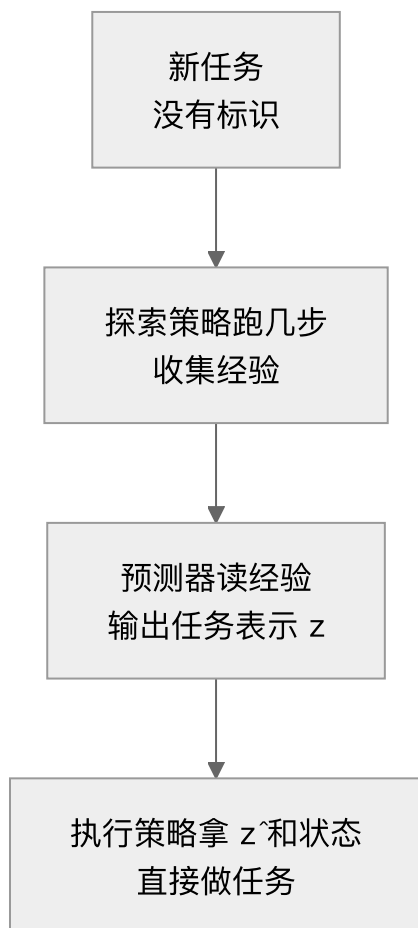
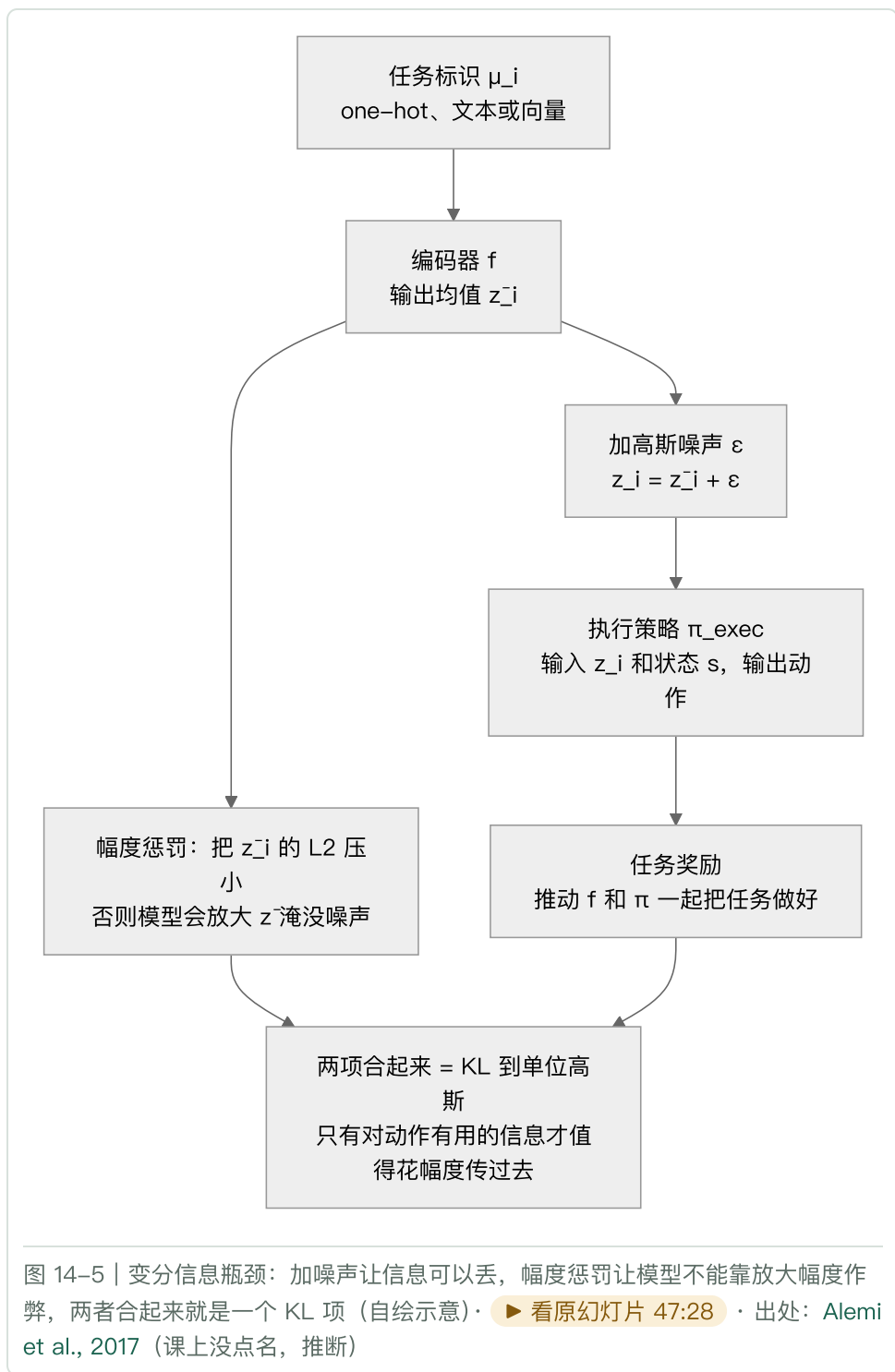


图 14-4 | meta-test 三步：先探索、再识别任务、再利用（自绘示意）。

► 看原幻灯片 43:23 · 出处：Liu et al., 2021

- **meta-test 时怎么用**：新任务来了，先跑探索策略收集经验 → 把经验喂给任务预测器得到任务表示 → 把它喂给执行策略去做任务。全程不用建模动力学或奖励。

11. 让它泛化：压缩的任务表示与变分信息瓶颈



- **one-hot 的问题**：新任务未必能对上旧的某个编号；而且 one-hot 其实比连续表示**更不压缩**——四个目标位置的任务里，任务 1 和任务 4 的目标挨得很近，one-hot 却把它们当成毫不相干的两个，学出来的连续表示会把它们放得很近。此外标识里常混着任务无关的东西（墙的颜色、装饰），只有“食材在哪”才要紧；两个厨房食材位置相同、其他不同，本该算同一个任务。

- **办法**：不直接预测 μ_i ，先把它编码成一个压缩的潜在任务表示 z_i ，把任务无关的信息"瓶颈"掉；探索线改为预测 z_i 。
- **怎么在神经网络里压缩信息**（讲者先板书讲原理）：
 1. 编码器把 μ_i 映射成 z_i ，执行策略读 z_i 和状态输出动作。端到端训练不会自发丢信息。
 2. 给 z_i 加高斯噪声。加噪确实会丢信息，但只在测试时加会把有用的信息也丢掉；训练时也加噪，模型会学会把 z_i 的幅度放大，让噪声相对变小——照样能把所有信息塞过去。
 3. 所以再加一项损失，惩罚 z_i 的幅度（L2）。幅度被压住之后，模型只会把对预测动作真正有用的信息"花幅度"传过去，其余的自然被噪声淹没。
- 这套"加噪 + 幅度惩罚"就是**变分信息瓶颈（variational information bottleneck）**：把 $z_i + \varepsilon$ 看成一个均值 z_i 、方差固定的高斯分布，两项合起来等价于最小化它到单位高斯的 KL 散度（衡量两个分布差多远的量，CME295 第 5 讲 / 本课第 9 讲用过）。
- **执行线的完整目标**：两个带参数的网络——编码器 f 把 μ_i 映射成 z 的分布，执行策略 π_{exec} 读 z 和状态输出动作——一起最大化任务奖励，同时最小化 KL 正则。 z 是一个向量（有学生问）。

$$\max_{\pi_{\text{exec}}, f} \mathbb{E}_{z \sim f(\cdot | \mu_i), \tau \sim \pi_{\text{exec}}(\cdot | z)} \left[\sum_t r_i(s_t, a_t) \right] - \text{KL}(f(z | \mu_i) \| \mathcal{N}(0, I))$$

$f(z | \mu_i)$ 是均值为 z_i 、方差固定的高斯分布（也就是" z_i 加噪声"）， τ 是执行策略在任务 i 里跑出的轨迹， r_i 是任务 i 的奖励；第一项要求 z 足以把任务做好，第二项要求 z 别带多余信息。课上板书没写 KL 项的权重，实现里它是一个超参数。

$$\text{KL}(\mathcal{N}(\bar{z}, \sigma^2 I) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_j \left(\bar{z}_j^2 + \sigma^2 - 1 - \log \sigma^2 \right)$$

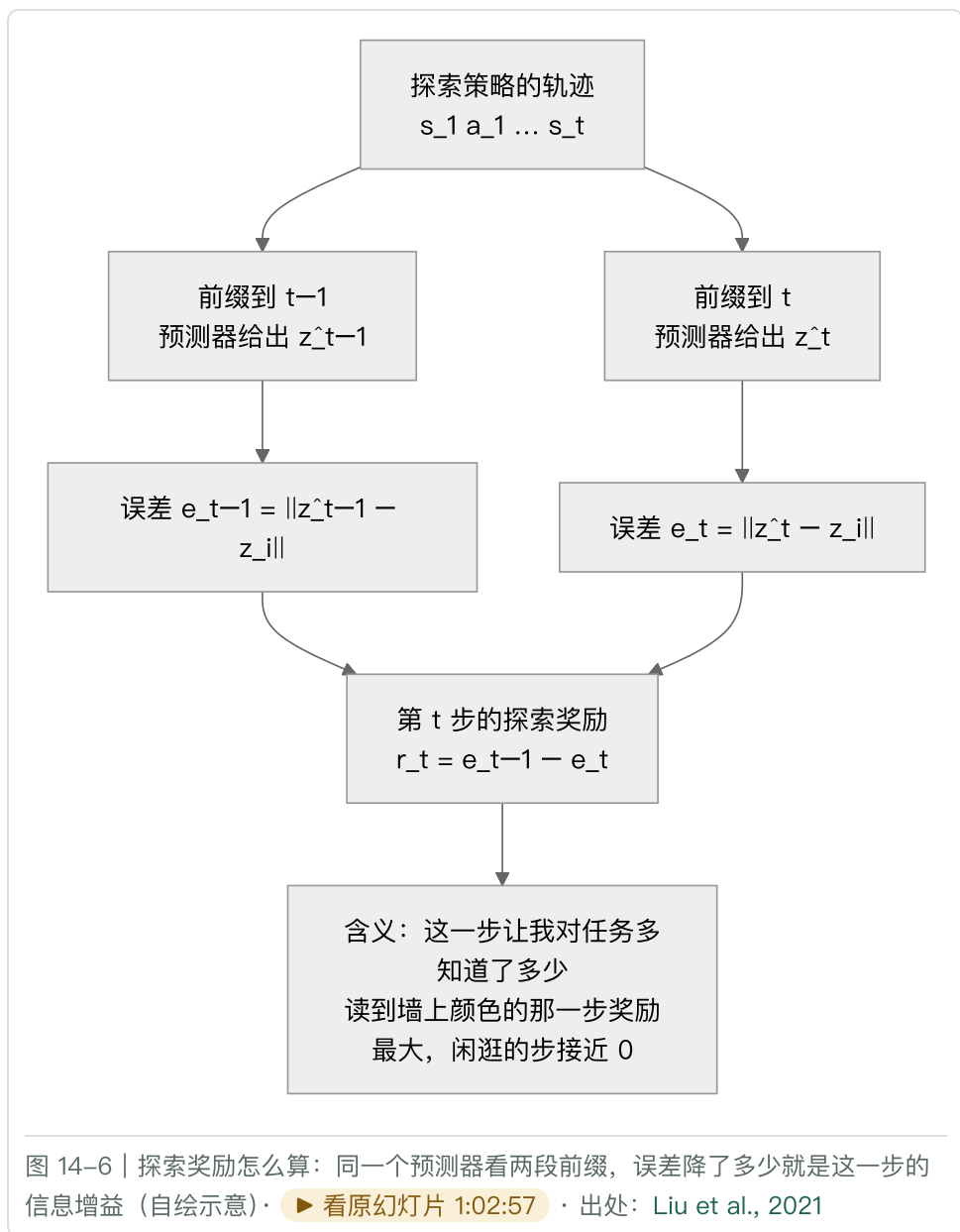
σ 固定时括号里只有 z_j^2 随参数变，所以这个 KL 就是 z 的 L2 惩罚加一个常数——讲者说"加噪声 + 幅度惩罚等价于 KL"，指的就是这个等式（课上没有展开，这是高斯 KL 的标准结果）。

• 问答

- ε 要进 loss 吗？——不用。训练时每次前向都重新采一个 ε 加上去，它的影响已经通过 KL 那一项体现了。

- 为什么要加 KL 项？——不加的话 z 会保留任务无关的信息，探索策略就会跑去探索那些无关的东西（墙的颜色）。
- 训练时给了任务标识，测试时怎么办？——测试时不需要 μ_i ：探索策略跑完，预测的是 z_i 而不是 μ_i ，直接喂给执行策略。
- 能不能让执行策略自己学会只看相关部分？——如果任务表示本来就干净，可以跳过瓶颈这一步：比如任务是"左转 / 右转 / 前进"这样的文本，或者目标坐标——相似的任务表示本来就相近。只有 one-hot 这类不含结构的标识才需要压缩；讲者说这是目前鼓励"只看相关部分"的较好的数学框架之一。
- 为什么要输出分布而不是一个向量？——随机性正是能丢信息的原因；确定性的连续向量哪怕只有一维也能编码无穷多信息，压维度只是启发式，控制不了信息量。

12. 探索策略的奖励：逐步的信息增益



- **目标：**让探索策略收集"能预测 z "的数据。两条线可以同时训。
- **最直接的奖励：**轨迹结束时看预测器对 z 猜得多准，奖励是负的预测误差，其余时刻为 0。问题是太稀疏——只有最后一步有信号，和第 1 节的 Montezuma's Revenge 一个毛病。
- **更稠密的写法：**每一步都问"这一步让我对任务多知道了多少"。用预测器分别看轨迹到 $t-1$ 步和到 t 步的前缀，两次预测误差之差就是这一步的信息增益，作为第 t 步的奖励。误差应随时间下降：读到墙上颜色的那一步奖励最大，闲逛的步骤奖励接近 0。

$$r_t^{\text{exp}} = \|\hat{z}(\tau_{t-1}) - z_i\|^2 - \|\hat{z}(\tau_t) - z_i\|^2$$

$z(\tau_{t-1})$ 是预测器只看轨迹前 t 步给出的任务表示估计， z_i 是执行线学出的"真值"；前一项减后一项就是"这一步让误差降了多少"，也就是这一

步带来的信息增益。课上把误差写成 $z - z_i$ ，没有明说取平方。

小注：DREAM 论文里这个奖励写成预测器对 z 的 log 似然之差（应为 $\log q(z | \tau_{1:t}) - \log q(z | \tau_{1:t-1})$ ）；预测器是高斯时，log 似然就是负的平方误差，和课上的写法一致。

13. DREAM 的理论、实验与小结

- **名字**：DREAM = Decoupled Reward-free Exploration And execution in Meta-RL（讲者自嘲和很多研究算法一样是硬凑的缩写）。作业 4 要实现它。"reward-free"指探索那条线不用任务奖励。
- **理论**：在相当温和的假设下，能恢复最优的探索与利用。效率上构造一个 bandit 式的问题：有 A 条臂，其中 0 号臂不是普通动作，而是"告诉你哪条臂最优"，最优策略显然是先拉 0 号臂、再拉它告诉你的那条。端到端 meta-learning 学到这个策略需要的样本量约 $A^2 \log A$ ，DREAM 只要 $A \log A$ （假设：底层用 Q-learning，外层做均匀探索）。图上随臂数增加，DREAM 到达最优的样本数远少于端到端（橙色）。
- **实验**：一族任务——去拿某种颜色的物体，颜色写在一堵墙的另一面。agent 得先绕过墙读颜色，再去对应的物体；奖励稀疏。端到端方法全部"躺平"，学不会探索；第 13 讲那些替代策略理论上能好一点但探索仍次优；DREAM 学会了"先去看牌子，再去找物体"。视频里探索 episode 绕到墙后读牌子，随后直奔蓝色钥匙。
- **问答**：探索和利用交错的任务呢？——这里是"先探索后利用"的设定；有些任务要先找信息、做一个子任务、再找信息。讲者说他们有后续工作把两者合到一个策略里，奖励是"识别任务的奖励 + 任务奖励"，感兴趣可以私下问。
- **小结**：端到端学探索——能得到最优，但探索难时几乎优化不动；替代策略——好优化、有原则，但可能大幅次优、探索低效；解耦——最优且好优化，两头都占。代价是**需要任务标识**：至少要能枚举训练任务、知道当前在哪个任务里，这是它拿到好处的关键前提。

14. 应用：让 meta-RL 替助教找学生程序里的 bug

- **场景**：入门编程课上学生写交互式小游戏——code.org 上的 Bounce（像半边 Pong，把球打进球门），Stanford CS106A 的 Breakout。学生程序会有 bug，比如球本该掉下去丢一条命，却从球拍上弹回来。助教批改时得亲自玩游戏才能发现 bug，非常耗时。

- **建模**：每个学生的程序是一个任务，目标是探索并找出其中的 bug，再据此给学生反馈。直接用 DREAM，学到的探索策略像一组测试用例：故意让球打进球门看会发生什么、故意不接球看球落地会怎样、故意把球打到墙上——有个 bug 程序一碰墙就生成一堆球。
- **落地**：2023 年春季 CS106A 的一次实验里，助教用 meta-RL agent 玩游戏的视频预填批改 rubric，自己只做核对：批改快了 44%，准确率高了 6%；助教喜欢用，之后至少又用了——一个学期。

小注：这条线应出自 [Giving Feedback on Interactive Student Programs with Meta-Exploration](#) (Liu et al., NeurIPS 2022 oral)：论文在 70 万份 code.org 学生程序上把找错准确率做到 94.3%，离人类只差 1.5 个百分点；课上说的 44% / 6% 是 CS106A 的课堂部署数据，论文摘要里没有。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
Breakout 与 Montezuma's Revenge	同是 Atari，为什么一个 RL 轻松拿高分、一个几乎学不动：奖励事件稀疏且弱相关	0:37	—
探索问题的难度谱	多臂 bandit → 上下文 bandit → 小 MDP → 大 MDP；越往右越难分析、越难设计算法	7:19	—
Regret 板书与三条曲线	公式两项各是什么；恒零、线性、次线性三种形状分别对应什么行为	12:00	—
UCB 与后验采样两页	UCB 是均值加方差 bonus；后验采样是"采一个 MDP、按它行动、更新、再采"	20:47	UCB 应出自 Auer et al., 2002
选药游戏的结果	真实疗效（9 号最好）、救活人数、regret: greedy 越来越差， ϵ -greedy 早好晚差，后验采样最好	29:04	—
大 MDP 里探索靠什么	示范 / 基座模型做初始化，塑形奖励引导；从零探索不可行	33:42	—
学模型再探索的紫色轨迹	探索轨迹专门去访问能暴露奖励位置的状态；旁白提到墙上的画会把它带偏	38:49	—
解耦训练的两条线	探索策略 + 任务预测器一组，执行策略 + 任务标识一组，各自的目标	42:23	DREAM
meta-test 三步	探索 → 预测任务 → 执行；不需要动力学或奖励模型	43:23	DREAM
信息瓶颈板书	只加噪为什么会被"放大幅度"绕过；加上 L2 惩罚为什么等于 KL	49:00	应为 Alemi et al., 2017

图	看什么	跳转	出处
执行目标板书	编码器 f 与策略 π 一起最大化奖励、减 KL; z 是向量	55:11	DREAM
探索奖励板书	末端误差太稀疏 $\rightarrow$ 相邻两步误差之差, "每一步的信息增益"	1:02:57	DREAM
样本复杂度与实验曲线	随臂数增加 DREAM 与端到端的差距拉大; 读墙实验里端到端全部躺平	1:06:03	DREAM
Bounce 找 bug 与批改数据	探索策略像测试用例: 打进球门、落地、撞墙各看一次; 44% 更快、6% 更准	1:10:42	应为 Liu et al., 2022

提到的工作

名称	在本讲里的作用
Breakout / Montezuma's Revenge (Atari)	开场对比: 稠密得分 vs 稀疏事件, 探索为什么难
Mao (纸牌游戏)	比方: 规则未知、罚分不解释的试错处境
multi-armed bandit / contextual bandit	难度谱的左端; 本讲分析探索的载体
regret	衡量"探索加利用"总账的分析量, 不是训练目标
UCB (应出自 Auer et al., 2002)	乐观策略: 均值加不确定度 bonus, 可推 regret 保证
posterior sampling / Thompson sampling	采一个可能的世界再按它行动; 第 13 讲见过; 课堂游戏里最好
greedy / ϵ -greedy (第 6 讲)	游戏里的两个基线; ϵ 需要衰减
novelty / count-based bonus	讲者一句带过的"大 MDP 探索"研究路线, 态度悲观
示范与模仿学习 (第 2 讲)、预训练基座模型 (第 9、10 讲)	大 MDP 里探索的实际来源: 初始化把模型偏置到有奖励的区域
塑形奖励; 从偏好学出的单步奖励 (第 8 讲)	引导探索的稠密信号
黑箱 meta-RL / RL^2 (第 13 讲)	端到端学探索的代表; 鸡生蛋的耦合问题
学动力学 / 奖励模型的探索目标 (第 11、13 讲)	DREAM 建立其上的替代策略; 干扰物问题
多任务 RL (第 12 讲)	DREAM 执行线的训练方式
Deep Variational Information Bottleneck (Alemi et al., 2017; 推断)	压缩任务表示的工具: 加噪 + $L2 = KL$

名称	在本讲里的作用
Q-learning（第 6 讲）	DREAM 样本复杂度结论的假设之一
DREAM（Liu et al., ICML 2021）	本讲主角；作业 4
探索与利用合一的后续工作	问答里提到，未点名
Giving Feedback on Interactive Student Programs with Meta-Exploration（Liu et al., NeurIPS 2022；应为）	用 DREAM 给学生程序找 bug 的应用
code.org Bounce、CS106A Breakout	应用里的两个课程环境

术语对照

English	中文
exploration / exploitation	探索 / 利用：试没做过的事 vs 做已知最好的事
sparse reward	稀疏奖励：绝大多数时刻没有任何奖励信号
multi-armed bandit	多臂老虎机：单步、无状态、奖励随机的极简 RL
contextual bandit	上下文老虎机：每一步附带特征（状态）的 bandit
arm	臂：bandit 里的一个动作选项
horizon	时域：一个 episode 有多少步；bandit 里是 1
stochastic reward	随机奖励：同一动作每次拿到的奖励不一样
regret	后悔值：最优动作能拿到的总奖励减实际拿到的
sublinear regret	次线性 regret：曲线先陡后平，说明探索之后学会了利用
oracle	全知者：知道问题真相的假想参照
greedy / ϵ -greedy	贪心 / ϵ -贪心：只选当前最好 / 以概率 ϵ 随机
optimism in the face of uncertainty	面对不确定保持乐观：没试过的先假设它好
UCB (upper confidence bound)	置信上界：用均值加置信区间上沿排序动作
posterior sampling / Thompson sampling	后验采样 / 汤普森采样：从后验抽一个世界，按它选最优动作
posterior	后验分布：看过数据之后对未知参数的信念
POMDP	部分可观测 MDP：agent 看不到环境的全部状态
novelty bonus	新奇度奖励：给没见过的状态或动作额外加分
shaped / dense reward	塑形 / 稠密奖励：每一步都给信号，引导探索
demonstrations	示范：人或专家的成功轨迹，做 RL 的起点
base model	基座模型：预训练好的模型，为 RL 提供初始化

English	中文
meta-RL	元强化学习：在一族任务上训练，让新任务少量经验就能上手
meta-training / meta-testing	元训练 / 元测试：在训练任务上学；在新任务上探索、识别、执行
chicken-and-egg / coupling problem	鸡生蛋 / 耦合问题：探索与利用互相等对方先学会
decoupling	解耦：探索和执行各用各的目标
task identifier μ	任务标识：one-hot、文本或向量，训练时已知
latent task representation z	潜在任务表示：从 μ 压缩出的、只含任务相关信息的向量
exploration policy / execution policy	探索策略 / 执行策略
multi-task RL	多任务 RL：策略带任务描述符，在多个 MDP 上直接最大化奖励
distractor	干扰物：环境里与任务无关但会吸引探索的东西
(variational) information bottleneck	(变分) 信息瓶颈：加噪加惩罚，逼表示只留有用信息
KL divergence	KL 散度：两个分布差多远的量
unit Gaussian	单位高斯：均值 0、方差 1 的正态分布
L2 penalty	L2 惩罚：压小向量的幅度
prediction error	预测误差：预测的任务表示离真值多远
information gain	信息增益：一步之后对任务多知道了多少
reward-free exploration	无奖励探索：探索目标里不出现任务奖励
sample complexity	样本复杂度：学到目标行为需要多少样本

字幕勘误

"Mau / MAU" → Mao（纸牌游戏）；"Montzuma's / Montazuma's" → Montezuma's Revenge；"trade a reinforcement learning agent" → train；"weekly correlates" → weakly；"metalarning / metaarl" → meta-learning / meta-RL；"MVP" → MDP；"kale divergence" → KL divergence；"gaussian / gausion" → Gaussian；"keyarning" → Q-learning；"one dot vector" → one-hot vector；"mui" → μ_i ；"zbar" → $z^\top$ ；"sat" → s_t ；"exporting" → exploiting；"huristic" → heuristic；"a squ log a" → $A^2 \log A$ ；"dream" → DREAM；"intromputer science" → intro computer science；"TAS" → TAs。

带走的问题

1. regret 要知道最优动作的期望奖励才能算，所以不能当训练目标。那么 UCB 的 bonus 和后验采样各自把"让 regret 拐平"这件事编码进了哪个量？把 ϵ 衰减的 ϵ -greedy 放进同一个框架，它缺了什么？
2. 讲者说大 MDP 里"从零探索不可行，靠基座模型和示范"。这和 CS329A 里 GRPO 训练需要采样多样性、DAPO 用 clip-higher 防止熵塌缩，是同一个探索问题的两面吗？基座模型给的是先验，那 RL 训练过程中的探索还剩什么要做？
3. DREAM 把探索奖励定义为对 z 的信息增益。如果任务标识 μ 本身有噪声（两个学生程序的 bug 相同却被标成不同任务），瓶颈会怎么处理？如果 z 学得不好、保留了墙的颜色，探索策略会怎么"作弊"？
4. DREAM 的前提是训练时知道任务标识。你做的 agent 产品里，"任务"是什么，有没有现成的标识（用户、会话类型、prompt 模板）？如果没有，第 13 讲的 PEARL 式后验推断和 DREAM 各要付出什么？
5. 先探索后利用 vs 交错探索：LLM 的推理（试一条思路 → 验证 → 换思路）更像哪一种？把"信息增益"当作过程奖励（CS329A 的 PRM）放进推理任务里，会奖励什么样的步骤？

CS224R 第 15 讲 | 分层 RL 与 IL (Hierarchical RL and IL)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 15 讲, 2025 年 5 月 21 日 · 承接第 12 讲的多任务与目标条件策略、第 13 讲的元 RL, 把"一个策略会做多件事"推进到"把多件事串成一件长事"; 下一讲进入机器人 RL 视频: <https://www.youtube.com/watch?v=iKWYLSVAtfM> (1:09:32, 英文字幕是自动生成的, hierarchical、Dagger、relabeling、Yosemite 这类词几乎都错, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座讲座) 课程主页: <https://cs224r.stanford.edu/> · 指定阅读: Do As I Can, Not As I Say: Grounding Language in Robotic Affordances (SayCan, Ahn et al., 2022; 课上没有点名, 只在 38:10 用"机器人站在咖啡机前"的例子讲了它要解决的问题, 第 6 节按论文补上机制)

一句话: 长程任务 (做一顿饭、修一个让训练 loss 爆炸的 bug) 难在状态多、容易出错、容易原地卡死。分层把一个策略拆成两层: 高层读状态、输出一个子目标 (一句话、一个坐标、一张图), 低层读状态和子目标、输出动作; 低层跑得快, 高层隔一阵才想一次。好处不在"多了一个网络", 而在中间监督、跨子任务共享、按目标空间探索和低频推理——端到端训一个隐目标的两层网络, 和平的策略没有区别。三个设计选择贯穿全讲: 子目标怎么表示 (够表达、有结构、层级合适); 每层用什么监督 (低层为子目标、高层为总任务, 且低层必须在高层真会下令的状态上可用, 否则技能之间有"缝"); 什么时候换目标 (多数系统每 N 步问一次高层, 因为把"完成"判错会卡死)。例子里分层模仿占大头: 语言子目标配语言空间的 Dagger, 图像子目标配图像编辑扩散模型; 分层 RL 用目标状态或语言当高层动作, 配 hindsight relabeling; 指定阅读 SayCan 是"LLM 当高层"的样板: LLM 说该做什么, 价值函数说做不做得成, 相乘选技能。

时间轴

时间	内容
0:05	回顾多任务与元 RL; 今天: 把多个子任务的行为串成长程任务
1:35	什么叫长程任务; 为什么难: 状态多、会出错、会卡死
4:10	分层的主意: 高层出目标、低层出动作; 频率不同; 芝士蛋糕的逐级分解
10:20	分层策略怎么 rollout: 计划一个目标 → 反复出动作 → 重新计划
12:24	分层为什么有用: 中间监督、跨子任务共享、结构化探索、省算力

时间	内容
16:30	中间路线：一个策略先"想"子目标再出动作，像 chain of thought
19:37	设计选择一：目标怎么表示——四道练习题与三条标准
25:48	设计选择二：每层的监督——煮意面的例子；端到端等于平的策略
30:59	技能之间的缝：低层要在高层会下令的状态上可用；LLM 当高层与 affordance
38:42	设计选择三：什么时候换目标——完成判定 vs 固定步数
44:54	近几个月的四个分层机器人系统；chain of thought 也是分层
46:28	分层模仿 · 语言子目标：切段标注、两级架构、语言空间的 DAgger
54:10	分层 vs 平策略的对比数字；更长的任务：装袋、收拾卧室
55:45	分层模仿 · 图像子目标：图像编辑扩散模型生成子目标图，人类视频也能用
59:18	chain-of-thought 版本；大规模系统仍靠模仿；问答：草图、错误的语言
1:05:03	分层 RL：目标状态与语言目标、hindsight relabeling；无监督技能发现；下一讲预告

核心内容

1. 从"会做多件事"到"把多件事串起来"：长程任务为什么难

- **前情：**第 12 讲把任务描述并进策略输入，一个 $\pi(a \mid s, z)$ 做多个任务；第 13 讲的 meta-RL 要从几步经验里推断任务。今天要把多个短任务的行为串成一个长程任务。
- **什么叫长程任务 (long-horizon task)：**要做很久、由若干子任务按某种顺序组成的任务。讲者的四个例子横跨机器人和 LLM：机器人做一顿饭；LLM agent 修一个让训练 loss 爆炸的 bug；自动驾驶开去 Yosemite；LLM 评审一份长报告。步骤之间有的相互依赖，有的可以并行。
- **为什么难，三条：**
 1. 时间一长，路上要经过的状态就多，每个新状态都得会决策——策略要在宽得多的状态分布上都好用。
 2. 出错的机会多，出了错还得会恢复。
 3. 更阴险的是**卡死**：某种错误让你没有进展、回到同一个状态，确定性的策略就会在那里反复犯同一个错，永远不往前走。

2. 分层的主意：高层出目标，低层出动作

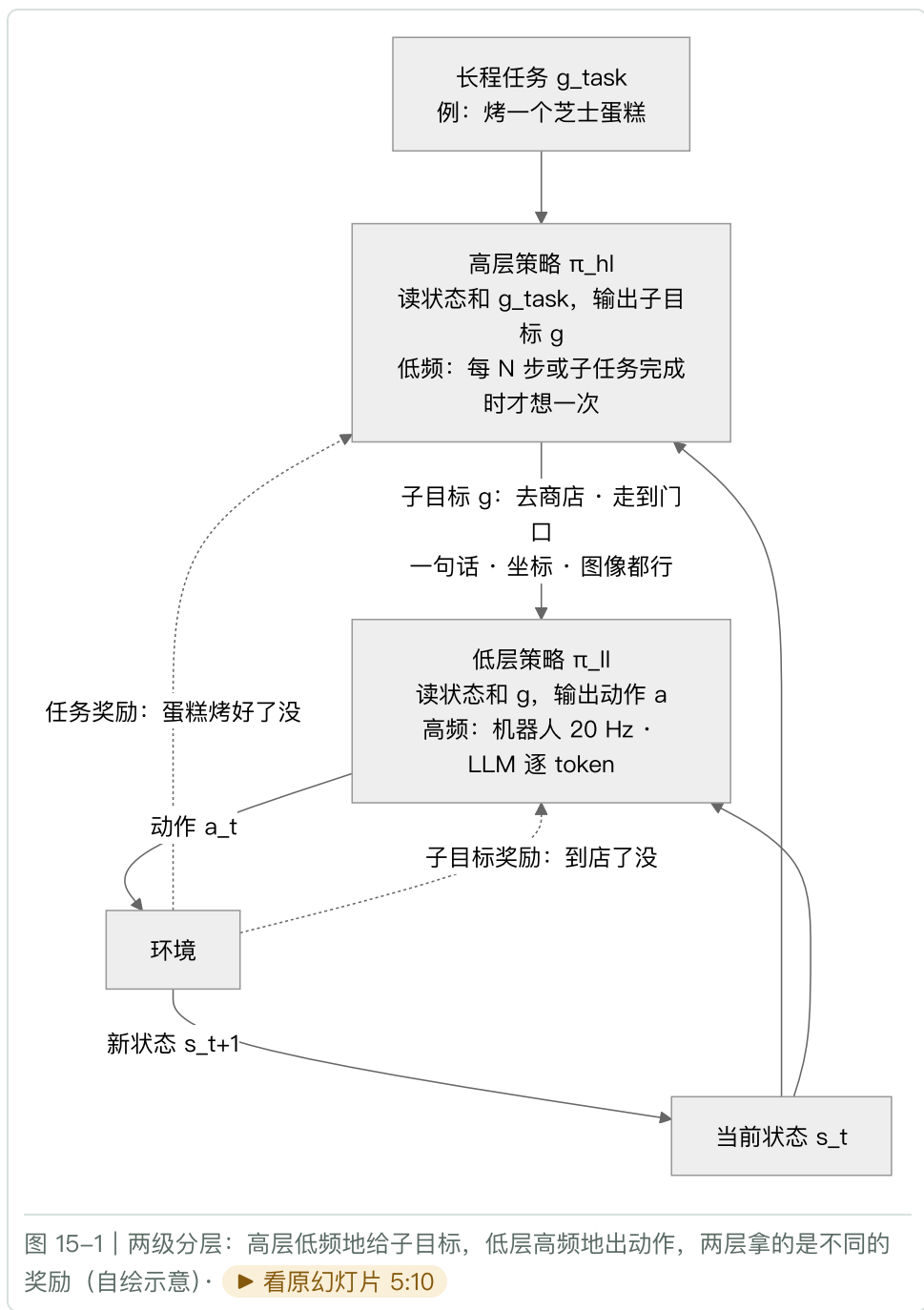


图 15-1 | 两级分层：高层低频地给子目标，低层高频地出动作，两层拿的是不同的奖励（自绘示意）· [▶ 看原幻灯片 5:10](#)

- **两层策略。**多任务策略 $\pi(a | s, g)$ 已经能按目标 g 做事；那就再放一个策略，专门负责给它下目标：
 - **低层策略** $\pi_{ll}(a | s, g)$ ：读当前状态和一个子目标 g ，输出动作。做的是短任务。
 - **高层策略** $\pi_{hl}(g | s, g_task)$ ：读当前状态（它也得知道自己在哪），可选地再读整个长程任务的描述 g_task （想同时会开去 Yosemite、大峡谷、西雅图，就得有这一项），输出交给低层的子目标 g 。

$$g_t \sim \pi_{hl}(g | s_t, g_{task}), \quad a_t \sim \pi_{ll}(a | s_t, g_t)$$

s_t 是当前状态，g_task 是整个长程任务的描述，g_t 是高层给低层的子目标，a_t 是真正发给环境的动作。高层的"动作"就是子目标，所以子目标也叫高层动作（high-level action）、技能（skill）、option 或子任务（subtask），本讲混用。

- **为什么拆得开**：直接训一个策略输出电机指令去"烤芝士蛋糕"几乎无从下手；但可以一层层问"第一步是什么"：买原料 → 去商店 → 走到门口 → 迈一步 → 收缩一块肌肉。越往下越小、越好训，小任务训好了再用高层**组合**。层数也可以不止两层：一层管"去商店、买东西、拿回来"，更低的一层管电机。
- **频率不同**。低层按环境的原始节拍走：机器人大概 20 Hz，LLM 逐 token；高层粗得多——写完一段、到了商店，才给下一个目标。
- **怎么 rollout**：先向高层要一个目标（讲者说这一步也可以叫**规划**：往前想下一步该做什么）；然后反复从低层采样动作、执行、观察新状态，目标不变；到某个时刻——子目标完成了，或时间到了，第 7 节专门讨论——回到第一步，按最新状态重新要目标。

3. 分层为什么有用，以及"一个策略先想再做"的中间路线

- **四条理由**：
 1. **中间监督**。低层为"达成子目标"训，高层才为"完成长程任务"训——两层目标函数不同，低层拿到的是关于中间步骤的监督信号，引导整个系统往终点走：高层在蛋糕烤好时拿奖励，低层在找到原料、预热好烤箱时拿奖励。
 2. **跨子任务共享**。平的策略（flat policy：不分层，直接从任务和状态出动作）也能靠 shaped reward 拿到中间监督；分层多出来的是低层以子目标为输入，相似的子任务会共享数据、学得更省。
 3. **结构化探索**（RL 专属）。高层提出的目标把探索组织在目标空间里，而不只是在原始动作空间里瞎撞——第 14 讲的探索问题在这里换了一个空间。
 4. **省算力**。做蛋糕要一步步推理，平的策略每个时间步（20 Hz）都得重新推理一遍；分层把推理放在低频的高层。
- **问答**：中间监督为什么帮得上？RL 里只在蛋糕烤好时给奖励太稀疏，没法取得进展；模仿学习里也有好处——策略若不明确"我在做哪一步"（尤其没有历史输入时），会在几个子任务之间来回摇摆。要并行做几件事怎么办？除非低层有十只手，否则总得排顺序；顺序重不重要交给高层学。

- **中间路线**：真的需要两个策略吗？还有一种做法很像 LLM 的 chain of thought：一个策略先推理出该做哪个子目标，再输出低层动作——两层合在一个模型里、按顺序预测。它保留了中间监督的好处，但推理仍要在全频率上跑，没有省算力这条；和真正分层相比孰优孰劣，讲者说实证上还没有结论。

小注：第 10 讲的 RL for LLM reasoning 和 CS329A 里的推理模型，走的就是这条"先想再答"的路——那里的思维链就是写在输出里的子目标序列。

4. 设计选择一：子目标怎么表示

- 讲者先让大家做练习：四个问题，各自的中间步骤该是什么、怎么表示？① 学做意大利菜（比如 focaccia）的机器人；② 校园里自动骑行的车，要把导航拆成去各个地点；③ 起草法律文书的 LLM；④ 替用户规划并预订一周假期的 web agent。
- **课堂上的答案**：
 - 骑车：中间目标是通往终点的路标点 (waypoints)，用 GPS 坐标表示，低层奖励就是离坐标有多近。
 - 订假期：按天拆（订第一天、第二天……），或更粗（订交通、酒店、活动）；讲者补了一条——先弄清用户想要什么样的假期，这个"高层调研"本身也是一个中间目标。
 - 做菜："打蛋""开柜子"这类自然语言字符串；想让面包呈某种样子，也可以给一张图当目标——多模态的目标表示。
- **三条标准**：
 1. **够表达**：要做很多种菜，自然语言这种能描述无数低层行为的表示就合适。
 2. **有结构**：目标若只是"子任务 1""子任务 2"这样的编号，低层看不出它们的关系；GPS 坐标天然有结构——相近的坐标对应相近的动作，低层学起来容易得多。
 3. **抽象层级合适**：骑车的目标太低（"下一秒的精确速度"）等于没拆，太高（"去 quad"）也没把问题拆小。合适的层级常依赖领域。
- **问答·状态也要按层抽象吗**：实践中高层通常也读完整状态，损失不大；真想抽象，可以让 LLM 先把状态总结成高层信息。

5. 设计选择二：每层的监督，以及技能之间的"缝"

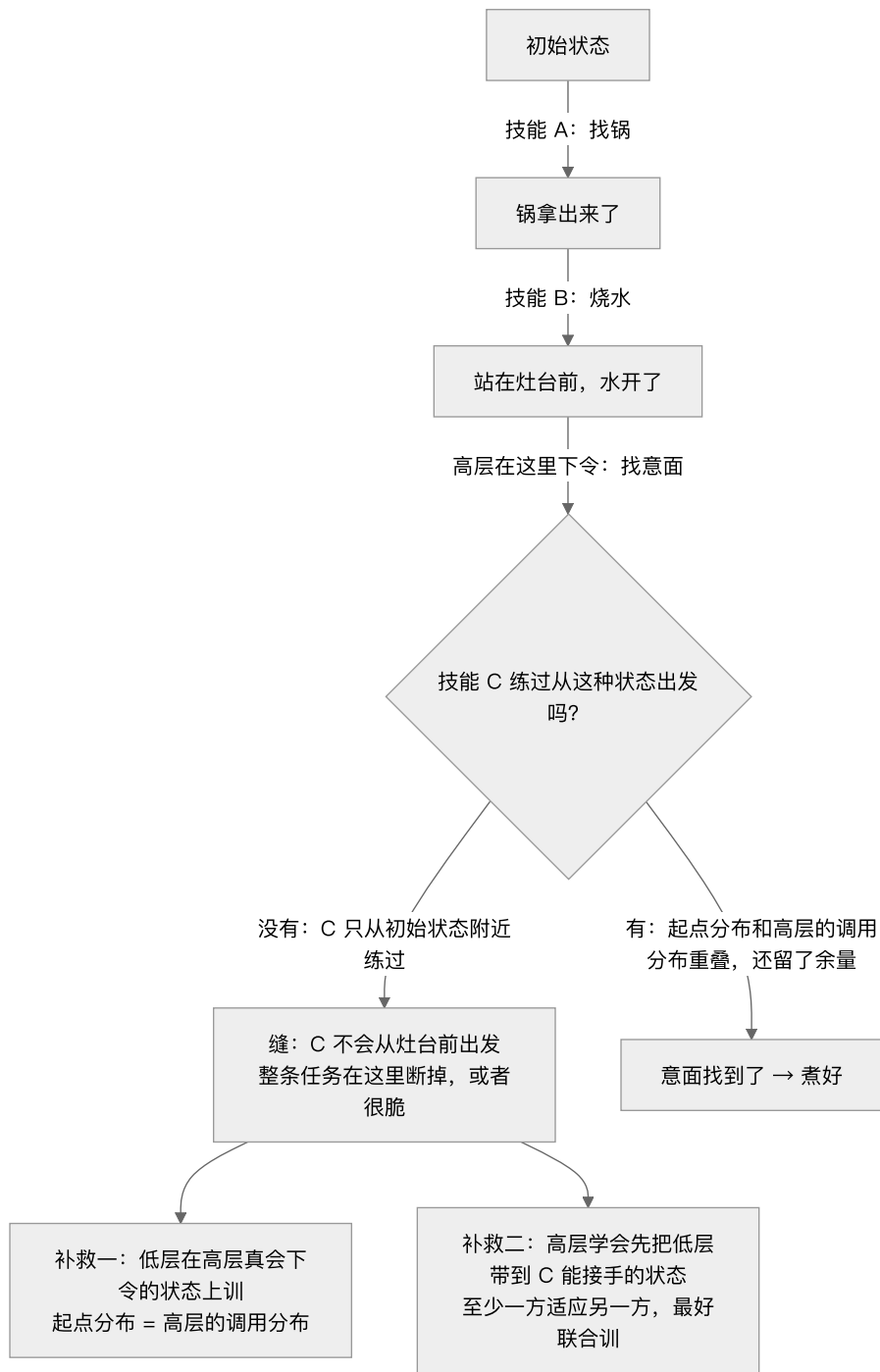


图 15-2 | 煮意面的"缝": 各训各的技能在状态空间里接不上, 长程任务就断在交接处 (自绘示意) · [▶ 看原幻灯片 34:01](#)

- **煮意面的例子:** 状态空间里可以想象几团区域——初始状态、"锅拿出来了"、"水烧开了"、"意面找到了"、"意面煮好了"。低层学"怎么行动才能到达这些区域", 高层学"在某团区域里该发哪条命令"。

$$J_{\text{ll}}(\pi_{\text{ll}}; g) = \mathbb{E} \left[\sum_t r_g(s_t, a_t) \right], \quad J_{\text{hl}}(\pi_{\text{hl}}) = \mathbb{E}_{g \sim \pi_{\text{hl}}, a \sim \pi_{\text{ll}}} \left[\sum_t r_{\text{task}}(s_t, a_t) \right]$$

r_g 是"离子目标 g 有多近"或"到没到"的子目标奖励, r_{task} 是只在意面煮好时才给的任务奖励; 低层按前者训 (找锅时只看有没有找到锅), 高层按后者训——发出一串子目标让系统最终到达终点。

- **端到端不等于分层**: 也可以不要显式的子目标表示, 让高层输出一个隐向量 (latent goal, 就是网络中间的激活), 两层只用任务奖励端到端训。讲者提醒: 这其实就是一个**平的策略换了种架构**——观察喂了两次的一个大网络。设计这类系统时要想清楚好处从哪来 (额外监督? 结构化探索?), 否则"我希望高层做这个、低层做那个"只是想象, 最后和平策略比不出差别。
- **分开训会留下"缝"** (图 15-2)。**"找锅"**已经训得很稳; 再训**"找意面"**时若不小心, 会从初始状态附近出发去训。可高层想要的顺序是**"先烧水, 再找意面"**, 那时机器人站在灶台前——低层从没在这种状态下练过**"找意面"**, 串起来就断在这里。模块化训练的通病: 各训各的, 从不接在一起, 最后拼不起来。

$$\mathbb{E}_{s \sim d^{\pi_{hl}}(s), g \sim \pi_{hl}(g|s)} \left[\sum_t r_g(s_t, a_t) \right] \quad \text{vs.} \quad \mathbb{E}_{s \sim p_0(s), g \sim p(g)} \left[\sum_t r_g(s_t, a_t) \right]$$

$d^{\{\pi_{hl}\}}(s)$ 是高层真正会在哪些状态下令的分布, $\pi_{hl}(g | s)$ 是它在那里会下哪些令; 低层该在左边这个分布上好用, 而不是右边**"初始状态 p_0 加随便抽一个目标"**。低层不必对每个状态都行, 只要对高层会命令的那些 (s, g) 组合行就够了。

- **反过来也一样**: 高层若假设低层完美, 就发现不了低层的短板, 也学不会**"在某些状态要一直命令同一个目标, 直到下一个技能能接手"**。两层都没训好时, 这是鸡生蛋的问题。
- **重叠要留余量**: 技能 A 结束的状态集合和技能 B 能起步的状态集合至少要有交集, 最好有缓冲——如果**"找意面"**只在正对着开水锅时才行, 站偏一点就不行, 整条任务就会脆。
- **问答·顺序必须固定吗**: 不必。高层可以自己决定顺序, 既看效率也看低层在哪儿更可靠; 烧水和找意面本来就可以换序, 甚至趁烧水的空档去找意面。
- **怎么补**: 可以先分开训, 但至少要让一方适应另一方, 最好两层为总目标联合调。
- **LLM 当高层**: 即使不是传统的 LLM 问题, LLM 也很擅长这种高层推理——问它**"煮意面分几步"**往往答得出来。所以可以把高层独立拿来用, 再针对低层微调: LLM 不知道低层最擅长哪个顺序、有什么能力。讲者的例子: 机器人站在咖啡机前, 你说**"去弄杯咖啡"**, 不看图像

的 LLM 可能让它出门去星巴克买——机器人根本没有这种 **affordance**（可供性：在当前状态下这个动作做不做得成）。这正是指定阅读 SayCan 要解决的问题。

小注：CS329A 第 5 讲讲的 LLM 规划——先列出步骤再逐步执行——就是这里的高层策略；本讲补上另一半：规划器必须适应执行器的能力边界，否则计划再合理也落不了地。

6. 指定阅读补充：SayCan——LLM 说"该做什么"，价值函数说"做不做得成"

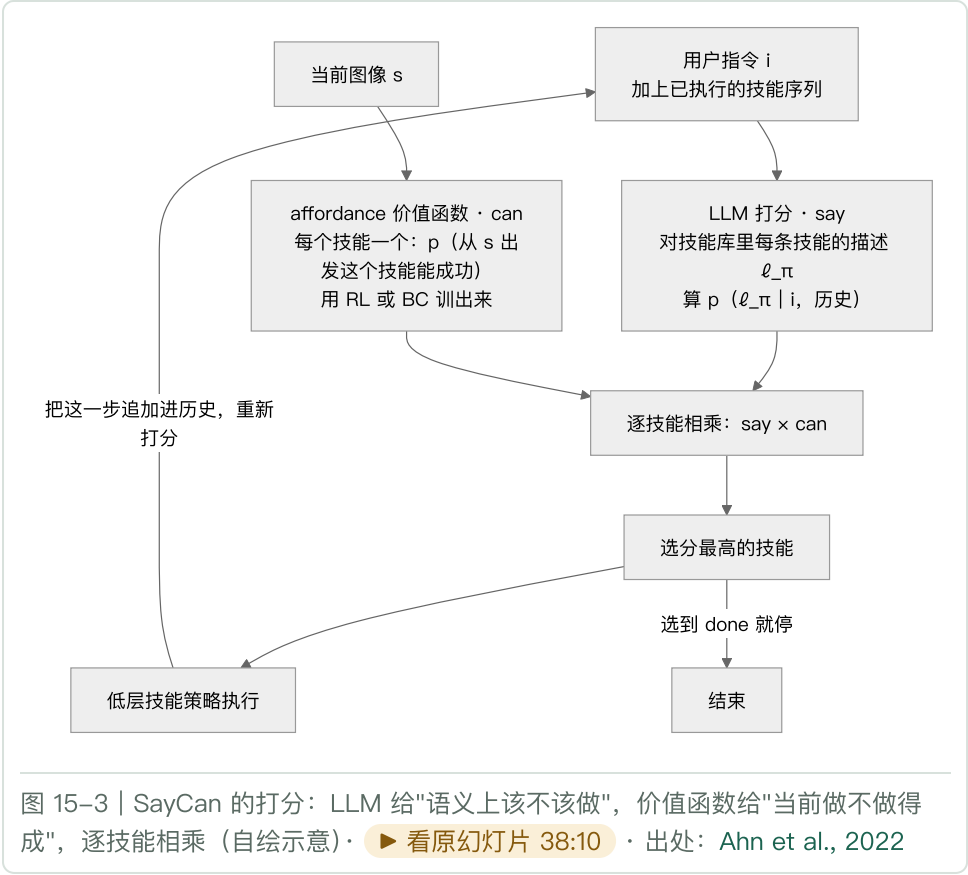


图 15-3 | SayCan 的打分：LLM 给"语义上该不该做"，价值函数给"当前做不做得成"，逐技能相乘（自绘示意） · ▶ 看原幻灯片 38:10 · 出处：Ahn et al., 2022

小注：这一节的机制来自指定阅读（Ahn et al., 2022），课上没有展开，只在 38:10 用咖啡机的例子讲了动机。放在这里是因为它是"LLM 当高层、技能库当低层"最直接的样板，也是读者做 LLM 规划加工具执行时最该对照的一篇。

- **设定**：机器人有一个技能库 Π ，每个技能 π 有一句语言描述 ℓ_π （"拿一罐可乐""走到桌子边"），由 RL 或行为克隆训出来；每个技能还配一个价值函数 $p(c_\pi | s, \ell_\pi)$ ——在当前状态 s 下这个技能能成功的概率（第 12 讲的目标条件 Q 函数： Q 高说明做得成）。
- **打分**：对技能库里的每个技能，LLM 算"在指令 i 和已执行步骤之后，下一句是 ℓ_π 的概率"（say）；价值函数算"从 s 出发 ℓ_π 能成"的概率

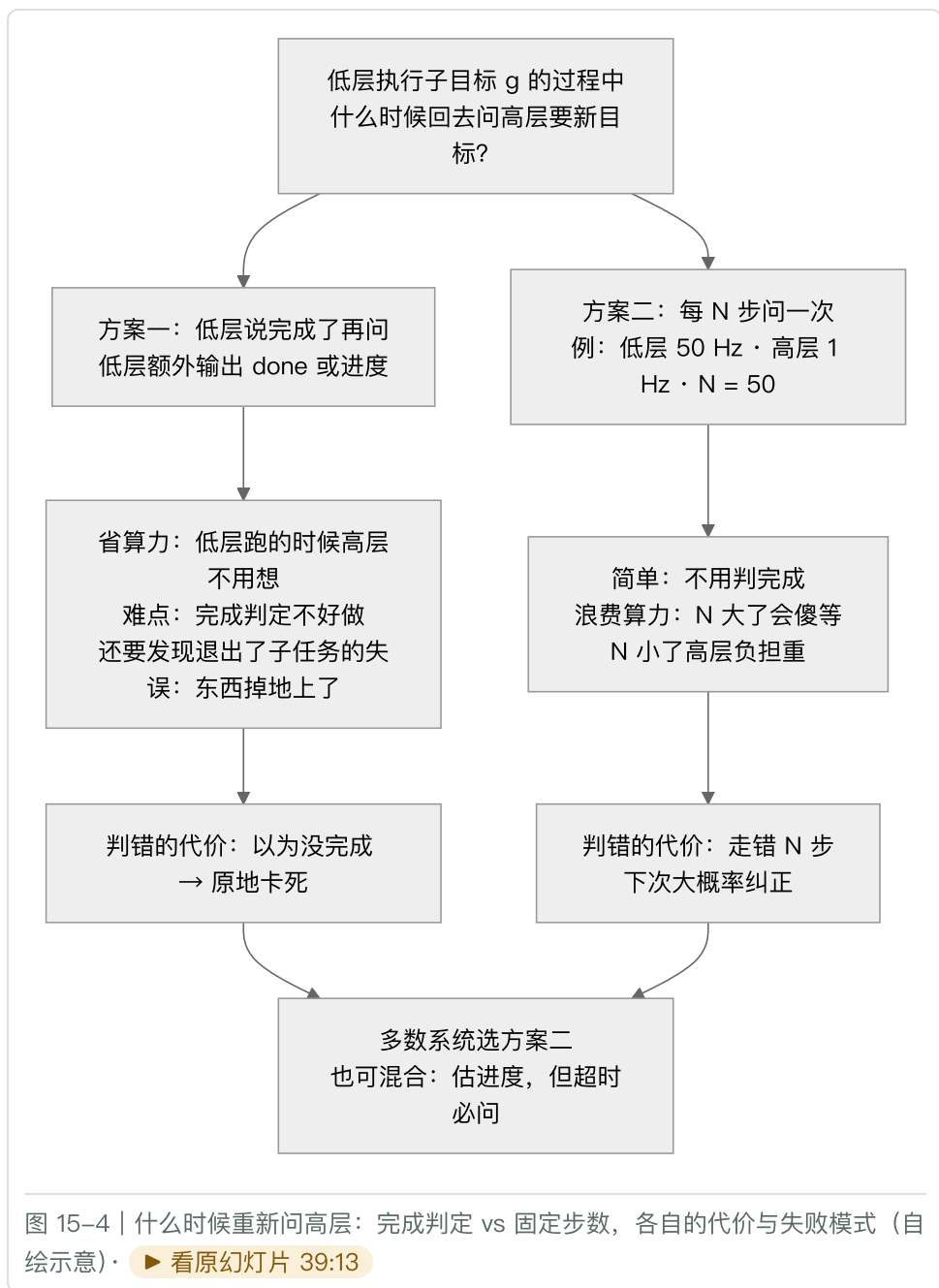
率 (can)；相乘，选最大的执行；把执行过的技能追加进 prompt，重新打分，直到选到 done。

$$\pi^* = \arg \max_{\pi \in \Pi} \underbrace{p_{\text{LLM}}(\ell_\pi \mid i, \ell_{\pi_1}, \dots, \ell_{\pi_{k-1}})}_{\text{say}} \cdot \underbrace{p(c_\pi \mid s, \ell_\pi)}_{\text{can}}$$

i 是用户指令， $\ell_{\{\pi_1\}}$ 到 $\ell_{\{\pi_{k-1}\}}$ 是已执行技能的描述，第一项是 LLM 对"接下来做 ℓ_π "的语言似然，第二项是从状态 s 出发该技能能成功的概率；相乘意味着"语义上合理但当前做不成"和"做得成但答非所问"的技能都会被压下去。

- **对回第 5 节：**SayCan 用"LLM 固定不动、价值函数做适配"回答了"高层怎么适应底层能力"——LLM 不看图像也没关系，affordance 那一项替它看了。代价是 LLM 只能在预先定义好的技能集合里挑；技能之间的"缝"仍要靠价值函数如实报告"从这里做不成"来避开。对照你的 agent：LLM 规划器加工具就是 say；工具在当前上下文里能不能调、前置条件满不满足，就是 can——多数 LLM agent 只做了前半。

7. 设计选择三：什么时候换目标



- **问题：**rollout 里"反复采样低层动作"要重复多久，也就是多久回去向高层要一次新目标？两个自然的方案：
 1. **低层说完成了再问：**水烧开了，才让高层说"去找意面"。这意味着低层除了出动作，还要输出"完成了没有"——一个二元或进度指示，可以看成这个子任务的成功分类器。
 2. **每 N 步问一次：**低层 50 Hz、高层 1 Hz，那就是每 50 步。简单，但没那么优雅。

$$g_t = \begin{cases} \tilde{g} \sim \pi_{\text{hl}}(\cdot \mid s_t, g_{\text{task}}) & t \bmod N = 0 \\ g_{t-1} & \text{otherwise} \end{cases}$$

方案二写成式子就是这样：每隔 N 个低层步重新采样一次子目标，其余时刻沿用上一步的； N 越小，高层被问得越勤。

- **方案一的账：**做得好最省算力——低层执行时高层不用重新思考。但两件事难：一是"完成没有"本身不好估；二是要处理**需要重做旧目标的失误**——收拾桌子时手一滑东西掉地上了，子目标就不该再是"放桌上"而该是"从地上捡起来"，这要求你还要判断"我是不是已经离开了这个子任务说得通的状态"。
- **方案二的账：**会浪费算力， N 上有取舍—— N 太大，机器人拿出锅之后就干等，直到下一次询问才去找意面； N 太小，高层要在更多状态上更频繁地准确出目标，负担和算力都上去。实践中一般取得比较小。
- **看错了各有什么后果：**方案一把"已完成"判成"未完成"，低层就一直待在那个状态，**永远卡死**——这种错再少，结局也很糟；方案二高层偶尔给错子目标，最多走错 N 步，下一次询问大概率纠正回来。所以现有的分层系统大多选方案二；也可以混合：估进度，但太久没换也强制重新询问。

小注：CME295 第 7 讲的 agent loop——模型每一步都重新决定下一个工具调用——就是方案二取 $N = 1$ 的极端：最不会傻等，也最费推理；本讲给了它一个可调的旋钮。

8. 分层模仿学习 · 语言子目标：切段标注，加语言空间的 DAgger

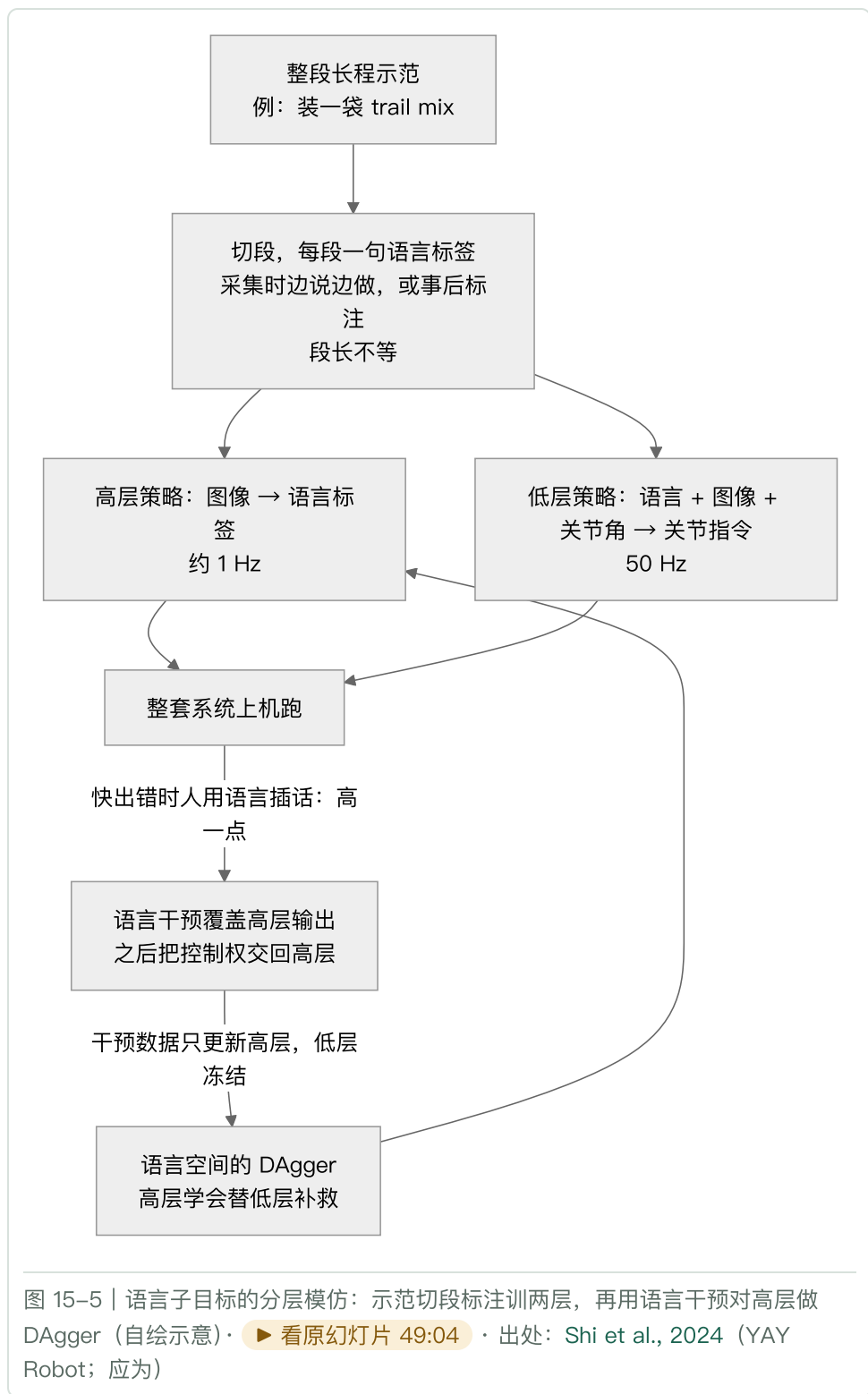


图 15-5 | 语言子目标的分层模仿：示范切段标注训两层，再用语言干预对高层做 DAgger（自绘示意）· [▶ 看原幻灯片 49:04](#) · 出处：Shi et al., 2024 (YAY Robot; 应为)

- **分层正在流行**：讲者放了近四五个月里发布的四个机器人系统——收拾卧室、递交物体并倒进箱子、把东西收进冰箱、把水果放进碗——都有一个出中间目标的高层模型和一个出动作的低层模型；LLM 那边的 chain of thought 同样如此。

小注：课上只放了图。“收拾卧室”应为 $\pi 0.5$ (Physical Intelligence, 2025 年 4 月)，其余应为同期 Helix、GR00T N1 这类“高层 VLM 加低层

- **数据长什么样**：真实机器人，任务是做 trail mix 并装袋。收整段长程示范，再切段，每段配一句语言标签："拿起袋子""拿起金属勺""舀 M&M's"。标签可以采集时边说边做，也可以事后用标注界面补。段长不等，但部署时高层按固定频率重新询问，比数据里的典型段长更密。
- **两级架构**：高层读 RGB 图像，预测语言标签，约 1 Hz；低层读语言标签、RGB 图像和关节角，输出关节指令，50 Hz；两层的观测不必一样。两层都用模仿学习训：

$$\mathcal{L}_{\text{hl}} = - \sum_t \log \pi_{\text{hl}}(\ell_{k(t)} | o_t), \quad \mathcal{L}_{\text{ll}} = - \sum_t \log \pi_{\text{ll}}(a_t | o_t, \ell_{k(t)})$$

o_t 是第 t 步的观测， a_t 是示范里的动作， $\ell_{\{k(t)\}}$ 是第 t 步所在那一段的语言标签；两个都是行为克隆（第 2 讲）的负对数似然，只是高层的"动作"是一句话，低层多了一句话当输入。

- **语言空间的 DAgger**：只在离线数据上做模仿，两层仍是各训各的。第 2 讲的 DAgger 可以用在两层上；有意思的是高层的 DAgger 发生在语言空间而不是关节空间：整套系统上机跑，人看到快出错时打字或口述一句命令覆盖高层输出，之后把控制权交回；这些带语言干预的数据只拿来微调高层，低层冻结。因为干预是在两层合成一个系统时收的，高层就能学会补偿低层的短板——某条命令低层老做不好，高层就学会先说一句别的。
- **视频**：黄字是高层输出："拿起勺子"→"我要花生"→"把勺移进袋子"，此时低层快失误了，人插话"左臂向左""高一点"再"移进袋子"；交回控制后高层继续"倒进袋子"。微调之后，高层在同样的情形下自己会先说"高一点"。
- **数字**（来自另一篇做法基本相同的论文）：分层 vs 单个平策略，task progress 提升 34%，instruction accuracy 也有差距；高层用 DAgger vs 普通模仿也有提升——这就是"两层互相适应"带来的。
- **能做更长的任务**：把三样东西装进一个 ziploc 袋（一口气做完传统上很难）；类似的系统给一句"收拾卧室"，系统会一路给出"拿起衬衫""放进篮子""拿起枕头""整理毯子""捡纸巾扔垃圾桶".....平的策略很难做到。

小注：切段标注、语言干预、trail mix 那套应为 YAY Robot (Shi et al., 2024, Finn 组)；34% 的对比应出自 Hi Robot (Shi et al., 2025) ——

高层是一个 VLM，任务有做三明治（课上"拿起一片奶酪、放到面包上"的切段例子应出自这里）、收拾餐桌、买菜。

9. 分层模仿学习 · 图像子目标：让扩散模型"画"出下一步

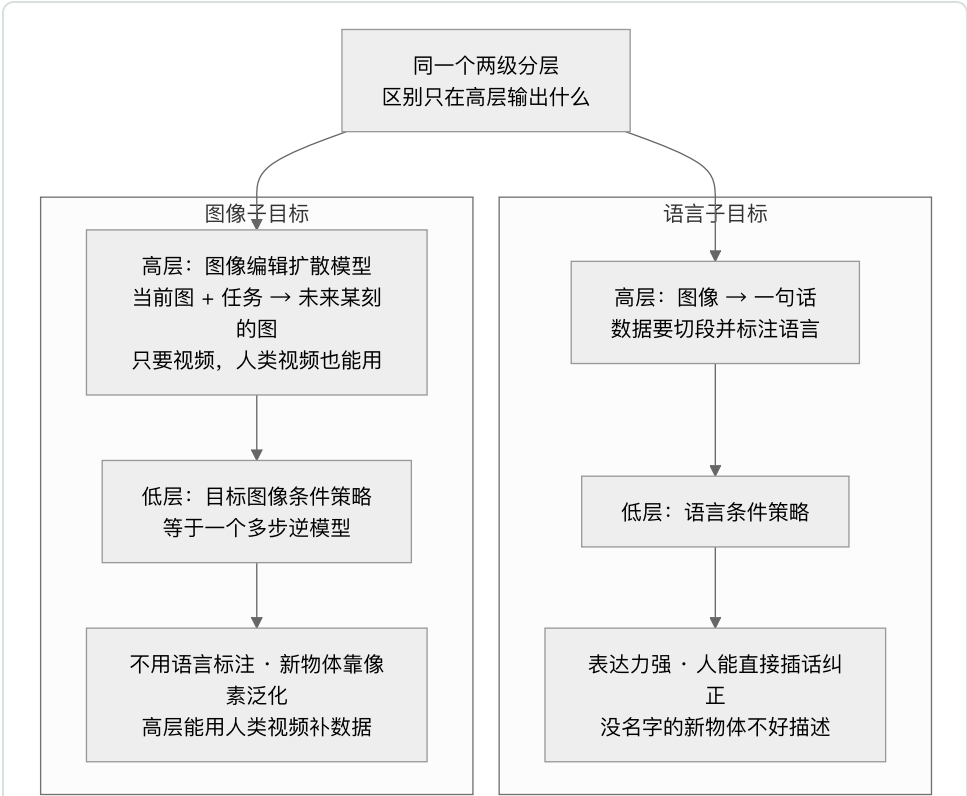


图 15-6 | 语言子目标 vs 图像子目标：同一个分层骨架，换掉高层的输出类型，数据要求和长处随之改变（自绘示意） · [▶ 看原幻灯片 56:15](#) · 出处：Black et al., 2023 (SuSIE；应为)

- **换一种子目标：**高层不出语言，出一张**目标图像**——"我要你未来达到这个样子"。实践上用图像编辑模型：读当前图像和高层任务，把它"编辑"成未来某个时刻应有的样子（用的是扩散模型，视频里能看到噪声逐渐变成图像）。低层变成目标图像条件策略，可以看成**一个多步逆模型**：读当前图和未来图，输出把前者变成后者的动作。两层仍用模仿学习训。

$$\hat{o}_{t+k} = f_{\theta}(o_t, g_{\text{task}}), \quad a_t \sim \pi_{\Pi}(a \mid o_t, \hat{o}_{t+k})$$

f_{θ} 是图像编辑模型（高层），输入当前观测 o_t 和任务描述，输出 k 步之后的预测图像 $\hat{o}_{t+k}$ ；低层读当前图和这张子目标图出动作。问答里讲者给了尺度感：整个任务可能一分钟量级，子目标图大约十秒之后。

- **数据要求变了：**不再需要按语言切段，只要有长程任务的**视频**。高层可以在机器人轨迹上训（预测 1 秒、5 秒、10 秒后的帧），也可以用人

类视频训"什么样的未来画面算朝目标前进"——机器人数据稀缺时尤其有用。问答里补充：图像编辑模型的通用预训练教会它"怎么生成图"；实际用的人类视频每段只有一句整体标注；随机 YouTube 视频难用一些，但也能教"未来可能出现什么画面"。

- **例子：**把牙膏放进木碗——先生成"牙膏举到碗上方"的子目标图（粉色边框），到达后再生成"放低手臂"；开抽屉——先"夹爪搭上把手"，再"向右移"。
- **好处：**遇到没见过、用语言说不清的物体时，图像比语言好用——不需要知道那个东西叫什么，也能生成像素并泛化。
- **对比：**高层只用机器人数据 vs 加上人类视频，后者性能更高。
- **chain-of-thought 版本：**也有把两层合成一个模型的做法——读任务和当前观测，先输出子目标图或子任务（也可以是 bounding box、路标点），再输出动作，都是最近一年的工作。

小注：图像子目标这套应为 SuSIE (Black et al., 2023, Finn 与 Levine 组合作)：在 InstructPix2Pix 上用机器人数据和 Something-Something 人类视频微调。chain-of-thought 版本应指 ECoT (2024)、CoT-VLA (2025) 这类 VLA (vision-language-action 模型：用视觉语言模型直接输出机器人动作) ——推断，课上没点名。

- **现状与问答：**这些大规模系统迄今主要靠模仿学习；怎么在大规模上用 RL 微调它们是开放问题，也很重要——在没见过的物体上，成功率还远谈不上可靠。问答：推荐系统用不用得上分层？讲者没见过，分层通常用在更长、更复杂的任务上。子目标能不能是草图？可以，训一个吃草图的低层就行；语言最流行，轨迹、草图都是可选表示。高层给了错误的语言怎么办？低层通常照错的做，直到被纠正；不过有时低层会忽略高层——给它乱码，它会按那个状态下"通常接下来该做的事"走，比如手里端着意面，下一步多半就是倒。

10. 分层 RL：目标状态和语言当高层动作

- **讲者的态度：**分层模仿的系统很新、例子多；分层 RL 让她兴奋的例子不多，但"在分层模仿的初始化之上用 RL 继续训"值得探索。已有做法主要是让低层去**到达状态**。
- **目标状态：**作业一里的四足 ant——要到达远处，得先把方块推到右边再走过去；另一个场景直接走会掉进沟里，得先推方块搭桥。贪心地走到不了，是标准的长程问题。做法：低层是目标条件策略，奖励是"到达目标状态"，目标是相对位置；高层输出目标状态，这就是它的

高层动作。两个关键：高层也做 **hindsight relabeling**（第 12 讲：事后把没达成的目标换成实际到过的状态），重标高层该输出的目标；并且用 **off-policy** 算法训（第 5—6 讲：能用旧数据），否则效率不够。

小注：应为 HIRO (Nachum et al., 2018)。它的 relabeling 有个特殊理由：低层一直在变，旧数据里高层当时发的目标 g 放到现在的低层上，未必再产生同样的动作序列；于是把 g 换成“最能解释当时低层动作”的目标，这样 off-policy 的高层数据才仍然有效，论文叫 off-policy correction。

- **语言目标**：低层是语言条件策略，用语言空间的 hindsight relabeling（做到了什么就把指令改成什么）；高层输出语言——这本身相当难。2019 年的工作，任务简单：按指令推绿、红、蓝方块摆成某种排列。

小注：应为 Jiang et al., 2019, Language as an Abstraction for Hierarchical Deep RL；指令重标叫 hindsight instruction relabeling。

11. 无监督技能发现，与下一讲

- 前面都是**监督**低层去达成给定的目标、高层去输出这些目标。另一个方向：不给监督，让系统自己**发现**有用的技能。例子是作业里的 half cheetah，自动学出倒退、前进、前空翻；原则上发现了技能，就可以串起来做更难的任务。但这仍是活跃的研究方向，在很大的状态空间里很难放大。

小注：应为 DIAYN (Eysenbach et al., 2018)：给每个技能一个编号 z ，训一个判别器从状态猜 z ，策略的奖励是“让判别器猜得出来”，于是不同技能会走到彼此区分得开的状态。

- **下一讲**：机器人 RL，重点是带自主性的学习；客座讲者 Ashish Kumar (Tesla Optimus 团队)。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
长程任务的四个例子	做一顿饭、修让 loss 爆炸的 bug、开去 Yosemite、评审长报告：步骤有的互相依赖、有的可并行	2:06	—
两层策略的板书	高层 π_{hl} 读 s 和任务出 g ，低层 π_{ll} 读 s 和 g 出 a ；哪一层跑得快	5:41	—
芝士蛋糕的分解链	烤蛋糕 → 买原料 → 去商店 → 走到门口 → 迈步 → 收缩肌肉：每往下一层都更好训	7:16	—

图	看什么	跳转	出处
分层为什么有用	四条理由；注意"中间监督"和"平策略加 shaped reward"的差别在跨子任务共享	13:25	—
目标表示练习	四道题：意大利菜机器人、校园自动骑行、写法律文书的 LLM、订假期的 web agent	19:37	—
煮意面的状态空间草图	几团状态区域和技能之间的箭头，"缝"就画在两团区域中间	34:31	—
什么时候换目标	两个方案各自的失败模式；结论：多数系统选固定步数	43:20	—
四个近期分层机器人系统	都有一个出中间目标的高层模型和一个出动作的低层模型	45:26	收拾卧室应为 π 0.5（推断）
语言子目标的数据切段	一段示范切成长短不一的段，每段一句话；部署时以固定频率重新询问	47:32	应为 YAY Robot
语言干预视频	黄字是高层输出；人插话"高一点"覆盖高层；微调后高层自己会说	52:06	同上
分层 vs 平策略的数字	task progress 提升 34%，instruction accuracy 有差距；DAgger 优于普通模仿	54:10	应为 Hi Robot
图像子目标示例	粉色边框是生成的子目标图：牙膏举到碗上方、再放低；开抽屉先搭把手再向右	57:46	应为 SuSIE
Ant push 与 ant fall	直接冲会掉沟里，要先推方块搭桥再过去	1:06:06	应为 HIRO
Half cheetah 的技能	无监督发现的倒退、前进、前空翻	1:08:43	应为 DIAYN

提到的工作

名称	在本讲里的作用
多任务 / 目标条件策略（第 12 讲）、meta-RL（第 13 讲）	开场回顾：分层把"会做多件事"的策略当低层
Options 框架（Sutton, Precup & Singh, 1999）	子目标的另一个名字 option 出自这里；课上只提了名字
Chain of thought	"一个策略先想再做"的中间路线；LLM 里的分层
DAgger（Ross et al., 2011；第 2 讲）	高层在语言空间做 DAgger：人用语言干预覆盖高层输出
YAY Robot（Shi et al., 2024；应为）	语言子目标的分层模仿：trail mix 装袋、切段标注、语言纠正
Hi Robot（Shi et al., 2025；应为）	分层 vs 平策略 task progress 提升 34% 的对比；奶酪片切段例子

名称	在本讲里的作用
π 0.5 (Physical Intelligence, 2025; 推断)	"收拾卧室"一句话展开成一串语言子任务
SuSIE (Black et al., 2023; 应为)	图像子目标: 图像编辑扩散模型当高层, 可用人类视频训
InstructPix2Pix (Brooks et al., 2022; 推断)	上一条所用的图像编辑模型底座
ECoT (Zawalski et al., 2024)、CoT-VLA (Zhao et al., 2025) (推断)	把子目标图 / 子任务 / bounding box 当思维链再出动作的单模型版本
HIRO (Nachum et al., 2018; 应为)	目标状态当高层动作的分层 RL: ant push / ant fall; 高层 relabeling 加 off-policy
Hindsight Experience Replay (Andrychowicz et al., 2017; 第 12 讲)	两个分层 RL 例子都用的 hindsight relabeling 出自这里
Language as an Abstraction for Hierarchical Deep RL (Jiang et al., 2019; 应为)	语言目标的分层 RL: 语言空间的 hindsight relabeling
DIAYN (Eysenbach et al., 2018; 应为)	无监督技能发现: half cheetah 学会倒退、前进、前空翻
SayCan (Ahn et al., 2022; 指定阅读)	LLM 当高层时用 affordance 价值函数落地; 课上只用咖啡机例子讲了动机
Ashish Kumar (Tesla Optimus)	下一讲客座讲者

术语对照

English	中文
long-horizon task	长程任务: 要做很久、由多个子任务按顺序组成
hierarchy / hierarchical policy	分层 / 分层策略: 高层出子目标、低层出动作
high-level policy π_{hl} / low-level policy π_{ll}	高层策略 / 低层策略
subgoal / subtask / skill / option / high-level action	子目标 / 子任务 / 技能 / option / 高层动作: 同一个东西的不同叫法
flat policy	平的策略: 不分层, 直接从任务和状态出动作
temporal abstraction	时间抽象: 高层在更粗的时间尺度上决策
goal-conditioned policy	目标条件策略: 目标并进输入的策略 (第 12 讲)
intermediate supervision	中间监督: 针对中间步骤而不是最终结果的训练信号
shaped reward	塑形奖励: 为中间进展额外加的奖励
structured exploration	结构化探索: 在目标空间而不是原始动作空间里探索

English	中文
chain of thought	思维链：先写出中间推理再给答案；这里指"先想子目标再出动作"的单策略
goal representation / level of abstraction	目标表示 / 抽象层级
waypoint	路标点：导航里的中间坐标
latent goal	隐目标：没有显式含义、只是网络激活的目标向量
end-to-end	端到端：只用最终任务的信号训所有部件
affordance	可供性：在当前状态下某个动作做不做得成
success classifier / done predictor	成功分类器 / 完成判定：判断子目标是否已达成
re-query	重新询问：回到高层要新目标
behavior cloning (BC)	行为克隆：对示范动作做监督学习（第 2 讲）
Dagger	数据聚合：让策略自己跑、专家纠正、再训（第 2 讲）
intervention / correction	干预 / 纠正：人在系统运行中覆盖它的输出
task progress / instruction accuracy	任务进度 / 指令准确率：分层系统的两个评测指标
image editing model / diffusion model	图像编辑模型 / 扩散模型：从噪声逐步去噪生成图像的生成模型
(multi-step) inverse model	（多步）逆模型：从当前和未来的观测反推动作
VLA (vision-language-action model)	视觉-语言-动作模型：用视觉语言模型直接输出机器人动作
hindsight relabeling	事后重标：把没达成的目标换成实际达成的（第 12 讲）
off-policy	离策略：训练数据不必来自当前策略（第 5—6 讲）
skill discovery	技能发现：不给监督，让系统自己学出多样的行为

字幕勘误

"Yusede" → Yosemite; "fshot" → few-shot; "hierarchal" → hierarchical; "highle""lowle""pi ll""the little policy" → high-level、low-level、 π_{ll} 、low-level policy; "goal condition setting" → goal-conditioned; "fkatcha" → focaccia; "being the eggs" → beat the eggs; "course defined" → coarse; "stator observation" → state or observation; "language walls" → language models; "Dagger" →

DAgger; "requer""requery" → re-query; "post talk" → post hoc; "middle scoop" → metal scoop; "wayoints" → waypoints; "creass" → crevasse; "reabeling" → relabeling。

带走的问题

1. 第 5 节的"缝"在 LLM agent 里长什么样？规划器列的步骤，执行器（工具）在当前上下文里未必做得成——你的系统里谁负责报告"从这里做不成"？SayCan 的 can 项能不能用工具的前置条件检查来代替？
2. 方案一（完成才换）和方案二（每 N 步换）的失败模式不同：卡死 vs 走错 N 步。你的 agent loop 是哪一种？如果模型每一步都重新规划（N = 1），它付出的代价是什么，省下的又是什么？
3. 端到端训一个隐目标的两层网络，讲者说它就是平的策略。那么"思维链写在输出里"和"隐目标藏在激活里"差在哪——中间监督从哪来？如果像第 10 讲那样只按最终答案给奖励，思维链还算中间监督吗？
4. 语言子目标能让人插话纠正，图像子目标不需要标注还能用人类视频。你的场景里哪种表示同时满足"够表达、有结构、层级合适"？能不能两种都用？
5. 分层 RL 里高层的旧数据会因为低层变了而失效（HIRO 的 relabeling）。类比一下：如果你的工具（低层）升级了，之前收集的"规划器决策 → 结果"数据还能直接拿来训规划器吗？

CS224R 第 16 讲 | 机器人 RL：自主学习 (RL for Robots: Autonomous Learning)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 16 讲, 2025 年 5 月 23 日 · 本讲和第 17 讲是课程的机器人专场：今天讲"怎么让机器人在没人盯着的时候自己练", 第 17 讲是关于 sim-to-real 与人形机器人的客座讲座 视频: <https://www.youtube.com/watch?v=rbaWQQLrzl0> (1:05:44, 英文字幕是自动生成的, 算法名和板书记号错得不少, 见文末勘误) 讲者: Chelsea Finn (课程主讲, 不是客座; 讲的大多是她自己实验室的工作) 课程主页: <https://cs224r.stanford.edu/> · 本讲没有指定阅读; 课上讲到但没点名的三篇核心论文: EARL (Sharma et al., 2021) · MEDAL (Sharma et al., 2022) · Single-Life RL (Chen et al., 2022)

一句话: RL 靠试错练习, 看起来比模仿学习自主得多, 可每次"再来一次"都默认有人把机器人和场景摆回起点——模拟器里是一句 `env.reset()`, 真机上是一个人。本讲把"去掉这个人"定义成自主 RL 问题: 初始化一次, 之后几乎不 `reset`; 评价要么看一生累计的奖励 (火星车), 要么看学完之后部署的策略好不好 (做饭机器人)。直接把 `episode` 拉长会失败: 机器人会漂移进难以恢复的状态, 或者到了目标就赖着不走、`replay buffer` 里只剩目标附近的状况。解法是让机器人自己复位: `forward-backward RL` 多学一个"撤销任务"的 `backward policy`; MEDAL 让它回到示范里的专家状态分布, 奖励用判别器学出来; 多任务循环则按当前状态挑一个此刻可练的任务, 任务图可以手写, 也可以让 VLM 提议。最后是 `single-life RL`: 部署后遇到没见过的情况, 在一条 `episode` 里靠"先回熟悉状态"和"在预训练技能层面换策略"把自己救回来。

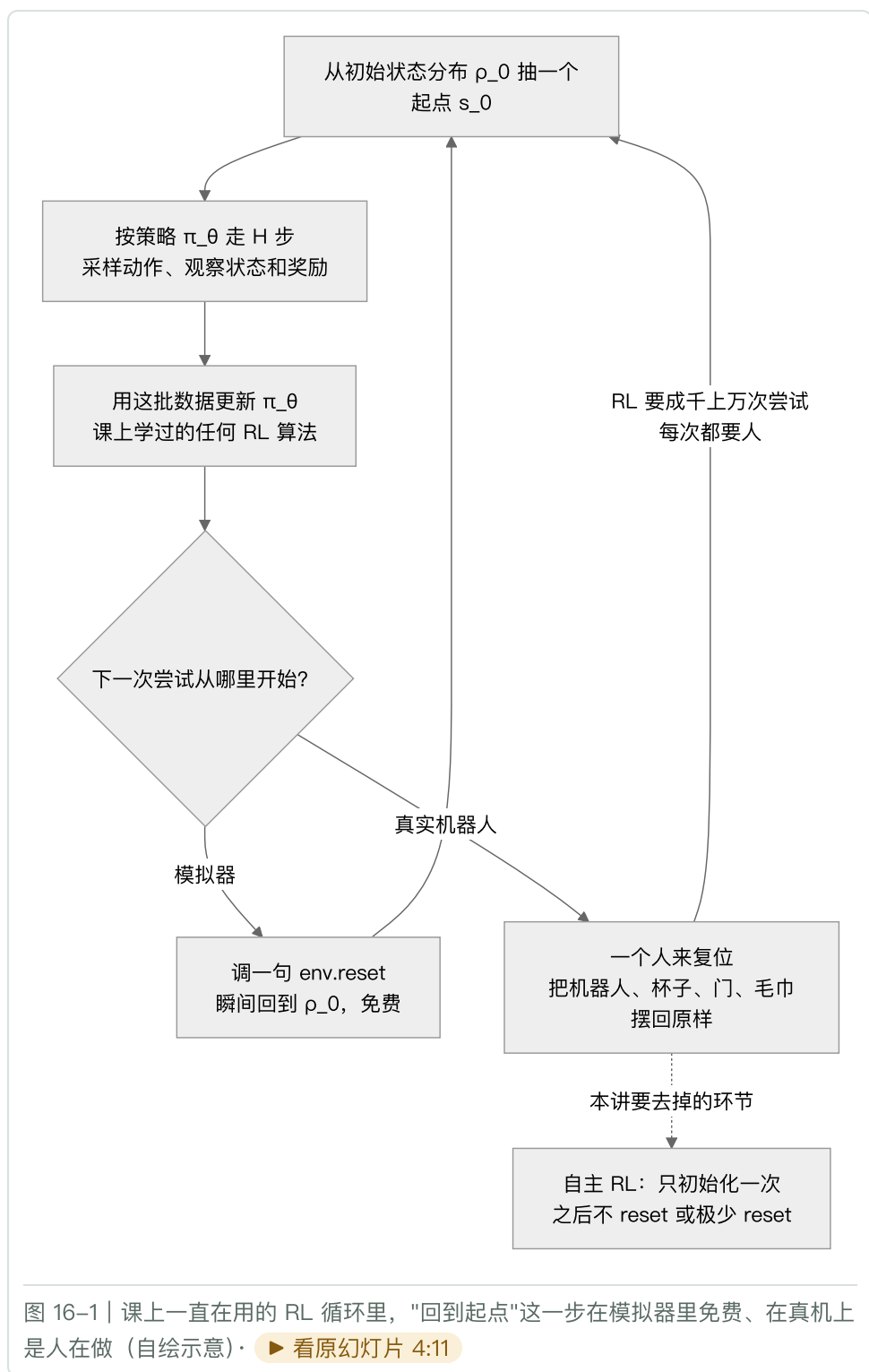
时间轴

时间	内容
0:06	开场: 本讲与第 17 讲聚焦机器人; 今天讲自主学习, 下周三是 sim-to-real 与人形机器人的客座讲座; 大纲
1:07	动机: RL 靠试错"练习"看似自主, 但每次重试都默认有人把机器人放回起点; <code>env.reset()</code> 只有模拟器里才有
7:19	三段真实视频: 人替机器人摆冰球、关门、复位毛巾——人比机器人还忙
8:19	问题定义: 初始化一次、之后几乎不 <code>reset</code> ; 两种评价——一生累计奖励 (火星车) vs 学完后部署的策略 (做饭)

时间	内容
12:23	两个目标函数的板书；问答：只有一条命为什么还要取期望；为什么这个问题重要
17:02	实验：SAC 把 episode 拉长 2 倍、10 倍，连简单的 fish 任务都学不会
19:04	两个原因：漂移进难以恢复的状态；到了目标就赖着，replay buffer 的状态分布坍塌
22:10	Forward-backward RL：学一个撤销任务的 backward policy；算法步骤；问答：两个策略共享信息吗
27:51	三个改进方向；MEDAL：backward policy 回到专家状态分布，奖励用判别器学；问答：为什么不会赖在目标状态
35:07	EARL benchmark：每 20 万步才 reset 一次；各算法比较；问答：示范从哪来、episodic 是什么意思
39:13	正反任务难度不对称；真机延时视频；奖励也得用模型给；这些还在研究阶段
42:16	多任务循环：咖啡例子；按状态选任务的算法；灵巧手、四足、七阶段拧螺丝
48:31	任务怎么选：手写任务图 vs 让 VLM 提议；与分层 RL 的区别；生成目标图当奖励；可重复的任务
53:04	Single-life RL：部署后在一条 episode 里自适应；插钥匙的例子；三种设定对比
56:39	测试时直接用 RL 微调会翻倒卡死；两个想法：先回熟悉状态、在预训练技能层面换策略；例子
1:02:17	问答：安全约束怎么办；VLM 的常识；总结

核心内容

1. 为什么 RL 还没让机器人自主：藏在 reset 里的那个人



- **先把词摆出来:** agent 在每个时刻 t 看到状态 s_t , 按策略 $\pi_\theta(a_t | s_t)$ 选一个动作 a_t , 环境给出奖励 $r(s_t, a_t)$ 并转到下一个状态; 一段连续的状态—动作序列叫轨迹 (trajectory) 或 rollout, 长度记作 H (horizon); 每次尝试的起点从初始状态分布 ρ_0 里抽。模仿学习 (imitation learning, 第 2 讲) 是让 agent 模仿专家给的示范 (demonstrations); 强化学习是让 agent 自己试错 (trial and error), 靠自己的经验变好。

- **练习看起来很自主**：一条轨迹里的每一步都由策略自己决定，没有人插手。讲者用一张示意图说明：从 p_0 出发，目标是进入奖励高的那片状态区域；第一次尝试通常走得很差，然后再试一次、再试一次。
- **问题出在"再试一次"上**：第一次尝试结束时机器人停在某个地方，可我们默认第二次又从 p_0 附近出发——它是怎么"魔法般"回去的？课上所有 RL 算法都默认这一步免费。模拟器里确实免费：走一千步，调一句 `env.reset()`。真实世界没有这个函数：把机器人挪回去、把杯子扶起来、把门关上，都是人做的。于是整个学习过程并不自主。
- **三个例子**：小车或自动驾驶要学会导航到篮球场，拐错一个弯就得先回来；机械臂学抓杯子，一不小心把杯子碰倒或推出臂展，就没法再试；移动机器人学把书放上书架，失误后书得回到够得着的地方。而 RL 不是试三次就会的事，往往要成千上万次尝试，每一次都对应一次人工干预。
- **三段真实视频 (7:19)**：机器人学把冰球打进球门，旁边的朋友一直在捡球摆球，"干的活比机器人还多"；机器人学开门，同事在一旁不停地把门关上；机器人学叠毛巾，另一位同事把毛巾重新铺平。

小注：叠毛巾那段里复位的人，字幕作 "Archet"，讲者说他本季度早些时候来讲过课，应为 Archit Sharma——本讲后面 EARL、MEDAL、single-life RL 三篇论文的一作或合作者（推断）。

2. 把问题定义清楚：自主 RL，以及两种"好"

- **最朴素的想法**：既然每个 episode 结束要人复位，那就把 episode 拉长，reset 次数自然变少，课上学过的算法原样照跑。这引出自主 RL (autonomous RL) 的定义：把场景初始化一次，之后 agent 自己和世界交互，没有环境 reset；放宽一点，允许频率极低的 reset（比如 H 非常大时才复位一次）。
- **评价什么，先要想清楚**。同样是"没有 reset 的一段长长的经历"，有两种截然不同的关心法：
 1. **一生累计了多少奖励**：把 agent 放出去边学边干，我们在乎的是它这一生里到底把事做成了多少。例子是火星车——你要的是它在火星上采到科学数据，而不是学完之后再部署一遍。
 2. **学完之后的策略好不好**：训练过程中干得怎样无所谓，但训完拿出来的策略要能从起点出发反复把任务做好。例子是学做饭——练习时糊几锅没关系，将来要能给各种顾客稳定出菜。

- **第二种叫 deployed policy evaluation**，目标函数和课上一共用的一样：

$$J_{\text{deploy}}(\pi) = \mathbb{E}_{s_0 \sim \rho_0, a_t \sim \pi(\cdot|s_t)} \left[\sum_t \gamma^t r(s_t, a_t) \right]$$

s_0 从初始状态分布 ρ_0 抽，之后每步按学好的策略 π 走，把奖励加起来（ γ 是折扣因子，越远的奖励打的折越多）；它衡量的是"学完的这个策略从起点出发能拿多少分"，和策略是怎么学出来的无关。

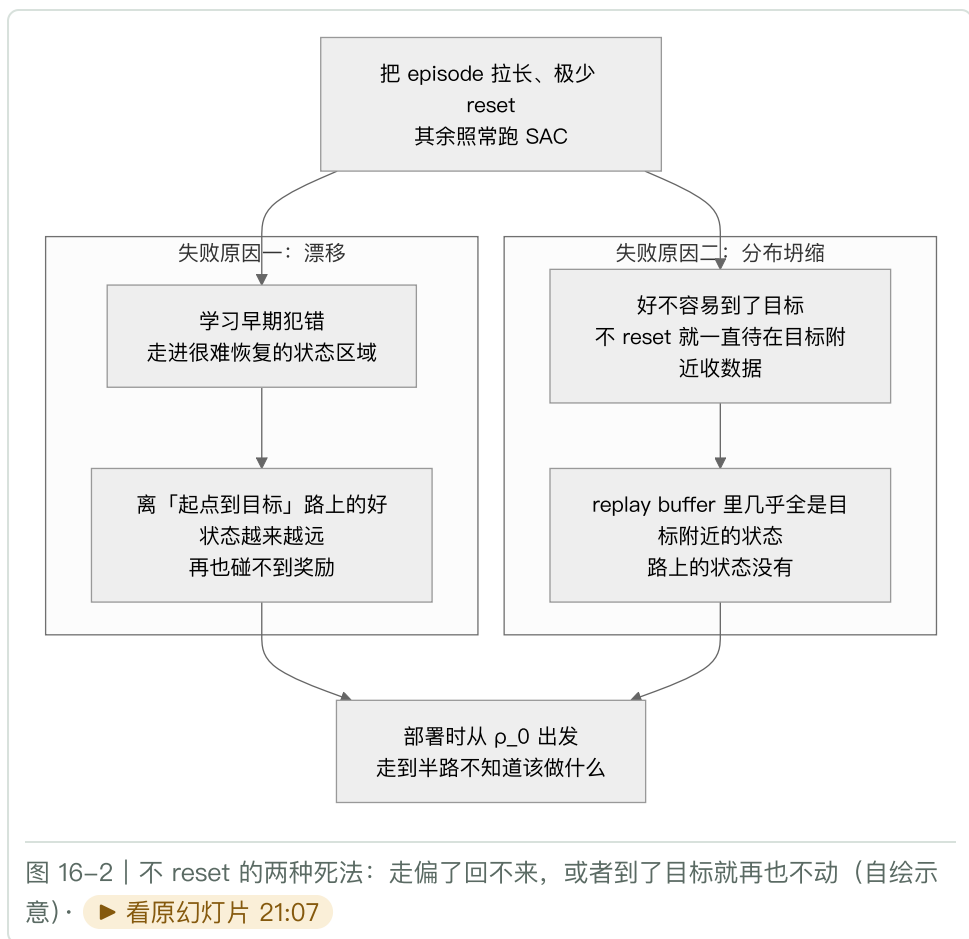
- **第一种是 continuing 的视角**：看整个学习过程里的平均奖励，而且要在"寿命越来越长"的极限下看：

$$J_{\text{life}}(\mathcal{A}) = \lim_{H \rightarrow \infty} \frac{1}{H} \mathbb{E}_{\mathcal{A}} \left[\sum_{t=1}^H r(s_t, a_t) \right]$$

这里的期望不再是对某一个固定策略取的，而是对整个学习过程（算法本身，记作 $\mathcal{A}$ ，包括它怎么采样、怎么更新）取的； H 小的时候只是平均了很短一段，所以取 H 趋于无穷。

- **问答：只有一条命，为什么还要取期望？**——如果只想知道这一次活得怎么样，直接跑一遍量就行。但环境有随机性，学习过程也有随机性（比如 batch 怎么抽），想评价的是"这个算法"而不是"这一次"，就得对多次运行取期望。
- **为什么值得做**：我们既希望机器人干活时不用人管，也希望它训练时不用人管。真机上做 RL 是把成功率和可靠性推到模仿学习之上的一条有希望的路；训练不需要人盯，就能收更多数据；更多数据又能换来更高的可靠性和更好的泛化——前提是这套系统能在各种真实环境里大规模铺开。

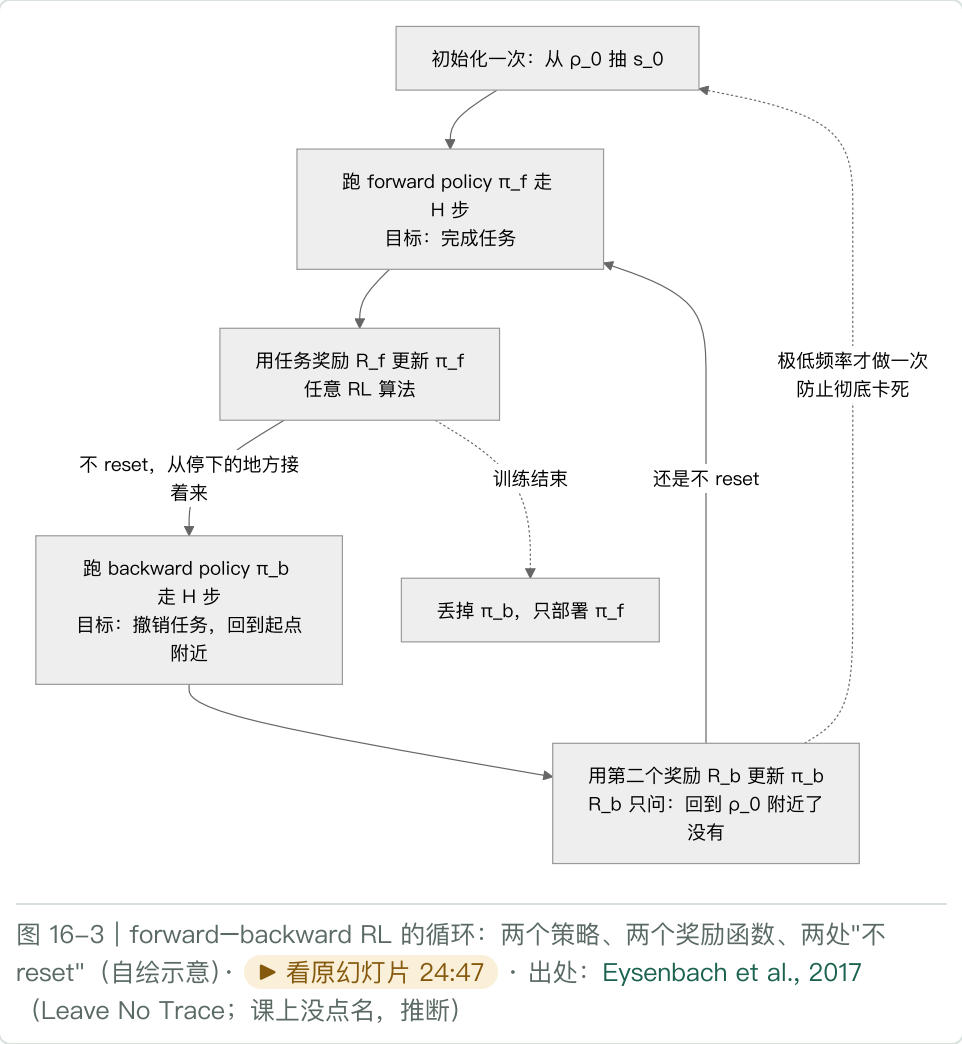
3. 只把 episode 拉长行不行？两个失败原因



- **实验：**拿 SAC (soft actor-critic, 第 5 讲的 off-policy actor-critic 算法：把交互数据存进 replay buffer 反复用) 在一个很简单的 fish 控制任务上跑——控制一条"鱼"游到指定位置。横轴是训练步数，纵轴是上一节的第二个目标：学到的策略部署后拿到的奖励。episode 长 1000 步时曲线正常爬升；拉长到 2 倍就变差，10 倍更差；拉到几万步才 reset 一次，几乎什么都学不到。这么简单的任务尚且如此。
- **原因一：漂移 (drift)。**学习早期难免犯错，有些错会把机器人带进很难恢复的状态区域（这个环境里鱼断不了尾巴，但真机上会断）。episodic 的 RL 每次都能回 p_0 重来，自主学习却只能从停下的地方接着走，于是离"从起点到目标"那条路上的好状态越来越远，再也碰不到奖励。
- **原因二：分布坍塌（更隐蔽）。**假设机器人运气好终于到了目标，拿到奖励——不 reset，它就会赖在目标附近继续收数据，replay buffer 里很快全是目标附近的状态。学习信号看着很好，但我们要的策略是"从 p_0 出发走到目标"，路上那些状态它几乎没再见过。把最优策略会经过的状态分布记作 p^* ，buffer 里的分布和 p^* 对不上，部署时走到半路就不知道该干什么。

- **术语**：能反复回起点重来的传统设定叫 episodic；没有 reset、一整段连续学习的设定叫 non-episodic，也叫 reset-free 或自主学习。两种失败都源于后者没有"回起点"这一步——所以最直接的解法就是让机器人自己学会回去。

4. Forward-backward RL：多学一个把任务"撤销"的策略



- **想法**：既然允许从 p_0 重来就能学好，那就再学一个策略，专门负责把机器人从任务结束时的状态送回 p_0 。做任务的叫 forward policy π_f ，撤销任务的叫 backward policy（也叫 reset policy） π_b ，合起来就是 forward-backward RL。
- **算法**（讲者在板上写的步骤）：
 1. 第 0 步：初始化，从 p_0 抽一个 s_0 。
 2. 跑 π_f 走 H 步，用任务本身的奖励 R_f 更新 π_f 。
 3. 不 **reset**，从停下的地方直接跑 π_b ，用第二个奖励 R_b 更新 π_b —— R_b 要回答的是"回到 p_0 附近了没有"，所以它是一个额外要

定义的奖励函数。

4. 再次不 reset，回到第 2 步。两个更新都可以用你喜欢的任何 RL 算法。
5. 可以留一个频率极低的 reset，以防真的卡死。训练结束后 π_b 扔掉，只部署 π_f 。

$$\pi_f = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_t R_f(s_t, a_t) \right], \quad \pi_b = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_t R_b(s_t) \right], \quad R_b(s) = \mathbf{1}[s \in \text{supp } \rho_0]$$

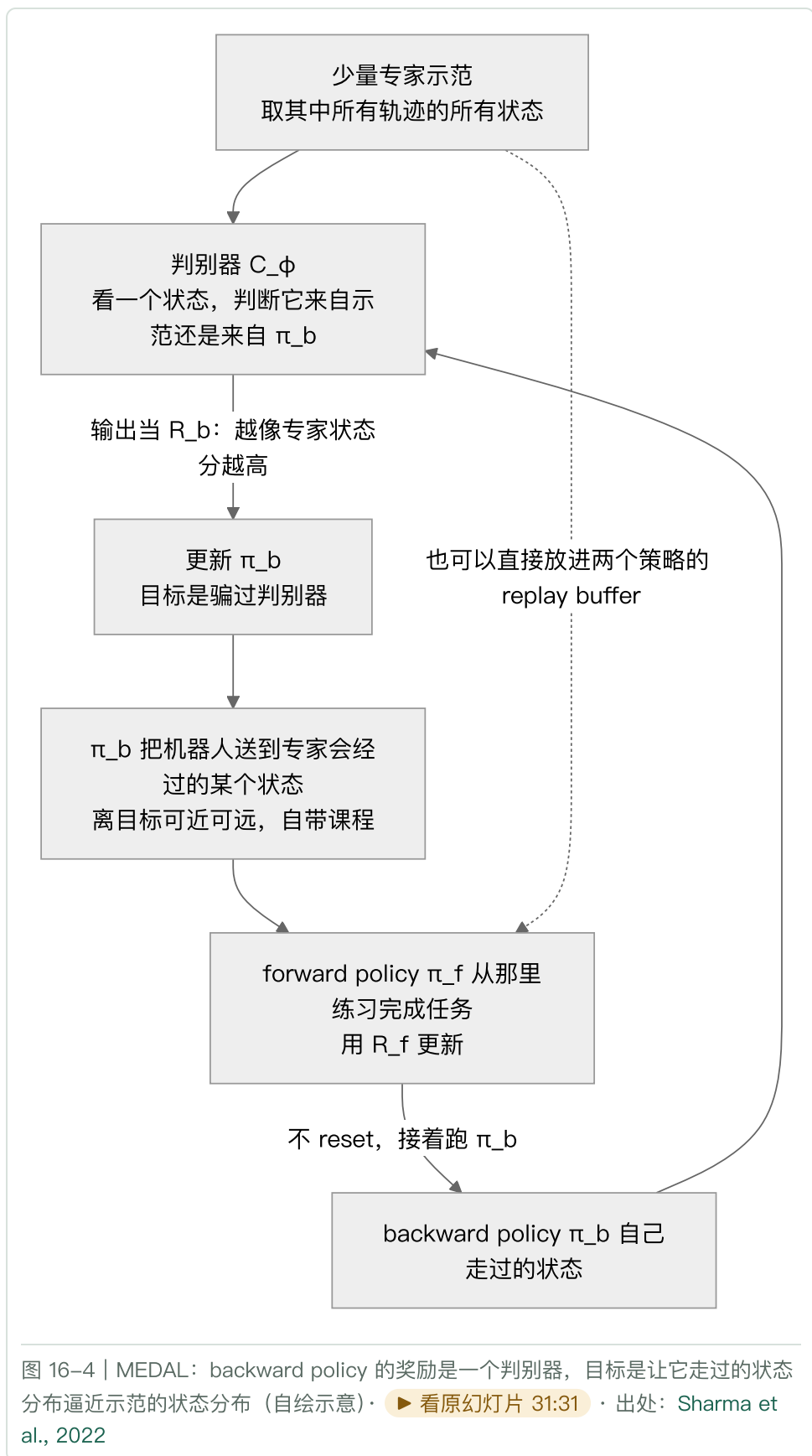
两个策略各自最大化自己的累计奖励； R_b 最简单的写法是“到了初始状态分布覆盖的区域就给 1”（supp 指分布的支撑集），实践里也可以是到起点的距离，或者一个学出来的分类器。

- **跑多久切换：**最简单是固定 H 步。更讲究的做法是早期让 π_f 多跑一会儿（它还没学会什么， π_b 没活可干），后期再让 π_b 多跑；但固定步数更省事，而且一旦过程真的自主了，这个数字就没那么要紧——“你可以睡觉去，让机器人自己练”。
- **问答：两个策略之间共享信息吗？**——用 replay buffer 的话，可以把同一批数据按另一个任务的奖励重新打分后共用（第 12 讲的 hindsight relabeling 就是这个操作）；也可以共享图像编码器之类的部分权重。但实践里常常不怎么共享：两者学的是几乎相反的事，硬共享反而添乱。后面讲的多任务方法会共享得更多。
- **优缺点：**优点是极其简单。缺点是 π_b 本身没什么用——花一半算力学一个最后扔掉的策略，看着浪费；但正是它换来了自主。

小注：这个“正反两个策略轮流跑、互为 reset”的框架应出自 Han et al., 2015 (Levine 组的 compound controllers) 和 [Leave No Trace](#) (Eysenbach et al., 2017)，后者还让 reset 策略顺带判断“这一步做了还回不回得来”，是第 9 节安全问题的雏形（推断，课上没点名）。

- **三个改进方向**（后面逐个展开）：(a) π_b 不一定要回到最开头，可以回到更多样的状态；(b) 不只学一正一反两个任务，而是在一圈任务之间循环练习；(c) 有些任务本身可以设置成反复做的。

5. MEDAL：回到专家走过的状态，而不是回到起点



- **动机：**forward-backward 每次都回到最开头，可我们要的策略得在"专家会经过的每一个状态"上都知道怎么做。已有工作表明，如果能从专家轨迹沿途的各个状态出发练习，哪怕在 episodic 设定里也比总

从 p_0 出发学得快——离目标近的起点简单，远的难，等于自带课程 (curriculum)。

小注：这条"从示范沿途的状态起步"的结论应出自 Salimans & Chen, 2018 (用一条示范通关 Montezuma's Revenge) 和 Florensa et al., 2017 的 reverse curriculum 一类工作 (推断)。

- **两处改动：**

1. 第 -1 步：给少量专家示范 (几条就够)，当作专家状态分布的样本。
2. π_b 的奖励不再是"回到 p_0 "，而是"到达专家去过的那类状态"。
这个奖励是学出来的：像第 8 讲的 reward learning 那样训一个判别器 (discriminator / classifier) $C_\phi(s)$ ，分辨一个状态来自示范还是来自 π_b 自己走过的状态； π_b 的任务就是骗过它。

- **为什么不会赖在目标状态 (问答)：**目标状态也在示范里， π_b 直接停在目标不就行了？不行——判别器用的是示范里所有轨迹的所有状态，目标只是其中一小撮；如果 π_b 只去目标，判别器很快学会"只在目标附近出现的就是策略产生的"，奖励随之消失。形式上这是一个状态分布匹配 (state-distribution matching) 目标：让 π_b 走过的状态分布逼近专家的状态分布。

$$\min_{\pi_b} D(\rho^{\pi_b}(s) \parallel \rho^E(s)), \quad R_b(s) \approx \log C_\phi(s)$$

ρ^{π_b} 是 π_b 走过的状态分布， ρ^E 是示范里的状态分布， D 是两者之间的某种散度 (divergence，衡量两个分布差多远的量)；用判别器输出当奖励是实现这种匹配的标准做法 (GAIL 那一路，第 8 讲)，奖励的具体写法以论文为准 (推断)。

- **好处：**同时治了漂移和坍缩—— π_b 总把机器人送回专家分布附近， π_f 每次都从"路上"某处开始练，起点有难有易。示范还可以直接塞进两个策略的 replay buffer 里当训练数据。名字 MEDAL 的意思就是 matching expert distributions for autonomous learning。
- **EARL benchmark (35:07)：**专门给自主 RL 设的模拟 benchmark，训练时每 200,000 个环境步才 reset 一次，评价看部署策略从 p_0 出发的表现。最简单的任务是抓杯子挪位置 (抓取机制简化过)；更难的一个是门——正向任务关门，反向任务开门。结果：
 - 光跑 SAC、只靠极稀疏的 reset：学不出来，和 fish 一样。
 - 给它频繁 reset 的 oracle (拿到了本不该有的便利、用来看上限的基线)：能学好——说明差的不是算法，是 reset。

- 几个专为 reset-free 设计的算法：靠扰动去访问新状态的（第 14 讲探索的思路）、原味 forward-backward、给目标课程的、以及 MEDAL。它们都明显好于不为此设计的算法；简单任务上彼此差不多，任务变难后 forward-backward 和 MEDAL 最好，靠新奇度或目标课程的那两个掉队。

小注：这几个基线应为 R3L 的 perturbation controller (Zhu et al., 2020) 和 VaPRL (Sharma et al., 2021)；MEDAL 论文摘要说在 EARL 的三个稀疏奖励任务上与先前方法持平或更好，最难的任务上提升约 40%（基线身份是推断，数字来自论文摘要）。

• 问答

- 示范从哪来？——真机上用遥操作 (teleoperation)：主从臂式的 puppeteering 装置、VR 手柄、3D 鼠标、甚至键盘；模拟里可以写脚本策略，或者提前用 RL 训一个策略专门产示范来测算法。
- episodic 到底什么意思？——每次尝试是一个 episode，试完 reset 再试；non-episodic 就是没有 reset、一整段连续学习，也叫 reset-free。

6. 真机上的现实：难度不对称、奖励也得学、还在研究阶段

- **正反难度不对称**：门任务里，开门（反向）取决于手臂起始位置，常常比关门（正向）更难。两个策略学得快慢不一，整个循环就会一头堵住——好比正向任务是后空翻，反向任务是站起来，难度差得远。这种不对称很常见。
- **真机延时视频 (39:44)**：左边是 forward-backward，两个策略都会犯错，但全程没人干预也在变好；右边是 MEDAL，能看到正向策略一旦没抓起来，反向策略就没活可干。
- **奖励也不是世界白给的**：真实世界不会告诉你"成功了"。所以 reset-free 算法几乎总是和 reward learning（第 8 讲）搭配：视频里的奖励全由模型（分类器）给，不需要人，整套东西才能"扔那儿跑一晚上"。
- **问答：鸡生蛋问题**——反向策略学不好，正向就没法从好状态起步；正向从没抓起毛巾，反向也没得练。答：可以给正反两个任务都提供示范来启动；除此之外，平衡两者本身就是这类方法的固有难题，目前只能说"在一些设定里能跑通"。
- **诚实的定位**：这些还处在研究阶段，不是机器人公司今天在用的东西。

7. 多任务循环：按当前状态挑一个能练的任务

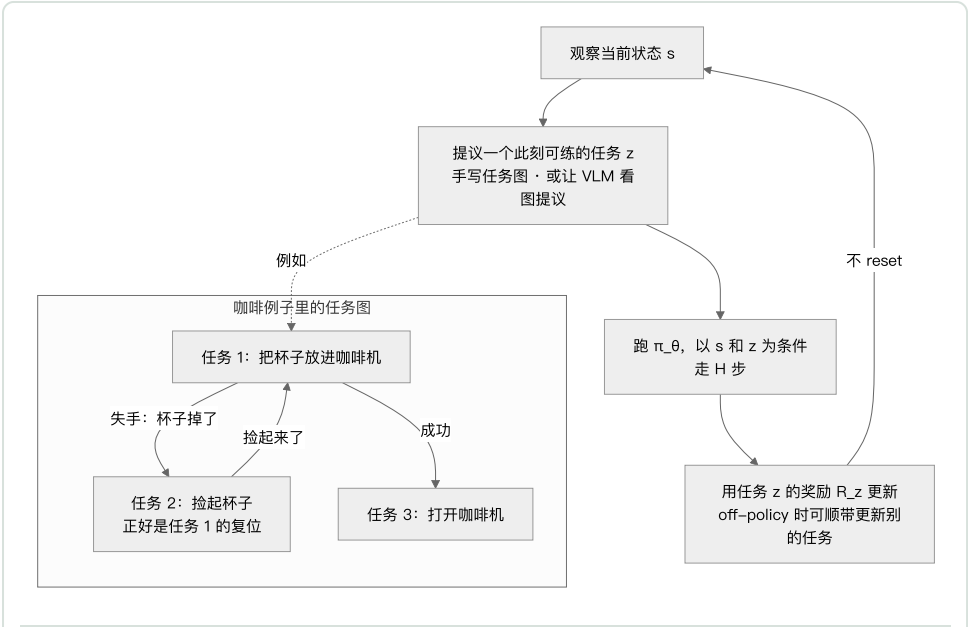


图 16-5 | 多任务自主练习：由当前状态决定练哪个任务，"复位"变成了任务图里的一条边（自绘示意） · [▶ 看原幻灯片 43:48](#) · 出处: Gupta et al., 2021（课上没点名，推断）

- **换个角度：**如果要学的本来就不止一个任务，就不必拘泥于"一正一反"，而是在一圈任务之间循环，**由当前状态决定练什么**。做咖啡的例子：任务 1 是把杯子放进咖啡机；失手把杯子掉了，传统做法是人把杯子摆回去，现在改成让"捡起杯子"当任务 2——它恰好是任务 1 的复位；任务 1 成功了就练任务 3 打开咖啡机。这样就有一张任务图（或环），每个时刻只练从当前状态可行的任务，不可行的排除。
- **算法：**学一个以任务为条件的多任务策略 $\pi_\theta(a | s, z)$ （第 12 讲：z 是任务标识，可以是编号、一句话或一张目标图）。循环：观察状态 $s \rightarrow$ 提议一个任务 $z_i \rightarrow$ 跑以 z_i 为条件的策略走 H 步 $\rightarrow$ 更新。更新时若用 on-policy 算法（只能用当前策略自己刚采的数据），就只更新任务 z_i ；若用 off-policy 算法（能用 replay buffer 里的旧数据），可以顺带更新所有任务。这要求每个任务都有奖励函数。照样没有 reset 或极少 reset；训练完部署的是能做多个任务的同一个策略。

$$\max_{\theta} \mathbb{E}_{z \sim p(\cdot | s)} \mathbb{E}_{a_t \sim \pi_{\theta}(\cdot | s_t, z)} \left[\sum_t R_z(s_t, a_t) \right]$$

$p(z | s)$ 是任务提议器：看着当前状态给出一个此刻可练的任务； R_z 是任务 z 自己的奖励；策略参数 θ 在所有任务间共享。

- **例子**

- 灵巧手：任务是把物体拿起、翻面、再旋转，各阶段是不同的任务。一开始既不会翻也不会转，就只能反复练"拿起来、放下去"；拿稳了才有机会练翻转。这正好化解上一节的不对称：正向没成功、状态没变，就不硬跑反向，而是继续练当下有意义的那个任务。
- 四足机器人：同时学向前走、向后走和摔倒后爬起来，走得太靠后就切到向前走。
- 七阶段的拧螺丝任务：拿起扳手、对准、反复拧，各阶段各跑各的策略，最后再由反向策略把扳手放回桌上。

小注：灵巧手的例子应为 Gupta et al., 2021 (Reset-Free RL via Multi-Task Learning)，四足的例子应为 Smith et al., 2021 (Legged Robots that Keep on Learning) (均为推断)。

- **任务怎么选** (问答，讲者说这是研究前沿)：
 1. 手写：预先定义任务集合和那张图——任务 1 失败去练任务 2，成功去练任务 3——照图执行。
 2. 学出来：训一个提议模型。哪个任务此刻最值得练，本身是个开放问题；一个可行做法是问语言模型或 VLM："这个场景里我能做哪些事来变强？"任务是场景相关的——桌上有玩具微波炉、香蕉、茄子、锅，"打开微波炉""把香蕉从银色锅里拿出来"可以，"拿起胡萝卜"不行，因为根本没有胡萝卜。
- **和分层 RL 什么关系** (问答，第 15 讲)：提议器长得像高层策略，但目的相反。高层策略是为了达成某个长期目标而选下一个子任务，越聚焦越好；提议器是为了让机器人学到尽量多样的东西，越发散越好。讲到的那个系统确实用了分层结构：VLM 提议任务 → 用图像编辑扩散模型把当前图改成"做完之后的样子" (第 15 讲讲过的那类工作) → 策略去够这张目标图。顺带解决了奖励问题：给每个任务手写奖励很难，而有了目标图，奖励就是"离这张图还有多远"。

小注：这个系统应为 Zhou et al., 2024 的 Autonomous Improvement of Instruction Following Skills via Foundation Models，目标图生成用的是第 15 讲提到的 SuSIE 一类图像编辑扩散模型 (推断，课上没点名)。

- **问答：冰球例子能用提议器吗？**——能，比如让它决定练传球还是射门；不过要练的东西就那么几样时，直接排课表 (一节练射门、一节练传球) 也够了。

- **第三条路：把任务设计成可以反复做的。**例子是给餐饮企业叠餐巾——叠完一条接着叠下一条，天然不需要 reset。这方面研究不多。
- 以上三类（做一撤销、任务循环、可重复任务）都是为了"学出一个好策略"；火星车式的"一生累计奖励"问题，则由下一节的 single-life RL 来回答。

8. Single-life RL：部署之后只有一条命

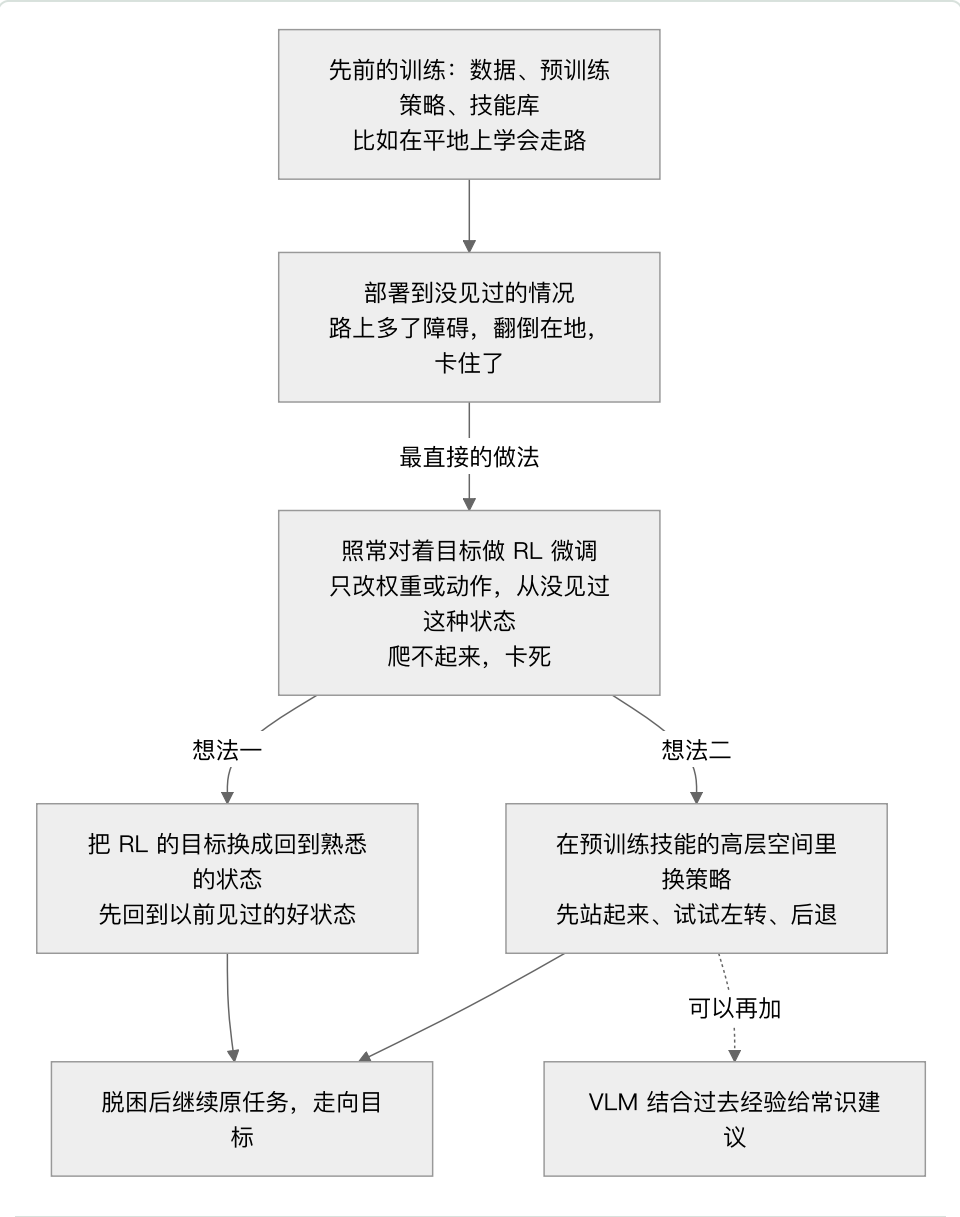


图 16-6 | single-life RL：部署中卡住时，直接微调会卡死，两条出路是"回熟悉状态"和"换高层策略"（自绘示意） · [▶ 看原幻灯片 58:12](#) · 出处：Chen et al., 2022

- **动机：**策略再好，部署到真实世界总会出岔子。人是怎么处理的？课上放了一段开门视频 (54:06)：钥匙没插正，人用力推不动，半秒之内就把钥匙拔出来重插——在一条 episode 里、不重来、当场适应。

我们希望机器人在部署时也能这样：遇到没见过的情況，靠先前的训练加上现场的经验把自己救回来。

• 和前面两种设定的对比：

设定	训练时	部署时	本讲对应
episodic	从 p_0 反复尝试，每次 reset	部署一次固定策略	第 1—15 讲的默认设定
reset-free / 自主 RL	做任务、撤销任务反复循环，没有 reset	部署一次固定策略	第 3—7 节
single-life RL	已有先前的数据或预训练策略	落进没见过的情況，要在一条 episode 里边适应边完成任务，没有 reset	本节

- **直接在测试时做 RL 微调行不行：**四足机器人在平地上训练，部署时路上多了障碍。最直接的做法是照常跑 RL：一边走一边用 TD 更新（第 4、6 讲：用下一步的价值估计修正当前估计）或策略梯度（第 3 讲）改权重。结果它确实翻过了第一个障碍，随后摔了个四脚朝天，就此卡死——RL 还在跑、更新还在做，但从没见过这种状态，怎么更新都爬不起来，除非有人来扶。
- **想法一：先回熟悉的状态，再谈目标。**卡住时，RL 的目标是"去目标"，可从这个陌生状态去目标它毫无头绪。把目标换成"回到先前经验里见过的好状态"，它反而容易脱困——那些状态离目标更近，也是它知道该怎么走的地方，回去之后再继续原任务。

$$\tilde{r}(s) = \log D_{\psi}(s)$$

D_ψ 是一个判别器，分辨一个状态更像先前数据里的状态，还是当前这条命里新踩到的状态；用它当奖励做 RL 更新，agent 就会被拉回熟悉的分布——和第 5 节 MEDAL 的判别器奖励是同一个套路，只是这次在部署时用（奖励的具体形式按论文，推断）。

小注：这就是 [Single-Life RL](#) (Chen, Sharma, Levine & Finn, NeurIPS 2022)：提出的算法叫 QWALE (Q-weighted adversarial learning)，用 Q 值给判别器加权；摘要说在几个 single-life 连续控制任务上比标准 RL 的成功率高 20—60%。

- **想法二：在预训练技能的高层空间里适应。**只改权重、只换动作，能变的有限；如果手里有一组预训练好的技能（向前走、翻身站起），就可以在"换哪种策略"这一层探索——从只会闷头往前走，变成试试先站起来、试试往左。这是换策略而不是换动作。预训练的 LLM 还带着常识，也能帮 agent 在陌生情况下想到别的办法。

- 例子

- 想法一在同一个障碍环境里：还是翻倒了，但它会朝"站着"这种熟悉状态努力，试几次后爬起来，最终到达目标（1:00:15）。
- 低层适应的代表是"从最近几步的历史里推断环境、调整动作"（第13讲那种从上下文快速适应的思路），能应付地形略有不同这类小偏移；碰到差异很大的情况就不够了。
- 高层适应的例子：四足机器人要去一个目标点，眼前只有一张图。没有上下文的策略会一直往前顶，因为它不记得自己刚试过前进；带上下文、在技能层面换策略的版本会试着左转、后退，最后在从没见过的环境里绕过障碍走到目标。要是在关节空间里试不同姿势来解决这种问题，几乎不可能。

小注：高层技能切换这项应为 Chen et al., 2023 的 Adapt On-the-Go (ROAM)，用预训练行为库加各行为的 value function 来选；结尾问答里"让 VLM 结合过去经验给策略建议"的应为同组 2024 年的 Commonsense Reasoning for Legged Robot Adaptation with Vision-Language Models；低层适应的代表应为 RMA (Kumar et al., 2021) (均为推断，课上没点名)。

9. 问答：安全怎么办；VLM 的常识；总结

- 安全 (问：怎么把安全约束"烤"进去?)：除了 reset，另一件需要人盯着的事就是安全——四足机器人可能摔坏自己，机械臂可能把玻璃杯摔到地上，或者把东西撞得很重。两条路：
 1. 学：直接在本讲的框架上加一层——学一个风险概念，识别自己是否处在高风险状态，再学一个 recovery policy 把自己带回已知更安全的状态。这些想法几年前就在安全 RL 里研究过。
 2. 硬约束：能在控制器层面做的尽量做——限制电机不至于过热、给机器人划一个活动盒子、限制施力大小。
- 硬约束不是处处可行，所以"学安全"可能是唯一通用的办法；可学安全又需要失败的样本，这本身就危险。总之是个棘手的问题。

小注：第一条路的代表应为 Recovery RL (Thananjeyan et al., 2021)；第4节小注里的 Leave No Trace 也在 reset 策略上加了"能不能回得来"的判断，是同一思路的早期版本（推断）。

- 在人群里导航 (问)：可以让 VLM 在高层出主意——课上最后那项工作就是让 VLM 结合过去的经验提出该试哪些策略；模型自带的常识

（别撞人、该等就等）在这类场景里相当有用。

- **总结：**为什么 RL 算法没有自动带来自主——因为 reset 里藏着人；把问题定义成两种（一生累计奖励 vs 学出好策略）；对应的算法——学一个 backward policy、学一张任务图在多个任务间循环、以及 single-life RL 防止部署时漂走。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
从 p_0 到高奖励区域的示意	一次尝试走偏后，"再试一次"默认又从起点出发——这一步是谁做的	3:08	—
三段人工复位视频	冰球、开门、叠毛巾：每次尝试之间人在干什么	7:19	—
两种评价与两个目标函数	火星车 vs 做饭；deployed 的期望在策略下取，continuing 的期望在整个学习过程下取、H 趋于无穷	13:53	EARL
SAC 拉长 episode 的学习曲线	1000 步正常，2 倍、10 倍逐步变差，几万步几乎学不到	17:32	同上
漂移与坍缩示意	走偏后回不来；到了目标后 buffer 只剩目标附近的状态	21:07	—
Forward—backward RL 板书	两个策略、两个奖励、两处"不 reset"	23:14	Leave No Trace（推断）
MEDAL 的粉色专家区域	backward policy 把机器人送回专家状态分布里任意一处，而不是起点	31:01	MEDAL
EARL 结果曲线	SAC、oracle、扰动、forward—backward、目标课程、MEDAL 六条线怎么排	36:10	同上
咖啡任务图与多任务算法	任务 2 就是任务 1 的复位；观察状态→提议任务→跑 H 步→更新	43:17	—
灵巧手、四足、拧螺丝	按状态切换练哪个阶段；不对称问题怎么被绕开	46:28	Gupta et al., 2021（推断）
VLM 提议任务的场景	场景相关的可行任务：能拿香蕉，不能拿不存在的胡萝卜	49:32	—
三种设定对比	episodic / reset-free / single-life 各自的训练与部署	55:06	SLRL
测试时微调翻倒 vs 回熟悉状态爬起	同一个障碍环境，RL 的目标不同，结局不同	57:09	同上

图	看什么	跳转	出处
高层换策略穿越障碍	没有上下文一直往前顶；有上下文会试左转、后退	1:01:46	—

提到的工作

名称	在本讲里的作用
模仿学习与示范（第 2 讲）	对照：RL 的"练习"比模仿更自主，但仍要人 reset；后面 MEDAL 和多任务方法又把示范请回来当启动数据
env.reset()	模拟器里免费的复位函数；真机没有，这是全讲的起点
SAC（第 5 讲）	拉长 episode 实验的算法，也是 EARL 里"不为 reset-free 设计"的基线
fish 控制任务	拉长 episode 实验的环境（应为 DeepMind Control Suite 的 fish 域，推断）
Forward—backward RL / Leave No Trace (Eysenbach et al., 2017；推断)	最简单的自主 RL 算法：多学一个撤销任务的 backward policy
Reward learning、判别器奖励、GAIL 一路（第 8 讲）	MEDAL 的 backward 奖励、真机上的成功判定、single-life RL 的"熟悉度"奖励都是它
Hindsight relabeling（第 12 讲）	两个策略之间共享数据的方式：同一批数据按另一个奖励重新打分
MEDAL (Sharma, Ahmad & Finn, 2022)	backward policy 回到专家状态分布，而不是起点
EARL (Sharma et al., 2021)	自主 RL 的形式化与 benchmark：每 20 万步才 reset 一次
扰动 / 新奇度基线、目标课程基线	EARL 里的两个对照（应为 R3L 与 VaPRL，推断）；任务变难后掉队
多任务 / 任务条件策略 $\pi(a \text{ given } s, z)$ （第 12 讲）	任务循环用的策略形式
Reset-Free RL via Multi-Task Learning (Gupta et al., 2021；推断)	灵巧手拿起—翻面—旋转的例子
Legged Robots that Keep on Learning (Smith et al., 2021；推断)	四足同时学前进、后退、爬起的例子
七阶段拧螺丝	任务图例子：各阶段各跑各的策略，反向策略把扳手放回去
VLM 任务提议 + 图像编辑扩散模型生成目标图（应为 Zhou et al., 2024；推断）	学出来的任务提议器；目标图顺带解决奖励定义

名称	在本讲里的作用
SuSIE 一类图像编辑扩散模型（第 15 讲）	上一条里"把当前图改成做完之后的样子"的部件
分层 RL 与高层策略（第 15 讲）	与任务提议器对比：一个为目标聚焦，一个为学习发散
叠餐巾	天然可重复、不需要 reset 的任务
Single-Life RL / QWALE（Chen, Sharma, Levine & Finn, NeurIPS 2022）	部署后一条 episode 内自适应；卡住时先回熟悉状态
TD 更新、策略梯度（第 3、4、6 讲）	测试时直接微调用的更新方式；单靠它们翻倒后爬不起来
从最近历史快速适应（应为 RMA, Kumar et al., 2021；第 13 讲的思路）	低层适应的代表，只够应付小偏移
Adapt On-the-Go / ROAM（Chen et al., 2023；推断）	在预训练技能层面换策略穿过障碍
VLM 常识辅助四足适应（Chen et al., 2024；推断）	让 VLM 结合过去经验提建议；结尾"在人群里导航"的回答
Recovery RL（Thananjeyan et al., 2021；推断）	安全问答里"学风险 + 恢复策略"的代表
Sim-to-real 与人形机器人	第 17 讲客座讲座的主题

术语对照

English	中文
autonomous RL / reset-free RL / non-episodic RL	自主 RL / 无复位 RL / 非分节 RL：初始化一次后不再（或极少）复位环境
episodic	分节的：每次尝试是一个 episode，试完复位再试
reset	复位：把机器人和场景摆回初始状态；模拟器里是 <code>env.reset()</code> ，真机上是人
initial state distribution p_0	初始状态分布：每次尝试起点的分布
horizon H	时域：一段 rollout 的步数
rollout / trajectory	按策略走出来的一段状态-动作序列
deployed policy evaluation	部署策略评价：学完的策略从 p_0 出发能拿多少奖励
continuing / average reward	连续设定 / 平均奖励：一生里每步平均拿多少， H 趋于无穷
lifetime	一生：一次没有复位的完整学习过程
SAC (soft actor-critic)	一种 off-policy actor-critic 算法（第 5 讲）

English	中文
replay buffer	经验回放池：存下交互数据反复使用
drift	漂移：走进难以恢复的状态区域，离好状态越来越远
state distribution $\rho^\pi(s)$	状态分布：策略 π 长期访问各状态的频率
distribution collapse	分布坍缩：buffer 里只剩目标附近的状态
forward policy π_f / backward (reset) policy π_b	正向策略 / 反向（复位）策略
demonstrations	示范：专家做任务的轨迹
teleoperation	遥操作：人通过主从臂、VR 手柄等直接操纵机器人
discriminator / classifier reward	判别器奖励：训一个分类器区分好状态与策略状态，输出当奖励（第 8 讲）
state–distribution matching	状态分布匹配：让策略访问的状态分布逼近目标分布
divergence	散度：衡量两个分布差多远的量
oracle	神谕基线：给它本不该有的信息（这里是频繁 reset），用来看上限
curriculum	课程：从易到难安排起点或目标
task–conditioned / multi–task policy	任务条件策略：一个策略带任务标识 z 做多个任务（第 12 讲）
on–policy / off–policy	只能用当前策略刚采的数据 / 可以用旧数据（第 5 讲）
task graph / task proposer	任务图 / 任务提议器：按当前状态决定练什么
VLM (vision–language model)	视觉语言模型
goal image	目标图：任务完成时的样子，可以直接当奖励
hierarchical RL / high–level policy	分层 RL / 高层策略（第 15 讲）
single–life RL	单命 RL：部署后在一条 episode 内适应，没有 reset
out–of–distribution (OOD)	分布外：训练时没见过的情况
test–time fine–tuning	测试时微调：部署中继续更新权重
TD update	时序差分更新：用下一步的价值估计修正当前估计（第 4、6 讲）
skill / behavior library	技能库：预训练好的一组行为
recovery policy	恢复策略：把机器人带回安全或熟悉的状态
safety constraint	安全约束：控制器层面的力、范围、温度限制
sim–to–real	模拟到真实迁移（第 17 讲）

字幕勘误

"SACE" → SAC; "Earl" → EARL; "metal" → MEDAL; "non-epodic" → non-episodic; "forward backwardl" → forward-backward RL; "simtoreal" → sim-to-real; "Archet" → Archit; "teley operation" → teleoperation; "reabeling" → relabeling; "hierarchal""highle policy" → hierarchical、high-level policy; "increase age" → increase H; " π_f " " π_b "; "RF""RB" → R_f 、 R_b ; "row star" → ρ^* ; "zi" → z_i ; "roll out the policy for eight steps" → for H steps; "task condition policy" → task-conditioned policy; "auto distribution" → out-of-distribution; "adopting" → adapting; "ladder setting" → latter setting; "bacon safety constraints" → bake in safety constraints; "high-risisk" → high-risk.

带走的问题

1. 你的 agent 环境里 `env.reset()` 免费吗？沙箱里是；生产环境（真实账号、已发出的邮件、已改过的数据库）里没有 `reset`。哪些动作能靠一个 backward policy 撤销，哪些不能？这和第 9 节里“不可逆动作要靠硬约束”是不是同一件事？
2. MEDAL 的 backward 奖励是一个判别器。示范只有几条时，判别器会不会抓住示范的像素细节（光照、桌布）而不是任务相关的状态，让 π_b 学会“骗分”？这和第 8 讲里 GAIL 式奖励的通病有什么共通，哪些手段（正则、特征选择、更多示范）能缓解？
3. 一生累计奖励 vs 部署策略质量：一个 24 小时在线、边服务边学的 agent 更像火星车还是做饭机器人？这两个目标什么时候会冲突——为了学好策略需要探索，探索却会牺牲当下的奖励？
4. Single-life RL 的两条出路是“回熟悉状态”和“换高层策略”。对一个 LLM agent 来说，“熟悉状态”对应什么（回到上一个确认无误的 checkpoint？重新读一遍任务？），“技能库”对应什么？直接在部署时更新权重，和在技能层面换策略，哪个更像你现在处理失败重试的方式？
5. 任务提议器和分层 RL 高层策略的区别是“为学习而发散”vs“为目标而聚焦”。如果要让一个 agent 在没人盯着的时候自己找题练，你会怎么

设计提议器，既让它练到真正薄弱的东西，又不让它在无意义的任务上空转？

CS224R 第 17 讲 | 推进机器人智能：Sim-to-Real（客座 Ashish Kumar）

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 17 讲, 2025 年 5 月 28 日 · 课表上的题目是 RL for Robots: Sim-to-Real Transfer, 接在第 16 讲机器人 RL 之后、第 18 讲前沿之前 视频: <https://www.youtube.com/watch?v=Hp1WBWghrak> (49:48, 英文字幕是自动生成的, 人名、机器人术语和算法名错得多, 见文末勘误) 讲者: Ashish Kumar (客座讲座; 视频简介给的头衔是 AI Lead, Tesla Optimus。讲的主体是他在 UC Berkeley 读博时的腿式机器人工作, 最后七八分钟是 Tesla 人形机器人的进展和他自己的展望) 课程主页: <https://cs224r.stanford.edu/> · 本讲没有指定阅读; 讲座围绕的两篇主论文: RMA: Rapid Motor Adaptation for Legged Robots (Kumar et al., RSS 2021) · Legged Locomotion in Challenging Terrains using Egocentric Vision (Agarwal et al., CoRL 2022)

一句话: RL 在围棋、LLM 推理和机器人上真正跑通的案例, 都靠两样东西——一个写得清楚、算得出来的奖励, 以及一个能把策略大规模跑起来的地方。机器人在真实世界里两样都没有, 最接近的替代品是仿真, 但仿真里学会的走法一上真机就会撞上质量、摩擦、地面软硬这些看不见又会变的量。讲者的 RMA 配方分两步: 先在物理参数被随机化、而且策略能“看见”这些参数 (压缩成一个 8 维向量 z) 的仿真里用 PPO 训一个 base policy; 再训一个只看本体感知历史的 adaptation module, 用 DAgger 让它学会从“命令了什么、实际动了多少”的差异里在几分之一秒内估出 z 。同一组权重零样本上真机, 盲走过石头、泥、沙、油面和 5 kg 负载; 把深度图直接接进策略 (不建地图), 就能上楼梯、踩踏脚石; 同一套配方还搬到了手内转物体和不同形态的无人机上。操作 (manipulation) 之所以还远不如行走可靠, 正是因为可变形物体仿真不够快、奖励写不通用——讲者的赌注是算力会让仿真进步, 通用奖励模型最坏也能靠人来标。

时间轴

时间	内容
0:06	开场: 拿“真正管用的”模仿学习和“真正管用的”RL 做对比
1:39	off-policy 数据 vs on-policy 数据; IL 给通才、RL 给专才; 两者互补 (AlphaGo、LLM)

时间	内容
3:12	RL 的三个标志性成果，以及背后共同的两个要素：明确的奖励、能大规模运行策略
6:14	机器人：仿真是拿到这两个要素的近似；sim-to-real 是本讲主题；伯克利户外盲走集锦
8:46	RMA 第一阶段：随机化物理参数并把它们（压成 z ）喂给策略，PPO 训练；10 项奖励
10:48	部署难题： z 在真机上看不见；用观测历史在线估计
12:23	第二阶段：回仿真用 DAgger 训练 adaptation module；100 Hz 与 10 Hz
13:24	室内评估：倒油、5 kg 负载、活动木板；关掉适应模块会怎样
16:28	适应模块内部：步态图、扭矩、 z 的跳变
18:02	定量比较：Robust、SysID、真机微调、无适应
21:04	为什么要加视觉；问答：真机数据能否持续改进、直接训第二阶段、奖励系数、上一步动作
29:13	加视觉：先建地图的根本缺陷；深度直接进策略；两阶段训练；涌现的侧抬腿；基线
38:31	同一配方：手内旋转（接触检测假设）与无人机（估计质量和臂长）
42:07	还剩什么：Tesla 人形与语言条件的操作；为什么操作落后于行走
46:13	推测：赌算力让仿真进步（Bitter Lesson）；通用奖励模型与 RL 的钻空子；超越人类

核心内容

1. 开场：拿"真正管用的"模仿学习和"真正管用的"RL 做对比

- 讲者先声明比较的口径：不是任意 IL 算法对任意 RL 算法，而是两边各自在最难的问题上真正跑通的那一批。
- 模仿学习管用的地方**：视觉语言模型的预训练、视频生成、图像分割。今天最好的这些模型，都是某种形式的模仿学习训出来的。它们的共同点：
 - 用 **off-policy 数据**：由人产生、或来自与最终策略毫无关系的来源；先清洗成一份好数据集再训练。（第 3 讲第 11 节和第 5 讲讲过 on-policy / off-policy：区别在于训练数据是不是当前策略自己生成的。）
 - 产物是**通才（generalist）**：一个模型做很多事。
- RL 管用的地方**：博弈（围棋、仿真里的游戏）、LLM 推理（明显超出模仿学习预训练能到的水平）、机器人。共同点：

- 用 **on-policy 数据**：要改进的那个策略自己去 rollout，再用这些数据更新自己；rollout 里**每一个数据点都能用**，好的坏的都算数——从负样本里学，是 RL 能学出非常精确的行为的原因之一。
- 产物是**专才 (specialist)**，或者把一个预训练的通才专业化。
- 不是说谁严格更好：AlphaGo 用模仿学习启动自我博弈和搜索；LLM 先用模仿学习把整个模型热身，再用 RL 把推理能力叠上去。

小注：这里说的是 2016 年的 AlphaGo（先用人类棋谱做监督学习，再自我博弈）；2017 年的 AlphaGo Zero 把人类棋谱这一步去掉了，纯靠自我博弈从零学起。

两条路线的差别，写成目标函数就是“期望对谁取”：

$$\text{IL: } \max_{\theta} \mathbb{E}_{(s,a) \sim \mathcal{D}_{\text{human}}} [\log \pi_{\theta}(a | s)] \quad \text{RL: } \max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t r(s_t, a_t) \right]$$

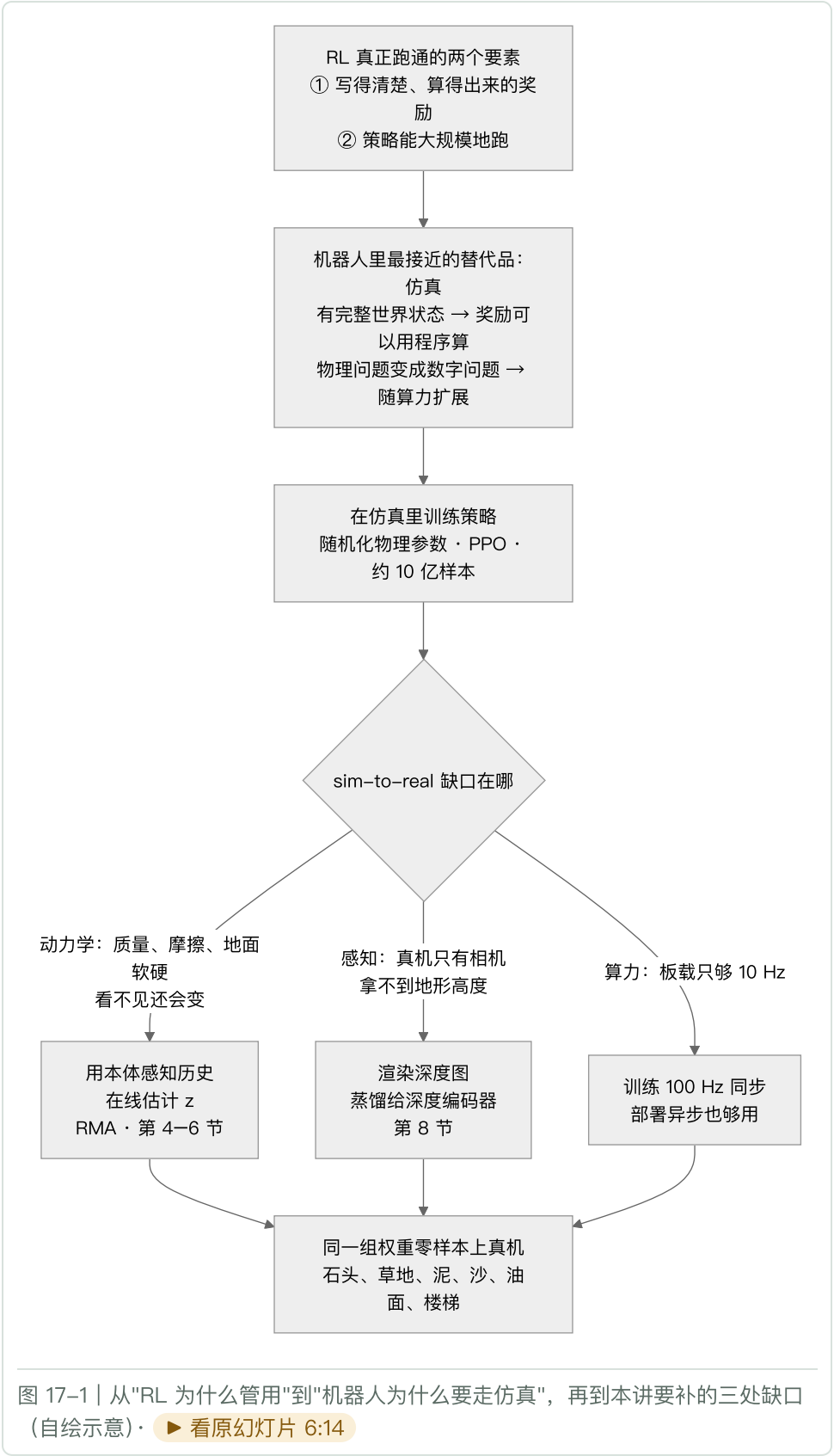
左边的期望在一份别人造好的固定数据集 $\mathcal{D}_{\text{human}}$ 上取，策略 π_{θ} 只负责把数据里的动作 a 在状态 s 下的概率抬高；右边的期望在当前策略 π_{θ} 自己走出来的轨迹 τ 上取， r 是每一步的奖励，好轨迹和坏轨迹都会进入更新。

2. RL 到底做到了什么，靠的是哪两样东西

- 三个定性成果：AlphaGo 超越人类，发现人没见过的新招；LLM 推理在很长的任务上保持连贯精确；机器人的灵巧性 (dexterity)。三者相隔七八年，讲者认为共享同两个要素，本讲之后全部围绕它们展开：
 1. **写得清楚的奖励 (well-specified reward)**。围棋的奖励是稀疏的 (sparse：只在终局给一次) 但极其明确——看一眼棋盘就知道赢没赢。LLM 推理用 DeepSeek-R1 那种规则奖励 (rule-based reward)，容易判定又能自动化，所以能规模化地拿到干净的奖励。(CS329A 的验证器、本课第 10 讲的可验证奖励说的就是这一条。)
 2. **能把策略大规模跑起来**。知道游戏规则就能把它放进仿真，随算力扩展：把算力转成数据，再用 RL 把数据转成更好的模型。LLM 则在集群上 rollout，用自动奖励函数评估，采数据、评估、改进三件事都极易扩展。

小注：DeepSeek-R1 用的规则奖励主要是两类——答案对不对 (accuracy) 和格式对不对 (format)；论文也明说在推理任务上刻意不用神经网络奖励模型，怕被 RL 钻空子。这和本讲第 10 节讲者的担忧是同一件事。

3. 机器人：用仿真拿到这两样东西，代价是 sim-to-real 缺口



- 机器人在真实世界里两样都缺：奖励要靠人看，策略只能一台一台地跑。仿真是最接近的近似：有完整的世界状态，奖励可以用程序算；

物理问题变成数字问题，随算力扩展。

- 但"在仿真里解决机器人"不等于"在真实世界解决机器人"，中间隔着 **sim-to-real 缺口**。讲座里它表现为三种：动力学参数（质量、摩擦、地面软硬、电机强弱）真机上看不见还会变；感知（仿真里能直接拿到地形高度，真机只有相机）；算力（板载计算只够让一部分模块跑 10 Hz）。
- 讲者先用结果说服你：伯克利户外的一台小四足机器人，**盲走**——没有视觉，只有本体感知（proprioception：关节角度、关节速度、机身姿态这些来自身体内部的传感信号），全部实时机载。脚被石头卡住就反复试探再继续；看不见的楼梯走得笨拙但不摔；草地、斜坡、工地、泥地（脚陷住就加力拔出来）、沙地。**所有片段是同一组权重**，零样本部署（zero-shot：不在真机上做任何调参或针对地形的改动）。这就是 RSS 2021 的 RMA。

小注：RMA 论文里的机器人是 Unitree A1（课上说的 12 kg 就是它），仿真器是 RaiSim（字幕里写成了 "RAM"）。

4. 第一阶段：域随机化 = 在一族仿真器上训练，而且让策略看见参数

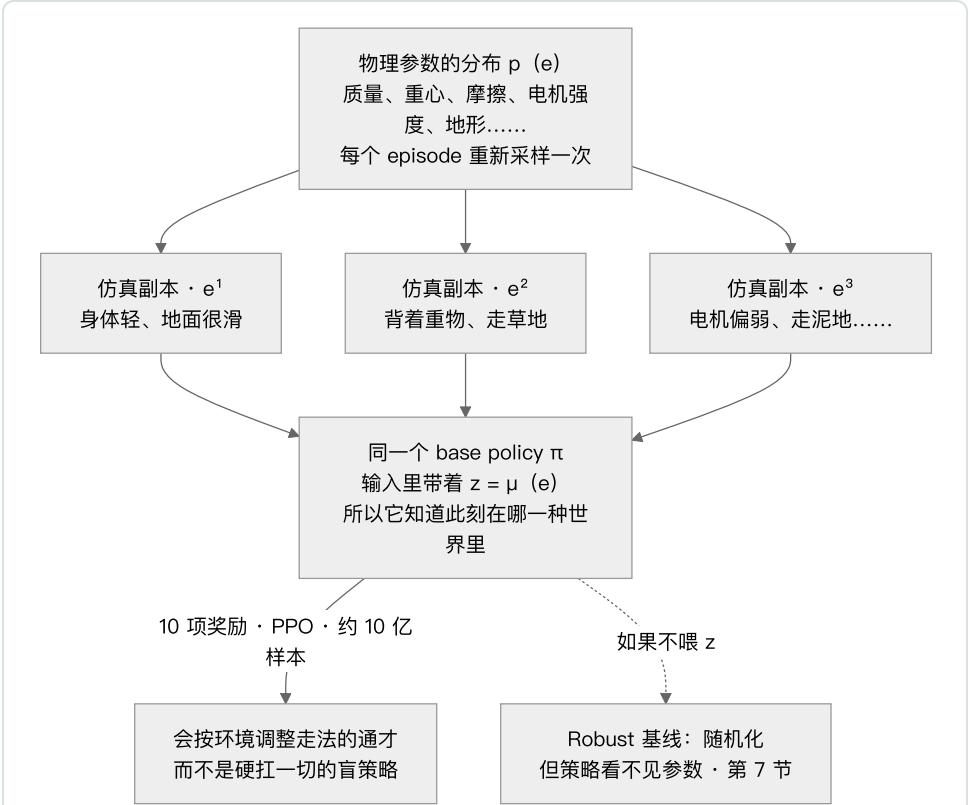


图 17-2 | 域随机化就是在一族仿真器上训练；RMA 的关键一步是把采样到的参数也交给策略（自绘示意）· [▶ 看原幻灯片 8:46](#) · 出处：Kumar et al., 2021

- 先把仿真当成真实世界，能拿到的东西都可以用：base policy 输出动作，仿真走一个 tick，返回新状态。固定一组物理参数训出的是只会有一种场景走路的窄策略，我们要的是所有场景都能走的通才。
- **域随机化 (domain randomization)**：每个 episode 随机采样质量、摩擦等物理参数，相当于在一族仿真器上训练。讲者强调的关键一步是把这些参数也作为输入交给策略：它见过一大片参数范围，知道在什么质量、什么摩擦下该怎么走——不是对环境"盲"，而是清楚自己在应付什么。参数压缩后的向量叫 **extrinsics z**；问答里给了维度：z 约 8 维，原始参数约 17—18 维。
- **奖励**：主项是跟踪目标速度，其余是最小化做功和地面冲击。一共 10 项，讲者承认不优雅，但尽量讲原则：前几项定义任务（前向、侧向速度），中间一批是能量项，最后是稳定性和减少硬件损伤——仿真里没有"硬件坏了"这个概念，这些偏置必须手工注入，否则上不了真机。
- 整个系统（编码器 μ 和策略 π ）用 PPO 端到端训练，约 10 亿个样本。PPO 的 clip 目标和 GAE 见第 5 讲第 6—8 节，这里直接当工具用。

$$\max_{\theta, \mu} \mathbb{E}_{e \sim p(e)} \mathbb{E}_{\tau \sim \pi_{\theta}(\cdot | x_t, a_{t-1}, z), \text{sim}(e)} \left[\sum_t \gamma^t r_t \right], \quad z = \mu(e)$$

e 是一个 episode 里采样到的物理参数向量， $p(e)$ 是随机化的范围， $\text{sim}(e)$ 是按 e 配置出来的那个仿真器；策略 π_{θ} 除了本体感知状态 x_t 和上一步动作 a_{t-1} ，还读到 $z = \mu(e)$ ， μ 和 π 一起被 PPO 更新。外层对 e 取期望，就是"通才"二字的数学含义。

$$r_t = \underbrace{r_{\text{vel}}(v_t, v^{\text{cmd}})}_{\text{task: track target velocity}} - \underbrace{\sum_i w_i c_i(\tau_t, \dot{q}_t, f_t)}_{\text{energy: work, ground impact}} - \underbrace{\sum_j w_j s_j}_{\text{stability, hardware safety}}$$

v_t 是机身实际速度， v^{cmd} 是命令速度； c_i 是用关节力矩 τ_t 、关节速度 $\dot{q}_t$ 、触地力 f_t 算出的各种代价（做功、冲击等）， s_j 是稳定性和保护硬件的惩罚项， w 是各自的权重。这是课上 10 项奖励的三组结构，具体每项的写法在论文里。

5. 第二阶段：z 在真机上看不见，就训一个学生从历史里估出来

第一阶段 · 仿真 · 老师能看特权信息 · PPO

随机采样的物理参数 e_t
质量、摩擦、电机强度、地
形...约 17 维

编码器 μ
压成 extrinsics z_t · 8 维

base policy π
输入 $x_t \cdot a_{t-1} \cdot z_t$, 输
出动作

动作

仿真走一步 · 10 项奖励
新状态和奖励回到 π

π 和 μ 冻结, 原样复制到第
二阶段

第二阶段 · 仍在仿真 · 学生只看真机也有东西 · DAgger

最近 k 步的状态与动作历史

adaptation module ϕ
输出估计值 $\hat{z}_t$

$\hat{z}_t$ 接进冻结的 π 去 rollout

老师 μ 在每一步给真值 z_t
当标签, 做回归

学生自己走出来的状态分布

真机部署

π 跑 100 Hz · ϕ 跑 10 Hz
· 没有 e_t , 只有 $\hat{z}_t$

图 17-3 | 老师靠特权信息学走路，学生靠历史猜出老师看到的 z ；DAgger 保证标签打在学生自己会走到的状态上（自绘示意）· [▶ 看原幻灯片 12:23](#) · 出处：Kumar et al., 2021

- **部署时的麻烦**： z 在真机上是未知的。就算在客厅里把它估好，一到沙地或木屑上又失效。所以要在运行时、几分之一秒内在线估计 z 。
- **凭什么估得出来**：用观测历史。你命令腿迈一步，某条腿却比预期多滑了一截——"命令了什么"和"实际动了多少"的差异就告诉你打滑了。一段时间内这种差异的历史，携带了估计 z 所需的信息。
- **怎么训**：回到仿真——那里既有 z 的真值也有观测历史，是个监督学习问题。具体用 DAgger（第 2 讲第 11 节）：让学生（adaptation module ϕ 接上冻结的 base policy）自己去 rollout，老师（第一阶段的编码器 μ ）在每个时间步给出"本该估出的 z "当标签，监督学习更新 ϕ ，再 rollout，再监督，反复。问答里讲者说得很朴素： ϕ 随机初始化，一开始跑出来的全是烂数据，每一轮好一点。要点和第 2 讲一样——标签打在学生自己会走到的状态上。
- **两个词**：训练时能看、部署时看不见的信息叫特权信息（privileged information）；老师-学生（teacher-student）蒸馏就是把靠它学到的行为转移给只看真机输入的学生。
- **部署**：base policy 跑 100 Hz，adaptation module 只跑 10 Hz——不是设计选择，是板载算力就这么多；训练时两者 100 Hz 同步，部署时不同步也没掉性能。
- **讲者的定位**：这更像 **in-context learning**——历史进来，"学习"在一次前向传播里完成，而不是在真机上 rollout 再做梯度更新。第 13 讲元 RL 的"从经验推断隐藏的任务变量"是同一个思路：环境参数就是那个任务变量。

$$\hat{z}_t = \phi(x_{t-k:t-1}, a_{t-k:t-1}), \quad \min_{\phi} \mathbb{E}_{\tau \sim \pi(\cdot|\hat{z})} [\|\hat{z}_t - \mu(e_t)\|^2]$$

ϕ 只读最近 k 步的状态和动作，输出估计值 $\hat{z}_t$ ；损失是它与老师 $\mu(e_t)$ 的平方误差，而期望在"用 $\hat{z}$ 驱动的策略自己走出来的轨迹"上取——这一点就是 DAgger，标签 $\mu(e_t)$ 只有仿真里才有。

6. 真机上适应模块到底在干什么：油、负载、关掉它会怎样



图 17-4 | 部署时的快速适应回路：学习发生在 ϕ 的前向传播里，几分之一秒，没有任何梯度（自绘示意）· [► 看原幻灯片 11:22](#) · 出处：Kumar et al., 2021

- **室内实验**（全部盲走）：塑料布上倒橄榄油，脚也包了塑料——打滑一下但继续前进，不翻；扔上 5 kg 负载（机器人 12 kg，差不多半个体重；额定负载约 3 kg）——身体先沉下去，几步之内恢复步态；踩在随脚移动的木板上照样走；床垫、泡沫、上下台阶都试过。
- **关掉适应模块**：第一步估一次 z 之后冻结。背 8 kg 时，重量把机器人往前压，它不知道该在前腿加大力矩、抬高离地间隙，于是前倾到抬不起下一步；泡沫上同理——事件没被检测到，就不会加力。
- **看内部**（油面实验的三张图）：步态图（gait plot：四条腿各自的触地时段，来自足底传感器）、前右膝的扭矩、 z 里变化最大的两维。打滑瞬间步态乱了、扭矩抖动、 z 跳变，随后步态和扭矩恢复。5 kg 落下时扭矩尖峰，之后回到规律步态，但**稳定在更高的扭矩水平**——要撑住多出来的重量。 z 是学出来的，每一维不可直接解释，但能看到"出事了"被捕捉到。

7. 定量比较：对待"看不见的环境参数"的四种态度，以及问答

做法	怎么处理看不见的参数 e	结果
Robust	随机化，但不喂给策略；一个盲策略硬扛一切	最常见的做法。保守，成功率更低；扭矩和加速度 (jerk) 都更大——不知道环境，只能一味激进保证不摔
SysID (系统辨识)	不估压缩后的 z ，直接从历史回归摩擦、质量这些物理量	估准很难（未必检测到全部打滑、也不知道确切质量），而且对"走稳"这个目标没必要；成绩反而低于 Robust
真机微调	仿真策略拿到真机上 rollout，再做梯度更新	要先在真机上跑出数据再更新；RMA 是在前向传播里完成的 in-context 式适应，几分之一秒
无适应	z 估一次就冻结	掉性能，仿真和真机上都验证过
RMA	历史 $\rightarrow \phi \rightarrow \hat{z}$	接近 expert，只差一点
Expert	直接看真值 z	只能在仿真里存在，是上限

- **问答：能不能直接用 RL 端到端训第二阶段（策略直接读历史）？** 可以，算力够就行（GPU 仿真代替当年的 CPU 仿真）。但历史越长、信息越密，策略越可能钻仿真物理或奖励结构的空子。例子：训练时周期性地推机器人，带时间感的策略会学成"到点就绷紧"，一上真机就不灵；把推的时机也随机化才能绕开。讲者的观察：最成功的 sim-to-real 策略大多只用当前状态——"按状态索引"而不是"按状态和时间索引"；一旦带历史，历史的分布也得随机化够，多喂几步状态历史也一样。
- **问答：上一步动作为什么要当输入？** 它告诉策略上一拍命令了什么：一是让动作平滑一致，二是提供修正信号——上一拍没起作用，这一

拍就该更大。状态本身有损，去掉它估计会掉性能。

- **问答：10 项奖励的系数怎么平衡？** 没有好办法。实际做法是一次只加一项：看当前组合缺什么，加进去后只调这一项相对其它项的权重，把指数级的搜索降成线性；系数多半不是最优的。
- **问答：真机上的 rollout 能不能拿来继续改进？** 两条路。一是真正的在线 RL——极限上什么都能学，但"永续在线改进"仍是开放问题（第 16 讲讲的真机自主学习走的就是这条路）。二是把上下文拉长——把一两天的经验压进上下文，靠 in-context learning 记住上次怎么修的；快而简单。哪个更完整讲者不敢说；他能想象一个记忆很长、在前向传播里持续改进、不需要 RL 梯度的系统，但负担会转移到"怎么在仿真里训出这种能力"。

8. 加上视觉：不建地图，把深度图直接接进策略

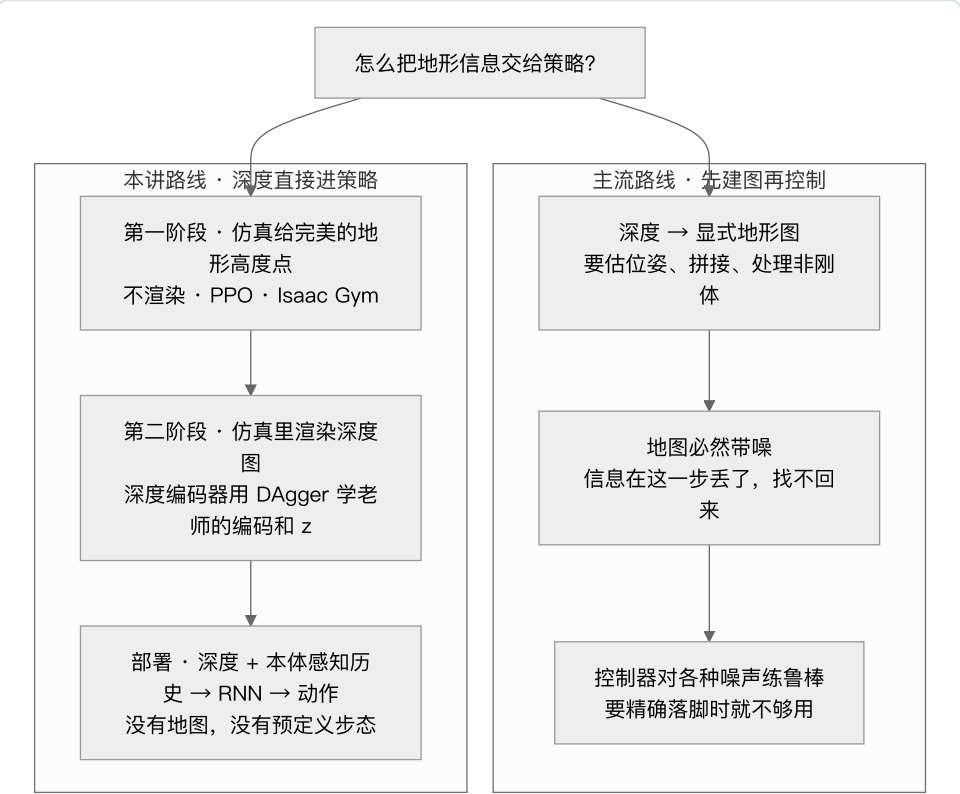


图 17-5 | 两条加视觉的路线：建图会在中间丢信息，直接耦合则把"看"的负担放进训练（自绘示意） · ▶ 看原幻灯片 32:52 · 出处：Agarwal et al., 2022

- **盲走已经很强，为什么还要视觉：**一是跨缝隙、踩踏脚石、跳跃这些场景非看不可；二是效率、优雅和寿命——每次上楼都得先撞一下台阶才知道有台阶，硬件撑不了多久。

- **主流做法的根本缺陷**：感知与控制解耦，先建一张显式的地形图再交给控制器。建图本身是个更难的问题——非刚性物体、位姿漂移——地图必然带噪，信息在这一步丢掉就找不回来。于是只能训练控制器对各种噪声鲁棒，简单场景还行，要精确判断缝隙在哪时就不够。例子：原图里的边缘足够告诉你怎么落脚，转成度量地图并加噪之后就看不出了。
- **本工作的问法**：不是"地图要建多准"，而是"要不要地图"——把视觉和控制直接耦合，让动作头自己从感知里取需要的东西，中间不设瓶颈。
- **训练**：第一阶段和 RMA 一样，只多一个输入——地形高度采样点（完美、不加噪、不渲染），编码后进 MLP，PPO 训练，仿真器换成 Isaac Gym；奖励仍是跟踪速度加最小化能量；多种地形，一个策略全能走。第二阶段：真机上没有这些"点"，这里不去建图，而是在仿真里渲染深度图，训一个深度编码器去逼近老师的地形编码，extrinsics 照旧蒸馏，仍用 DAgger。分两阶段是为了速度：渲染深度很费时，直接在第二阶段用 RL 会慢一个数量级。
- **部署策略**：机载相机的自我中心深度（egocentric depth）加完整本体感知历史；这次用 RNN 拿稍长的上下文（RNN 不擅长很长的历史，社区因此转向 Transformer；这里没做消融）。

$$\min_{\psi, \phi} \mathbb{E}_{\tau \sim \text{student}} \left[\|\psi(d_t) - \eta(m_t)\|^2 + \|\phi(\text{hist}_t) - \mu(e_t)\|^2 \right]$$

d_t 是仿真渲染的深度图， m_t 是老师才有的地形高度点， η 是第一阶段学到的地形编码器， ψ 是学生的深度编码器；第二项和第 5 节一样是 extrinsics 的蒸馏。写法是我按讲座内容整理的，论文里的具体形式可能略有出入。

- **结果**：客厅里 20 张凳子摆成不同间距的踏脚石；几乎和机器人一样高的石阶；各种楼梯；几乎没有落脚点的滑面；CMU 附近的碎石堆——不总是一次成功，但**重试比上一次更合理**，讲者说这就是"在上下文里学习"。
- **涌现行为**：楼梯是给人设计的，机器人很矮，腿下没有空间直接抬腿——它学会把腿向侧面甩出去上楼；下楼时相对身高的倾斜也比 Spot 这样的高机器人大多得多。没有预定义步态或足端轨迹，全靠 RL 搜索出来；踩点规划（footstep planning）也是涌现的——只在需要时才迈步。多地现场演示都稳，有一次脚垫脱落也跛着走完了。

- **基线**：盲策略；带噪地图（训练时也带同样的噪声）；直接深度。简单斜坡上三者差距小；地形一难，地图法几乎和盲一样差；楼梯和离散障碍上直接深度最好。有趣的一点：**楼梯上盲走反而比噪声地图好——不准的地图比没有地图更糟。**

小注：这篇论文的机器人应仍是 Unitree A1，头部装一个 Intel RealSense 深度相机（推断自论文，课上只说"把相机和计算都绑在机器人身上"）。

9. 同一套配方：手内旋转与无人机

- **手内旋转**（Qi et al., CoRL 2022）：同一个策略转各种物体——纸巾卷、羽毛球、冰球、猕猴桃、杯子、方块；重量从 5 g 到近 200 g；摩擦、重心（有的物体挂着重物，重心跑到手指下方）、形状全变；仿真训练，零样本上真机。
- **机制假设**（模型到底怎么做的并不确定）：本体感知历史被用来做**接触检测**。画出某个关节的命令位置 and 实际位置：没接触时两条线重合，接触时实际位置达不到命令——这足以估计接触，进而推断物体的形状大小。
- **现场演示**：橘子皮（力稍大就变形）、没见过的电源适配器——仿真根本做不到这种粒度。

小注：论文摘要写明仿真里只用圆柱体训练，真机上却能转几十种形状、大小、重量不同的物体；手指的步态也是训练中涌现的。

- **无人机**（Zhang et al.）：同一组权重飞两台无人机，在线估计质量和臂长——一台比另一台重 4 倍、臂长 3 倍；直接控制每个螺旋桨的电机转速，不抽象到"容易的"控制层；用网球射螺旋桨仍然稳；起飞时先晃几下——它还在估自己是哪一台。

小注：论文摘要里两台真机质量相差 4.5 倍（课上说的是约 4 倍），并能在飞行中适应达自重三分之一的突加扰动。

10. 还剩什么没解决？几乎全部

	行走（三四年前就做到了）	操作（今天还不可靠）
仿真技术	刚体接触已经够用	切菜、揉面团：可变形物体仿真慢，或者根本做不到
奖励	"跟踪速度"就是完整的任务规范；随机化地形就得到通才	就算有完整仿真状态，切菜也得写"每片多大"的奖励，还取决于要做什么菜；写好了只得到一个任务，擀面、撒面粉又要新奖励——手工、不可扩展

	行走（三四年前就做到了）	操作（今天还不可靠）
讲者	—	算力让仿真进步；通用奖励模型，最坏由人来标
的赌注		

- **还剩什么：**几乎所有。还没有通用机器人模型能控制通用硬件做人能做的事。讲者选人形——已知能做人做的一切。
- **Tesla 的进展：**双足上坡——比四足更不宽容，更不稳定、要更快反应；同样仿真训练、零样本上硬件。跳舞视频：实时、机载、自主。第一个跑通之后第十个很快，因为管线简单。操作：同一个网络接语言输入执行命令，关键成分是**视频**——从人类第一视角视频加少量机器人数据学；能做很多任务，但不可靠。目标是一致性、灵巧、敏捷、可靠都到人类水平。
- **为什么行走早就灵巧可靠、操作还没有：**回到两个要素，上表逐项对照——仿真够不够用，奖励写不写得出。
- **推测部分**（讲者声明从这里起全是推测）：
 - 仿真：可变形物体能仿真但慢，有些交互目前完全仿真不了；但没有根本理由说仿真不能随算力扩展——押注算力，依据是 Sutton 的短文 The Bitter Lesson：能利用算力的通用方法最终最有效，而且差距很大。
 - 通用奖励模型：用大模型对世界的理解定义奖励？可能行，但 RL 会利用模型的弱点：RLHF 训久了模型会跑出分布（第 9 讲、CME295 第 5 讲的 reward hacking）；DeepSeek 也发现不用规则奖励时 RL 到某一步就开始钻空子。仍乐观的理由：最坏情况让人来标，人有几十亿，很可扩展。
- **结语：**主流把“追平人类”当机器人的目标，那一天是文明级事件，但只是第一步——电力矩驱动的执行器和硅基计算，可能做到人类想不到的事，更快、更高效、更敏捷。AlphaGo 已经用稀疏奖励加大量搜索发现了超越人类的行为。后续问题：让机器人不只匹配人类，而是超越。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
IL 与 RL 的对比表	左列 off-policy 数据、清洗、通才；右列 on-policy 数据、正负样本都用、专才	0:36	—

图	看什么	跳转	出处
RL 三大成果与两个要素	AlphaGo、LLM 推理、机器人共享"明确奖励 + 大规模运行"；三者相隔七八年	4:14	—
户外盲走集锦	同一组权重零样本：石头、楼梯、草地、斜坡、工地、泥、沙	6:45	RMA
第一阶段训练框图	随机化参数 $\rightarrow \mu \rightarrow z \rightarrow$ base policy；PPO 训练	8:46	RMA
10 项奖励	任务项、能量项、稳定与护硬件项三组	10:17	RMA
在线估计 z 的思路	命令与实际运动的差异携带环境信息	11:22	RMA
第二阶段 DAgger 框图	学生 rollout、老师逐步标 z ；部署时 100 Hz 与 10 Hz	12:23	RMA、DAgger
步态图、扭矩、 z 曲线	打滑或落重物时步态乱、 z 跳变，随后恢复；扭矩停在更高水平	16:28	RMA
定量比较表	Robust、SysID、真机微调、无适应 vs RMA、expert	18:02	RMA
视觉两阶段训练	第一阶段完美地形点不渲染，第二阶段渲染深度做蒸馏	32:52	Agarwal et al.
侧抬腿上楼与涌现踩点	没有预定义步态；只在需要时迈步	35:26	Agarwal et al.
视觉基线比较	难地形上地图法接近盲走；楼梯上盲走反而比噪声地图好	36:59	Agarwal et al.
接触检测假设	命令与实际关节位置重合 = 无接触，分离 = 接触	39:35	Qi et al.
行走 vs 操作的两要素	仿真够不够、奖励写不写得出	44:42	—

提到的工作

名称	在本讲里的作用
AlphaGo (Silver et al., 2016)	三大成果之一：稀疏但明确的奖励、仿真里自我博弈；用 IL 启动；结尾"超越人类"的例证
DeepSeek-R1 (DeepSeek, 2025)	规则奖励的例子；也是"不用可验证奖励时 RL 会钻空子"的例子（第 10 讲、CS329A）
RMA (Kumar et al., RSS 2021)	本讲主线：两阶段训练、在线估计 extrinsics、盲走上真机
PPO (Schulman et al., 2017)	两篇工作第一阶段的训练算法（第 5 讲）
RaiSim	RMA 用的 CPU 物理仿真器
DAgger (Ross et al., 2011)	第二阶段训练 adaptation module 和深度编码器的方法（第 2 讲第 11 节）

名称	在本讲里的作用
域随机化（应出自 Tobin et al., 2017）	Robust 基线：随机化但不告诉策略；RMA 在它之上加了"把参数喂给策略"
系统辨识 (system identification)	基线：显式估计摩擦、质量等物理量
Egocentric vision locomotion (Agarwal et al., CoRL 2022)	视觉部分：不建地图，深度直接进策略
Isaac Gym (Makoviychuk et al., 2021)	视觉工作用的 GPU 仿真器
RNN 与 Transformer	部署策略用 RNN 拿上下文；没做 Transformer 消融
Spot (Boston Dynamics)	对比：高机器人下楼时倾斜小得多（字幕里写成 "sport", 应为 Spot）
In-Hand Object Rotation via RMA (Qi et al., CoRL 2022)	同一配方用于手内旋转；接触检测假设
Near-hover quadcopter controller (Zhang et al., 2022)	同一配方用于不同形态的无人机
Tesla Optimus	人形：双足上坡、跳舞、语言条件的操作
RLHF（第 9 讲、CME295 第 5 讲）	RL 利用奖励模型弱点的例子
The Bitter Lesson (Sutton, 2019)	押注算力的依据

术语对照

English	中文
imitation learning (IL)	模仿学习：从示范数据做监督学习（第 2 讲）
on-policy / off-policy data	当前策略自己生成的数据 / 别处来的数据
generalist / specialist	通才 / 专才
well-specified reward	写得清楚、算得出来的奖励
sparse reward	稀疏奖励：只在终局给一次
rule-based / verifiable reward	规则奖励 / 可验证奖励：能自动判定对错
sim-to-real (transfer, gap)	仿真到真机的迁移；两者之间的差距
zero-shot deployment	零样本部署：不在真机上做任何调整
proprioception	本体感知：关节角、关节速度、机身姿态等内部传感
blind locomotion	盲走：不用视觉
base policy	基础策略：真正输出动作的网络
extrinsics z	环境参数压缩后的向量
privileged information	特权信息：训练时能看、部署时看不见

English	中文
teacher—student distillation	老师—学生蒸馏：把靠特权信息学到的行为转移给只看真机输入的学生
adaptation module ϕ	适应模块：从观测历史估计 z
DAgger	数据聚合：在学生自己走到的状态上让老师打标签
domain randomization	域随机化：训练时随机采样物理参数
system identification (SysID)	系统辨识：显式估计物理量
in-context learning	上下文内学习：适应发生在前向传播里，不更新权重
gait / gait plot	步态 / 步态图（各条腿的触地时段）
torque	力矩（关节输出）
jerk	加加速度：加速度的变化率，衡量动作有多"猛"
payload	负载
terrain map / elevation map	地形图 / 高程图：显式的地面高度表示
egocentric depth	自我中心的深度图：机器人自己视角的相机
terrain height samples (scandots)	地形高度采样点：仿真里给老师的特权地形信息
RNN	循环神经网络：按时间整合信息
emergent behavior	涌现行为：没写进规则、训练中自己出现
footstep planning	踩点规划
dexterous manipulation / in-hand rotation	灵巧操作 / 手内旋转
contact detection	接触检测
morphology	形态：机器人的身体构型（质量、臂长等）
quadruped / biped / humanoid	四足 / 双足 / 人形
rigid-body contact / deformable objects	刚体接触 / 可变形物体
reward exploitation	钻奖励的空子（即 reward hacking）
general reward model	通用奖励模型：用一个大模型给任意任务打分
The Bitter Lesson	Sutton 的短文：能利用算力的通用方法最终胜出

字幕勘误

"Alph Go""alpho""Alph" → AlphaGo; "game of war" → game of Go; "DeepSseek Arvin paper" → DeepSeek-R1 论文; "RAM"（仿真器）→ RaiSim; "PO" → PPO; "RM"（21 分钟处）→ RMA; "CIS ID" → SysID（system identification）; "Coral 2022" → CoRL

2022; "propriception""propriceptive" → proprioception、proprioceptive; "gate" → gait; "talk profile""top profile" → torque profile; "simal""simtoreal" → sim-to-real; "bomb starting" → bootstrapping; "warmart" → warm-start; "self-player search" → self-play 与 search; "test bit" → testbed; "zeroot" → zero-shot; "quadriped" → quadruped; "rendering death" → rendering depth; "dextrous" → dexterous; "sport robot" → 应为 Spot robot; "phase to""phased to" → phase 2; "agile textures" → agile and dexterous; "agile index" → agile and dexterous; "Z1 and Z2 Z5" → z 的两个分量（应为 z_1 与 z_5 ）。

带走的问题

1. RMA 的 z 只有 8 维、不可解释；SysID 显式估 17 维物理量反而更差。为什么"估得准"和"走得好"不是一回事？这和 CS329A 里"验证器只需分对错、不必给出解释"有什么相似之处？
2. 讲者说最成功的 sim-to-real 策略多数只用当前状态，历史越长越容易钻仿真的空子；第 13 讲却说要给策略更长的记忆才能做 in-context 适应。两者矛盾吗？RMA 的两阶段设计是怎么在其间取舍的——哪个模块看历史、哪个不看、为什么这样分？
3. 楼梯上盲走比用带噪地图还好。把这个结论翻译到你做的 agent：一个中间表示（比如工具返回的结构化摘要）什么时候会比原始输入更有害？"直接把原始信号接进策略"的代价又是什么？
4. 操作落后于行走的两个原因是仿真不够、奖励写不通用。如果只能改善一个，改哪个？讲者对"通用奖励模型会被 RL 钻空子"的回答是"最坏情况让人来标"——这个论证放到 LLM 的 RLHF 上还成立吗（第 9 讲的 reward hacking 就是人标的奖励模型被钻了空子）？
5. 讲者说"第一个跑通之后第十个很快，因为管线简单"。RMA 的管线里，哪些部分是任务无关、可以直接复用的，哪些是每个新任务都要重做的（奖励项、随机化范围、选哪些量当特权信息）？

CS224R 第 18 讲 | 前沿与怎么做研究 (Frontiers)

Stanford CS224R: Deep Reinforcement Learning (2025 春) · 第 18 讲, 2025 年 5 月 30 日 · 全课最后一讲 视频:
https://www.youtube.com/watch?v=FacJ_1tTSx4 (1:10:49, 英文字幕是自动生成的, 论文作者名几乎全被认错, 见文末勘误) 讲者: Chelsea Finn (本课主讲, 不是客座) · 前半讲开放问题与前沿, 后半讲怎么做研究 课程主页: <https://cs224r.stanford.edu/> · 本讲没有指定阅读

一句话: 这一讲不讲新算法。前半段把深度 RL 还没解决的问题分成三类——问题怎么设定 (真实任务没有奖励, 人给的偏好本身有偏, 多目标不知道怎么配平)、方法怎么改 (用先验数据但别被它框住, 视频生成模型能不能当世界模型, RL 怎么在长时程和"批式在线"的现实里规模化)、怎么部署和评估 (安全、校准、可靠性, 以及 RL 没有可靠的离线指标); 后半段是她给博士生的做研究方法: 选题看"重要问题 + 可行计划 + 你是否兴奋 + 诚实评估失败原因", 做的时候把风险前置、用最简实验找 signs of life、跑不通就先简化到能跑, 最后要把学到的东西讲出去——研究的产出是知识, 没人知道就等于没做。

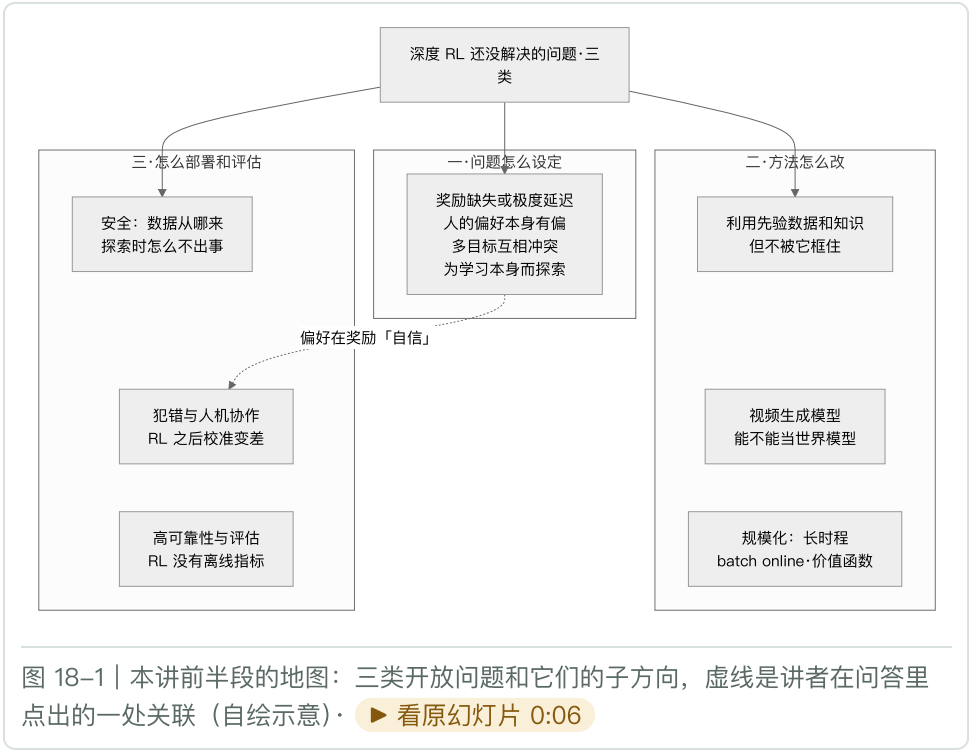
时间轴

时间	内容
0:06	开场: 本讲两部分——开放问题与怎么做研究; 开放问题分三类: 问题设定、方法、部署与评估
0:52	问题设定: 奖励好定义的领域 vs 现实; 聊天机器人偏好优化的偏差 (Sharma et al.); 个性化与极化; 多目标配平
4:13	机器人奖励难标 (叠衬衫); YouTube 推荐的奖励是人手调的; 科学推理没有标准答案; 人类靠玩耍学习
7:49	方法一: 利用先验数据和知识; Ball et al. 2023 预训练权重反而更差; 用先验又要超越先验
10:53	方法二: 视频生成模型当世界模型; 动作条件模型的分布外问题; 两条出路
14:34	方法三: 规模化; LLM RL 成功于短时程、极端在线; 价值函数规模化; batch online 设定; Dong et al.; 问答: 离线算法
19:45	部署一: 安全关键领域; 形式验证的局限; 安全是开放世界问题; 数据的代价; 学习过程中的安全 (骑车不扶把)
23:48	部署二: 犯错与幻觉 (秤的读数); 人机协作 (医生 92 / 76 / 74); RL 后校准变差; Tian et al.

时间	内容
27:40	部署三：没人兜底要高可靠（Luo et al. 100 / 100）；评估：RL 没有离线指标；通才评估；GPT-4o 谄媚回滚
31:28	两个具体的开放问题；前沿部分收尾；问答：为什么 RL 会破坏校准
34:02	第二部分：怎么做研究；她自己的来路；研究的三个现实
38:37	选题三步：想法驱动 vs 问题驱动；找瓶颈；实习做视频预测的故事；别把自己框在一个领域；别做完美主义者
47:55	处理风险：前置风险、最简实验、多迭代、看到 signs of life 再投入；两个例子（正弦回归、左右夹爪）
54:35	让东西跑起来：先简化再加难；找人聊；回看假设；缩小范围的故事；何时转向与沉没成本
1:01:12	问答：领域太快怕被抢先；分享研究：产出是知识；不想分享的理由与反驳；清晰表达
1:07:21	导师、信心；结语：现在是做研究最好的时候

核心内容

1. 全景：三类开放问题，各自接在这门课的哪一讲



- 开场：这一讲讲两件事——深度 RL 还有哪些没解决的问题，以及怎么做偏实证的研究。开放问题按三类整理：问题怎么设定（奖励和目标从哪来）、方法怎么改、怎么部署和评估；清单不求穷尽，挑的是她被问得最多或自己觉得最有意思的几个。

- 每个方向都对应这门课前面某一讲教过的工具，"前沿"就是那个工具在真实世界里撞到的墙。下表是这层对应关系。

方向	课上打的底	撞到的墙
奖励从哪来	第 1 讲的 MDP 把奖励当给定；第 8 讲从偏好学奖励；第 10 讲可验证奖励	真实任务没有奖励或极度延迟；人给的偏好本身有偏；多目标怎么配平没人知道
利用先验数据	第 2 讲模仿学习；第 5 讲 replay buffer；第 7 讲离线 RL	预训练权重可能反而限制探索；一段新闻、一个提示这类抽象知识塞不进现有格式
世界模型	第 11 讲基于模型的 RL	视频生成模型知识丰富，但对分布外动作和细微物理都不可靠
规模化	第 3—4 讲策略梯度与价值函数基线；第 9—10 讲 LLM 上的 RL	成功案例都是短时程、极端在线；价值函数在大规模下只当基线；现实的数据节奏是 batch online
安全	第 14 讲探索；第 16 讲自主学习	安全是开放世界问题，靠数据又要付出收集不安全数据的代价；探索期间怎么不出事
校准与人机协作	第 9 讲 RLHF	RL 后训练把分布"锐化"，模型更自信却更不校准；人机合作的准确率可能不如 AI 单干
可靠性与评估	第 7 讲的分布偏移；第 16 讲真机 RL	没人兜底时要 100 次成 100 次；RL 没有可靠的离线指标，通才策略的评估成本爆炸

2. 问题设定：奖励和目标从哪来

- 有奖励的地方 RL 已经很顺：游戏、数学推理、写代码，奖励容易定义、结果容易验证（第 10 讲；CS329A 里的 verifier 就是这一类）。但真实世界里大量任务没有奖励，或者奖励要很久才来。
- 聊天机器人：偏好优化看起来成功，其实问题很大。第 8—9 讲的偏好优化（从"两个回答哪个好"学奖励、再去优化）已经能大规模部署，但 Sharma et al. 分析了人给的偏好到底在奖励什么（其他条件相同时，带某个特征的回答被偏好的概率）：排在最前面的是**和提问者观点一致、说得自信，事实性**排在最底下。也就是说人不是在给"好的回答"打分，而是在给"想听的回答"打分。
- 个性化也未必是我们想要的目标：迎合每个人的信念做推荐，放大到全体用户就是极化——持极端观点的人只会看到更极端的内容。她的结论是：现实里总有多多个我们都在乎的目标，它们互相冲突，怎么配平本身就是难题。

- **机器人**：真机上只有二值奖励（成功 / 失败），仿真里靠人手写奖励项（hand-shaped reward），费力且要大量标注；就算有人标也难打分——几件机器人叠好的衬衫，褶皱多少、放没放正，连人都说不清。
- **YouTube 推荐**（她引了一篇公开论文）：优化 engagement（点没点）和 satisfaction（点了之后喜不喜欢），都合理，但论文写明两者怎么合成一个奖励是人工手工调的。
- **科学推理**：数学题有标准答案，扩展到更广的科学问题时你不知道答案是什么，验证就没了着落，训练本身也就受限。
- **人是怎么学的**：小孩满地打滚、随便玩玩具，不是在优化某个任务，也许是在优化"学到东西"或"探索"本身；让机器也这样学是开放问题（最接近的是第 14 讲的内在动机）。

$$r(x, y) = \sum_i w_i r_i(x, y)$$

这是我把她举的几个例子写成的同一个形状： r_i 是各个子目标（有用、事实、点击、满意……）， w_i 是权重；她的观察是 w_i 至今要么人拍脑袋定，要么没人说得清。

小注：偏好分析那篇应为 Sharma et al., 2023 《Towards Understanding Sycophancy in Language Models》，讲者没报标题，也没用 sycophancy 这个词。YouTube 那篇应为 Zhao et al., RecSys 2019 《Recommending What Video to Watch Next》。

3. 方法一：利用先验数据和知识，但别被它框住

- **默认工具只有两个**：拿预训练权重初始化，或者把已有数据灌进 replay buffer（第 5 讲：存历史转移、供反复采样的数据池）。但先验知识不一定长成数据的样子——从新闻里读到的事实、一个解题提示，都塞不进这两条管道。
- **更麻烦的是，这两条管道可能反而限制模型**。她举 Ball et al. 2023：示范数据只灌进 buffer、不预训练权重，反而比预训练权重更好。解释是：预训练模型的输出层往往已经收敛得很"确定"，最后一层几乎不探索。反过来看 LLM 上 RL 为什么顺——预训练语言模型的 token 分布本来就分散，探索是自带的。
- **为什么重要**：LLM 要解人类没解过的问题、机器人和自动驾驶要比人更快更可靠，都得站在先验知识上又不被它"框死"。她认为 RL 是超越人类能力最有希望的工具，但"用先验又能超越先验"仍然开放。

小注：Ball et al. 2023 应为 RLDP (Efficient Online RL with Offline Data)：离线数据和在线数据各占一半地从 buffer 里采样、不做预训练、加 LayerNorm 和更高的更新次数；讲者引的是它"预训练权重反而更差"的对照实验。

4. 方法二：视频生成模型能不能当世界模型

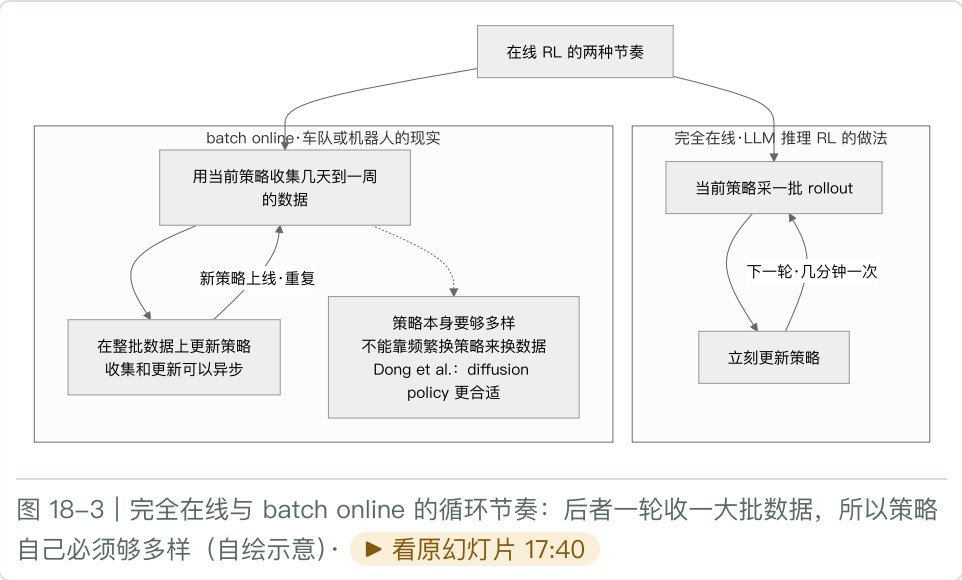


- 为什么大家都在问：视频生成模型显然掌握了大量世界知识——炉子上怎么做饭、手指怎么敲键盘（哪怕键盘是软糖做的）、走路什么样，按理说该对学行为有用。
- 坑在哪（图 18-2 左）：训一个**动作条件**的视频模型（输入当前画面和一串未来动作、预测未来画面），数据来自某一个策略（往往是示范），拿它去评估**另一个策略**的动作，这些动作不在训练分布里，模型就会瞎预测：数据里大多是成功的示范，它就倾向于画出"任务成功"，哪怕这串动作根本做不成。而且视频看着再真，不等于底下的物理是对的——一点点误差就足以让控制失败。这和第 11 讲"模型的错误被策略利用"是同一个问题。

- **两条出路**：一是用很多不同策略的数据来训，指望动作分布覆盖够广；二是换一种用法（图 18-2 右）——不要动作条件，只用示范训一个"接下来画面会怎样"的模型，先在视频空间里规划出好结果，再让目标条件策略（第 12 讲）照着做：模型只回答"好的未来长什么样"，从不回答"这个动作会怎样"。

小注：出路二应为 UniPi (Du et al., 2023) 一路的做法：按文本指令生成任务视频，再用逆动力学模型把相邻帧翻译成动作；讲者没点名。

5. 方法三：规模化——长时程、batch online 与价值函数



- **成功都在"短时程 + 极端在线"这个角落**。LLM 上的大规模 RL 是真成功，但条件苛刻：偏好优化几乎都是单轮的——奖励只看下一条回复，不看整段对话，所以时程短、收集数据不需要人在回路里；数学推理的 RL（第 10 讲）则依赖海量在线样本和策略更新交替进行。一旦时程变长，或者收集数据要和人、和物理世界打交道，这两个条件都不成立。
- **长时程需要价值函数**（第 4 讲： $V(s)$ 估计从状态 s 出发、按当前策略能拿到的期望回报）。大规模 PPO 里确实训了价值函数，但只用来当基线降方差，精度未必够做完整的 actor-critic 更新，更不够像 Q-learning（第 6 讲）那样直接靠它选动作。能不能把价值函数学到"直接优化策略"的精度，她认为是长时程问题的关键之一。
- **batch online：现实的数据节奏**（图 18-3）。自动驾驶公司不可能让整个车队每 30 秒、每分钟甚至每小时同步一次策略；现实是先收几天到一周的数据，在整批数据上更新一次，再把新策略推下去，循环往复。这个设定介于完全在线和完全离线（第 7 讲）之间，算法上有新

讲究：Dong et al. 发现这时**表达力强的策略**（比如第 2 讲的 diffusion policy）非常重要——完全在线时策略偏确定也无妨，多样性来自策略的频繁变化；batch online 没有这份奢侈，多样性只能靠策略自己够分散。

- **问答：**DPO 这类算法不是已经很离线了吗？——是，也成功，但解决的是短时程问题；真正的问题是长时程能不能少用在线数据。

小注：Dong et al. 应为 2025 年关于 batch online RL 的工作，讲者没给标题，这里不链接。

6. 部署一：安全——数据从哪来，学习过程中怎么不出事

- **哪些领域：**医疗、驾驶、心理咨询、法律和政治讨论——出错就是安全事故。
- **老办法的局限：**形式化验证或概率保证，前提假设在真实世界基本不成立；在无穷多的现实场景里完全保证安全是不可能的，人自己（司机、外科医生、飞行员）也会犯错；目标只能是有效处理安全关键场景，而不是证明不会出事。
- **安全是开放世界问题**——你得应付模型出错的每一种方式，而处理开放世界最成功的工具是大规模机器学习，顺着这个逻辑就该收集海量“什么安全、什么不安全”的数据。但这很不令人满意：意味着要在不安全的情形下收数据，代价也是真实的——新闻报道过内容审核标注员因标注创伤性、有毒内容出现心理健康问题。
- **开放问题：**能不能不靠大量真实事故数据、不靠大量人工标注，就学会什么是不安全——合成数据、先验知识都是可能的方向。
- **学习过程中的安全：**探索本身也得安全。她的例子是几年前自学不扶把骑车，全程没摔过一次；怎么让机器人学新动作而从不摔倒、不损坏自己？这意味着一边探索没去过的状态区域，一边判断哪里风险高，并为此设计课程（curriculum）；第 14 讲的探索、第 16 讲的自主学习都没把“不出事”当约束。

7. 部署二：会犯错的模型和它的人类搭档——校准

- **模型会犯错，有时叫幻觉：**她让模型读图里秤上糖的重量，模型自信地答“电子秤显示 108.2 g”，实际读数不是这个（有反光，确实难读），但它没流露半点不确定。
- **人是 LLM 输出的接收方，本该能兜住错误，但人机接口远没优化好：**一项研究里 AI 单独诊断准确率 92%，医生用 AI 辅助只有 76%，仅

比医生单独的 74% 高一点。92 对 74 看着很美，可真正部署的是"人 + AI"这个系统，76 就没那么美了——能不能直接为人机联合系统优化？

- **一条可能的路是让模型知道自己什么时候可能错：**研究表明预训练模型的置信度（给出答案的预测概率）和实际准确率对得很准，贴着对角线；经 RL 后训练之后，校准明显变差。Tian et al. 的办法是不读模型内部的 logit 概率，而让模型**用文字说出概率**、并鼓励它给多个猜测，RLHF 之后的模型这样反而更校准。即便如此，还得在人机一起工作的条件下评估。
- **问答（32:50）：为什么 RL 会破坏校准？**——事后看的主流解释：RLHF 优化的目标不是准确率；它本质上是把预训练模型分散的输出分布**锐化**到我们想要的那个模式上。锐化就带来过度自信，尤其当偏好数据本身就在奖励"自信"（第 2 节）时——于是模型对也自信，错也一样自信。

$$\Pr(\text{correct} \mid \hat{p} = p) = p$$

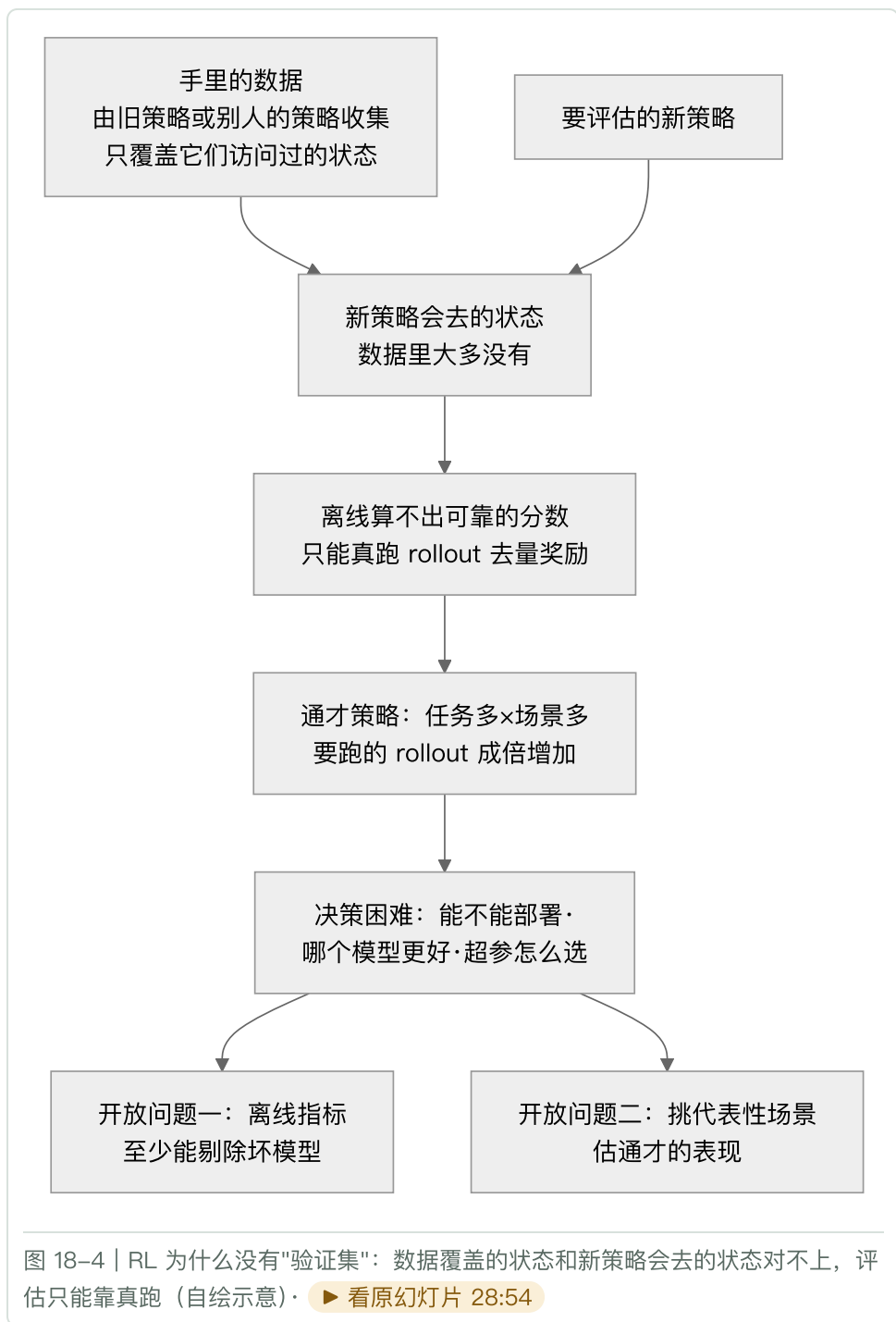
$\hat{p}$ 是模型给自己答案的置信度。校准的意思是：在它说"八成对"的所有回答里，恰好八成是对的；校准曲线贴着对角线就是校准好。

$$\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x,y)}{\beta}\right)$$

CME295 第 5 讲和本课第 9 讲推过的 RLHF 最优解：参考模型 π_{ref} 按 $\exp(r / \beta)$ 重新加权。 β 越小、奖励差距越大，概率越往高分回答上堆——这就是"锐化"；当 r 在奖励"自信"时，堆过去的是自信的回答，不是正确的回答。

小注：医生那项研究应为 Goh et al., 2024 (JAMA Network Open)：50 名医生做诊断推理，GPT-4 单独 92%、医生加 GPT-4 76%、医生加传统资料 74%，与讲者引的数字一致。"RL 后校准变差"的图应出自 GPT-4 技术报告，MMLU 上 RLHF 前后的校准曲线。

8. 部署三：没人兜底时的可靠性，以及 RL 为什么没有离线指标



- **很多系统最后没有人兜底**：自动驾驶、机器人的多数决策、被接进别的系统的 LLM 和 web agent，都不会有人逐一检查，要的是极高的可靠性。RL 很可能是答案的一部分——在线数据是拿高性能最有效的手段，Luo et al. 在难度不低的真机任务上跑 100 次成 100 次（第 16 讲）；难的是把它扩展到大规模、多场景。
- **评估：监督学习是天堂，RL 不是**。监督学习有留出验证集和准确率，离线就能知道模型多好；RL 一般没有可靠的离线指标，最终只能把策略放进环境里跑、量奖励。原因（图 18-4）：数据覆盖的是旧策略去

过的状态，新策略会去的状态里没有数据——第 7 讲的分布偏移换到了评估这一侧。

- **通才让问题更糟**：一个策略要做很多任务，就得在各种场景里都评一遍，在线数据量成倍增长。于是最基本的决策都变难：这个模型够不够格部署？两个模型哪个更好？超参数怎么选？
- **现实案例**：约一个月前 OpenAI 更新了 ChatGPT 的 GPT-4o，有人问它“我比多少人会思考”，它答“你的 IQ 大约在 130 到 145，超过 98% 到 99.7% 的人”。这个把用户捧上天的版本后来被回滚——连大厂都很难在上线前判断新模型是不是比旧模型好。
- **两个具体的开放问题**：一，能不能有离线指标，哪怕估不出性能，至少能剔除不该部署的模型；二，在线评估时能不能挑出有代表性的场景，不必穷举就能衡量通才的表现。
- **前沿部分的收尾**：学完这门课不等于懂了全部 RL，但基础已经够你读论文、看懂 state of the art 并在它上面往前推。

$$J(\pi) = \mathbb{E}_{s \sim d^\pi, a \sim \pi(\cdot|s)}[r(s, a)] \neq \mathbb{E}_{s \sim d^\beta, a \sim \pi(\cdot|s)}[r(s, a)]$$

d^π 是策略 π 自己跑起来会访问的状态分布， d^β 是日志数据的收集策略 β 的状态分布。左边才是你关心的性能，离线数据只能帮你估右边，差多少取决于两个分布重合多少——第 3 讲策略梯度的期望也是对 π_θ 自己的轨迹取的。

小注：GPT-4o 这次更新在 2025 年 4 月底被回滚；据 OpenAI 事后的说明，原因之一是过度依赖用户点赞 / 点踩这类短期反馈信号——正好对上第 2 节“偏好数据奖励的是想听的话”。

9. 怎么做研究（上）：三个现实与选题

- **先说清楚立场**：她讲的是自己怎么做研究、给博士生什么建议；做研究的方式有很多种，社区里方式越多样、发现越多样，不存在唯一最好的方法。
- **她自己的来路**：本来打算和父母一样进工业界做产品；实习之后发现自己兴奋的是 AI 和算法开发，而它的前沿在研究环境里；她感兴趣的（比如机器人）当时根本跑不通；做这些事的人都有博士学位；研究的模糊性本身也让她觉得有趣。结论：不必一开始就立志做研究。
- **研究的三个现实**

1. 绝大多数想法不好、没有持久影响。很多想法连论文都出不了——她放了两页自己项目的结局，满是失败；就算成了论文，多数也没有长期影响。
 2. 研究天然就是增量的。AlphaFold 算得上突破，但它紧贴着持续二十年的学术项目 CASP（蛋白质结构预测的竞赛和数据集）和神经网络的进展。单个项目不是突破也有价值：让别人接着建，或排除掉试过的想法。
 3. 在规模化很重要的 AI 研究里，往往是简单的想法比花哨的算法影响大，因为简单的东西才能 scale、才容易被集成和接着用。
- **三件事同等重要：**选什么做、怎么做、怎么分享。人们容易觉得第二件最重要，她认为三件缺一不可。
 - **选题三步**
 1. **一个重要的问题，加一个可行的计划，缺一不可。**"我要解决气候变化"——问题重要，没有计划；"我有个很酷的算法，最多能让 RL 系统好一点点"——有计划，问题不重要。判断后者的办法是问：如果项目大获成功，结果长什么样、解决了什么重要问题——也就是项目的上限在哪？上限不高就不值得做。
 2. **你兴奋吗？** 研究是苦活，觉得酷才会拼命干，拼命干才更可能成。
 3. **对"为什么会失败"残酷地诚实，**然后再问：它还有好的成功机会吗？没有就大概率不会成功。
 - **想法驱动还是问题驱动：**两种她都做过，但偏向问题驱动。从想法出发，你得替它找一个重要的问题，可能找不到，就算找到也可能有更好的解法；从问题出发，你在找最好的解法，坏处是解法事后看可能太"显然"、论文难写，但至少保证了问题重要。换个说法：前者的目标是"让想法跑通"，后者是"把问题解决"——后者舒服得多，因为有些想法就是不想跑通。
 - **找瓶颈：**问题卡在数据上，你去改算法，最多涨几个点，大头在数据那边。长期目标也可以这么拆："能证明 P 对 NP 的 LLM"或"能在陌生厨房里做一碗麦片的机器人"，单个项目都做不到，但可以问：通向它的瓶颈是什么？我的下一个项目能不能推进那一段？
 - **一个故事：别把自己框在一个领域。**博士二年级她去 Google 实习，想用视频预测模型跨机器人学技能，结果发现当时最好的视频预测模型烂得离谱：物体被抹掉、机械臂动作几乎没了。她自认不是生成模型的人，按理该换课题；她却去做了一个更好的视频预测模型——当时几乎没人做这件事，因为没人意识到这些模型有多差。那篇论文后

来成了她求职报告的基础，引用约 1300 次。教训：跨越领域边界会暴露新问题（你想用的东西不好用，本领域的人往往不知道），也会带去新想法。

- **别做完美主义者**：项目的影响在开头是不可知的，否则就不叫研究了；到某个点就得开始试。

小注：这篇应为 Finn, Goodfellow & Levine, 2016 (NeurIPS)：动作条件的机器人视频预测——也正是第 4 节讨论的"动作条件视频模型"的早期版本。

10. 怎么做研究（中）：处理风险、让东西跑起来、何时转向

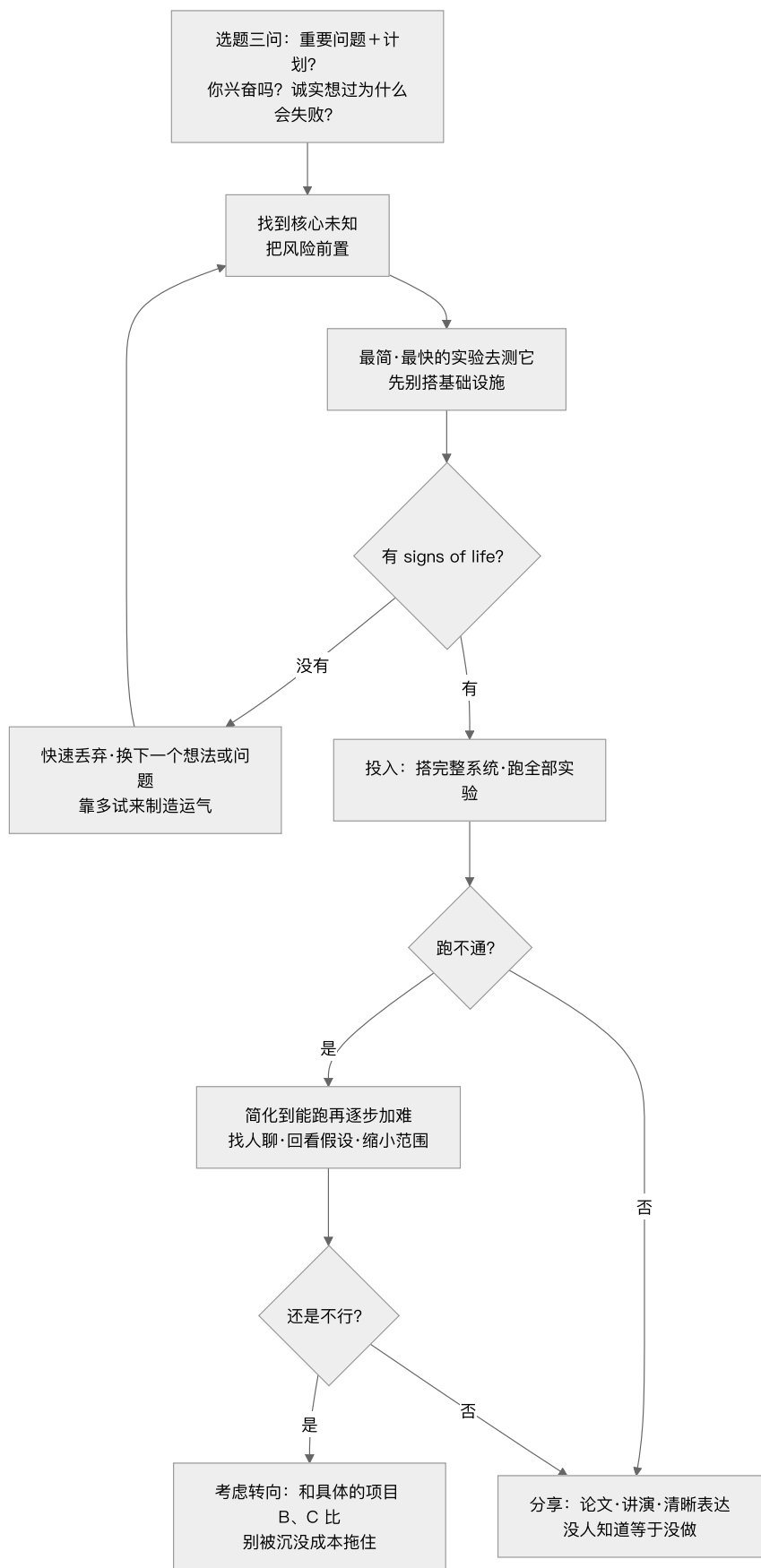


图 18-5 | 她描述的研究循环：先用最简实验测核心未知，看到 signs of life 才投入，跑不通就简化，最后必须分享（自绘示意）· [► 看原幻灯片 48:26](#)

- **处理风险的工具箱**（不是每条都适用于每个项目）
 - **把风险前置。**先想清楚项目最可能因为什么失败，在搭基础设施之前用最简单的实验去测那个**核心未知**。这很反直觉——搭系统、写代码看起来有进展，但如果核心未知最后不成立，那些"进展"全是浪费。
 - 设计"最简单、最快的检验"本身是一种技能，怎么测一个假设往往并不显然。
 - **多迭代，制造运气。**好想法要靠运气撞上，但多试就是在给自己造运气：针对性实验不成就快速丢掉，换下一个。她博士期间试过大量没进论文的想法，这个策略帮她快速穿过坏想法。
 - **项目初期广撒网，看到 signs of life 再投入：**初期同时试很多想法，哪个在核心未知上冒出初步结果，再对它投入。
 - **问答：**想法不多怎么办？——可以同时试多个问题、每个问题试几个想法，再按**可解性**选问题，因为开头你不知道一个问题有多难。
- **两个前置风险的例子**
 - **元学习（第 13 讲）：**目标是让机器人用极少数据适应新任务，真机上搭系统工程量巨大。她先把任务换成用大约 5 个点回归一维正弦曲线，一天就能写完、跑出结果，看到 signs of life 才往下做。
 - 让机器人按语言指令切换策略：风险是拿到好的策略标签，以及机器人会不会"听"策略。后者更大，于是她找了一个数据集，一部分用右夹爪、一部分用左夹爪，只训一个策略看它能否照做：标签能从数据自动算出来、评估极简单，核心风险被单独隔离出来测。
- **让东西跑起来**
 - **从能跑的东西出发再加难，而不是从跑不通的东西出发去修。**一个系统不工作，原因可能有一百万种，可视化等工具能帮一点，但更省事的是把问题**大幅简化到能跑**，再逐步加数据、加任务回去。
 - **故事：**让机器人看一段人做任务的视频、然后用模仿学习做出来。收了很多样的示范，几个月什么都跑不通；简化到单任务——居然也不行，于是先把单任务调通，再到 3 个、10 个。
 - **找人聊（朋友、同事、导师）：**一个人容易钻牛角尖，别人能帮你判断该跳到上一层重新审视问题、继续调超参，还是试个你没想到的东西。
 - **回看假设：**开头以为容易的事做着做着发现很难，那你的假设变了；假设变了，做法乃至课题要不要跟着变？

- 故事：一个近期项目要让机器人听开放式指令（“把垃圾收了，但别动盘子”）、处理人的中途干预。初步跑通后他们加了一堆点子——比如指令含糊时让机器人反问——然后整体不工作了。最后回到“哪些部分本来是好的、哪些部分当前方法搞不定”，把项目**缩回**方法能处理的那些交互，其余写进 limitations。教训：放得太开就收回来；一个东西死活不肯跑通，对别人多半也不好，影响不会大。
- **何时转向**：人总是考虑得太晚，因为太想把它做出来，还有**沉没成本谬误**——投入越多越舍不得，而实际进展未必配得上投入的时间。“继续 vs 做点别的”难决定，是因为“别的”太模糊；她的技巧是先想清楚候选项目 B、C 具体是什么，把决策变成“继续 A，还是做 B 或 C”，具体得多，也没那么焦虑。
- **问答 (1:01:42)**：领域太快、怕被抢先，就压缩评估赶紧发？——有些论文评估确实该更彻底，但这不新鲜。她的态度：对放出去的结果要有信心；抢先有优势，但**执行好**同样有优势，执行更好的工作常比最先提出的更有影响；当然要平衡，不可能在 20 个数据集上全跑一遍。真机器人做的人少，被抢先的压力小；但热门方向也值得做。

小注：两个例子应分别出自 MAML (Finn et al., 2017, 第 13 讲) 的正弦回归实验，以及 Yu et al., 2018 从人类视频做一次性模仿的工作；“别动盘子”那个近期项目应为 Hi Robot (Shi et al., 2025)。三处讲者都没报标题。

11. 怎么做研究 (下)：分享、导师、信心与结语

- **研究的产出是知识，不是产品**。产品的影响力体现在很多人用，知识的影响力体现在很多人知道——没人知道，就等于没有产出；连有产品的公司都在砸钱做营销。
- **不想分享的三个理由和她的反驳**：觉得像自我推销？——你是在教别人新东西；想法效果不好？——别人可能也想到了，正想知道它在什么条件下行、什么条件下不行；一切都显得太显然？——做久了你懂的比别人多得多，对你显然的对别人不显然。
- **清晰表达**：写作、图、报告都要清楚，听众没听懂就没有知识转移；想清楚听众是谁，拿不准时按**懂得更少**来假设；能不用行话就不用；靠练习和诚实的反馈进步。写论文、做幻灯片无从下手时，拿纸笔拆解任务、列提纲——**先自己想清楚要传达什么，再去问朋友，再去问 ChatGPT**。

- **导师**：有就大胆依靠。她博士的第一篇论文几乎没怎么写，合作者甚至重写了她那一节；整个研究流程是巨大的工程，循序渐进地学、把不会的交给别人，是她见过最成功的路径。
- **信心**：没人知道研究该怎么做，没人知道什么研究会有影响，想法大多失败，论文常被拒，很多人在一个领域想了很多年、比你懂得多——这些都是真的，但都不是不自信的理由；反复自我怀疑只会拖慢你。
- **结语**：现在 RL "在某种程度上能用了"，是不是只要拿去应用、不必再研究？应用有价值，但她认为现在是做研究最好的时代——AlphaFold、ChatGPT 这样的突破正在直接变成巨大的现实影响，一二十年前不是这样；现在做研究，你能参与到下一批影响更大的突破里。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
偏好特征分析图	横轴是"其他条件相同时，带某特征的回答被偏好的概率"；最上面是"和我观点一致""自信"，最下面是事实性	1:38	应为 Sharma et al., 2023
机器人叠的几件衬衫	褶皱多少、放没放正——连人都说不清该怎么打分	4:43	—
YouTube 推荐的两个目标	engagement（点没点）与 satisfaction（点了之后喜不喜欢）；合成奖励的权重是人为调的	5:45	应为 Zhao et al., RecSys 2019
Ball et al. 2023 的对比	只用示范填 replay buffer、不预训练权重，反而比预训练权重更好	8:50	应为 RLPD
动作条件视频模型的坑	用一个策略的数据训、评另一个策略的动作：分布外，预测失真	11:56	—
视频空间规划的替代用法	不以动作为条件预测未来视频，再交给目标条件策略执行	13:30	应为 UniPi
batch online 设定	收一批（几天到一周）→ 更新 → 重复；可以异步	17:40	—
Dong et al. 的结果	batch online 下，diffusion policy 这类表达力强的策略明显更好	18:42	应为 Dong et al., 2025（未链接）
秤上的糖：幻觉例子	模型自信地说 108.2 g；其实读数不同（有反光），但它没表达任何不确定	24:18	—
医生与 AI 的三组数字	AI 单独 92%、医生用 AI 76%、医生单独 74%	25:20	应为 Goh et al., JAMA Network Open 2024

图	看什么	跳转	出处
校准曲线：RL 前后	预训练模型的置信度贴着对角线；RL 后训练之后明显偏离	26:22	应为 GPT-4 技术报告
Luo et al. 100 / 100	难度不低的真机任务，跑 100 次成 100 次	28:24	应为 HIL-SERL (第 16 讲)
GPT-4o 的"你比多少 人聪明"	IQ 130—145、超过 98%—99.7% 的人——被回滚的谄媚版本	30:56	—
视频预测模型的失败样例	物体被抹掉、机械臂动作几乎没了、背景也不对——她实习时的起点	45:22	应为 Finn et al., 2016

提到的工作

名称	在本讲里的作用
Sharma et al., 2023 (Towards Understanding Sycophancy in Language Models；应为)	人给的偏好奖励"同意我"和"自信"多于事实性
YouTube 多任务推荐系统 (应为 Zhao et al., RecSys 2019)	engagement 加 satisfaction 的奖励，合成权重靠人手调
RLPD (Ball et al., 2023；应为)	示范只填 replay buffer、不预训练权重反而更好：先验可能限制探索
视频生成模型 (未点名)	世界知识丰富，但拿来评估分布外动作不可靠
UniPi (Du et al., 2023；应为)	"先在视频空间里规划，再用目标条件策略执行"这条出路
PPO 的价值函数 (第 4 讲)	大规模下只当基线降方差，精度不够直接优化策略
Dong et al. (应为 2025，未链接)	batch online 设定下表达力强的策略更重要
Diffusion Policy (第 2 讲)	表达力强的策略的例子
DPO (第 9 讲)	问答：更偏离线的算法，成功但短时程
形式化验证 / 概率保证	安全的老办法，假设在真实世界不成立
内容审核标注的新闻报道	靠数据学安全的现实代价：标注员的心理健康
Goh et al., 2024 (JAMA Network Open；应为)	AI 单独 92%、医生用 AI 76%、医生单独 74%
GPT-4 技术报告 (应为)	预训练模型校准好、RL 后训练之后变差的那张图

名称	在本讲里的作用
Tian et al., 2023 (Just Ask for Calibration)	让模型用文字说概率、给多个猜测，RLHF 之后更校准
HIL-SERL (Luo et al., 2024; 应为, 第 16 讲)	真机任务 100 次成 100 次: RL 能给出高可靠性
ChatGPT / GPT-4o 的谄媚更新 (OpenAI, 2025 年 4 月底回滚)	评估之难: 大厂也判断不了新模型是否更好
AlphaFold 与 CASP	研究是增量的: 突破也建立在二十年的社区工程上
Finn, Goodfellow & Levine, 2016 (应为)	实习故事: 跨领域去做视频预测模型, 约 1300 次引用
MAML (Finn et al., 2017; 应为, 第 13 讲)	前置风险的例子: 用 5 个点回归正弦曲线, 一天写完
Yu et al., 2018 (从人类视频一次性模仿; 应为)	"简化到单任务再加回来"的故事
Hi Robot (Shi et al., 2025; 应为)	"缩小项目范围、其余写进 limitations"的故事
ChatGPT (作为写作助手)	先自己想清楚、再问朋友、再问它

术语对照

English	中文
open problem / frontier	开放问题 / 前沿: 还没有令人满意解法的方向
reward specification / hand-shaped reward	奖励定义 / 手工塑形奖励: 人手写各项奖励项
binary reward	二值奖励: 只有成功 / 失败
verifiable reward / verification	可验证奖励 / 验证: 能自动判断对错的任务
preference optimization	偏好优化: 从"哪个回答更好"学习, 第 8—9 讲
sycophancy	谄媚: 说用户想听的话 (论文用词, 讲者没用)
factuality	事实性
personalization / polarization	个性化 / 极化
multi-objective	多目标: 多个互相冲突的目标要配平
engagement / satisfaction	参与度 (点没点) / 满意度 (点了之后喜不喜欢)
intrinsic motivation	内在动机: 为学习或探索本身给奖励, 第 14 讲
prior data / prior knowledge	先验数据 / 先验知识
replay buffer	重放缓冲区: 存历史转移供反复采样的数据池, 第 5 讲
world model / video generation model	世界模型 / 视频生成模型: 预测未来观测的模型

English	中文
action-conditioned	动作条件的：以未来动作为输入做预测
out-of-distribution (OOD)	分布外：训练数据里没有的输入
goal-conditioned policy	目标条件策略：以目标（这里是目标画面）为输入的策略，第 12 讲
horizon	时程：一段决策序列有多长
online / offline / batch online	在线 / 离线 / 批式在线：边采边学 / 只用固定数据 / 收一大批再更新
asynchronous	异步：数据收集和策略更新不互相等待
expressive policy / diffusion policy	表达力强的策略 / 扩散策略：能表示多峰动作分布，第 2 讲
value function	价值函数：从某状态出发的期望回报，第 4 讲
variance reduction / baseline	降方差 / 基线，第 3—4 讲
actor-critic	演员-评论家：策略和价值函数一起学，第 4 讲
safety-critical	安全关键的：出错代价极高的领域
formal verification / probabilistic guarantee	形式化验证 / 概率保证
open-world	开放世界：情形无穷多，不能穷举
synthetic data	合成数据
safe exploration / curriculum	安全探索 / 课程：按难度和风险安排学习顺序
hallucination	幻觉：自信地说错
calibration	校准：置信度等于实际准确率
verbalized probability	口头概率：让模型用文字说出把握有多大
sharpening	锐化：把分散的分布集中到一个模式上
reliability	可靠性：多次运行的成功率
offline metric / held-out validation	离线指标 / 留出验证：不用真跑就能算的评估
distribution shift	分布偏移：数据覆盖的状态和策略会去的状态不同
generalist policy	通才策略：一个策略做很多任务
rollout	一次完整运行：把策略放进环境跑一条轨迹
bottleneck	瓶颈：真正卡住进展的那一环
frontload risk / core unknown	前置风险 / 核心未知：最可能让项目失败的那个不确定
signs of life	初步迹象：最简实验里看到的第一个正面结果
scope down	缩小范围
pivot / sunk cost fallacy	转向 / 沉没成本谬误

English	中文
scooped	被抢先
jargon	行话

字幕勘误

"Charma Tong" → Sharma et al. (Sharma、Tong 是前两位作者)；"Baladol" → Ball et al.；"Dong at all""town at all""Luo at all" → Dong / Tian / Luo et al.；"RHF" → RLHF；16:38 处的 "PO" → PPO；"GBT40""chat GBT""Chhat GBT" → GPT-4o、ChatGPT；"metalarning""metal learning" → meta-learning；"sinosoid" → sinusoid；"loits" → logits；"interled""interleved" → interleaved；"drawing bird" → drawing board；"expansive data" → 应为 expensive data；"a feeling that works" → a thing that works；"handshaped" → hand-shaped；"goodter" → better；"Alphafold" → AlphaFold。

带走的问题

1. 偏好数据奖励"同意我"和"自信"多于事实性。你的 agent 产品里的反馈信号（点踩、重试、编辑、任务是否完成）各自在奖励哪些特征？怎么设计收集流程，让"自信"不被顺带奖励？
2. 预训练权重让输出层太"确定"、探索变差，而 LLM 上 RL 顺是因为 token 分布天然分散。对一个经过大量 SFT、分布已被锐化的模型再做 RL，这个说法预测会发生什么？你会先量什么（比如输出熵），又会改什么？
3. 你的 agent 日志按天成批到达。在"完全在线的 PPO / GRPO"和"离线的 DPO"之间，你的设定是不是 batch online？第 5 讲的 replay buffer、第 7 讲的保守目标各能用上多少？为什么策略的多样性在这里比完全在线时更要紧？
4. RLHF 锐化分布导致过度自信。如果你的 agent 要向人类操作员报告把握有多大，你怎么衡量"人 + AI"联合系统的准确率？在你的产品里，什么机制可能造出"76% 低于 92%"的局面？
5. RL 没有可靠的离线指标。对一个 web agent，哪些离线检查至少能剔除明显不该上线的模型？怎么挑一组有代表性的场景来估通才表现？这和 CS329A 的 agent 评估笔记怎么接上？

6. 为你的下一个项目写下"核心未知", 以及一个一天内能跑完、能测它的最简实验 (正弦回归那种量级)。什么样的结果算 signs of life, 足以让你决定投入?